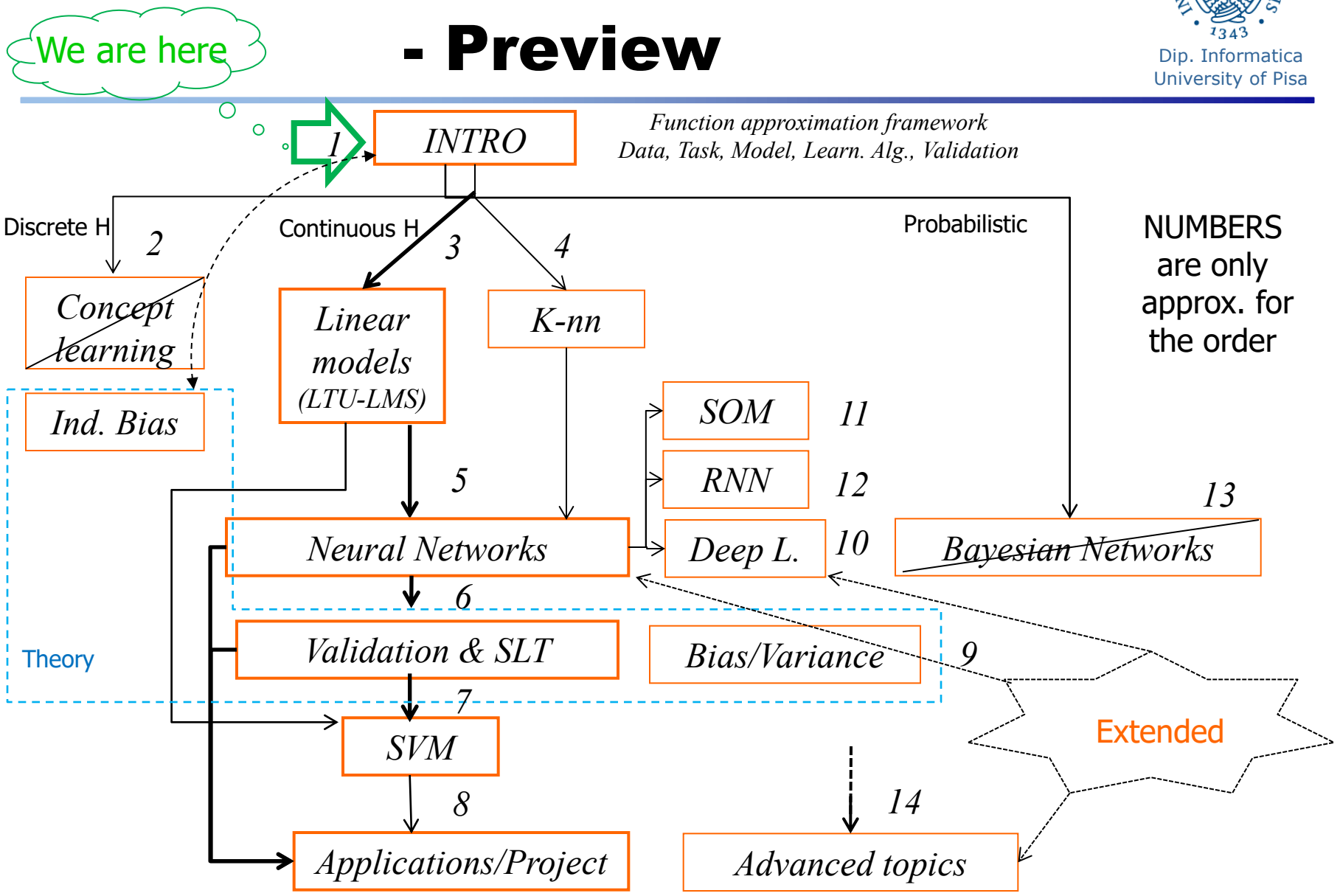
ML Course structure - Preview

Dip. Informatica
University of Pisa



Course contents (skip now)
(Indicative synthetic programme)



Dip. Informatica
University of Pisa

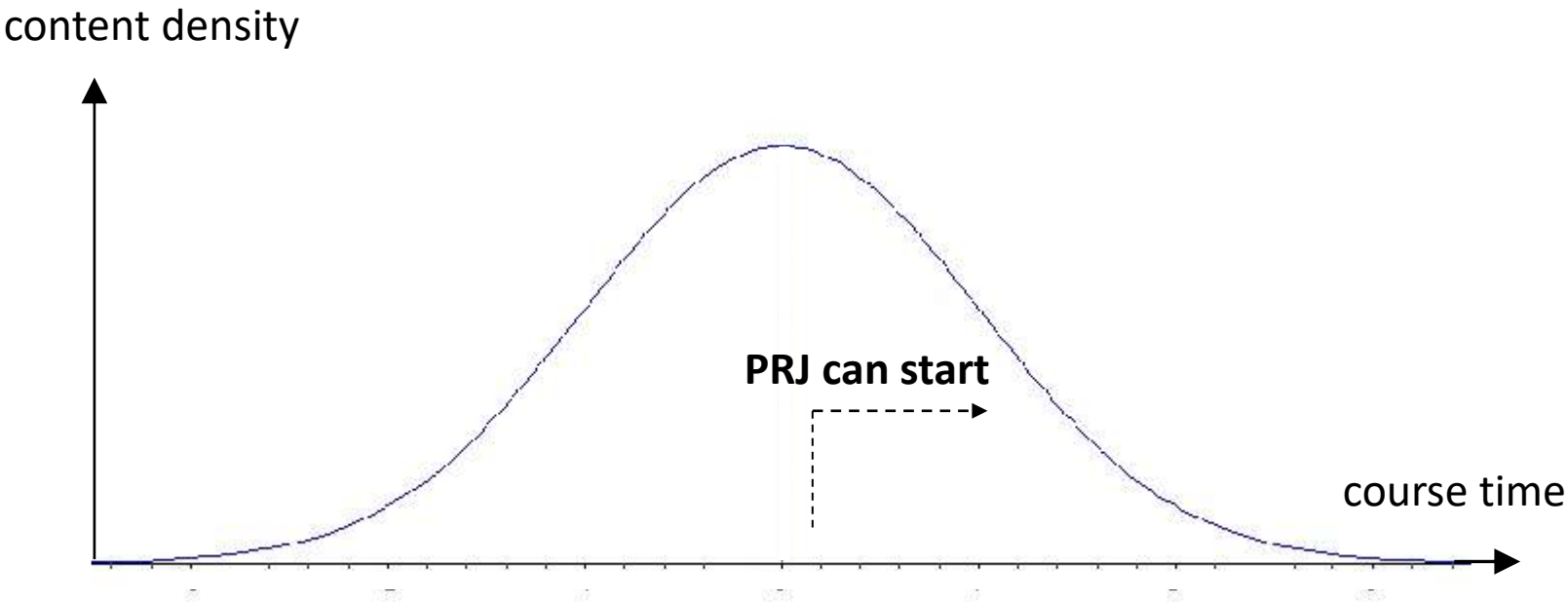
WARNING: See the final program provided at the end of the course

- **Introduction:** Computationl learning tasks, prediction, discrete and continuous spaces, inductive bias, generalization
- **Basic concepts and models**: linear models (models and properties), nearest neighbor (models and properties), propositional and rule based models,.
- **Neural Networks (NN):** Perceptron and computational properties. Introduction to multilayer feedforward NN: architectures and learning algorithms*. Regularization. Recurrent NN*. *Intro to recent NN paradigms**: Deep Learning, Randomized NN.
- **Principles of learning processes and general practical aspects:** Validation: model selection and model assessment, Bias-Variance analysis. Elements of Statistical Learning Theory, VC-dimension.
- **Support Vector Machines:** linear case, kernel-based models.
- **Bayesian and Probabilistic/Graphical models** [removed]
- **Unsupervised learning:** vector quantization, self-organizing map.
- **Introduction to Benchmarks and Applications** *
- **Advanced approaches**: Learning for Structured data (intro)*

*** These parts have been progressively extended**

Course Intensity/Speed Distribution through Time

Dip. Informatica
University of Pisa



Introduction: slow and soft

Core: Main models (NN, SVM) and Validation: dense (faster)

Advances: fast but consequential (not dense) and stimulating

Summary of the Intro to ML

Dip. Informatica
University of Pisa

- Part I (now)
  - Motivations, contextualization in CS
  - Course info
- Part II (in Lect.s 2 and 3)
  - Utility of ML
  - Learning as function approximation (pilot example)
  - Design components of a ML system, including
    - Learning tasks
    - Hypothesis space (and first overview)
    - Inductive bias (examples in discrete hypothesis spaces)
    - Loss and learning tasks
    - Generalization (first part)
- Part III (in Lect. 4)
  - Generalization and Validation

Aim: *overview and terminology*
before starting to study models and learning algorithms

Machine Learning

Dip. Informatica
University of Pisa

A new extended definition (looking to the pilot example)

- The ML studies and proposes methods to build (infer) dependencies / **functions** / hypotheses from examples of observed data
  - that ***fits*** the know examples
  - able to ***generalize***, with reasonable accuracy for new data
    - According to verifiable results
    - Under statistical and computational conditions and criteria
  - Considering the expressiveness and algorithmic complexity of the models and learning algorithms

DATA

Dip. Informatica
University of Pisa

- The data *represent* the available facts (*experience).*
  - Representation problem: to capture the structure of the analyzed objects

Types: Flat, Structured, …

- **Flat** (*attribute-value language*):

fixed-size vectors of properties (*features)*, single table of tuple (measurements of the objects)

Fruits	Weight	Cost $	Color	Bio
Fruit 1 (lemon)	2.1	0.5	y	1
Fruit 2 (apple)	3.5	0.6	r	?

→ Attributes (categorical/discrete or continuous)

→ missing data

Data can be subject to
preprocessing: e.g. Variable scaling, encoding*, feature selection…

DATA Encoding

Dip. Informatica
University of Pisa

Flat case:

- **Numerical encoding for categories: e.g.**
  - 0/1 (or –1/+1) for 2 classes
  - More classes:
    - 1,2,3… Warning: grade of similarity (1 vs 2 or 3): useful for "order categorical" variables (e.g small, medium, large)
    - *1-of-k* (or *1-hot*) encoding: useful for symbols

A	**1**	0	0
B	0	**1**	0
C	0	0	**1**

It will be useful for the project !

Useful both for input or output variables

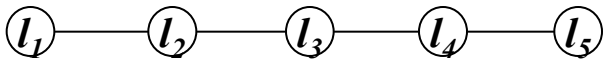
DATA : Structures

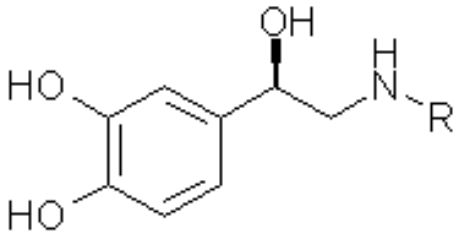
Dip. Informatica
University of Pisa

- **Structured:** Sequences (lists), trees, graphs, Multi-relational data (table) (in DB)

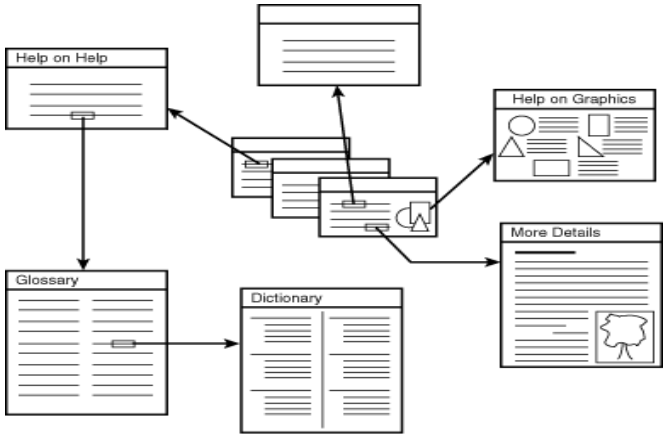
Examples: images, microarray, temporal data, strings of a language, DNA e proteins, hierarchical relationships, molecules, hyperlink connectivity in web pages, ...

Which natural representation?





Graph/network data

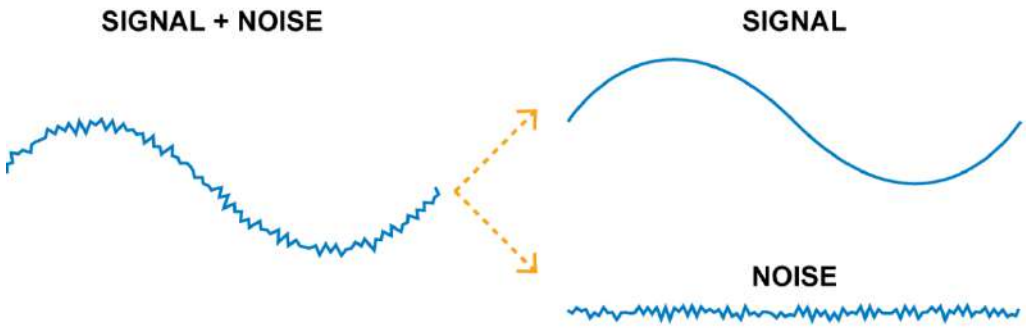


DATA
Further terminologies



Dip. Informatica
University of Pisa

- ***Noise***: addition of external factors to the stream of (target) information (*signal*); due to randomness in the measurements, not due to the underlying law: e.g. Gaussian noise



- ***Outliers***: are unusual data values that are not consistent with most observations (e.g. due to abnormal measurements errors)
  - outlier detection – preprocessing: removal
  - Robust modeling methods

- ***Feature selection***: selection of a small number of informative features: it can provide an optimal input representation for a learning problem

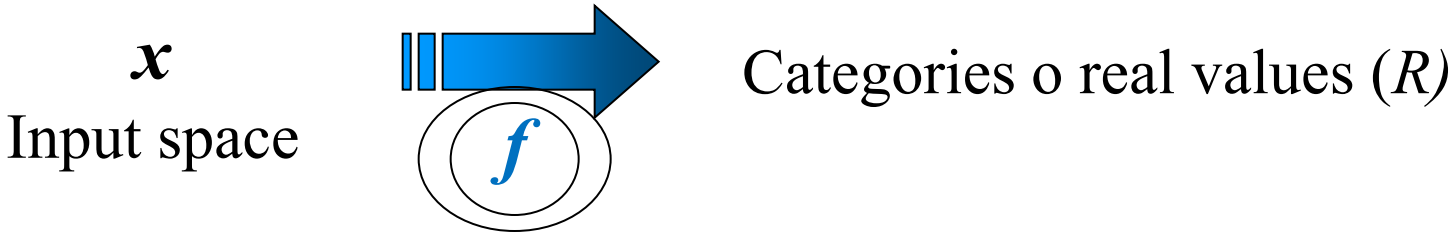
TASKS

Dip. Informatica
University of Pisa

- The task defines the purpose of the application:
  - Knowledge that we want to achieve? (e.g. pattern in DM or model in ML)
  - Which is the helpful nature of the result?
  - What information are available?

Mainly in the ML course

- **Predictive** (Classification, Regression): function approximation



E.g. recall the *"pilot" example on handwritten digits:* Build a *function* from examples

- **Descriptive** (Cluster Analysis, Association Rules): find subsets or groups of unclassified data

Tasks: Supervised Learning

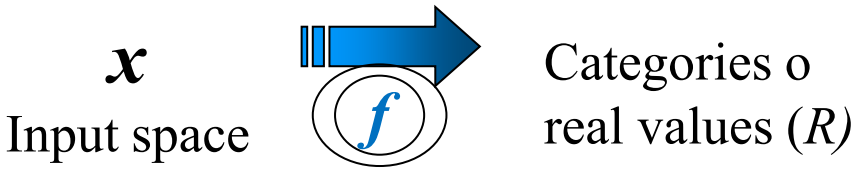
Dip. Informatica
University of Pisa

- Given: Training examples as $<input,output>=<\boldsymbol{x},d>$ (***labeled* examples**)

Def for an unknown function f (known only at the given points of example)

  - Target value: desiderate value d or t or y … is given by the teacher according to $f(\boldsymbol{x})$ to label the data

- Find: A *good* approximation to f (a hypothesis h that can used for prediction on unseen data $\boldsymbol{x}'$, i.e. that is able to generalize)



- Target d (or t or y): a categorical or numerical *label*
  - ***Classification:*** discrete value outputs:
    $f(\boldsymbol{x}) \in \{1,2,\ldots,K\}$ *classes* (*discrete-valued function*)
  - ***Regression:*** real continuous output values (approximate a real-valued target function, in R or R^K)

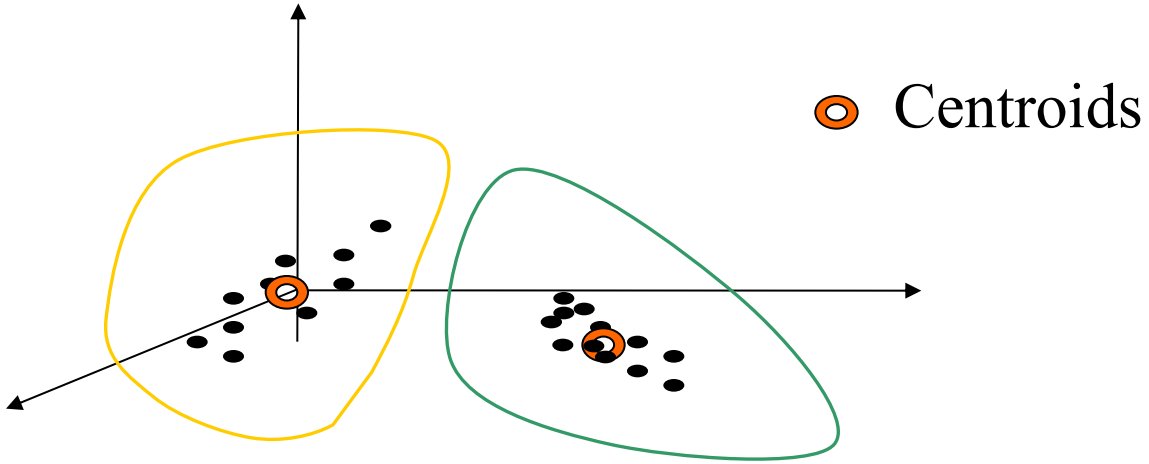
Unified vision thanks to the formalism of a **function approximation** task

Tasks: Unsupervised Learning

Dip. Informatica
University of Pisa

Unsupervised Learning: No teacher!

- TR (Training Set)= set of unlabeled data <x>
- E.g. to find *natural groupings* in a set of data
  - Clustering
  - Dimensionality reduction/ Visualization/Preprocessing
  - Modeling the data density



- Clustering:

Partition of data into clusters (subsets of "similar" data)

Tasks: Classification

Dip. Informatica
University of Pisa

(Supervised) Classification: Patterns (features vectors) are seen as members of a class and the goal is to assign the patterns observed classes (label)

- *Classification*: $f(\boldsymbol{x})$ return the correct class for $\boldsymbol{x}$
- Number of classes:
  - **=2** : $f(\boldsymbol{x})$ is a Boolean function: binary classification, **concept learning** (T/F or 0/1 or –1/+1 or negative/positive),
  - **> 2**: multi-class problem (C_1,C_2,C_3 ….C_K)

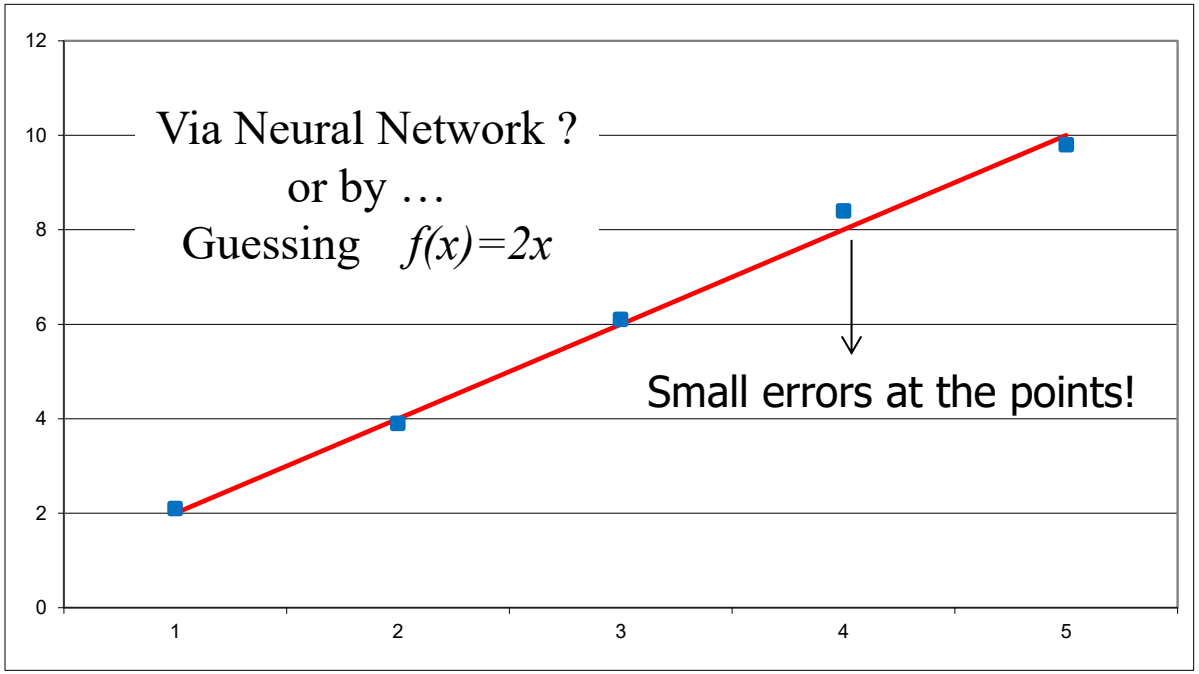
Tasks: Regression: example

Dip. Informatica
University of Pisa

- Process of estimating of a real-value function on the basis of finite set of noisy samples (supervised task)
  - known pairs $(\boldsymbol{x}, f(\boldsymbol{x})+random\ noise)$

Task (exercise): find f for the data in the following table:

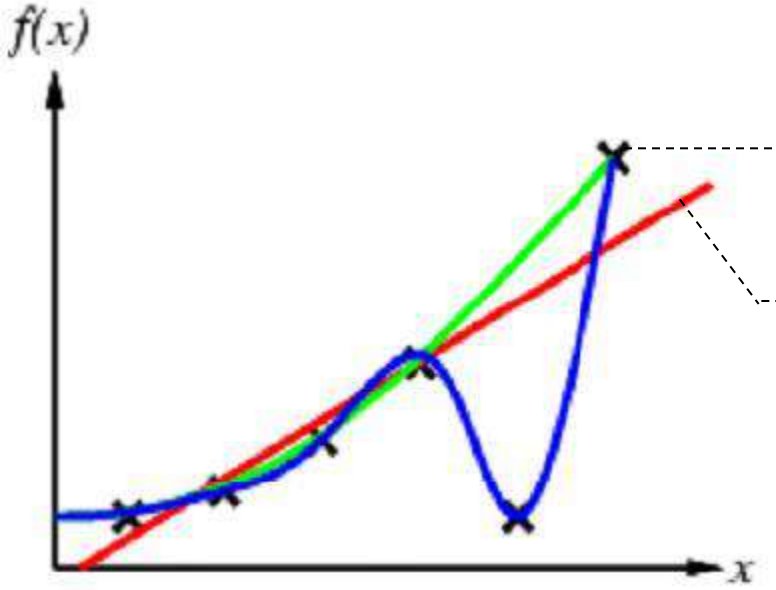
x	*target*
1	2.1
2	3.9
3	6.1
4	8.4
5	9.8
…	…



Tasks: regression

Dip. Informatica
University of Pisa

- *Regression:* x = variables (e.g. real values), $f(x)$ *real values*: curve fitting (x is 1-dim in the example but it becomes *k-dim* in general)
- Process of estimating of a real-value function on the basis of finite set of noisy samples
  - known pairs $(x, f(x)+random\ noise)$



Point where we know the value of $f(x)$

Linear hypothesis

Among the infinite possibilities, what is the most appropriate?

An example (linear hypothesis): $h_w(x)=w_1x+w_0=0.2\ x\ -0.4$

Models
and survey of useful concepts

Dip. Informatica
University of Pisa

- MODEL:
  - Aim: to capture/describes the relationships among the data (on the basis of the task) by a "language" (numerical, symbolic, …)
  - The "language" is related to the *representation* used to get knowledge
  - The model defines the class of functions that the learning machine can implement (*hypotheses space*)
    - E.g. set of functions $h(\mathbf{x},\mathbf{w})$, where $\mathbf{w}$ is the (abstract) parameter

- **Training example** (superv.): An example of the form ($\mathbf{x}$, $f(\mathbf{x})+noise$)
  $\mathbf{x}$ is usually an input vector of features, *(d or t or)* $y=f(\mathbf{x})+noise$ is called the target value
- **Target function**: The true function *f*
- **Hypothesis**: A proposed function *h* believed to be similar to *f*. An expression in a given *language* that describes the relationships among data
- **Hypotheses space** H: The space of all hypotheses (specific models) that can, in principle be output by the learning algorithm

Models: few trivial examples....

Dip. Informatica
University of Pisa

Just to have a preview of different *representation* of hypothesis (because you already know the *language* of equations, logic, probability):

- **Linear models** (representation of H defines a **continuously** parameterized space of potential hypothesis);
  each assignment of w is a different hypothesis, e.g:
  - $h(\mathbf{x}) = sign(\mathbf{w}^T \mathbf{x} + w_0)$ *binary classifier*
  - $h_{\mathbf{w}}(x) = w_1 x + w_0$ $\quad E.g.\ h_{\mathbf{w}}(x) = 2x + 150$ *simple linear regression*

- **Symbolic Rules**: (hypothesis space is based on **discrete** representations); different rules are possible , e.g:
  - *if* $(x_1=0)$ *and* $(x_2=1)$ *then* $h(\mathbf{x})=1$
  - *else* $h(\mathbf{x})=0$

  binary classifier

- **Probabilistic models**: estimate $p(x,y)$
- K Nearest neighbor regression: Predict mean y value of nearest neighbors (memory-based)

Learning Algorithms



Dip. Informatica
University of Pisa

- LEARNING ALGORITHM Basing on data, task and model

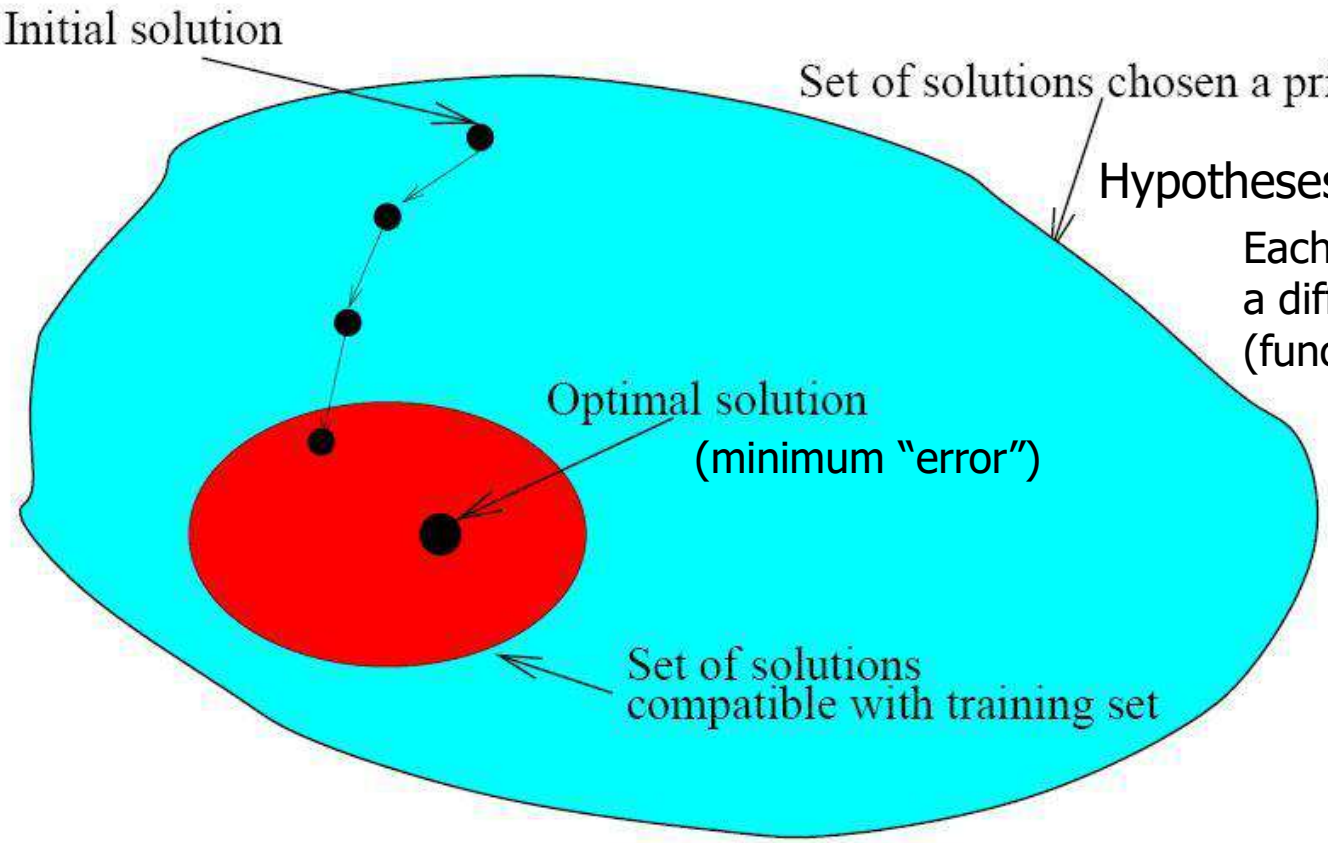
- (Heuristic) *search through the hypothesis space* H *of the* ***best hypothesis***
  - i.e. the best approximation to the (unknown) target function
  - Typically searching for the *h* with the *minimum "error"*

  - E.g. free parameters of the model are fitted to the task at hand:
  - Examples: best w in linear models, best rules for symbolic models, ….
  - Remember the regression example, we proposed $h(x)=2x$, for $h_w(x)=w_1x+w_0$ assuming $w_1=2$ and $w_0=0$ as the best parameter value: *how?*

- **H** may not coincide with the set of all possible functions and the search can not be exhaustive: we need to make assumptions → (we will see the role of) *Inductive bias*

Learning Algorithms: search

Dip. Informatica
University of Pisa



Typically local search approaches

1. The Role of the Inductive Bias

Dip. Informatica
University of Pisa

In order to set up a model and a learning algorithm we can make *assumptions* (about the nature of the target function) concerning either

- Constraints in the model (in the hypothesis space H, due to the set of hypotheses that we can express or consider) (**Language Bias**)
- Constraints or preferences in learning algorithm/search strategy (**Search Bias**)
- Or **Both**.

- We will see that such assumptions are strictly need to obtain an useful model for the ML aims, i.e. a model with generalization capabilities

- We start to discuss it within examples in *discrete hypotheses spaces (rules)*, ***learning a concept*** (a Boolean function) [Mitchell chapt. 2]
  - E.g. $\boldsymbol{x}$ is a "cat" if $h_{cat}(\boldsymbol{x}) = 1$, otherwise is 0 for $\boldsymbol{x}$ in "animals"

Find the Version Space

Dip. Informatica
University of Pisa

- Given the def.: a hypothesis h is **consistent** with the TR, if $h(\boldsymbol{x})=d(\boldsymbol{x})$ for each training example $<\boldsymbol{x},d(\boldsymbol{x})>$ in TR.

- It is possible to perform a *complete search* (finding the set of *all* h consistent with the TR set) *in an efficient way* in this reduced space (of conjunctive rules) by cleverer algorithms (Mitchell chap. 2)
  - Instead of searching enumerating all the possible combination of literals, i.e. every h in H

- We are only interested to say that these algorithms find the VS:
- Call the **version space**, $VS_{H,TR}$, with respect to hypothesis space H, and training set TR, the *subset of hypotheses* from H *consistent* with all training examples

Unbiased Learner II (formal)

Dip. Informatica
University of Pisa

Recall that the **version space**, $VS_{H,TR}$, with respect to hypothesis space H, and training set TR, is the subset of hypotheses from H consistent with all training examples

The only examples that are unambiguolsy classified by an ***unbiased learner*** represented with the VS are the training examples themselves I.e. the *lookup table* !

Property: An unbiased learner is *unable to generalize* (on new instances):
Proof: Each unobserved instance will be classified 1 (positive) by precisely half the hypothesis in VS and 0 (or negative) by the other half (*rejection:* no answer is made by the VS for new input instances).
Indeed:
$\forall$h consistent with x_i(test), $\exists$ h'identical to h except $h'(x_i) <> h(x_i)$,
$h \in$VS $\rightarrow$ $h' \in$ VS (because they are identical on TR)

Language or search bias?

Dip. Informatica
University of Pisa

Why the *search bias* can be preferred over the *language bias*?

- In ML typically use ***flexible*** *approaches* (expressive hypothesis spaces, universal capability of the models, e.g. Neural Networks, DT)
- avoiding the *language bias*, hence without excluding a priori the unknown target function,
- retaining an inductive bias but focusing on the *search bias* (which is ruled by the learning algorithm).
  - In practice using an *incomplete* search strategy.

Conclusions:

- Learning without bias cannot extract any regularities from data (lookup-table: no generalization capabilities)
- Every state-of-the-art ML approach shows an *inductive bias*
- Issue: characterize the bias for different models/learning approaches

2. Tasks & Loss

Dip. Informatica
University of Pisa

We said … A *"good"* approximation to f from examples.

How to measure the quality of the approximation?

- Recall that we produce $h(\boldsymbol{x})$ value (output of the model for input $\boldsymbol{x}$)
- We want to measure the "distance" between $h(\boldsymbol{x})$ and d
  (objective function for minimization of errors in training, check of errors in test)

We use a ("inner") *loss function/measure*: $L(h_w(\boldsymbol{x}),d)$ (for a pattern $\boldsymbol{x}$)

e.g. high value → poor approximation

The *Error* (or *Risk or Loss*) is an expected value of this L

e.g. a "sum" or mean of the inner loss L over the set of samples

$$Loss(h_{\boldsymbol{w}}) = E(\boldsymbol{w}) = \frac{1}{l}\sum_{p=1}^{l} L(h_{\boldsymbol{w}}(\boldsymbol{x}_p), d_p)$$

Note:
index p is used for the samples $p=1..l$

We will change L for different tasks



Note: at moment Error, Risk and Loss are considered equivalent, we will specify differences later through the course

Regression

Dip. Informatica
University of Pisa

- Regression: *predicting a numerical value*

- **Output**: $d_p = f(\boldsymbol{x}_p) + e$ *(real value function + random error)*
- **H**: a set of real-valued functions

- **Loss function** L : measures the approximation accuracy/error
- A *common* loss function for regression: the squared error

$$L(h_{\boldsymbol{w}}(\boldsymbol{x}_p), d_p) = (d_p - h_{\boldsymbol{w}}(\boldsymbol{x}_p))^2$$

- The mean over the data set provide the *Mean Square Error (MSE)*

Classification

Dip. Informatica
University of Pisa

- Classification of *data into discrete classes*

- **Output**: e.g. {0,1}
- **H**: a set of indicator functions

- **Loss function** L : measures the classification error

$$L(h_{\boldsymbol{w}}(\boldsymbol{x}_p), d_p) = \begin{cases} 0 & if \quad h_{\boldsymbol{w}}(\boldsymbol{x}_p) = d_p \\ 1 & otherwise \end{cases}$$

0/1 Loss

Def

- The mean over the data set provide the *number/percentage of misclassified patterns*
- *E.g. 20 out of 100 are misclassified → 20% errors, i.e. 80% of* ***accuracy***

3. Machine Learning & generalization

Dip. Informatica
University of Pisa

This is a fundamental concept of the course

- *Learning*: search for a *good function* in a function space from known data *(typically minimizing an Error/Loss)*

- Good w.r.t. generalization error: it measures how accurately the model predicts over novel samples of data (*Error/Loss measured over new data)*

Generalization: crucial point of ML!!!

Easy to **use** ML tools *versus* **correct/good use** of ML

Generalization

Dip. Informatica
University of Pisa

- **Learning** phase **(training, fitting)**: build the model from know data – *training data* (and bias)
- **Predictive** or **Test** phase (deployment/ *Inference* use of the ML built model): apply the model to new examples:
  - we take the new inputs ***x'*** and we compute the response by the model
  - we compare with its target *d'* that the model has never seen
  - i.e. we make evaluation of the generalization capability of our predictive hypothesis

Note: *performance* in ML = generalization accuracy/ *predictive accuracy*

estimated by the error computed on the (hold out) Test Set

- **Theory**: E.g. Statistical Learning Theory [Vapnik] :
  - *under what (mathematical) conditions is a model able to generalize?* → see next lecture (just basic notions)

ML in a Nutshell

Dip. Informatica
University of Pisa

- **DATA**: available experience represented as vectors, structures,…
- **TASK**: supervised (classification, regression), unsupervised, …
  - E.g. Given data as labeled examples, find good approximation of the unknown *f.*
- **MODEL:**
  - describes the relationships among the data / the knowledge
  - defines the class of functions that the learning machine can implement (*hypothesis space*)
- **LEARNING ALGORITHM:**
  - (given data, task and model) the learning algorithm performs a (heuristic) *search* through a space of hypotheses that are valid in the given data
  - E.g. it adapts the free parameters of the model to the task at hand

- VALIDATION: evaluate generalization capabilities (of your hp)

ML issues (II) Generalization

Dip. Informatica
University of Pisa

- **Learning phase**: to build the model (including training)
- **Prediction phase**: evaluate the learned function over novel samples of data (generalization capability)
- Inductive learning hypothesis
  - Any h that approximates f well on training examples will also approximate f well on new (unseen) instances x (?)

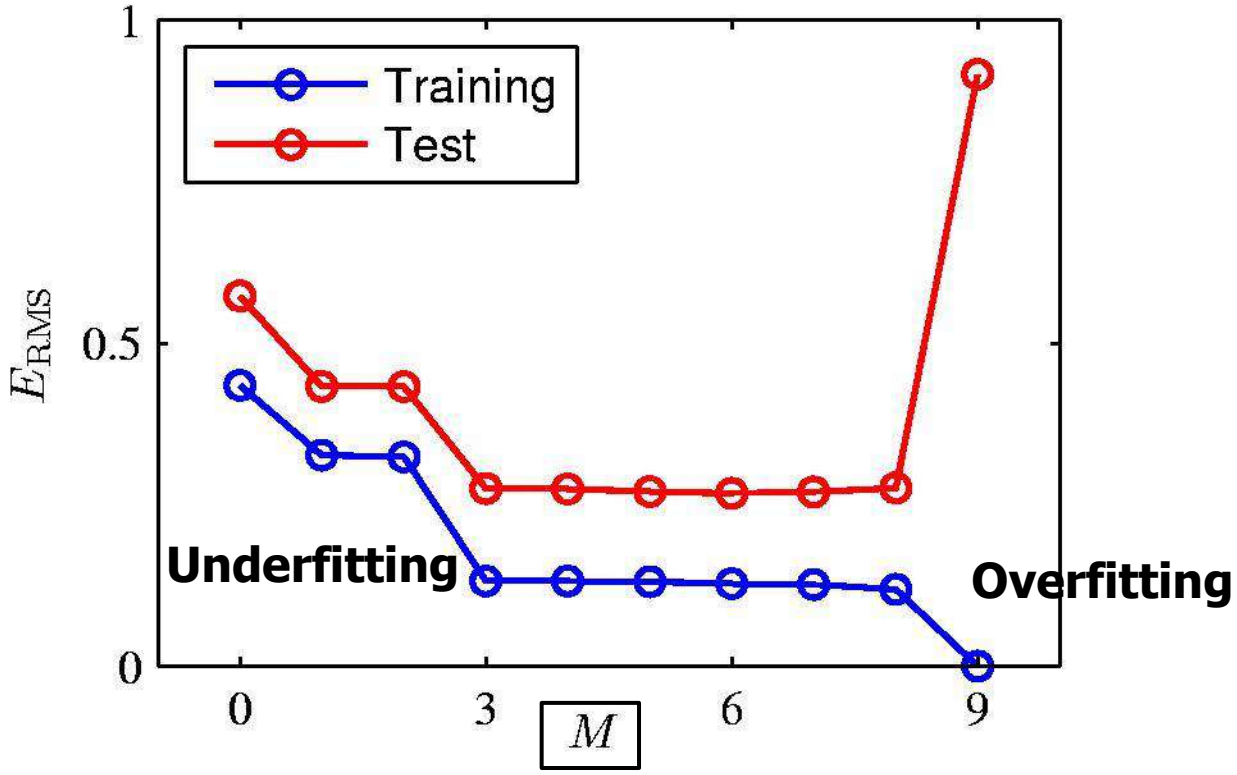
Def

- **Overfitting**: A learner overfits the data if
  - it outputs a hypothesis $h(\cdot) \in H$ having true/generalization error (risk) R and empirical (training) error E, but there is another $h'(\cdot) \in H$ having $E' > E$ and $R' < R$ (so that $h'(\cdot)$ is the better one, despite a worst fitting).

- Critical aspect: accuracy / performance estimation
  - Theoretical
  - Empirical (training, test) and cross-validation techniques

Underfitting and Overfitting with the complexity (*M*)

Dip. Informatica
University of Pisa



Root-Mean-Square (RMS) Error: $E_{\mathrm{RMS}} = \sqrt{2E(\mathbf{w}^\star)/\ l}$

Where $E(\mathbf{w}^*)$ is the error for the trained model

Toward SLT

Dip. Informatica
University of Pisa

Putting all together:

- We want to investigate on the *generalization* capability of a model (measured as a risk or test error)
  - with respect to the training error
  - overfitting and underfitting zones

- The role of model complexity
- The role of the number of data

- *Statistical Learning Theory (SLT):* a general theory relating such topics

(Simplified) Formal Setting

Dip. Informatica
University of Pisa

- Approximate unknown $f(\boldsymbol{x})$, d *is the target* (d=*true* f +*noise*)
- Minimize *risk function* $R = \int L(d, h(\boldsymbol{x}))dP(\boldsymbol{x}, d)$ True Error Over *all* the data
- Given
  - value from teacher (d) and the probability distribution $P(\boldsymbol{x}, d)$
  - a loss (or cost) function, e.g. $L(h(\boldsymbol{x}), d) = (d - h(\boldsymbol{x}))^2$

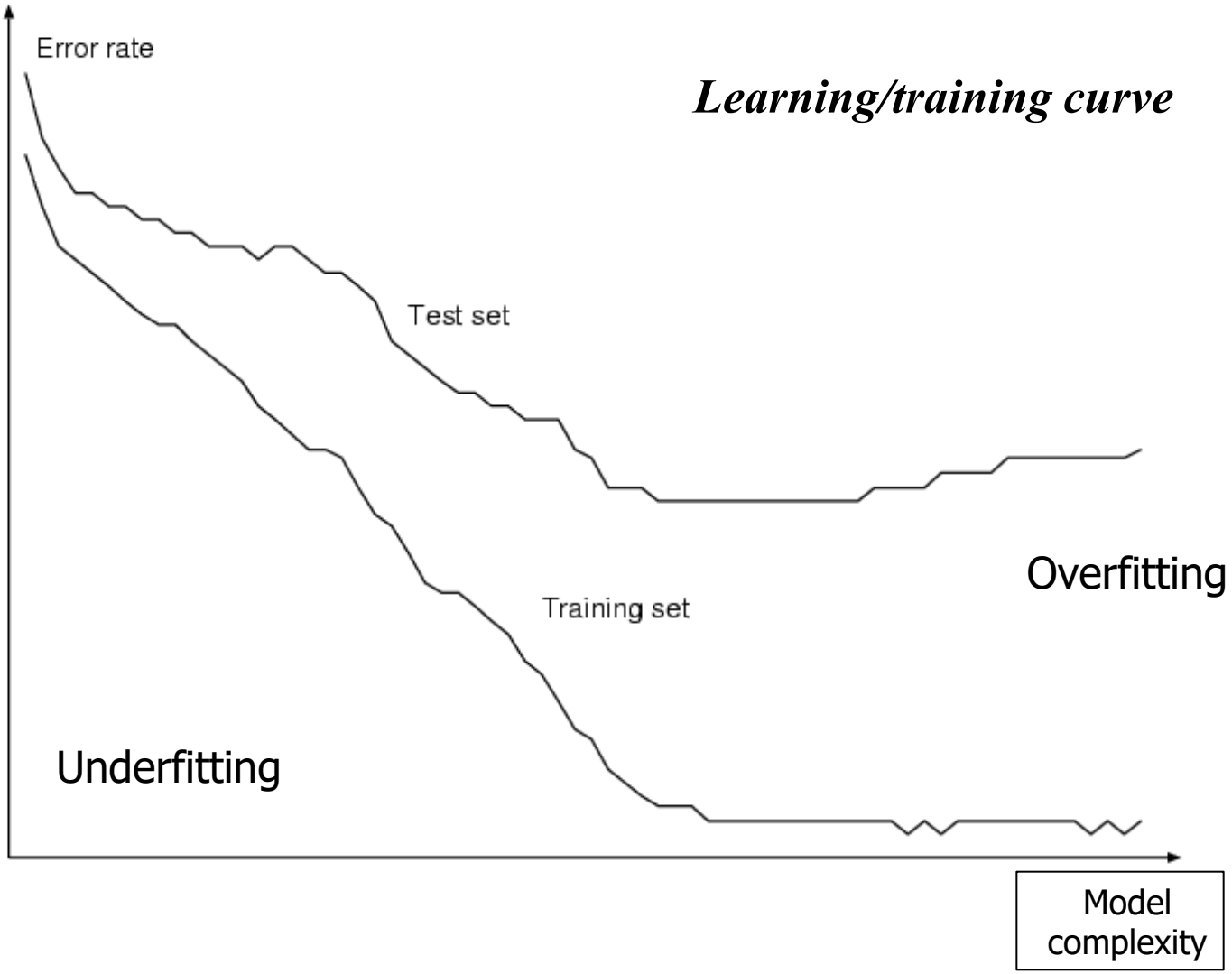
- Search h in H : Min R
- But we have only the finite data set $TR = (\boldsymbol{x}_p, d_p), \quad p = 1 \ldots l$
- To search h: minimize empirical risk (training error E), finding the best values for the model free parameters

$$R_{emp} = R_{emp}(h, TR) = \frac{1}{l}\sum_{p=1}^{l}(d_p - h(\boldsymbol{x}_p))^2$$

- Empirical Risk Minimization (ERM) Inductive Principle
- *Can we use R_{emp} to approximate R?*

Typical behavior of learning

Dip. Informatica
University of Pisa



Vapnik-Chervonenkis-dim and SLT: a general theory (I)

Dip. Informatica
University of Pisa

- Given the *VC-dim* (VC), a measure *complexity* of *H (flexibility to fit data)* (e.g. Num. of parameters for linear models/polynomials)

Repetita: Can we use R_{emp} to approximate R?

Def.

- *VC-bounds in the form:* it holds with probability 1-δ that

Very important!

guaranteed risk

$$R \leq R_{emp} + \varepsilon(1/l, VC, 1/\delta)$$

VC-confidence

- First (basic) explanation, ingredients:
  - ε is a function that grows with VC *(VC-dim)*, and that decreases with (higher) l and δ (i.e. l and δ are in the denominator)
  - We know that R_{emp} decreases using complex models (with high *VC-dim*) (e.g. the polynomial degree in the example)
  - δ is the confidence, it rules the probability that the bound holds (e.g. low delta 0.01 → the bound holds with probability 0.99)

- Now we can see how it can "explain" the *underfitting* and *overfitting* and the aspects that control them.

Vapnik-Chervonenkis-dim and SLT: a general theory (II)

Dip. Informatica
University of Pisa

Comments:

Very important!

- *VC-bounds in the form:* it holds with probability 1-δ that

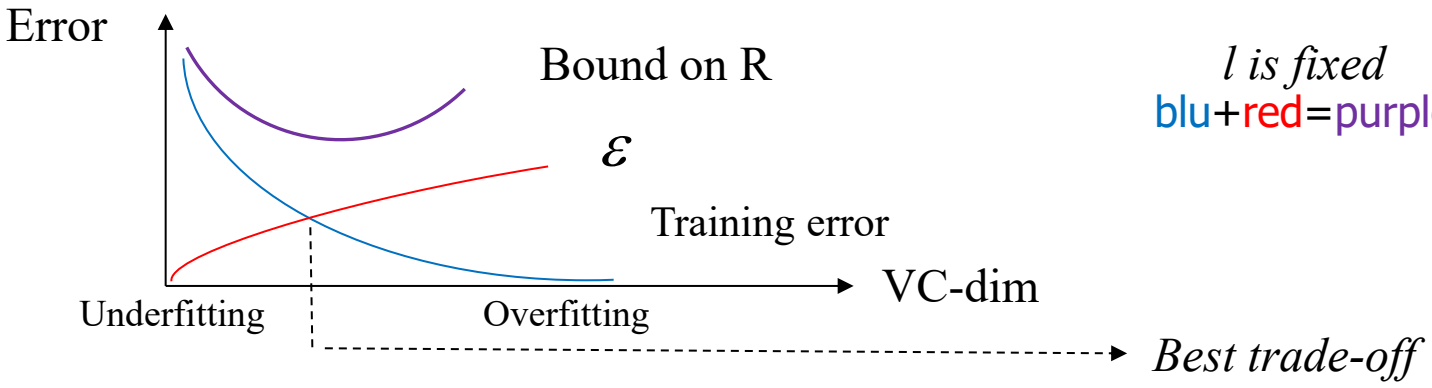
guaranteed risk $R \leq R_{emp} + \varepsilon(1/l, VC, 1/\delta)$

R_{emp}: empirical risk; ε: *VC-confidence*

Intuition:

- Higher l (data) → lower *VC-confidence* and a bound close to R
- Too simple models (low VC-dim) can be not suff. due to high R_{emp} (underfitting)
- Higher VC-dim (fix l) → lower R_{emp} but *VC-conf.*, and hence R, may increase (overfitting)

- *Structural risk minimization*: minimize the bound !



- Concept of control of the model complexity (flexibility):
  trade-off between model complexity (VC-dim) and TR accuracy (fitting)

Validation: Two aims

Very important!

Dip. Informatica
University of Pisa

- After models training on the training set
- ***Model selection:*** estimating the performance (*generalization error*) of different learning models in order to choose the best one (to generalize).
  - this includes search the best *hyper-parameters* of your model (e.g. polynomial order, …).

  It returns a model

- ***Model assessment:*** having chosen a final model, estimating/evaluating its prediction error/ risk (*generalization error*) on new *test* data (measure of the quality/performance of the ultimately chosen model).

  It returns an estimation

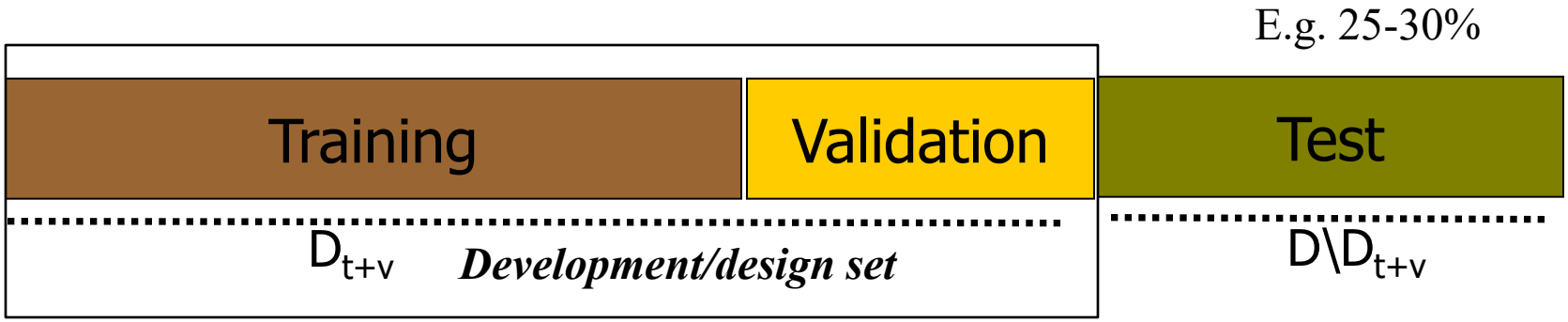
Gold rule: Keep separation between goals and use separate data sets

Hold out cross validation

Dip. Informatica
University of Pisa

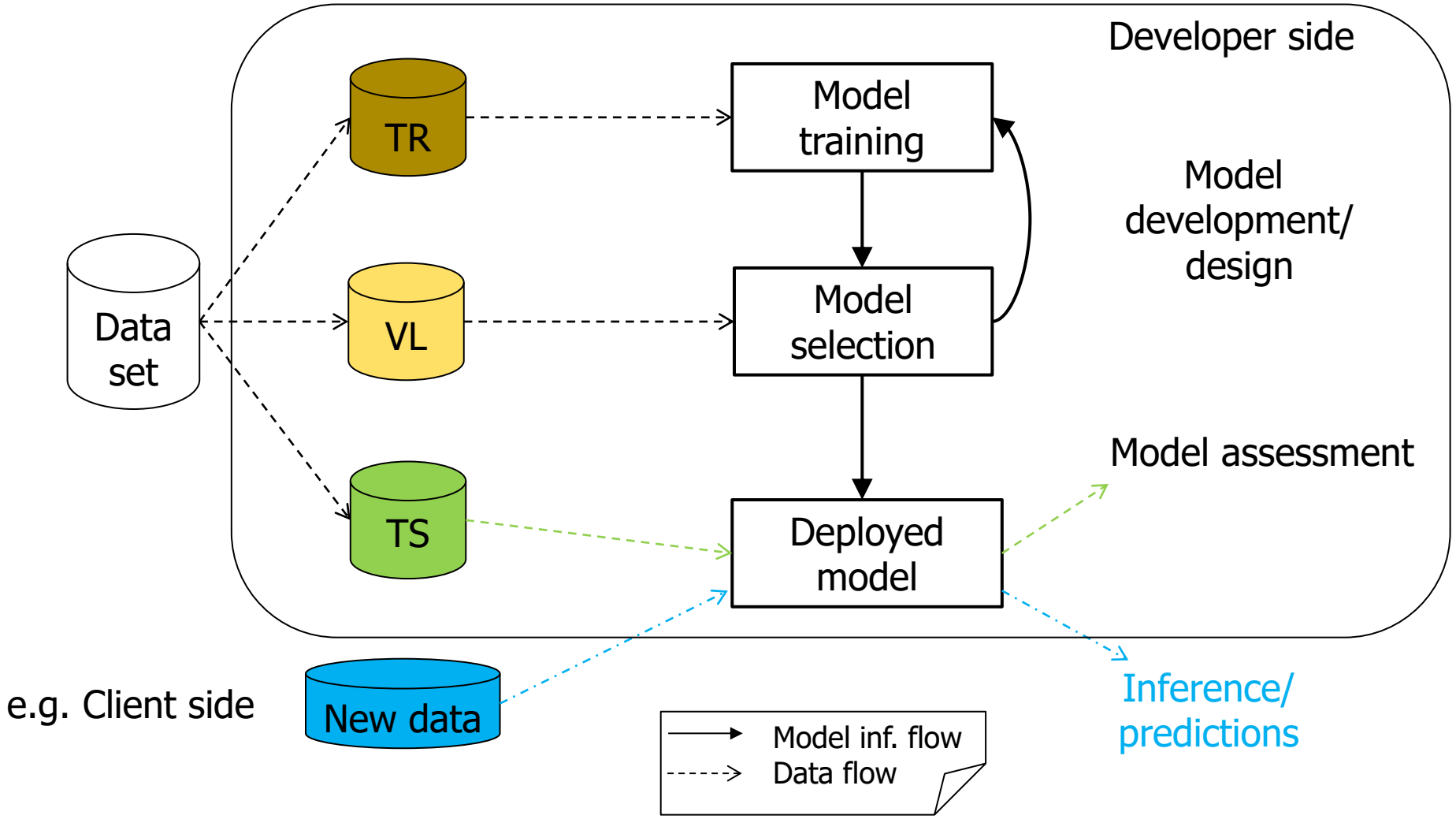
Hold out: basic setting

- Partition data set D into *training set* (TR), *validation or selection set* (VL) *and test* set (TS)
  - All the three sets are disjoint sets !!!
  - **TR** is used to run the **training** algorithm
  - **VL** can be used to **select the best model** (e.g hyper-parameters tuning)
  - **Test** set (result) is *not* to be used for tuning/selecting the best h: it is only for **model assessment**



TR/VL/TS by a schema

Dip. Informatica
University of Pisa



Classification Accuracy

Dip. Informatica
University of Pisa

Def

Confusion matrix

Actual \ Predicted	Positive	Negative
Positive	TP	FN
Negative	FP	TN

false positive (FP) :eqv. with false alarm

Specificity = TN / (FP + TN)

(True Negative rate =1 -FPR)

Sensitivity = TP / (TP + FN)

(True Positive rate or Recall)

(Precison= TP/(TP+FP))

Accuracy: % of correctly classified patterns = TP +TN / total

Note: for binary classif.: 50% correctly classified = "coin" (random guess) predictor!

Other topics:

- unbalanced data (e.g. 99% +) → trivial classifier exists ,
- ….

ROC curve

Dip. Informatica
University of Pisa

Confusion matrix

Actual \ Predicted	Positive	Negative
Positive	TP	FN
Negative	FP	TN

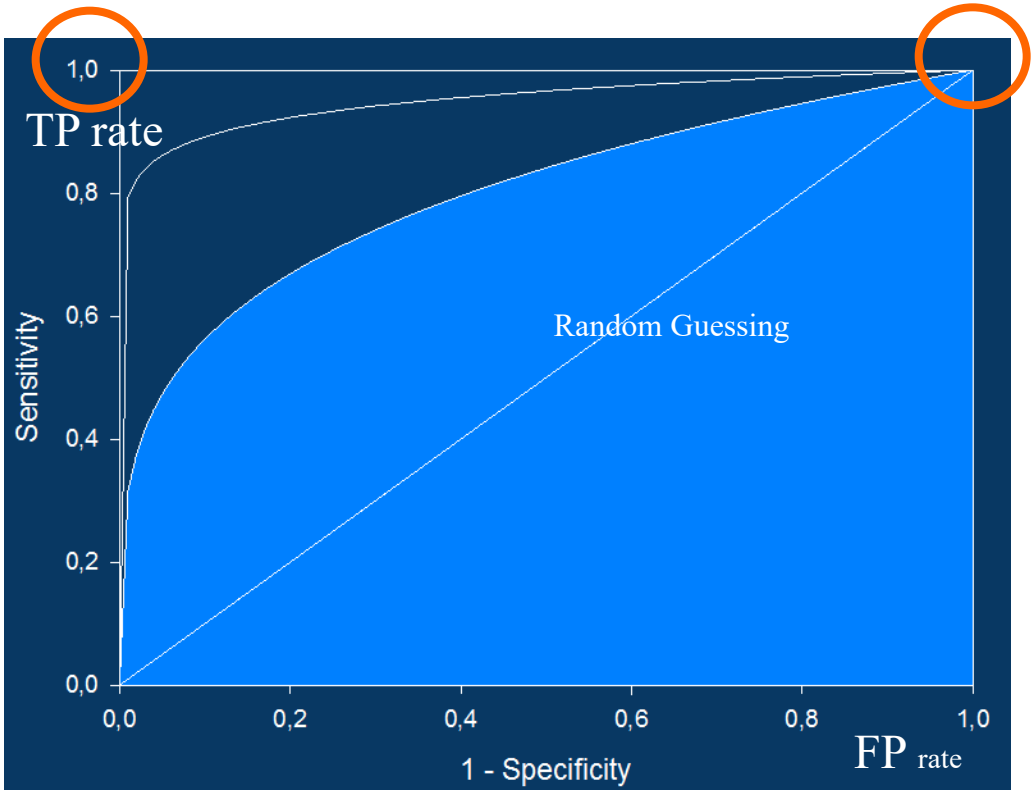
Specificity = TN / (FP + TN)

TPr or Sensitivity = TP / (TP + FN)

ROC curve

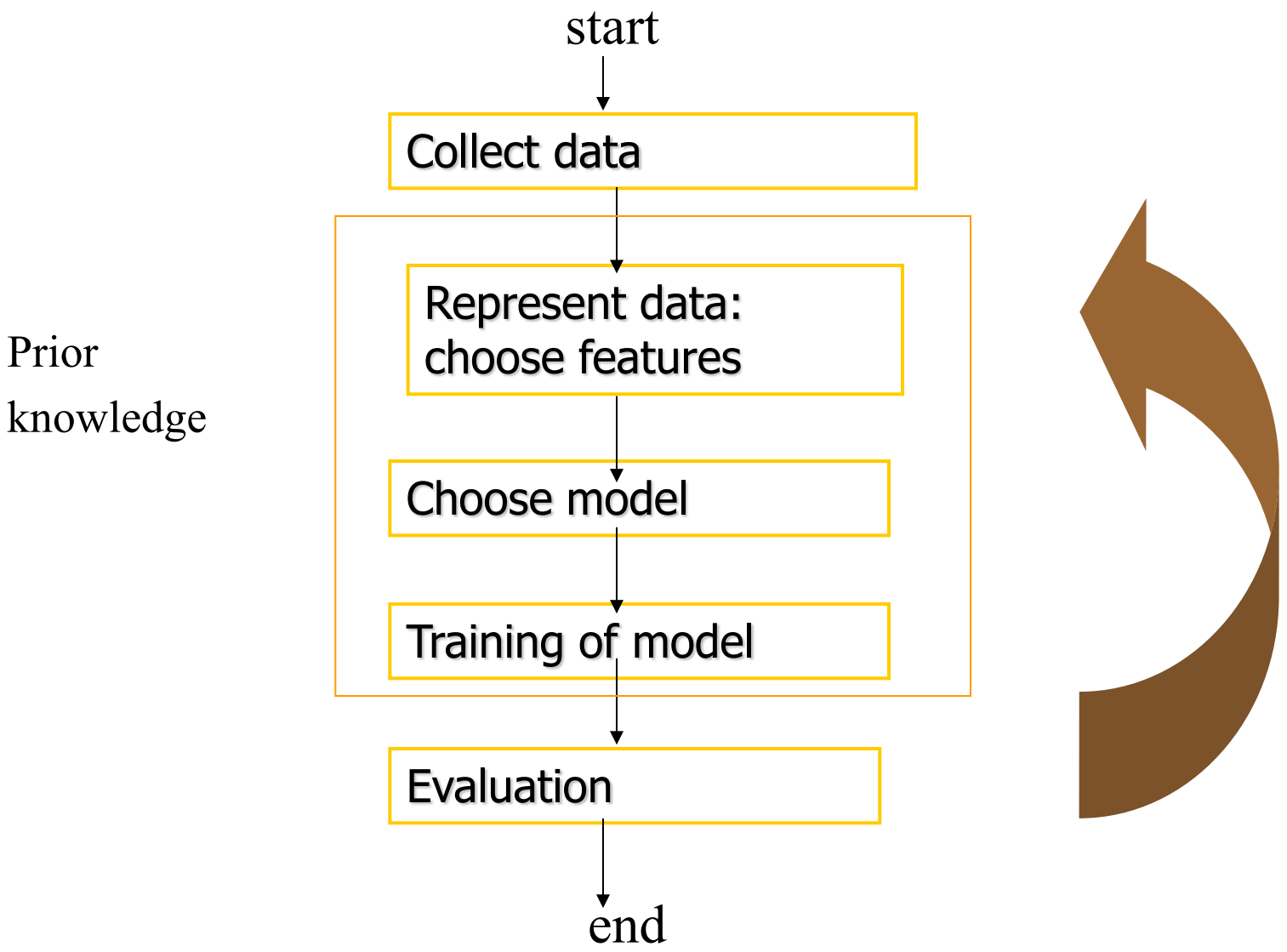
The diagonal corresponds to the worst classificator.

- Better curves have higher AUC (Area Under the Curve).



The Design Cycle

Dip. Informatica
University of Pisa

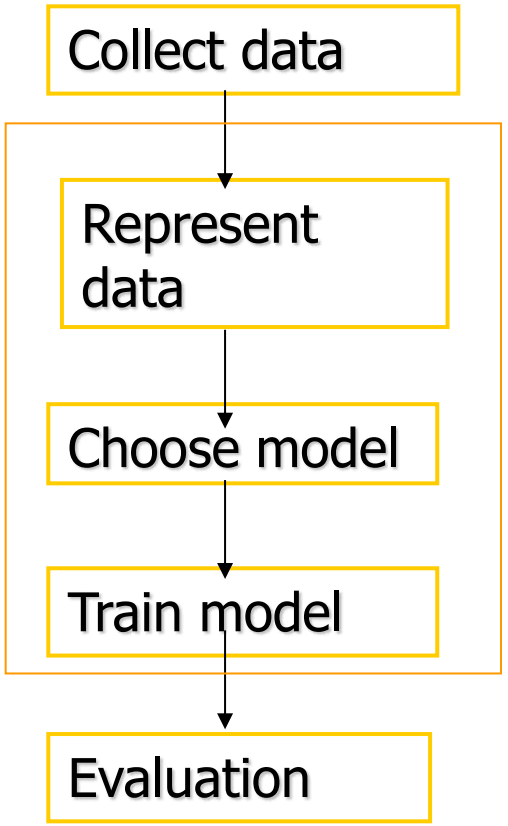


Design cycle (I)

Dip. Informatica
University of Pisa

- Data collection:
  - Selection, integration, data cleaning etc.
  - Adequately large and representative set of examples for training and test
- Data representation
  - Domain dependent, exploit prior knowledge of the application expert
  - Feature selection
  - Outliers detection
  - Other preprocessing: variable scaling, missing data,..

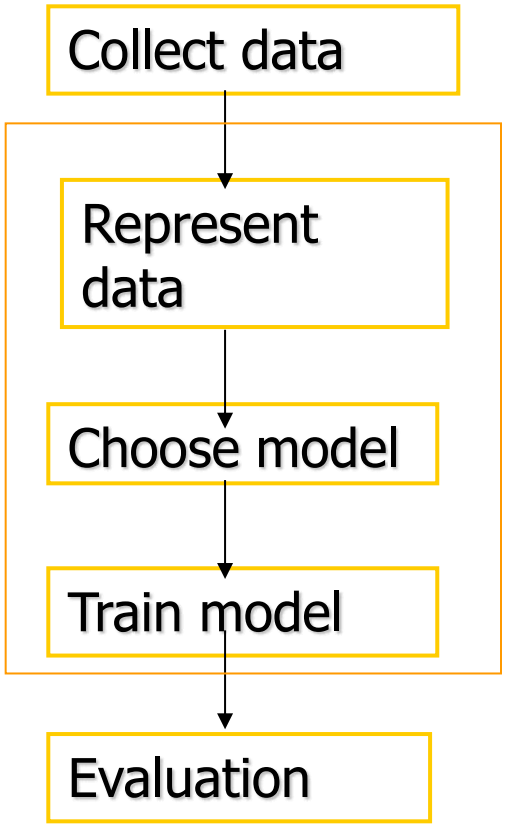
  Often the most critical phase for an overall success!
- Model choice:
  - Statement of the problem
  - Hypothesis formulation
    - You must know the limits of applicability of your model
  - complexity control



Design cycle (II)

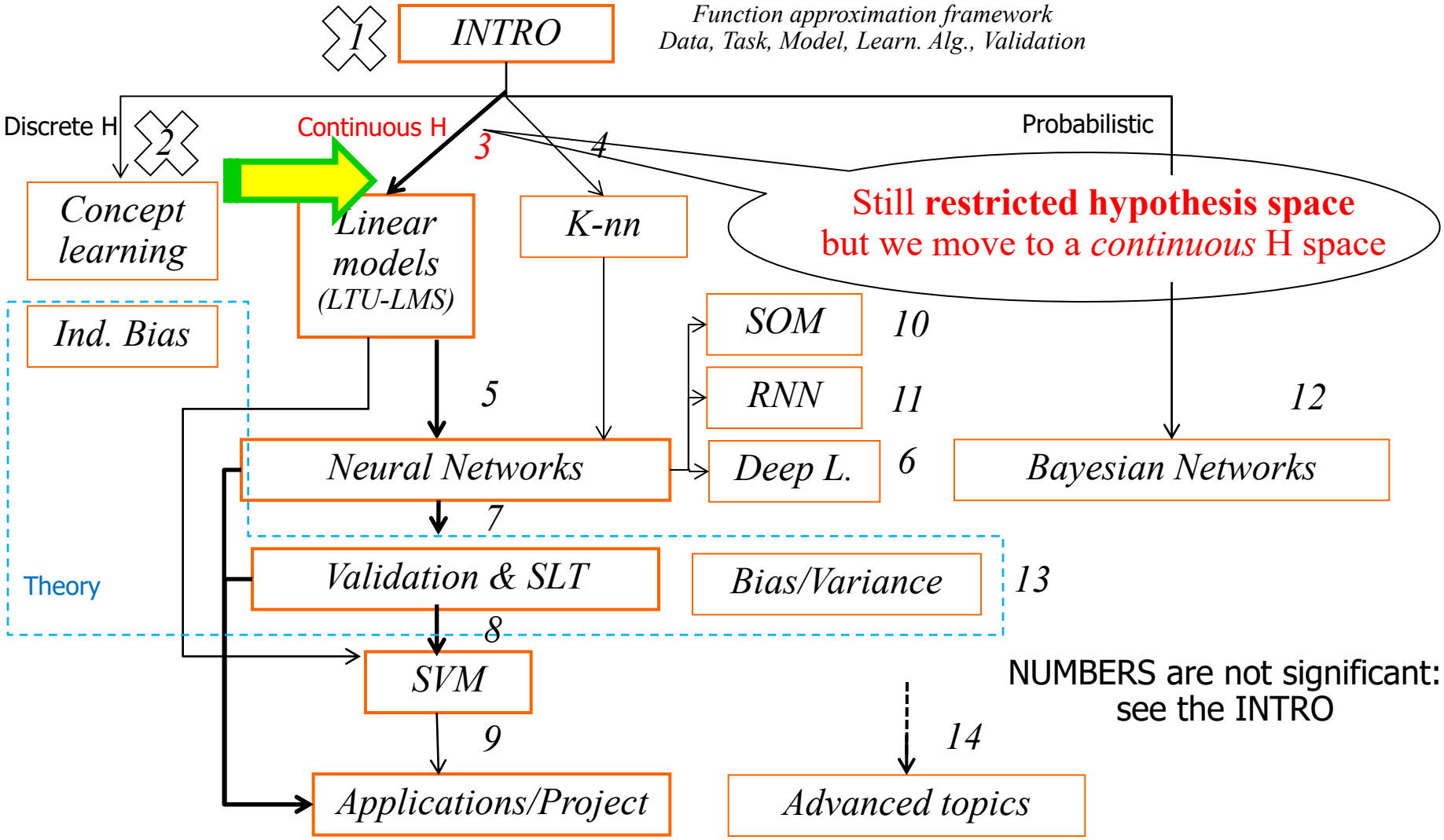
Dip. Informatica
University of Pisa

- Building of the model (core of ML):
  - through the learning algorithm using the training data
- Evaluation:
  - Performance = predictive accuracy !
  - Also *interpreation* of model outcomes and *explanation* of the results
  - "knowledge" extraction
- Deployment…



ML Course structure
Where we go

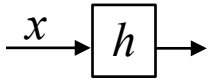
Dip. Informatica
University of Pisa



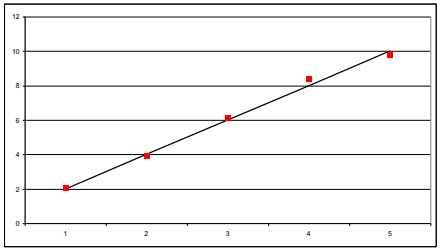
Univariate Linear Regression

Dip. Informatica
University of Pisa

- Univariate case, simple linear regression :
- We start with 1 input variable x, 1 output variable y
- We assume a model $h_{\mathbf{w}}(x)$ expressed as $out = w_1 x + w_0$



- *where* ***w*** *are real-valued coefficients/free parameters (weights)*
- ***Fitting*** the data by a "straight line"



- Infinite hp space (continuous w values) but we have nice solution from classical math (going back to Gauss/Legendre ~1795!)
  - Surprisingly we can "learn" by this basic tool
  - Although simple it includes many relevant concept of modern ML and it is a basis of evolved methods in the field

Build it: Learning via LMS (I)

Dip. Informatica
University of Pisa

- Training → find $\boldsymbol{w}$ such that minimize **error**/empirical **loss** (best data fitting – on the training set with l examples): i.e. we are now focusing on the R_{emp}
- **Given** a set of l training examples $(\boldsymbol{x}_p, y_p) \quad p=1..l$
- **Find**: $h_w(x)$ in the form $w_1 x + w_0$ (hence the values of $\boldsymbol{w}$) that minimizes the expected loss on the training data.

- For the loss we use the square of errors:
- Least (Mean) Square: Find $\boldsymbol{w}$ to *minimize* the residual sum of squares $[argmin_{\mathbf{w}}\ Error(\mathbf{w})\ in\ L_2]$:

$$Loss(h_{\mathbf{w}}) = E(\boldsymbol{w}) = \sum_{p=1}^{l} (y_p - h_{\mathbf{w}}(x_p))^2 = \sum_{p=1}^{l} (y_p - (w_1 x_p + w_0))^2$$

p runs over patterns/examples

where x_p is p-th input/pattern/example, y_p the output for p, w free par., l num. of examples

Note: to have the mean divide by l

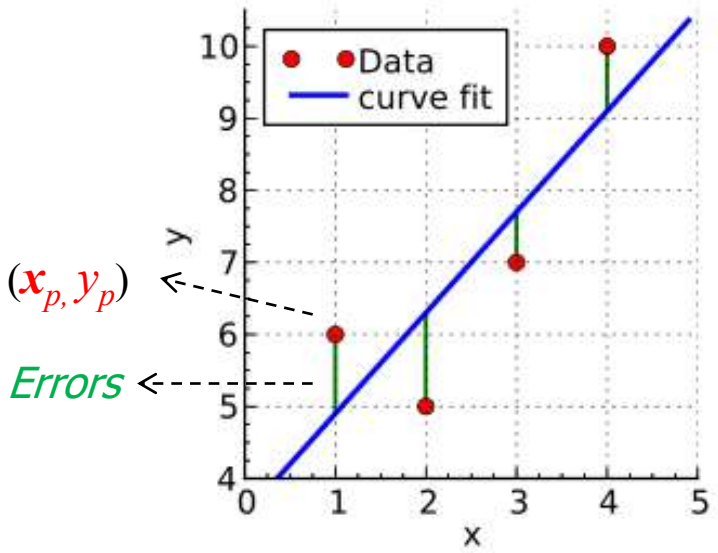
On the notation: Indeed for the univariate case, with 1 variable: $x_p = x_{p,1} = (\boldsymbol{x}_p)_1$

Build it: Learning via LMS (II)

Dip. Informatica
University of Pisa

Why LMS to fit the data with *h* ?

- Least (Mean) Square: Find w to *minimize* the residual sum of squares $[argmin_{\mathbf{w}}\ Error(\mathbf{w})\ in\ L_2]$:



$h_w(x)$

$y = w_1 x + w_0 + noise$

Different blue lines will have different green bars.
Minimizing the green bars (residuals /errors)
is a way to find the best approximation/fitting of the data
(i.e. our $h_w(x)$ or blue line).
The squares of errors $E(\mathbf{w})$ quantify such green bars:

$$E(w) = \sum_{p=1}^{l} \left(y_p - h_w(x_p)\right)^2$$

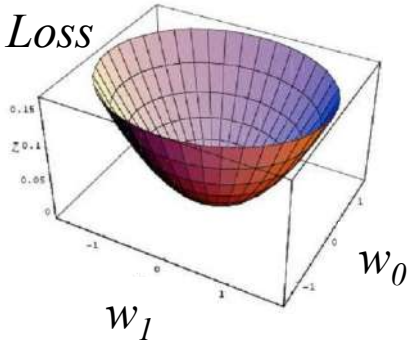
- The method of **least squares** is a standard approach to the approximate solution of over-determined systems, i.e., sets of equations in which there are more equations than unknowns.

How to solve?

Dip. Informatica
University of Pisa

- Remember: local minimum as stationary point: the gradient is zero

$$\frac{\partial E(\mathbf{w})}{\partial w_i} = 0, \qquad i = 1,\ldots,\dim_input + 1 = 1,\ldots,n+1$$



- For the simple Lin. Regr. (2 free parameters)

Search the $\boldsymbol{w}$ such that $\quad \frac{\partial E(\mathbf{w})}{\partial w_0} = 0 \quad \frac{\partial E(\mathbf{w})}{\partial w_1} = 0$

Convex loss function → we have the following solution (no local minima)

(just to know that it exists!)

$$w_1 = \frac{\sum_{p:1\to l} x_p y_p - \frac{1}{l}\sum x_p \sum y_p}{\sum_{p:1\to l} x_p^2 - \frac{1}{l}(\sum x_p)^2} = \frac{\mathrm{Cov}[x,y]}{\mathrm{Var}[x]}, \qquad w_0 = \overline{y} - w_1\overline{x}$$

$$\frac{1}{l}\sum_{p->l} y_p \qquad \frac{1}{l}\sum_{p->l} x_p$$

Exercise: *compute w_0 and w_1 according to the next slide results for the gradient (extended to l patterns)*

Compute the gradient for 1 (each) pattern *p*

Dip. Informatica
University of Pisa

Redo this by yourself as an *Exercise*

Basic rules:

$$\frac{\partial}{\partial w}k = 0, \frac{\partial}{\partial w}w = 1, \frac{\partial}{\partial w}w^2 = 2w$$

$$\frac{\partial (f(w))^2}{\partial w} = 2f(w)\frac{\partial (f(w))}{\partial w}$$

Der. sum = sum of der.

We will call (y-$h(x)$) "delta"

$$\frac{\partial E(\mathbf{w})}{\partial w_i} = \frac{\partial (y - h_{\mathbf{w}}(x))^2}{\partial w_i} =$$

$$= 2(y - h_{\mathbf{w}}(x))\frac{\partial (y - h_{\mathbf{w}}(x))}{\partial w_i} = 2(y - h_{\mathbf{w}}(x))\frac{\partial (y - (w_1 x + w_0))}{\partial w_i}$$

$$\frac{\partial E(\mathbf{w})}{\partial w_0} = -2(y - h_{\mathbf{w}}(x))$$

$$\frac{\partial E(\mathbf{w})}{\partial w_1} = -2(y - h_{\mathbf{w}}(x)) \cdot x$$

Then we will sum up for l patterns (x_p, y_p) …

And we will extend to multidimensional $\boldsymbol{x}$ and $\boldsymbol{w}$ *(n=1 here)* …

Linear model: notation for multidimensional inputs

Dip. Informatica
University of Pisa

- Assuming column vector for $\boldsymbol{x}$ and $\boldsymbol{w}$ *(in bold)*
- Number of data l, dimension of input vector n, y_p *(targets)* $p=1..l$

d_p or t_p

$$\boldsymbol{w}^T\boldsymbol{x} + w_0 = w_0 + w_1x_1 + w_2x_2 + \cdots + w_nx_n = w_0 + \sum_{i=1}^{n} w_ix_i \qquad \text{(eq. 1)}$$

- Note that sometimes (in NN) the transpose notation T in $\boldsymbol{w}^T$ is omitted

- w_0 is the *intercept, threshold, bias, offset*.....

Often it is convenient to include the constant $x_0 = 1$ so that we can write eq.1 as :

$$\boldsymbol{w}^T\boldsymbol{x} = \boldsymbol{x}^T\boldsymbol{w}$$

Inner product

$$\boldsymbol{x}^T = [1, x_1, x_2, \ldots, x_n]$$

$$\boldsymbol{w}^T = [w_0, w_1, w_2, \ldots, w_n]$$

So, the "linear "model can be written as a function that for each $\boldsymbol{x}_p$ compute:

$$h(\boldsymbol{x}_p) = \boldsymbol{x}_p{}^T\boldsymbol{w} = \sum_{i=0}^{n} x_{p,i}\, w_i$$

***w*: continuous (free) parameters: "weights"**

Learning algorithm: just wait!

Dip. Informatica
University of Pisa

- For the learning algorithm in the multidimensional case for linear regression: please wait.
- We will provide it along with the learning algorithm of the linear classifier in the next few slides.

Summing up:

- Given the data set and the linear model, we can state the learning problem as LMS problem
- Once we find the best w parameters values, we have our $h_w(x)$ for regression purposes

- For students that need a soft intro to just LMS Regression:
  - https://svivek.com/teaching/machine-learning/lectures/slides/linear-models/lms-regression.pdf

Classification

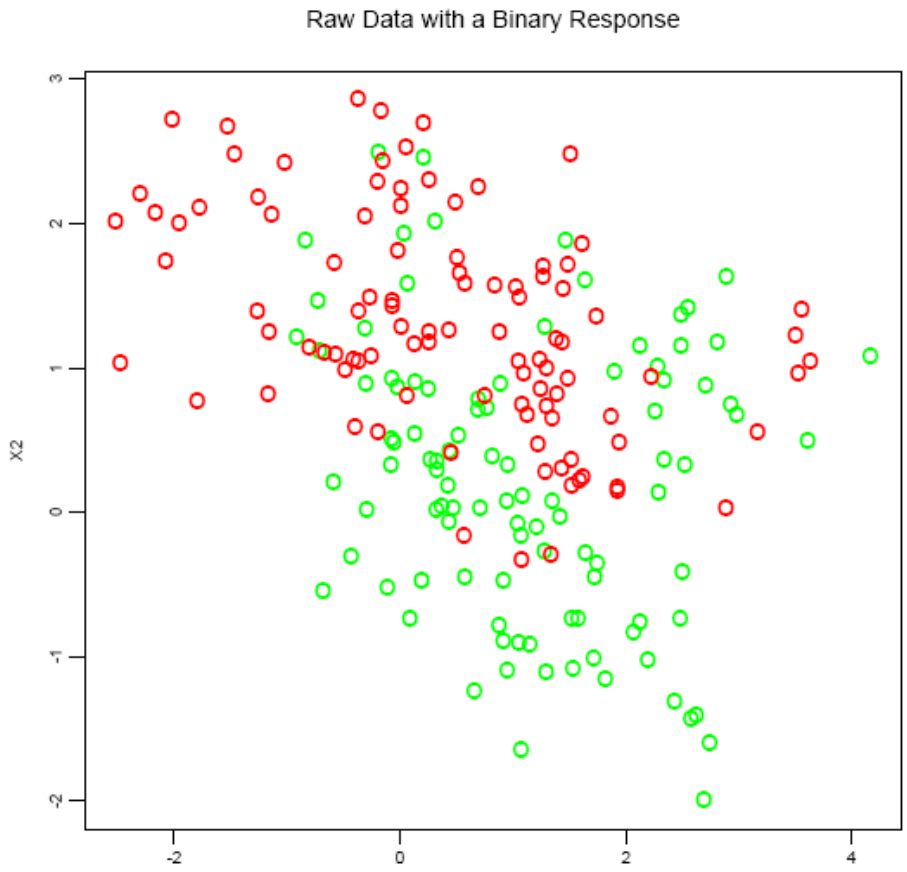
What we are looking at

Dip. Informatica
University of Pisa

1. The classification by hyperplanes
2. See how the model works after training ("use it" slides)
3. How to state/formulate a (regression)/classification learning problem for a linear model by LMS
4. How to derive the learning algorithm
5. Proposing two learning algorithms to build a linear classifier

Problem: example

Dip. Informatica
University of Pisa



200 points generated in IR^2 from an unknown distribution; 100 in each of two classes.

Can we build a rule to predict the color of future points?

Data may be generated by gaussian distribution (for each class) with different means
or by a mixture of different low variance gaussian distributions.

We reuse the linear model

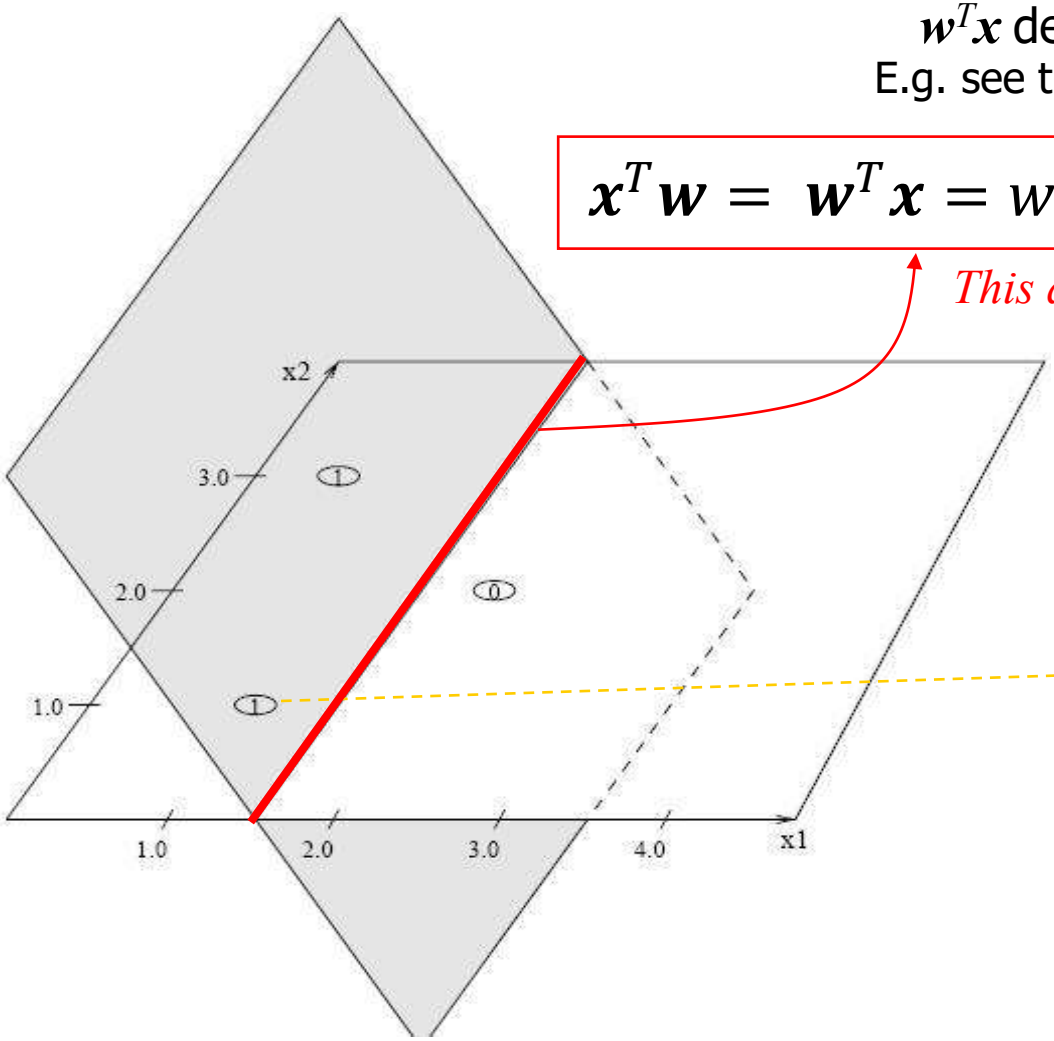
Dip. Informatica
University of Pisa

- The same models (used for regression) can be used for **classification**: categorical targets (*y* or *d*), **e.g. 0/1 or -1/+1.**

  - In this case we use a hyperplane ($\boldsymbol{w}^T\boldsymbol{x}$) assuming negative or positive values
  - We exploit such models to decide if a point $\boldsymbol{x}$ belong to positive or negative zone of the hyperplane (to classify it)
  - So we want to set $\boldsymbol{w}$ (by learning) s.t. we get good classification accuracy

Geometrical view: hyperplane

Dip. Informatica
University of Pisa



$\boldsymbol{w}^T\boldsymbol{x}$ define an hyperplane.
E.g. see the picture for 2 variables

$$\boldsymbol{x}^T\boldsymbol{w} = \boldsymbol{w}^T\boldsymbol{x} = w_0 + w_1x_1 + w_2x_2 = 0$$

This defines the decision boundary

Can be used to classify:

Examples $\langle(x_1,x_2), y\rangle$:

<(1.0, 1.0), 1>

<(0.5, 3.0), 1>

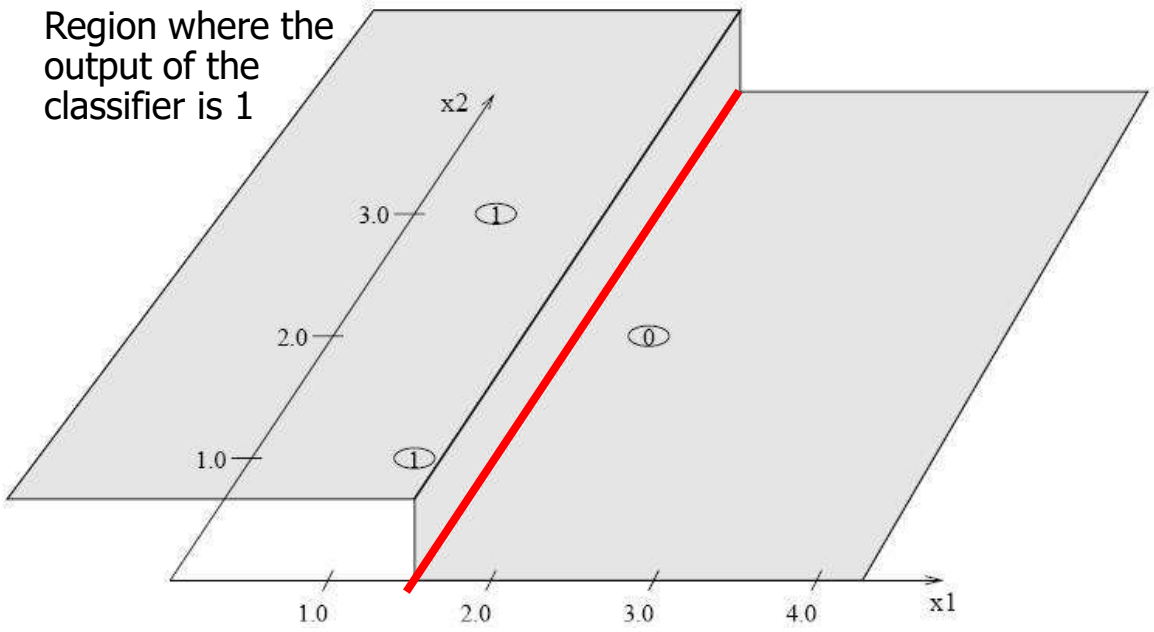
<(2.0, 2.0), 0>

Geometrical view: classifier

Dip. Informatica
University of Pisa

Introducing a threshold function

Region where the output of the classifier is 1



Examples:

<(1.0, 1.0), 1>

<(0.5, 3.0), 1>

<(2.0, 2.0), 0>

[0,1] output range

$$h(\boldsymbol{x}) = \begin{cases} 1 & \text{if } \boldsymbol{w}^T\boldsymbol{x} + w_0 \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

or

[-1,+1] output range

$$h(\boldsymbol{x}) = sign(\boldsymbol{w}^T\boldsymbol{x} + w_0)$$

Using $\boldsymbol{x}_p$ and including w_0 in $\boldsymbol{w}$

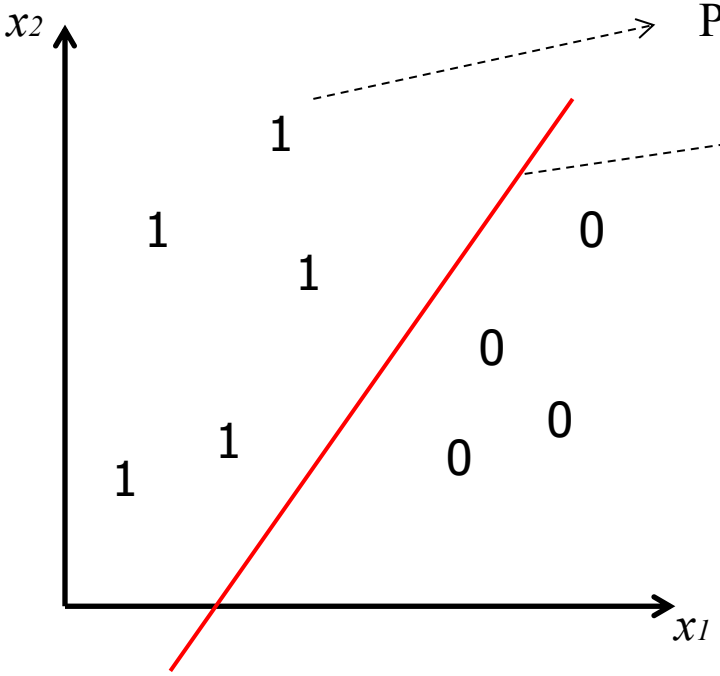
$$h(\mathbf{x}_p) = sign(\mathbf{x}_p{}^T\mathbf{w}) = sign\left(\sum_{i=0}^{n} x_{p,i} w_i\right)$$

In this slide $\boldsymbol{w}$ is omitted from $h_{\boldsymbol{w}}$ (we use just h)

Classification by linear decision boundary [repetita]

Dip. Informatica
University of Pisa

The classification may be viewed as the allocation of the input space in decision regions (e.g. **0/1**)
Example: linear separator on
2-dim instance space $\boldsymbol{x}=(x_1,x_2)$ *in* IR^2 , $f(\boldsymbol{x})=0/1$ *(or -1/+1)*



Point belonging to class 1

Separating (hyper)plane : $\boldsymbol{x}$ s.t.

$$\boldsymbol{w}^T\boldsymbol{x} + w_0 = w_1x_1 + w_2x_2 + w_0 = 0$$

$$h(\boldsymbol{x}) = \begin{cases} 1 & if \quad \boldsymbol{w}^T\boldsymbol{x} + w_0 \geq 0 \\ 0 & otherwise \end{cases}$$

[0,1] output range

or

$$h(\boldsymbol{x}) = sign(\boldsymbol{w}^T\boldsymbol{x} + w_0)$$

[-1,+1] output range

Linear threshold unit (LTU)

Indicator functions

How many? (H): set of dichotomies induced by hyperplanes

Threshold (bias w_0)

Dip. Informatica
University of Pisa

Note that, given the bias w_0, in the LTU

saying $h(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + w_0 \geq 0$

is equivalent to say $h(\mathbf{x}) = \mathbf{w}^T\mathbf{x} \geq -w_0$

with $-w_0$ as the «threshold» value

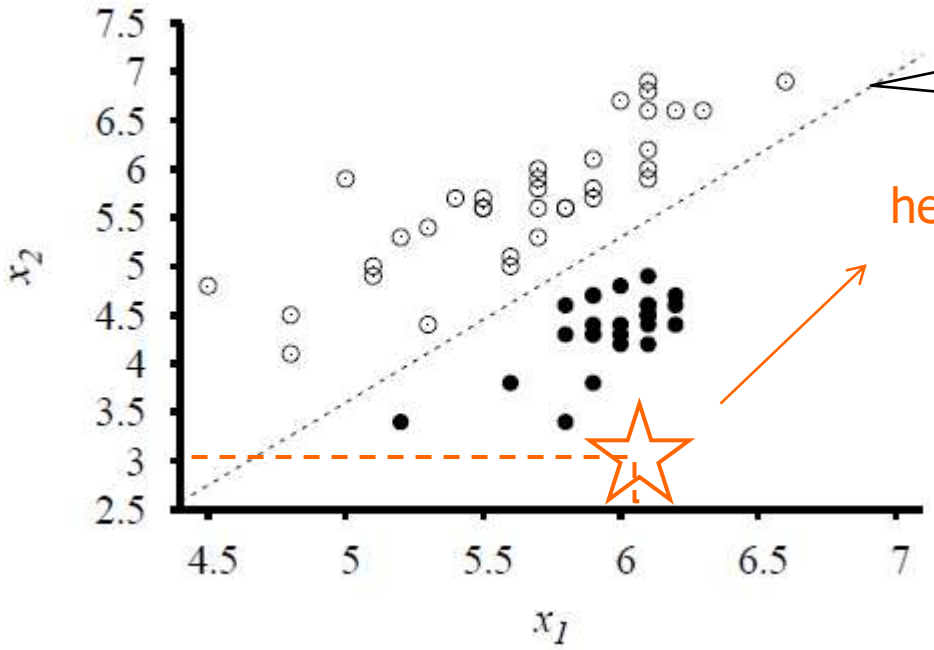
- The two forms identify the same positive zone of the classifier
- The second one emphasizes the role of the bias as a threshold value to "activate" the +1 output of the classifier.

Use it: Example (AIMA)
Classify a new data (x_1,x_2)

Dip. Informatica
University of Pisa

Find *h* s.t. given (x_1,x_2) return 0/-1 for *Earthquakes* and 1 for *Nuclear Explosion*

Some alg. finds this decision boundary:



$$w_0 + w_1 x_1 + w_2 x_2 = 0$$
$$\Updownarrow \quad \Updownarrow \quad \Updownarrow$$
$$-4.9 + 1.7 x_1 - 1 x_2 = 0$$

hence ***h(6,3)***$=sign(w_0 + w_1 * 6 + w_2 * 3) =$

$=$***sign(- 4.9 + 1.7*6 -1*3)***

$=$***sign(2.3) =+1*** ➔ ***nuclear expl.***



Seismic data.

x_1 body wave magnitude, x_2 surface wave magnitude

Earthquakes (white) Nuclear Explosion (black) 1982-1990 Asia

Getting new examples is expensive;-)

Use it: Example (Spam)

Dip. Informatica
University of Pisa

- Find $h(mail)$ +1 for $spam$, -1 $not\text{-}spam$
  - Features Phi(*mail*) = words [0/1] or phrases ("free money") [0/1] or length [integer]
  - e.g $\phi_k(\mathbf{x}) = contain(word_k)$ [*bag of words* representation]
- $\boldsymbol{w}$ → weight contribution of the input features to prediction
  - e.g. positive weight for "free money", negative for ".edu" or "unipi"
- $\boldsymbol{x}^T\boldsymbol{w}$ is the weight combination
- $h_{\mathbf{w}}(x)$ provides the threshold to decide spam/not spam

$$h_{\mathbf{w}}(\mathbf{x}) = sign\left(\sum_k w_k \phi_k(\mathbf{x})\right)$$

>0 ➔ + 1 = Spam !

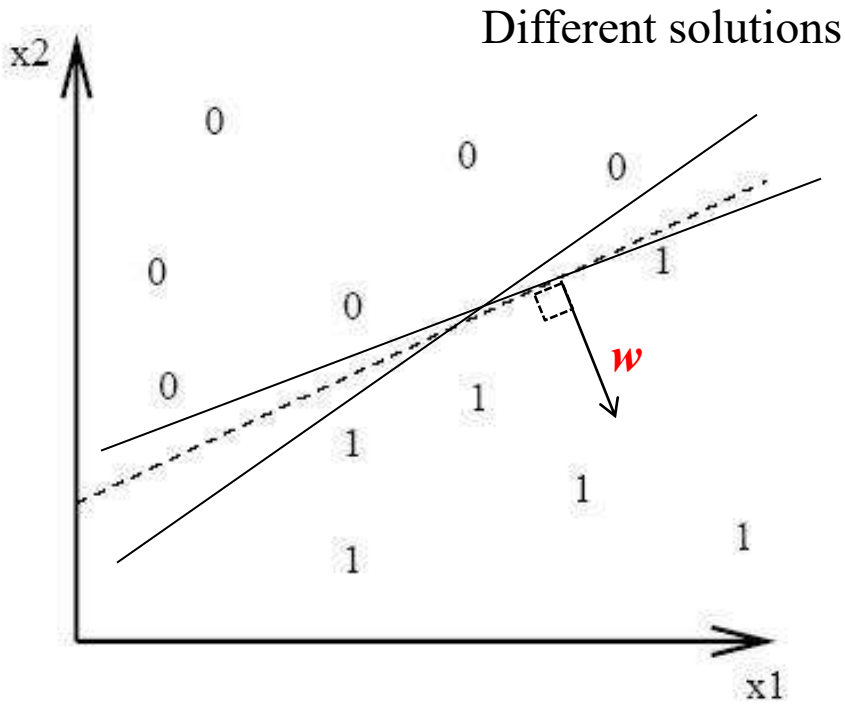


Some useful properties
(we will use them in future lectures)

Dip. Informatica
University of Pisa

$$\boldsymbol{w}^T \boldsymbol{x} = w_1 x_1 + w_2 x_2 + w_0 = 0$$

Exercise: Draw $x_2 = - x_1 \, w_1/w_2 - w_0/w_2$ with different $\boldsymbol{w}$ values



A linearly separable problem

- If w_0=0 the line goes through the *origin* of the coordinate system.
- If $n > 2$ → *hyperplane (decision boundary)*
- *Scaling freedom*: the same decision boundary multiplying $\boldsymbol{w}$ by K
- $\boldsymbol{w}$ is a vector *orthogonal* to the hyperplane:

Given $\boldsymbol{x_a, x_b}$ ***(belonging to the sep. hyperplane):***
$\boldsymbol{w^T x_a + w_0 = 0;\ w^T x_b + w_0 = 0}$ *(take the diff.)* →
$\boldsymbol{w^T (x_a - x_b) = 0}$ → ***orthogonal vectors*** *(dot prod. 0)*

- If it exist, there are many possible hyperplanes separating these points: also many ML alg.!!!

Learning Algorithms

Dip. Informatica
University of Pisa

- We are going to introduce 2 learning algorithms for the regression and for the classification task using a linear model, both based on LMS:

  1. A direct approach based on **normal equation** solution

  2. An iterative approach based on **gradient descent**

- We start redefining the learning problem and the loss for them (for l data and multidimensional inputs)

The learning problem (classification tasks)

Dip. Informatica
University of Pisa

- **Given** a set of l training examples $(\boldsymbol{x}_p, y_p)$ and a loss function (measure) L

$$y_p = \{0,1\} \text{ or } y_p = \{-1,+1\}$$

- **Find**: The weight vector $\boldsymbol{w}$ that minimizes the expected loss on the training data

$$R_{emp} = \frac{1}{l}\sum_{p=1}^{l} L(h(\boldsymbol{x}_p), y_p)$$

- For classification: Using a piecewise constant (over $sign(\boldsymbol{w}^T\boldsymbol{x})$) for the loss can make this a difficult problem.

- Assume we still use the *least squares (as for the regression case)*

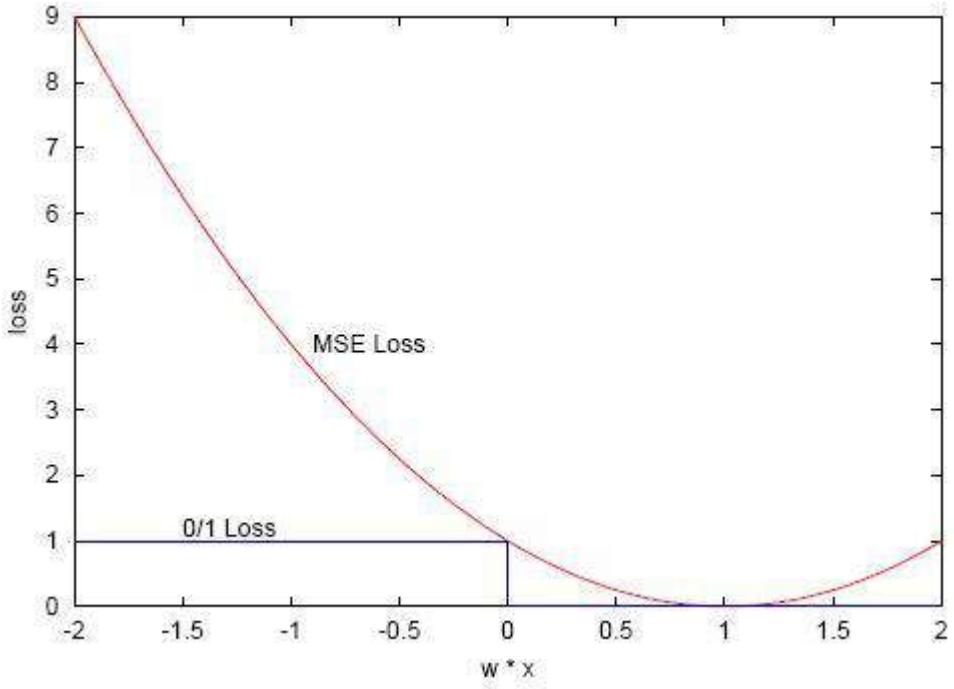
$$E(\boldsymbol{w}) \propto \sum_{p=1}^{l}(y_p - \boldsymbol{x}_p{}^T\boldsymbol{w})^2 = \sum_{p=1}^{l}(y_p - \boldsymbol{w}^T\boldsymbol{x}_p)^2$$

Approximating expected loss by a smooth function (#)

Dip. Informatica
University of Pisa

- Initially, we can make the optimization problem easier by replacing the original objective function *L (0/1 loss)* by **a smooth, *differentiable function***. For example, consider the popular *mean squared error (MSE loss)*:

Both losses satisfy the minimization of error *(*)*,
Let us start with LMS avoiding to introduce combinatorial problems



(*) Example: y (target)=1,
$h(\boldsymbol{x})=1$ if $\boldsymbol{w}^T\boldsymbol{x}>0$,
→ No err. when $\boldsymbol{w}^T\boldsymbol{x}>0$
for the classifier

Hence no classif. error
minimizing either

- *0/1 loss*
- *or MSE loss*

(solution is for both
on the right part in the plot)

Learning (a classifier) by Least Squares

Dip. Informatica
University of Pisa

- Find optimal values for $\boldsymbol{w}$ (for fitting of training (TR) data) by *least squares*:
- **Given** a set of l training examples $(\boldsymbol{x}_p, y_p)$, **find** $\boldsymbol{w}$ to *minimize* the residual sum of squares:

$$E(\boldsymbol{w}) = \sum_{p=1}^{l} (y_p - \boldsymbol{x}_p{}^T \boldsymbol{w})^2 = \quad ||\boldsymbol{y} - \boldsymbol{X}\boldsymbol{w}||^2$$

Where $\boldsymbol{x}_p$ is p-th input vector, y_p the output for p, $\boldsymbol{w}$ free par., l num. of examples, n input dim.

Min error: if $y_p = 1$ then $\boldsymbol{x}_i{}^T\boldsymbol{w}$ go toward 1 → *no class. error*;
if $y_p = -1$ then $\boldsymbol{x}_p{}^T\boldsymbol{w}$ go toward -1 → *no class. Error*

***Note**: in $E(\mathbf{w})$ we do **not** use $h(\boldsymbol{x})$, as for regression, to hold a continuous **differentiable** loss (because $h(\boldsymbol{x}) = \text{sign}(\boldsymbol{w}^T\boldsymbol{x})$ for classification) !!!*

- This is a quadratic function → minimum always exists (but may be not unique) [see course of CM or O4DS (@DSBI)]
- $\boldsymbol{X}$ is a matrix $\boldsymbol{l}\,\boldsymbol{x}\,\boldsymbol{n}$ with a row for each input vector $\boldsymbol{x}_p$
- Note: The same approach is used for a *regression problem*

Normal equation & direct approach solution

Dip. Informatica
University of Pisa

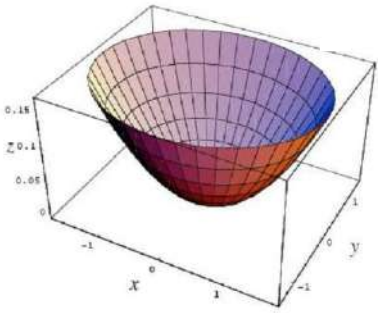
- Differentiate $E(w)$ with respect to w:
  Blackboard or Exercise (a next slide).

  Result synthesis:

- In the derivation we find that $$\frac{\partial E(\boldsymbol{w})}{\partial w_j} = -2\sum_{p=1}^{l}\left(y_p - \boldsymbol{x}_p{}^T\boldsymbol{w}\right)x_{p,j}$$

- We can get the **normal equation**
  (point with gradient of E w.r.t w =0):

$$(\boldsymbol{X}^T\boldsymbol{X})\boldsymbol{w} = \boldsymbol{X}^T\boldsymbol{y}$$



See also CM course

- If X^TX is not singular the unique solution is given by

$$\boldsymbol{w} = (\boldsymbol{X}^T\boldsymbol{X})^{-1}\boldsymbol{X}^T\boldsymbol{y} \quad = \boldsymbol{X}^+\boldsymbol{y}$$

`+` Moore-Penrose *pseudoinverse*
(also if $\boldsymbol{X}$ is not invertible)

- Else the solution are infinite (satisfying the normal equation):
  we can choose the *min norm* (w) solution

Direct approach by SVD

Dip. Informatica
University of Pisa

- The *Singular Value Decomposition (SVD)* can be used for computing the pseudoinverse of a matrix (X^+)

$$X = U\Sigma V^T \Rightarrow X^+ = V\Sigma^+ U^T$$

Σ: diagonal

Σ^+: by replacing every nonzero entry by its reciprocal

- Moreover we can apply directly SVD to compute $w = X^+ y$

obtaining the minimal norm (on w) solution of least squares problem.

- Note: THIS IS the learning alg. for the *direct* approach solution on w (or say in *closed form*)

- A practical tool. E.g. see "numerical recipes" in C and many numerical/statistical tools and scientific library (also in R, Octave, Matlab,…)
  - e.g. ARMADILLO: since 2011/12 C++ linear algebra library, NumPy etc.
- Many algorithms addressing the problems of efficiency and stability
- See also CM course (may be later) or O4DS (@DSBI)

Other approaches to LS

Dip. Informatica
University of Pisa

Many, for instances by an **iterative/gradient descent technique** we can search for:

- More efficient solutions (previous is cubic with dim of matrix X)
- Regularization (to reduce complexity of the model)
- Better approximation with noisy data by stopping searching before the minimum

- Other approaches that can be applied also to NON-LINEAR models !!!

Hence, we are going to present the second (more important) learning algorithm

Gradient descent

Dip. Informatica
University of Pisa

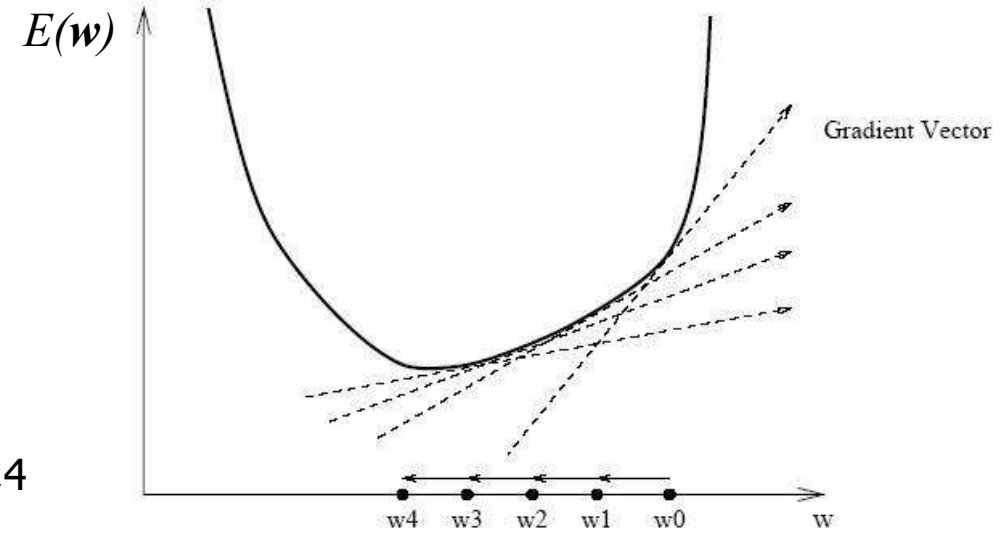
- Previous derivation suggest the line to construct an iterative algorithm based on:

$$\frac{\partial E(\boldsymbol{w})}{\partial w_j} = -2\sum_{p=1}^{l}(y_p - \boldsymbol{x}_p{}^T\mathbf{w})(\boldsymbol{x}_p)_j$$

Where $\boldsymbol{x}_p$ is p-th input pattern, y_p the output for p, $\boldsymbol{w}$ free par., l num. of examples

→ *Component j of pattern p, also* $x_{p,j}$

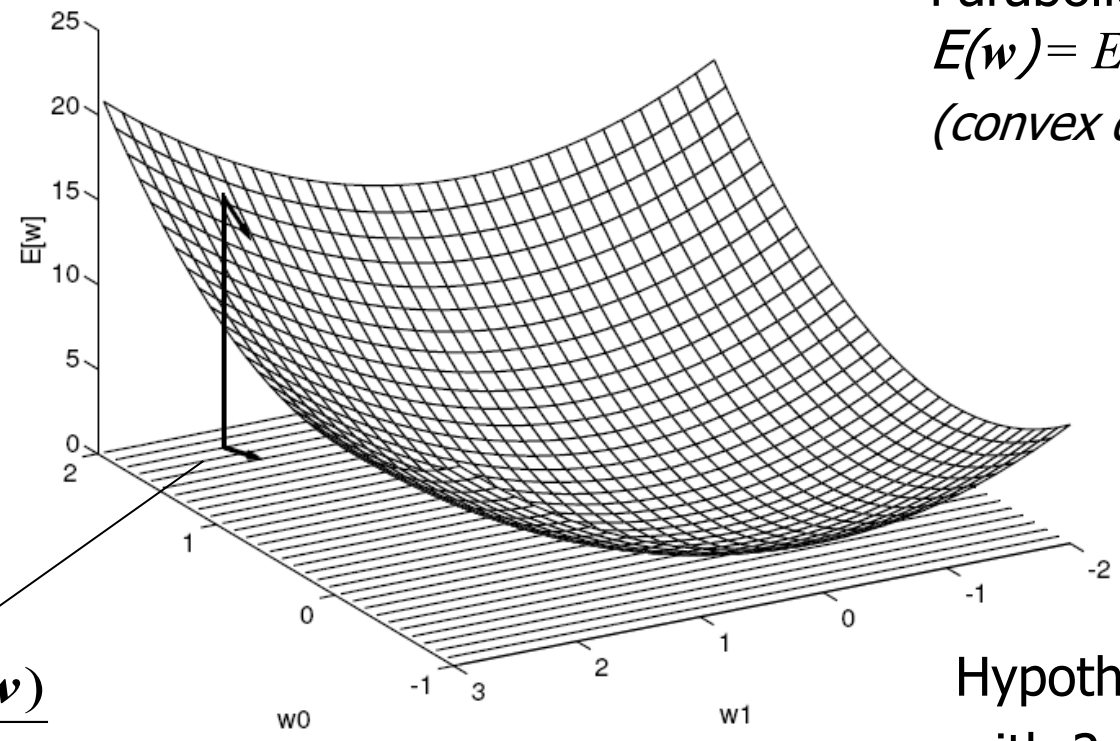
- **Gradient** = **ascent direction**: we can move toward the minimum with a gradient **descent** (changing $\boldsymbol{w}$ with $\Delta\boldsymbol{w}$= - gradient of $E(\boldsymbol{w})$)
  - *Local search*: it begins with an initial weight vector. We modify it iteratively to decrease up to minimize the error function (steepest descent).



A single w at step 0,1,2,3,4

Error surface for linear model with 2 weights (w)

Dip. Informatica
University of Pisa



Parabolic for the
$E(\boldsymbol{w}) = E([w_0, w_1]^T)$
(convex quadratic function)

$$-\frac{\partial E(\boldsymbol{w})}{\partial \boldsymbol{w}}$$

Our "compass" to find the minimum

Hypothesis space with 2 parameters

w_0, w_1

The gradient vector

Dip. Informatica
University of Pisa

$$\Delta \boldsymbol{w} = -\frac{\partial E(\boldsymbol{w})}{\partial \boldsymbol{w}} = \begin{bmatrix} -\frac{\partial E(\boldsymbol{w})}{\partial w_0} \\ -\frac{\partial E(\boldsymbol{w})}{\partial w_1} \\ -\frac{\partial E(\boldsymbol{w})}{\partial w_2} \\ -\frac{\partial E(\boldsymbol{w})}{\partial w_j} \\ \dots \\ -\frac{\partial E(\boldsymbol{w})}{\partial w_n} \end{bmatrix} = \begin{bmatrix} \Delta w_0 \\ \Delta w_1 \\ \Delta w_2 \\ \Delta w_j \\ \dots \\ \Delta w_n \end{bmatrix}$$

We can work in a multi-dim space without the need to visualize it

Using the Delta Rule

Dip. Informatica
University of Pisa

- Hence, as iterative approach we will move using a learning rule based on a «delta» (changing) of w proportional to the (opposite) of the local gradient
- The «movements» will be made iteratively according to

$$w_{new} = w + eta * \Delta w$$

(or component-wise, i.e. for each w_j)

- that is the "***learning rule***"
- and *eta* (η) is the "step size" (learning rate) parameter (ruling the speed of our gradient descending)

Gradient descent algorithm

Dip. Informatica
University of Pisa

A simple algorithm:

1) Start with weight vector $\boldsymbol{w}_{\text{initial}}$ (small), fix *eta (0<eta<1)*.

2) Compute $\Delta \boldsymbol{w} = -$"gradient of $E(\boldsymbol{w})$" $= -\frac{\partial E(\boldsymbol{w})}{\partial \boldsymbol{w}}$ (or for each w_j)

3) Compute $\boldsymbol{w}_{\text{new}} = \boldsymbol{w} + \eta * \Delta \boldsymbol{w}$ (or for each w_j)

Repeat (2) until convergence or $E(\boldsymbol{w})$ is "sufficiently small"

- $\Delta \boldsymbol{w}/l$: *least mean squares* (dividing by l, that will be the standard case)
- Batch versions ($\Delta \boldsymbol{w}$ after each "epoch" of l training patterns)
- *eta* (η): step size = *learning rate*: speed/stability trade-off: can be (gradually) decreased to zero (guarantee convergence, avoiding oscillation around the min.): many variants will be introduced later



Batch/On-line

Dip. Informatica
University of Pisa

- For **batch version** the gradient is the sum over all the l patterns:

$$\frac{\partial E(\boldsymbol{w})}{\partial w_j} = -2\sum_{p=1}^{l}(y_p - \boldsymbol{x}_p{}^T\boldsymbol{w})\, x_{p,j}$$

  - provide a more "precise" evaluation of the gradient over a set of l data

  And we upgrade the weights after this sum

- For the **on-line/stochastic version** we upgrade the weights with the error that is computed for each pattern
  - hence, the 2nd pattern output is based on weights already updated from the 1st, and so ahead
  - It makes progress with each examples it looks at: it can be the faster, but need smaller *eta:*

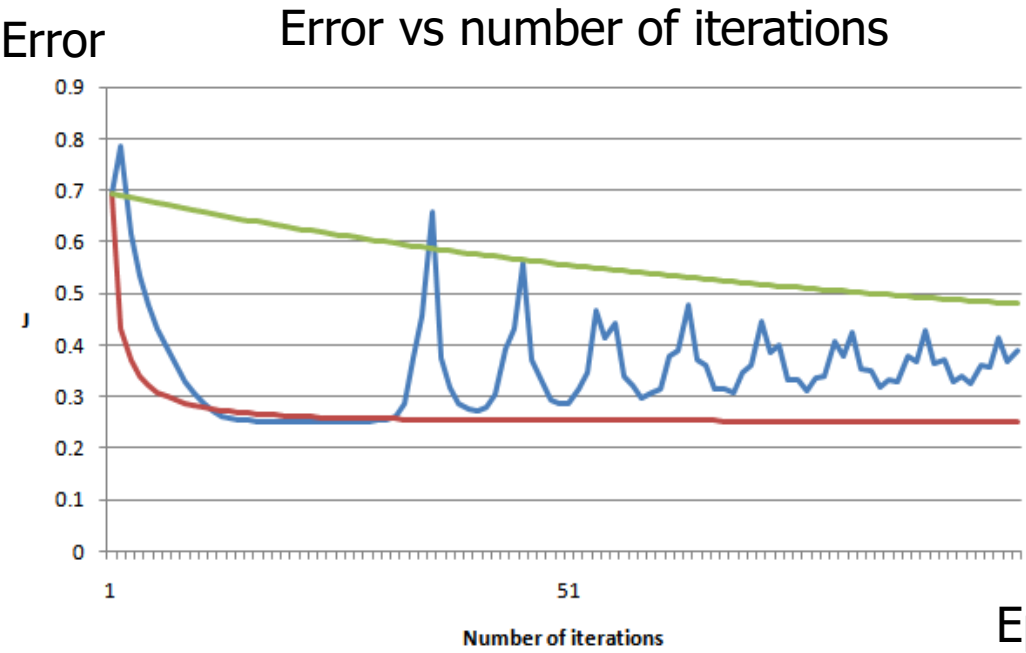
$$\frac{\partial E_p(\boldsymbol{w})}{\partial w_j} = -2(y_p - \boldsymbol{x}_p{}^T\boldsymbol{w})x_{p,j} = -\Delta_p w_j$$

- We will see intermediate cases later (as **mini-batch**)



Learning curve examples

Dip. Informatica
University of Pisa



Epochs

These are **learning curves**:

They show how the error decreases through gradient descent iterations

P.S. No relation with color in the previous slide!

Exercise: *1 is slow, 1 is unstable, 1 is good: which one?*
Which with high or low eta value?

Gradient descent as Error correction delta rule

Dip. Informatica
University of Pisa

$$\Delta w_j = 2 \sum_{p=1}^{l} \overbrace{(y_p - \boldsymbol{x}_p{}^T \boldsymbol{w})}^{\delta_p} x_{p,j}$$

$$\boldsymbol{w}_{\text{new}} = \boldsymbol{w} + eta * \Delta \boldsymbol{w}$$

Where $x_{p,j}$ is the component j of the input pattern p, y_p the output for p, $\boldsymbol{w}$ free par., l num. of examples.
The constant 2 can be omitted

- This is an *"error correction" rule* (Widrow-Hoff or **delta rule**) that change each w_j proportionally to the error (target y - output):
  - E.g. (target y – output) = err=0 → no correction
  - (input$_j$>0) if err + (output is too low), positive delta → increase w_j → increment the output → less err
  - (input$_j$>0) if err - (output is too high), negative delta → decrease w_j → reduce output → less err
  - … [*exercise*: all the cases]
- We improve by learning from previous errors

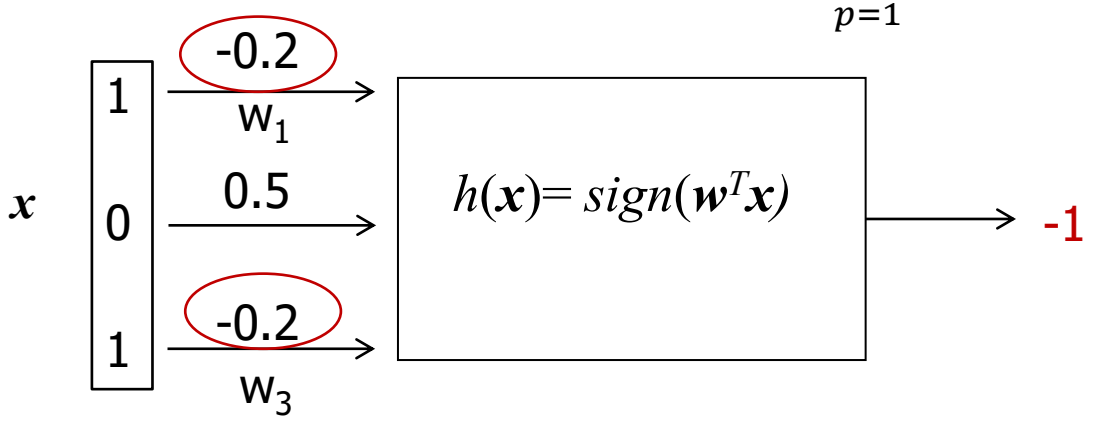
  "seeking and blundering we learn (Goethe)"

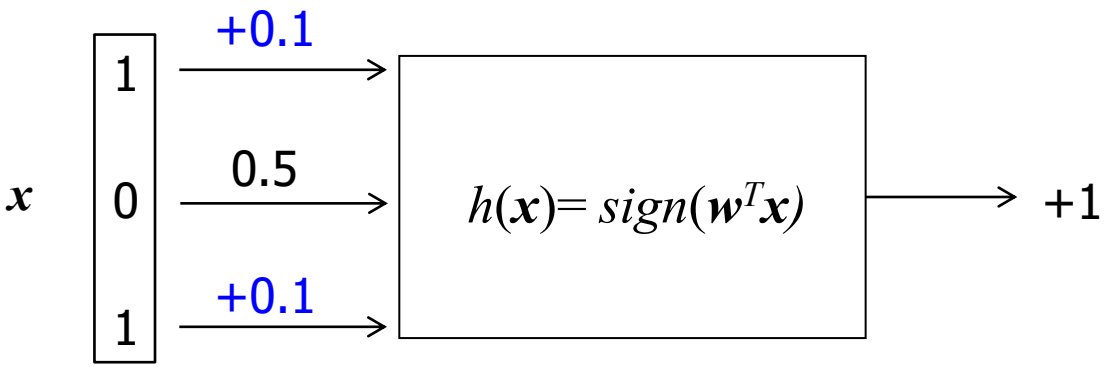


Delta-W as Error Correction Learning rule (II)

Dip. Informatica
University of Pisa

Inputs Weights $(w_0=0)$ $\Delta w_j = 2 \sum_{p=1}^{l} (y_p - \boldsymbol{x}_p{}^T \boldsymbol{w})\, x_{p,j}$





If misclassified (because target is +1) → (1-(???)) → High positive delta for w_1 and w_3 → increase them proportionally (with eta) to the *delta*, hence a positive value in this case [*error correction rule*] *!!!*

e.g. (see figure):

Now is correct !!!

Exercise: compute the values for delta (hint: see eq. above) and for also eta

Gradient descent: final discussion

Dip. Informatica
University of Pisa

Gradient descent approach it is simple and effective *local search* approach to LMS solution and

- It allows us to search through an *infinite (continuous) hypothesis* space!
- It can be easily always applied for *continues* H and *differentiable loss*
- NOT ONLY to linear models !!!! (we will see for Neural Networks and deep learning models)

- *Efficient?* Many improvements are possible, e.g. Newton & quasi-newton methods; Conjugate Gradient, …! → CM course*
  (we will mention examples later, discussing Neural Networks)

Summarizing (Classification)

Dip. Informatica
University of Pisa

- Model trained (on TR set) with LS (LMS) on $\boldsymbol{w}^T\boldsymbol{x}$
  - by the simple gradient descent algorithm used for linear regression
- Model used for classification applying a threshold function, obtaining $h(\boldsymbol{x}) = sign(\boldsymbol{w}^T\boldsymbol{x})$
- The error can be computed as *classification error* or number of misclassified patterns (not only by the Mean Square Error)

0/1 Loss

$$L(h(\mathbf{x}_p), d_p) = \begin{cases} 0 & \text{if } h(\mathbf{x}_p) = d_p \\ 1 & \text{otherwise} \end{cases}$$

$$mean_err = \frac{1}{l}\sum_{p=1}^{l} L(h(\boldsymbol{x}_p), d_p)$$

num_err

- **ACCURACY** = mean of correctly classified = $(l\text{-}num_err)/l$

Linear model (in ML): Inductive Bias (alla Mitchell)

Dip. Informatica
University of Pisa

- Language bias: the H is a set of linear functions (may be very restrictive and rigid)

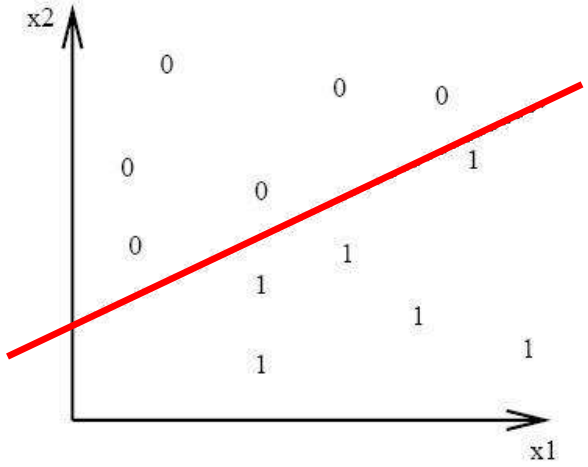
- Search bias: ordered search guided by the Least Squares minimization goal
  - For instance, we could prefer a different method to obtain a restriction on the values of parameters, achieving a different solutions with other properties (in particular to consider the generalization issue), …

It shows that even for a "simple" model there are many possibilities. We need a principled approach! (see theory of ML)…

Limitations (classification) (language bias)

Dip. Informatica
University of Pisa

- In geometry, two set of points in a two-dimensional plot are ***linearly separable*** when the two sets of points can be completely separated by a single line
- In general, two groups are *linearly separable* in *n*-dimensional space if they can be separated by an (*n* – 1)-dimensional hyperplane.



- The linear decision boundary can provide exact solutions only for linearly separable sets of points

Example: Conjunctions

Dip. Informatica
University of Pisa

- We can represent conjunctions by the linear models, e.g.:
- **Conjunctions (**see the example in the introduction lectures**):**

4 var.: $x_1 \wedge x_2 \wedge x_4 \leftrightarrow y$

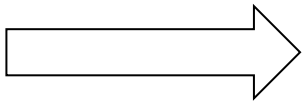
- $1\ x_1 + 1\ x_2 + 0\ x_3 + 1\ x_4 \geq 2.5$

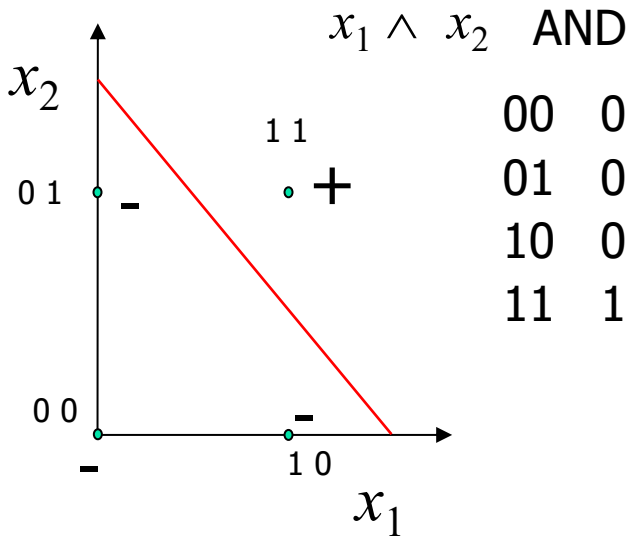
$$h(\boldsymbol{x}) = \begin{cases} 1 & \text{if } \boldsymbol{w}^T\boldsymbol{x} + w_0 \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

In the plot:

2 var.: $x_1 \wedge x_2 \leftrightarrow y$

- $1\ x_1 + 1\ x_2 \geq 1.5$





$\boldsymbol{w}$ can be learned to find this solution

Limitations: classification tasks

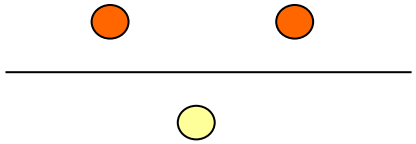


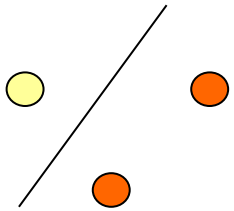
Dip. Informatica
University of Pisa

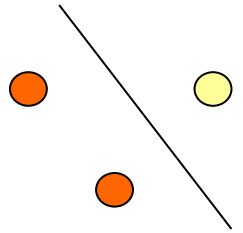
- Given **3 points**, can we always find a separation plane for every assignment of *f(**x**)*?

  No, 3 aligned points with 0 in the middle and others 1; yes if they are not aligned (existence!).

Here NOT complete:
There are 2^3 cases



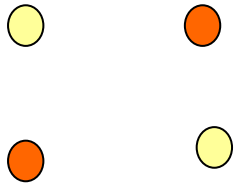




- Given **4 points**, can we always find a separation plane for every assignment of *f(**x**)* ?

  No (XOR)

  we can find a labeling such that the linear classifier fails to be perfect



XOR	
00	0
01	1
10	1
11	0

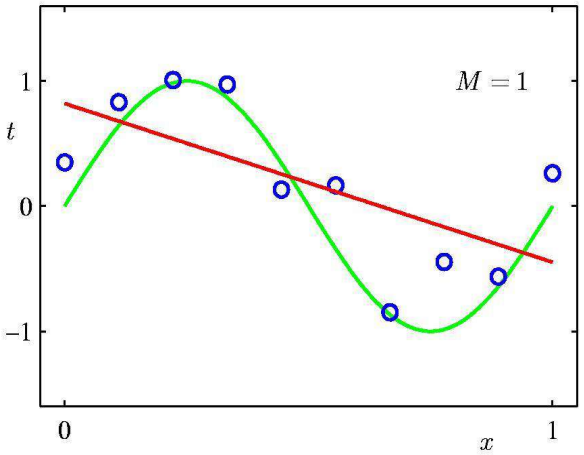
***Exercise**: try to redo it by yourself!*

How to extend the linear model

Dip. Informatica
University of Pisa

- Note that in $h_{\mathbf{w}}(x)=w_1 x+w_0$ or $h_{\boldsymbol{W}}(\boldsymbol{x}) = \boldsymbol{w}^{\mathrm{T}} \cdot \boldsymbol{x}$
- As Statistical Parametric models:

  "linear" does not refer to this (red) straight line, but rather to the way in which the regression coefficients $\boldsymbol{w}$ occur in the regression equation



- Hence, we can use also transformed inputs, such are x, x^2, x^3, x^4, ... with *non-linear* relationship inputs and output, holding the learning machinery (Least Square solution) developed so far…

$$h_{\mathbf{w}}(x) = w_0 + w_1 x + w_2 x^2 + \ldots + w_M x^M = \sum_{j=0}^{M} w_j x^j$$

polynomial regression

A generalization (LBE)

(shown for regression*)

Dip. Informatica
University of Pisa

- Basis transformation: **linear *basis expansion* (LBE):**

$$h_{\boldsymbol{w}}(\boldsymbol{x}) = \sum_{k=0}^{K} w_k \phi_k(\boldsymbol{x})$$

- Augment the input vector with additional variables which are transformations of $\boldsymbol{x}$ according to a function phi (ϕ_k: $R^n \rightarrow R$)
- E.g
  - Polynomial representation of $\boldsymbol{x}$: $\phi(\boldsymbol{x})=x_j^2$ *or* $\phi(\boldsymbol{x})= x_j x_i$, *or* ....
  - Non-linear transformation of single inputs: $\phi(\boldsymbol{x})=log(x_j)$, $\phi(\boldsymbol{x})= root(x_j)$, ...
  - Non-linear transformation of multiple input: $\phi(\boldsymbol{x})= ||\boldsymbol{x}||$
  - ***Splines*, …, ….**
- Typically: Number parameters $K > n$ (before it was n)
- The model is *linear in the parameters (also in phi, not in* $\boldsymbol{x}$*): we can use the* ***same learning alg.*** *as before!*
- *Note: it can be applied for regression (here) or classification (Exercise: HOW?)*

Basis Expansion: examples



Dip. Informatica
University of Pisa

- Basis transformation: **linear *basis expansion* :**

$$h_{\boldsymbol{w}}(\boldsymbol{x}) = \sum_{k=0}^{K} w_k \phi_k(\boldsymbol{x})$$

EXAMPLES:

- *[1-dim* $\boldsymbol{x}$*]* $\quad \phi_j(x) = x^j.$

$$h(\boldsymbol{x}) = w_0 + w_1 x + w_2 x^2 + \ldots + w_M x^M = \sum_{j=0}^{M} w_j x^j$$

1-dim polynomial regression (K=M)

Already seen in the introduction

- Or *"any other", e.g.* $\phi(\boldsymbol{x})=\phi([x_1,x_2,x_3])$

$$h(\boldsymbol{x}) = w_1 x_1 + w_2 x_2 + w_3 log(x_2) + w_4 log(x_3) + w_5 (x_2 x_3) + w_0$$

Improvements: How to control model complexity? Important !

Dip. Informatica
University of Pisa

- Many approaches… e.g. coefficient shrinkage:
- **Ridge regression**: (**Tikhonov regularization**): smoothed model → possible to add constraints to the sum of value of $|w_j|$ penalizing models with high values of $|w|$, i.e. favoring "sparse" models using less terms due to weights $w_j = 0$ (or close to 0) (it means a less complex model)

"Error" data term [(M)SE] (R_{emp} in the SLT)

Regularization/penalty term

$$Loss(\boldsymbol{w}) = \sum_{p=1}^{l} (y_p - \boldsymbol{x}_p{}^T \boldsymbol{w})^2 + \lambda ||\boldsymbol{w}||^2 \rightarrow \sum w_j{}^2$$

Lambda (λ): regularization (hyper)parameter
(a small positive value chosen by the "model selection" phase)

The sum is over the number of w

- Note that for the ***objective function*** we use here the name *Loss* (used for the model training cost function) to distinguish from the *Error E* (useful to evaluate the model error and used for the *data term* inside this Loss).
Hence, differently from previous assumptions the two terms are not more equivalent in our use (in the course).

Tikhonov Regularization: Solving it

Dip. Informatica
University of Pisa

$$Loss(\boldsymbol{w}) = \sum_{p=1}^{l} (y_p - \boldsymbol{x}_p{}^T \boldsymbol{w})^2 + \lambda ||\boldsymbol{w}||^2$$

– For the direct approach:

$$\mathbf{w} = \boxed{(\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}}\mathbf{X}^T\mathbf{y}$$

this matrix is always invertible

– For the gradient approach: we still apply –gradient of the Loss:

- As an **Exercise** compute the gradient of the two terms (error and penalty terms) of the *Loss*(**w**) w.r.t. weights w_i separately, using *eta* only for term *E*
- Formulate the new update rule :
- You will obtain:

$$\boldsymbol{w}_{\text{new}} = \boldsymbol{w} + eta * \Delta\boldsymbol{w} \; - 2\,\lambda\,\boldsymbol{w}$$

- That is a **weight decay** technique (basically add $2\lambda w$ to the gradient)
  - E.g. with 0 gradient, it decreases the value of each w with a fraction of the old w

Tikhonov regularization: trade-off

Dip. Informatica
University of Pisa

$$Loss(\boldsymbol{w}) = \sum_{p=1}^{l} (y_p - \boldsymbol{x}_p{}^T \boldsymbol{w})^2 + \lambda ||\boldsymbol{w}||^2$$

- Note the balancing (trade-off) between the two terms:
  - Small ***lambda*** (λ) value → minimizing the loss the focus is on obtaining a small error data term (first term, minimize just the training error) with a too complex model (high norm of the weights), the risk is of ***overfitting***,
  - High ***lambda*** (λ) → minimizing the loss the focus is on the second term, hence the data error (first term) could grow too much, i.e. moving to ***underfitting***
  - The trade-off is ruled by the value of ***lambda*** (λ)
  - We will see soon some examples

- The main advantage is that we have a concrete realization of the control of model complexity, easy to be implemented and of general applicability.

Tikhonov and SLT

Dip. Informatica
University of Pisa

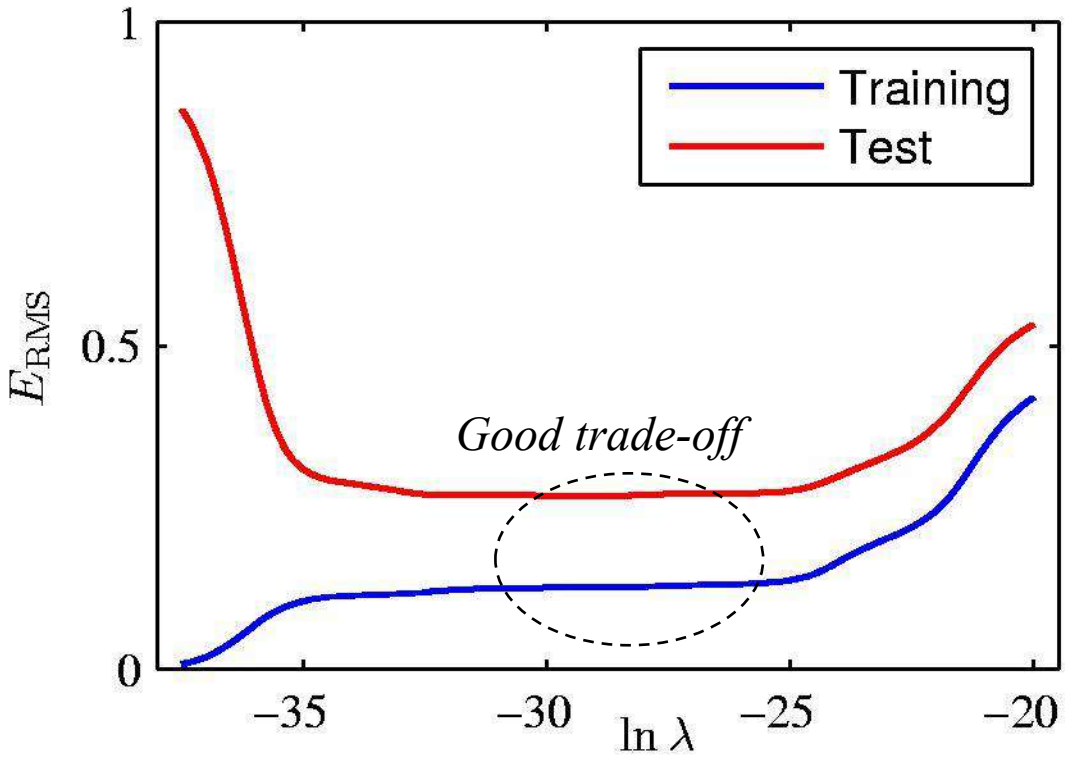
- The penalty term penalizes high value of the weights and tends to drive all the weights to smaller values
  - E.g. Some weights values can go even to zero
- It implements a control of the model complexity
- This leads to a model with less (or proper) *VC-Dim,* with a trade-off obtained through **just a (1) parameter** that you can control: the λ

- **Exercise**: connect Tikhonov regularization with the slide on SLT (see the introduction) to see
  - why this can help to have a better bound on *R*, and
  - how lambda values can rule the underfitting/overfitting cases

- **Exercise:** derive the new learning rule with weight decay using the Tikhonov loss: computing again the partial derivatives of *Loss* with respect to w, separating data term (• eta) and penalty term (• λ)

Regularization: E_{RMS} vs $\ln \lambda$

Dip. Informatica
University of Pisa



Low lambda → overfitting

High lambda → underfitting

Polynomial Coefficients

Dip. Informatica
University of Pisa

0 lambda

Very high lambda

	$\ln \lambda = -\infty$	$\ln \lambda = -18$	$\ln \lambda = 0$
$w_0^\star$	0.35	0.35	0.13
$w_1^\star$	232.37	4.74	-0.05
$w_2^\star$	-5321.83	-0.77	-0.06
$w_3^\star$	48568.31	-31.97	-0.05
$w_4^\star$	-231639.30	-3.89	-0.03
$w_5^\star$	640042.26	55.28	-0.02
$w_6^\star$	-1061800.52	41.32	-0.01
$w_7^\star$	1042400.18	-45.95	-0.00
$w_8^\star$	-557682.99	-91.53	0.00
$w_9^\star$	125201.43	72.68	0.01

Lessons Learned

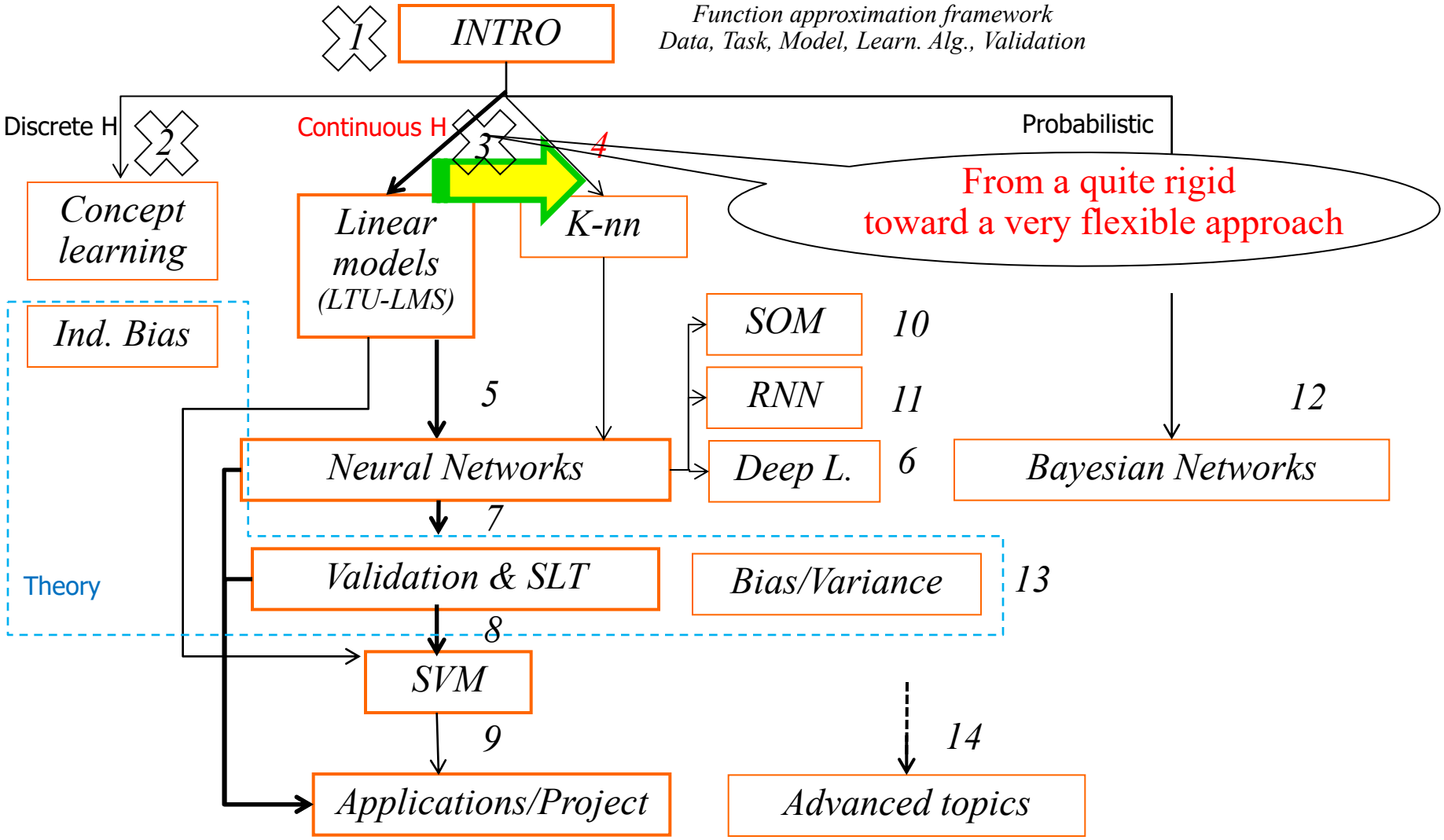
Dip. Informatica
University of Pisa

- We concretely showed that is possible to *learn* by tuning free parameters ***w***, searching over a continuous space guided by a loss f.
  - How to state/formulate a regression/classification learning problem for a linear model by LMS
  - How to derive the learning algorithm (basic univariate, direct and **gradient** based forms)
  - Two learning algorithms
  - How the model works after training
  - Linear models have limitations (strong language bias)
- How to extend to non-linear modeling (by **LBE**)
- How to implement a **regularization approach** changing the loss f.
- The role of regularization for the **control of the model complexity**

Also useful for your self assessment (try to check if you can answer)

ML Course structure
Where we go

Dip. Informatica
University of Pisa



Learning… timing

Dip. Informatica
University of Pisa

- LEARNING ALG.

- Timing.
  - **Eager:** Analyze the training data and construct an explicit hypothesis.
  - **Lazy**: Store the training data and wait until a test data point is presented, then construct an ad hoc hypothesis to classify that one data point.

1-Nearest Neighbor (1-nn)

Dip. Informatica
University of Pisa

The algorithm:

- Simply store the training data $<\boldsymbol{x}_p, y_p>\ p=1\ldots l$
- Given an input $\boldsymbol{x}$ (with dimension n)
- Find the nearest training example $\boldsymbol{x}_i$
  - Find i s.t. we have min $d(\boldsymbol{x},\boldsymbol{x}_i)$ → $i(\boldsymbol{x}) = \arg\min_p d(\boldsymbol{x}, \boldsymbol{x}_p)$
  - E.g. Euclidian distance :

$$d(\boldsymbol{x}, \boldsymbol{x}_p) = \sqrt{\sum_{t=1}^{n} (x_t - x_{p,t})^2} = ||\boldsymbol{x} - \boldsymbol{x}_p||$$

$x_{p,t}$: Pattern $\boldsymbol{x}_p$, component t

- Then output y_i

1-nn

Dip. Informatica
University of Pisa

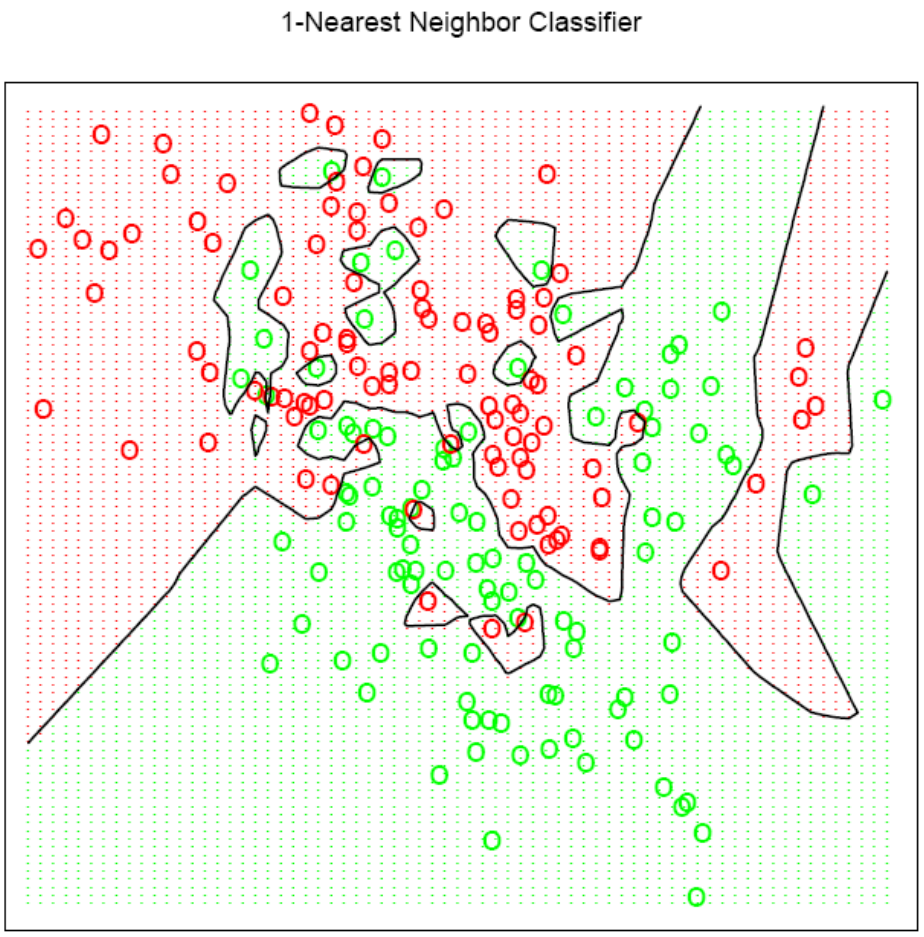


Figure 2.3:
The same classification example in two dimensions as in Figure 2.1.
GREEN = 0, RED = 1

- Very flexible !
- No misclassifications in TR data: 0 training error: what for test data ?
- Decision boundaries is *not* linear: it is quite *irregular*
- May be unnecessary noisy (e.g. for scenario 1)

K-Nearest Neighbors

Dip. Informatica
University of Pisa

- A natural way to classify a new point is to have a look at its neighbors,
- and take an average:

$$avg_k(\boldsymbol{x}) = 1/k \sum_{\boldsymbol{x}_i \in N_k(\boldsymbol{x})} y_i$$

- where $N_k(\boldsymbol{x})$ is a neighborhood of $\boldsymbol{x}$ that contains exactly k neighbors (closest patterns according to $distance\ d$):
  - k-nearest neighborhood: **K-nn**.

- If there is a clear dominance of one of the classes in the neighborhood of an observation ***x***, then it is likely that the observation itself would belong to that class, too. Thus the classification rule is the **majority voting** among the members of $N_k(\boldsymbol{x})$. As before,

$$h(\boldsymbol{x}) = \begin{cases} 1 & _if \quad avg_k(\boldsymbol{x}) > 0.5 \\ 0 & _____otherwise \end{cases} \qquad for\ targets\ y\ \ in\ \{0,1\}$$

- For regression task: use directly the avg: mean over K-nn

15-nn

Dip. Informatica
University of Pisa

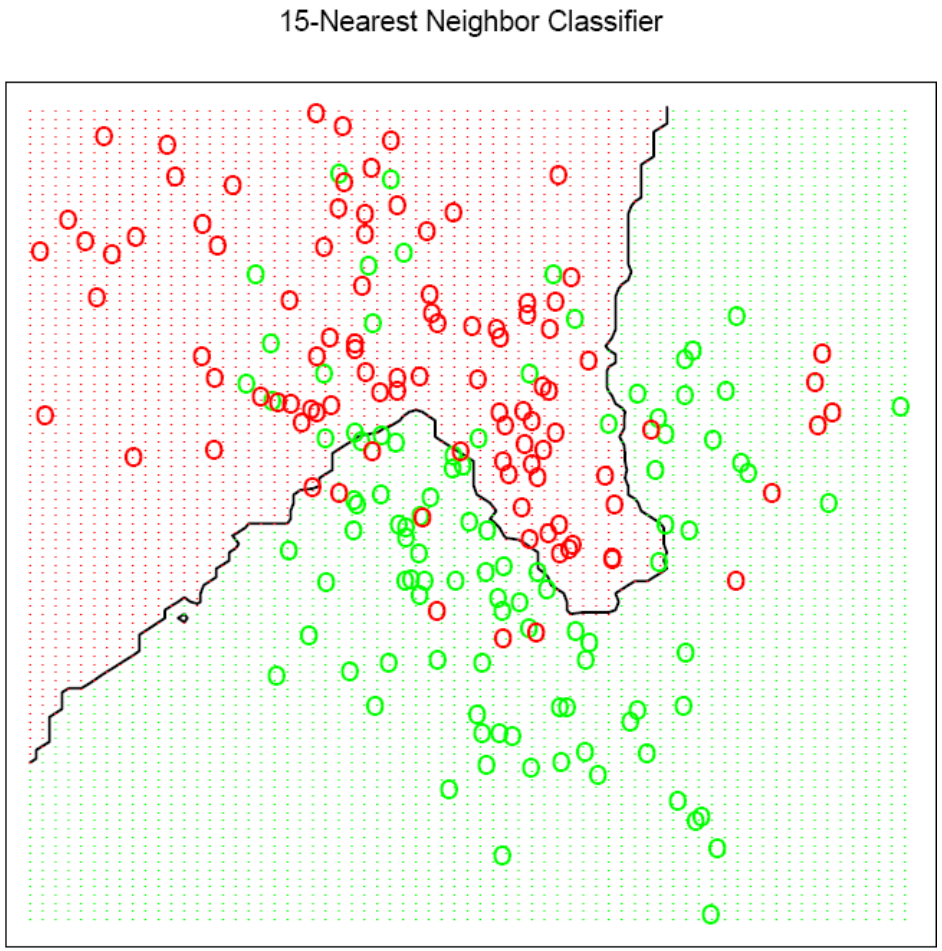


Figure 2.2:
The same classification example in two dimensions as in Figure 2.1.
GREEN = 0, RED = 1
The predicted class is hence chosen by majority vote amongst the 15*-nearest neighbors.*

- Still very flexible !
- Some misclassifications in TR data
- Decision boundaries is *not* linear: it is still quite, although less, *irregular*
- Decision boundary adapts to the local densities of the classes

K-nn for multi-class

Dip. Informatica
University of Pisa

- Return the class most common amongst its *k* nearest neighbors

$$h(\boldsymbol{x}) = \arg\max_{v} \sum_{\boldsymbol{x}_i \in N_k(\boldsymbol{x})} \boldsymbol{1}_{v,y_i}$$

$$\boldsymbol{1}_{v,y_i} = \begin{cases} 1 & if \quad v = y_i \\ 0 & otherwise \end{cases}$$

We count the classes in the neighbor (by the $\boldsymbol{1}_{v,y}$ for each v) taking the most frequent class ($\arg\max$)

K-nn variants: Weighted distance



Dip. Informatica
University of Pisa

- It can be useful to weight the contributions of the neighbors, so that the nearer neighbors contribute more to the average than the more distant ones.

$$h(\boldsymbol{x}) = \arg\max_{v} \sum_{\boldsymbol{x}_i \in N_k(\boldsymbol{x})} \boldsymbol{1}_{v,y_i} \bullet \frac{1}{d(\boldsymbol{x},\boldsymbol{x}_i)^2}$$

If $d = 0$ for a i , return y_i

$$\boldsymbol{1}_{v,y_i} = \begin{cases} 1 & if \quad v = y_i \\ 0 & otherwise \end{cases}$$

K-nn: An extreme

Dip. Informatica
University of Pisa

- Not a global hypothesis for all the instances → no model to be fit
  - We need to memorize the input examples
- Local estimations (by locally constant functions) vs global linear approximation/estimation of the target function (over the instance space)
- Lazy, memory based, instance-based, **distance-based methods**

Bayes error rate

Dip. Informatica
University of Pisa

Bayes optimal solution (called Bayes classifier):

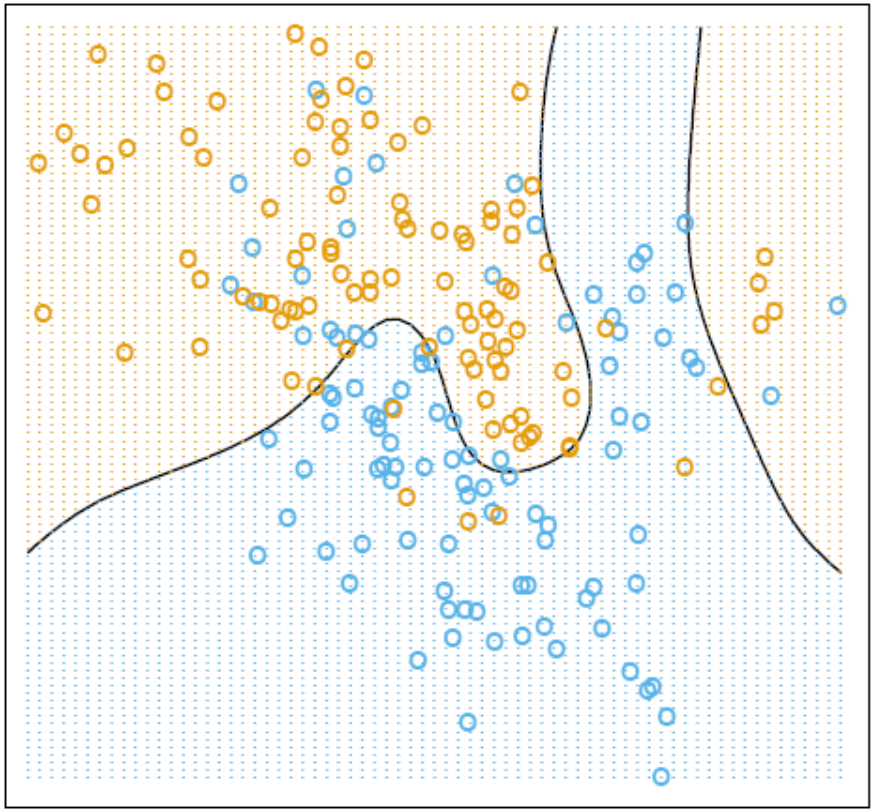
- If we know the density $P(\boldsymbol{x},y)$ we classify to the most probable class, using the conditional (discrete) distribution as:
  - Output the class v s.t. is max $P(v|\boldsymbol{x})$ (v *in* $\{C_1,C_2,C_3 \ldots C_K\}$)

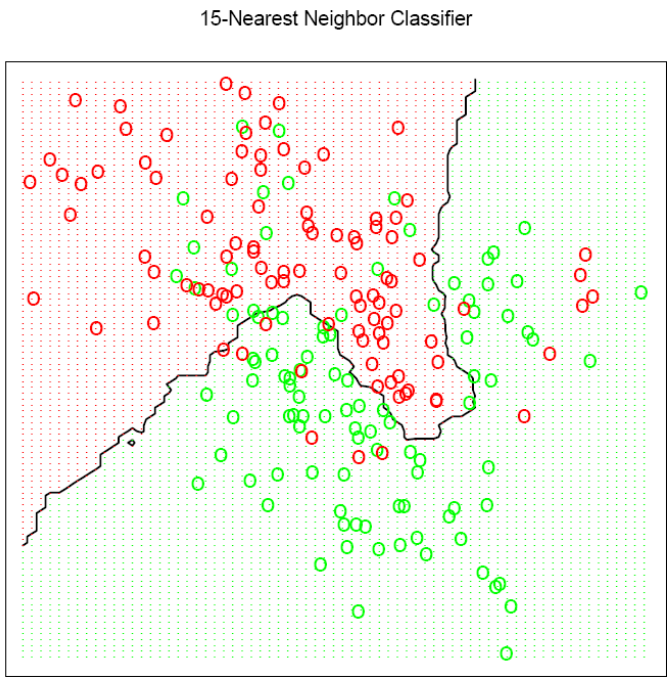
- The error rate of the (optimal) Bayes classifier is called the Bayes rate.
- I.e. the minimum achievable error rate given the distribution of the data (assuming that the generating density is known !!!)

- Note on K-nn: we see that the k-nn classifier directly *approximates this solution* (a majority vote in a nearest neighborhood amounts to exactly this), except that
  - conditional probability at a point is relaxed to conditional probability within a neighborhood of a point (local approximation), and
  - probabilities are estimated by training-sample proportions.

Bayes Optimal Classifier: results

Dip. Informatica
University of Pisa





- The optimal Bayes decision boundary for the simulation example.
- Since the generating density is known for each class, this boundary can be calculated exactly
- 15-nn was *close to this* (indeed it shown a min. test err in the plot a couple of slides ago)

Inductive Bias of k-nn

Dip. Informatica
University of Pisa

1. The assumed **distance** tells us which are the most similar examples
   - The classification is assumed similar to the classification of the neighbors according to the assumed metric
     - E.g. we can use other metric than the Euclidian
     - Symbolic data require "ad-hoc" metrics (e.g. Hamming distance between two strings of equal length is the number of positions for which the corresponding symbols are different).
2. **Locality**: local approximation (local smoothness prior): see before, and later in the course (vs Deep Learning bias)

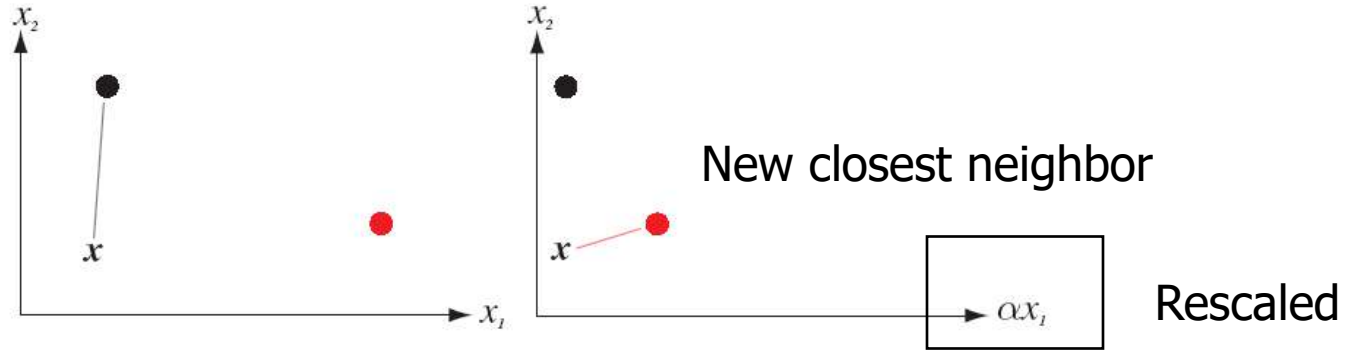
- Next slides:
  - criticism and limitations

1. Scale changes and 2. other metrics

Dip. Informatica
University of Pisa

- **Scale changes**: Domain knowledge dependent choice
- **If** variables should contribute equally:
  - Pay attention to disparity in the ranges of each variables
  - Rescale data to equalize inputs ranges (→ change the metric)!
    E.g. mean zero and variance 1 normalization

- Variable scaling can have a high impact (i.e. k-nn is fragile even with respect to basic preprocessing)



(some) Limits of K-nn: computational cost

Dip. Informatica
University of Pisa

- Note that K-nn makes the local approximation to the target function for each new example to be predicted:

➢ The computational cost is deferred to the **prediction phase**!

Moreover: high retrieval cost:

- Computationally intensive (in time) for each new input: computing the distances from the test sample *to all* stored vectors
  - The time is proportional to the number of stored patterns
  - "ad-hoc" proximity search algorithms to optimize
  - E.g. by indexing the patterns or other approximate nearest neighbor search techniques
- Cost in space (all the training data)

(some) Limits of K-nn: curse of dim.

Dip. Informatica
University of Pisa

K-nn provides a good approximation if we can find a significant set of data close to any ***x***, with dense sampling

When can it fail?

- When we have a lot of input variables (high n, *i.e. high dim.*), K-nn methods often fails because of the ***curse of dimensionality***:

Many manifestations of this problem: we will examine a few

1. *It is hard to find nearby ("similar") points in high dimensions!*
2. *Low sampling density for high-dim data*
3. *Irrelevant features issue*

These are important in terms of ML,
as they inherently affect the ***generalization capability*** *!!!*

(some) Limits of K-nn: curse of dim.

Dip. Informatica
University of Pisa

1. It is hard to find nearby points in high dimensions!

- K-nearest neighbors can fail in high dimensions, because it becomes difficult to gather K observations "close" (in terms of of variables values, i.e. "similar") to a query point x_q:
  - near neighborhoods tend to be **spatially large**, and estimates are **not longer local**
  - reducing the spatial size of the neighborhood means reducing K, and the variance of the estimate increases (→ overfitting).

- Why? See the next slide…

(some) Limits of K-nn: curse of dim.

Dip. Informatica
University of Pisa

2. Low sampling density for high-dim data

- Sampling density (*l/volume)* is proportional to $l^{1/n}$ (*l* data, *n*=data dim). Hence, for example:
  - if 100 points are sufficient to estimate a function in IR^1 (1-dim input),
  - 100^{10} (=100.000.000.000.000.000.000) are needed to achieve similar accuracy in IR^{10} (10-dim input)

(some) Limits of K-nn: curse of dim.

Dip. Informatica
University of Pisa

3. Irrelevant features: The Curse of Noisy

if the target depends on only few of many features in $\boldsymbol{x}$ (e.g. 2 out 20), we could retrieve a "similar pattern" with the similarity dominated by the large number of irrelevant features

This issue grows with the dimensionality

An improvement

Dip. Informatica
University of Pisa

Irrelevant features:

- We may weight features according to their relevance
  - Stretching the axes along some dimension
  - Weights can be searched by a (expensive) model selection approach or other approaches …

- Feature selection approaches: it eliminates some variables → reduce input dimension

K-nn in the course: some general lessons

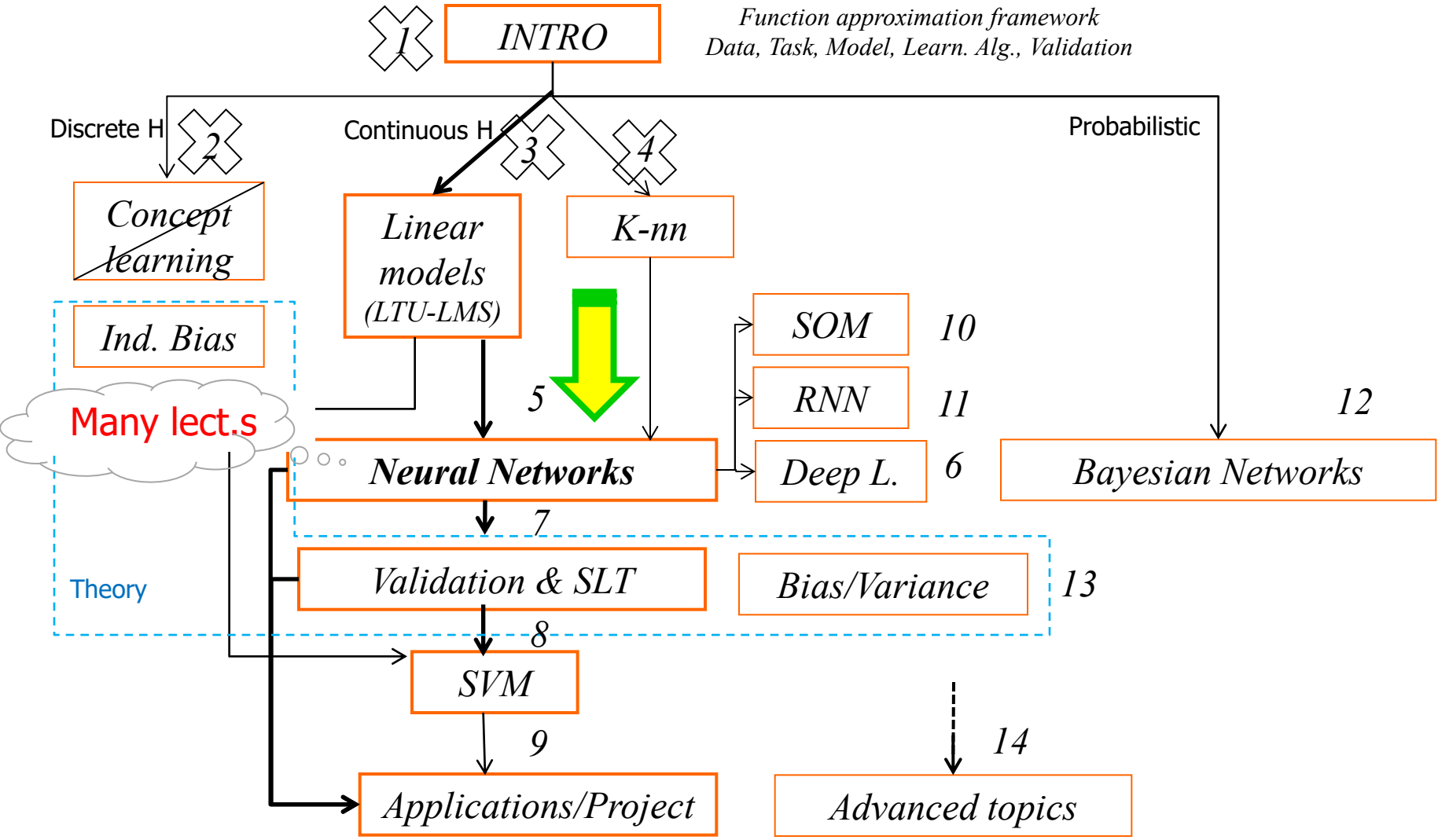
Dip. Informatica
University of Pisa

- Too low variance is poor (rigid linear models), too high variance is dangerous (K-nn)

- Other concepts presented todays that are interesting in general:
  - **Smoothing** techniques (in K-nn by increasing K)
  - **Curse of dimensionality** (the volume of the problem space increases so fast that the available data become sparse)
  - Again an **instance of the Statistical Learning Theory** bound plot: see the misclassification plot moving K
  - **Inductive Bias** for K-nn (metric based issue, which is underestimated in many books)

ML Course structure
Where we are

Dip. Informatica
University of Pisa



A first look to NN (as ML tool)

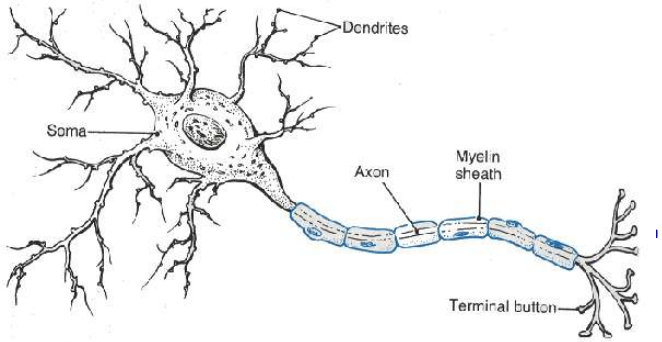
Dip. Informatica
University of Pisa

A historical learning machine,
while still a powerful approach to ML

- NN can learn from examples
- NN are universal approximators (Theorem of Cybenko):
  flexible approaches for arbitrary functions (including non–linear) !
- NN can deal with noise and incomplete data
  - Performance degrades gracefully under adverse operating conditions
- NN can handle continuous real and discrete data
  for both regression and classification tasks
- Successful model in ML due to the flexibility in applications

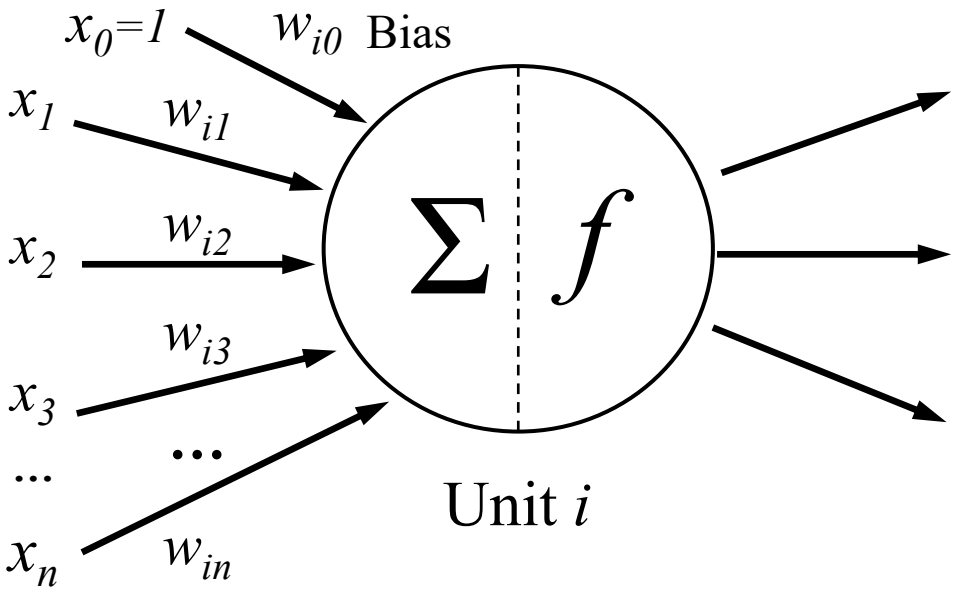
- NN encompasses a wide set of models: it is a paradigm !
  (in the class of subsymbolic approaches)

Artificial Neuron: processing unit



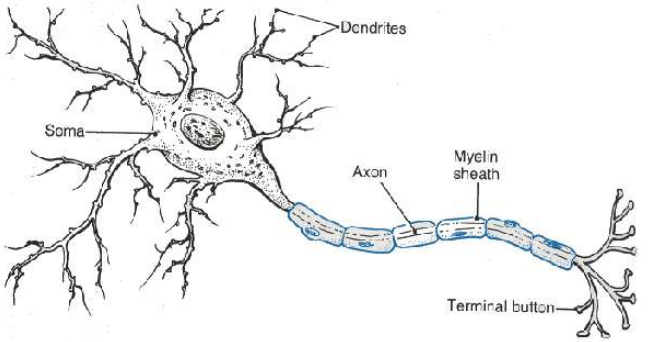
- Node, neuron or **unit**
- Input: from extern source or other units (R)
- Input Connections: **weights** w: free parameters: these can be modified by learning (synaptic strength)

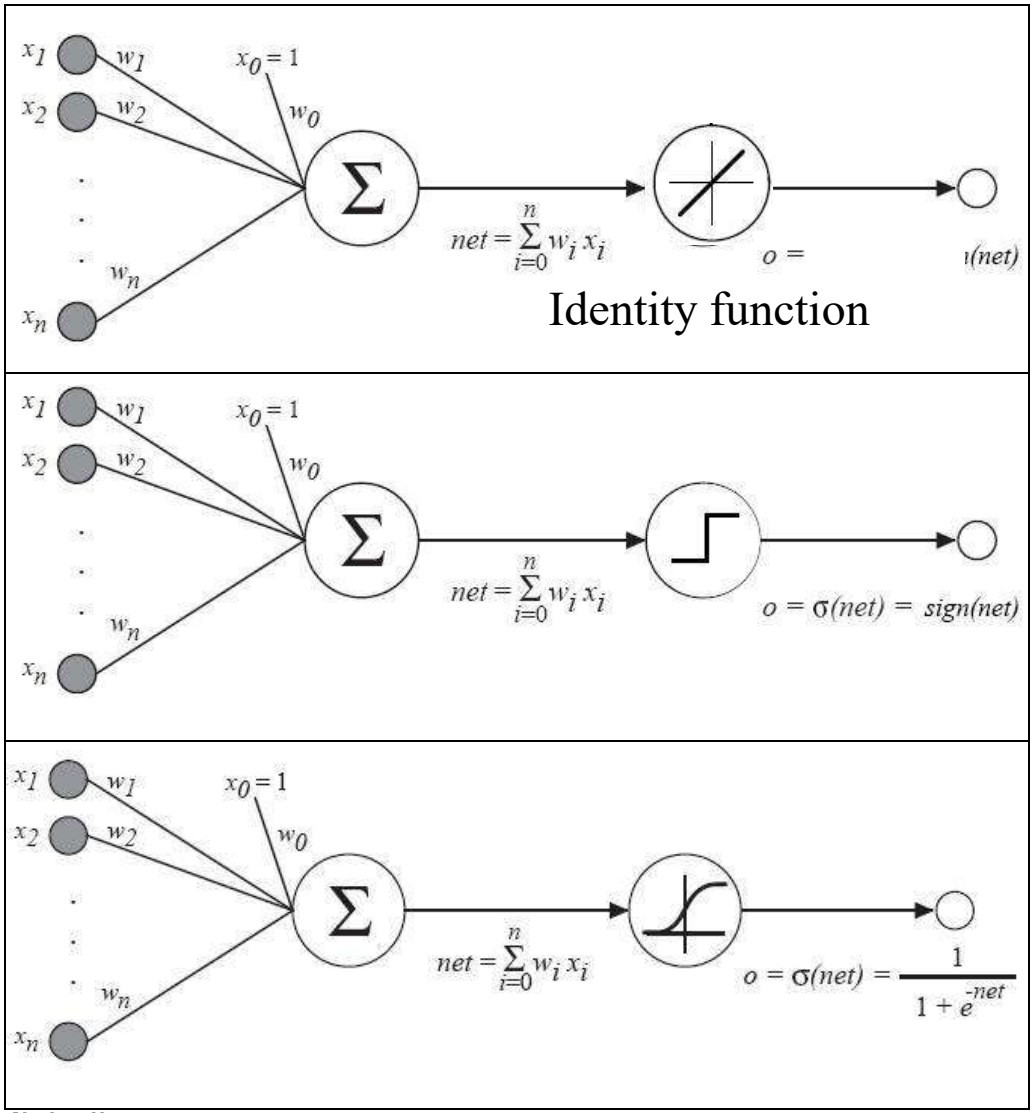
$$\begin{cases} net_i(\boldsymbol{x}) = \sum_j w_{ij} x_j \\ o_i(\boldsymbol{x}) = f\ (net_i(\boldsymbol{x})) \end{cases}$$



- The weighted sum net_i is called the **net input** to unit i
- Note that w_{ij} refers to the weight of the unit i, i.e.from unit/input j to unit i (not the other way around, e.g. see the next slide).
- The function f is the unit's **activation function (e.g linear, LTU, ...)**.

Neuron: Three activation functions





$$h(\boldsymbol{x}) = \sum_i w_i x_i$$

Linear activation function

Perceptron (see LTU)
Threshold activation function

Logistic function : sigmoid
(see later)

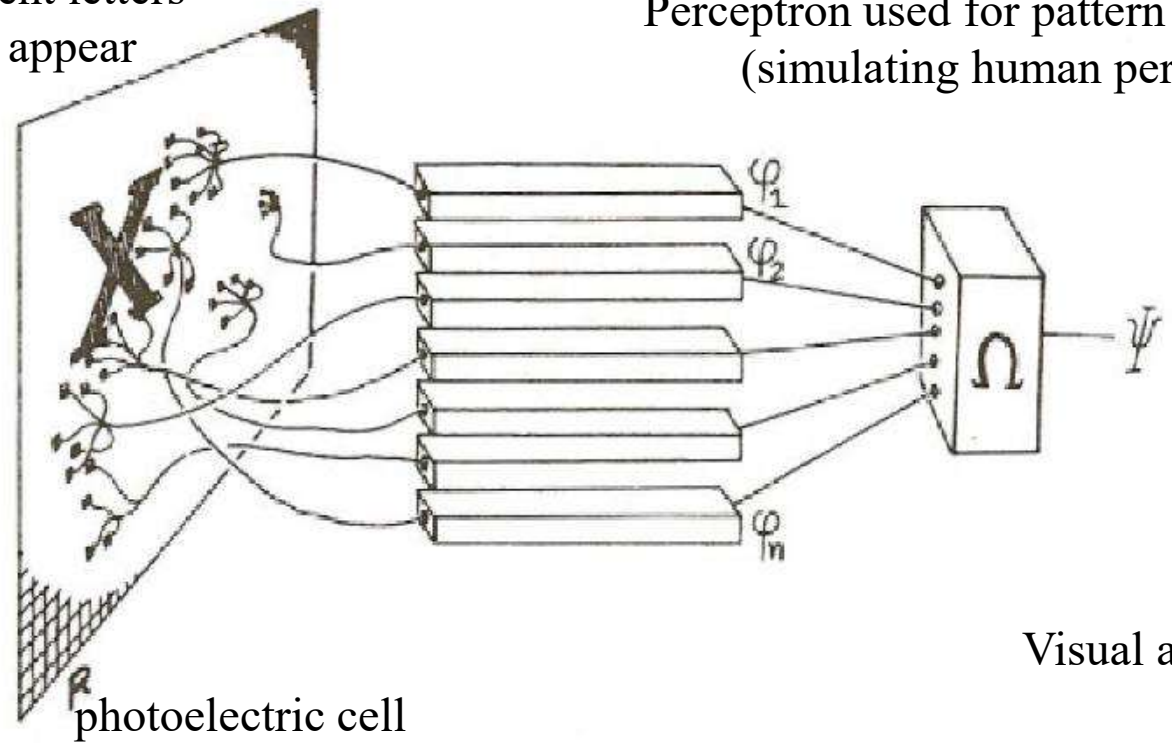
Perceptron

Dip. Informatica
University of Pisa

- Frank Rosenblatt (1957-1958,1960, ...)

Different letters can appear

Perceptron used for pattern classification (simulating human perception)



© 1969 Mit Press

Models with Perceptron

Dip. Informatica
University of Pisa

- Perceptrons can be composed and connected to build a networks:
  *from LTU to NN !!! (**MLP** NN = Multi Layer Perceptron)*
  - Paradigmatic for the transition from linear to non-linear models towards the flexible models at the state-of-the-art in ML
- *First, by an historical model…McCulloch&Pitts*
- Note: actually NN are realized by software (*simulators*) or hardware on chips

McCulloch & Pitts Networks

Dip. Informatica
University of Pisa

- ``A logical calculus of the ideas immanent in nervous activity" 1943
  - What does a given net compute?
  - Can a given net compute a given logical sentence?
- Neurons are in two possible states: firing (1) and not firing (0)
- All synapses (connections) are equivalent and characterized by a real number (their strength w), which is positive for excitatory connections and negative for inhibitory connections;
- a neuron i becomes active when the sum of those connections w_{ij} coming from neurons j connected to it which are active, plus a bias, is larger than zero.
- Binary inputs
- Again binary output (aka binary classification task)

- "Basically" the LTU/Perceptron

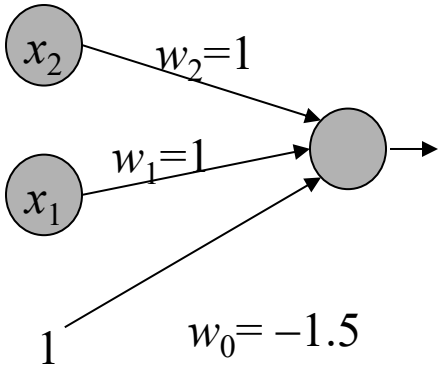
AND, OR Boolean functions

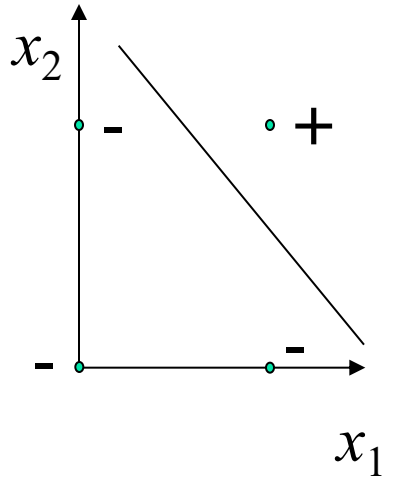
Dip. Informatica
University of Pisa

Assume $net = \mathbf{w}^T\mathbf{x} = \sum_{i=0}^{n} w_i x_i$ $\quad \text{sign}(net) = \begin{cases} 1 \text{ if } net > 0 \\ 0 \text{ (or } -1\text{) otherwise} \end{cases}$

AND

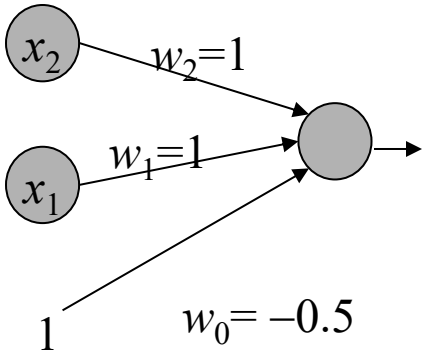
x_1	x_2	$f(x)$
0	0	0
0	1	0
1	0	0
1	1	1

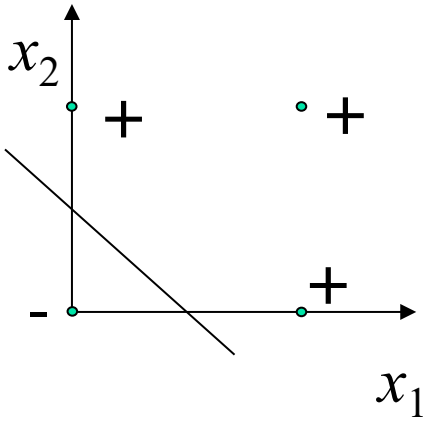




OR

x_1	x_2	$f(x)$
0	0	0
0	1	1
1	0	1
1	1	1



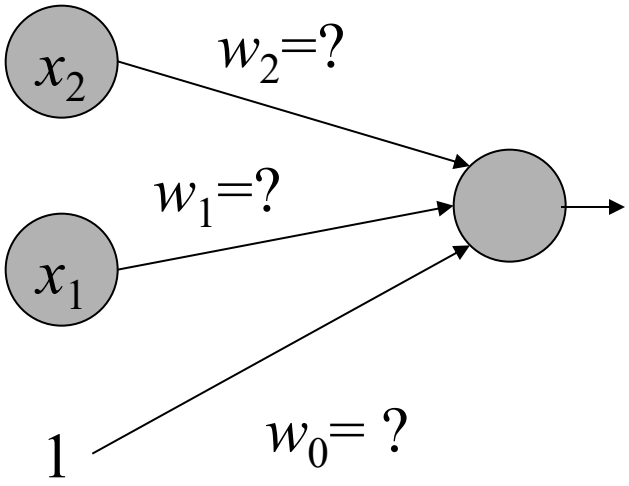


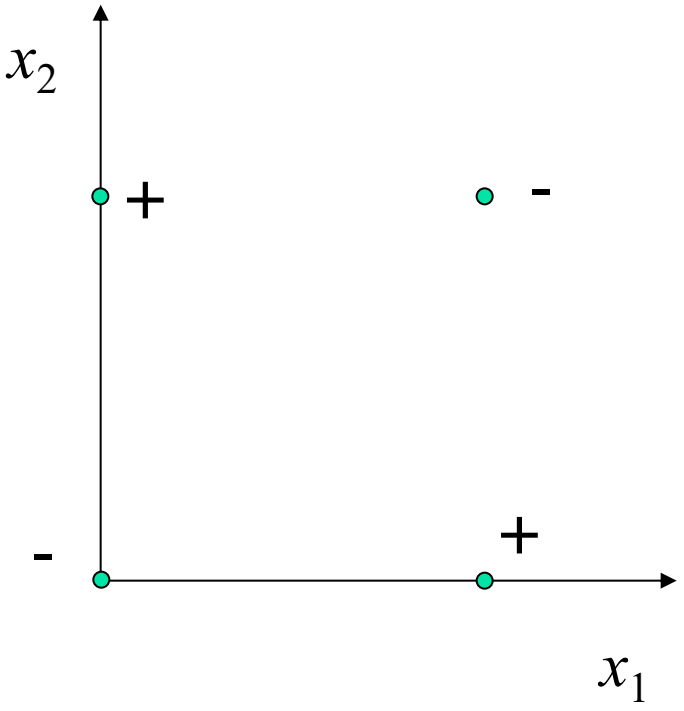
Exercise: find a network for the NOT

Exclusive OR (XOR)

Dip. Informatica
University of Pisa

x_1	x_2	$f(x)$
0	0	0
0	1	1
1	0	1
1	1	0





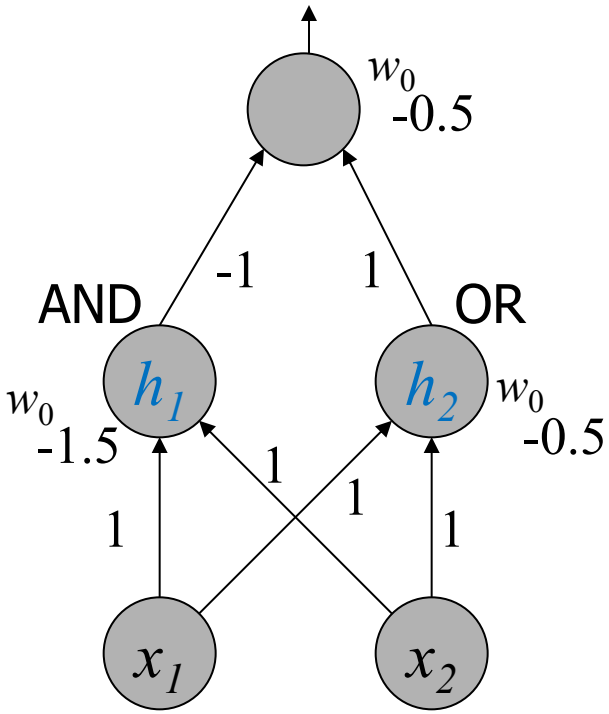
No linear separation surface exists!!
Minsky and Papert (1969)

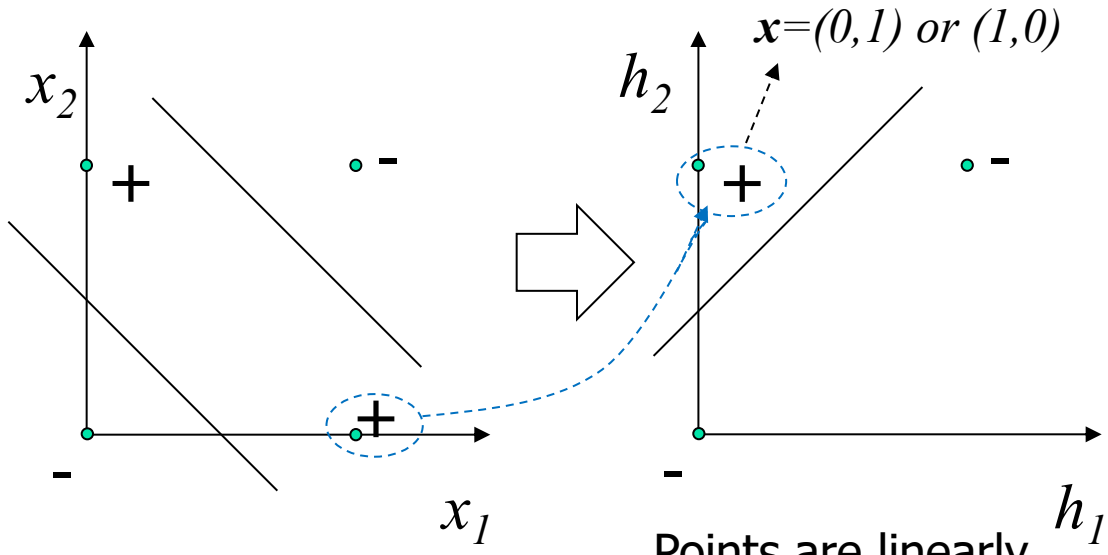
XOR by a Two Layers Network

Dip. Informatica
University of Pisa

$$x_1 \oplus x_2 = x_1 \cdot \overline{x_2} + \overline{x_1} \cdot x_2 \qquad \text{let :} \quad h_1 = x_1 \cdot x_2 \quad (\text{dot} = \textbf{and}) \qquad h_2 = x_1 + x_2 \quad (+ = \textbf{or})$$

$$\text{then we have :} \; x_1 \oplus x_2 = \overline{h_1} \cdot h_2 \qquad \text{(next slide)}$$





E.g. (1,0) becames h_1=0 h_2=1

Points are linearly separable in the *new space!*

Exercise: separate by other planes→ find different networks solving XOR

Detalis to solve the XOR (proof)



Dip. Informatica
University of Pisa

- DE MORGAN rule : ***not***$(a$ ***and*** $b) =$ ***not*** (a) ***or not***(b)

- Hence, if $h_1 = (a$ ***and*** $b)$ → ***not***$(h_1)=($***not***(a) ***or not***$(b))$
- And if $h_2 = (a$ ***or*** $b)$ →

not(h_1) ***and*** $h_2 =$ ***(not***(a) ***or not***$(b))$ ***and*** $(a$ ***or*** $b)=$
$=$ ~~***(not***(a) ***and***~~ $a)$ ***or*** ***(not***(a) ***and*** $b)$ ***or*** $(a$ ***and not***$(b))$ ***or***
~~$(b$ ***and not***$(b))$~~ $=$
$=$***(not***(a) ***and*** $b)$ ***or*** $(a$ ***and not***$(b)) =$ *(by* ***xor*** *def.)*
$=$ a ***xor*** b *(C.V.D.)*

Exercise: redo by yourself with x, +, etc

Hidden layer & Representation

Dip. Informatica
University of Pisa

- Useful concept of internal **(re)representation** of input variables via internal (hidden) units
- Developing *high level (hidden) features* is a key factor in NN
- The representation in the hidden layer makes easier the task to the output layer (last unit).
  - E.g. See the figure on the right: point *(1,0)* (in the sud-est corner) becames $h_1=0\ h_2=1$ (north-west corner) → now it is a linear separable problem

A look ahead:

- Such ***composition*** of sub-/intermediate operations to perform a complex task can be extended through *many layers* of abstraction
- In NN such internal representation/intermediate features can be ***learned***
- Learning internal distributed representation → *representation learning* and *deep learning* concepts (link to next parts of the ML course)

McCulloch&Pitts Summary

Dip. Informatica
University of Pisa

- Properties of Networks of Perceptrons:

- Perceptrons can represent AND, OR, NOT (NAND, NOR)
- → every Boolean function can be represented by some network of interconnected perceptrons.

- Only **two** levels deep is sufficient (or more can be more efficient*).
- … but **single** layer cannot model all the possible functions (Minsky&Papert 1969): limitations of single perceptrons due to the linear separable problems!

- Note: **no** provided (up to now) learning algorithm (even for one unit)!!!!
- Let us start (again) with a single unit model

Learning for one unit model - Overview

Dip. Informatica
University of Pisa

Two kinds of method for learning (also historical view):

1. **Adaline** = Adaptive Linear Neuron (Widrow, Hoff): linear unit during training: LMS direct solution and gradient descent solution
   - Regression tasks: See the LMS algorithm
   - For classification: See the LTU and LMS algorithm of a previous lectures (on linear model)
   - An approach that we will generalize to MLP

2. **Perceptron** (Rosenblatt): non-linear unit during training: with hard limiter or Threshold activation function
   - Only classification: Capabilities studies, convergence theorem
   - Next slides

Introduction to 2.: the Perceptron Learning Alg.

Dip. Informatica
University of Pisa

- Rosenblatt's Perceptron Learning Algorithm (1958-1962)

- Minimize the number of misclassified patterns:
  - find $\boldsymbol{w}$ s.t. $sign(\boldsymbol{w}^T\boldsymbol{x}) = d$

  - On-line algorithm: a step can be made for each input pattern

  - Note that to build linear classifier we will have from now **3 learning alg:**
    1. Adaline LMS direct solution
    2. Adaline LMS gradient based alg.
    3. Perceptron learning algorithm (now)
    - And others will come (later in the course, e.g. SVM)



The "Perceptron Learning Algorithm"

Dip. Informatica
University of Pisa

1. Initialize the weights (either to zero or to a small random value)
2. pick a learning rate η (this is a number between 0 and 1)
3. until stopping condition is satisfied (e.g. weights don't change):

For each training pattern $(\boldsymbol{x}, d)$ $(d = +1 \text{ or } -1)$ [let *out* be the output]:

- Compute output activation $out = sign(\boldsymbol{w}^T \boldsymbol{x})$ $[+1,-1]$

- If $out = d$, don't change weights I.e. "minimize (only) misclassifications"
- If $out \neq d$, update the weights:
  - $\boldsymbol{w}_{new} = \boldsymbol{w} + \eta\ d\ \boldsymbol{x}$

add $+\ \eta\, \boldsymbol{x}$ if $\boldsymbol{w}^T\boldsymbol{x} \leq 0$ and $d=+1$
$-\ \eta\, \mathbf{x}$ if $\boldsymbol{w}^T\boldsymbol{x} > 0$ and $d=-1$

Or (in a different form):

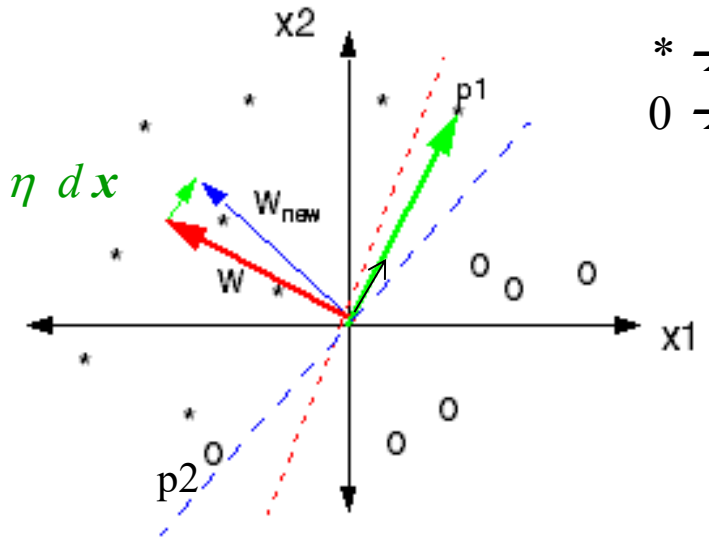
- $\boldsymbol{w}_{new} = \boldsymbol{w} + 1/2\,\eta\ (d\text{-}out)\ \boldsymbol{x}$

Note: Δw *in LMS was different because we have here the sign() in the out computation*

Geometrical view

Dip. Informatica
University of Pisa

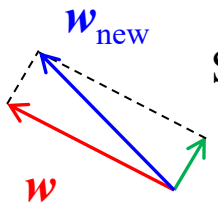
An example: Before updating the weight $\boldsymbol{w}$ (pointing the positive region), we note that both p1 and p2 are incorrectly classified (the red dashed line is decision boundary).
Suppose we choose p1 to update the weights as in picture below.
p1 has target value d=1, so that $\boldsymbol{w}$ is moved a small amount in the direction of p1.
The new boundary (blue dashed line) is better than before (and $\boldsymbol{w}_{new}$ closer to p1).
Exercise: what happens if the training pattern p2 is chosen ?



* → d=+1
0 → d= -1

$$\boldsymbol{w}_{new} = \boldsymbol{w} + \eta\ d\,\boldsymbol{x}$$

Note: only for this example (w_0) bias =0



Sum of vectors

Note the *Hebbian rule*: $\boldsymbol{w}$ moves towards $\boldsymbol{x}$

Delta Rule

Dip. Informatica
University of Pisa

- The form $\boldsymbol{w}_{\text{new}} = \boldsymbol{w} + \eta\ d\,\boldsymbol{x}$ is seen as *Hebbian learning*
- The other form $\boldsymbol{w}_{\text{new}} = \boldsymbol{w} + \eta\ (d\text{-}out)\,\boldsymbol{x} = \boldsymbol{w} + \eta\,\delta\,\boldsymbol{x}$

  is in the form of *error-correction* learning

- Recall from LMS: This is an "*error correction*" rule (delta**/** Widrow-Hoff**/Adaline/LMS rule**) that changes the $\boldsymbol{w}$ proportionally to the error (target d -output):
  - E.g. (target d – output) = err=0 → no correction
  - (input>0) if err + (output is too low), increase $\boldsymbol{w}$ → incr. $\boldsymbol{w}^T\boldsymbol{x}$ → reduce err
  - …
- In terms of "neurons": *the adjustment made to a synaptic weight is proportional to the product of error signal and the input signal that excite the synapse*
- Easy to compute when errors signal δ is directly measurable (we know the desired response for each unit)

Perceptron Convergence Theorem

Dip. Informatica
University of Pisa

- *The perceptron is guaranteed to converge (classifying correctly all the input patterns) in a finite number of steps if the problem is linearly separable.*
- (independently of the starting point, although the final solution is not unique and it depends on the starting point)

- May be "unstable" if the problem is not separable
  - In particular it develop cycles (with a set of weights that are not necessarily optimal)

Note: I'll simply the notations in the proof (e.g. assume the *not* reported scalar product with ^{T} as usual among vectors)

Perceptron Convergence Theorem: preliminaries (1)

Dip. Informatica
University of Pisa

We can focus on a "all positive patterns" task (as shown in the following):

Assume $(\boldsymbol{x}_i, d_i)$ in the TR set, with $d_i = +1 \text{ or } -1 \text{ and } i=1\ldots l$

Linearly separable → $\exists\ \boldsymbol{w}^*$ solution s.t. $(\text{omitting } ^T)$

$$d_i\ (\boldsymbol{w}^* \boldsymbol{x}_i) \geq \alpha, \text{ with } \alpha = \min_i d_i\ (\boldsymbol{w}^* \boldsymbol{x}_i) > 0$$

Hence, $\boldsymbol{w}^* (\ d_i\ \boldsymbol{x}_i) \geq \alpha$

Defining $\boldsymbol{x}_i' = (d_i\ \boldsymbol{x}_i)$, then $\boldsymbol{w}^*$ is a solution iff

$\boldsymbol{w}^*$ is a solution of $(\boldsymbol{x}_i',\ +1)$

(if) $\boldsymbol{w}^*$ solve → $d_i\ (\boldsymbol{w}^* \boldsymbol{x}_i) \geq \alpha$ → $(\boldsymbol{w}^*\ d_i \boldsymbol{x}_i) \geq \alpha$ → $(\boldsymbol{w}^* \boldsymbol{x}_i') \geq \alpha$ → $\boldsymbol{w}^*$ is a solution of $(\boldsymbol{x}_i',\ +1)$

(only if) $\boldsymbol{w}^*$ is a solution of $(\boldsymbol{x}_i',\ +1)$ → $(\boldsymbol{w}^*\ d_i \boldsymbol{x}_i) \geq \alpha$ → $d_i\ (\boldsymbol{w}^* \boldsymbol{x}_i) \geq \alpha$ → $\boldsymbol{w}^*$ solve for $\boldsymbol{x}_i$

Perceptron Convergence Theorem: preliminaries (2)

Dip. Informatica
University of Pisa

Assuming $\boldsymbol{w(0)=0}$ *(at step 0),*

η =1,

$\beta = max_i ||\boldsymbol{x}_i||^2$ (i=1...l, *and* || || denotes the Euclidian norm, see first lesson)

After q errors (all false negative)

$$\boldsymbol{w}(q) = \sum_{j=1 \rightarrow q} \boldsymbol{x}_{(i_j)}$$

i_j is used to denote the patterns belonging to the subset of misclassified patterns e.g., the indices 2, 8, 9,14 in the TR set.

Because $\boldsymbol{w(j)} = \boldsymbol{w(j-1)} + \boldsymbol{x}_{i_j}$

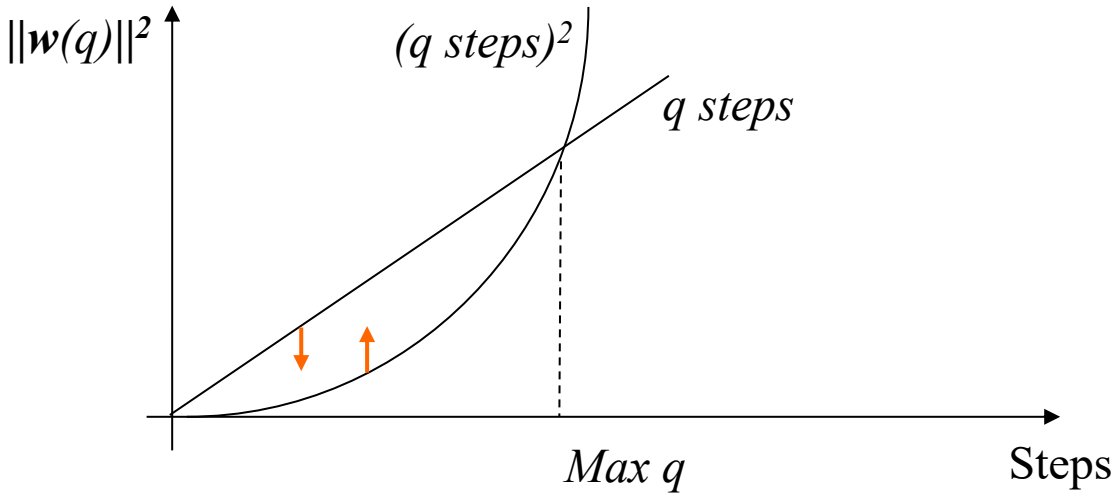
Rule: $\boldsymbol{w}_{new} = \boldsymbol{w} + \eta\ d\ \boldsymbol{x}$ (but all d are +1 here, only positive patterns, and η=1)

Perceptron Convergence Theorem: Proof

Dip. Informatica
University of Pisa

Theorem "For linearly separable tasks, the perceptron algorithm converges after a finite number of steps"

Basic idea: we can find lower and upper bound to $\|w(q)\|^2$ as a function of q^2 steps (lower bound ↑) and q steps (upper bound ↓) → we can find q (number of steps) s.t. the algorithm converges:
[Proof also in Haykin 2nd ed. pages 139-141]



Perceptron Convergence Theorem: Proof (part 1)

Dip. Informatica
University of Pisa

- Lower bound on $\|w(q)\|^2$:

$$(\boldsymbol{w}^*)^T \boldsymbol{w}(q) = (\boldsymbol{w}^*)^T \sum_{j=1 \to q} \mathbf{x}_{i_j} \geq q\alpha$$

Recall that :

- $\boldsymbol{w}^*$ is the solution
- $\alpha = min_i \ ((\boldsymbol{w}^*)^T \boldsymbol{x}_i)$

Cauchy-Swartz: $(\boldsymbol{a}^T \boldsymbol{b})^2 \leq \|\boldsymbol{a}\|^2 \|\boldsymbol{b}\|^2 \implies \|\boldsymbol{a}\|^2 \|\boldsymbol{b}\|^2 \geq (\boldsymbol{a}^T \boldsymbol{b})^2$

Use $\boldsymbol{a} = \boldsymbol{w}^*$ and $\boldsymbol{b} = \boldsymbol{w}(q)$

$$\|\boldsymbol{w}^*\|^2 \|\boldsymbol{w}(q)\|^2 \geq ((\boldsymbol{w}^*)^T \boldsymbol{w}(q))^2 \geq (q\alpha)^2$$

$$\|\boldsymbol{w}(q)\|^2 \geq (q\alpha)^2 / \|\boldsymbol{w}^*\|^2$$

The lower bound

Proof (part 2)

Dip. Informatica
University of Pisa

$$||\mathbf{a}+\mathbf{b}||^2 = \sum_i (a_i + b_i)^2 = \sum_i a_i^2 + \sum_i 2a_i b_i + \sum_i b_i^2 = ||\mathbf{a}||^2 + 2\mathbf{ab} + ||\mathbf{b}||^2$$

- Upper bound on $||w(q)||^2$:

$$||\boldsymbol{w}(q)||^2 = ||\boldsymbol{w}(q-1) + \mathbf{x}_{i_q}||^2 = ||\boldsymbol{w}(q-1)||^2 + 2(\boldsymbol{w}(q-1)^T \mathbf{x}_{i_q}) + ||\mathbf{x}_{i_q}||^2$$

For the *q-th error, this is* <0; hence, deleting a negative term:

$$||\boldsymbol{w}(q)||^2 \leq ||\boldsymbol{w}(q-1)||^2 + ||\mathbf{x}_{i_q}||^2$$

And by iteration ($\boldsymbol{w}(0)=0$)

$$||\boldsymbol{w}(q)||^2 \leq \sum_{j=1->q} ||\mathbf{x}_{i_j}||^2 \leq q\beta$$

$$\beta = max_i ||\boldsymbol{x}_i||^2$$

The upper bound

Proof (part 3): bring them together

Dip. Informatica
University of Pisa

Upper bound Lower bound

$$q\beta \geq ||\boldsymbol{w}(q)||^2 \geq (q\alpha)^2 / ||\boldsymbol{w}^*||^2$$

$$q\beta \geq q^2\alpha'$$

$$\beta \geq q\alpha'$$

$$\boxed{q \leq \beta / \alpha'}$$

i.e. a finite number of steps

Differences between "Perc. Learning Alg." and LMS Alg. (I)

Dip. Informatica
University of Pisa

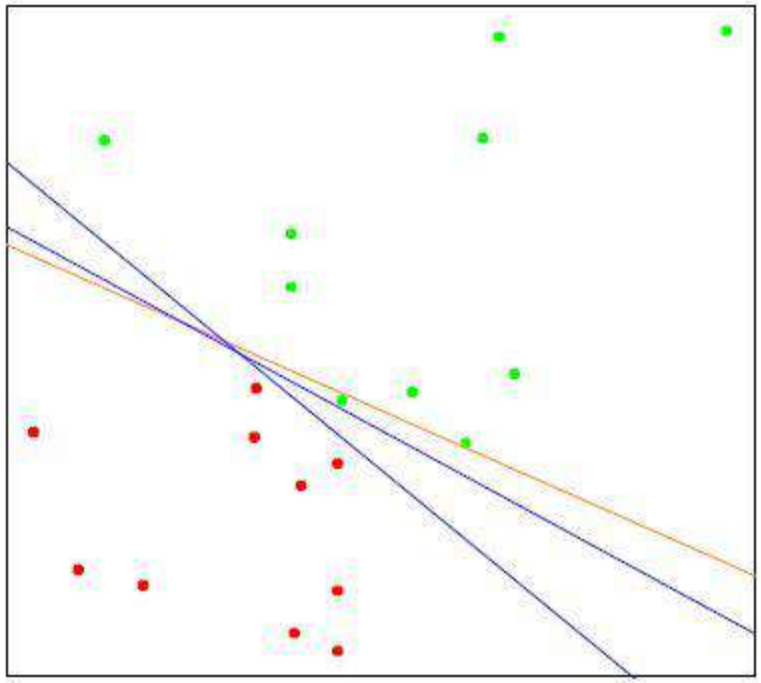
$$\boldsymbol{w}_{\text{new}} = \boldsymbol{w} + \eta\ (d\text{-}out)\ \boldsymbol{x}$$

- Apparently similar but note that:
  - LMS rule was derived (by the gradient) without threshold activation functions: minimization of the error of the linear unit (using directly $\boldsymbol{w}^T\boldsymbol{x}$)
  - The perceptron use $out = sign(\boldsymbol{w}^T\boldsymbol{x})$
- Hence, for training:
  - $\delta = (d - \boldsymbol{w}^T\boldsymbol{x})$ for the LMS approach
    versus
  - $\delta = (d - sign(\boldsymbol{w}^T\boldsymbol{x}))$ for the perceptron learning alg.
- Of course the model trained with LMS can still be used for classification applying the threshold function $h(\boldsymbol{x}) = sign(\boldsymbol{w}^T\boldsymbol{x})$ (LTU)

LMS can misclassify in linear separable problems

Dip. Informatica
University of Pisa

- LMS not necessarily minimize the number of TR examples misclassified by the LTU



$$E(\mathbf{w}) = \sum_{p} (y_p - \mathbf{x}_p{}^T \mathbf{w})^2$$

It changes the weights **not only** for the misclassified patterns!

FIGURE 4.14. *A toy example with two classes separable by a hyperplane. The orange line is the least squares solution, which misclassifies one of the training points. Also shown are two blue separating hyperplanes found by the perceptron learning algorithm with different random starts.*

Differences Summary

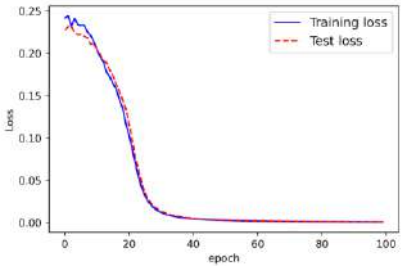
Dip. Informatica
University of Pisa

Perceptron Learn. Alg.

- Min. misclassifications ($out = sign(\mathbf{w}^T \mathbf{x})$)
- it always converges for linear separable problems (in a finite number of steps) to a perfect classifier
- (else no convergence)
- difficult to be extended to networks of units (NN)

LMS Alg.

- Min. $E(\mathbf{w})$ with $out = \mathbf{w}^T \mathbf{x}$
- asymptotic convergence (*), also for not linear separable problems
- not always zero classification errors, linear separable problems
- it can be extended to networks of units (NN), using the gradient based approach

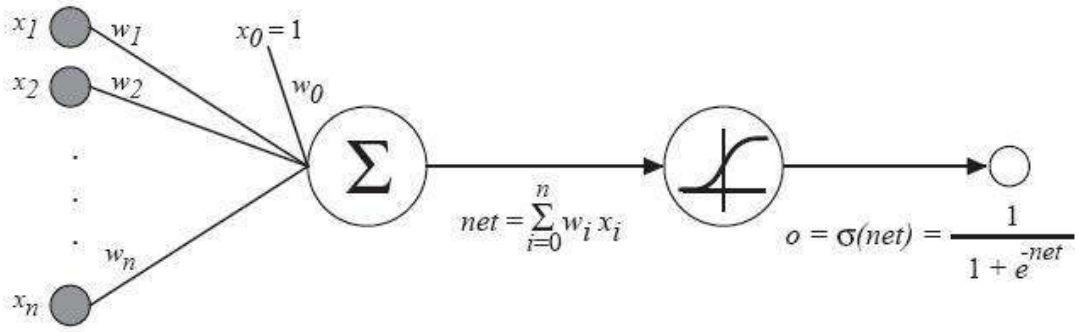


(*) example of *asymptotic* convergence (training with MSE loss)

Activation functions

Dip. Informatica
University of Pisa

1. Linear function
2. Threshold (or step) function (Perceptron/LTU)
3. A non-linear *squashing* function like the **sigmoidal** logistic function: assumes a continuous range of values in the bounded interval [0,1]



$$f_\sigma(x) = \frac{1}{1+e^{(-ax)}}$$

- The sigmoidal-logistic function has the property to be a *smoothed differentiable threshold function*.
- a is the *slope* parameter of the sigmoid function

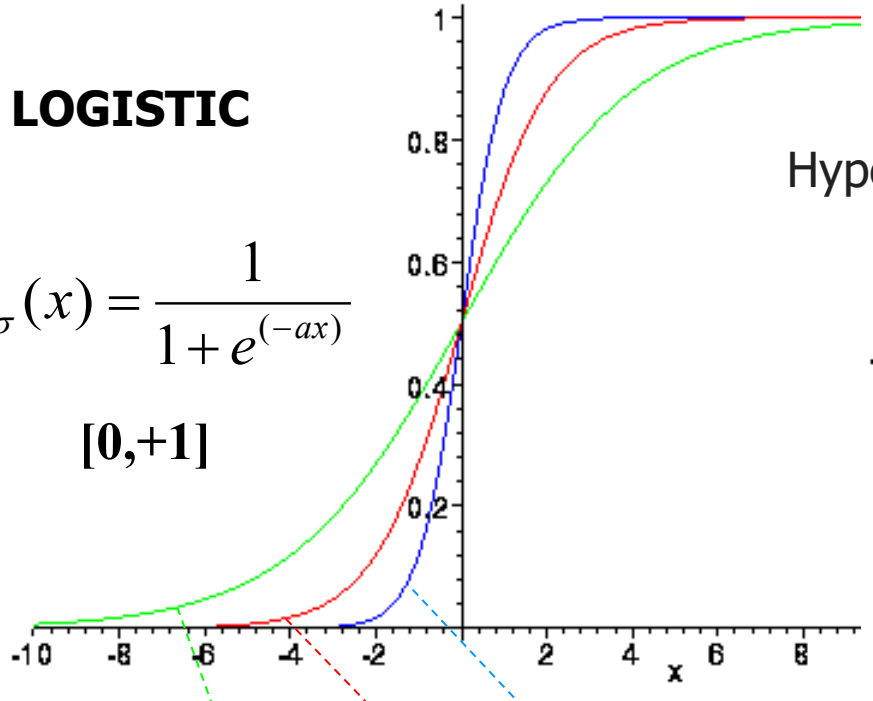
Activation functions (type 3.)

Dip. Informatica
University of Pisa

LOGISTIC

$$f_\sigma(x) = \frac{1}{1+e^{(-ax)}}$$

[0,+1]



a=0.5 green, *a*=1 red, *a*=2 blue

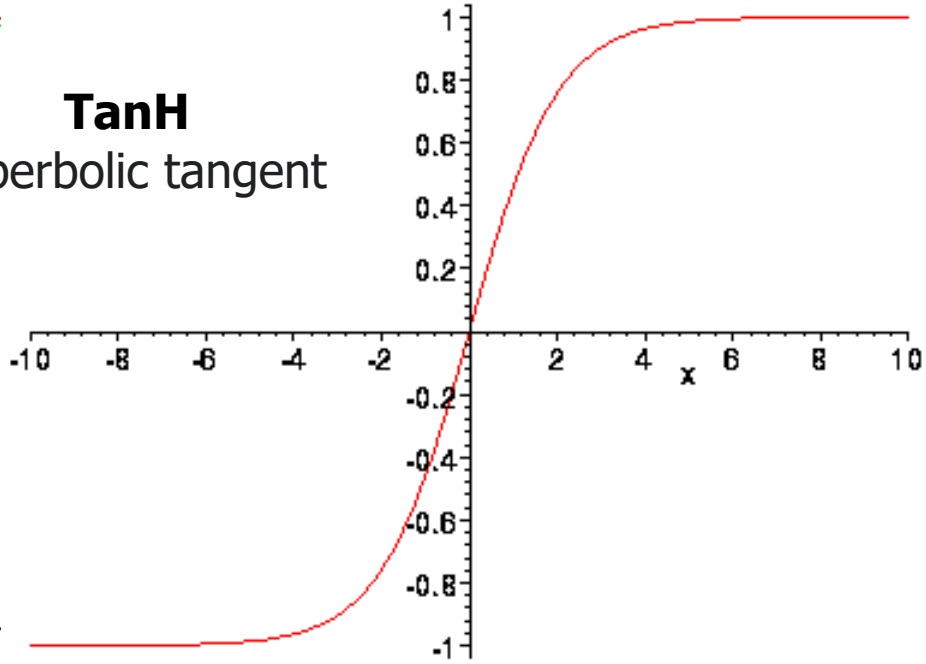
Semilinearity close to 0

What if $a \rightarrow 0$? (to linear)

$a \rightarrow \infty$? (to LTU)

TanH

Hyperbolic tangent



$f_{symm} = 2\, f_\sigma(x) - 1 = tanh(ax/2)$

[-1,+1]

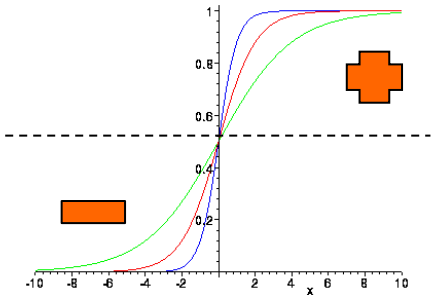
Sigmoidal functions and classifier outcomes

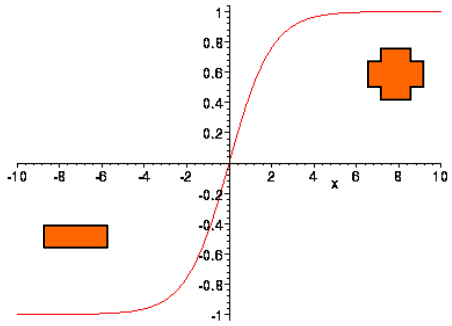
Dip. Informatica
University of Pisa

These functions provide continues outputs but…

- For the Logistic function an output value
  - ≥0.5 (**threshold**) correspond to the **positive class**
  - <0.5 correspond to the **zero or negative class**
- It is possible to change this threshold (e.g. studying the effect on FP/FN etc, or by a ROC),
- and even to consider a *rejection zone* in an interval around the threshold value (to avoid fragile decisions)

- For the TanH the **threshold is in 0** (with the analogues possibilities)





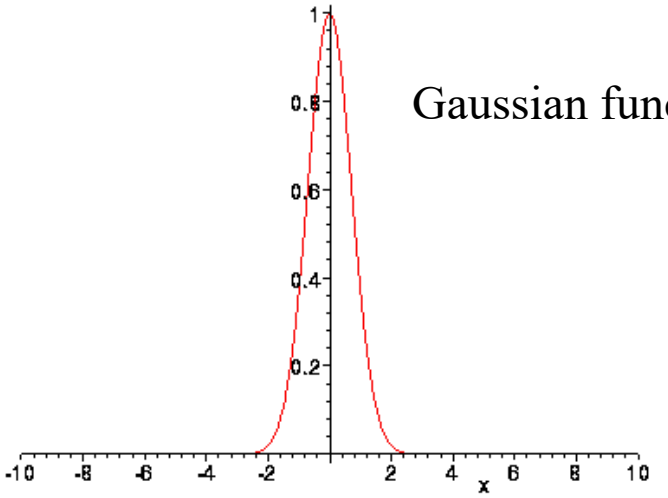
Other Activation Functions

Dip. Informatica
University of Pisa

- Radial Basis Functions → *RBF networks*

$$f(x) = e^{(-ax^2)}$$

$$\left\| \boldsymbol{w} - \boldsymbol{x}_{input} \right\|$$



Gaussian function

- Softmax: see next lectures (multi-output case)
- Stochastic neurons: output is +1 with probability $P(net)$ or –1 with $1-P(net)$ → *Boltzmann machines and other models rooted in statistical mechanics*

Other Activation Functions (2)

Dip. Informatica
University of Pisa

- Tanh-like (piecewise linear approximation) for efficient computation

$$f(x) = \text{sign}(x)\left[1 + \frac{1}{2^{\lfloor 2^n |x| \rfloor}}\left(\frac{2^n |x| - \lfloor 2^n |x| \rfloor}{2} - 1\right)\right]$$

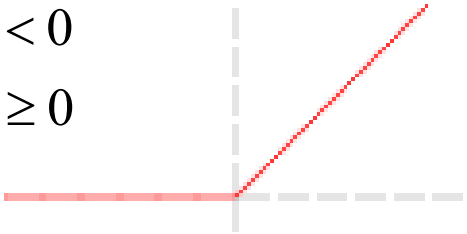
n steepness (slope) of the function
Rounding operator (floor)
Shift operation in binary rep.

- Rectifier →**ReLU** (Rectified Linear Unit): see *Deep Learning* section

It has become a default choice for Deep models, so it (and its variants) deserves more attention discussing Deep Learning

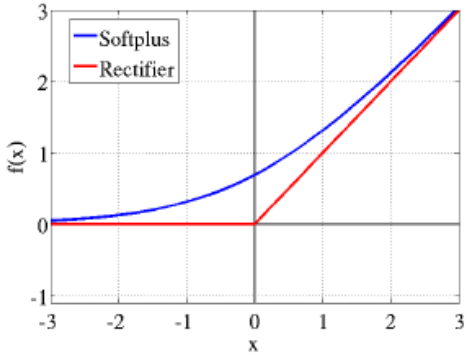
$$f(x) = \begin{cases} 0 & for \quad x < 0 \\ x & for \quad x \geq 0 \end{cases}$$

$$f(x) = \max(0, x)$$



- **Softplus** (smooth approximation)

$$f(x) = \ln(1 + e^x)$$



- An updated list (Note: many perform comparably!):

https://en.wikipedia.org/wiki/Activation_function

Activation functions: derivatives

Dip. Informatica
University of Pisa

- The derivative of the **identity function** is 1.
- The derivative of the **step** (threshold) **function** is not defined, which is exactly why it isn't used with LMS .
- **Sigmoids**: for asymmetric and symmetric case we have ($a=1$):

$$\frac{df_\sigma(x)}{dx} = f_\sigma(x)(1-f_\sigma(x)) \qquad \frac{df_{\tanh}(x)}{dx} = 1-f_{\tanh}(x)^2$$

Show by exercise: also with any a

LMS with f_σ

Dip. Informatica
University of Pisa

- The sigmoidal-logistic function has the property to be a *smoothed differentiable threshold function*

- Hence, we can derive a Least (Mean) Square algorithm by computing the gradient of the mean square loss function as for the linear units (also for a classifier)

- From $o(\mathbf{x}) = \mathbf{x}^T\mathbf{w}$ to $o(\mathbf{x}) = f_\sigma(\mathbf{x}^T\mathbf{w})$ *where* f_σ *is a logistic function*
- Find $\mathbf{w}$ to *minimize* the residual sum of squares:

$$E(\mathbf{w}) = \sum_p (d_p - o(\mathbf{x}_p))^2 = \sum_p (d_p - f_\sigma(\mathbf{x}_p{}^T\mathbf{w}))^2$$

LMS with 1 non-linear unit Compute the gradient

Dip. Informatica
University of Pisa

Important *Exercise*: compute the gradient of the error function

(do it before the next lecture, also in the simplifed form for E_p)

$$\frac{\partial E(\boldsymbol{w})}{\partial w_j} \quad \textit{with } o(\boldsymbol{x}_p)=out(\boldsymbol{x}_p)=f_\sigma(\boldsymbol{x}_p{}^T\boldsymbol{w})=f_\sigma(net(\boldsymbol{x}_p))$$

Result (1 unit, all the patterns $p = 1..l$):

$$\frac{\partial E(\boldsymbol{w})}{\partial w_j} = -2\sum_{p->l}(\boldsymbol{x}_p)_j(d_p - f_\sigma(\boldsymbol{x}_p{}^T\boldsymbol{w}))f'_\sigma(\boldsymbol{x}_p{}^T\boldsymbol{w}) = -2\sum_{p->l}(\boldsymbol{x}_p)_j \boxed{\text{new } \delta_p \quad \delta_p f'_\sigma(net(\boldsymbol{x}_p))}$$

We can simplify the notation as:

$$\frac{\partial E(\boldsymbol{w})}{\partial w_j} = -2\sum_{p=1}^{l} \boxed{\text{new } \delta_p \quad (d_p - o(\boldsymbol{x}_p))f'_\sigma}\, x_{p,j}$$

Note: here we have just 1 unit, more indices will be used with more units, see backprop.

Gradient descent algorithm

Dip. Informatica
University of Pisa

- The same as for linear unit using the *new delta rule* (batch/on-line):

$$\boldsymbol{w}_{\text{new}} = \boldsymbol{w} + \eta \; \delta_p \, \boldsymbol{x}_p$$

Pattern p
The unique Unit has not index here

$$(d_p - f_\sigma(net(\boldsymbol{x}_p)))f'_\sigma(net(\boldsymbol{x}_p))$$

…or simplifying the notation

$$(d_p - o(\boldsymbol{x}_p))f'_\sigma$$

Gradient descent algorithm as an error correction rule

Dip. Informatica
University of Pisa

- The same as for linear unit using the *new delta rule* (batch/on-line):

$$\boldsymbol{w}_{\text{new}} = \boldsymbol{w} + \eta\ \delta_p\ \boldsymbol{x}_p \qquad \boxed{\delta_p = (d_p - o(\boldsymbol{x}_p))f'_\sigma}$$

Again, an **error correction rule**. Moreover:

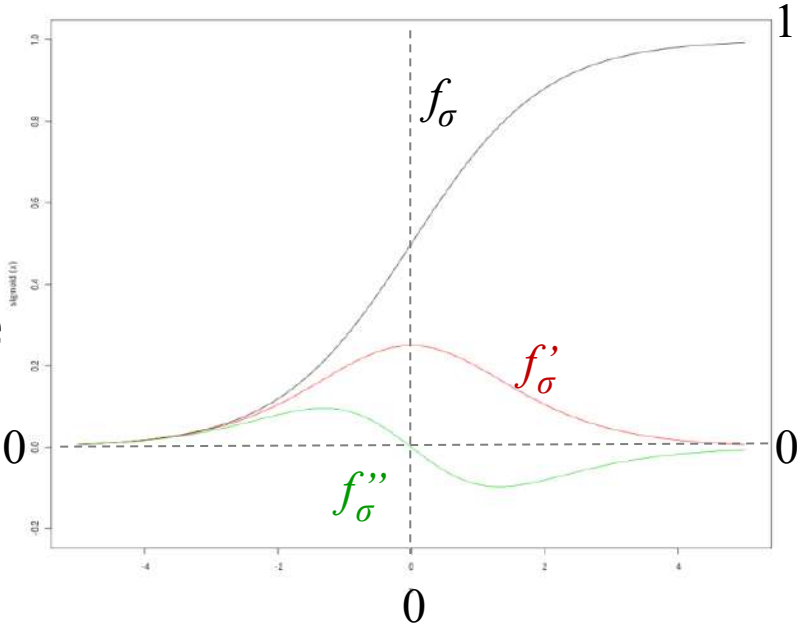
• The parameters a (slope of f) can affect the step of gradient descent

• Max of f_σ' for inputs (*net*) close to 0 (~ linear unit). Here high value of the δ are possible

• Minimum of f_σ' are for **saturated cases** (where f go to 0 or 1 asymptotically) → better to avoid a premature saturation, small δ, and hence very slow changes of $\boldsymbol{w}$, at the beginning (starting with small weights) or later



• Insight: we see also a bridge between the two alg.s: toward no corrections for correct outputs also for LMS alg. (as it was for the perceptron learn. alg.).
I.e. Using f_σ we are approximating not only the LTU but also the perceptron learn. alg.



Plot of f_σ (black) f'_σ (red) f''_σ (green)

Neural Networks (NN)

Dip. Informatica
University of Pisa

All the ingredients are ready:
NN by two views of Multi-Layer Perceptron (MLP)-NN:

A. A network of interconnected units

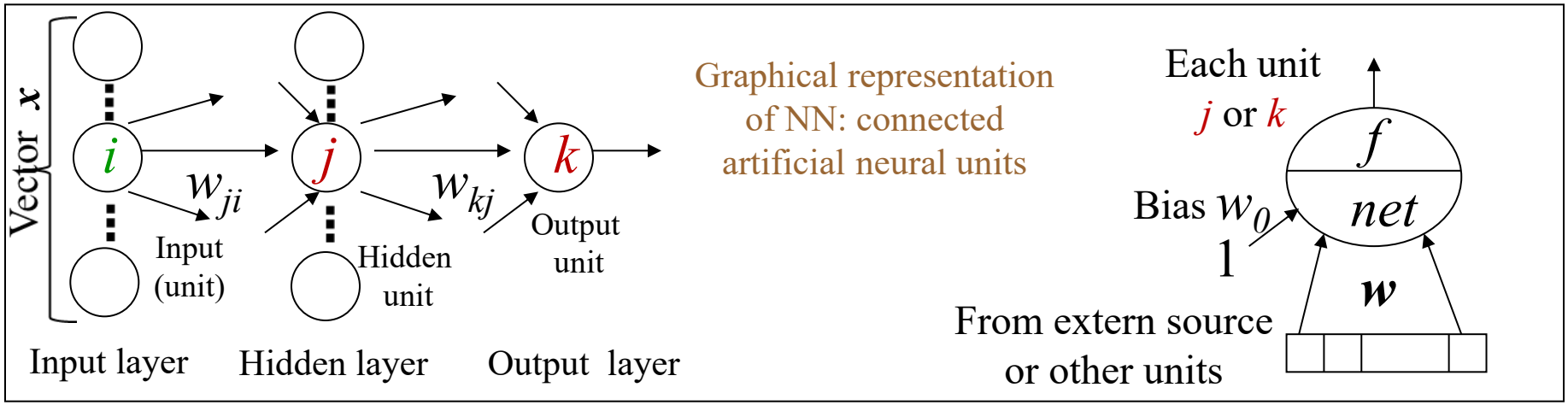
B. A flexible function $h(x)$ (as nested non-linear functions)

The Neural Network (NN)

Standard feedforward NN (with one hidden layer)

Dip. Informatica
University of Pisa

A. A network of units



In a **MLP** (Multi Layer Perceptron) architecture

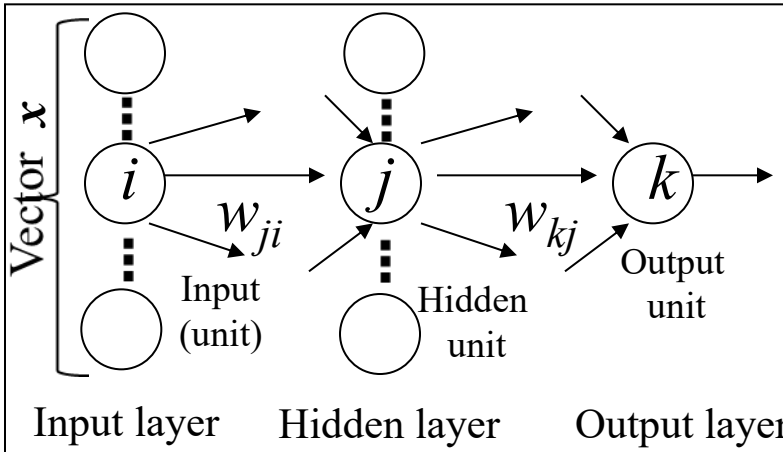
- the units are connected by **weighted links** and they are organized in the form of **layers**
- the **input layer** is simply the source of the input ***x*** that projects onto the hidden layer of units: it loads (copy) the (extern) input patterns ***x*** (the input units *i* do <u>not</u> compute the *net* and *f*)
- the **hidden layer** projects onto the **output layer** (feedforward computation of this two-layers network in the fig.) or to another hidden layer

Neural Networks: two views

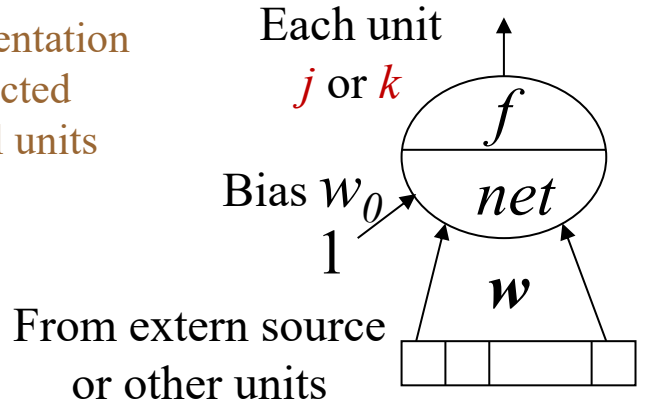
Standard feedforward NN (with one hidden layer)

Dip. Informatica
University of Pisa

A. A network of units



Graphical representation of NN: connected artificial neural units



B. A flexible function

$$h(\boldsymbol{x}) = f_k\left(\sum_j w_{kj} f_j\left(\sum_i w_{ji} x_i\right)\right)$$

Free parameters (weight matrix)

Non-linear function (sigmoid)

Input variables (vector $\boldsymbol{x}$)

Recall of a linear function

$$h(\boldsymbol{x}) = \sum_i w_i x_i$$

Neural Networks: components

Dip. Informatica
University of Pisa

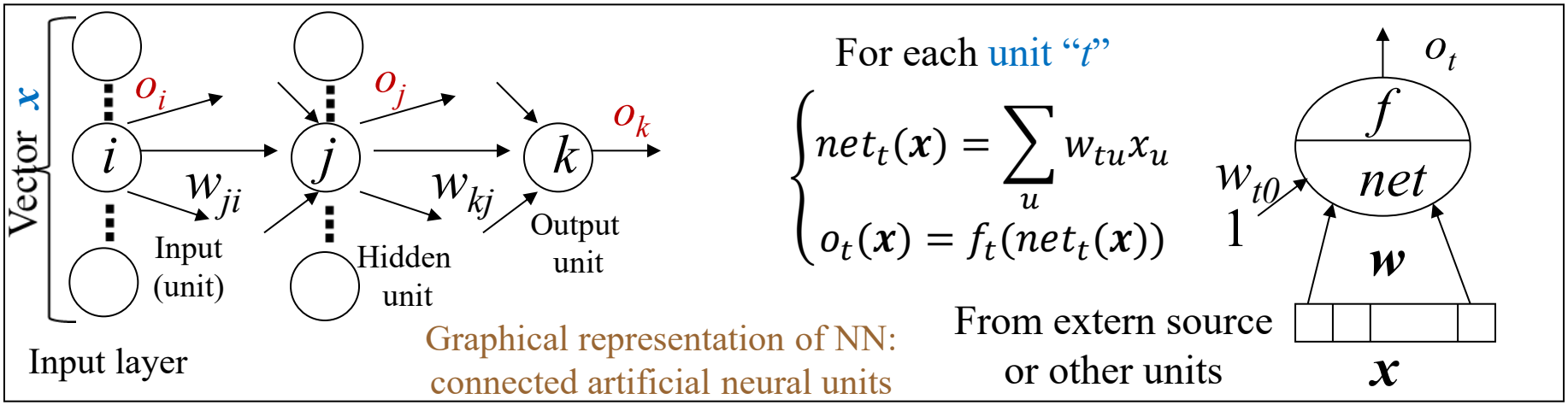
- NN model traditionally presented by the type of
  - UNIT: net, activation functions
  - ARCHITECTURE: number of units, topology (e.g. also number of layers)
  - LEARNING ALGORITHM

The NN and their Units

Standard feedforward NN (with one hidden layer)

Dip. Informatica
University of Pisa

A. A network of units



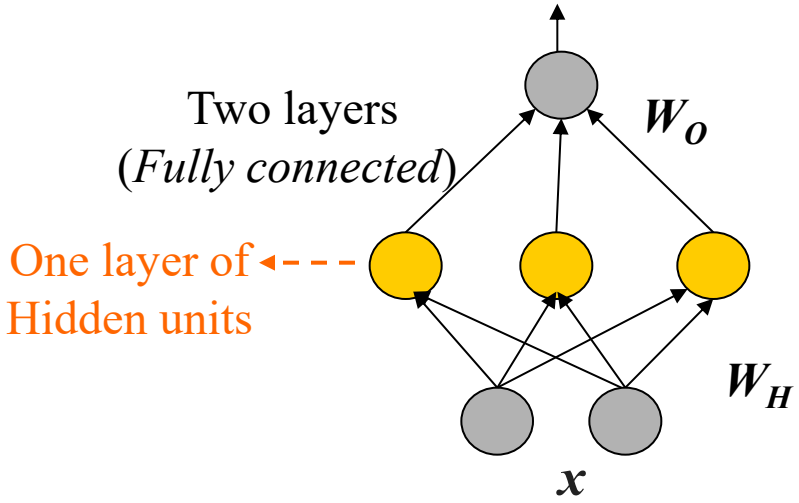
Notation:

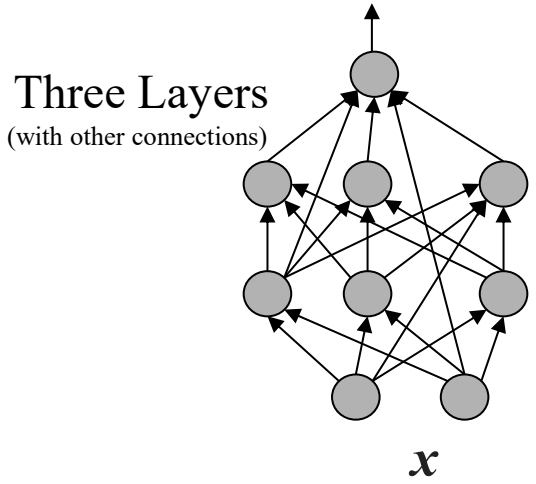
- The index t denotes a generic unit, it can be either j or k (*)
- The index u denotes a generic input component, it can be either i or j (*)
- In the unit, $\boldsymbol{x}$ is a generic input from an external source (input vector) or from other units (*) *according to the position of the unit in the network*
- If we load the pattern $\boldsymbol{x}$ in the input layer, we can use the notation with $\boldsymbol{o}$ for both the inputs and the hidden units outputs. Hence, *inside the network*, the input to each unit t from *any source* u (through the connection w_{tu}) is typically denoted as o_u (we will use this style in the back-prop derivation).

MLP Architecture
Standard feedforward NN

Dip. Informatica
University of Pisa

- The *architecture* of a NN defines the topology of the connections among the units.
- The two-layer **feedforward neural network** described in Equation $h(x)$ corresponds to the well-know **MLP** (Multi Layer Perceptron) architecture:

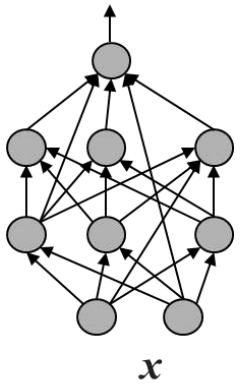




Feedforward processing

Dip. Informatica
University of Pisa

The processing of a pattern for feedforward NN precedes as in the following (from the input layer to the output layer):



- For each input pattern $\boldsymbol{x}$

1. the input pattern is load in the input layer
2. we compute the output of all the units of the 1st hidden layer
3. we compute the output of all the units of the 2st hidden layer and so on for all the hidden layers
4. we compute the output of all the units of the output layer (NN output $h(\boldsymbol{x})$)
5. we can now compute the error (delta) at the output level

Architectures: Feedforward versus Recurrent

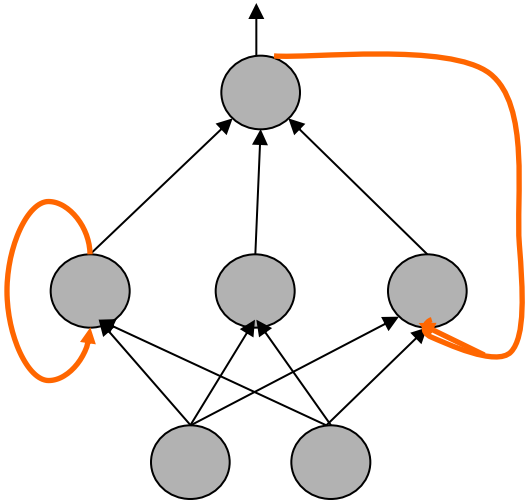
Dip. Informatica
University of Pisa



- **Feedforward**: direction: input→ output
- **Recurrent** neural networks: A different category of architecture, based on the addition of *feedback loops* connections in the network topology,
  - The presence of **self-loop** connections provides the network with dynamical properties, letting a memory of the past computations in the model.
  - This allows us to extend the representation capability of the model to the processing of sequences (and structured data).

Recurrent neural networks:

- See separate slides in this course
- They will be the subject (further developed) of the CNS (Computation Neuroscience) course.

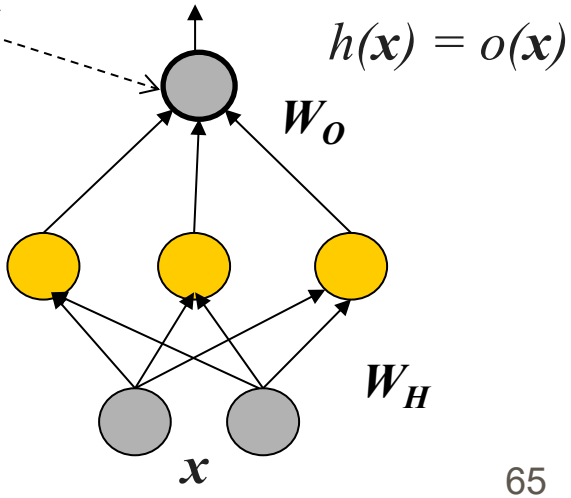


1. Notes on NN flexibility: Which Tasks?

Dip. Informatica
University of Pisa

- **Hypothesis space**: *continuous* space of all the functions that can be represented by assigning the weight values of the given architecture.
- Note that, depending on the class of values produced by the network output units, discrete or continuous,
the model can deal, respectively, with **classification tasks** (sigmoidal output *f.*) or **regression tasks** (linear output *f.*) tasks.

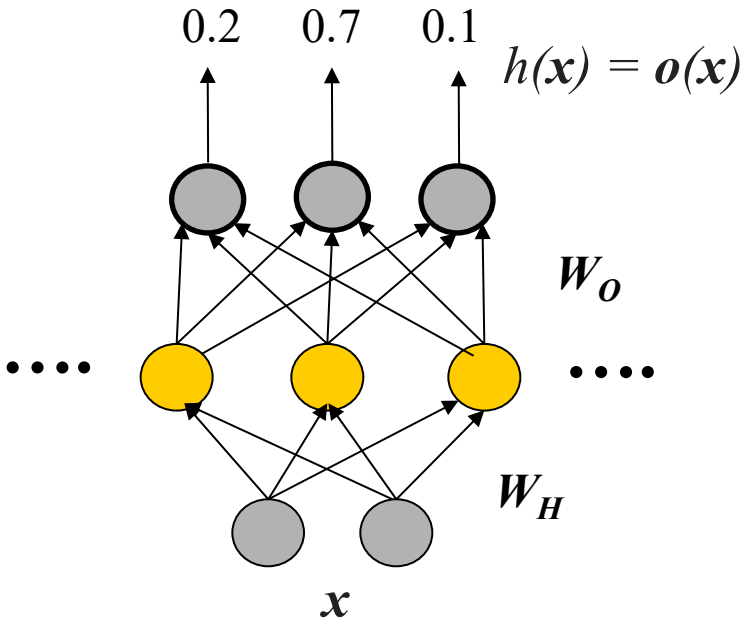
$$h(\boldsymbol{x}) = f_k\left(\sum_j w_{kj} f_j\left(\sum_i w_{ji} x_i\right)\right)$$



1. Notes on NN flexibility
Which Tasks?

Dip. Informatica
University of Pisa

- Also **multi-regression** or **multi-classes classifier** can be obtained by using **multiple output units**



1. NN as a *function*

Dip. Informatica
University of Pisa

$$h(\boldsymbol{x}) = f_k\left(\sum_j w_{kj} f_j\left(\sum_i w_{ji} x_i\right)\right)$$

- This is the *function* computed by a two-layer feedforward neural network
- Units and architecture just a graphical representation (of the data flow process)
- Each $f_j\left(\sum_i w_{ji} x_i\right)$
- can be seen as computed by an independent processing element (unit, a hidden unit), or a **special kind of phi** (ϕ) of LBE
- Also, NN is a function **non linear** in the parameters $\boldsymbol{w}$

1. NN as a dictionary approach (LBE) (1, equations)

Dip. Informatica
University of Pisa

- **LBE**: (Recall) **fixed** linear basis functions

$$h(\boldsymbol{x}) = \sum_j w_j \phi_j(\boldsymbol{x}) \qquad h(\boldsymbol{x}) = f\left(\sum_j w_j \phi_j(\boldsymbol{x})\right)$$

Regression

Classification
(either regression with f = identity function)

- **Neural Network**

$$h(\boldsymbol{x}) = f_k\left(\sum_j w_{kj} f_j\left(\sum_i w_{ji} x_i\right)\right) = f\left(\sum_j w_j \phi_j(\boldsymbol{x}, \boldsymbol{w})\right)$$

using

$$\phi_j(\boldsymbol{x}, \boldsymbol{w}) = f_j\left(\sum_i w_{ji} x_i\right)$$

hidden units' output

because now ϕ depends also on $\boldsymbol{w}$

1. NN as a dictionary approach (LBE) (2, discussion)

$$h(\boldsymbol{x}) = f_k\left(\sum_j w_{kj} f_j\left(\sum_i w_{ji} x_i\right)\right)$$

Dip. Informatica
University of Pisa

- **LBE**: (Recall) **fixed** linear basis functions (phi preselected: does not change with training data),
  - And linear w.r.t. parameters w:

$$\sum_j w_j \phi_j(\boldsymbol{x})$$

- **NN: Adaptive** (flexible) basis functions approach:

the basis functions themselves are **adapted** to data (by fitting the $\boldsymbol{w}$ in phi)

$$\phi_j(\boldsymbol{x}, \underline{\boldsymbol{w}}) = f_j\left(\sum_i w_{ji} x_i\right)$$

- Note that we fix the same type of basis functions for all the terms in the basis expansion (given by the activation function)

- $h(\boldsymbol{x})$ as (nonlinear function of weighted) sums of nonlinearly transformed linear models + the important enhancement of adaptivity

Please came back to these slides after SVM (another form of LBE) to compare

1. Hidden Layer Relevance and Interpretation

Dip. Informatica
University of Pisa

- Each basis function (**hidden unit**) computes a **new** nonlinear derived features, **adaptively** (by learning, according to the training data)
- I.e. the parameters of the basis function $\boldsymbol{w}$ are learned from data by learning

$$\phi_j(\boldsymbol{x}, \underline{\boldsymbol{w}}) = f_j\left(\sum_i w_{ji} x_i\right)$$

In other words:

- The representational capacity of the model is related to the presence of a hidden layer of units, with the use of non-linear activation function, that transforms the input pattern into the **internal representation** of the network.
- The learning process can define a suitable internal representation, also visible as new hidden features of data, allowing the model to extract from data the higher-order statistic that are relevant to approximate the target function.

1. Note: Non-linear hidden layer is need

Dip. Informatica
University of Pisa

- Non-linear units are essential:
  MLP with linear units = 1 layer NN !

$$h(\boldsymbol{x}) = f_k\left(\sum_j w_{kj} f_j\left(\sum_i w_{ji} x_i\right)\right)$$

- Exercise: using f as identity function you can rewrite $h(\boldsymbol{x})$ using a singular layer of units

- Anyway for the learning algorithm this introduces an issue: The model is non-linear in the parameters, i.e. w.r.t. to $\boldsymbol{w}$ → we have a non linear optimization problem

2. Universal approximation (important!)

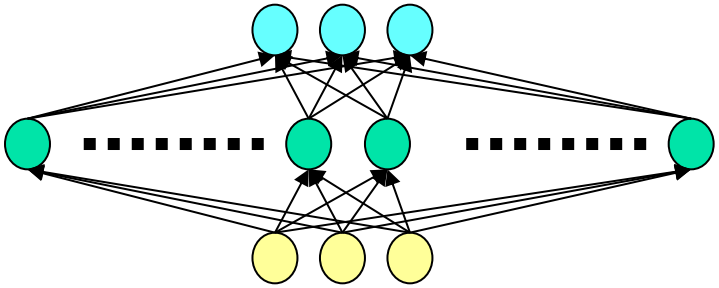
Dip. Informatica
University of Pisa

- Many early results (Cybenko 1989, Hornik et al. 1993, etc…)
- Shortly:
  - A single hidden-layer network (with logistic activation functions) can approximate (arbitrarily well) every continuous (on hyper cubes) function (provided enough units in the hidden layer)

    [e.g. image it generalizes approximation by finite Fourier series]
  - A MLP network can approximate (arbitrarily well) every input-output mapping (provided enough units in the hidden layers)

$$\text{Given } \varepsilon,\ \exists\ h(\boldsymbol{x}) \text{ s.t. } |(f(\boldsymbol{x})-h(\boldsymbol{x})|<\varepsilon \quad \forall\ \boldsymbol{x} \text{ in the hypercube}$$

$$h(\boldsymbol{x}) = f_k\left(\sum_j w_{kj} f_j\left(\sum_i w_{ji} x_i\right)\right) \quad \text{linear}$$



Existence theorem! This not provide the algorithm nor the number of units

NN Expressive power

Dip. Informatica
University of Pisa

- The ***expressive power*** of NN is strongly influenced by two aspects: the number of units and their configuration (architecture)

- The number of units can be related to the discussion of the **VC-dimension** of the model.

  Specifically, the network capabilities are influenced by the number of parameters w, that is proportional to the number of units *,

  and further studies report also the dependencies on their value sizes, e.g.:
  - Weights = 0 → minimal VC-dim
  - Small weights → linear part of the activation function (small VC-dim)
  - Higher weights values → more complex model

* Number of parameters (fully-conn. MLP)= #input-units X #hidden-units X #output-units

- Next lectures: NN as a *variable-size hypothesis space*
- Indeed, we start with weight values near 0, then w values increase during training → higher VC-dim → it is a variable size hp space

How many layers?

Dip. Informatica
University of Pisa

A look ahead (toward **deep learning**).

- The univ. approx. theorem is a fundamental contribution
- It show that 1 hidden layer is sufficient in general, but it does **not** assure that a "small number" of units could be sufficient (it does not provide a limit on such number)
- It is possible to find boundaries on such number (for many f. families)
- But also to find "*no flattening*" results (on efficiency, not expressivity):
  - cases for which the implementation by a single hidden layer would require an exponential number of units (w.r.t *input dim.*), or non-zero weights,
  - while more layers can help (it can help for the number of units/weights and/or for learning such approximation)
  - But is it easy to optimize (training) a MLP with many layers?
- See later for deep learning.

On the inductive bias of NN

Dip. Informatica
University of Pisa

- NN with backpropagation learning algorithm:
- Generally related to the ***smoothness*** properties of functions:
  - Small input variations → small output variations
  - E.g. a locally limited value of the first derivative
  - A very common assumption in ML

- Why make sense? A target function that is **Non-smooth**: random number generator → generalization cannot be achieved !

3. NN Learning Algorithm



Dip. Informatica
University of Pisa

- Repetita: **The learning algorithm** allows adapting the free-parameters $\boldsymbol{w}$ of the model, i.e. the values of the connection-weights, in order to obtain the best approximation of the target function.

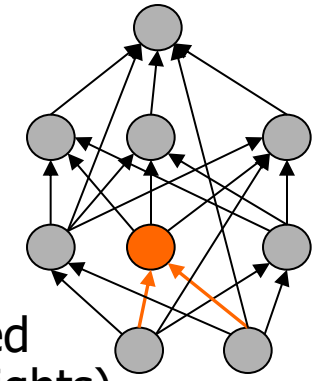
- As usual, we will realize this in terms of minimization of an error (or loss) function on the training data set.

From Perceptron to NN

Dip. Informatica
University of Pisa



- **Perceptron**:
  - We have a Perceptron learning algorithm (delta rule)
  - cannot represent all the Boolean functions (e.g. XOR)
- A **network of Perceptrons** can represent every Boolean function
- Problem: define a **Learning Algorithm for the network (MLP)**



- *What is different? Credit assignment problem*:
  - Which credit to the **hidden units**?
  - not easy (as for single Percetron) when errors signal is not directly measurable: we *don't* know the error(***delta***)/desired response for the **hidden units** (useful to change their weights).
- (Repetita) Non-linear wrt to w → *non linear optimization problem*
- Supposed too difficult by Minsky-Papert (1969), while faced by many researchers (see historical notes) with the *backpropagation* algorithm popularized by Rumelhart, Hinton, Williams in the PDP book (1986) → renaissance of NN

The loading problem

Dip. Informatica
University of Pisa

Note that in general, it is indeed a difficult problem:

- *Loading problem*: ("loading" a given set of TR data into the free par. of the NN)
  - given a network and a set of examples
  - answer yes/no: is there a set of weights so that the network will be consistent with the examples?

- The loading problem is *NP-complete* (Judd, 1990),
  it is *not* known a polynomial alg. to solve it.

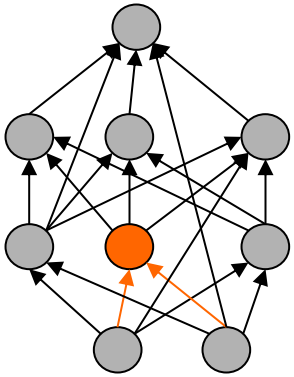
- In practice networks can be trained in a reasonable amount of time (see the back prop alg.) although optimal solution is not guaranteed.

How to solve?

Dip. Informatica
University of Pisa

Key steps:

- Credit assignment problem: how to change the hidden layer weights?
- The ***gradient descend approach***, minimizing a loss function, can be extended to MLP, provided that the *loss function* and *the activation functions* are *differentiable functions*
  - So, to find the delta for every units in the network



Intro to Backpropgation (BP)

Dip. Informatica
University of Pisa

- **The problem is the same we have seen** for the other models, starting again with the **LMS** approach (error/data term):
  - **Given** a set of l training example $(\boldsymbol{x}_p, d_p)$ and a (inner) loss measure L *(e.g.* $L(h(\boldsymbol{x}_p), dp) = (d_p - h_{\boldsymbol{w}}(\boldsymbol{x}_p))^2$ *for the MSE)*
  - **Find**: The weight vector $\boldsymbol{w}$ that minimizes the expected error on the training data (we first focus on the data term), **by computing the gradient of the error function** :

$$E(\boldsymbol{w}) = R_{emp} = \frac{1}{l}\sum_{p=1}^{l}(d_p - h(\boldsymbol{x}_p))^2$$

- **What we need**: differentiable loss, differentiable activation functions, a network to follow the information flow (!)

Backprop. Properties

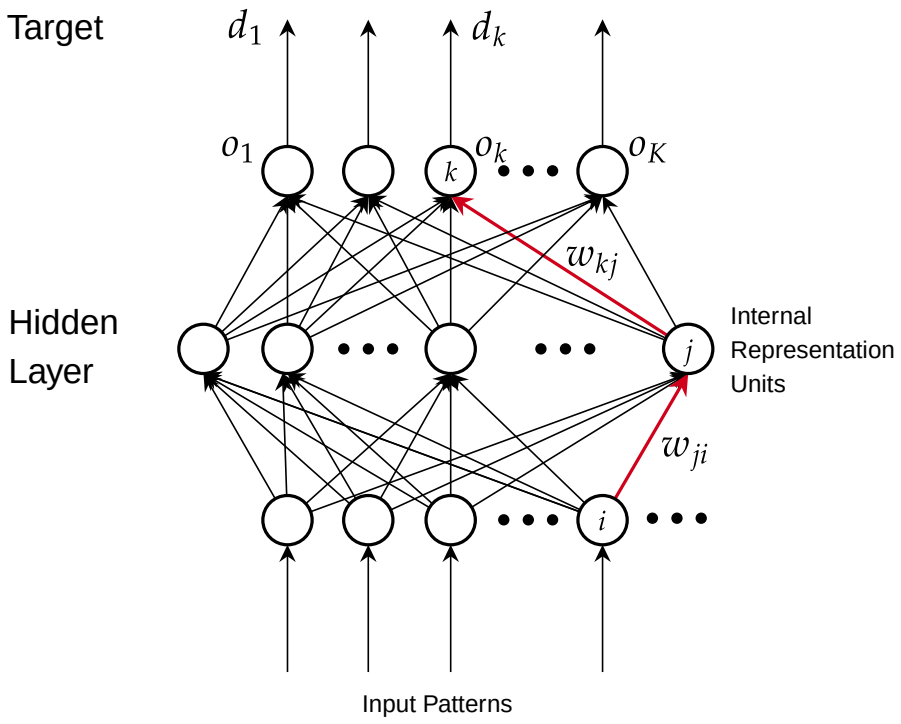
Dip. Informatica
University of Pisa

- **Nice properties of the Backprog. alg.** (also for programming):
  - Easy because of the *compositional* form of the model
  - It keeps track only of quantities local to each unit (by local variables) → *modularity* of the units is preserved
  - *Efficiency*: O(#W) instead of O(#W^2) (factorization of deltas, see later)
  - On the *biological plausibility*: controversial, but it has been supposed that in the brain, to learn, we have a local suboptimal approx. of BP

Caminante no hay camino, se hace camino al andar. **Antonio Machado**
(There is no path, traveler; the path is made by walking)

Back-propagation

The basic architecture is a feed-forward fully connected neural network (MLP). We use different indices, k, j and i, as generic indices for the different layers of the MLP.



The problem: to estimate the contribution of hidden units to the error at the output level. We do this by computing the Generalized Delta Rule.

Training Set (TR) = $\{(\mathbf{x}^1, d^1), \ldots, (\mathbf{x}^l, d^l)\} \rightarrow$ Supervised Learning.
Each input unit i is loaded with the input x_i (component i of the input vector $\mathbf{x}$), hence $o_i = x_i$ per each pattern $p = 1...l$.

Total Error (of the network on the TR set): $E_{tot} = \sum_p E_p$, where $E_p = \frac{1}{2}\sum_{k=1}^{K}(d_k - o_k)^2$ is computed for the input pattern p for all the K units.
Note that, in order *to simplify*, p and x_p (or o_p) are and will be omitted in the following. E.g. o_k is for $o_k(\mathbf{x}_p)$ and d_k is for $d_{p,k}$. Above, for this reason, the patterns in the TR set were denoted in a special form as $(\mathbf{x}^p, d^p)$ without using the subscript form for $p = 1...l$.

Objective: find the values of parameters $\mathbf{w}$ that **minimize** E_{tot} (we first focus on the data term).

Least Mean Square (LMS): the minimization of the above Total Error.

Idea: Perform a descent on the surface of E_{tot} on the basis of the gradient (Figure 1).

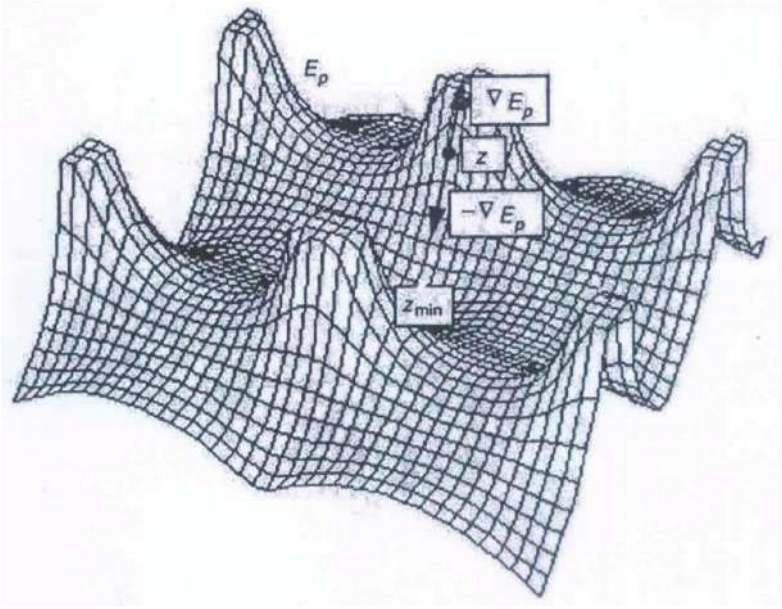


Figure 1: Example of E_{tot} landscape in the space of two weights (w_1 and w_2); the local gradient is shown in the point Z. Along the direction of (−gradient) we can reach the Z_{min} point.

Iterative gradient descent training algorithm

Description: Compute the loss (error) function (i.e. E_{tot}) → compute the gradient → update all the weights $\mathbf{w}$ in the network. Then repeat until convergence or until other stopping criteria.

Algorithm 1: Back-propagation (the same presented in general for the LMS)

Initialize all the weights $\mathbf{w}$ in the network and η;
Compute units outputs and E_{tot};
while $E_{tot} > \epsilon$ *(desired value or other criteria)* **do**
  $\forall w \in \mathbf{w}$: $\Delta w = -\frac{\partial E_{tot}}{\partial w}$ (Step 1);
    $w^{new} \leftarrow w + \eta \Delta w + \dots$ (Step 2) ;
  Compute units outputs and E_{tot};
end

Main Back-propagation Version: stochastic, batch

Distinguish between:

1. Gradient computation → $\frac{\partial E_{tot}}{\partial \mathbf{w}}$ (we now compute this for each weight in the network)
2. Update rule for $\mathbf{w}$: Standard Back-prop, Quick-prop, R-prop, ...

We focus on (Step 1)

$$\Delta \mathbf{w} = -\frac{\partial E_{tot}}{\partial \mathbf{w}} = -\sum_p \frac{\partial E_p}{\partial \mathbf{w}} \stackrel{def.}{=} \sum_p \Delta_p \mathbf{w}$$

whereas p is p-th pattern to be fed as input to the neural network.

Then, for w_{tu} of a generic unit t, pattern p, and the input u from a generic unit u to the unit t, we have:

$$\Delta_p w_{tu} = -\frac{\partial E_p}{\partial w_{tu}} = \boxed{-\frac{\partial E_p}{\partial net_t}} \cdot \boxed{\frac{\partial net_t}{\partial w_{tu}}} = \boxed{\delta_t} \cdot \boxed{o_u}$$

(where w_{tu} connects t with the input from a generic unit u, either i or j), since

$$\begin{cases} net_t = \sum_s w_{ts} o_s \\ o_t = f_t(net_t) \end{cases} \implies \boxed{\frac{\partial net_t}{\partial w_{tu}}} = \frac{\partial \sum_s w_{ts} o_s}{\partial w_{tu}} = \boxed{o_u} \quad \text{(all 0 but for } s = u,$$

note that o_s are inputs to the unit t)

Now we expand on $\boxed{\delta_t}$ (the delta of a unit t):

$$\boxed{\delta_t} = -\frac{\partial E_p}{\partial net_t} = \boxed{-\frac{\partial E_p}{\partial o_t}} \cdot \frac{\partial o_t}{\partial net_t} \overset{o_t = f_t(net_t)}{=} \boxed{-\frac{\partial E_p}{\partial o_t}} \cdot f_t'(net_t).$$

Now recall that $E_p = \frac{1}{2}\sum_{k=1}^{K}(d_k - o_k)^2$. We distinguish 2 cases, depending on whether o_t is an output unit k (Case 1) or an hidden unit j (Case 2).

Case 1: Output Unit ($t = k$, see the exercise of previous lecture)

$$\boxed{-\frac{\partial E_p}{\partial o_k}} = -\frac{\partial \frac{1}{2}\sum_{r=1}^{K}(d_r - o_r)^2}{\partial o_k} = (d_k - o_k) \implies \boxed{\delta_k} = -\frac{\partial E_p}{\partial net_k} = \boxed{(d_k - o_k)} \cdot f_k'(net_k)$$

Case 2: Hidden Unit ($t = j$)

$$\boxed{-\frac{\partial E_p}{\partial o_j}} = \sum_{k=1}^{K} -\frac{\partial E_p}{\partial net_k} \cdot \frac{\partial net_k}{\partial o_j} = \sum_{k=1}^{K} \delta_k \cdot w_{kj}$$

since $\quad \frac{\partial net_k}{\partial o_j} = \frac{\partial \sum_s w_{ks} \cdot o_s}{\partial o_j} = w_{kj}$

$$\implies \boxed{\delta_j} = \boxed{\left(\sum_{k=1}^{K} \delta_k \cdot w_{kj}\right)} \cdot f_j'(net_j)$$

Crucial: We have **already computed** $-\frac{\partial E_p}{\partial net_k}$ before! (see the def. of delta above with $t = k$). We can back-propagate the signals δ_k to the hidden unit j!

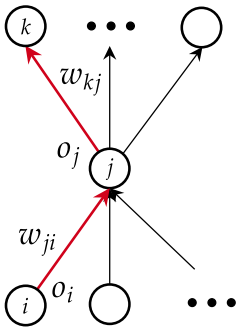


Figure 2: HINT: Use the network graph to localize the computation! (here by indices)

Notes on Case 2:

- It expresses the variation of E_p considering all the output units o_k
- Each o_k (and net_k) depends on o_j, hence we introduce a sum over k (see Fig. 2)
- This step can be obtained directly, using the def. of E_P (exercise), but in this form is more instructive (by computing the partial derivatives flow over the network, it shows the propagation over the provided network)

Summary

We derived, for two layers,

$$\Delta_p w_{tu} = \boxed{\delta_t} \cdot o_u$$

where δ_t is the error signal available to the unit t, and o_u is the input to t from a generic unit u, i.e. the input through the connection weighted by w_{tu}. In particular:

$$\Delta_p w_{kj} = \delta_k \cdot o_j \quad \text{if } t = k \text{ (output unit) and the input is from unit } j$$
$$\Delta_p w_{ji} = \delta_j \cdot o_i \quad \text{if } t = j \text{ (hidden unit), and input } i$$

The other two equations specify the error signals:

$$\boxed{\delta_k} = (d_k - o_k) \cdot f'_k(net_k) \qquad \text{if } t = k \text{ (output unit)}$$
$$\boxed{\delta_j} = \Big(\sum_{k=1}^{K} \delta_k w_{kj}\Big) \cdot f'_j(net_j) \qquad \text{if } t = j \text{ (hidden unit)}$$

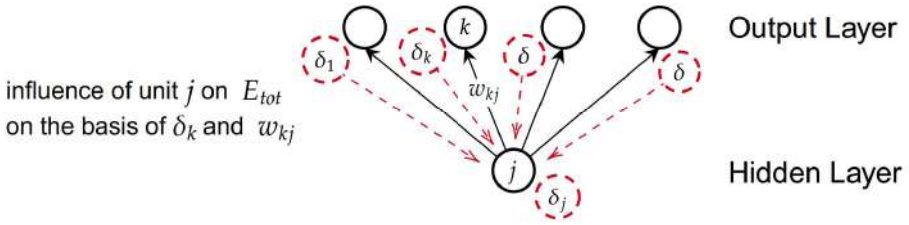


Figure 3: Retropropagation of delta (δ) values from the output layer to obtain the error signal for the hidden layers. This explains the name "Backpropagation".

This derivation can be applied (generalized) to m hidden layers! In fact, the deltas come not only from the output layer but also from any generic layer above the current one (a generic upper layer h). More in general, the deltas come from any units to which the current unit is connected to.

Hence, for each pattern p (again, omitting p):

$$w_{tu}^{new} = w_{tu} + \eta \cdot \delta_t \cdot o_u$$

where δ_t is the delta at **that** level for t and o_u is the input to the unit t from u through the connections w_{tu}. In this way we are using the $\Delta_p \mathbf{w}$ for each pattern p, and we have the on-line version; for the batch version we have to sum over the patterns (see $\Delta \mathbf{w}$ and Alg. 1.)

Further comments (and repetita):

- Summarizing the entire **training cycle** (as sketched in the Algorithm 1) we have (in order):
  - forward computation (computation of units outputs);
  - computation of errors and delta at output layer level;
  - propagation of the deltas backwards from the output layer to the hidden layers;
  - updating of the weights;
  - and we can restart again up to reach a criteria for stopping the training.
- The computation and the update must be applied also to all the **bias terms** of the units: We assumed the first input component of each unit =1, and the bias term present as w_{t0}.
- We compute only **local values**. We directly exploit only information around the unit (below or over), and not from far layers not connected to the unit, while the other information are received indirectly (deltas flow).
  Or in other words: Despite local computation, the changes of weight values have a global effect on the network. What is the influence of each w_{tu} on the total (global network) error? We have decomposed this complex effect in simpler components through the concept of the delta values local to each units.
- **Diffusion process** point of view: Backprogation as a local spatial temporal propagation from unit's parents to the unit and from the unit to unit's children (toward Hamiltonian view).
- **Efficiency**: "magic factorization" due the computation of deltas for each unit, which can be used for all the weights of that units. For all the w_{tu}, we use the same δ_t, computed just one time for all them (see the equations above) and not for each couple (t, u). Hence, we do not have a cost proportional to the square of number of weights, but to the number of the weights (check by yourself) .
  With models using bilions of weights this makes a huge difference: e.g. 10^9 versus $(10^9)^2 = 10^{18}$ operations (1 trillion, not feasible) !!!

General (didatics) Notes:

- Read the derivation at 3 levels: (i) singular math steps, (ii) interpretation of the (local) changes of a quantity with respect to the others (due to the partial derivatives) (iii) general framework of the meaning of the provided decomposition, i.e. to find delta values for each level decomposing the delta error ON THE NETWORK at hand.
- Don't say "backprop NN", NN is a model. Instead, this is a training algorithm based on the backpropagation of errors, related to the direct computing of the gradient through the network.

Suggestions:

- Repeat the derivation by yourself (don't just memorize).

Solution to the Exercise proposed in the "Back-propagation Notes"

Alessio Micheli

Direct derivation (alternative) for Case 2: hidden unit j

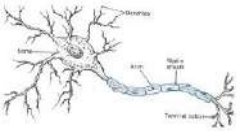
$$
\begin{aligned}
\boxed{-\frac{\partial E_p}{\partial o_j}} &= -\frac{\partial}{\partial o_j}\left\{\frac{1}{2}\sum_{k=1}^{K}(d_k - o_k)^2\right\} \\
&= -\sum_{k=1}^{K}(d_k - o_k)\cdot\frac{\partial(-o_k)}{\partial o_j} \\
&= \sum_{k=1}^{K}(d_k - o_k)\cdot\frac{\partial f_k(\mathrm{net}_k)}{\partial o_j} \\
&= \sum_{k=1}^{K}\boxed{(d_k - o_k)f_k'(\mathrm{net}_k)}\cdot\frac{\partial \mathrm{net}_k}{\partial o_j} \\
&= \sum_{k=1}^{K}\boxed{\delta_k}\, w_{kj}
\end{aligned}
$$

where $\boxed{\delta_k}$ is that of Case 1 in the "Back-propagation Notes".

Course structure: zoom in the *Neural Networks* box

NEW

Dip. Informatica
University of Pisa



1 unit NN

Perceptron

Adaline

LMS

As a linear model (previous lects)

Neural Networks

***MLP**: Mack. &Pits*

***LEARNING:** Perceptron learning alg + Convergence theorem*

Non-linear out (f)

MLP: feed-forward NN

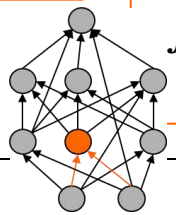
LEARNING: Backprop

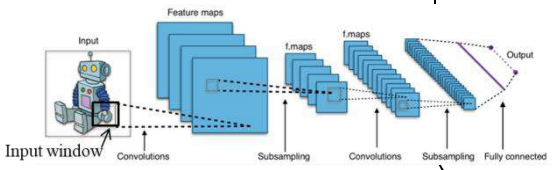
CasCor

Heuristics for BP

Regularization

We are here





Applications: CNN example

Introduction to recent NN paradigms (deep, random.)

After the Validation and SVM (to favorite the PRJ)

Issues in Training NN

Dip. Informatica
University of Pisa

- Heuristic guidelines in setting the back-prop training

General problems:

- The model is often over-parametrized
- Optimization problem is not convex and potentially unstable
- We discuss few of them (see e.g. Haykin for details)
  - **Hyperparameters**: Starting values, on-line/batch, learning rate
  - Multiple Minima
  - Stopping criteria
  - Overfitting and regularization
  - **Hyperparameters**: Number of hidden units
  - Input scaling/ Output representation
  - ……..
- A more specific discussion for DL is in the DL book (we will see later)

Hyper-parameters= values that **you** have to set-up to run the training

Philosophy

Dip. Informatica
University of Pisa

- We don't like to introduce too much heuristic details
  - Historically: From a collection of heuristic methods (often collections of tricks originating from the contributing fields of statistics, neural networks, fuzzy logics, …) to the construction of general conceptual frameworks …
- They can be experimental investigated in the ***project*** !
- Some of them have a *theoretical ground* (guess which one!) and most of them can be applied in a more general framework (e.g. regularization)





- *Some are very common & must be used (in the prj) see the symbol* → **(!)**
- Remember also the symbol for tech arguments / *important topics, fast in class, because* not strictly need at a first glance, but useful for your successive reading (e.g. because they are easier to be understood *later*, or useful to develop the *project*, etc)





- *Avoid rules of thumb from blogs (unless proved in scientific papers or with an improvement empirically assessed by yourself)*

Nice interpretation

Dip. Informatica
University of Pisa

- Back-prop as a **path** trough the loss/weight space.
- The path depends on:
  - The data (assumed already given)
  - The NN model
  - The starting point (initial weight values)
  - The rate of convergence
  - The final point (stopping rule)

- They define a control for the *search* over the Hypothesis space

The Basic Alg. (AGAIN!!!)



Dip. Informatica
University of Pisa

Gradient descent: Given training data in the form *(x,d)*

1) Start with weight vector $\boldsymbol{w}_{\text{initial}}$ (small), fix *eta (0<eta<1)*.
2) Compute $\Delta\boldsymbol{w}$= -"gradient of $E(\boldsymbol{w})$" = $-\frac{\partial E(\boldsymbol{w})}{\partial \boldsymbol{w}}$ (or for each w_j)
3) Compute $\boldsymbol{w}_{\text{new}} = \boldsymbol{w} + eta*\Delta\boldsymbol{w}$ (or for each w_j) [upgrade weights]

   where *eta* is the "step size" (learning rate) parameter
4) Repeat (2) until convergence or $E(\boldsymbol{w})$ is "sufficiently small"

- But now $\Delta\boldsymbol{w} = -\frac{\partial E(\boldsymbol{w})}{\partial \boldsymbol{w}}$ were obtained through **backpropagation** derivation/algorithm for any weights in the network
  - At step2) to compute the Error we first apply inputs to the network computing a **forward phase** up to the outputs computation
  - then we start the **backward phase** to retro-propagate the deltas for the gradient
- See "back-prop" algorithm in (the last page of) the Back-prop pdf file
  - computing the delta for each unit in the network from the output layer to the hidden layers and then the Δw for each weight,
  - and summing over the patterns: we will see how soon!

The Basic Alg. (AGAIN!!!)

Dip. Informatica
University of Pisa

Gradient descent: Given training data in the form (x,d)

1) Start with weight vector $\boldsymbol{w}_{initial}$ (small), fix *eta* (*0<eta<1*).
2) Compute $\Delta \boldsymbol{w}$= -"gradient of $E(\boldsymbol{w})$" = $-\frac{\partial E(\boldsymbol{w})}{\partial \boldsymbol{w}}$ (or for each w_j)
3) Compute $\boldsymbol{w}_{new} = \boldsymbol{w} + eta * \Delta \boldsymbol{w}$ (or for each w_j) [upgrade weights]

   where *eta* is the "step size" (learning rate) parameter
4) Repeat (2) until convergence or $E(\boldsymbol{w})$ is "sufficiently small"

E.g. How to choice for the [ ] and other aspects of the training algorithm for MLP by back-prop?

Issues in Training NN

Dip. Informatica
University of Pisa

- Heuristic guidelines in setting the back-prop training

General problems:

- The model is often over-parametrized
- Optimization problem is not convex and potentially unstable
- We discuss few of them (see e.g. Haykin for details)
  - **Hyperparameters**: Starting values, on-line/batch, learning rate
  - Multiple Minima
  - Stopping criteria
  - Overfitting and regularization
  - **Hyperparameters**: Number of hidden units
  - Input scaling/ Output representation
  - ……..
- A more specific discussion for DL is in the DL book (we will see later)

Hyper-parameters= values that **you** have to set-up to run the training

Issues in Training NN

Dip. Informatica
University of Pisa

- Heuristic guidelines in setting the back-prop training

General problems:

- The model is often over-parametrized
- Optimization problem is not convex and potentially unstable
- We discuss few of them (see e.g. Haykin for details)
  - **Hyperparameters**: Starting values, on-line/batch, learning rate
  - Multiple Minima
  - Stopping criteria
  - Overfitting and regularization
  - **Hyperparameters**: Number of hidden units
  - Input scaling/ Output representation
  - ……..
- A more specific discussion for DL is in the DL book (we will see later)

Starting values ($w_{initial}$ in the basic alg.)

Dip. Informatica
University of Pisa

- **(!)** Initialize weights by random values near zero
  - Avoid for ***W***: all zero, high values (e.g. saturation issue), or all equals (symmetry): these can hamper the training
  - E.g. in the range [-0.7,+0.7] (for standardized data, see later)
- Also: considering the Fan-in : e.g. range * 2/fan-in
  - Not if fan-in is too large
  - Not for output units (else deltas start close to zero !)
- Other heuristics …orthogonal matrix, …. (enjoy if you like)

  Very popular among practitioners: Glorot, Bengio AISTATS 2010:
  - **Basic**: 0 for biases and w random from Uniform distribution in [-1/*a*, +1/*a*] with *a=sqrt(fan-in)* or (later results) *a* depending also on fan-out
  - Glorot, Bengio (**New proposed**): see inside the paper.

Fan-in is the number of inputs to a (hidden) unit.



- **(!)** Note (practical for the first part of the project): a very small NN (very few units) can have a stronger dependencies w.r.t. initialization

Issues in Training NN

Dip. Informatica
University of Pisa

- Heuristic guidelines in setting the back-prop training

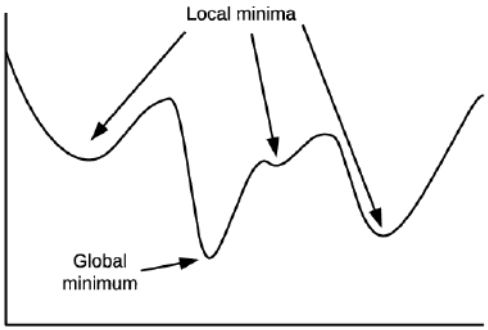
General problems:

- The model is often over-parametrized
- Optimization problem is not convex and potentially unstable
- We discuss few of them (see e.g. Haykin for details)
  - **Hyperparameters**: Starting values, on-line/batch, learning rate
  - Multiple Minima
  - Stopping criteria
  - Overfitting and regularization
  - **Hyperparameters**: Number of hidden units
  - Input scaling/ Output representation
  - ……..
- A more specific discussion for DL is in the DL book (we will see later)

Multiple Minima

Dip. Informatica
University of Pisa

- Loss is *not convex*: many *local minima*;
  Effects:



- The result depends on starting weight values, hence:
- **(!)** Try a number of random starting configurations (e.g. **5-10** or more training runs or **trials**)
- Useful for the project: Take the **mean results** (mean of errors) and look to **variance** to evaluate your model and then, if you like to have only 1 response:
  - you can choose the solution giving lowest (penalized) validation error (or the median)
  - or you can take advantage of different end points by an average (committee) response: mean of the outputs/voting
    - A lecture of the course will explain why committee/ensembling approaches can be advantageous in general (later)



- Local minima stopping training with too high error is not really a big issue:
  - you can see the final training error (and restart if you are not happy)
  - a "good" local minima is often sufficient (see the next slide)

But is global minima needed?

Dip. Informatica
University of Pisa

Two general observations:

1. Indeed, in ML we don't need neither local or global minima on R_{emp} as we are searching the min of R (that we cannot compute)
   - Often we stop * in a point that has even *not* null gradient and hence it is also neither a local or global minima for the empirical (training) error.
     **(we will see later how)*

2. The NN build a *variable size hp space* (see at the end of NN-first part slides) → it increases VC-dim during training → the training error decreases toward zero (or global minimum) while the NN becomes too complex (high VC-dim) [we have a major problem to avoid it]

- Stopping before this condition of *overtraining* (and hence overfitting) is needed
  (independently from the issue on local/global minima)

Issues in Training NN

Dip. Informatica
University of Pisa

- Heuristic guidelines in setting the back-prop training

General problems:

- The model is often over-parametrized
- Optimization problem is not convex and potentially unstable
- We discuss few of them (see e.g. Haykin for details)
  - **Hyperparameters**: Starting values, on-line/batch, learning rate
  - Multiple Minima
  - Stopping criteria
  - Overfitting and regularization
  - **Hyperparameters**: Number of hidden units
  - Input scaling/ Output representation
  - ……..
- A more specific discussion for DL is in the DL book (we will see later)

On-line/Batch





Dip. Informatica
University of Pisa

- See batch/on-line in "Gradient descent algorithm" slide, lecture on linear models!!!

Repetita

- For **Batch** version: sum up all the gradients of each pattern over an epoch and then update the weights (Δw after each "epoch" of l patterns)
  - Memento: *"Epoch":* an entire cycle of training patterns presentation (all the training examples)

- **Stochastic/on-line** version: upgrade w for each pattern p (by $\Delta_p w$) without wait the total sum over l:
  - It makes progress with each examples it looks at: it can be faster, but it need smaller eta.

- Let see the two algorithms in details:

On-line/Batch Algorithms

Dip. Informatica
University of Pisa

- See batch/on-line in "Gradient descent algorithm" slide, lecture on linear models!!!
- Steps 2 and 3 change to express an *inner* or *external* cycle on the patterns:

Batch (as before)

1) Fix $\boldsymbol{w}_{\text{initial}}$ and *eta*
2) Compute $\Delta\boldsymbol{w}$= -"gradient of $E(\boldsymbol{w})$" on the entire Training set (a cycle, epoch, i.e. a sum, on p is inside this step)
3) Compute $\boldsymbol{w}_{\text{new}} = \boldsymbol{w} + eta*\Delta\boldsymbol{w}$
4) Repeat until convergence

On-line

1) Fix $\boldsymbol{w}_{\text{initial}}$ and *eta*
For each pattern p:
2) Compute $\Delta_p\boldsymbol{w}$ = -"gradient of $E_p(\boldsymbol{w})$"
3) Compute $\boldsymbol{w}_{\text{new}} = \boldsymbol{w} + eta* \Delta_p\boldsymbol{w}$
4) Repeat until convergence* consider also epoch wise repetition

$$-\frac{\partial E(\mathbf{w})}{\partial w_{tu}} = -\sum_{p=1}^{l}\frac{\partial E_p(\mathbf{w})}{\partial w_{tu}} = \sum_{p=1}^{l}\delta_{p,t}\ o_{p,u} \qquad -\frac{\partial E_p(\mathbf{w})}{\partial w_{tu}} = \delta_{p,t}o_{p,u}$$

p= pattern, $\delta_{p,t}$: delta of unit t for pattern p

$o_{p,u}$: component u, computed for the input pattern p

See later to add the *l* (Least **Mean** Square)

On-line/Batch
First Comments

Dip. Informatica
University of Pisa

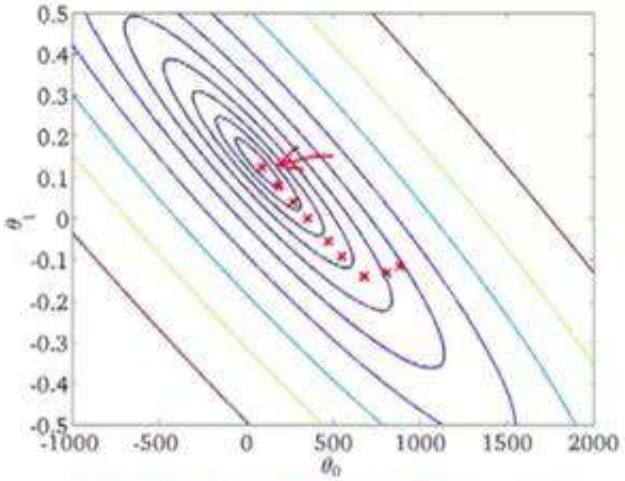
- **Batch** version: more accurate estimation of gradient
- **On-line or Stochastic** version: since the gradient for a single data point can be considered a noisy approximation to the overall gradient, this is also called **stochastic** (noisy) gradient descent.
  Zig-zag (stochastic) descent *(see figures in the next slide)*
  - It can help avoiding local minima
  - A different order of patterns in different epochs should be used (to avoid a bias/drift in the gradient descending)
  - **(!)** Usually random order (*shuffling* the patterns: i.e. randomly mixing their order in each epoch)

- Many **variations** exist: for instance
  - Stochastic Gradient Descent (SGD) using *minibatch – mb* (few training examples) for each step of gradient descent
- → see a next slide for *mb* with further comparisons

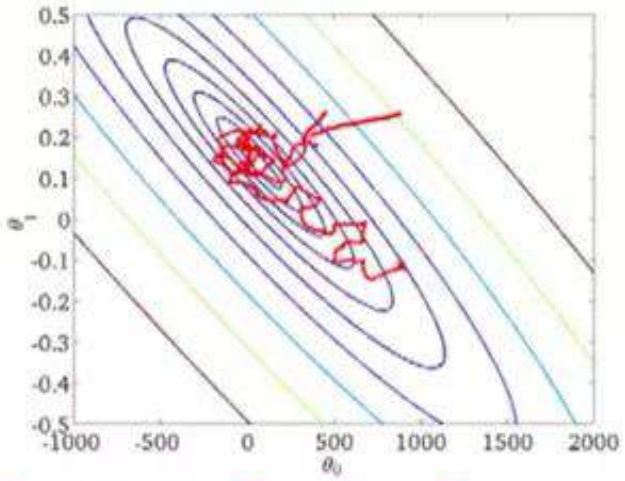
Plots stochastics vs batch

(we have already seen them)

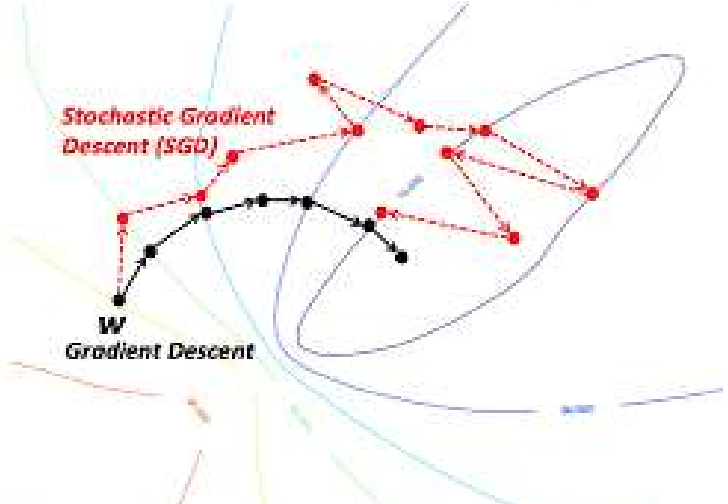
Dip. Informatica
University of Pisa



Batch Gradient Descent



Stochastic Gradient Descent



Here a case for SGD with high learning parameter

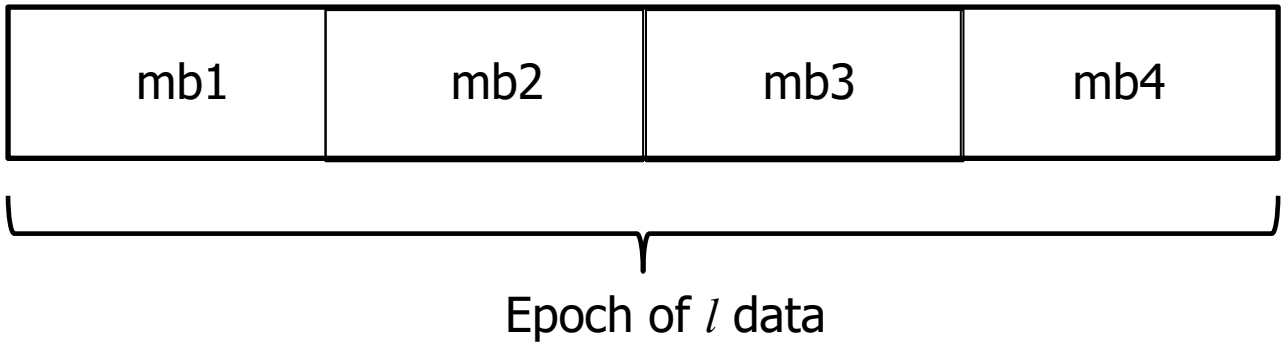
The Mini-Batch

Dip. Informatica
University of Pisa

In the Mini-Batch we divide each *epoch* in parts

- We sum the gradient up to ***mb*** patterns before to update the weights
- And we repeat the cycle until all the training data have been used (for an *epoch*).

- So the weight update is every **mini-batch** instead then of singular pattern (on-line) or an entire epoch (batch)



Mini-batch SGD

Dip. Informatica
University of Pisa

Mini-batch (with *mb* examples) to estimate the gradient

$$1 \leqslant mb \leqslant l$$

Stochastic/on-line:
- cannot exploit parallel processing of examples
- may be unstable
+ can have a regularization effect (adding noise to the learning process*)

e.g. 100

Batch (full training set)
+ more accurate gradient estimation
- but slower! (wait all the data)

+ the parallelism can be compatible with e.g. a GPU memory

Minibatch stochastic method → aka ***SGD***

- Use random sampling to implement the minibatch (unbiased estimation). By *shuffling* (at the beginning for all the TR data or for each mini batch)
- With very large TR set or *online streaming* → even no epochs!
- Along with the advantage of gradient descent (linear cost wrt examples) SGD with *mb* is a key factor in extending learning to very large data sets (over millions of examples) because *mb* (and hence the cost of each SGD update) does not depend on *l : typically just mini-batch and (again) no epochs repetition*

Issues in Training NN

Dip. Informatica
University of Pisa

- Heuristic guidelines in setting the back-prop training

General problems:

- The model is often over-parametrized
- Optimization problem is not convex and potentially unstable
- We discuss few of them (see e.g. Haykin for details)
  - **Hyperparameters**: Starting values, on-line/batch, learning rate
  - Multiple Minima
  - Stopping criteria
  - Overfitting and regularization
  - **Hyperparameters**: Number of hidden units
  - Input scaling/ Output representation
  - ……..

And also
- Momentum
- Speed-up?

- A more specific discussion for DL is in the DL book (we will see later)

Learning rate – eta (η) (also w.r.t batch / on line)

Dip. Informatica
University of Pisa

Recap

- **Batch**: more accurate estimation of gradient→higher eta (η) value
- **On-line:** it can make the training faster but can be more unstable → smaller eta (η) value

More in general (on the eta values):

- High versus low eta values : fast (but possible unstable if too large) versus stable (but possible too slow if too small)

- And we will see various techniques to improve the convergence speed
- Anyway, do you still ask for some values ?!? **Typically**, search in the range [0.01-0.5] in LMS (see also later)

Practical hints on eta values and training behavior

NEW

Dip. Informatica
University of Pisa

- Learning curve, plotting errors during training, it allows you to check the behavior in the early phases of the model design for your task at hand (preliminary trials)
  - Please check the Learning Curve while training a NN!

(!)

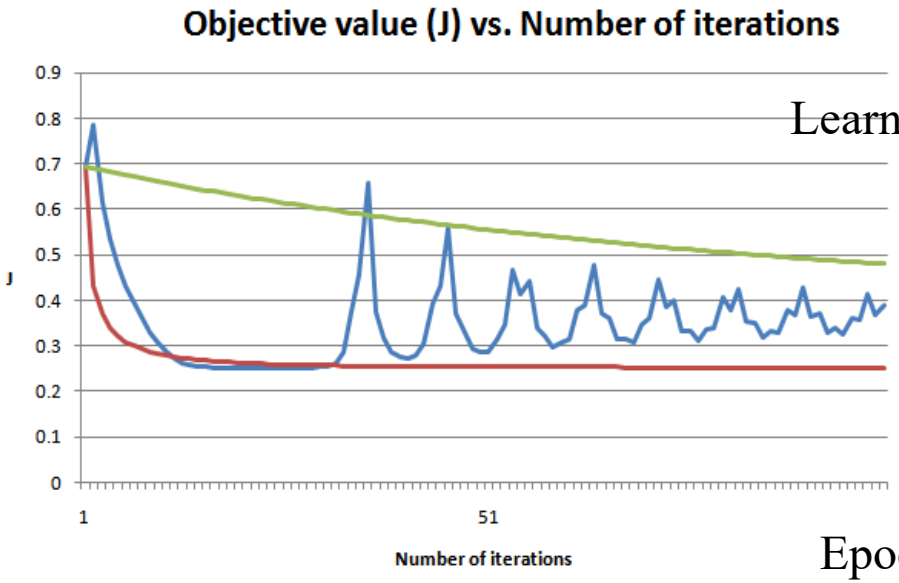
- We are now looking to the **curve quality** and **convergence speed**: of course, the absolute value for the training error depends also on the *model capacity* (see later for the number of units, regularization etc.) and the other hyperparameters

Look to the Learning Curve (repetita)

Dip. Informatica
University of Pisa

Which curve do you prefer?

Error



Learning curves

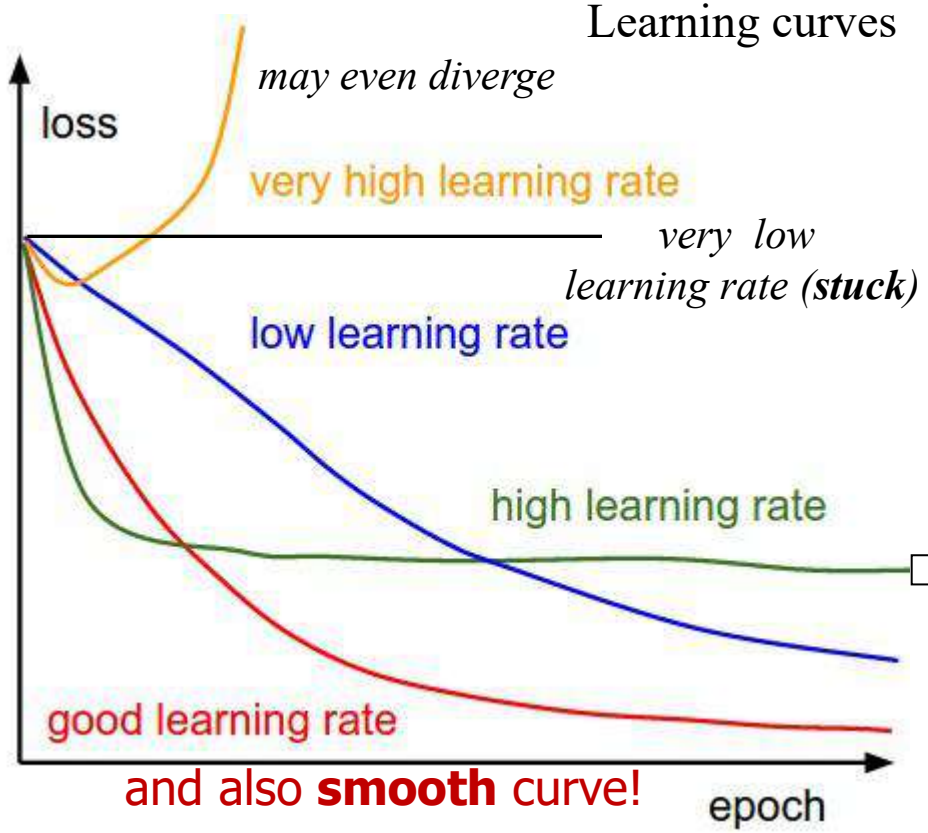
Epochs

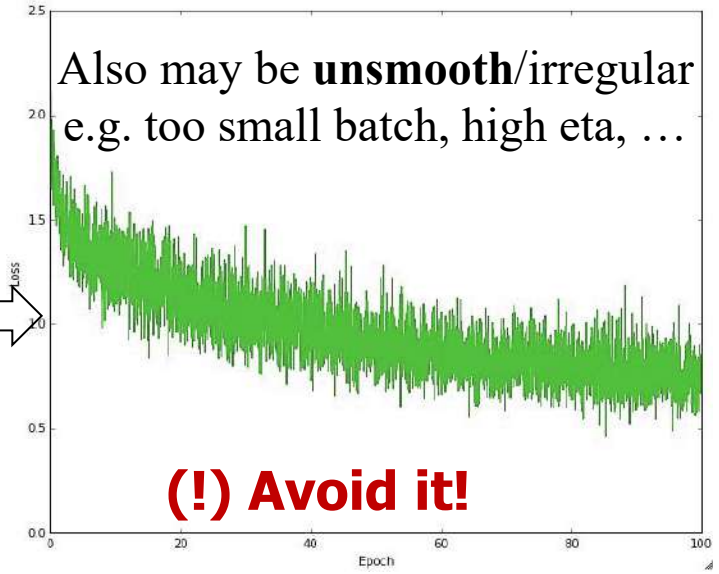
These are learning curves:
They show how the error decreases through gradient descent iterations

It means that you can decide the best approach by looking to this plot

Examples with eta (η)

Dip. Informatica
University of Pisa





Here just looking to the optimization "failures" (on the TR set), not the generalization errors

Learning rate – eta (η): Least Mean Square

Dip. Informatica
University of Pisa

- Practical approach: it is useful to use of the mean (average) of the gradients over the epoch: it helps to have an "uniform" approach with respect to the number of input data (*Sum of gradients/* l). Hence:

(!) – Repetita from "Gradient descent algorithm": ***dividing by*** l ***correspond to have the Least Mean Squares***

– *Note that it is equivalent to have:* η/l

NOTE:
$0 < \eta < 1$
when we use LMS

(!) Note that if you compare on-line with batch using LMS (i.e. $/l$), then using **on-line** training will require much smaller eta *to be comparable*!!!!
(in order of the same factor /l used to divide the gradient in the batch version)

(!) For **mini batch** (if you divide by *mb* in each mini-batch) you have to tune eta as well (to obtain again an eta proportional to the batch case)
or simply divide eta by l also for mini-batch (exercise)

Learning rate - Improvements

Dip. Informatica
University of Pisa

In the following slides (**guide**):

1. **(!)** Momentum
   - Nesterov Momentum
2. Variable learning rate (initially high then decrease)
3. Adaptive learning rates (change during training and for each *w*)
   (see also later in Deep Learning lectures)
4. In NN with many layers (see deep learning lectures):
   - It can be varied depending on the layer (higher for deep layers) or fan-in (higher for few input connections)

1. Momentum (!)

Dip. Informatica
University of Pisa

$$\Delta w_{new} = -\eta \frac{\partial E(\boldsymbol{w})}{\partial w}$$

Without momentum

(we now include η in the Δw)

$$\Delta w_{new} = -\eta \frac{\partial E(\boldsymbol{w})}{\partial w} + \alpha \Delta w_{old}$$

With momentum

$$\boldsymbol{w}_{new} = \boldsymbol{w} + \Delta \boldsymbol{w}_{new}$$

Here η is in $\Delta \boldsymbol{w}$

After you can save Δw_{new} for ther next step ($\Delta w_{old} = \Delta\, w_{new}$)

$0 <$ momentum parameter Alfa $(\alpha) < 1$ e.g [0.5 - 0.9]

«Heavy Ball method»:

- Faster in plateaus:

same sign of grad. for consecutive iterations → momentum increase the delta

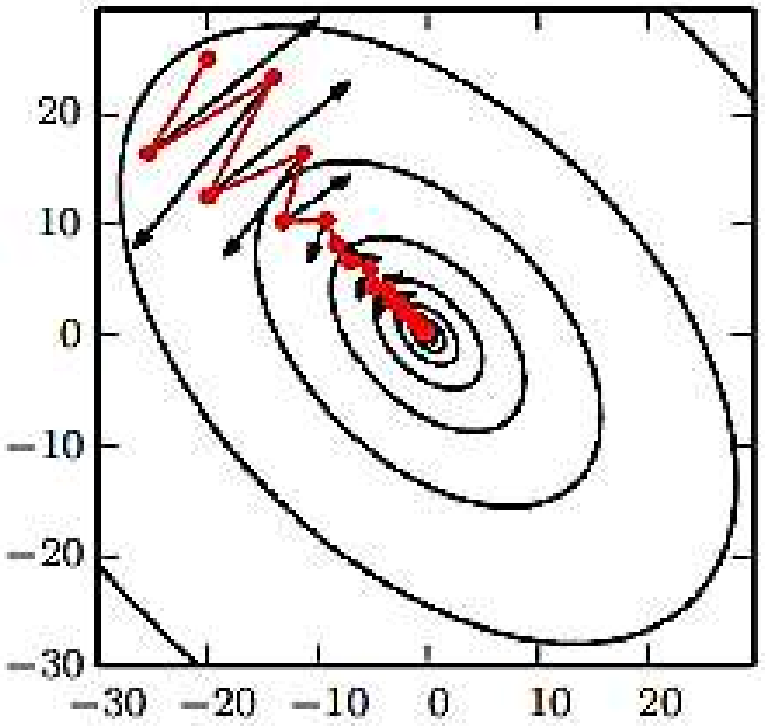
- Damps oscillations: stabilizing effect in directions that oscillate in sign
- *Inertia effect*: Allow us to use higher eta η (learning parameter: e.g. 0.5-0.9)

- NOTE: Haykin Vol 3 eq. 4.47 "$\alpha\ w_{old}$" should be "$\alpha\ \Delta w_{old}$"

1. Momentum: Plot

Dip. Informatica
University of Pisa

- Effect on a canyon of the error surface (# poor conditioning of the Hessian matrix).
- In **red** the path with the momentum, lengthwise traversing the valley toward the minimum …
- instead of zig-zag along the canyon walls (following the **black** directions given by the gradient)



It shows the gradient for each red point descending with the moment

1. Momentum (II)





Dip. Informatica
University of Pisa

- Momentum born with "**batch**" learning, and it is commonly assumed that helps more in batch mode
- In the on-line version can be used, in this case:
  - It considers the Δw_{old} as the Δw_{p-1} i.e. of the *previous example*
- And so, it is connected to a *moving average of the past gradients* concept to smooth out the stochastic gradient samples (*inside the epoch*)

  [e.g. see it in the form $\overline{grad} = \alpha\ \overline{grad} + (1-\alpha)\ grad_{now}$]
  - It smooths the gradient over *different examples* (a different meaning w.r.t. the batch version that uses the delta computed for same batch of examples, among epochs)
  - You can use it multiplying by eta (as usual) to obtain the $\Delta \boldsymbol{w}$
  - See if really needed (as any other heuristic)

- For Minibatch: again, moving average of past gradients over the progressive minibatchs (that have different examples, not the step before for the same examples as for batch).

1. Nesterov Momentum



Dip. Informatica
University of Pisa

A variant:

- Evaluate the gradient after the momentum is applied
  - First apply the momentum ($\underline{\boldsymbol{w}} = \boldsymbol{w} + \alpha \Delta \boldsymbol{w}_{old}$)
  - Evaluate the *new gradient* at this interim point (i.e. with such $\underline{\boldsymbol{w}}$)
  - Compute and apply the DeltaW (to the original $\boldsymbol{w}$) as before (summing the momentum and the *new gradient*)

- Shown to improve the rate of convergence *for the batch mode* (not for the stochastic case!)
- Inspired by Nesterov's accelerated gradient (with nice formal properties in the convex case)*

- See DL book 8.3.3, popular in SW libraries such is *Caffe* (DL software)

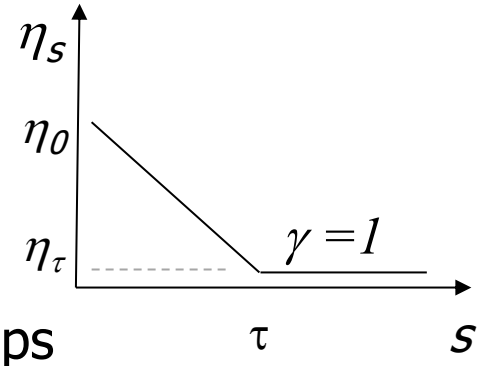
2. Variable Learning rate and Mini Batch



Dip. Informatica
University of Pisa

- Using mini-batch → the gradient does not decrease to zero close to a minimum (as the exact gradient can do)
- Hence fixed learning rate should be avoided:
- For instance, we can **decay linearly** eta for each step until iteration tau (τ), using $\gamma = (step\ s)/\tau$:

$$\eta_s = (1-\gamma)\eta_0 + \gamma\eta_\tau$$



- Then stop and use fix (small) η ($=\eta_\tau$)
- Set-up e.g.: η_τ as ~1% of η_0, τ few hundred steps
- And η_0 *?* Again: no instability/ no stuck trade-off (preliminary trials or grid-search)

- See Section 8.3.1 DL book

3. Adaptive Learning Rates



Dip. Informatica
University of Pisa

- Automatically adapt the learning rate during training
  possibly avoiding/reducing the fine-tuning phase via hyperparameters selection
- Also, they can include even the usage of a separate eta for each parameter of the network

E.g. (popular):

- AdaGrad
- RMSProp (becoming quite popular for DL, e.g. caffe SW)
- Adam (also very popular, often robust with the *default value,* but warning!)
- … *continuously evolving*



- Combined with moment etc. (many variants*)

- Popular for DL: see DL book 8.5 if interested

Issue

Is the good old basic SGD still the best approach to NN training?

Slow ?

Dip. Informatica
University of Pisa

- As any iterative approaches…. Basic SGD is generally efficient (linear in the number of free parameters)
  but depends on the number of training data and on the number of epochs

- Lots of other heuristics approaches (in the NN literature):



  - *Quick-prop*: go to to the minima of the local estimated convex function (assumed as a parabolic curve) (see Bibliography @ end of slides)
  - *R-prop* does not use the value of the gradient (it can vanish for deep layers) but the sign of gradient (see Bibliography @ end of slides)
  - Deep Learning need special care (see later)

- Second-order techniques, or other from CM/O4DS? See next slide!

Second order methods

Dip. Informatica
University of Pisa

Use the *second order derivatives* (Hessian matrix): more information about the curvature of the objective function → better descent (*Newton's methods)*

- But the Hessian can be too large (i.e. computationally expensive), and see the next "saddle points" slide

In the form of *approximate* second order methods we find:

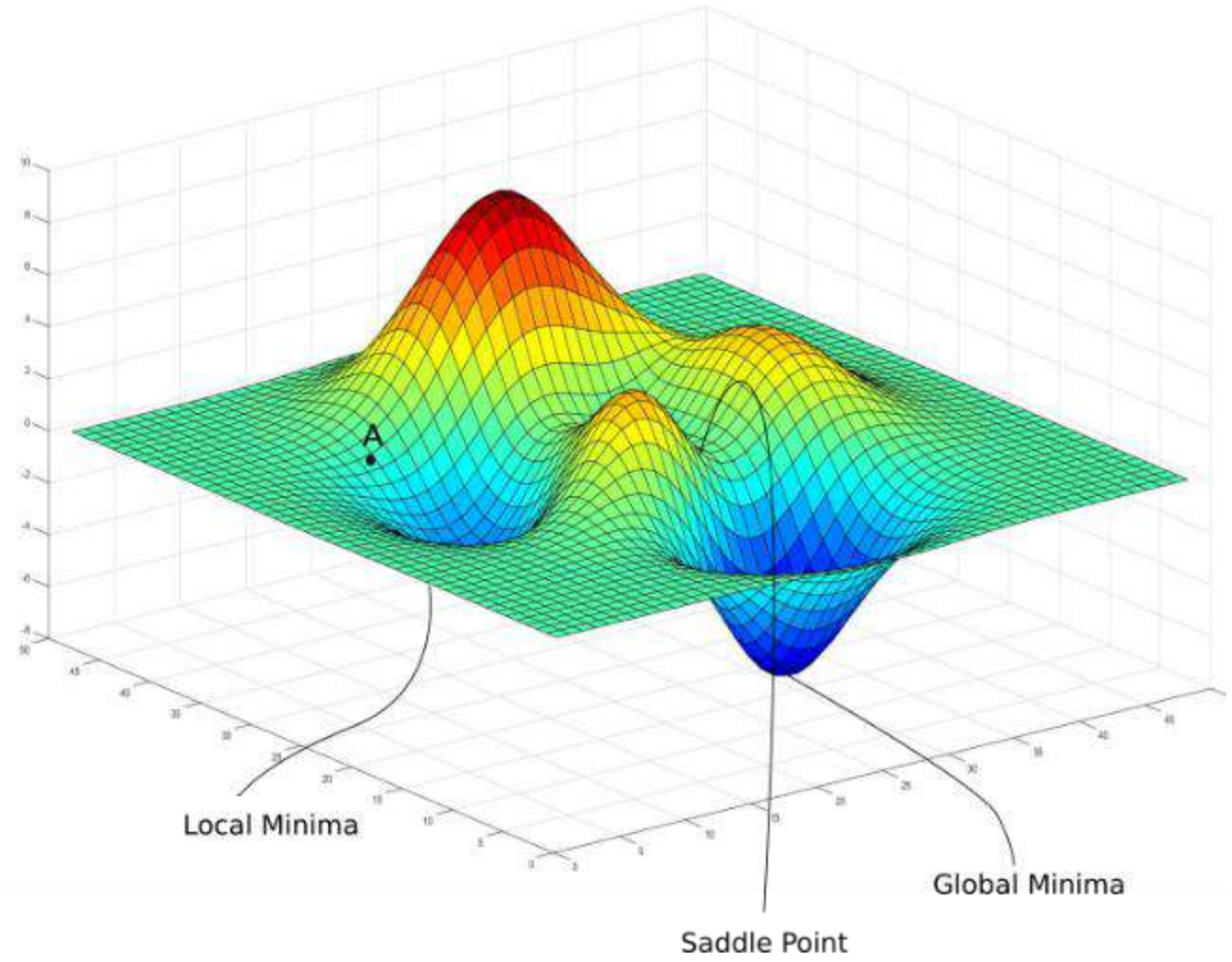
- *Approx. Newton's methods* (also *Gauss-Newton* and *Levenberg-Marquardt* approximation)
- Hessian free approaches (without computing H and H^{-1})
  - *Conjugate gradients*
    - can be nice for faster converge! (see e.g. in "Numerical Recipes in C").
  - *Quasi-Newton methods: BFGS* (Broyden–Fletcher–Goldfarb–Shanno Algorithm)
- ***...***

- References: Haykin book, DL book

- Is possible to try them within a CM/ML prj? Yes, especially with ad hoc variations for NNs

Minima and Saddle Points

Dip. Informatica
University of Pisa



Means technical aspect: Not needed at the beginning

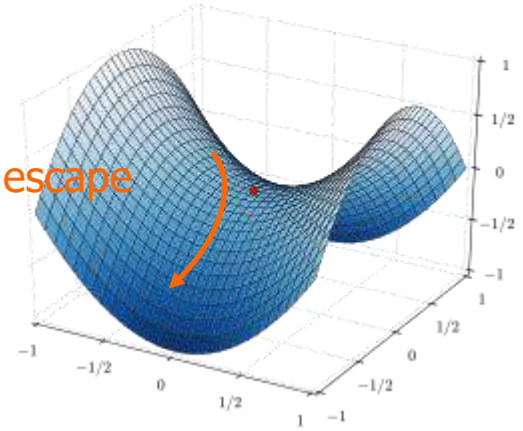
Saddle points #tech



NEW

Dip. Informatica
University of Pisa

More intriguing discussion on saddle points (see e.g. 8.2 Deep Learning book) i.e. Gradient (close to) zero but not a local minima



- Saddle points can grow exponentially (wrt to local minima) in high dimensional spaces
- Newton search for (jump to) gradient zero points – more sensible to find saddle points
- This can explain why (standard) *second-order methods* (looking to Hessian, e.g. *Newton's method*) have **not succeed** in replacing gradient descent for NN training
- Instead, gradient descent empirically shows to escape saddle points […]
- But the length of the training path can be high for many other reason not related to local minima or saddle points (small gradient values), such as time to "circumnavigate" local maxima and so on…
- But such difficult cases are not frequent! Why? Research is on-going…!
- *Typically, we are able to find a low value of the loss quickly enough to be useful!* And especially useful to generalize! (we don't need a minimum at all, see previous slides on multiple/global minima!).

In practice?

Dip. Informatica
University of Pisa

- For optimization: Don't worry, the main basics approaches e.g. with **(!)**, often are enough *if you apply them correctly*
  - The **experience** with well-know techniques have a major role than sophisticated approaches *that you cannot manage*.
  - The optimization technique is relevant only if there are obstacles for training. Validation will have a major role than the optimization on the training set (we will have next lectures on this): remember, optimize on the training data is not our purpose!
  - The others are an opportunity to explore different approaches (especially using libraries)
- SGD with momentum (ad we will see regularization) is in any case a **starting point**, also as comparison if you move toward other approaches
  - You have to start from this, also for didactic reasons!

Issues in Training NN

Dip. Informatica
University of Pisa

- Heuristic guidelines in setting the back-prop training

General problems:

- The model is often over-parametrized
- Optimization problem is not convex and potentially unstable
- We discuss few of them (see e.g. Haykin for details)
  - **Hyperparameters**: Starting values, on-line/batch, learning rate
  - Multiple Minima
  - Stopping criteria
  - Overfitting and regularization
  - **Hyperparameters**: Number of hidden units
  - Input scaling/ Output representation
  - ……..
- A more specific discussion for DL is in the DL book (we will see later)

Stopping criteria

Dip. Informatica
University of Pisa

- The basic is the used error (*mean error<E*): it is the best if you know the **tolerance of data (expert knowledge),** but we often have not this info.
  - Also: Max (instead of mean) tolerance on data
  - For classification: Number of misclassified (error rate)
- But we often need to use "internal criteria"
  - No more relevant ***weight changes*** / ***near zero gradient*** (Euclidian norm of gradient < epsilon)/ No more significant ***error decreasing*** for a epoch (e.g. less than 0.1%):

    NOTE: may be premature (e.g. if eta is too small)! It can be applied observing *k* epochs (know as *patient*).
- **(!)** In any case stop after an excessive number of epochs (just to escape from too slow convergence),
  - but *avoid* to stop fixing an arbitrary number of epochs (*)
- *and not necessarily stop with very low training error*...(see next)



  - (*) We will discuss soon the "early stopping" and also later FAQ and typical errors for stopping (discussing the validation issues, in validation lectures)

Issues in Training NN

Dip. Informatica
University of Pisa

- Heuristic guidelines in setting the back-prop training

General problems:

- The model is often over-parametrized
- Optimization problem is not convex and potentially unstable
- We discuss few of them (see e.g. Haykin for details)
  - **Hyperparameters**: Starting values, on-line/batch, learning rate
  - Multiple Minima
  - Stopping criteria
  - Overfitting and regularization **Much more important!**
  - **Hyperparameters**: Number of hidden units
  - Input scaling/ Output representation
  - ……..
- A more specific discussion for DL is in the DL book (we will see later)

Stopping at the minimum?

Dip. Informatica
University of Pisa

- Typically we don't want the global minimizer of $R_{emp}(w)$, as this is likely to be an overfitting solution
- This is also a difference w.r.t. optimization (CM) methods
  (see the previous slide <But is global minima needed?>)

- The *control of complexity* is our main aim to achieve the best generalization capability
- For instance we need to add some *regularization*:
  - this is achieved directly through a penalty term, or indirectly by early stopping.
  - plus model selection (cross-validation) on empirical data to find the trade-off

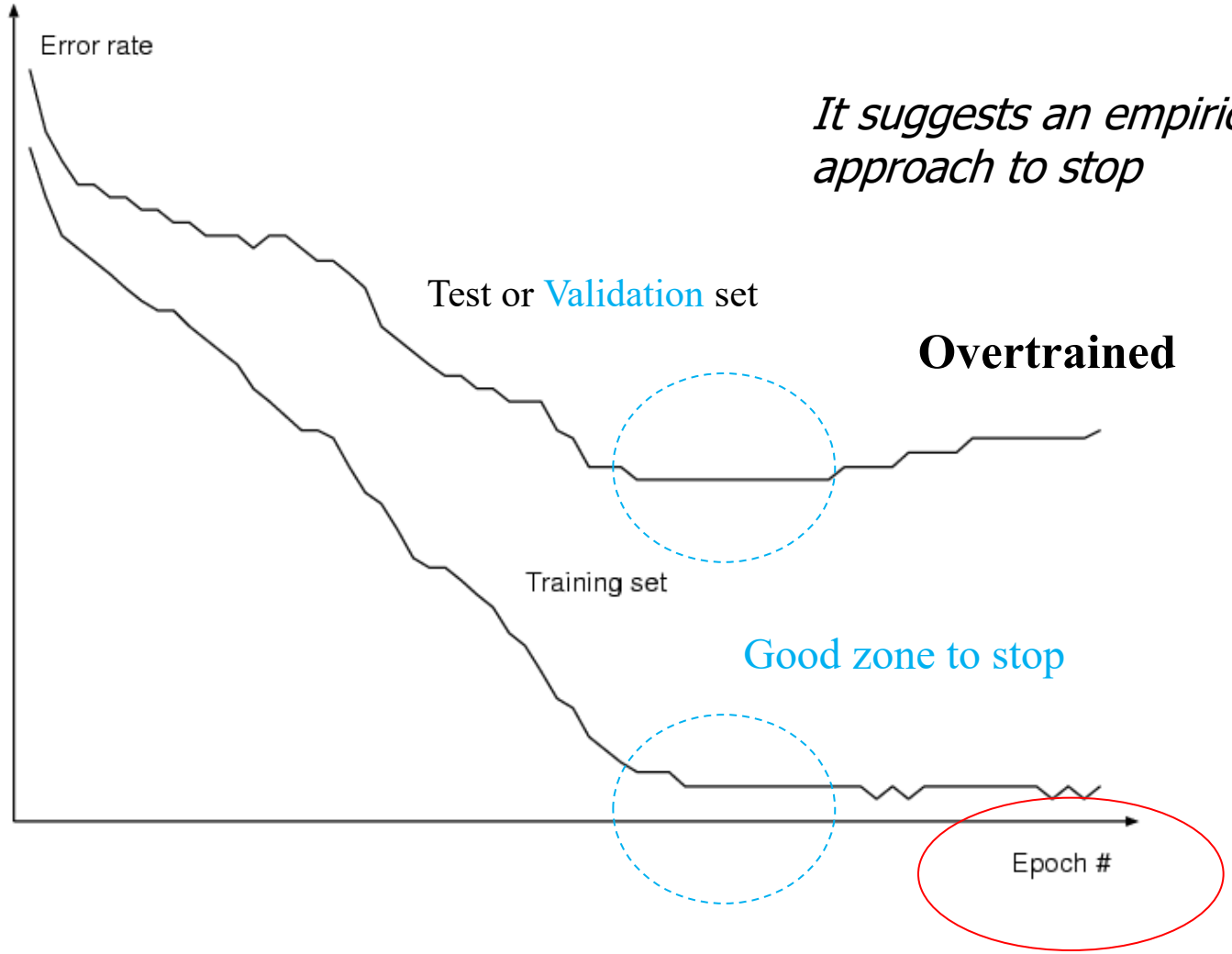
Overfitting in NN

Dip. Informatica
University of Pisa

- Start learning with small random weights (symmetry breaking)
- The mapping input→output is nearly linear:
  - *number of effective* free parameters (and VC-dim) nearly as in perceptron
- As optimization proceeds hidden units tend to *saturate*, increasing the *effective number of free parameters* (and introduce non-linearities) and hence increase *VC-dim*
- As we discussed, this is a *variable-size hypothesis space* (it changes during training)

Typical behavior of backprop: **learning curve**

Dip. Informatica
University of Pisa



Learning curve analysis

Dip. Informatica
University of Pisa

Recall the VC-bound plot ! (theoretical ground)

How to act?

1. *Early stopping*: use a **validation set** for determining when to stop: at the *validation error increasing*: **...** quite vague*!
   - * Use more than one epoch before estimating (called "*patience*")
   - Backtracking approach
   - Note that since *effective* numb. of parameters grows during the course of training, halting training corresponds to limit the effective complexity
   - See "Early Stopping — But When?» L. Prechelt, LNCS 2012



   - We will discuss also later about *Early Stopping (FAQ & typical errors)*
2. *Regularization* (on the loss: our main warhorse!)
3. *Pruning methods*: see later

2. Regularization (!)

Important !

Dip. Informatica
University of Pisa

- Training → weight increases: We can optimize the loss considering the weight values
- A well know principled approach (related to **Tikhonov theory,** again):
- Add a penalty term to the error function:

$$Loss(\boldsymbol{w}) = \sum_p (d_p - o(\boldsymbol{x}_p))^2 + \lambda \| \boldsymbol{w} \|^2$$

Loss

"Error" term [(M)SE]

Regularization/penalty term

A sum over *all* the weights in the network

$$\sum w_i^2$$

- Effect: **weight decay** (basically add $2\lambda w$ to the gradient for each weight)

$$\boldsymbol{w}_{new} = \boldsymbol{w} + eta * \Delta \boldsymbol{w} - 2\,\lambda\,\boldsymbol{w} \quad (of\ course\ 2\ can\ be\ omitted)$$

- Chose the lambda regularization parameter (generally very low value, e.g. 0.01): selected by the *model selection* phase (cross-validation)
- Note: if applied to linear model → it is the *ridge regression* *(see lect .on linear models)*
- More sophisticated penalty terms have been developed (e.g. for *weight elimination*: see Haykin, you can try them)



2. Term issue: Error and Loss
Repetita



Dip. Informatica
University of Pisa

Recall what we already discussed about the objective function:

- When we have, as in the Tikhonov regularization, a penalty term in the objective function is better to clarify:
  - We can use **Loss** term for the objective function (MSE+Penalty) for model training
    (coherently with the CM course, in a broader use of the term)
  - And **Error** /Risk for the "data term" (the MSE, depending on data)

$$\sum_{p}(d_p - o(\boldsymbol{x}_p))^2$$

"Error" term [(M)SE]

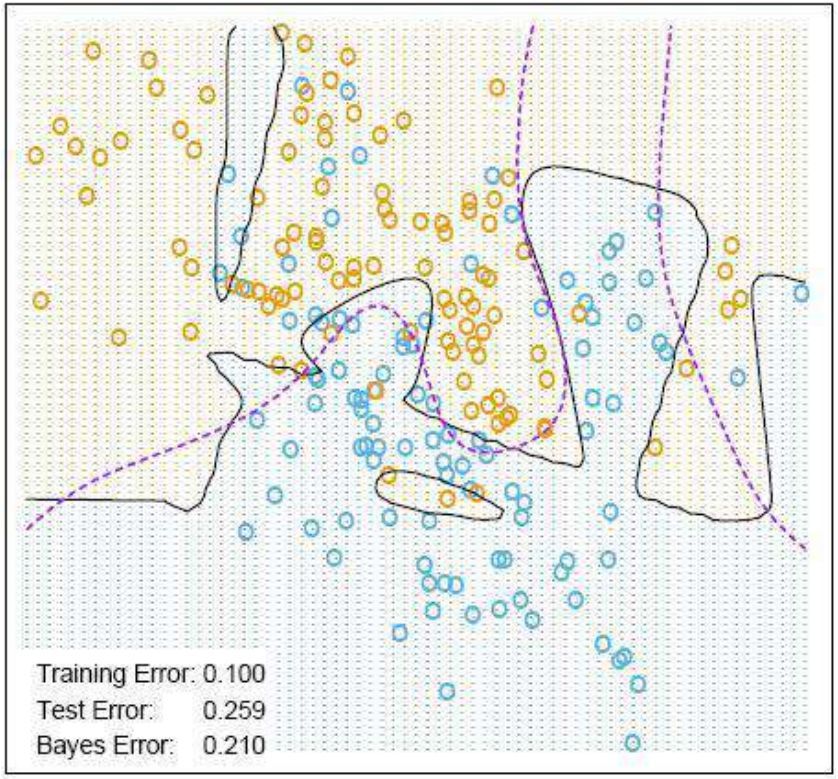
to evaluate *model error*, because this is the measure useful to the user (using the model) and to be reported in the table or plots etc. **(!)**

2. Effect of regularization

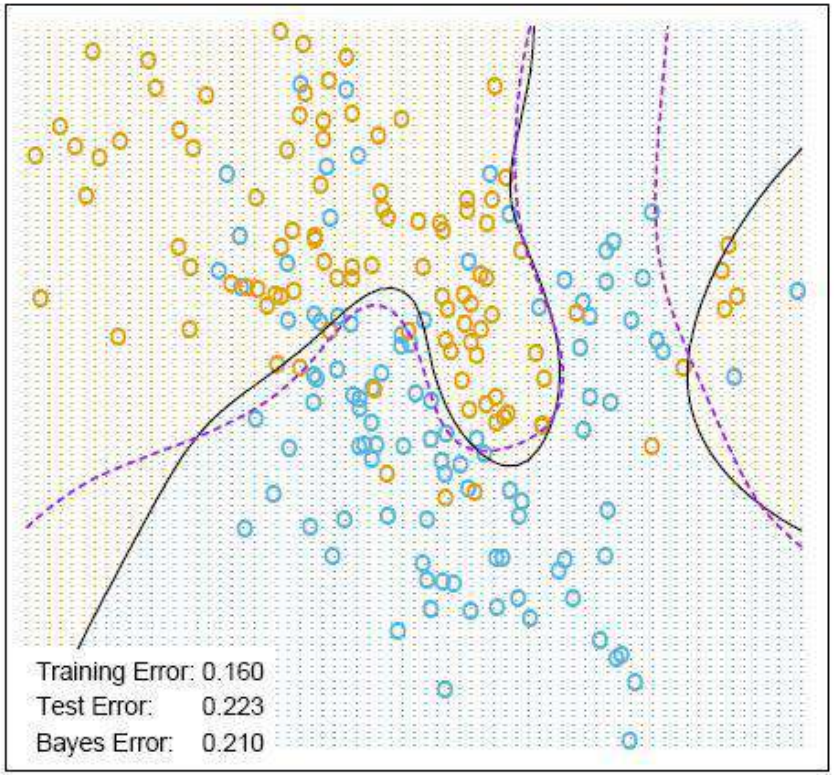
Dip. Informatica
University of Pisa

Weight decay with λ=0.02

Neural Network - 10 Units, No Weight Decay



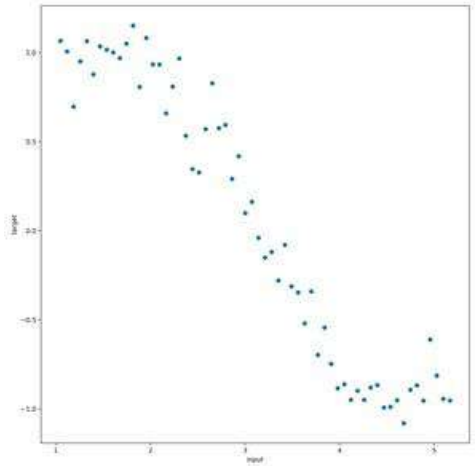
Neural Network - 10 Units, Weight Decay=0.02



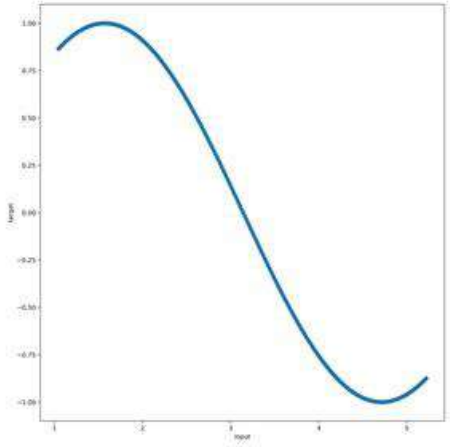
© Hastie, Tibshirani & Friedman 2009

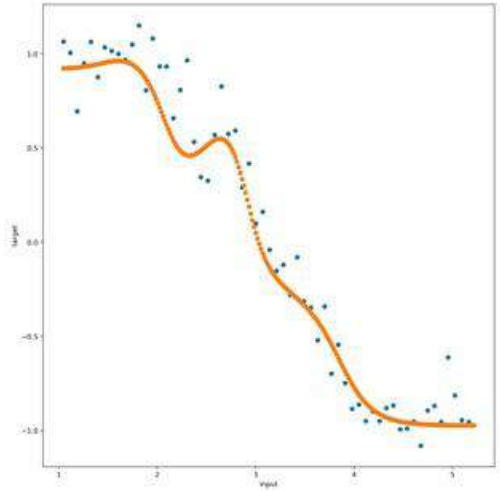
Univariate regression example

Training data

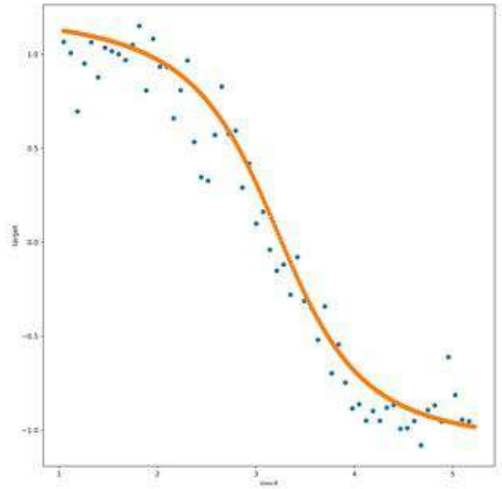


True function



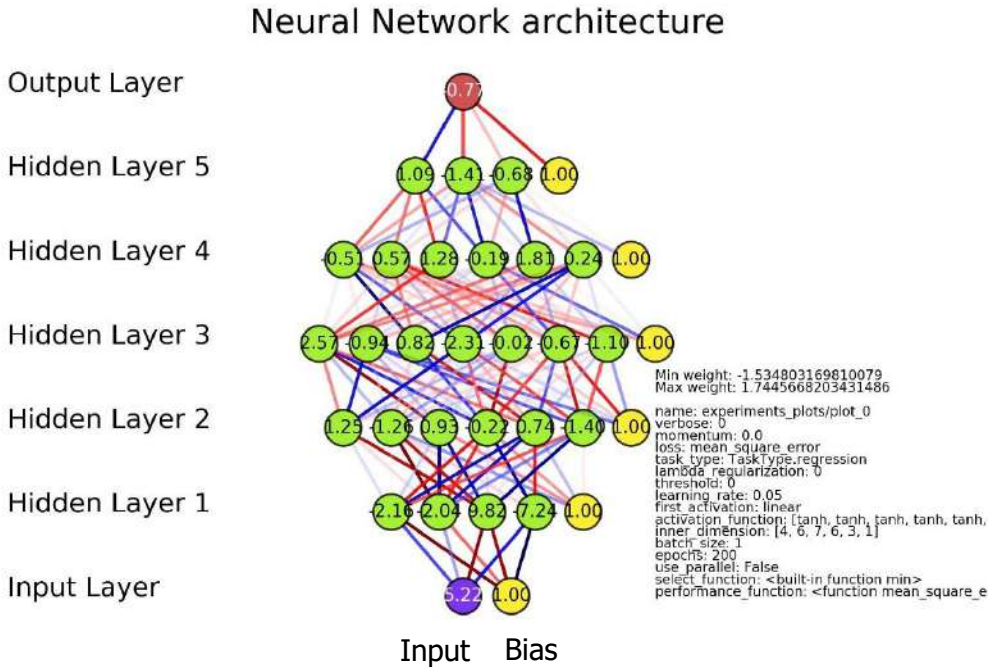


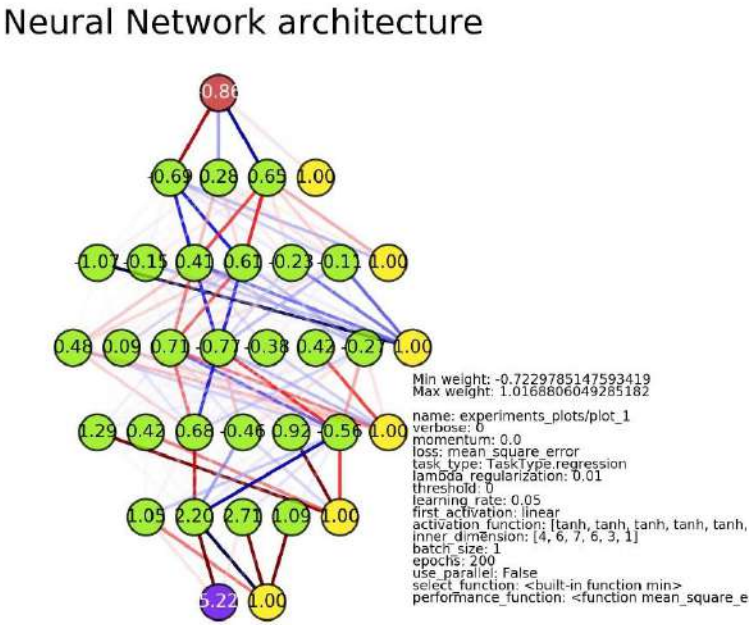
NN output λ=0



Regularized (λ= =0.01)

Plots by Aurora, Gravina 2019





NN trained with $\lambda=0$:
all the weights are significantly higher (or lower) than 0.

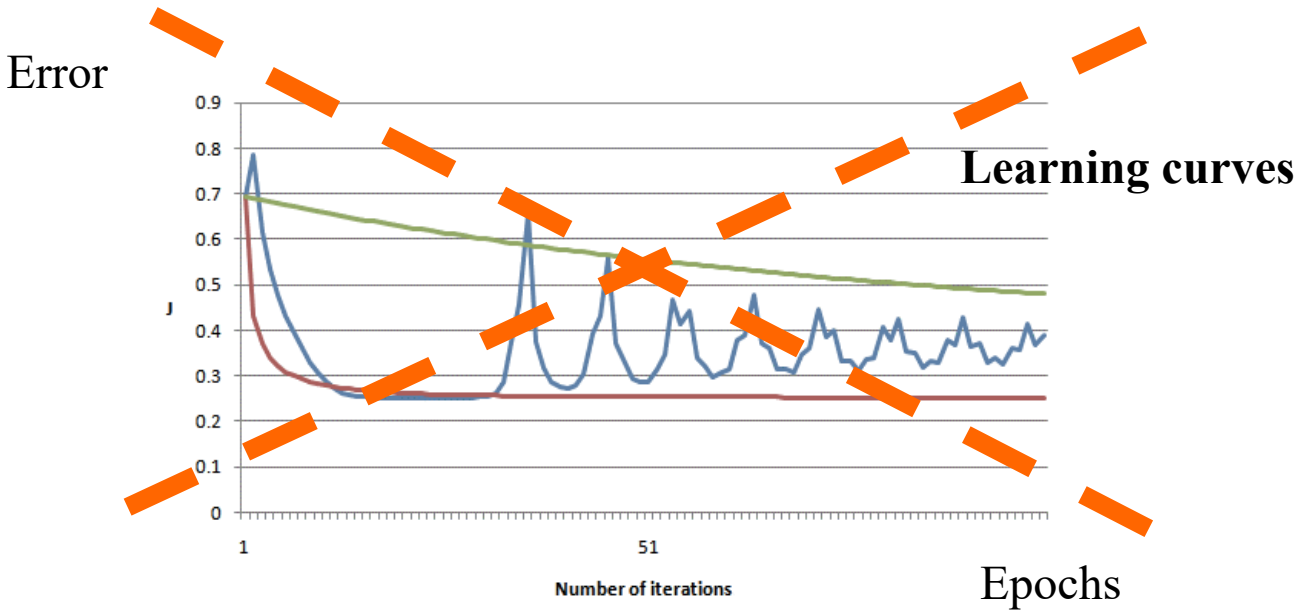
Regularized ($\lambda=$ =0.01):
most of the weights are close to zero (light color → lower in module).
Hence lower model complexity!

Figures by Aurora, Gravina 2019

2. Regularization: Frequent misunderstandings (I)

Dip. Informatica
University of Pisa

I. Does the regularization help the **convergence stability?**
It is important to highlight that the regularization **is not** a technique to control the *stability* of the training converge (*learning curve*) but, as you well know, it allows the control of the *complexity* of the model (measured by the VC dimension according to the SLT theory), which is related to the number of weights and values of the weights in NNs.



2. Regularization: Frequent misunderstandings (II)



Dip. Informatica
University of Pisa

II. Better Early Stopping (**E.S.**) **or** Regularization (**Regul.**)**?**
We will discuss later, but note that the E.S. is an *empirical approach* that needs a VL set to decide the stopping point, which sacrifices a part of data.
The Regul. in the Loss is a *principled approach*: it allows the VL curve to follows the TR curve (provided that you avoid that the VL curve is close to TR curve just due to underfitting with a too high lambda);
in that case you don't need (or strictly need) of the E.S. heuristic and you can stop at training convergence.
Anyway, you can also use both!

(and the E.S. will not enter in action if VL error does not increase)



2. Regularization: details

Dip. Informatica
University of Pisa

- Note that often the **bias** w_0 is omitted from the regularizer (because its inclusion causes the results to be not independent from target shift/scaling) or it may be included but with its own regularization coefficient (see Bishop book, Hastie et al. book)



- Typically apply it in the batch version
- For **on-line/mini-batch** take care of possible effects over many steps (patterns/examples): hence to compare w.r.t. batch version in a fair way do not force equal *lambda* but it would be better to use



$$\lambda \text{ x } (mb \text{ /\#total patterns } l \text{ })$$

  - Of course if you choose lambda by model selection they will automatically select different lambda for on-line and batch (or any mini-batch)

- Other techniques: **Dropout** → see later

Q&A lecture set-up

Dip. Informatica
University of Pisa

- Please send to me further questions by **email** so to set up Q&A lectures

  Please, **USE in the subject the TAG [ML-24-Q]**

  - We can discuss
    - Self-evaluation quiz results
    - New questions to better understand some parts of the course (no questions, no Q&A lecture ;-))

- Memento
  - Special interactive classes will be used to assess activities and make a ML discussion forum (**Q&A classes**).
    This makes the course a class, not just a set of records
  - Within a **cooperative** and collaborative context!

2. Putting together I (FAQ)



Dip. Informatica
University of Pisa

- Merging (1) Moment with (2) Weight decay

$$\Delta w = -\eta \frac{\partial E(\mathbf{w})}{\partial w} + \alpha \Delta w_{old}$$

(1) We wrote for the Momentum, but what is $E(w)$? We used the MSE Error

- (2) Moving to include the weight decay (Tikhonov), we move to the *Loss*:

$$\Delta w_{tu} = -\eta \frac{\partial Loss(\mathbf{w})}{\partial w_{tu}} + \alpha \Delta w_{old\,tu} \overset{\text{assumed}}{\doteq} \eta \delta_t o_u - \lambda w_{tu} + \alpha \Delta w_{old\,tu}$$

Where (p are omitted and) we consider the *Loss=Error+Penalty* separating the 2 terms in the derivation and using eta only for E term (we already did it!: see the exercise in the linear model part). In this way, eta does NOT multiply lambda → changing eta we do not change lambda and vice versa (i.e. we make **independent** the two hyperparameters and their effects, and we make easer to control them).

- In this context (because, as for the momentum, η is in the Δw), we have

$$\boldsymbol{w}_{\text{new}} = \boldsymbol{w} + \Delta \boldsymbol{w}$$

2. Putting together II (FAQ)



Dip. Informatica
University of Pisa

Hence, $\Delta w_{tu} = \eta \delta_t o_u - \lambda w_{tu} + \alpha \Delta w_{old\,tu}$

- And now, does Δw include the lambda part for the momentum **?**
  Often yes, e.g. in the toolbox of Matlab, Caffe simulator etc.
- But we can also keep separation of the 3 hyperparameters writing:

$$\Delta w_{tu} = \eta \delta_t o_u + \alpha \Delta w_{old\ tu}$$

$$w_{new\ tu} = w_{tu} + \Delta w_{tu} - \lambda w_{tu}$$

i.e. by adding the lambda part directly to w update (*separating it from the momentum memory*)

- In this form lambda, eta and alfa become ***independent*** !
- And you can better control the different role of 3 hyperparameters

- Further notes:
  - divide by l only the gradient of E (i.e. η) where you use a sum over the patterns

Issues in Training NN

Dip. Informatica
University of Pisa

- Heuristic guidelines in setting the back-prop training

General problems:

- The model is often over-parametrized
- Optimization problem is not convex and potentially unstable
- We discuss few of them (see e.g. Haykin for details)
  - **Hyperparameters**: Starting values, on-line/batch, learning rate
  - Multiple Minima
  - Stopping criteria
  - Overfitting and regularization
  - **Hyperparameters**: Number of hidden units
  - Input scaling/ Output representation
  - ……..
- A more specific discussion for DL is in the DL book (we will see later)

Number of Units

Dip. Informatica
University of Pisa

- Related to the **control of complexity** issue !!!
- Related also to the input dimension and to the size of the TR data set

In general: this is a "***model selection***" issue

- The number of units along with regularization parameter can be selected by a cross-validation, i.e. the model selection phase (on a validation set)
  - To few hidden units → underfitting; to many → overfitting (can occur)
  - *The number of units can be higher if appropriate regularization is used*

- **Constructive approaches**: the learning algorithm decides the number starting with small network and then adding new units
- **Pruning methods**: start with large networks and progressively eliminate weights or units

Constructive approaches

Dip. Informatica
University of Pisa

- *Incremental approaches*: algorithms that build a network starting with a minimal configuration and
- add new units and connections during training.

- Examples:
  - For classification: Tower, Tiling, Upstart
  - For both regression and classification: Cascade Correlation

  - See **Scott E. Fahlman** and **Christian Lebiere:**

    The Cascade-Correlation Learning Architecture 1991 CMU-CS-90-100 Carnegie Mellon University

    Also in NIPS 2, 1990, pages 524-532



:-)

Constructive approaches

Dip. Informatica
University of Pisa

- An exemplificative and effective approach :
- the **Cascade Correlation** (**CC**) learning algorithm

- Start with a minimum networks
- Add units if they are needed until the error is low.

- The CC alg. learns both the network weights and network topology (number of units)
  - **automatic** determination of network dimension and topology:

    CC allows to deal with hypothesis spaces of flexible size, since the number of hidden units is decided by the learning algorithm;
  - training of a **single unit** for each step.

The CC algorithm

Dip. Informatica
University of Pisa

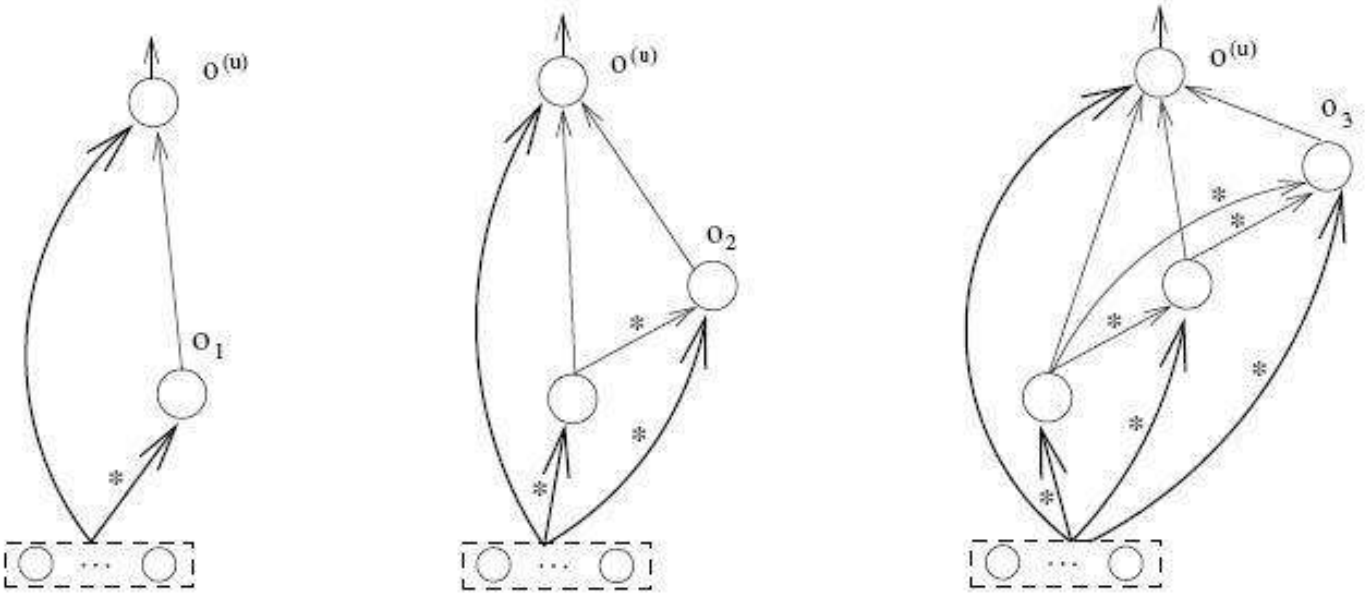
- **Start** with N0 network, a network without hidden units: Training of N0. Compute error for N0. If N0 cannot solve the problem: go to N1.
- In the N1 network a hidden unit is added such that the ***correlation*** between the output of the unit and the **residual error** of network N0 is maximized (by training its weights, see next slide).
- After training, the weights of the new unit are **frozen** (they cannot be retrained in the next steps) and the remaining weights (output layer) are retrained.
- If the obtained network N1 cannot solve the problem, new hidden units are progressively **added** which are connected with all the inputs and previously installed hidden units.
- The process continues until the residual errors of the output layer satisfy a specified **stopping** criteria (e.g the errors are below a given threshold).

The method dynamically builds up a neural network and terminates once a sufficient number of hidden units has been found to solve the given problem.
Cascade of units

CC architecture building

Dip. Informatica
University of Pisa

- The evolution of a CC network with up to 3 hidden units



© A. Micheli 2003

* = weights frozen after candidate training

Learning for CC

Dip. Informatica
University of Pisa

- Specifically, the algorithm works interleaving the minimization of the total error function (LMS), e.g. by a simple backpropagation training of the output layer, and the **maximization** of the (non-normalized) correlation, i.e. the covariance, of the new inserted hidden (candidate) unit with the residual error:

$$S = \sum_k \left| \sum_p (o_p - \underbrace{mean_p(o_p)}_{\bar{o}})(E_{p,k} - \underbrace{mean_p(E_{p,k})}_{\bar{E}}) \right|$$

$$E_{p,k} = (o_{p,k} - d_{p,k})$$

E: residual error with $o_{p,k}$ at the output layer

p: pattern, k: output unit

o (in eq. for S): candidate output

- **Exercise:** derivation (sketch on blackboard)
  hints: $d|f|/dx = sign(f)\ df/dx$; assume no effect of variations of o_p over $\bar{o} = mean_p(o_p)$

- Result:

$$\frac{\partial S}{\partial w_j} = \sum_k \operatorname{sgn} S_k \sum_p (E_{p,k} - mean_p(E_{p,k}))\, f'(net_{p,h}) I_{p,j}$$

$I_{p,j}$ → Input

h : candidate index (not needed since we have just one candidate at time)

- Weight update: why + dS/dw_j ?

Notes on CC

Dip. Informatica
University of Pisa

- The role of hidden units is to reduce the residual output error
  - Each unit solves a specific sub-problem
  - Each unit becomes a permanent "feature-detector"

- POOL: Typically, since the maximization of the correlation is obtained using a gradient ascent technique on a surface with several maxima, a pool of hidden units is trained and the best one selected. This helps avoiding local maxima.

- Greedy algorithms: easy to obtain converge, may also find a minimal number of units but it may lead into overfitting (need regularization)

- Loss: Not necessarily "max S": e.g.,for regression, LMS in Prechelet 1997, Neural Networks Vol. 10 pp. 885-896

Issues in Training NN

Dip. Informatica
University of Pisa

- Heuristic guidelines in setting the back-prop training

General problems:

- The model is often over-parametrized
- Optimization problem is not convex and potentially unstable
- We discuss few of them (see e.g. Haykin for details)
  - **Hyperparameters**: Starting values, on-line/batch, learning rate
  - Multiple Minima
  - Stopping criteria
  - Overfitting and regularization
  - **Hyperparameters**: Number of hidden units
  - Input scaling/ Output representation
  - ……..
- A more specific discussion for DL is in the DL book (we will see later)

Input/Output

Dip. Informatica
University of Pisa

- Preprocessing can have large effect
- For different scale values, *normalization* via :
  - Standardizations (often useful):

    For each feature we obtain zero mean and standard deviation 1:
    v-mean /standard dev.
  - Rescaling: [0,1] range: *v-min /(max-min)*



- *Categorical inputs*: *see Lectures for Introduction (e.g. 1-of-k)*

  A →100, B → 010, C →001
- *Missing data*: Warning: 0 does not mean "no input" if 0 is in the input range !

Input/**Output**



Dip. Informatica
University of Pisa

- **Regression**: Output **linear** unit(s) **(!)**
  - (1 unit or more output units for multiple quantitative response)

- **Classification**: 1unit (binary) or 1-of-K with multioutput
  - Sigmoid → choose the *threshold* to assign the class
  - *Rejection zone* (is possible, se slide on sigmoidal functions)
  - *1-of-K encoding*: to choose the winner class take the highest value among the output values
  - Often symmetric logistic *(TanH*) learns faster (see Haykin book)
  - 0.9 can be used instead of 1 (as d value) to avoid asymptotical convergence (-0.9 for –1 for TanH, or 0.1 for 0 for logistic)

Of course, target range must be in the output range of units

Regression and Classification Tasks



Dip. Informatica
University of Pisa

- **Classification**: multi output , 1-of-K encoding (each coded as 0/1 or –1/+1)
  - **Sigmoidal activation (i.e. Tanh)**
  - **For 0/1 targets**: Softmax function (sum to 1, can be interpreted as probability of the class p(class=1|x) !!!)

Popular nowadays for classification

For numerical issues:
DL book (sec 4.1 and 6.2)

Unit k $$o_k(\boldsymbol{x}) = \frac{e^{(-net_k)}}{\sum_{j=1}^{K} e^{(-net_j)}}$$

  - Also we can use an alternative error/cost function: minimize the "cross entropy" → maximum likelihood estimations

e.g for 1 unit: $$-\sum_{i \in TR} \{d_i \log(out(x_i)) + (1-d_i)\log(1-out(x_i))\}$$

If loss =0 → no classification errors!

Ref: Bishop 1995
Also DL book (sec. 6.2)

Exercises

Dip. Informatica
University of Pisa

Open questions
(for your self-assessment and to make a good project):

- Think on the role of the different hyper-parameters
- Connect to the theory (also later after next lectures)
  - E.g. relate the optimization of the loss with penalty term to the Statistical Learning Theory (VC-bound in the introduction and next lectures) to understand the role of the *number of units , lambda, and others,* in controlling under/overfitting cases.
- Exercise: Hence, which are the most relevant for your prj (e.g. to win the blind competition)?

Applications (guide slide)

Dip. Informatica
University of Pisa

- A benchmark, anticipating the MONK example to "start" the projects
- CNN & character recognition (next lectures)
- Final lectures at the end of the course

BUT we will anticipate
other parts to favorite the project starting

PRJ: Ready to go?

Dip. Informatica
University of Pisa

Ready to start the project? **NO**,

BUT You can start to think about NN implementation (core of the training part, with benchmark data) or exploring first usage of the simulators; and I'm going to introduce the **Monk** benchmark.

Not ready because we have still to present

- VALIDATION (hyperparameters selection, min. 2 lectures)
- The PRJ assignment (1 lecture)

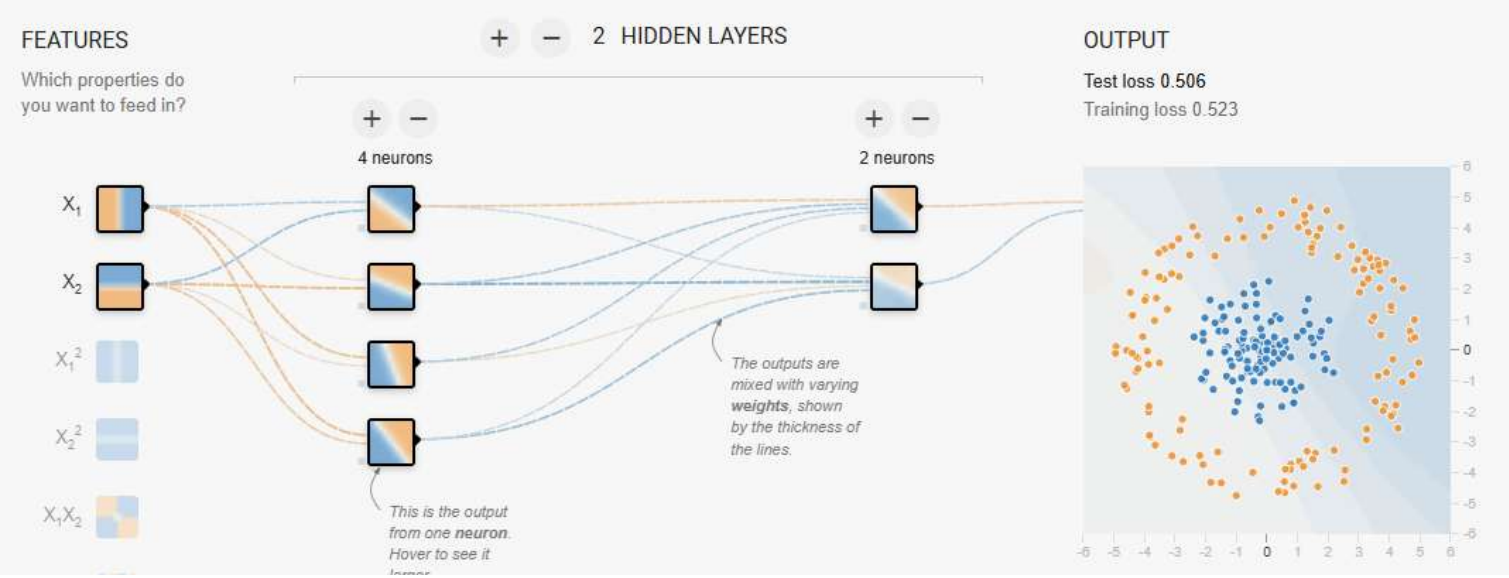
The project (type A) will have

- Backprop (gradient computation) implementation (ready)
- DeltaW rules implementation (ready)
- Validation (TR/VL/TS) (NOT READY yet!!!) both for type A and B

Play with a simple NN

Dip. Informatica
University of Pisa

- https://playground.tensorflow.org/
- Enjoy



"Tinker With a Neural Network Right Here in Your Browser. Don't Worry, You Can't Break It. We Promise."

(a preview for the project, <anticipo di slides sul prj>)

General hints: benchmark



Dip. Informatica
University of Pisa

- Difficulty of assessing implementation correctness
  - Because often a NN still works (even if hampered) with small implementation errors
- A first test *("collaudo")* **MONK's Problems Data Set**
  - (the results must be reported in the prj report)

    http://archive.ics.uci.edu/ml/datasets/MONK's+Problems
  - 3 tasks binary classification, small artificial data set, "not difficult" (a small NN with few units achieve a very high accuracy, up to 100%, with small time of convergence)
  - There is a report with previous results using MLP (and others ML models):
    - chapter 9 http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.45.2363 *
  - Input encoding: 6 variables with 2, 3 or 4 symbolic values each: 1-of-k → correspond to introduce 17 input units (see also sec 1.1 and check it!)
  - TS includes TR, which is a bad practice, but does not change here (since 100% accuracy is 100% accuracy also on the test set !!)
- Other sources of data sets for software tests: UCI Machine Learning Repository (see later)

Other suggestions for your first trials



Dip. Informatica
University of Pisa

- That results in the following were obtained with 1 output unit for classification (using 0/1 or -1/+1 encoding both for target and outputs of course!):
  - *tanh* is suggested for the output unit
- (repetita) 1-hot encoding for the inputs (previous slide): *frequent fault*!!!
- Batch with grad/#patterns allows to converge in few hundred of epochs

- How many hidden units? Follow the manual as baseline (very few units: 2-5!!!)
  - The test works if you achieve the state of the art results with few units

- MONK can be sensible to weights initialization (due to the use of few units),
  - So, for some random initialization with high range the convergence could be slow

→ try small initial random weight ranges (or using fan-in) [see lect.s before*]

- If you use on-line approaches use shuffling (patterns are ordered according to the first and second class)

NOTE!

Dip. Informatica
University of Pisa

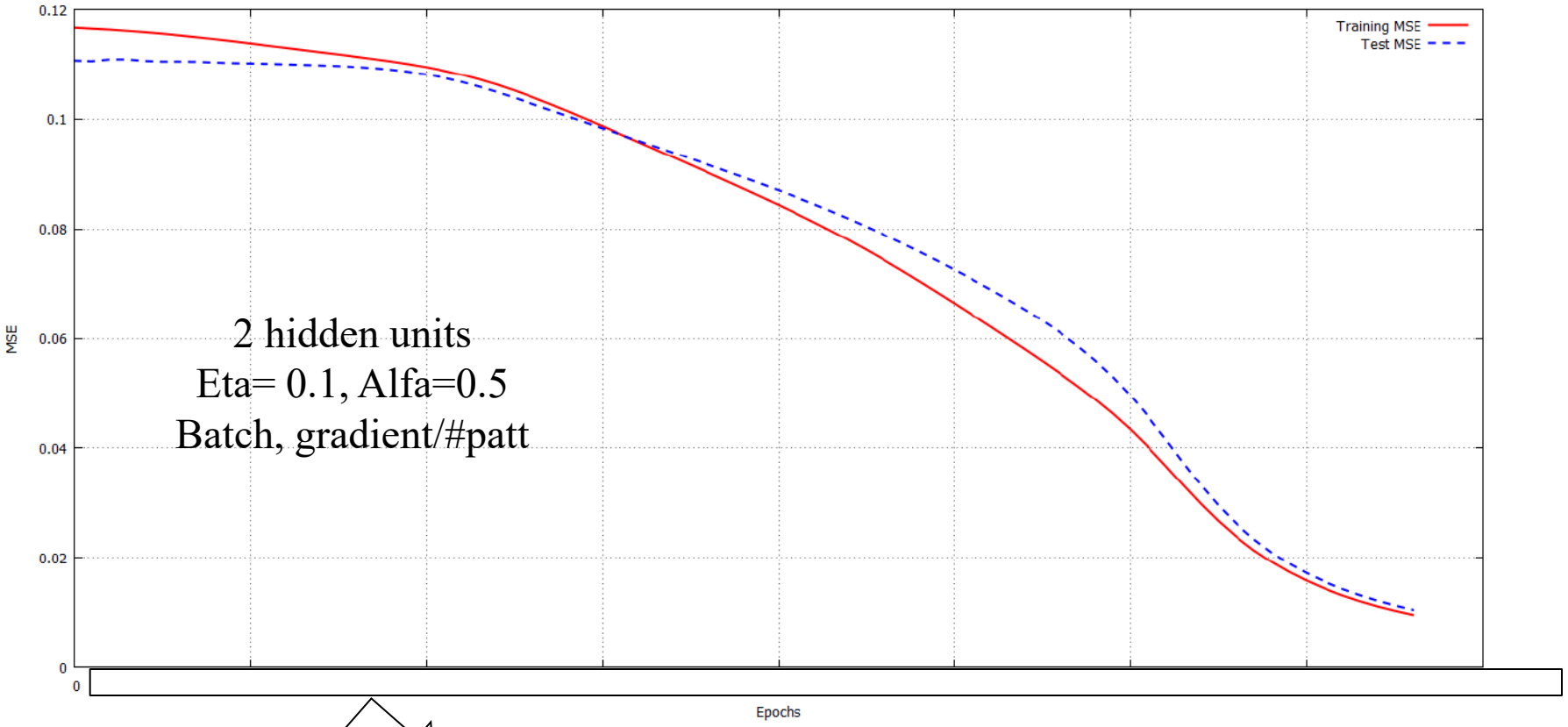
- In the following QUALITATIVE results
  - Does not care about the values in itself, just the shape of the learning curve to see if you are close or far from a good convergence!
  - Note: student simulator results and the results from some well-know libraries are similar.
  - Note that ALL the plots are smooth in the MSE
  - We also use regularization (with a penalty term) in case of Monk3

- Good results on the MONK benchmark does not guarantee the simulator correctness
- Bad results on the MONK benchmark for sure requires revision of the code/setting

Monk2: example (MSE) Student simulator

Dip. Informatica
University of Pisa

- MSE vs epochs

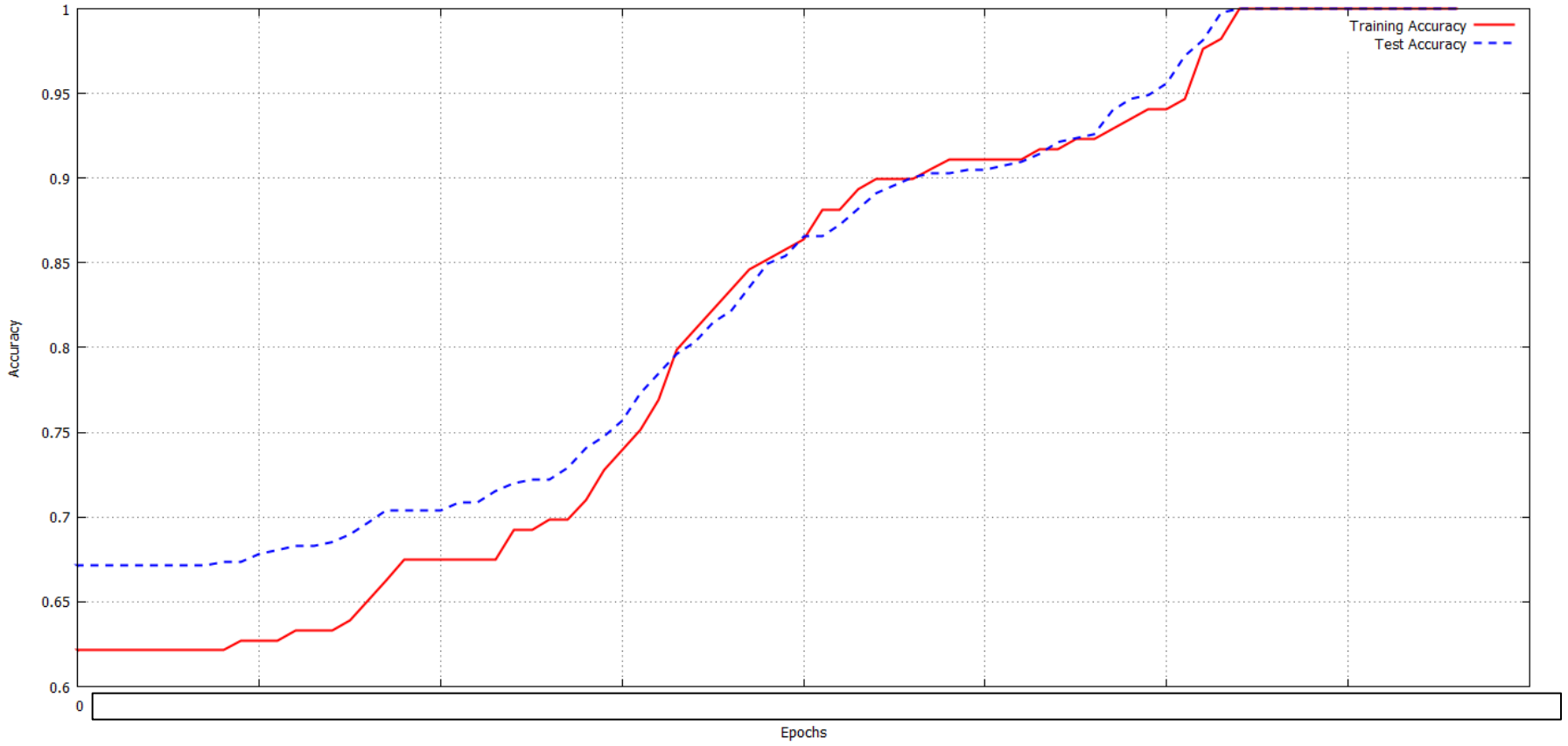


The numbers are intentionally omitted, don't care now
Depending on many factors could be e.g. 80-300/400 epochs

Monk2: example (Accuracy) Student simulator

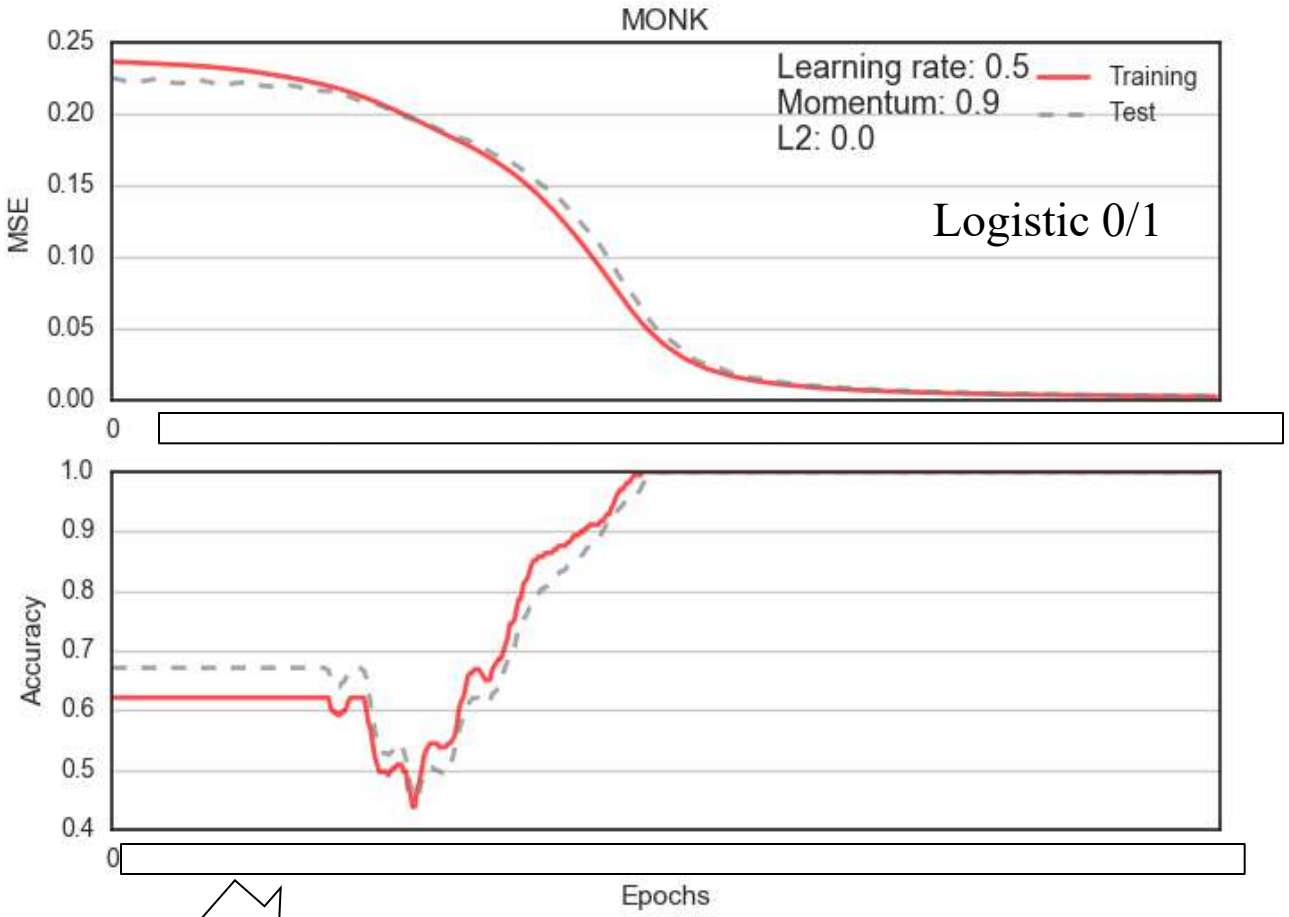
Dip. Informatica
University of Pisa

- Accuracy vs epochs. 100% accuracy is achieved on test! (which is *unusual* in general)



Monk2: With Theano

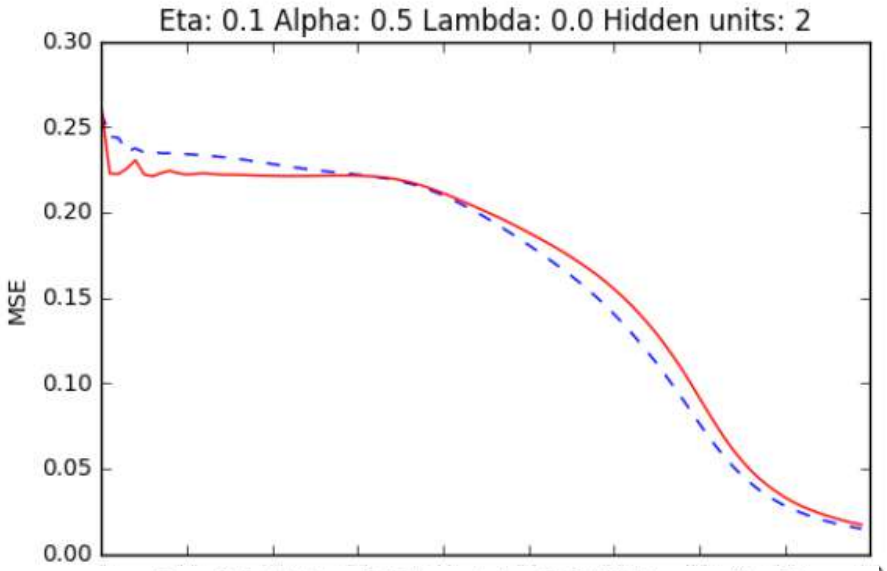
Dip. Informatica
University of Pisa

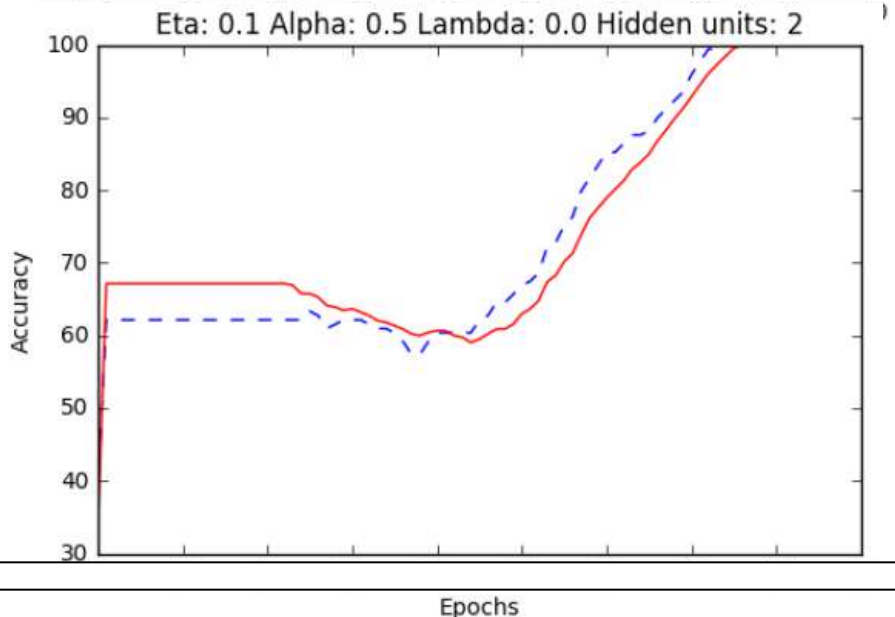


The numbers are intentionally omitted, don't care now
Depending on many factors could be e.g. 80-300/400 epochs

Monk2: Student simulator

Dip. Informatica
University of Pisa

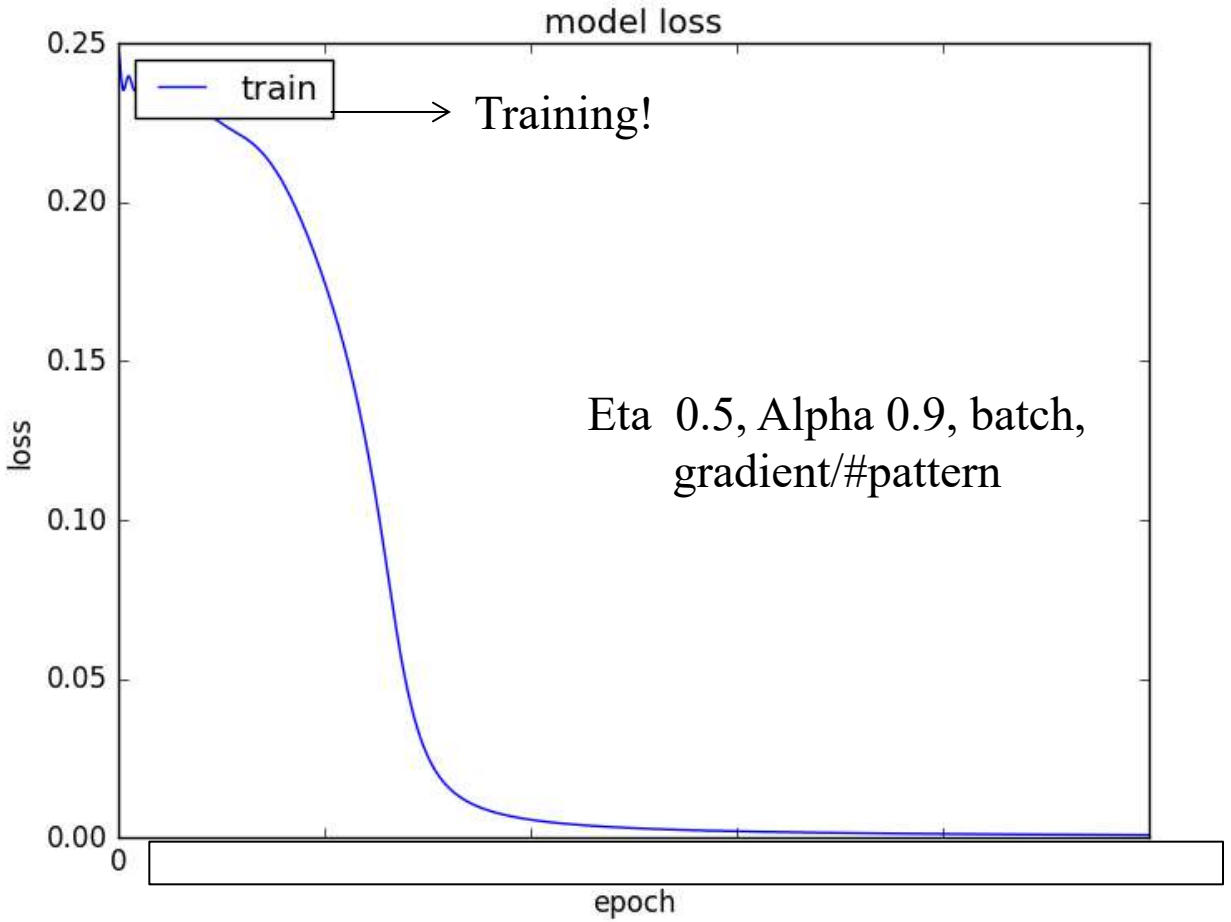




Red is test
Blu training

Monk2: Keras

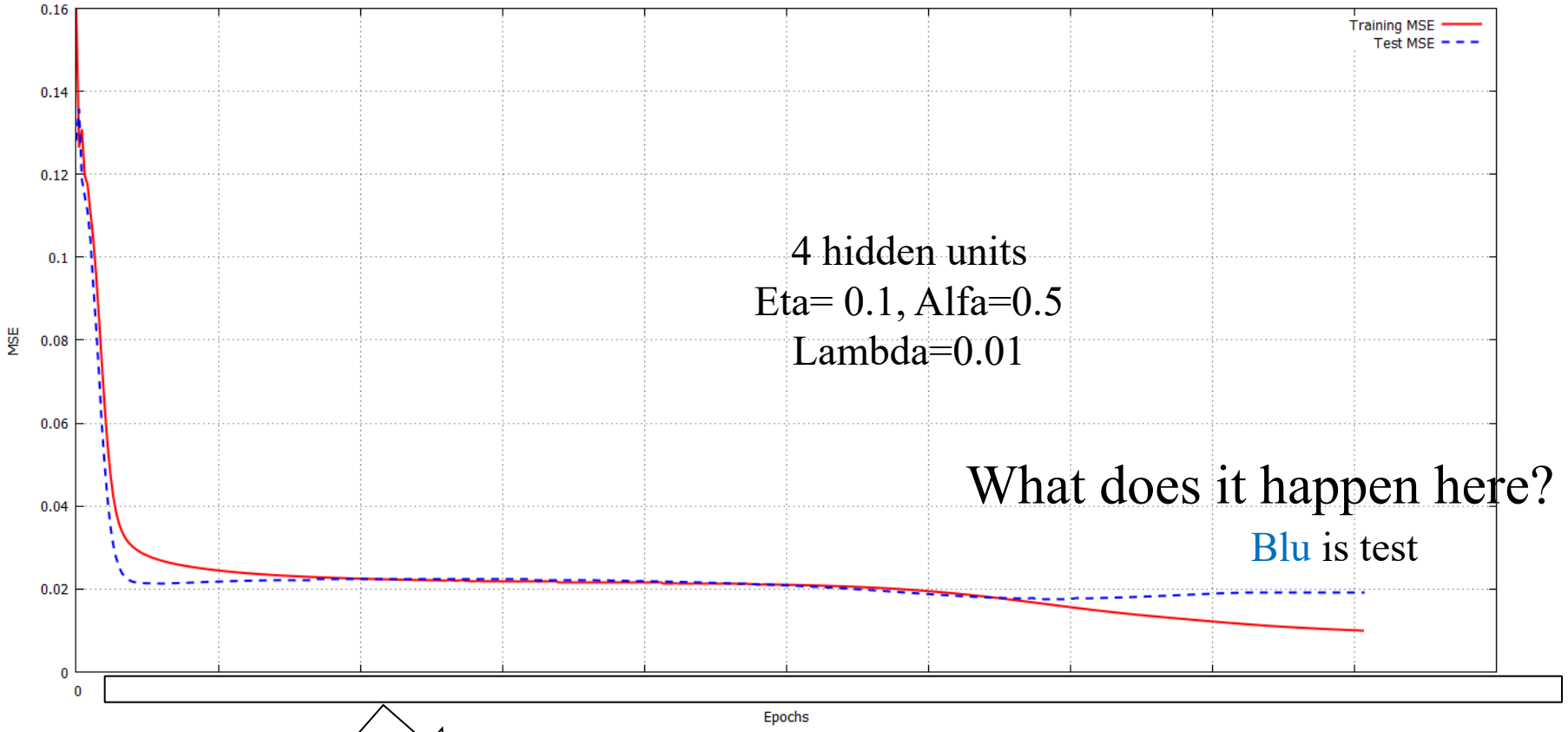
Dip. Informatica
University of Pisa



Monk3: example (with regulariz.)
Student simulator

Dip. Informatica
University of Pisa

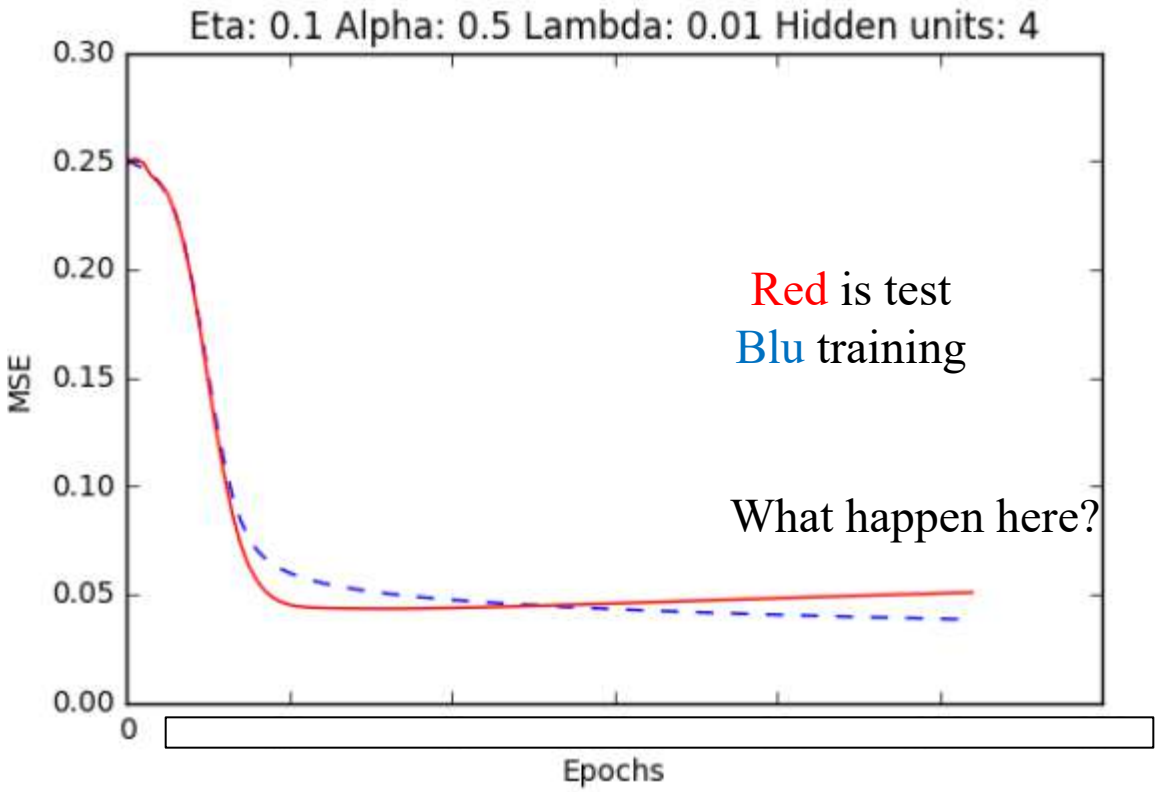
- MSE vs epochs (monk3 can occur in overfitting, test is **not** at 100%, but improve with lambda)



The numbers are intentionally omitted, don't care now
Depending on many factors could be 100-300 epochs

Monk3: a different student simulator

Dip. Informatica
University of Pisa



(See more in the PRJ slides) Data Sets: other benchmark istances

Dip. Informatica
University of Pisa

You can also select a benchmark useful to your case (e.g. to work with many data, many features etc.) from a public data sets (with hundreds of data sets), e.g.

UCI Machine Learning Repository: Data Sets
https://archive.ics.uci.edu/datasets



Examples (NOT requested for the PRJ)

DataSet Name	N. Instances	N. Attributtes	Attribute (type)	N. Classes
Adult/Census	48842	14	Categ./Intero	2
Mushroom	8124	22	Categorical	2
(WBC)BreastCancer Wisconsin (Diagnostico)	569	32	Real	2
Nursery	12960	8	Categ.	5 (4)
Spambase	4601	57	Real	2
Zoo	101	17	Categ.	7

Other practical hints

Dip. Informatica
University of Pisa

- You can start *to think about* NN implementation (the core) or to a first exploration/usage of simulators
- I will communicate soon also the name of the *course assistants*

- To check your code (backpropagation part) you can use this VERY INFORMAL example (suggested by a student, not verified):

http://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/

Warning: in that exercise the bias (w_0) was omitted from the analysis, indeed it is need to develop the example also for the bias

- Or use any "reliable" library as an "oracle" (we will introduce them later)
  - **Keras (NN)** (running on top of TensorFlow, Theano, et al., see bibliography), **Scikit-learn**, Caffe, Torch, **PyTorch (NN)**, R, **Matlab** ….
    We will discuss many more later

- NOTE: We will provide in the next lectures the guide for the *model validation*, that are fundamentals for any applications!

PRJ: a preview!

Dip. Informatica
University of Pisa

- In the following: few slides for some main FAQs, (mostly repetita from the introduction of the course) but we will have a ***specific lecture on the project*** **later**!

Assessment methods
(REPETITA from introduction)





Dip. Informatica
University of Pisa

Exam:

- **Project** (Written exam)
  - Students have the opportunity to develop a project realizing/applying a learning system simulator (typically a simple neural network) and to validate it through benchmarks. A written report will show the results.
    - Great *opportunity* to apply the concepts by yourself
    - Great *opportunity* to show your concrete understanding and effort for the exam
  - Deadline: ~(around /or more than)10-14 days before the oral exam session (see the Moodle folder of your session for the exact deadline)
  - **See details in the lecture for project presentation**
  - Last years: *competition* with *blind-test*
    - which is part of the benchmark results in the prj
    - also some joint proposals with CM course
- **Oral exam** (date according to the exams sessions): see introduction slides



(REPETITA from introduction)

The main hints





Dip. Informatica
University of Pisa

- Follow the lectures and slides as a guide, studying *progressively* during the course
  - Special interactive classes will be used to assess activities and make a ML discussion forum (**Q&A classes**). This is a class, not a set of records*.
  - Within a **cooperative** and collaborative context!
  - The **intermediate tests** are also meant to stimulate you to **stay "*online*"** and to have constant *self-evaluations*

- A major hint *from past students*:

1. **FIRST study the course content**
2. **THEN apply for the project**
   - This help to boost efficacy and efficiency of your work

PRJ Preview: Possible Aims

Dip. Informatica
University of Pisa

A) Realize a NN model simulator and apply it (implementation)

- Programming language is a free choice (C++, Python are popular for ML, or even environments as Matlab or R can be considered)

B) Extensive experimental applications of existing ML/NN simulators or code made by LLM (NEW): **(comparison)**

- You will consider different models as NN, SVM, K-NN, …
- Simulator/s is/are a free choice (a list will be discussed within prj lecture)

C) Contact me (e.g. PhD students etc)

A) or B): participate to the "**ML cup**" competition.

Details for A) B) [C)] will be discussed in the Prj lecture

PRJ Preview: other FAQs: REPETITA

Dip. Informatica
University of Pisa

- **Groups**: you will participate in **groups of 2/3 people.**
  - Please, search immediately a partner or parterners
  - See FAQ section in the Moodle to search a partner
    - You can use the link to the shared file (see the FAQ)
- **Repetita: Deliverables**: ~around / or more than 10-14 working days in advance w.r.t. to the oral exam: BUT the exact and strict deadline will be indicated in the Moodle folder for each session.
- …questions?



→ **Details will be discussed later in the Prj lecture**

DISCUSSION ON NN

NN historical notes

Dip. Informatica
University of Pisa

- See Haykin (end of chapter 1) and Russel, Norvig book

- **E.g.** McCulloch and Pitts (1943), Hebb (1949), Minsky (1954 and 1969), von Neumann (1956), Kohonen (1972 and 1982), Rosenblatt (1958), Widrow and Hoff (1960), Grosseberg (ART 1980), Hopfield (*) (1982), Rumelhart, Hinton, Williams, Werbos, Parker, LeCun and others (1975-1985 for backprop), Poggio and Girosi (RBF-theory 1990), Vapnik (90s' SVM), … deep learning (see later, Hinton (*)(*), Bengio (*), LeCun (*) et al. 2007 and ahead) … and many others!

  (*) Nobel prize 2024; (*) Turing award 2018

- The field continues to grow in

  theory, new models and applications… *you are welcome!*

NN summary

Dip. Informatica
University of Pisa

- NN provide **flexible method** for ML
  - extend linear methods to arbitrary function estimation
  - construct an explicit hypothesis (with respect to K-NN)

- Expressive power: num. of units + architecture + training
- Hypothesis space is a rich continues space of functions + loss function is differentiable → Training bases on error minimization by gradient descent technique

NN summary (+)

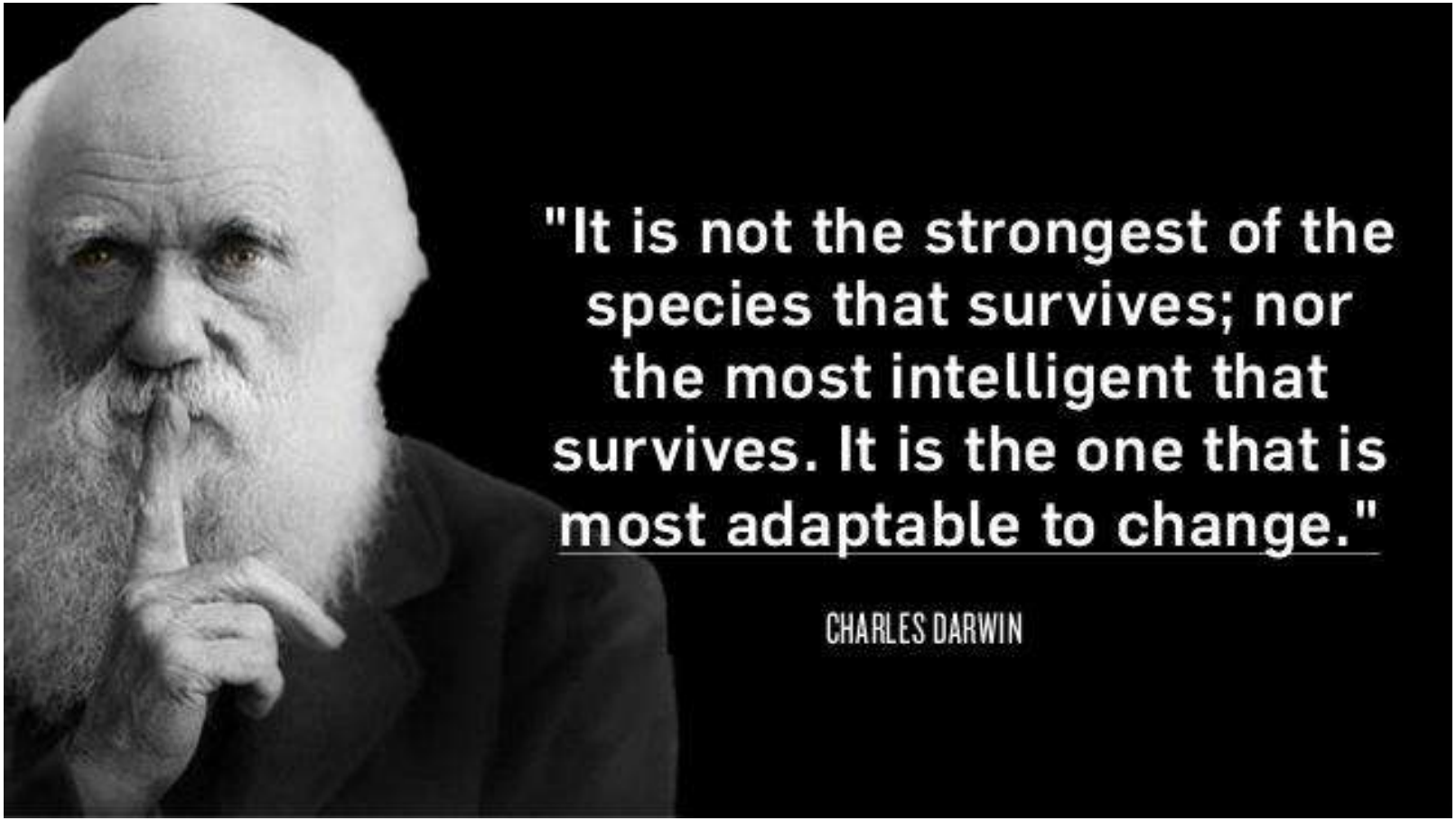
Dip. Informatica
University of Pisa

- NN can learn from examples
- NN are universal approximators (Theorem of Cybenko):
  flexible approaches for arbitrary functions (including non–linear) !
- NN can deal with noise and incomplete data
  - Performance degrades gracefully under adverse operating conditions
- NN can handle both continuous real and discrete data
  for both regression and classification tasks
- Successful model in ML due to the flexibility in applications

- NN encompasses a wide set of models: it is a paradigm !
  (in the class of subsymbolic approaches)
- E.g. also unsupervised tasks, associative memories, stochastic approaces, ….

NN after more than 70 years?

Dip. Informatica
University of Pisa



Others (+)

Dip. Informatica
University of Pisa

- Neural plausibility
- Distributed representation of knowledge
  - The knowledge acquired is represented compactly as weight matrices
- Fault tolerance – robustness (related also to distributed representation)
- Adapt to changes (nonstationary environment): on-line learning is possible (a.k.a. *continual learning*)→ stability-plasticity dilemma

NN summary (-) critical points

Dip. Informatica
University of Pisa

- Black-box problem (inherent inability/difficulty to explain, i.e. it is difficult to interpret the acquired knowledge):
  - extraction and representation of knowledge for human experts
  - related to the current trend in the research for interpretability of ML models and the "*explainable artificial intelligence*", **XAI**, issue

Faced by many approaches, e.g.:

- Projecting/clustering internal representation
- Computing sensitivity analysis (since 1995)
- LRP: Layer-wise Relevance Propagation [Bach et al. 2015]. Also for Deep NNs. Klaus-Robert Müller @ ICANN 2019 "*NN's black box issue is not sense, not true anymore*" (http://www.heatmapping.org/) (see figures below)
- Discovering symbolic rules associated to the trained model !!!
- ...





Why is it a rooster?
Rooster crest in the heatmap
of the internal NN represenntation

NN summary (-) critical points (cnt)

Dip. Informatica
University of Pisa

- Architectural design :
  - Constructive/pruning methods to optimize the architecture
  - New approaches (e.g. SVM, we will see later)

- Training process dependency: training realization (we have seen many variants and possible values of the hyperparameters) has effect on the final solution

- Not the best for Missing data: standard strategies including removing data, imputations, …

Conclusions

Dip. Informatica
University of Pisa

- When to consider NN:
  - Input is high-dimensional discrete or real-valued / classification and regression tasks
  - Possibly noisy data
  - Training time is not critical
  - Form of target function is unknown
  - Human readability of result is not critical
  - The computation of the output based on the input has to be fast

Next for NN

Dip. Informatica
University of Pisa

Applications:

- **CNN example**

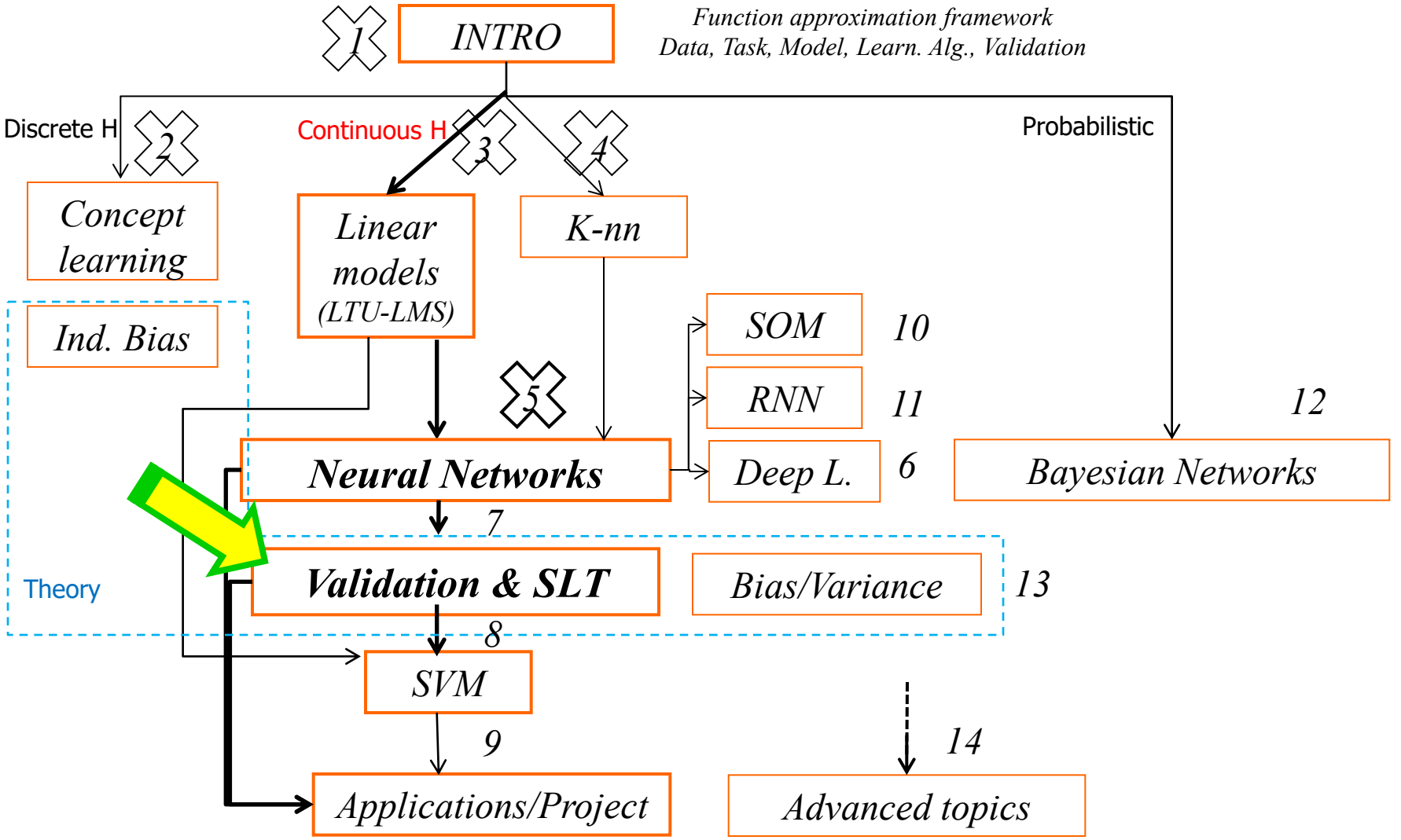
Introduction to recent NN paradigms

- Other architectures…..
  - **Deep learning**
  - **Random weights NN**

- But we will anticipate the Validation part!!!
  (so to favorite prj starting)

ML Course structure
Where we go

Dip. Informatica
University of Pisa



A premise: Bias-Variance

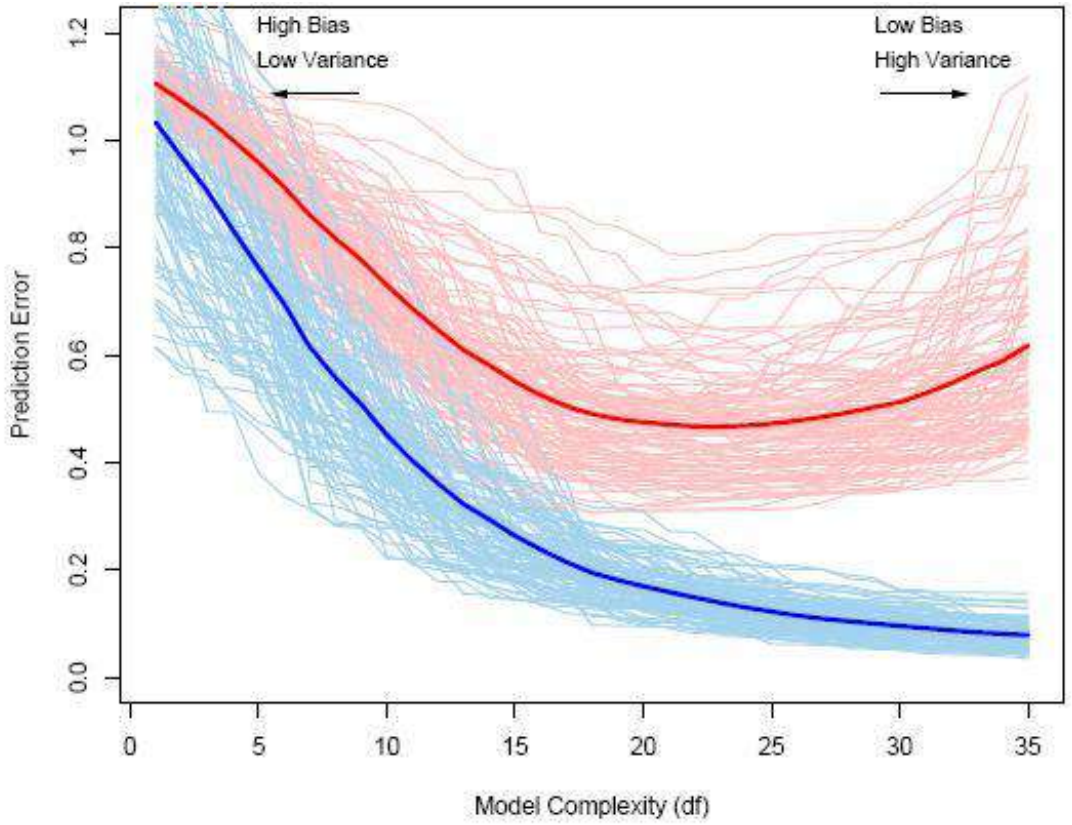
Dip. Informatica
University of Pisa

- Bias-Variance decomposition provides an useful framework to understand the validation issue, showing how the estimation of a model performance *is difficult*
  - Considering also the role of different training set realizations
  - Showing again the need of a trade-off between fitting capability (bias) and model flexibility (variance) in different way

- Anyway we will discuss it later by a specify lecture
- Let me introduce just this plot, to recall the need to optimize a model in the right way.

Under & Overfitting in a Bias-Variance plot*

Dip. Informatica
University of Pisa



unlucky

lucky

Complex model : lower bias, higher variance → possible overfitting

FIGURE 7.1. *Behavior of test sample and training sample error as the model complexity is varied. The light blue curves show the training error* $\overline{\text{err}}$*, while the light red curves show the conditional test error* $\text{Err}_{\mathcal{T}}$ *for 100 training sets of size 50 each, as the model complexity is increased. The solid curves show the expected test error* Err *and the expected training error* $\text{E}[\overline{\text{err}}]$.

Motivations

Dip. Informatica
University of Pisa

- Recall that we are looking for the best solution (minimal prediction – test error) searching a balancing between fitting (accuracy on training data) and model complexity
- Training set is not a good estimate of test error
  - Initially ok but you have too many bias (underfitting, high TR error) than too high variance (low TR error but overfitting)

- Assuming a tuning hyper-parameters θ (implicit or explicit) that varies the model complexity, we wish to find the value of θ that minimizes test error
- Look for methods for estimating the expected error for a model (or each model of a class of models or even of a set of models*)

Approximating estimation step (intro: a guide)

Dip. Informatica
University of Pisa

We can approximate the estimation

- Analytically:
  - AIC, BIC Akaike/Bayesian Information Criterion
    - limited to linear models (in par.)
  - MDL (Minimum Description Length)
  - SRM: **Structural Risk Minimization & VC-dimension →**
    - In lecture INTRO + a next lecture on the VC-DIM

In practice, often we can approximate the estimation on the data set:

- By resampling: direct estimation of extra-sample error via
  - cross-validation (hold-out, K-fold CV, …)
  - bootstrap

Validation: Two aims

Dip. Informatica
University of Pisa

Recall the slides in the intro (on the validation)! *THE MOST IMPORTANT SLIDE of the course*

- After models training on the training set
- ***Model selection:*** estimating the performance (*generalization error*) of different learning models in order to choose the best one (to generalize)
  - this includes search the best *hyper-parameters* of your model (e.g. polynomial order, #units in a NN, lambda, eta, … …).

  It returns a model

- ***Model assessment:*** having chosen a final model (or a class of models), estimating/evaluating its prediction error/ risk (*generalization error*) on new *test* data (measure of the quality of the ultimately chosen model)

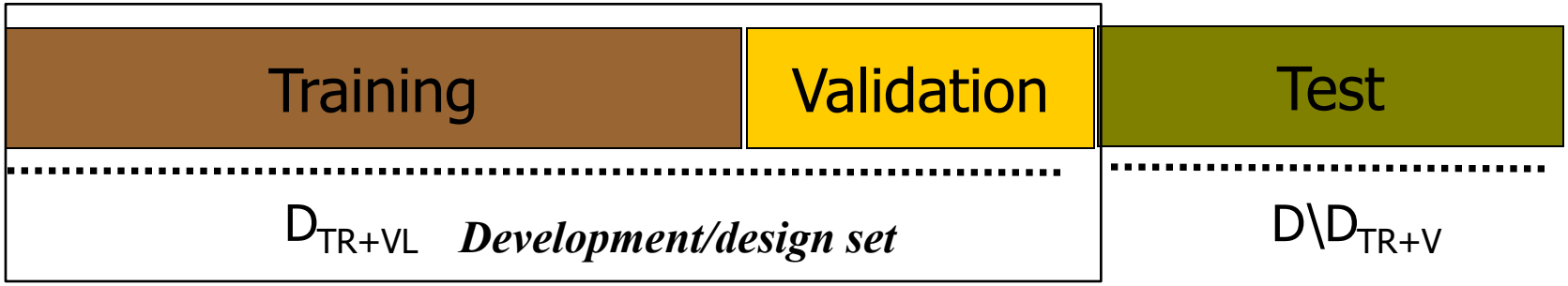
  It returns an estimation value

Gold rule: Keep separation between goals and use separate data sets in each phase

Hold out

Dip. Informatica
University of Pisa

- If data set size is sufficient: e.g. 50% TR, 25% VL, 25% TS **disjoint sets**



- TR: *Training set* is used to fit (to minimize R_{emp} in SLT) [**training**]^{L}
- VL*: Validation set* (or *selection set*) is used to select the best model (among different models and/or hyper-parameters configurations) [**model selection**]
- TR+VL sometimes are joitnly called development/design set , i.e. used to build the final model (training different models and selecting the best final model)
- TS*:Test set* is used for estimation of generalization error (of the final model) (to estimate *R* in SLT) [**model assessment**]

Notes:

1) the estimation made for model selection (on VL set) [is for model selection purpose], it is not a good estimation for the assessment phase/risk test.

2) **Test set <u>results</u>** cannot be used for model selection (or call it validation set)

Test or model selection?

Dip. Informatica
University of Pisa

- What if test set is used in a (repeated) design cycle?
  - We are making *model selection* and not reliable *assessment* (estimation of expected generalization error)
    - and *we wouldn't be able to do that on future examples*.
    - **Blind test set** concept (e.g. for ML competitions)
    - Image an exam exercise: if you see the solutions, it is not a test!
  - In that case, used test set error provides an overoptimistic evaluation of the true test error (→ we will see how easy is to obtain very high classification accuracy over random task even using the test set only implicitly)

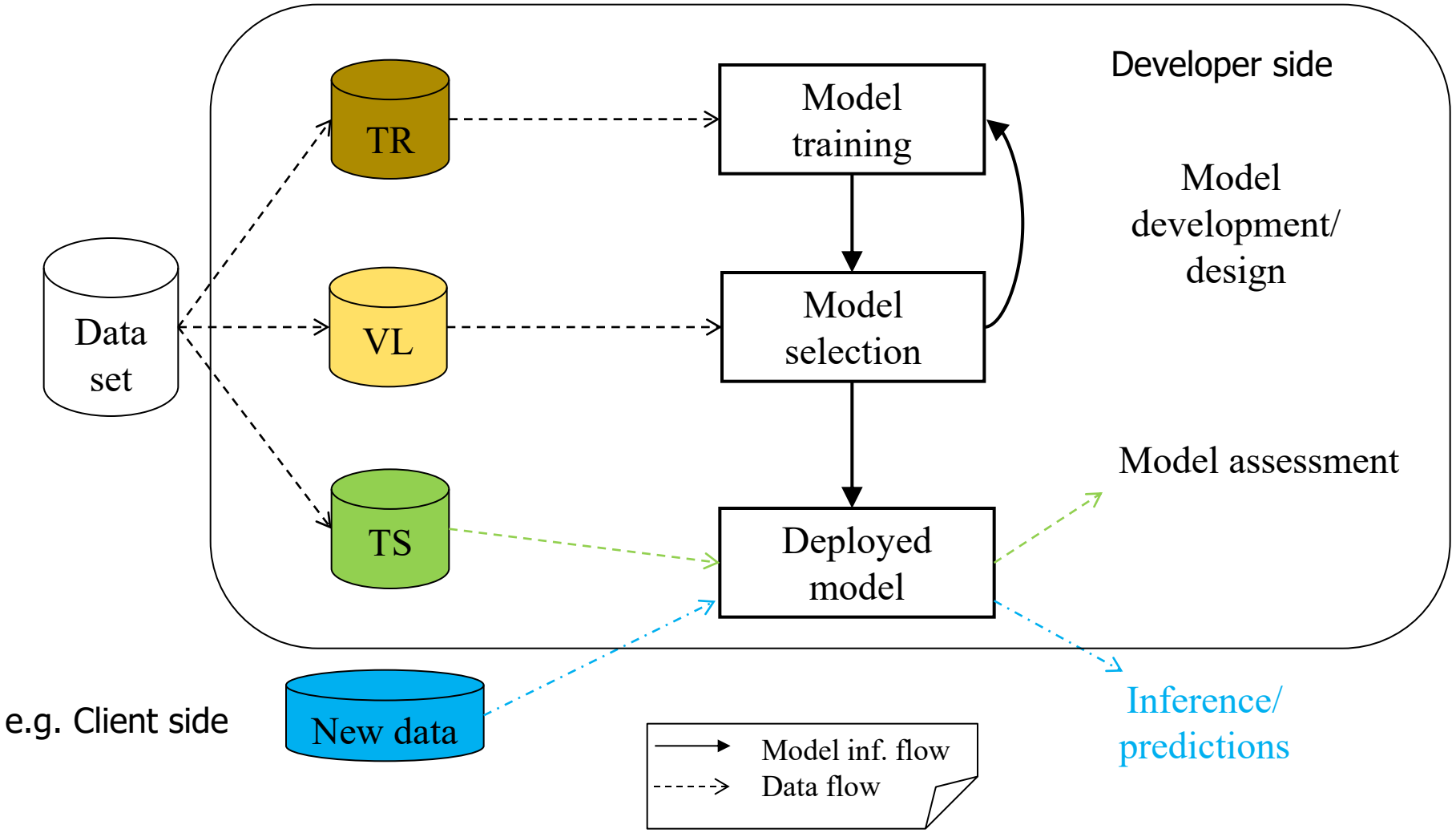
Gold rule:

Keep separation between goals and use separate sets in each phase (TR for training, VL for model selection, TS for risk estimation)



TR/VL/TS by a simple schema

Dip. Informatica
University of Pisa



Counterexample
(relevance of a correct assessment)

Dip. Informatica
University of Pisa

- 20-30 examples, 1000 random value input variables,
- ***Random*** *target* 0/1
- We select 1 model with 1 input var/feature that guess (for chance) 99% (or 100%) on any successive TR and VL and TS set splitting

? Perfect result (a model with accuracy 99%)? What is wrong?

99%/100% is not good estimation of the test error (the right one is 50%)

1) Estimation of error on TR and VL is NOT good risk estimation.

2) Using the entire dataset for feature/model selection prejudice the estimation (biased estimation, called *subset selection bias*)

- Test set was implicitly used at the beginning (*).
- Test set must be separated in advance, before making any model selection (even of the Feature selection) !

  On external test the result is 50% (random coin result)!!!!!

We are discussing the correctness of the estimation, not the possibility to solve the task !

K-fold CV and other techniques does not solve the issue

[but can confuse you ! ;-)]

The table for the Counterexample

Dip. Informatica
University of Pisa

Input variable value

Pattern	1	2	…	26	27	28	…	1000	**Target**
1	…	…	…	1	1	1	…	…	1
2				0	0	0			0
3				1	1	1			1
4				0	0	0			0
…				0	0	0			0
…				1	1	1			1
…				0	0	0			0
20	…	…	…	1	1	1	…	…	1
TS1				1	0	0			1
TS2				0	1	0			0
TS3				1	1	1			1
Accuracy				100%	33%	66%			

Grid search (model selection phase)

Dip. Informatica
University of Pisa

- *Hyper-parameters*. i.e. parameters that are not directly learnt within estimators, can be set by searching a *hyper-parameters* space (of course by model selection on a *validation set*).
  Search best hyper-parameter values.
- **Exhaustive Grid Search**

  exhaustively generates candidates from a grid of parameter values,

  TRIVIAL EXAMPLE:

Hyper-param.	Lambda 0.1	Lambda 0.01	Lambda 0.001
1 unit	Res1	Res4	Res7
10 units	Res2	Res5	Res8
100 units	**Res3**	Res6	Res9

Example: The best one is *Res3* → (Units=100, lambda=0.1) is the winner

AUTOMATIZE IT!!!
Parallelization is easy (independence of trials)

- Alternatives: Randomized Parameter Optimization (*) → see later
- And many others (also in current ML libraries) → see later

Grid search -2

Dip. Informatica
University of Pisa

- The cost of search can be high: Cartesian product between the sets of values for each hyper-parameter
- #(values in the range)$^{\#\text{hyperparameters}}$:
  - e.g. before 3X3=3^2=9
  - What with 5-6 hyperpar.,
    each with 10 possible different values?

Hyper-param.	Lambda 0.1	Lambda 0.01	Lambda 0.001
1 unit	Res1	Res4	Res7
10 units	Res2	Res5	Res8
100 units	Res3	Res6	Res9

- Can be useful to fix some hyper-parameters values in a preliminary experimental phase (since they show to be not relevant) [but take care, see FAQ later on seq. selection error]

Two (or more) levels of **nested grid search:** Useful!

- Apply a first *coarse grid* search to the hyperparameters with a table on the combinations of **all the possible** (e.g. growing exponential) values too find good *intervals* (regions)
- Then *finer* grid-searchs can be performed (over smaller and smaller intervals and with selected hyper-parameters) still using ***all*** the significant hyper-par.

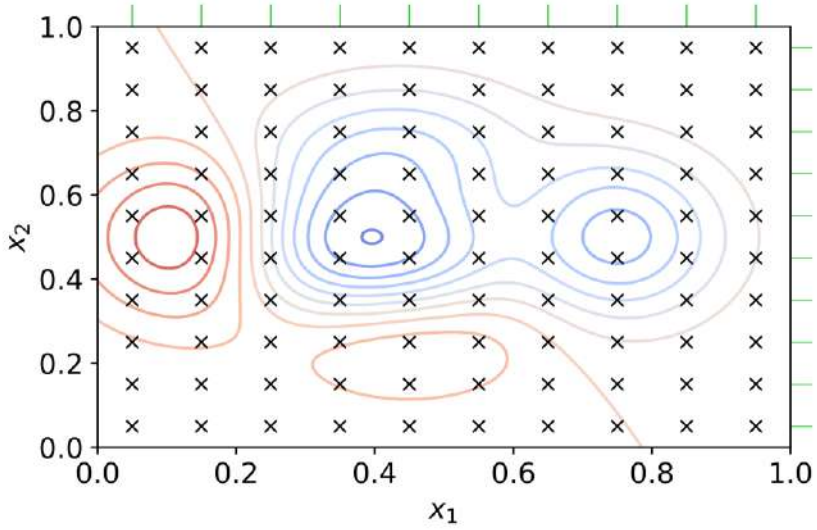
Alternatives to grid search

Dip. Informatica
University of Pisa

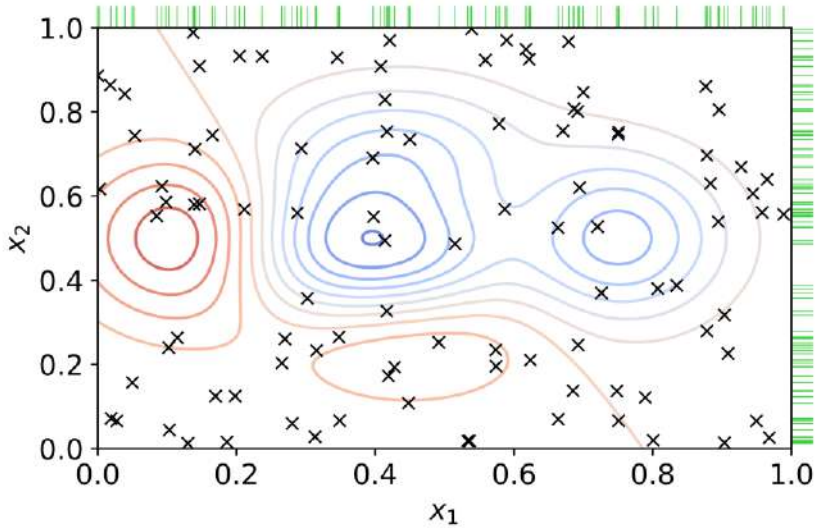
- Grid search is very useful to observe the behavior of the model changing the hyper-parameters values (to explore the hyper-parameter space → very important for your understanding)
  - But it is *computationally demanding* for large spaces and does not scale well with the increase in number of hyperparameters
- Alternatives exist, trying:
  - to reduce the cost: e.g. *random search*, which avoid to resample the same values of influent hyperparameters (in case some other are not-influent), and allows to *fix the budget* of samples independently from the # of hyper-par. See Bergstra, Y. Bengio, (2012). "Random Search for Hyper-Parameter Optimization" J. Machine Learning Research 13: 281–305.
  - to automatize the search: e.g. *Bayesian approaches, evolutionary approaches*, *configuration evaluation methods* (e.g. Hyperband*), et al.
- Current libraries, as Keras tuner for NN and Scikit-Learn, include various alternatives (toward *AutoML*)→ anyway, don't try them uncritically, compare with the grid search that teaches you more for the first approach to ML

Grid versus Random search

Dip. Informatica
University of Pisa



Grid search for 2 hypepar.s.:
a total of 100 (10x10) different combinations are evaluated and compared. Blue contours indicate regions with strong results, whereas red ones show regions with poor results.
ONLY 10 samples for each axis!

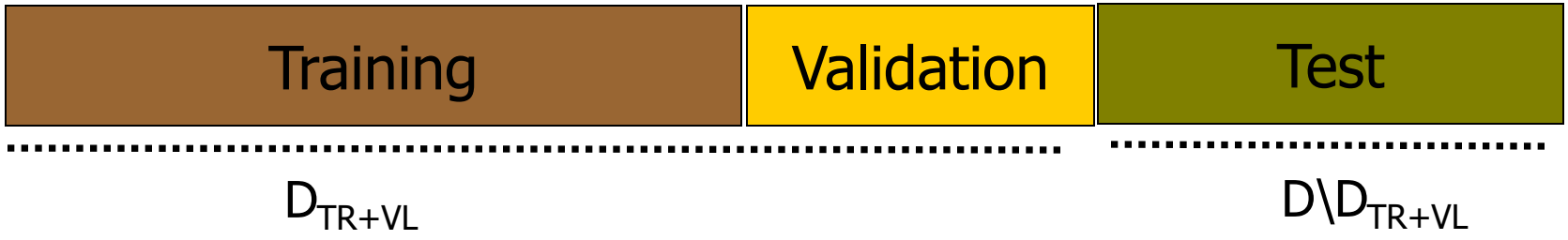


Random search for 2 hypepar.s:
100 different random choices are evaluated. The green bars show that more individual values for each hyperparameter are considered compared to a grid search.
MANY samples for each axis

Hold out - rethinking

Dip. Informatica
University of Pisa

- If data set size is sufficient: e.g. 50% TR, 25% VL, 25% TS



- How much data is enough?
  - It dependes on model complexity, signal-to-noise ratio of the underlying function (e.g. Few for linear model, linear separable task, no noise)
- How to deal with data that are insufficient to make 3 significant/large size splits?
- Can we avoid to be sensible to the particular partitioning of examples?

K-fold CV can help!

Notes and Terminologies

Dip. Informatica
University of Pisa

- Generically we have used the term "*cross validation*" for both:
  - An approach to ***model selection*** based on direct estimation of prediction error **by resampling** (instead of analytical)
    - Choose hyperparameters estimating errors
  - More specifically for implementations of estimation methods with resampling specifying how to divide data for both *model selection and assessment*
    - How to make partitions of the data set : e.g. by **K-fold CV**

K-fold CV recall

Dip. Informatica
University of Pisa



K-fold Cross-Validation

- Split the data set D into k mutually exclusive subsets $D_1,D_2,\ldots,D_K$
- Train the learning algorithm on $D\backslash D_i$ and test it on D_i
- *It uses all the data for training and testing*

- You don't get an unique model (see next lect.) but
- You get also a **variance** (standard deviation) over different folders for your <**class of models/alg**.>

It can be used both for the validation set or for the test set

Fold-1	Res on D_1
Fold-2	Res on D_2
Fold-3	Res on D_3
….	…
TOTAL	**MEAN +/- SD**

Issues:

- How many folds? 3-fold, 5-fold , 10-fold, …., 1-leave-out
- Often computationally very expensive
- Combinable with validation set, double-K-fold CV, ….

An example of model selection and assessment with K-fold CV

Dip. Informatica
University of Pisa

- Split data in TR and Test set (here simple hold-out or a K-fold CV)
- [Model selection] Use K-fold CV (internal) over TR set, obtaining *new* TR e VL set in each folder, to find best hyper-parameters of your model (e.g. polynomial order, λ of ridge regression, #units in a NN, …): **Grid-search** with many possible values of the hyper-parameters
  - i.e. for example a k-fold-CV for (λ =0.1, #units=20, …), a k-fold-CV for (λ =0.01 #units=20), … i.e. k-fold-CV for each cell of the grid matrix, ... and then take the best configuration (λ, #units, …)
    (comparing the mean error computed over the validation sets obtained by the all the folds of each k-fold CV, i.e. the result on the "diagonal")
- Train on the *whole* (original) TR set the final model
- [Model assessment] Evaluate it on the external Test set
  - Question: Can we retrain the model again with all the data?

We will see more combinations in the next lecture!

Before…

Dip. Informatica
University of Pisa

- Before enter in general method for model selection/assessment we face (in brief) some particular cases

Lucky/Unlucky sampling





Dip. Informatica
University of Pisa

- Can we avoid to be sensible to the particular partitioning of examples? (bias of the particular sample)
- *Stratification* procedure:
  - *Stratification* is the process of grouping members of the population into relatively homogeneous subgroups before sampling.
- Classification: for each (random) partition (TR and TS in hold-out/CV) each class is represented in approximately the same proportions as in the full data set
- Folder composition: check if order in data related to specific meaning

- *Repeated* hold-out method or CV: repeat splitting with different random sampling
  - E.g. repeat the CV 10 times !
  - Average the results to yield an overall estimation
  - Useful especially for comparing different models !

Very few data (informal)



Dip. Informatica
University of Pisa

- With very few data: difficult to say whether a sample is representative or not ….
- Stratification
- Avoid (or consider in evaluation):
  - Missing classes or features in training data
  - Special classes of data not sampled in TR or TS (again stratification)
  - Prior known outliers can affect the mean test results
  - On the opposite: selection of only "easy" cases
- Also: blind test set can be misleading if it is
  - From a different distribution
  - Measured with different scale, different tolerance, etc.
  - Uncleaned, unpreprocessed, …
  - Extrapolation (out of range data)

Error Functions for Evaluation

Dip. Informatica
University of Pisa

- Measuring error (addenda to slides in the intro)
- Classification: see first lessons:
  - Accuracy (correct classification rate/error rate often in %), confusion matrix (specificity, sensitivity, ROC curve)
- Regression: residue $r_i=(y_i-o_i)$ $y = target$
  - MS error ($mean_i\ [r^2]$)→ RMS (root mean square) or S
  - Mean absolute error (mean of the absolute error |r|)
  - Max absolute error ($max_i\ [|r_i|]$)
  - R (correlation coefficient/index) (R^2 coeff. of determination):

  E.g. $R=sqrt(1- (S^2/S_y^2))$ where $S_y^2=mean_i\ [\ (y_i-mean_i\ [y])^2\ ]$ *(variance of y)*
    - degree of linear dependence between the variables y e o in [0-1], where 1 is the best
  - Outputs versus targets plots
  - Various statistical tests and statistical significance analysis

Other approaches



Dip. Informatica
University of Pisa

- Bootstrap: random resampling with replacement and repeat it to from different subsets (vl or ts)

REPETITA: 2 gold rules

Dip. Informatica
University of Pisa

- TR result is not a good estimation of VL and TS result
- VL result is not a good estimation of TS result

Hence:

- Do **not** use validation set results for test estimation [*risk estimation/model assessment*]
- Do **not** use test set results for *model selection* (in any form*!!!) (or call it validation set)

Or even **1 rule**:

Keep separation between goals and use separate sets in each phase



In the following

Dip. Informatica
University of Pisa

- Correct use of cross-validation methods for model selection and/or estimation of risk (assessment)
- Simple concepts with a rich formal notations, can help to avoid ambiguities:

- Very useful guide. It is not intended as a recipe book but help you to rationalize, to understand in a systematic form issues of a rigorous approach.
- Then you have to be *reasonable (when you are aware*)
- In any case use them if you have uncertainties, it is better than to use *fantasy*!

Exercise (in the slide of the next lecture):
relate wrong cases 1 and 2 of slide "counterexample" to misuses of such schemas!

Model Assessment (Evaluation/Validation) and Model Selection Part 2

- Correct use of cross-validation methods for model selection and/or estimation of risk (assessment)
- Simple concepts with a rich formal notation, they can help to avoid ambiguities:
- Very useful guide. But it is not intended as a recipe book but to help you to rationalize, i.e. to understand in a systematic form issues of a rigorous approach.
- Then you have to be *reasonable (when you are aware*) (which effort deserve? Most of the *time* for running the experiments depends on the validation choices! We have to avoid both a too rough evaluation and a never ending evaluation process)
- In any case use them if you have uncertainties, it is better than to use *fantasy*!
- Note that we will discuss selection among different hyper-parameters configurations (represented by θ), but also for selecting among *different models* you have to properly apply "model selection" and "model assessment" approaches (think θ to represent also different models).

Memento: our exercise:

1) Estimation of error on TR and/or VL are NOT a good risk estimation.

2) Using the entire dataset for feature/model selection prejudices the estimation

Credits: by slides of S. Bengio, adapted and integrated by A. Micheli with the notation and figures for our course

Premise (refresh and slight changes in notations**)**

The data:

- Let $Z_1, Z_2, \dots, Z_l$ be a l-tuple random sample of an unknow distribution $p(z)$
- All the Z_i are independently and identically distributed (i.i.d.)
- Let D_r be a particular instance $= \{z_1, z_2, \dots, z_r\}$

Tasks:

- Classification: $Z = (X, Y) \in \mathbb{R}^n \times \{-1, +1\}$
- Regression: $Z = (X, Y) \in \mathbb{R}^n \times \mathbb{R}$

Loss: (to search for a good h in a function space H) (we called it "inner" loss function)

- Classification: $L(z, h) = L\big((x, y), h\big) = 0 \;\; if\; h(x) = y\,;\; 1\; otherwise$
- Regression: $L(z, h) = L\big((x, y), h\big) = (h(x) - y)^2$

Aim: Minimize the expected risk (true error) on H, defined for a given h as

$$R(h) = E_Z[L(z, h)] = \int_Z L(z, h)p(z)dz$$

- Select $h^* = \arg\min_{h \in H} R\,(h)$
- Problems: $p(z)$ is unknow, and we don't have access to all $L(z, h)$

Empirical Risk : $R_{emp}(h, D_r) = \frac{1}{r}\sum_{p=1}^{r} L\big(z_p, h\big) = \frac{1}{r}\sum_{i=1}^{r} L(z_i, h)$
seen therein in a general sense as a *surrogate of R* on a finite set of data (D_r) with *r* data, within the purpose of training, model selection or model assessment as specified in the following algorithms, according to the used partition of data. *Don't* confuse it with the use in the SLT as training error, whereas, in SLT presentation, $R_{emp} = R_{emp}(h, D_{TR})$ according to the notation of this document.

Methodology

Identify the goal. It could be:

1. To give the best model you can obtain given a (*training*) data set?
2. To give the expected performance of a model obtained by the Empirical Risk Minimization given a (*training*) data set?
3. To give the best model and its expected performance that you can obtain given a (*training*) data set?

For (1): you need to do model selection
For (2): you need to do model assessment (i.e. to estimate the risk)
For (3): you have to do both

We will systematically see in the following the different methods

Note: the provided original *training* data set will be used *by us* for the 3 different purposes

Model Selection: Hold-out Validation

- Select a hypothesis space with hyper-parameter θ
- Divide your dataset (training set) D_l in two parts
  - $D_{TR} = \{z_1, z_2, \dots, z_{tr}\},\ D_{VL} = \{z_{tr+1}, z_{tr+2}, \dots, z_{tr+vl}\},\ tr + vl = l$
- For each value θ_m of the hyper-parameter θ [grid search]
  - select $h^*_{\theta_m}(D_{TR}) = \arg\min_{h \in H_{\theta_m}} R_{emp}(h, D_{TR})$ [training]
  - estimate(#) $R\left(h^*_{\theta_m}\right)$ with $R_{emp}\left(h^*_{\theta_m}, D_{VL}\right) = \frac{1}{vl}\sum_{z_i \in D_{VL}} L\left(z_i,\ h^*_{\theta_m}(D_{TR})\right)$
- select $\theta^*_m = \arg\min_{\theta_m} R\left(h^*_{\theta_m}\right)$ [best on VL set]

Note: it does **not** return this value as estimation for test [(case 1) in the exercise]

- return $h^*(D_l) = \arg\min_{h \in H_{\theta^*_m}} R_{emp}(h, D_l)$ [retraining on l data]

(#)This estimation of R is only for model selection purpose (see note at page 2)

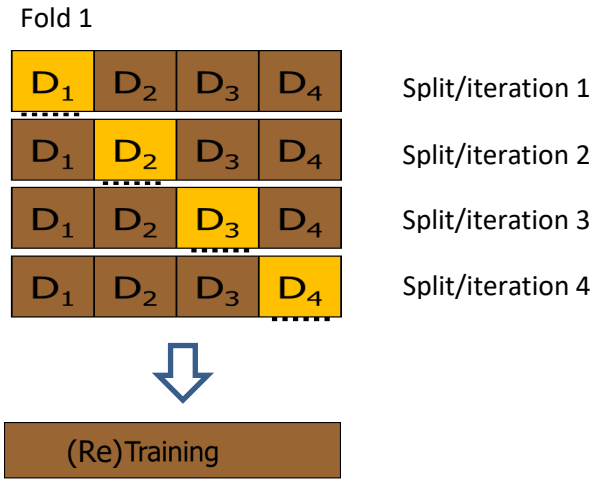


Model Selection: Cross Validation (K-fold CV)

- Select a hypothesis space with hyper-parameter θ
- Divide your dataset (training set) D_l into K distinct and equal parts $D_1, \ldots, D_k, \ldots, D_K$
- For each value θ_m of the hyper-parameter θ [grid search]
  - For each part D_k (and its counterpart $\bar{D}_k$) [folds cycle]
    - select $h^*_{\theta_m}(\bar{D}_k) = \arg\min_{h \in H_{\theta_m}} R_{emp}(h, \bar{D}_k)$ [training]
    - estimate $R\left(h^*_{\theta_m}(\bar{D}_k)\right)$ with $R_{emp}\left(h^*_{\theta_m}(\bar{D}_k), D_k\right) = \frac{1}{|D_k|}\sum_{z_i \in D_k} L\left(z_i, h^*_{\theta_m}(\bar{D}_k)\right)$
  - estimate(#) $R\left(h^*_{\theta_m}(D_l)\right)$ with $\frac{1}{K}\sum_k R\left(h^*_{\theta_m}(\bar{D}_k)\right)$
- select $\theta^*_m = \arg\min_{\theta_m} R\left(h^*_{\theta_m}(D_l)\right)$ [best on all θ_m values on the CVs]
- return $h^*(D_l) = \arg\min_{h \in H_{\theta^*_m}} R_{emp}(h, D_l)$ [retraining on l data]

Note the ORDER: Each iteration starts from scratch with training and estimation local to the split (else e.g. using a model trained on split 1 for split 2: you are using VL of split 2 that was used in training in split 1: the part D_2)

(#)This estimation of R is only for model selection purpose (see note at page 2)



Estimation of the Risk (model assessment): Hold-out

Note the ORDER:
the test set is separated NOW and not below: [(case 2) in the exercise]

- Divide your dataset (training set) D_l in two parts
  - $D_{TR} = \{z_1, z_1, \dots, z_{tr}\},\ D_{TS} = \{z_{tr+1}, z_{tr+2}, \dots, z_{tr+ts}\},\ tr + ts = l$
- select $h^*(D_{TR}) = \arg\min_{h \in H} R_{emp}(h, D_{TR})$ [training]

Note: it does not return here an estimation for test, but below [(case 1) in the exercise]

(this optimization process could include a model selection on the D_{TR} part)

- estimate $R\left(h^*(D_{TR})\right)$ with $R_{emp}(h^*(D_{TR}), D_{TS}) = \frac{1}{ts}\sum_{z_i \in D_{TS}} L\left(z_i,\ h^*(D_{TR})\right)$ [test error]



Estimation of the Risk (model assessment): Cross-validation (K-fold CV)

- Divide your dataset (training set) D_l into K distinct and equal parts $D_1, \ldots, D_k, \ldots, D_K$
  - For each part D_k [folds cycle]
    - let the counterpart $\overline{D}_k$ be the set of examples that are in D_l and not in D_k
    - select $h^*(\overline{D}_k) = \arg\min_{h \in H} R_{emp}(h, \overline{D}_k)$ [training]

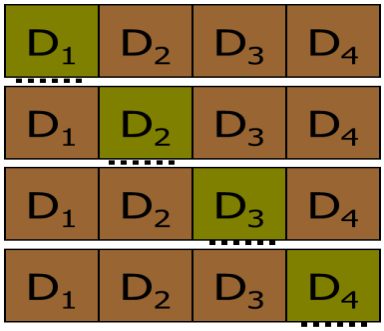
      (this process could include a model selection)

    - estimate $R\left(h^*(\overline{D}_k)\right)$ with [test error in each iteration]

      $$R_{emp}(h^*(\overline{D}_k), D_k) = \frac{1}{|D_k|} \sum_{z_i \in D_k} L\left(z_i, h^*(\overline{D}_k)\right)$$
  - estimate $R\left(h^*(D_l)\right)$ with $\frac{1}{K}\sum_k R\left(h^*(\overline{D}_k)\right)$ [the mean over the folds]

Notes: when $K = l$: *leave-one-out* CV

We do not violate the principia: the test set results are never used for training or model selection in any K-fold CV splitting/iteration



Merge: Model selection + Model assessment (estimation of the Risk)

- When you want both the best model and its expected risk.
- You then need to merge/combine the methods already presented.
- For instance:
  - hold out with training-validation-test: 3 separate data sets are necessary
  - cross-validation + test: cross-validate on training set, then test on separate set
  - external CV for test and internal hold out for the model selection
  - double-cross-validation: for each subset (of a K-fold CV), need to do a nested cross-validation with the K − 1 other subsets

- Other aspects:
  - take care to consider the different trials (different weigths *init* for a NN) when computing the mean values for TR, VL, TS

- More general methodological aspects:
  - compare your results with other methods
  - use statistical tests to verify significance
  - verify your model on more than one datasets

Merge: Training – Validation – Test (Hold Out)

- Select a hypothesis space with hyper-parameter θ
- Divide your dataset (training set) D_l into three parts D_{TR}, D_{VL}, D_{TS} , $tr + vl + ts = l$
- For each value θ_m of the hyper-parameter θ [grid search]
  - select $h^*_{\theta_m}(D_{TR}) = \arg\min_{h \in H_{\theta_m}} R_{emp}(h, D_{TR})$ [training]
  - let $R_{emp}\left(h^*_{\theta_m}(D_{TR}), D_{VL}\right) = \frac{1}{vl}\sum_{z_i \in D_{VL}} L\left(z_i,\ h^*_{\theta_m}(D_{TR})\right)$

- select $\theta^*_m = \arg\min_{\theta_m} R_{emp}\left(h^*_{\theta_m}(D_{TR}), D_{VL}\right)$ [best on VL set]
- select $h^*(D_{TR} \cup D_{VL}) = \arg\min_{h \in H_{\theta^*_m}} R_{emp}(h, D_{TR} \cup D_{VL})$ [retraining]

- estimate $R\left(h^*(D_{TR} \cup D_{VL})\right)$ with $\frac{1}{ts}\sum_{z_i \in D_{TS}} L\left(z_i,\ h^*(D_{TR} \cup D_{VL})\right)$

- Question: Can we now retrain the model again with all the D_l data? Does it use the test results?

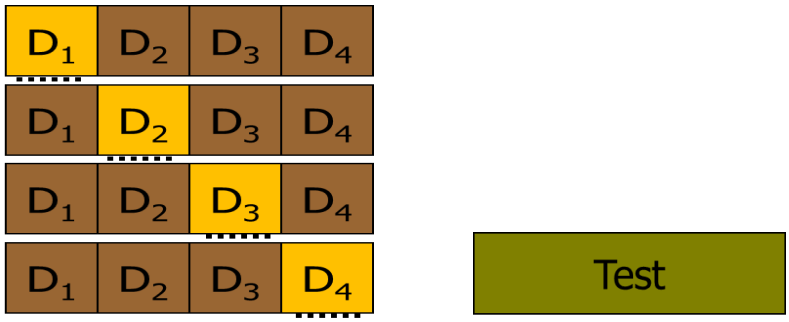


(retraining is not shown, it can have a VL set for the Early Stopping)

Merge: K-fold CV for model selection + Hold Out Test

- Select a hypothesis space with hyper-parameter θ
- Divide your dataset D_l in two parts D_{TR} and D_{TS}, $tr + ts = l$
- For each value θ_m of the hyper-parameter θ [grid search]
  - estimate(#) $R\left(h^*(D_{TR})\right)$ with D_{TR} using a K-fold CV
- select $\theta_m^* = \arg\min_{\theta_m} R\left(h_{\theta_m}^*(D_{TR})\right)$ [best from the K-fold CVs]
- select $h^*(D_{TR}) = \arg\min_{h \in H_{\theta_m^*}} R_{emp}(h, D_{TR})$ [retraining]
- estimate $R\left(h^*(D_{TR})\right)$ with $\frac{1}{ts}\sum_{z_i \in D_{TS}} L\left(z_i,\ h^*(D_{TR})\right)$

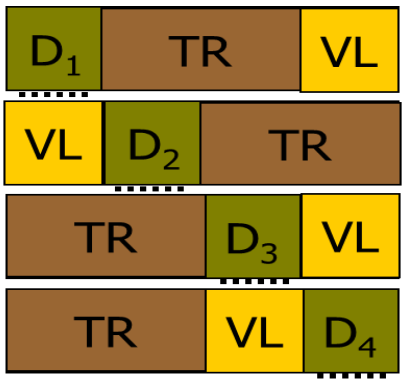
(#)This estimation of R is only for model selection purpose (see note at page 2), and this is the validation error over the K-fold CV on the "design set" called therein D_{TR}



(retraining is not shown, it can have a VL set for the Early Stopping)

Merge: K-fold CV for Test with internal Hold-out for model selection

The description by the meta algorithm can be an Exercise



(retraining is not shown)

The internal TR/VL Hold Out in each raw can be with arbitrary partitions and with shuffling (the figure is just illustrative)

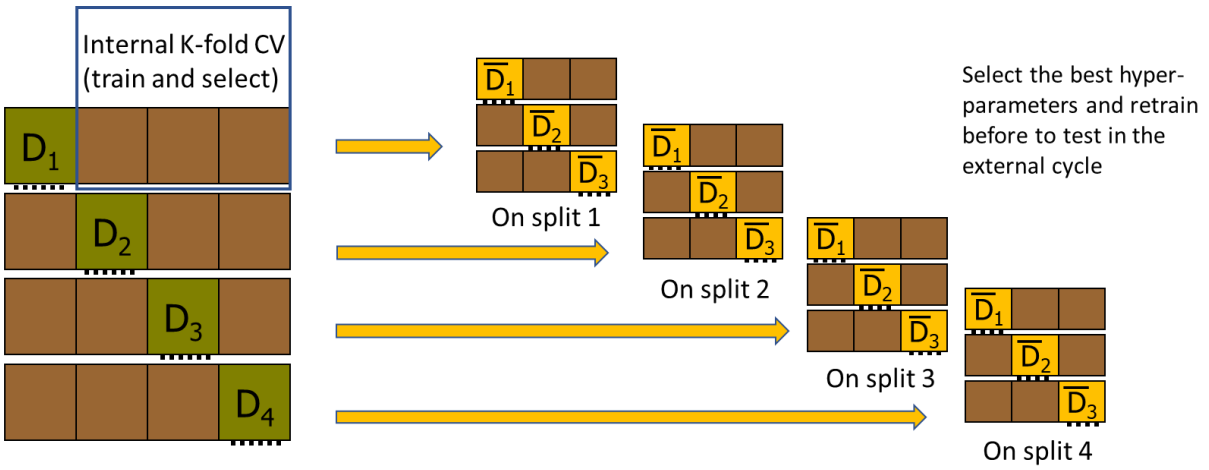
Note: this process **does not** provide a final model yet, it only gives an estimate of the risk (of the class of the considered models), but not a unique model (because you potentially have a different model for each raw). See the note in the following page.

Merge: Double K-fold CV (also known as Nested K-fold CV)

- Select a hypothesis space with hyper-parameter θ
- Divide your dataset (training set) D_l into *K* distinct and equal parts D_1, …, D_k, … , D_K
- For each part D_k (and its counterpart $\bar{D}_k$) [*external cycle*]
  - select $h^*(\bar{D}_k)$ by an internal K-fold CV on $\bar{D}_k$ (with *K'* that can be ≠ *K*) [*inner cycle*]
  - estimate $R\left(h^*(\bar{D}_k)\right)$ with [test error for each external split]

$$R_{emp}(h^*(\bar{D}_k), D_k) = \frac{1}{|D_k|} \sum_{z_i \in D_k} L\left(z_i, h^*(\bar{D}_k)\right)$$

- estimate $R\left(h^*(D_l)\right)$ with $\frac{1}{K}\sum_k R\left(h^*(\bar{D}_k)\right)$ [the mean over the test folds]



(retraining in each internal cycle is not shown)

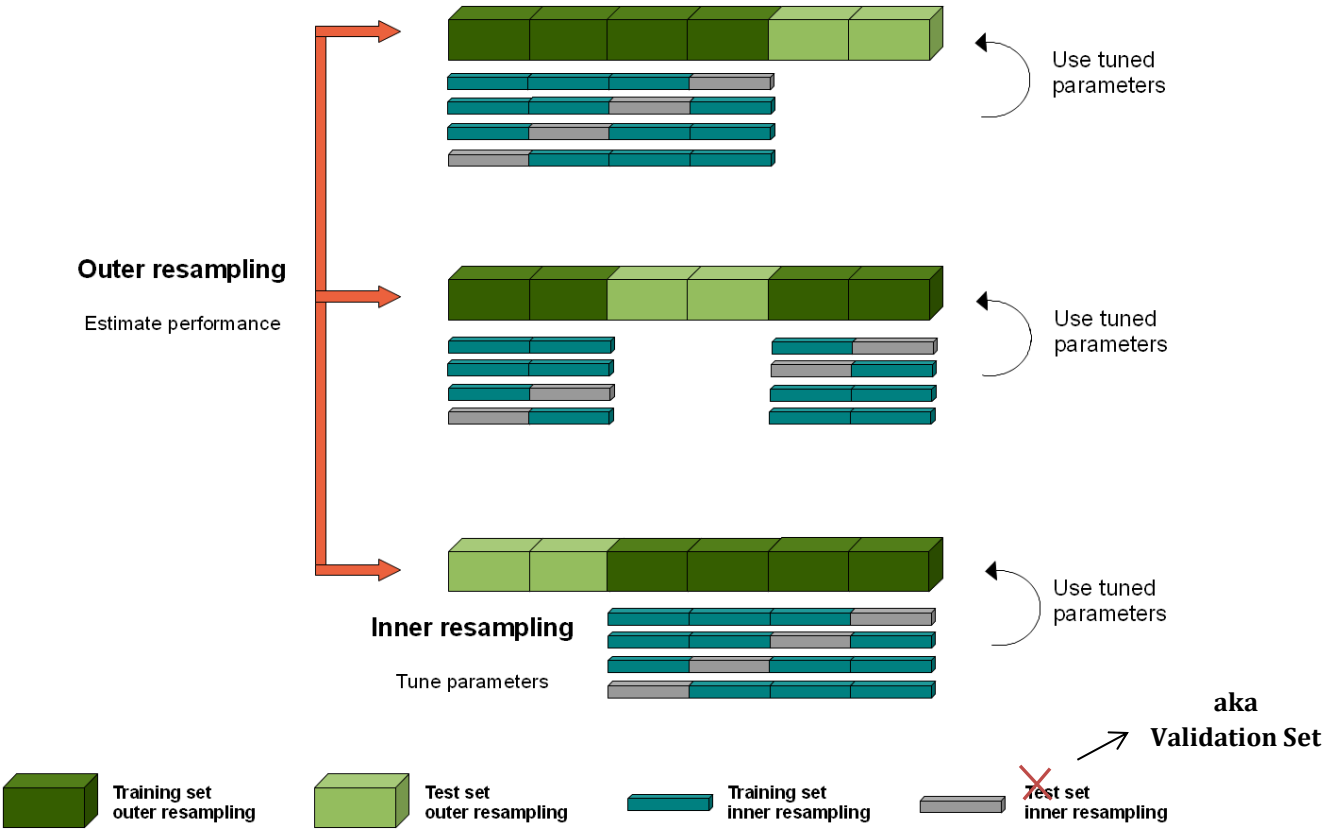
Note: this process **does not** provide a final model, it *only* gives an estimate of the risk, but not a unique model, because you potentially have a different hyperparameters for each external cycle step (external split).

If you need a final unique model, you can perform a **separate** model selection process (hold out or K-fold CV)! And a possible final retraining.
This approach does not violate the rules; the *test error has been already estimated* above (for the class of model/algorithm), with also the estimation of the variance (standard deviation - std) through the folds. We are <u>not</u> (and never) using the <u>test results</u> for any model selection, and the model will have an expected error within the estimated interval.

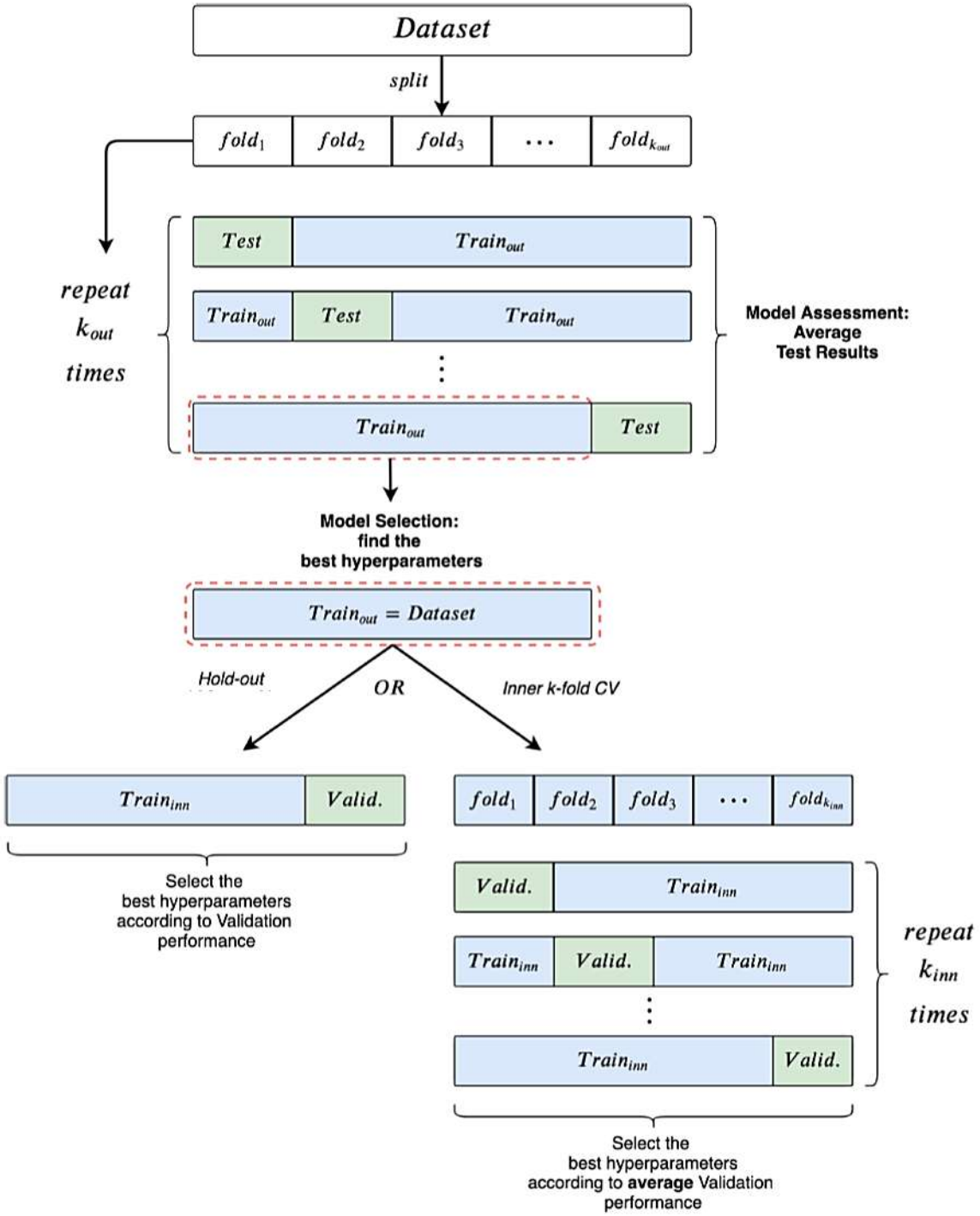
Moreover, note that for this reason it is not simply to say that the test happens *always* as a final step, in this case it is before the last model selection phase (even if it can appear a bit anti-intuitive).

Finally, note that the double CV makes *better exploitation of all the data* for training and evaluation (both for selection and assessment aims) even if at the cost of a *high computational effort*!

Double CV (a different view)

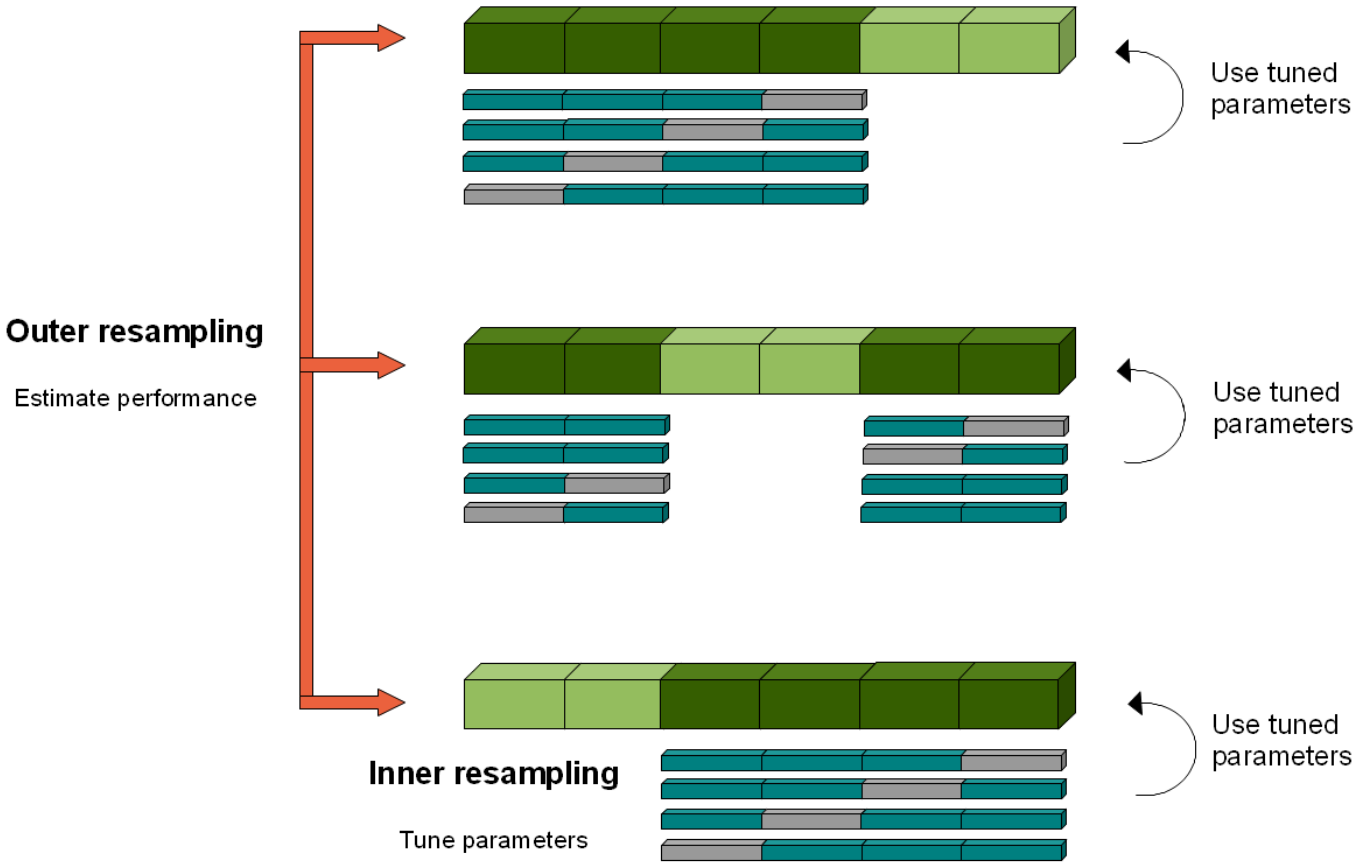


Nested/Double CV (a further different view, also with the inner Hold Out case)



Double CV (a different view)

Dip. Informatica
University of Pisa



Special cases: *Stopping where?* (FAQ/Frequent Error)



Dip. Informatica
University of Pisa

Avoid to stop the NN training **fixing an arbitrary number of epochs**
Reasons:

- If it is **small** it may be *too early* (underfitting or just in favor of high eta that need less epochs) or
- if it is **large** it may be *too late* (overfitting or losing time),
- It cannot hold the same value for **all the configurations** and iteration of a K-fold CV, ...
- And what number to use if you **retrain** the model at the end?

- Also selecting the number epochs by *model selection* it is not the best practice: it is a bit better than a fixed number but still *not* accurate, considering the K-fold CV issue and that we have **many other criteria**:
  - See the slides on "Stopping criteria" in the NN-part2 file, where we discussed solutions that works well with a *well regularized system* e.g. looking to the training convergence, i.e. using a threshold for training error decrease or on gradient value, etc. or the *Early Stopping* (ES) if you can/want use a VL set (see also the next slide)

Special cases: *Early Stopping* (FAQ/Frequent Error)



Dip. Informatica
University of Pisa

- **E.S. criteria** should be part of the **model selection** as well (**not** on the TS set, in principle it needs of a **VL set** anytime you train the model).
- It is tricky as it is a strategy that uses part of the data just to *decided when to stop*, without giving a concrete value of a hyperparameter, and it can be complex within CV!
- **Problem**: how to select the best model with ES for the final retraining? (e.g. after a CV you have a different stop point for each validation set)
  - **If** you consider the #epochs a possible hyperpar. (E.g. Mitchell book p.112), you can take the mean #epochs over the folds. **Issue**: varying the data (size) for retraining after model selection could lead to a different (best) stop point! What do you risk? *
  - So, much BETTER, if you consider the *mean of TR error* at the best point for VL and use it to gain the same level of fitting in the retraining (avoiding undertraning effects).
  - **Else** you need a VL set anytime you are training the model.
  - For instance: CV, ES activated for each validation fold (together with the other hyperpar.). After you get the best hyperparameters values estimation, you retrain the model over the recomposed dataset still using part of the data to find the best for ES (VL set again).
- It is your choice: Using ES or prefer to relay on regularized models with standard stop criteria (e.g. no improvement in training). In any case regularization helps to avoid high overtraining and the need of strict dependences on ES.

Special cases: *Random init.* (FAQ/Frequent Error)



Dip. Informatica
University of Pisa

Random weight initialization (MLP) and model selection

- Different $\boldsymbol{W}_{\text{initial}}$ → different models
- **Problem**: how to choose one for the final model**?** *(and you must report the final model choice in the report)*
- You compute the *mean* and *variance* of error/accuracy for the different trials→ a random case will be in this range. Hence, the risk is under control unless we get high variance! (but high variance is bad in general - see bias/variance lecture)
- You can take all of them (see "Multiple minima" slide and *ensemble approaches*, see again bias/variance lecture)
- If you chose one of them: it is a *model selection* → you need a VL set
  - the best (i.e. with minimum error on the VL set)
  - the median (to use more "cautious")
  - random (in that case you don't need a VL set, see above)
    - you can also avoid unlucky cases just looking to training error
- More sophisticated approaches are not central for this class.

Other Typical *errors:* *Sequential selection*



Dip. Informatica
University of Pisa

- **Sequential selection** of the hyperparameters: the order introduce a bias. E.g. eta on 10 units used also for 100 units (!!!)
- After initial trials to determine the right *range/interval* (e.g. by looking to stability of training), the most influential hyperparameters (e.g. looking to variance of results) , etc.,
  the search should be performed **on a grid** with *all the combinations* of the hypar. (of course higher #hyper. and #values → larger table!).
- The grid-search can be repeated to refine the search in a *smaller range* found in the previous grid-search (**nested** grid searches, from coarse to fine grain) so to reduce the effort, but **all** the significant hyper. must be considered together (to account the cross effects).

- But….Think, select relevant aspects, don't try with (just) automatic systematic expensive approach.
- Time constraints of the prj should force you to search for basic and fast schema, while still correct /reasonable.

FAQ on validation/assessment: the final reflection (repetita)



Dip. Informatica
University of Pisa

What to do after model selection and model assessment ?

Can I decide to make a new design on such basis **?**

- ***Now*** you know this is *not* the correct way, it is not really advantageous (unless you will have new data to make a new test set)
  - Repeating this approach with the same data you can achieve any result that you like **fitting this data set** (your model can be perfect on it and not in the future use, you are not estimating the risk).
  - In other words: We shouldn't manually or automatically tune our predictor based on the test error, because we wouldn't be able to do that on future examples.
  - ***E.g. Blind test*** in competition does not allow you to check and make redesign !
  - But your model assessment (internal test set) is useful to check your ***approach***!
- It is well-know the people are tempted to decide on the basis of the results, but this is a "psychological" problem outside of ML methods ;-)

  At least be aware that you are doing it (and be reasonable) **OR better ….:**

❖ If you like to make a good model you have to make a good model selection

❖ If you like to estimate the real performance you must make rigorous test estimation.

Overfitting CV (repetita but with CV)

Dip. Informatica
University of Pisa

Intensive use of cross validation can overfit? Yes.

Also does it retain previous issues? An example on **feature selection (FS)**

- Imagine a dataset with 20-30 records and 1000 attributes.
- You try 1000 linear regression models, each one using one of the attributes.
- The best of those 1000 looks good! (validation set)
- But you realize it would have looked good even if the output had been purely random!

- Or [Wrong use] image to approximate a random target finding less then the true error rate 50%(say 3%) **by k-fold CV** (for testing) *after* the FS by best correlation with targets
- Why? Selection were made on the basis of all the examples
- What can be done about it? You know:"left out" samples before selecting !
  - Hold out an additional test set *before* doing any model selection.
  - Check the best model performs well even on the additional test set.

  Model selection CV: Use "CV+test" or "double-CV" (see prev. slides: make a new **selection for each fold**)

  Assessment CV [vs wrong use]: FS for each fold: reliable test estimation:
  - You will achieve a (correct) average of ~50% for the random task

Which kind of CV?

Dip. Informatica
University of Pisa

	Downside	Upside
Test-set	Variance: unreliable estimate of future performance Lucky/unlucky	Cheap
Leave-one-out	Expensive. Has some weird behavior	Doesn't waste data
10-fold	Wastes 10% of the data. 10 times more expensive than test set	Only wastes 10%. Only 10 times more expensive instead of R times.
3-fold	Wastier than 10-fold. Expensivier than test set	Slightly better than test-set
R-fold	Identical to Leave-one-out	

Where $R = l$, i.e. the number of data

Model Selection: example 1 Different algorithms/models

Dip. Informatica
University of Pisa

i	f_i	TRAINERR	10-FOLD-CV-ERR **VL**	Choice
1	f_1			
2	f_2			
3	f_3			⊠
4	f_4			
5	f_5			
6	f_6			

- Whichever model class gave best CV score:

train It with all the data, and that's the predictive model you'll use.

- Note: the best is chosen looking at the **validation** set result, **NOT** considering (or averaging with) the **TR** error !!!

Model Selection: example 2 # hidden units in 1HL-NN

Dip. Informatica
University of Pisa

- One hidden layer NN

Algorithm	TRAINERR	10-FOLD-CV-ERR **VL**	Choice
0 hidden units			
1 hidden units			
2 hidden units			⊠
3 hidden units			
4 hidden units			
5 hidden units			

- Whichever model class gave best CV score:

train it with all the data, and that's the predictive model you'll use

- Note; the search can be for 10, 20, 30 units… (or other step size)

Model Selection: example 3 K-NN

Dip. Informatica
University of Pisa

- Compute LOOCV error for six different model Classes
- (why LOO?)

Algorithm	TRAINERR	10-fold-CV-ERR **VL**	Choice
K=1			
K=2			
K=3			
K=4			⊠
K=5			
K=6			

Model Selection: example 3
K-NN: details



Dip. Informatica
University of Pisa

- Example: Choosing "k" for a k-nearest-neighbor regression.
- Step 1: Compute LOOCV error for six diffe
  classes:

Algorithm	TI
K=1	
K=2	
K=3	
K=4	
K=5	
K=6	

Why did we use 10-fold-CV for neural nets and LOOCV for k-nearest neighbor?

The reason is Computational. For k-NN (and all other nonparametric methods) LOOCV happens to be as cheap as regular predictions.

And why stop at K=6

No good reason, except it looked like things were getting worse as K was increasing

Are we guaranteed that a local optimum of K vs LOOCV will be the global optimum?

Sadly, no. And in fact, the relationship can be very bumpy.

What should we do if we are depressed at the expense of doing LOOCV for K= 1 through 1000?

Idea One: K=1, K=2, K=4, K=8, K=16, K=32, K=64 … K=1024

Idea Two: Hillclimbing from an initial guess at K

Conclusions on practical issues

Dip. Informatica
University of Pisa

In practice:

- Simple Hold Out is widely applied

  Sometimes the test set is already given separately or "taken" by previous applications
  - See Warnings for few data
  - Check your TR and VL error rates

- 5 or 10-fold CV are recommended as a good compromise (among bias and variance of estimation) with respect to LOO (Hastie et al. and the references therein)

- We continue with an analytic approach to approximate the estimation step
- Interesting insights for model selection/comparison (relative size of error is what matters) instead of pure empirical evaluation

Next Lecture

Dip. Informatica
University of Pisa

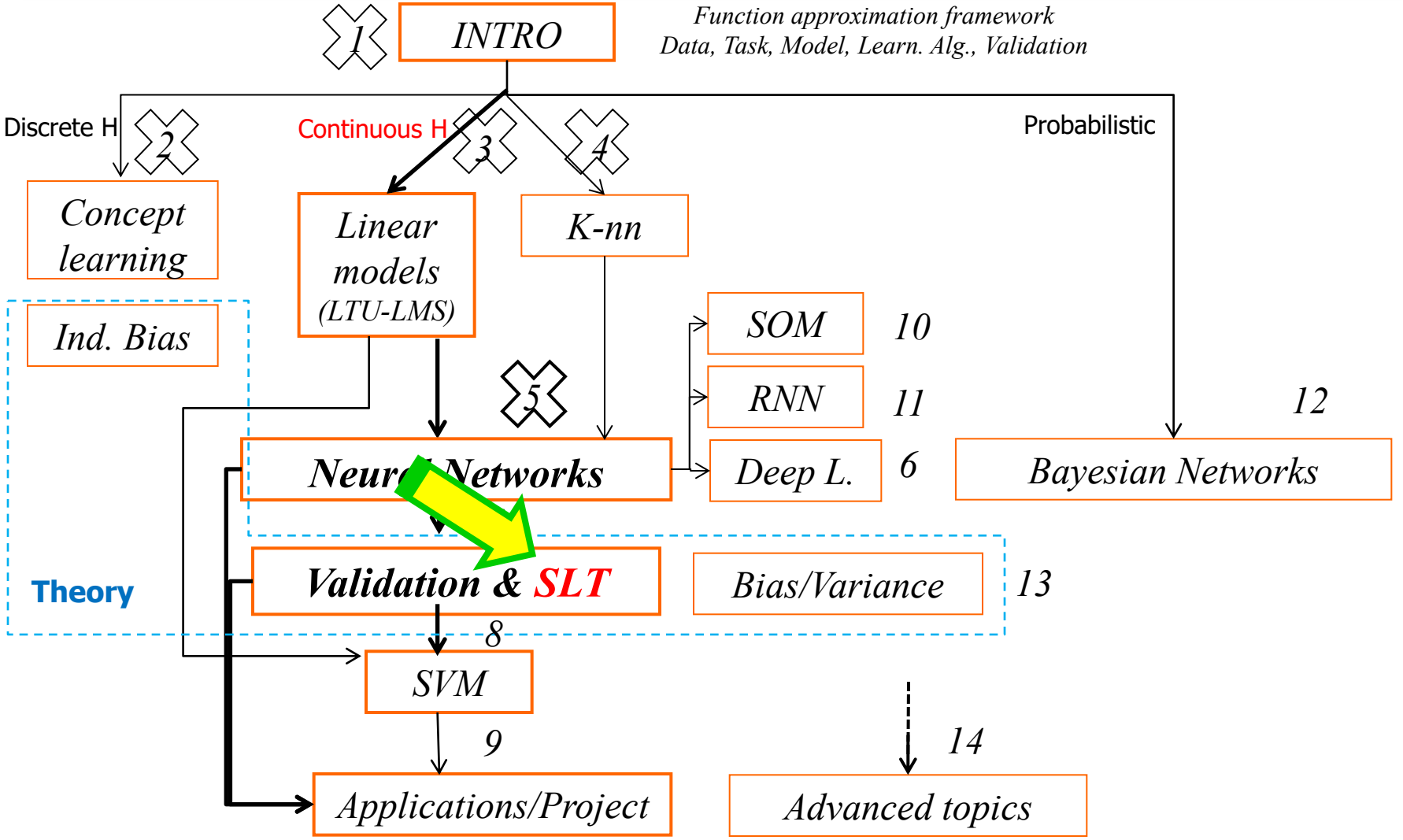
Let us now go back to the analytical estimation of the expected error (validation or test error)

SRM: **Structural Risk Minimization & VC-dimension**

Please, in the while have a look back to the lect. 4 on SLT (in the introduction of the course)

ML Course structure
Where we go

Dip. Informatica
University of Pisa



Approximating estimation step

Dip. Informatica
University of Pisa

We can approximate the estimation of validation set error (for model selection) or test set error (for model assessment)

- Analytically:
  - AIC, BIC Akaike/Bayesian Information Criterion
    - limited to linear models (in par.)
  - MDL
  - SRM: **Structural Risk Minimization & VC-dimension**
- By resampling: direct estimation of extra-sample error via
  - cross-validation
  - bootstrap

VC-dim

Dip. Informatica
University of Pisa

- To provide the analytical bound we need to evaluate H
- A measure of complexity (capacity/expressive power/flexibility) of a class of hypotheses (also for infinite H):
  - **V**=Vapnik
    C=Chervonenkis

- Not by the |H| but instead by the number of distinct instances that can be completely discriminated using H
- Intuitively, restricting to the classification case:

  how much H can discriminate points ?
  - max number of points that can be correctly classified / learned without error for all the possible labelings

Shattering (it: frammentazione)

Dip. Informatica
University of Pisa

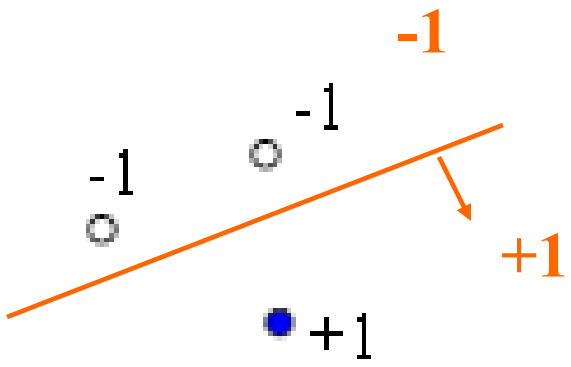
- X: instance (or input) space
- Assume N the size of (num. of instances in the) instance space
  (instead of l used for the num. of data, which can be a different set in concrete cases)
- H: hypothesis space
- Binary classification
- There are 2^N possible **dichotomies** (partitions or labeling of the N points by –1 or +1)
- A particular dichotomy is **represented** in H if there exists a hypothesis h in H that realizes the dichotomy

- **Def: H shatters X iff H can represent all the possible dichotomies on X** (0 errors)
  - the points in X can be separated by an h in H in all the possible ways
  - for every possible dichotomy of X, there exists a consistent hypothesis h in H

Example: dichotomy and H

Dip. Informatica
University of Pisa

- 3 (M) points in IR^2

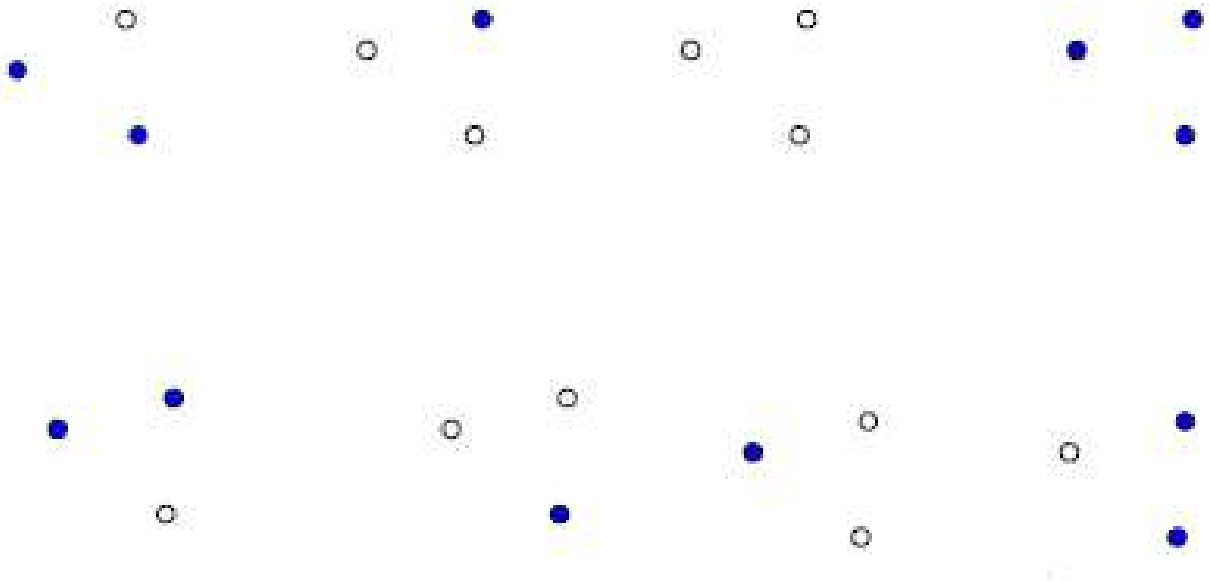


- H as set of lines $h(\boldsymbol{x}) = sign(\boldsymbol{w}\boldsymbol{x} + w_0), \boldsymbol{x} \in IR^2$
- A dichotomy is a particular labeling (in {-1,+1}) of the points
- This specific dichotomy can be represented in H:
  - there exist a line that correctly separates the points

Shattering: example: dichotomies

Dip. Informatica
University of Pisa

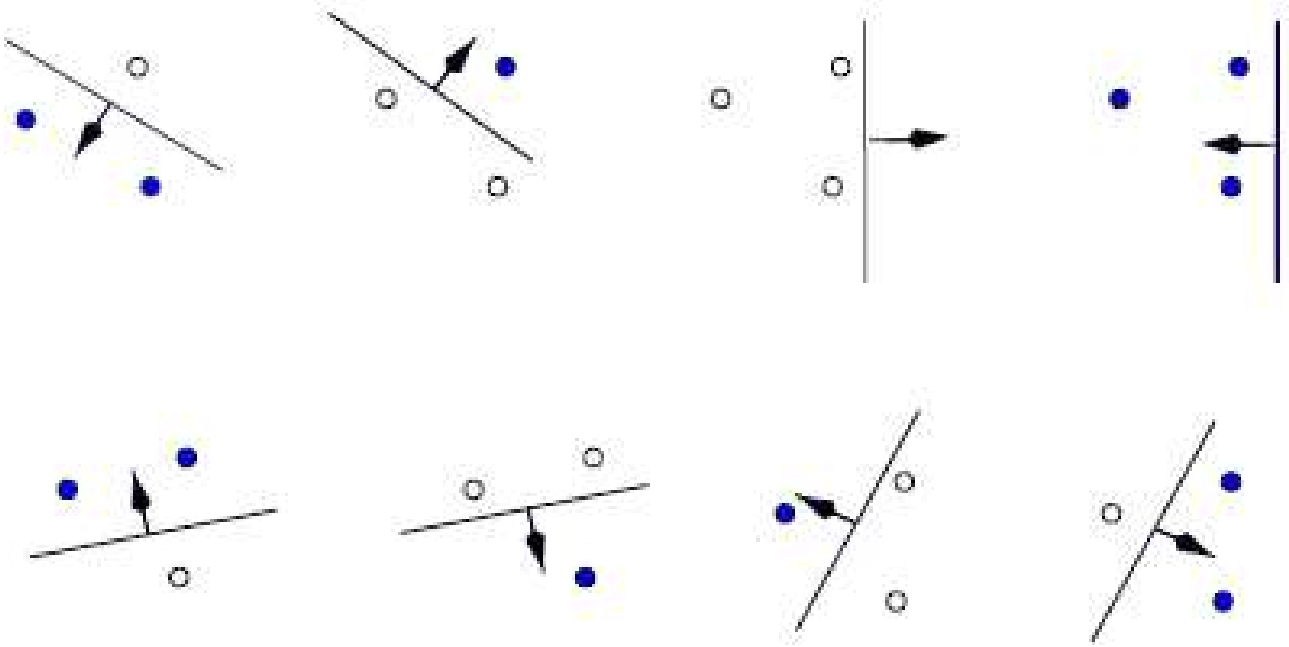
- All the possible 2^N dichotomies on 3 points in IR^2



Shattering: example: by linear functions

Dip. Informatica
University of Pisa

- The class of linear functions H shatters X



VC -dimension

Dip. Informatica
University of Pisa

- **Definition**: The **VC dimension** of a class of functions H is the maximum cardinality of a set (configuration) of points in X that can be shattered by H

- VC(H) = p
  - H *shatters* at least one set (configuration) of p points
  - H *cannot shatter* any set (configuration) of p + 1 points

- If arbitrarily large (but finite) sets of X can be shattered by H, then VC is infinite

VC-dim of linear functions (1)

Dip. Informatica
University of Pisa

- **VC(H)≥3** (already shown on the three points before)
- Note that however not all the possible configurations of 3 points can be shattered !

Not linearly separable

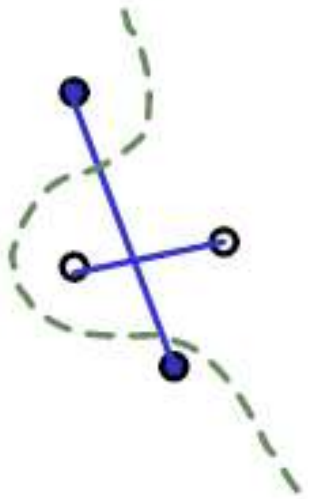


- But it is sufficient to find **one** configuration of three points which is separable for every labeling!

VC-dim of linear functions (2)

Dip. Informatica
University of Pisa

- **VC(H)≥3** (shown on the 3 points)
- **VC(H)<4**



Can always draw six lines between pairs of four points.

Two of those lines will cross.

If we put points linked by the crossing lines in the same class they can't be linearly separated

(do you remember the XOR?)

So a line can shatter 3 points but not 4

- Conclusion: **VC(H) = 3**, H hyperplanes in IR^2 (LTU)

VC-Dim: various H

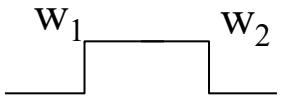
Dip. Informatica
University of Pisa

- In general: The VC dimension of a class of linear (separator/decision) hyperplanes (LTU) in a **n**-dimensional space is **n + 1**

Note: upper-bound, it can be less constraining the model, we'll see how…

Exercise: Try also:

- Lines in *IR* (VC(H)=2)
- Intervals on *IR* (VC(H)=2)



A circle:

$$x_1^2 + x_2^2 = r^2$$

- Circles centered in the origin in 2D (f(***x***,b) = sign(***x***•***x***-b) VC(H)=1)
  (f (***x***,b,k) = sign(k***x***•***x***-b) VC(H)=2)
- Circles (free radius and center) in 2D (VC(H)=3)
- Circles (fix radius, free center) in 2D (VC(H)=3)

- Can be difficult to be calculated using non-linear parameters

Number of parameters

Dip. Informatica
University of Pisa

- Is VC the number of free-parameters?
- Related but not really the same:
  - see the example in the previous slide for circles (with the same VC but different number of free-parameters)
  - and of course, we may add redundant free-parameters
  - there exist models with one parameter and infinite VC dim
    - See Haykin/Burges in bliblio
- Moreover, for nearest neighbor VC-dim is infinite

SLT: Analytical Bound on R

Dip. Informatica
University of Pisa

Assume N = *number of data (l)*

$$R[h] \leq \underbrace{R_{emp}[h] + \overbrace{\varepsilon(VC, N, \delta)}^{\textit{VC-confidence}}}_{\text{guaranteed risk}}$$

- *Example*: For 0/1 loss

$$\varepsilon(VC, N, \delta) = \sqrt{\frac{VC(\ln \frac{2N}{VC} + 1) - \ln \frac{\delta}{4}}{N}}$$

with probability at least $(1 - \delta)$ for every $VC < N$

- There are different bounds formulations according to different classes of functions, of tasks, etc.
- Interpretation: see **introduction slides on SLT** ($\varepsilon \rightarrow 0$ increasing N, ε grows with VC, U shape plot for R increasing the VC-dim)
- Now, this gives us a way to estimate the error on future data based only on the training error and the VC-dimension of ***H*** (*)
- And the VC-confidence can be computed prior to learning
- Note: The resulting bounds are "*worst case*", because they must hold for *all* but (1–delta) of the possible approximation function/ training sets.

Remarks

Dip. Informatica
University of Pisa

Example of consequences:

- For many reasonable hypothesis classes (e.g., linear approximators) the VC dimension is linear in the number of "free-parameters" of the hypothesis.
- This shows that to learn "well", we need a number of examples that is linear in the VC dimension (so linear in the number of parameters, in this case).

Here's an example of a possible heuristic (on the num .of examples for the linear model) explained with the principles of the theory

Structural Risk Minimization

Dip. Informatica
University of Pisa

- SRM uses VC dimension as a controlling parameter for minimizing the generalization bound on R
- Assuming finite VC we can define a nested *structure* of models-hypothesis spaces according to the VC-dim as in the following

$$- \mathcal{H}_1 \subseteq \mathcal{H}_2 \subseteq \cdots \subseteq \mathcal{H}_n$$

$$- VC(\mathcal{H}_1) \leq \cdots \leq VC(\mathcal{H}_n)$$

Some example?

- NN with increasing number of Hidden units (roughly), but also the #epochs (we know that also the size of weights care and a NN is a variable-size H)
- Polynomial of increasing degree *M*
- Increasing values for c, where ||w||< c for regularization (or controlled by λ)
- Increasing number of nodes in a decision tree (roughly)

- *Note: important* hyper-parameters → related to VC-dim

SRM for model selection

Dip. Informatica
University of Pisa

Growing VC-dim: empirical (TR) error decreases, VC-confidence increases

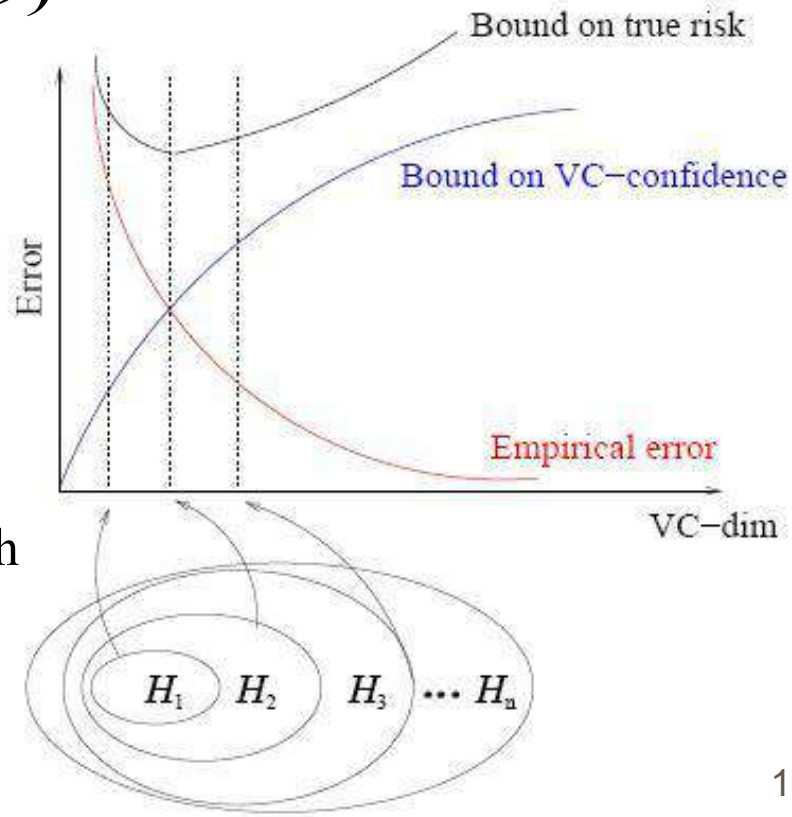
Structural **R**isk **M**inimization: find a trade-off on the bound (as seen in the introduction using l):

$$R \leq R_{emp} + \varepsilon(1/l, VC, 1/\delta)$$

- $\mathcal{H}_1 \subseteq \mathcal{H}_2 \subseteq \cdots \subseteq \mathcal{H}_n$
- $VC(\mathcal{H}_1) \leq \cdots \leq VC(\mathcal{H}_n)$

e.g. NN with increasing number of Hidden units

Model selection: choose the model (h) with the better bound on the true risk



SRM: model selection

Dip. Informatica
University of Pisa

i	H_i	TR err	VC-Conf	Probable upper bound on R	Choice
1	H_1				
2	H_2				
3	H_3				x
4	H_4				
5	H_5				
6	H_6				

Beware: However, the VC-Conf can result very conservative: hundreds of times larger than the empirical overfitting effect

Use of the bound (1)

Dip. Informatica
University of Pisa

- Provide a fundamental theoretical ground for principled ML
  - Independently from specific models details or learning alg.
  - Highlight the role of complexity control
  - The optimal choice of the model complexity (on the structure) provides the minimum of the expected risk:

    the inductive principle of SRM

- As estimation of predictive errors? Rarely used ….
  - See next slide
- Provide a direction for new models development guided by SRM

Use of the bound (2)

Dip. Informatica
University of Pisa

- Provide a fundamental theoretical ground for principled ML
- Estimation of predictive errors? Rarely used ….
  - It can be very loose: but it can still be useful for model selection (relative size/ranking of error is what matters)
  + Probabilistic upper bound for all the functions *h(x,theta)*: we can search over the class
  - Difficulty in calculating the VC-dim of specific class of H
  - The crude upper bound (over pessimistic) may be not suitable for reliable evaluation of the value of generalization error (for model assessment)
  - Tighter bounds are under development

- Provide a direction for new model development guided by SRM
- Toward *principled approaches*: less based on empirical trial&erros

SRM Approaches

Dip. Informatica
University of Pisa

$$R \leq R_{emp} + \varepsilon(1/l, VC, 1/\delta)$$

VC-confidence

For instance, two practical approaches:

1. Choose appropriate structure/complexity, fix the model (and hence the VC-confidence), minimize TR error (e.g. can be used in NN)
   - However, regularization due to training heuristics can already introduce an *implicit* SRM implementation (early stopping, …)
   - Or SRM by the Tikhonov regularization, where we have min loss with R_{emp} + *complexity term*, considering both the terms

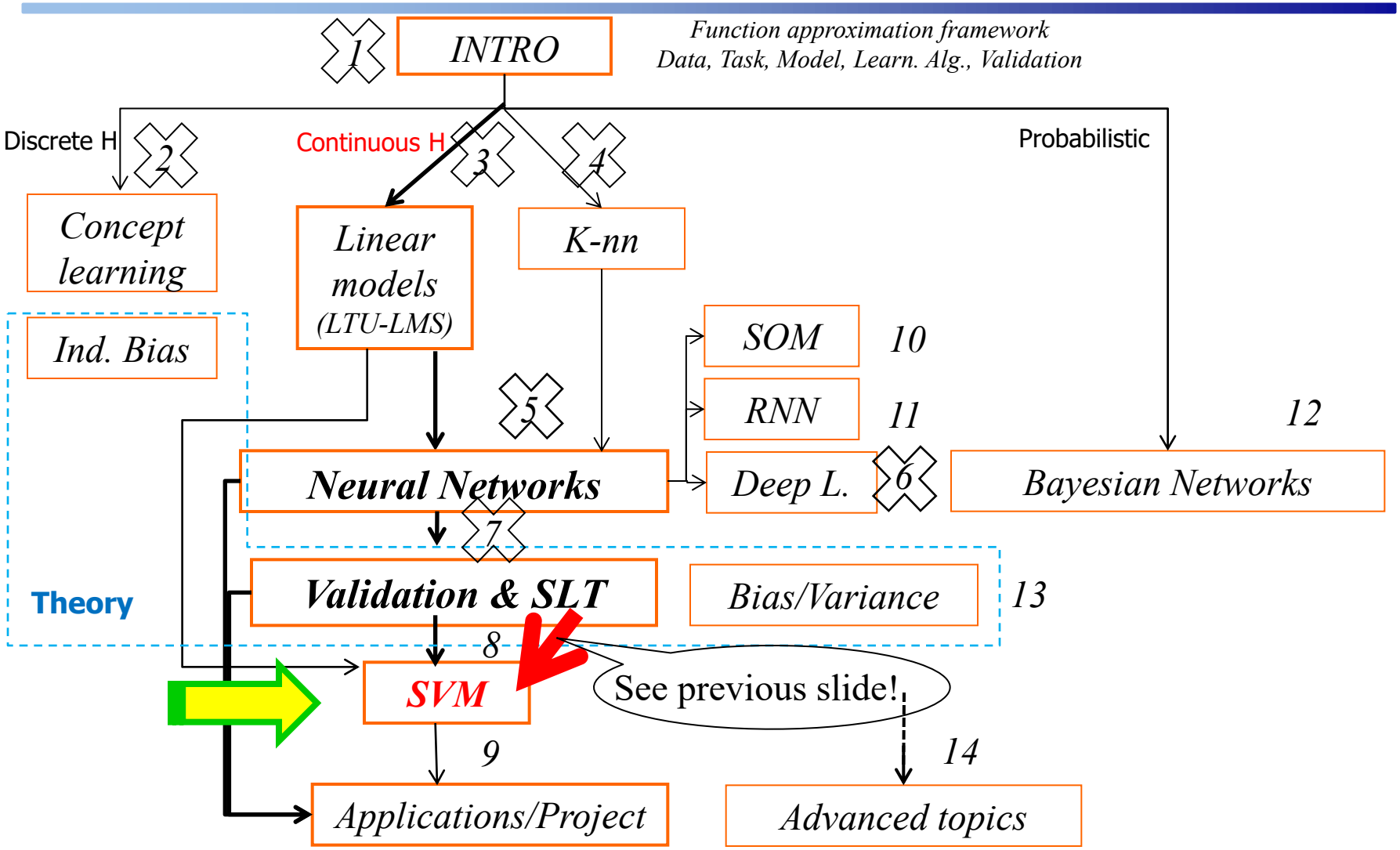
2. Fix the TR error,

   automatically minimize the VC-confidence: how ?

   We will … (see next lecture on SVM)

ML Course structure
Where we go

Dip. Informatica
University of Pisa



Introduction to Support Vector Machines

Machine Learning course

University of Pisa

Credits: Claudio Gallicchio, Alessio Micheli

Info: Email: micheli@di.unipi.it

Introduction to SVM

- Basic of SVM: Linear SVM, again a **LTU** (relating to Margin, VC-dim, ...)
- Non linear tasks: Soft Margin SVM
- Non linear tasks: Kernels for SVM
- SVM for Regression

Notation

Notation from *Haykin, Chapter 6*

- N in the number of examples (previous l) and also i and j as indices for the patterns
- m is the dimension of the input vector (previous n)
- use b instead of w_0 as the intercept or bias

* star slides/sections mean material useful to be complete (but not mandatory for the use of the model "and for a standard oral exam")

Part I

Support Vector Machines for binary classification

Support Vector Machines

Support Vector Machine (SVM):

- linear machine (LTU, Perceptron, ...)
- maximization of the separation margin
- structural risk minimization

Initially (hard margin SVM) we assume:
- linear separable problems
- no errors in the data

Linearly Separable Patterns

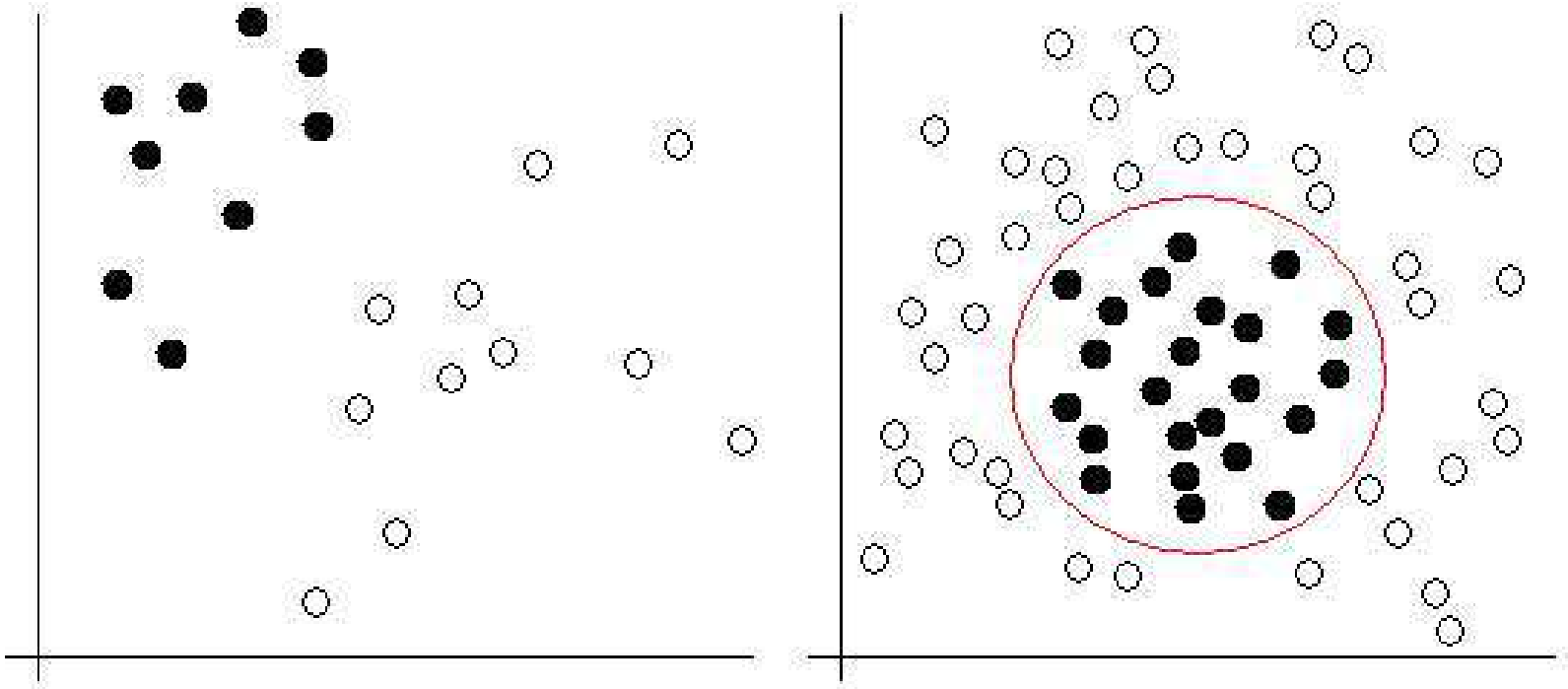


Figure: Linearly Separable Patterns and Non-Linearly Separable Patterns

Separating Hyperplane

Given the training set $T = \{(\mathbf{x}_i, d_i)\}_{i=1}^{N}$

find an hyperplane of equation $\mathbf{w}^T\mathbf{x} + b = 0$ to separate the examples:

$$\begin{aligned} \mathbf{w}^T\mathbf{x_i} + b \geq 0 \quad & \text{for } d_i = +1 \\ \mathbf{w}^T\mathbf{x_i} + b < 0 \quad & \text{for } d_i = -1 \end{aligned}$$

- $g(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b$ is the discriminant function
- $h(\mathbf{x}) = sign(g(\mathbf{x}))$ is the hypothesis

Separation Margin

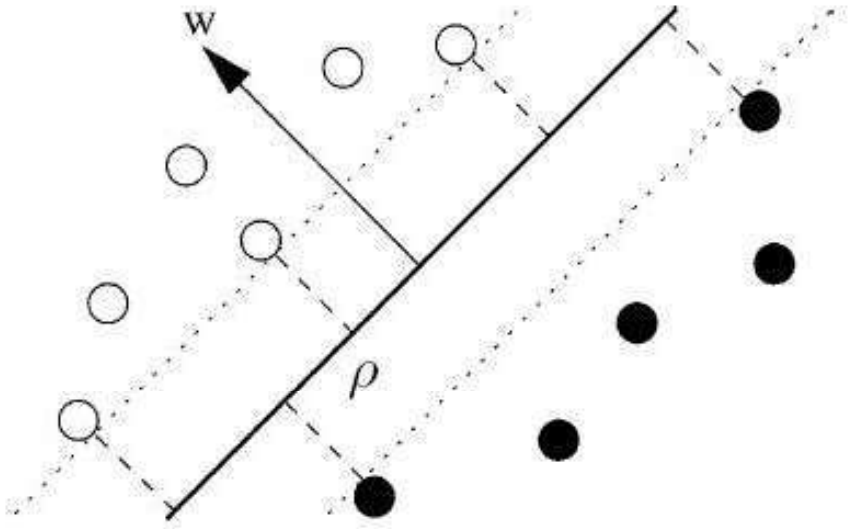


Figure: The separation margin

In this case the hyperplane has equal distance to the closest postive and negative examples.
The separation margin (ρ) is evaluated as (the double of) the distance between the linear hyperplane and the closest data point. We can think to a "safe zone".

Separation Margin

Not all the hyperplanes solving the task are equals....

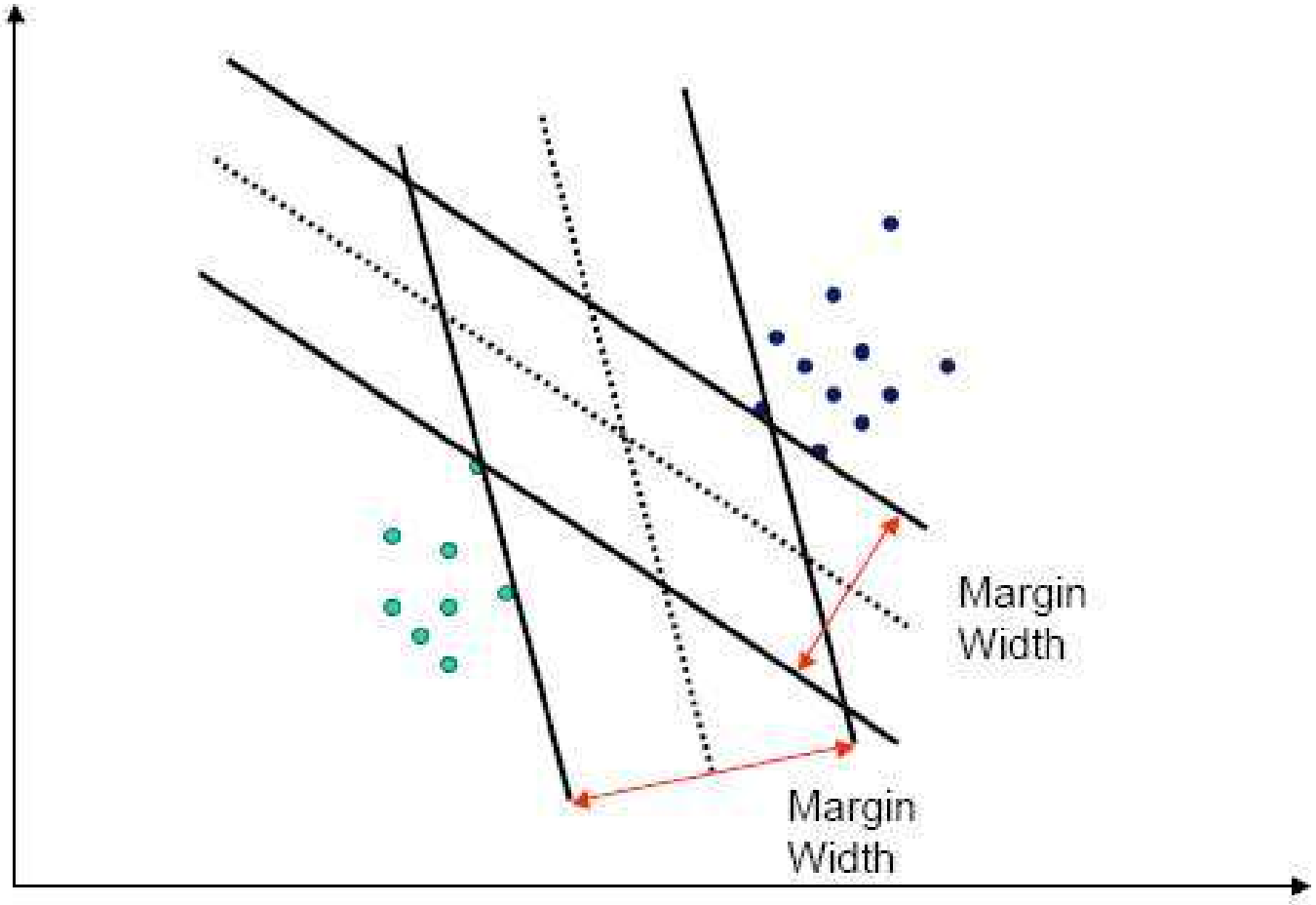


Figure: Varying the separating hyperplane the margin varies as well.

Optimal Hyperplane

The optimal hyperplane is the hyperplane which maximizes the margin ρ

$$\mathbf{w_o^T x} + b_o = 0$$

Note: In $\mathbf{w_o^T}$ "o" is for "Optimal"

We will find $\rho = \frac{2}{\|\mathbf{w}\|}$

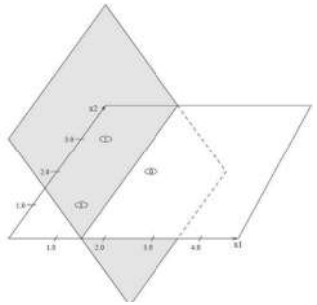
Maximize $\rho \Leftrightarrow$ Minimize $\|\mathbf{w}\|$

Canonical representation of the hyperplane

$$\mathbf{w}^T\mathbf{x_i} + b \geq 0 \quad \text{if } d_i = +1$$
$$\mathbf{w}^T\mathbf{x_i} + b < 0 \quad \text{if } d_i = -1$$

Re-scaling $\mathbf{w}$ and b (see scaling-freedom property) so that the closest points to the separating hyperplane satisfy $|g(\mathbf{x}_i)| = |\mathbf{w}^T\mathbf{x_i} + b| = 1$, we can write:

$$\mathbf{w}^T\mathbf{x_i} + b \geq 1 \quad \text{if } d_i = +1$$
$$\mathbf{w}^T\mathbf{x_i} + b \leq -1 \quad \text{if } d_i = -1$$



In a compact form:

$$d_i(\mathbf{w}^T\mathbf{x_i} + b) \geq 1 \; \forall i = 1, \cdots, N$$

Support Vector

A support vector $\mathbf{x}^{(s)}$ satisfies the previous equation exactly:

$$d^{(s)}(\mathbf{w}^T\mathbf{x}^{(s)} + b) = 1$$

Support Vectors

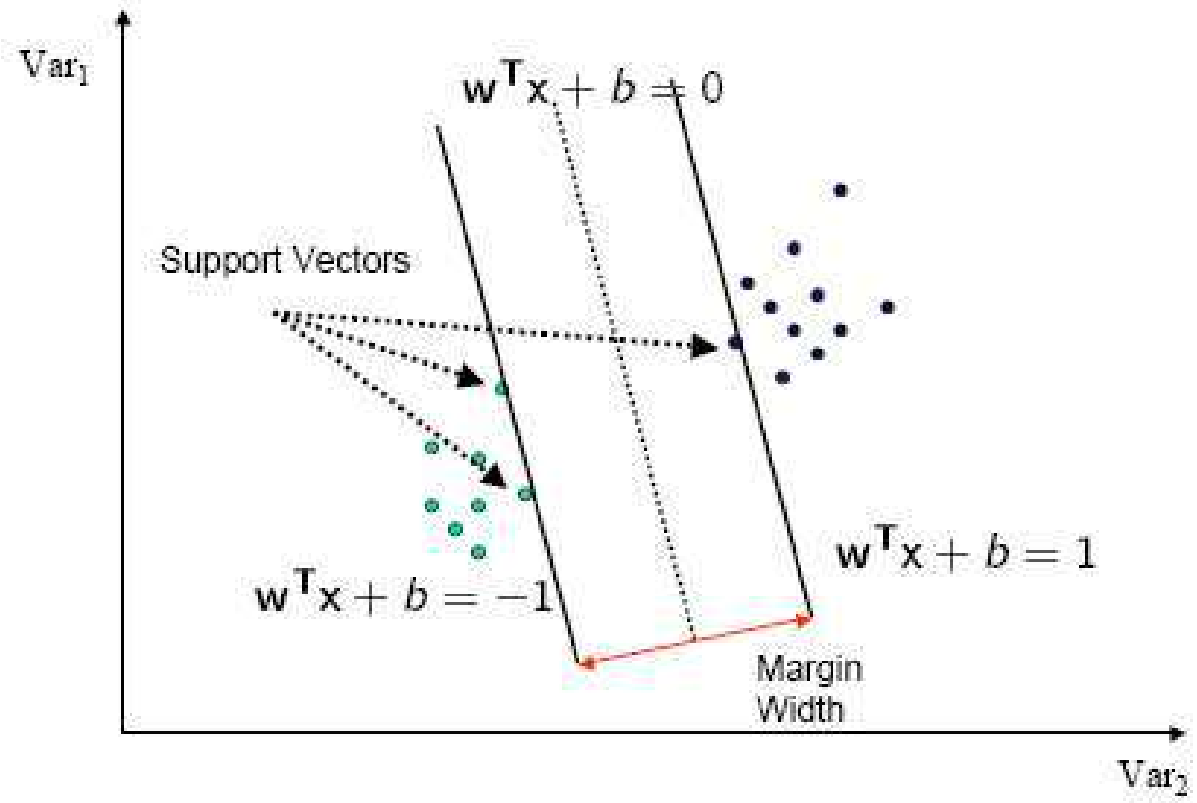


Figure: Support vectors.

The support vectors are the closest data points to the hyperplane

Distance to the hyperplane

Let's denote $g(\mathbf{x})$ as the discriminant function: $g(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b$.
Recall that $\mathbf{w_o}$ is a vector orthogonal to the hyperplane
Let's denote the distance between $\mathbf{x}$ and the optimal hyperplane with r

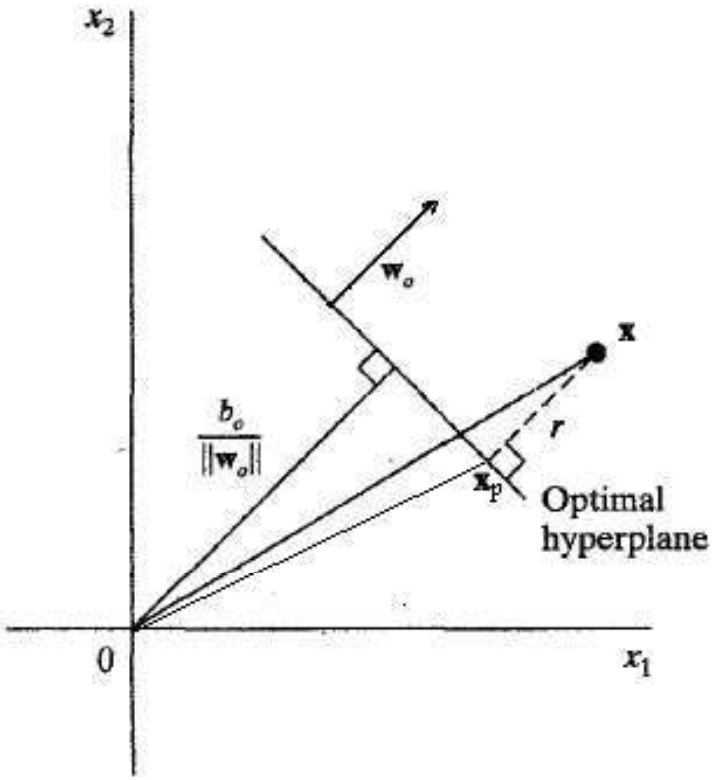


Figure: Distance between a point and the optimum hyperplane.

$$\mathbf{x} = \mathbf{x_p} + r\frac{\mathbf{w_o}}{\|\mathbf{w_o}\|}$$

Computing the distance

$$\mathbf{x} = \mathbf{x_p} + r\frac{\mathbf{w_o}}{\|\mathbf{w_o}\|}$$

Evaluating $g(\mathbf{x}) = \mathbf{w_o^T x} + b_o$ we obtain

$$g(\mathbf{x}) = g(\mathbf{x_p} + r\frac{\mathbf{w_o}}{\|\mathbf{w_o}\|}) = \mathbf{w_o^T x_p} + b_o + \mathbf{w_o^T} r\frac{\mathbf{w_o}}{\|\mathbf{w_o}\|} = g(\mathbf{x_p}) + r\mathbf{w_o^T}\frac{\mathbf{w_o}}{\|\mathbf{w_o}\|} = r\frac{\|\mathbf{w_o}\|^2}{\|\mathbf{w_o}\|} =$$

$r\|\mathbf{w_o}\|$

Thus

$$r = \frac{g(\mathbf{x})}{\|\mathbf{w_o}\|}$$

Computing the margin

Consider the distance between the hyperplane and a positive support vector $\mathbf{x}^{(s)}$

$$\text{i.e. } r \text{ for } \mathbf{x}^{(s)} = \frac{g(\mathbf{x}^{(s)})}{\|\mathbf{w_o}\|} = \frac{1}{\|\mathbf{w_o}\|} = \frac{\rho}{2}$$

$$\Downarrow$$

$$\rho = \frac{2}{\|\mathbf{w_o}\|}$$

- the optimum hyperplane maximizes ρ
- the optimum hyperplane minimizes $\|\mathbf{w}\|$

Quadratic Optimization Problem

The formal derivation of the SVM solution requires techniques in the constrained optimization framework (which is assumed).
In particular, we assume quadratic programming solved elsewhere (packages).
In the first derivation is just useful to understand:

- how the problem is formally stated
- what is the relationships between support vectors and Alfa
- the final model and the result for $\mathbf{w}_0$

* slides/sections is material useful to be complete (but not mandatory for the use of the model)

Quadratic Optimization Problem

Finding the optimum values of $\mathbf{w}$ and b in order to maximize the margin yields to a quadratic optimization problem

Hard margin SVM

Optimization Problem - Primal form

Given the training samples $T = \{(\mathbf{x_i}, d_i)\}_{i=1}^{N}$, find the optimum values of $\mathbf{w}$ and b which minimize

$$\Psi(\mathbf{w}) = \frac{1}{2}\mathbf{w}^T\mathbf{w} \quad (min.\|\mathbf{w}\|)$$

satisfying the constraints (zero classification errors)

$$d_i(\mathbf{w}^T\mathbf{x_i} + b) \geq 1 \; \forall i = 1, \cdots, N \qquad \text{(N =number of data)}$$

- The objective function $\Psi(\mathbf{w})$ is quadratic and convex in $\mathbf{w}$
- The constraints are linear in $\mathbf{w}$
- Solving this problem scales with the size of the input space m

(the computational cost) ...which however depends on the algorithm of the solver, see CM course

Solving the quadratic problem*

To solve this problem the Lagrangian multipliers method is used:

Construct the Lagrangian function corresponding to the quadratic opt. problem

$$J(\mathbf{w}, b, \alpha) = \frac{1}{2}\mathbf{w}^T\mathbf{w} - \sum_{i=1}^{N} \alpha_i(d_i(\mathbf{w}^T\mathbf{x_i} + b) - 1)$$

- $\alpha_i \geq 0 \, \forall i = 1, \cdots, N$ are the N Lagrangian multipliers
- Each term in the sum corresponds to a constraint of the primal problem
- J must be minimized with respect to $\mathbf{w}$ and b and maximized with respect to α
- The solution corresponds to a saddle point of J

Optimal values of $\mathbf{w}$ and b

We will find:

$$\mathbf{w_o} = \sum_{i=1}^{N} \alpha_{o,i} d_i \mathbf{x_i}$$

And thus the optimal hyperplane is expressed as

$$\mathbf{w_o^T x} + b_o = 0 \Leftrightarrow \sum_{i=1}^{N} \alpha_{o,i} d_i \mathbf{x_i}^T \mathbf{x} + b_o = 0$$

Optimal Conditions*

$$J(\mathbf{w}, b, \alpha) = \frac{1}{2}\mathbf{w}^T\mathbf{w} - \sum_{i=1}^{N} \alpha_i (d_i(\mathbf{w}^T\mathbf{x_i} + b) - 1)$$

- Minimize J with respect to $\mathbf{w} \Rightarrow \frac{\partial J}{\partial \mathbf{w}} = 0 \Rightarrow \mathbf{w} = \sum_{i=1}^{N} \alpha_i d_i \mathbf{x_i}$
- Minimize J with respect to $b \Rightarrow \frac{\partial J}{\partial b} = 0 \Rightarrow \sum_{i=1}^{N} \alpha_i d_i = 0$

and we may substitute these in J to study the dual form ...

Kuhn - Tucker Conditions

From the KT conditions it follows:

$$\alpha_i(d_i(\mathbf{w}^T\mathbf{x_i} + b) - 1) = 0 \,\forall i = 1, \cdots, N$$

in the saddle point of J.

- if $\alpha_i > 0$, then the $d_i(\mathbf{w}^T\mathbf{x_i} + b) = 1$ and $\mathbf{x_i}$ is a support vector
- if $\mathbf{x_i}$ is not a support vector than $\alpha_i = 0$

Hence we can restrict the computation to N_s: $\mathbf{w_o} = \sum_{i=1}^{N_s} \alpha_{o,i} d_i \mathbf{x_i}$.
The hyperplane depends only from support vectors !

Dual Problem

To obtain the Lagrangian multipliers $\{\alpha_i\}_{i=1}^N$ we solve the problem in dual form:

Optimization Problem - Dual form *

Given the training samples $T = \{(\mathbf{x_i}, d_i)\}_{i=1}^N$, find the optimum values of the Lagrangian multipliers $\{\alpha_i\}_{i=1}^N$ which maximize

$$Q(\alpha) = \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \alpha_i \alpha_j d_i d_j \mathbf{x_i^T x_j}$$

satisfying the constraints

- $\alpha_i \geq 0 \, \forall i = 1, \cdots N$
- $\sum_{i=1}^{N} \alpha_i d_i = 0$

α can be found solving the quadratic programming (QP) problem or by more recent (efficient) approaches (e.g. SMO:Sequential Minimal Optimization).... Please note that solving this problem scales with the number of training samples (N), less with size of space dimensions

(the computational cost) ..which however depends on the algorithm of the solver, see CM course

Obtaining $\mathbf{w_o}$ and b_o

- Solve the dual problem obtaining $\{\alpha_{o,i}\}_{i=1}^{N}$
- Compute $\mathbf{w_o} = \sum_{i=1}^{N} \alpha_{o,i} d_i \mathbf{x_i}$
- Compute $b_o = 1 - \mathbf{w_o^T x^{(s)}}$ corresponding to a positive support vector $\mathbf{x}^{(s)}$

$$b_o = 1 - \sum_{i=1}^{N} \alpha_{o,i} d_i \mathbf{x_i}^T \mathbf{x}^{(s)}$$

How do we use it?

- We do NOT need to explicitly know $\mathbf{w_o}$
- We only need to know Lagrangian multipliers (by solving the dual problem)
- Then we compute the optimal bias b_o

Remember:

$$\mathbf{w_o} = \sum_{i=1}^{N} \alpha_{o,i} d_i \mathbf{x_i}$$

And the decision surface is given by

$$\mathbf{w_o}^T \mathbf{x} + b_o = 0 \Leftrightarrow \sum_{i=1}^{N} \alpha_{o,i} d_i \overbrace{\mathbf{x_i}^T \mathbf{x}}^{\text{dot product}} + b_o = 0$$

So, given the input pattern $\mathbf{x}$:

1. Compute $g(\mathbf{x}) = \sum_{i=1}^{N} \alpha_{o,i} d_i \mathbf{x_i}^T \mathbf{x} + b_o$
2. Classify $\mathbf{x}$ as the sign of $g(\mathbf{x})$

Note that it is not necessary to calculate $\mathbf{w_o}$ and the sum over N can be restricted to the support vectors N_s

Why does this improve the generalization?

We fixed the training error on linearly separable problems. Than minimizing the norm of $\mathbf{w}$ is equivalent to minimizing the VC dimension and thus to minimizing the capacity term (VC-confidence) ϵ in

$$R[h] \leq R_{emp}[h] + \epsilon(VC, N, \delta)$$

Relevant!!!

Theorem - Vapnik

Let D be the diameter of the smallest ball around the data points $\mathbf{x_1}, \cdots, \mathbf{x_N}$. For the class of separating hyperplanes described by the equation $\mathbf{w^T x} + b = 0$ the upper bound to the VC dimension is

$$VC \leq min(\lceil \frac{D^2}{\rho^2} \rceil, m_0) + 1$$

since $\frac{D^2}{\rho^2} = Radios^2 \|\mathbf{w}\|^2$, VC can be less than $m_0 + 1$ computed for general hyperplanes by constraining the search to the "regularized" one with maximum margin

Why does this provide an elegant approach?

For **linearly separable** data there are many solutions. Vapnik propose an "optimal separating hyperplane" maximizing the margin providing

- an unique solution (not for iterative Perceptron Learn. Alg.) with zero errors (not for LMS) for the binary classifier
- an automatized approach to SRM that minimizes VC-confidence (by maximizin the margin) as part of the training process (directly into the optimization process), without hyper-parameters in the linear-separable case
- the use of solver in the class of constrained Quadratic Programming (instead of gradient descent) with a nice dual form (showing support vectors and the dot product among patterns)
- a solution focused on "selected" training data, the support vectors
  - (nice) not depending directly on the point far away from the decision boundary
  - (less nice) hoping that the points at the boundaries are not the noisier ones.

But what for noisy or ñon l̃inearly separable data?

Soft margin : admit errors

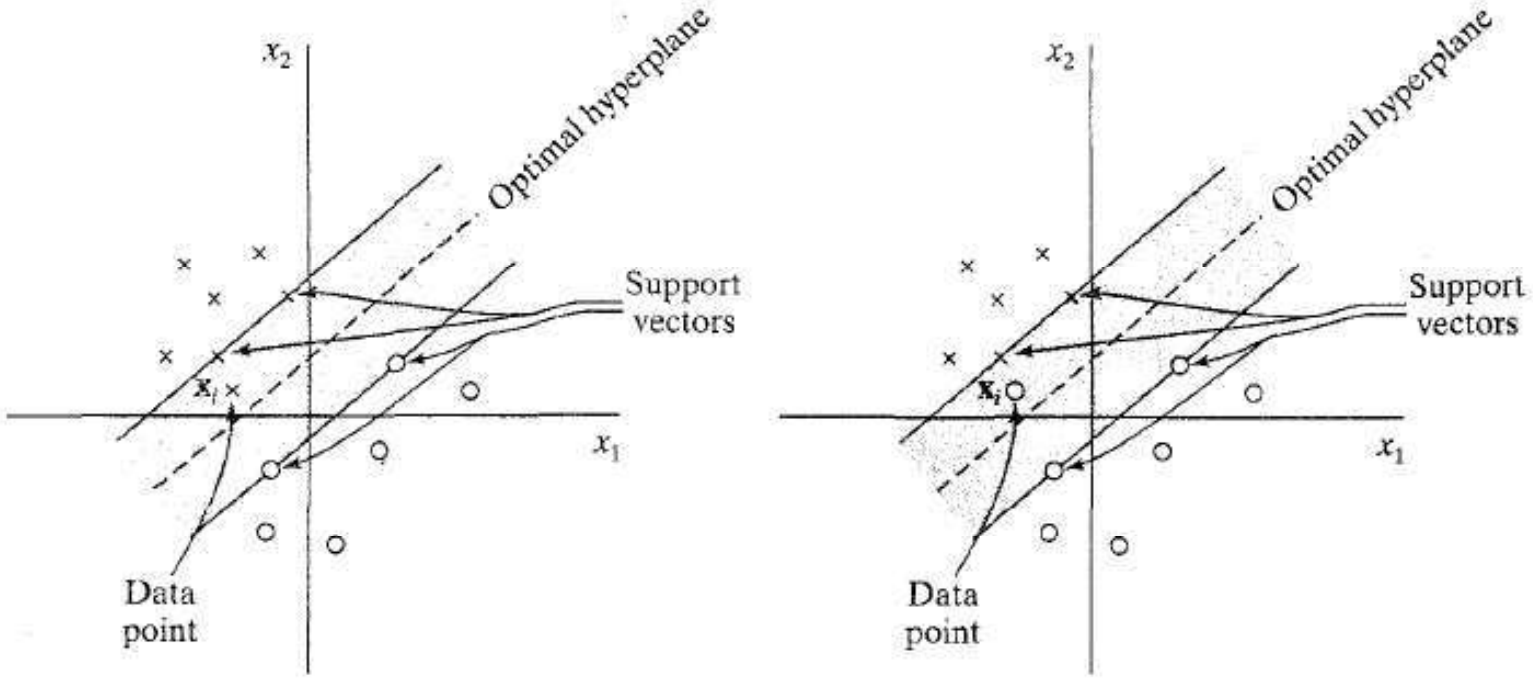


Figure: Patterns violating the margin (with and without classification error).

At least one point violate the exact-fit condition:

$$d_i(\mathbf{w}^T\mathbf{x_i} + b) \geq 1$$

Exercise: find the margin for the figures above without admitting points inside the margin. This explain the reason for soft margin: noise tolerance and larger margin

Soft Margin.

Note: admitting points inside the margin allows us to have a larger margin.

Soft Margin Conditions

We introduce the non-negative scalar variables:

$$\xi_i \geq 0 \, \forall i = 1, \cdots, N$$

called *slack variables*.

$$d_i(\mathbf{w}^T \mathbf{x_i} + b) \geq 1 - \xi_i \, \forall i = 1, \cdots, N$$

Support Vector

A support vector $\mathbf{x_i}$ satisfies the previous exactly:

$$d_i(\mathbf{w}^T \mathbf{x_i} + b) = 1 - \xi_i$$

Soft Margin and slack variables

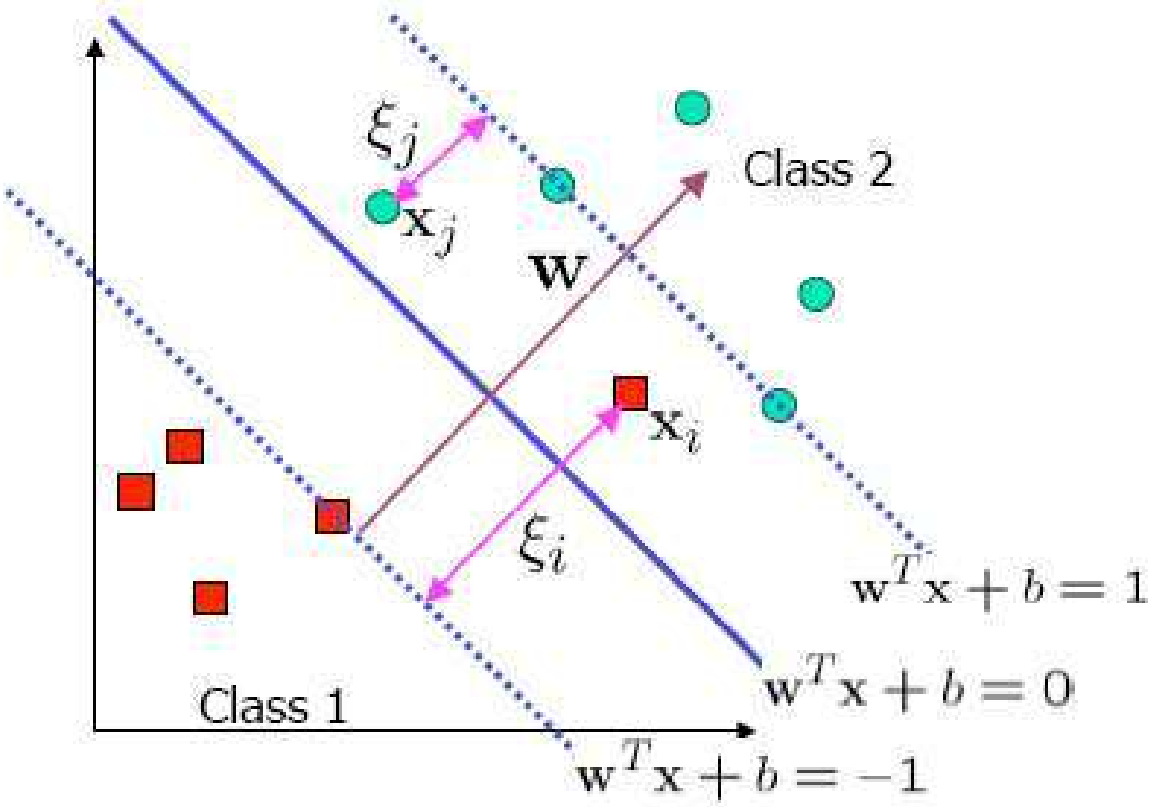


Figure: Soft margin and slack variables.

Note

Note that the Vapnik's Theorem does not hold anymore! (hard margin does not admit points inside the margin)

Primal Problem

Given the training set $T = \{(\mathbf{x_i}, d_i)\}_{i=1}^{N}$, find the values of $\mathbf{w}$ and b which minimize the objective function

$$\Psi(\mathbf{w}, \xi) = \frac{1}{2}\mathbf{w}^T\mathbf{w} + C\sum_{i=1}^{N}\xi_i$$

Soft margin SVM

under the constraints:

$$d_i(\mathbf{w}^T\mathbf{x_i} + b) \geq 1 - \xi_i \; \forall i = 1, \cdots, N$$

$$\xi_i \geq 0 \; \forall i = 1, \cdots, N$$

C as a regularization hyper-parameter (we lost fully automatic SRM!), a trade-off between empirical risk minimization and capacity term (VC-confidence) minimization:

For which C we obtain hard margin?

- Low C -> many TR errors are allowed -> possible underfitting
- High C -> no TR errors are allowed -> smaller margin -> possible overfitting

Dual Problem *

Given the training set $T = \{(\mathbf{x_i}, d_i)\}_{i=1}^{N}$, find the values of the Lagrangian multipliers $\{\alpha_i\}_{i=1}^{N}$ which maximize the objective function

$$Q(\alpha) = \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \alpha_i \alpha_j d_i d_j \overbrace{\mathbf{x}_i^T \mathbf{x}_j}^{\text{dot product!}}$$

satisfying the constraints:

$$\sum_{i=1}^{N} \alpha_i d_i = 0$$

$$0 \leq \alpha_i \leq C \; \forall i = 1, \cdots N$$

Note that C is a user defined parameter

Kuhn - Tucker Conditions

The KT conditions are now defined as:

- $\alpha_i(d_i(\mathbf{w}^T\mathbf{x_i} + b) + \xi_i - 1) = 0 \, \forall i = 1, \cdots, N$
- $\mu_i \xi_i = 0 \, \forall i = 1, \cdots, N$

where the μ_i are Lagrange multipliers introduced to enforce the non-negativity of the slack variables (in the Lagrange primal function)
$0 < \alpha_i < C \rightarrow \xi_i = 0$ (on the margin), (i.e. on the edge of the margin)
$\alpha_i = C \rightarrow \xi_i \geq 0$ (inside the margin)

- Solve the dual problem with respect to $\{\alpha_i\}_{i=1}^N$
- Find $\mathbf{w_o}$ and b_o from $\{\alpha_{o,i}\}_{i=1}^N$

$$\mathbf{w_o} = \sum_{i=1}^{N} \alpha_{o,i} d_i \mathbf{x_i}$$

$$b_o = d_j - \sum_{i=1}^{N} \alpha_{o,i} d_i \mathbf{x_i}^T \mathbf{x}_j \text{ for a pattern j such that } 0 < \alpha_j < C$$

(or an average of all the solutions for numerical stability)
The non zero Lagrangian Multipliers correspond to support vectors.

How do we use it?

As before:

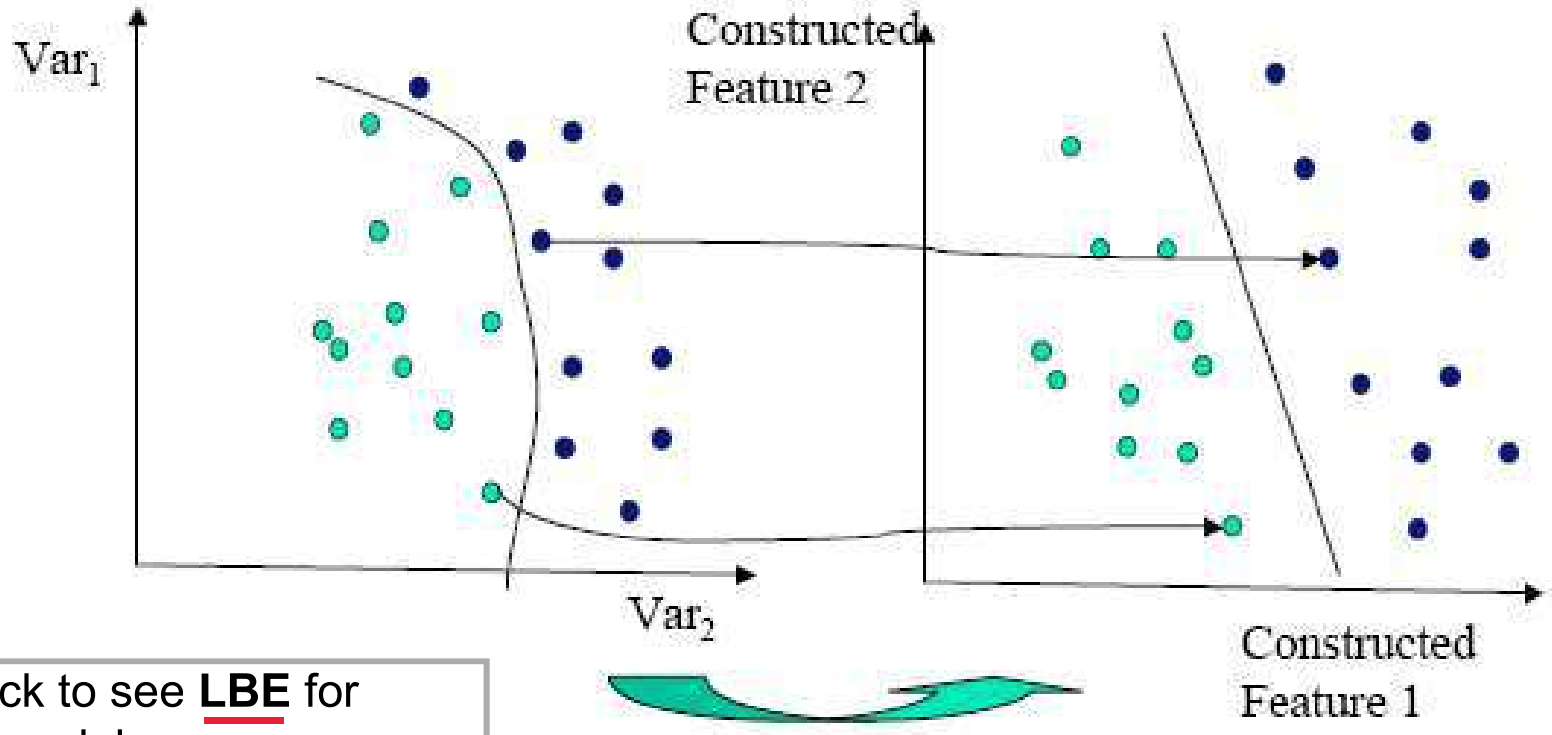
$$g(\mathbf{x}) = \sum_{i=1}^{N} \alpha_{o,i} d_i \mathbf{x_i}^T \mathbf{x} + b_o$$

$$h(\mathbf{x}) = sign(g(\mathbf{x}))$$

Mapping to a High-Dimensional Space (for non linearly separable patterns)

Idea

Map the data points in the input space to a high-dimensional **feature space**, where they can be linearly separable



Go back to see **LBE** for linear models

Example

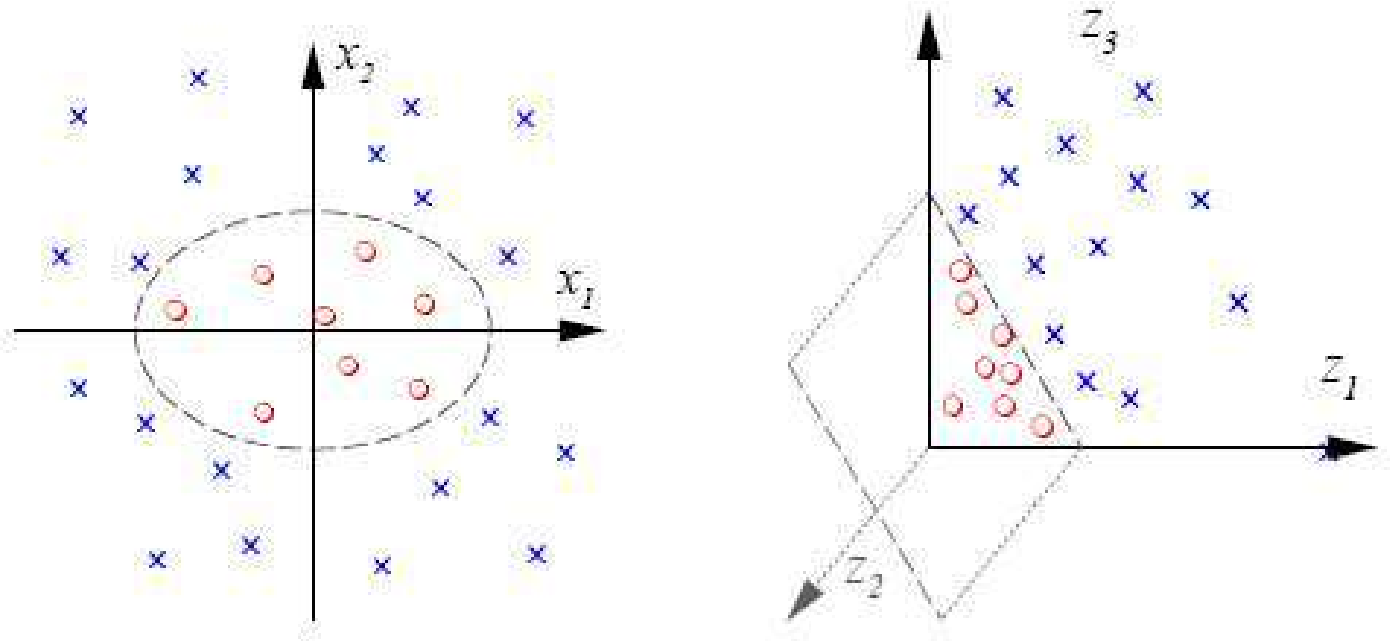


Figure: In the input space we need a quite complex decision surface, while in the second order monomials feature space a linear function is sufficient.

$$\Phi : \mathbb{R}^2 \rightarrow \mathbb{R}^3$$

$$(x_1, x_2)^T \mapsto (x_1^2, \sqrt{2}x_1x_2, x_2^2)^T$$

Example: comments

The previous mapping is built such that it solve the problem (the built features correspond to ellipsoids in the original input space).

We shown that even adding a dimension to the feature space we can translate the original non linear problem into a linearly separable problem.

But it is not automatic to find the "best" transformation! (unless you have prior knowledge allowing you to select a proper feature space).

Often, general transformations (see in the following popular choices) can be effective in the general case.

However, the choice of Φ is the critical issue of the SVM.

High-dimensional feature spaces

1. Nonlinear mapping of input patterns to a high-dimensional feature space (Cover's Theorem: the patterns are linearly separable with high probability in the feature space under such conditions)
2. Finding the optimal hyperplane to separate the patterns in the feature space

However, we know that using high dimensional feature spaces (large basis function expansion) can be computationally unfeasible and can lead to over-fitting without controlling the dimension of feature space and the complexity* of the classifier (image to have a new unconstrained free-parameter for each transformation)

We are going to propose the Kernel approach to manage (implicitly) the feature space in the context of regularized modeling ...

(where, you know, the margin rules the complexity and not the dimension of the input space)

*(complexity is for LBE related directly to the model input dimensionality, increased after the phi expansion)

Solving the problem in the Feature-Space

Non linear function map

$$\Phi : \mathbb{R}^{m_0} \rightarrow \mathbb{R}^{m_1}$$

$$\mathbf{x} \mapsto \Phi(\mathbf{x})$$

- The problem is formulated as before, but the training set is now $T = \{(\Phi(\mathbf{x}_i), d_i)\}_{i=1}^{N}$
- The hyperplane is now $\mathbf{w}^T \Phi(\mathbf{x}) + b = 0$

Incorporating the bias in the weight vector:

- $w(0) = b$
- $\Phi_0(\mathbf{x}) = 1$

$$\Phi(\mathbf{x}) = (\phi_0(\mathbf{x}) = 1, \phi_1(\mathbf{x}), \cdots, \phi_{m_1}(\mathbf{x}))^T$$

Decision surface equation: $\mathbf{w}^T \Phi(\mathbf{x}) = 0$ i.e. a linear **expansion of the basis functions** (where (i) the dimension of the enlarged space is allowed to get very large and (ii) the complexity dependes from the margin, not directly on the space dimension)

Linear hyperplane in feature space

The weight vector is a linear combination of the feature vectors:

$$\mathbf{w} = \sum_{i=1}^{N} \alpha_i d_i \Phi(\mathbf{x_i})$$

Thus the hyperplane equation is:

$$\sum_{i=1}^{N} \alpha_i d_i \underbrace{\Phi^T(\mathbf{x_i})\Phi(\mathbf{x})}_{\text{dot product}} = 0$$

Note the dot product is now in the feature space

Evaluating $\Phi(\mathbf{x})$ could be intractable

Kernel trick

Fortunately under certain conditions we do NOT need to evaluate directly $\Phi(\mathbf{x})$.

We do NOT even need to know the feature space itself!

This is possible using a function to compute directly the dot products in the feature space

$$k : \mathbb{R}^{m_0} \times \mathbb{R}^{m_0} \rightarrow \mathbb{R}$$

k is called *inner product kernel function*

$$k(\mathbf{x_i}, \mathbf{x}) = \Phi^T(\mathbf{x_i})\Phi(\mathbf{x})$$

Property: a kernel function is a symmetric function $k(\mathbf{x_i}, \mathbf{x}) = k(\mathbf{x}, \mathbf{x_i})$
The kernel function provides us a powerful abstraction tool.

An example

Computing the inner product

Recalling the previous example of non-linear mapping function Φ

$$\Phi(\mathbf{x}) = \Phi((x_1, x_2)^T) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)^T$$

Given $\mathbf{x} = (x_1, x_2)^T$ and $\mathbf{y} = (y_1, y_2)^T$ in $\mathbb{R}^2$ we compute $\Phi^T(\mathbf{x})\Phi(\mathbf{y})$ in this way:

$$(x_1^2, \sqrt{2}x_1x_2, x_2^2)(y_1^2, \sqrt{2}y_1y_2, y_2^2)^T = x_1^2y_1^2 + 2x_1x_2y_1y_2 + x_2^2y_2^2$$

$$= (x_1y_1 + x_2y_2)^2 = ((x_1, x_2)(y_1, y_2)^T)^2 = (\mathbf{x}^T\mathbf{y})^2 = k(\mathbf{x}, \mathbf{y})$$

Very important

The dot product in feature space is evaluated without considering the feature mapping and the feature space itself.
We evaluate the dot product in terms of the input patterns (from 3D to 2D in this simple ex.)

Kernel Matrix

We can arrange the dot products in the feature space between the images of the input training patterns in a N by N matrix called *kernel matrix*:

$$K = \begin{pmatrix} k(\mathbf{x}_1, \mathbf{x}_1) & \cdots & k(\mathbf{x}_1, \mathbf{x}_N) \\ k(\mathbf{x}_2, \mathbf{x}_1) & \cdots & k(\mathbf{x}_2, \mathbf{x}_N) \\ \cdots & & \\ k(\mathbf{x}_N, \mathbf{x}_1) & \cdots & k(\mathbf{x}_N, \mathbf{x}_N) \end{pmatrix}$$

$$K = \{k(\mathbf{x}_i, \mathbf{x}_j)\}_{(i,j)=1}^{N}$$

The kernel matrix is symmetrical as the inner product kernel is symmetrical.

Kernels Vs Inner product kernels

Does any kernel function computes the inner product in a feature space?

NO

This property only holds for kernels gaining positive semi-definite kernel matrices (Mercer's Theorem) , i.e. "valid kernels"

Related to having not-negative Eigenvalues of the kernel matrix.

Some properties of kernels

Let k_1 and k_2 be kernels over $\mathbb{R}^{m_0} \times \mathbb{R}^{m_0}$.

The following are kernel functions as well:

- $k_1(\mathbf{x}, \mathbf{y}) + k_2(\mathbf{x}, \mathbf{y})$
- $\alpha k_1(\mathbf{x}, \mathbf{y}) \, \forall \alpha \in \mathbb{R}_+$
- $k_1(\mathbf{x}, \mathbf{y}) k_2(\mathbf{x}, \mathbf{y})$

Primal problem in feature space

Primal Problem

Given the training set $T = \{(\Phi(\mathbf{x_i}), d_i)\}_{i=1}^{N}$ find the optimal value of $\mathbf{w}$ which minimizes the objective function

$$\Psi(\mathbf{w}, \xi) = \frac{1}{2}\mathbf{w}^T\mathbf{w} + C\sum_{i=1}^{N}\xi_i$$

under the constraints

$$d_i(\mathbf{w}^T\Phi(\mathbf{x_i})) \geq 1 - \xi_i \; \forall i = 1, \cdots, N$$

$$\xi_i \geq 0 \; \forall i = 1, \cdots, N$$

Please note that $\mathbf{w}$ is now in the feature space and thus its dimensionality can lead to an intractable problem

Dual Problem in feature space

Dual Problem

Given the training set $T = \{(\Phi(\mathbf{x_i}), d_i)\}_{i=1}^N$ find the optimal values of $\{\alpha_i\}_{i=1}^N$ which maximize the objective function

$$Q(\alpha) = \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{N} \alpha_i \alpha_j d_i d_j \overbrace{k(\mathbf{x}_i, \mathbf{x}_j)}^{K_{i,j}}$$

satisfying the constraints

$$\sum_{i=1}^{N} \alpha_i d_i = 0$$

$$0 \leq \alpha_i \leq C \; \forall i = 1, \cdots, N$$

Remember C is a user specified non-negative parameter (regularization parameter)
The kernel includes the mapping to the feature space. The dependences on N becames now an advantage (due to the possible high dim of the feature space)

Wrapping Up

- We have the training set $T = \{(\mathbf{x_i}, d_i)\}_{i=1}^{N}$
- We choose the trade-off parameter C, and the kernel function k
- We compute the kernel matrix K
- We solve the optimization problem (*Quadratic Programming Algorithms*) and find $\{\alpha_i\}_{i=1}^{N}$
- We compute the bias b knowing the Lagrangian multipliers and the kernel matrix
- Given an input pattern $\mathbf{x}$ we compute $\mathbf{w}^T\Phi(\mathbf{x}) = \sum_{i=1}^{N} \alpha_i d_i k(\mathbf{x}, \mathbf{x}_i) + b$ (here is fundamental that we don't need to compute $\mathbf{w}$)
  (or include it as $\phi_0 = 1$)
- We classify $\mathbf{x}$ as the sign of the discriminant function $\sum_{i=1}^{N} \alpha_i d_i k(\mathbf{x}, \mathbf{x}_i) + b$

Sparsity

Please remember that the opt. problem solution gains a sparse solution in $\{\alpha_i\}_{i=1}^{N}$, as all the multipliers corresponding to a non-support vector are zero.

N= number of data

Test phase

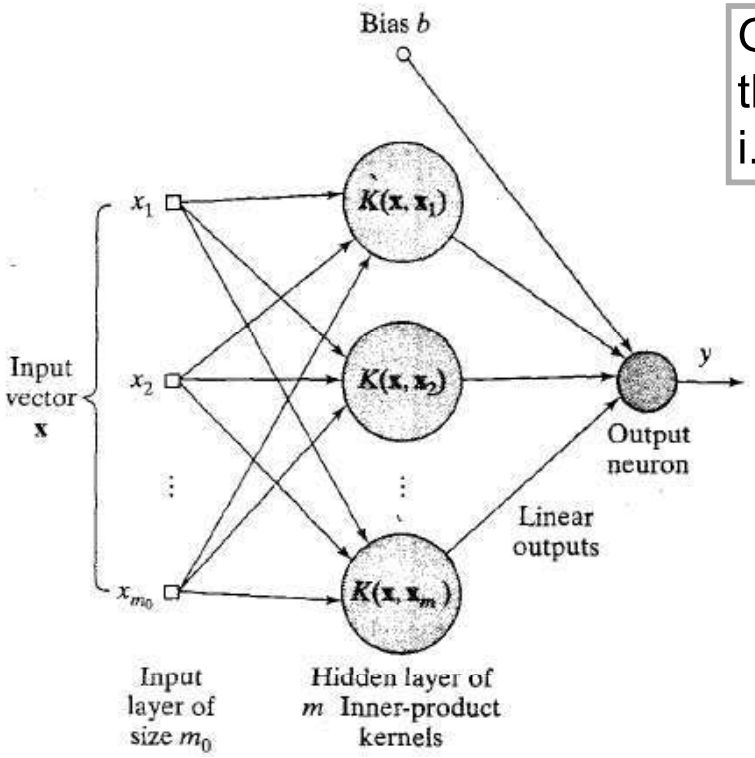
Remember: at this step we solved the optimization problem and obtained the Lagrangian multipliers and the kernel matrix.

How do we use the machine for an unseen input pattern $\mathbf{x}$?

1. Compute $\sum_{i=1}^{N} \alpha_i d_i k(\mathbf{x}, \mathbf{x}_i)$
2. Classify $\mathbf{x}$ as the sign of the previous: $h(\mathbf{x}) = sign(\sum_{i=1}^{N} \alpha_i d_i k(\mathbf{x}, \mathbf{x}_i) + b)$

Note: we memorize the $\mathbf{x}_i$ for the test phase (does it resemble k-nn?)

Architecture view of a SVM



Go back to see the slides on the NN as (a special kind, i.e. adaptive, of) **LBE**

Figure: "Architecture" of a SVM.

- Number of hidden units = Number of support vectors
  (... but not exactly the same unit, e.g. hidden units in NN have adpative free parameters)!

This is just a "picture" view of the SVM. Think by yourself on the comparison in terms of complexity control, efficiency, and other characteristics.

Some examples of kernels

- Polynomial Learning Machine: $k(\mathbf{x}, \mathbf{x}_i) = (\mathbf{x}^T \mathbf{x}_i + 1)^p$ where p is a user specified parameter
- Radial Basis Function Net: $k(\mathbf{x}, \mathbf{x}_i) = e^{-\frac{1}{2\sigma^2}\|\mathbf{x}-\mathbf{x}_i\|^2}$ where σ^2 is a user specified parameter A.k.a. Gaussian kernel

  [narrow peaked Kernels (small sigma) imply that reply for $\mathbf{x}_i$ is only d_i (what does it resemble?)]
- Two-layer perceptron $k(\mathbf{x}, \mathbf{x}_i) = tanh(\beta_0 \mathbf{x}^T \mathbf{x}_i + \beta_1)$ where $\beta_0 (> 0)$ and $\beta_1 (< 0)$ are user specified parameters

Important

For polynomial learn. machines and RBN you always get an inner product kernel, while for a two-layer perceptron the Mercer's th. only holds for some choices of β_0 and β_1

Note

Also note that using a kernel function of RBF type always leads to a feature space with an infinite number of dimensions

Part II

Support Vector Machines for non-linear regression

Recall: Regression Problem

We have a function that describes the dependency of a scalar from a vector:

$$d = f(\mathbf{x}) + v$$

where:

- the function f is unknown
- the noisy term v is statistically independent of the input vector $\mathbf{x}$

We only have a training set $T = \{(\mathbf{x}_i, d_i)\}_{i=1}^{N}$

We estimate d using a linear expansion of non-linear functions $\{\phi_j(\mathbf{x})\}_{j=0}^{m_1}$

$$y = h(\mathbf{x}) = \mathbf{w}^T \Phi(\mathbf{x})$$

Where: $\mathbf{w} = (w(0) = b, w(1), \cdots, w(m_1))^T$

$\Phi(\mathbf{x}) = (\phi_0(\mathbf{x}) = 1, \phi_1(\mathbf{x}), \cdots, \phi_{m_1}(\mathbf{x}))^T$

ϵ-insensitive loss function

$$L_\epsilon(d,y) = \begin{cases} |d-y| - \epsilon & \text{if } |d-y| \geq \epsilon \\ 0 & \text{otherwise} \end{cases}$$

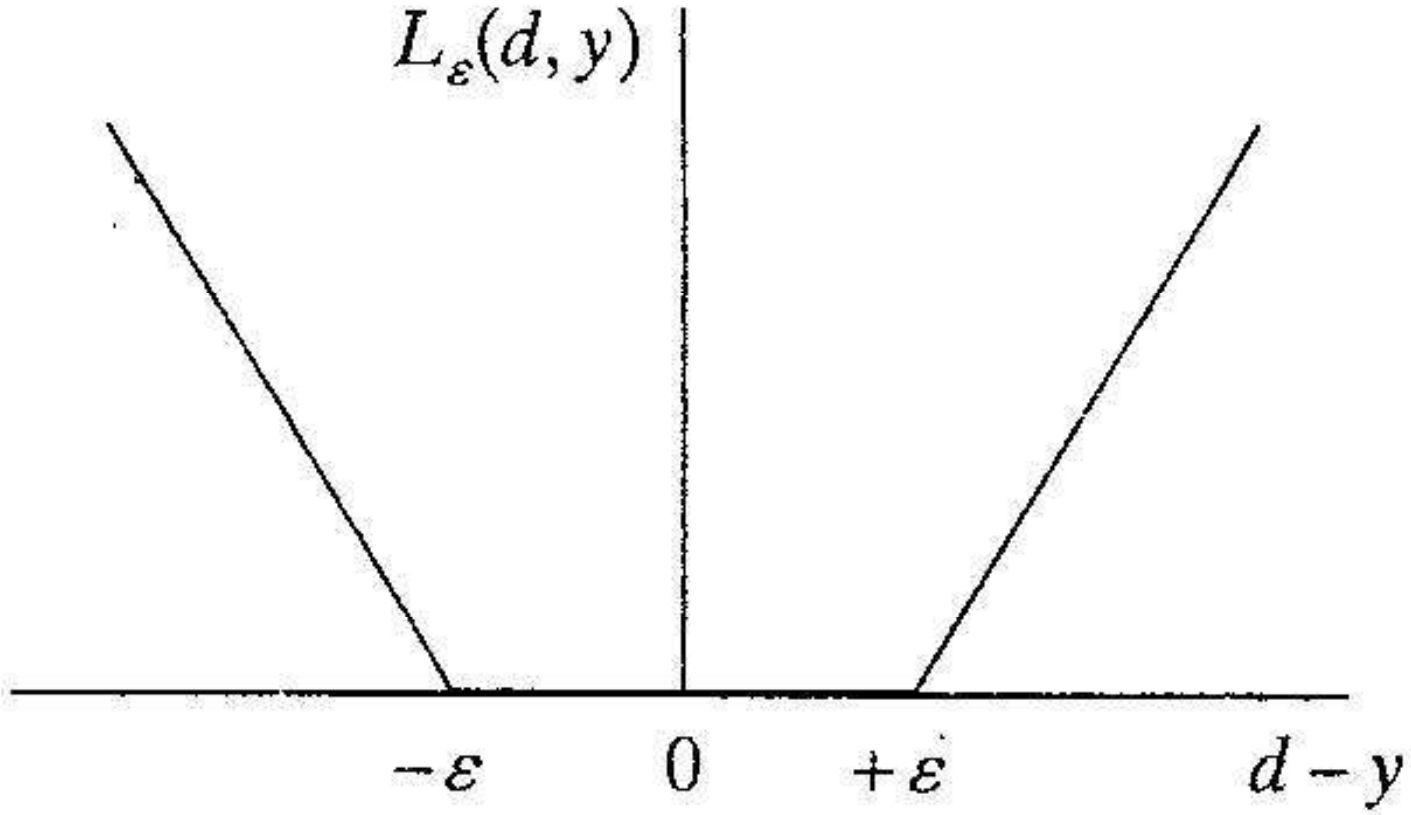
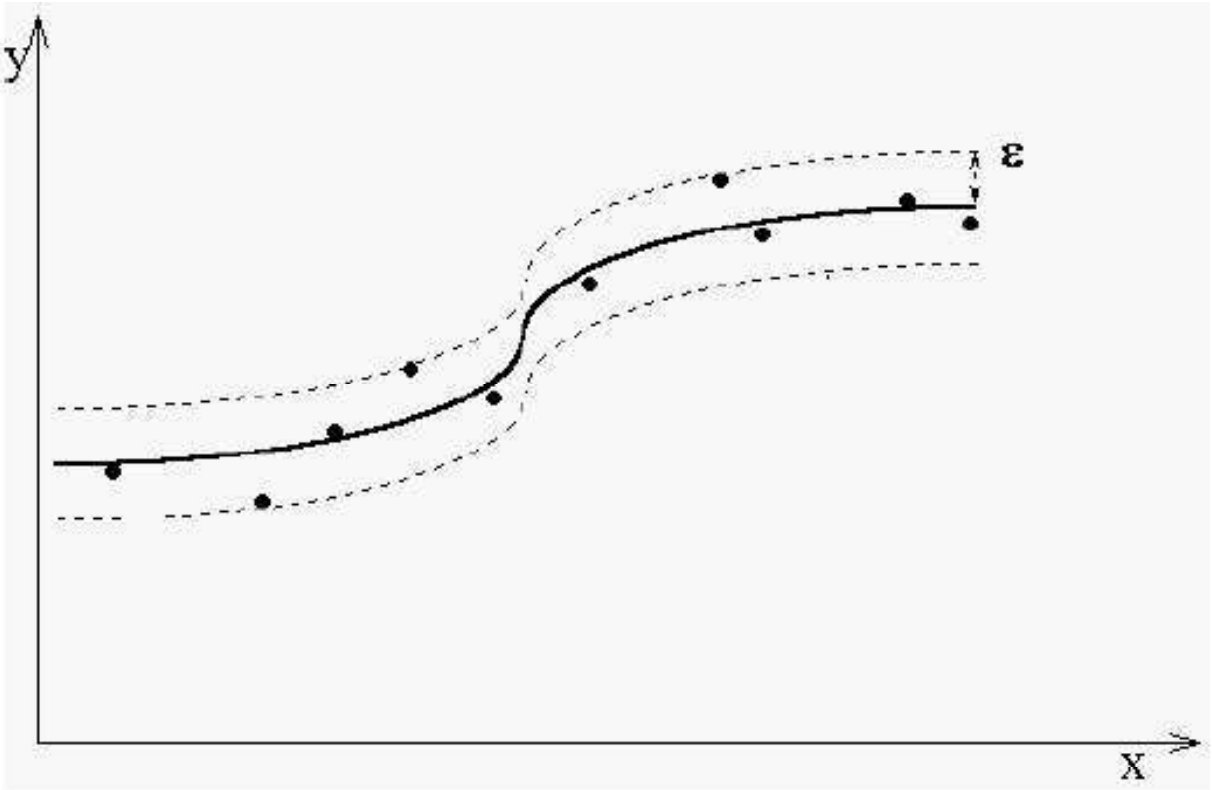


Figure: ϵ-insensitive loss function or soft margin loss.

ϵ-tube



Close points do not care, only support vectors care (as for classification)

Figure: ϵ-tube around d.

Note: again only the support vectors guide the solution, as the cost function ignores any training data close to the model prediction, i.e. within a threshold ϵ (for class. were for points that lie beyond the margin)

Formulating the optimization problem

Introduce the non-negative slack variables ξ_i' and ξ_i for all i=1,...,N

$$-\xi_i' - \epsilon \leq d_i - \mathbf{w}^T \Phi(\mathbf{x}_i) \leq \epsilon + \xi_i \; \forall i = 1, \cdots, N$$

This leads to the following constraints:

$$\left.\begin{array}{l} d_i - \mathbf{w}^T \Phi(\mathbf{x}_i) \leq \epsilon + \xi_i \\ \mathbf{w}^T \Phi(\mathbf{x}_i) - d_i \leq \epsilon + \xi_i' \\ \xi_i \geq 0 \\ \xi_i' \geq 0 \end{array}\right\} \forall i = 1, \cdots, N$$

Slack variables

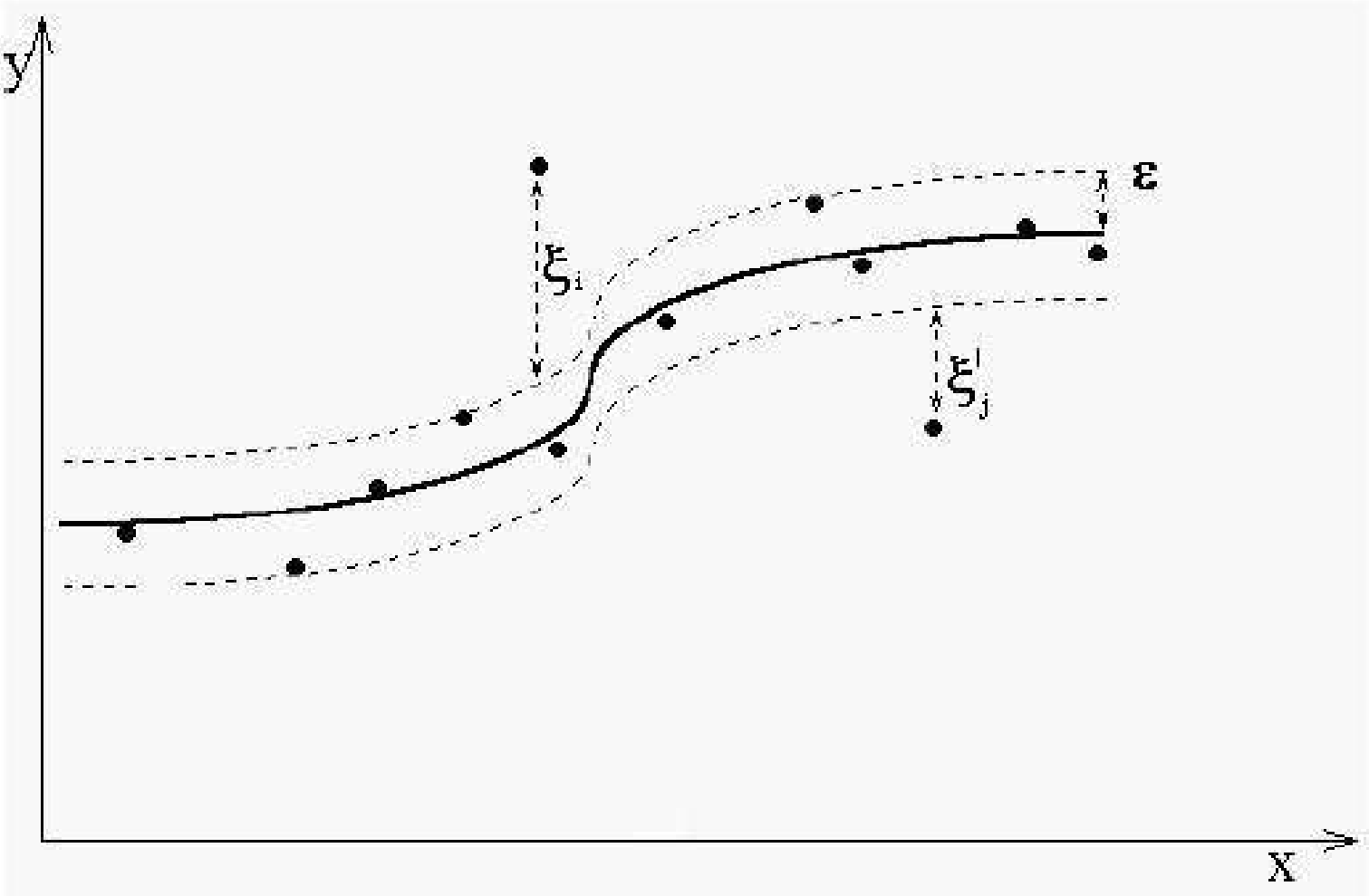


Figure: ϵ-tube and slack variables.

Primal Problem

Given the training set $T = \{(\mathbf{x}_i, d_i)\}_{i=1}^{N}$ find the optimal values of $\mathbf{w}$ such that the following objective function is minimized

$$\Psi(\mathbf{w}, \xi, \xi') = \frac{1}{2}\mathbf{w}^T\mathbf{w} + C\sum_{i=1}^{N}(\xi_i + \xi_i')$$

under the constraints

$$\left.\begin{array}{l} d_i - \mathbf{w}^T\Phi(\mathbf{x}_i) \leq \epsilon + \xi_i \\ \mathbf{w}^T\Phi(\mathbf{x}_i) - d_i \leq \epsilon + \xi_i' \\ \xi_i \geq 0 \\ \xi_i' \geq 0 \end{array}\right\} \forall i = 1, \cdots, N$$

Dual Problem

Dual Problem *

Given the training set $T = \{(\mathbf{x}_i, d_i)\}_{i=1}^N$ find the optimal values of $\{\alpha_i\}_{i=1}^N$ and $\{\alpha_i'\}_{i=1}^N$ which maximize the objective function

$$Q(\alpha, \alpha') = \sum_{i=1}^{N} d_i(\alpha_i - \alpha_i') - \epsilon \sum_{i=1}^{N}(\alpha_i + \alpha_i') - \frac{1}{2} \sum_{(i,j)=1}^{N} (\alpha_i - \alpha_i')(\alpha_j - \alpha_j')k(\mathbf{x}_i, \mathbf{x}_j)$$

under the constraints:

$$\sum_{i=1}^{N}(\alpha_i - \alpha_i') = 0$$

$$\left.\begin{array}{l} 0 \leq \alpha_i \leq C \\ 0 \leq \alpha_i' \leq C \end{array}\right\} \forall i = 1, \cdots, N$$

Remember that C is a user specified parameter

Computing the weight vector

Solving the dual problem we obtain the optimal values of the Lagrangian multipliers $\{\alpha_i\}_{i=1}^N$ and $\{\alpha_i'\}_{i=1}^N$

Then we can compute the optimal value of vector $\mathbf{w}$:

$$\mathbf{w} = \sum_{i=1}^{N} \underbrace{(\alpha_i - \alpha_i')}_{\gamma_i} \Phi(\mathbf{x}_i)$$

$$\Updownarrow$$

$$\mathbf{w} = \sum_{i=1}^{N} \gamma_i \Phi(\mathbf{x}_i)$$

Support vector

In this case support vectors correspond to non-zero values of γ_i

Computing the estimate

The estimate function is defined as $h(\mathbf{x}) = y = \mathbf{w}^T \Phi(\mathbf{x})$

Using the linear expansion for $\mathbf{w}$ we get:

$$h(\mathbf{x}) = \sum_{i=1}^{N} \gamma_i \Phi^T(\mathbf{x}_i)\Phi(\mathbf{x}) = \sum_{i=1}^{N} \gamma_i k(\mathbf{x}_i, \mathbf{x})$$

Wrapping Up

1. Important: we firstly have to select values for the user specified parameters C and ϵ
2. Important: we also have to choose an inner product kernel function k
3. We compute the kernel matrix K
4. We solve the optimization problem in the dual form and get the optimal values of the Lagrangian multipliers (we get the values of $\{\gamma_i\}_{i=1}^N$)
5. We compute the optimal value for the bias
6. We obtain the estimate function as a linear combiner of dot products in a feature space we can ignore –> $h(\mathbf{x}) = \sum_{i=1}^N \gamma_i k(\mathbf{x}_i, \mathbf{x})$

Test

Given an input pattern $\mathbf{x}$ we estimate the value of the unknown function f in that point using the estimate computed before:

$$h(\mathbf{x}) = \sum_{i=1}^{N} \gamma_i k(\mathbf{x}_i, \mathbf{x})$$

What You Should Know

- The definition of a maximum margin classifier
- How Maximum Margin can be turned into a QP problem ?
- What QP can do for you ? (but, for this class, you don't need to know how it does it)
- Why SVM approximate the SRM ?
- How we deal with noisy (non-separable) data?
- How we permit non-linear boundaries?
- How SVM Kernel functions permit us to pretend we're working with ultra-high-dimensional basis function terms?

Some references

- A Practical Guide to Support Vector Classification
  http://www.csie.ntu.edu.tw/ cjlin/papers/guide/guide.pdf (related to LIBSVM)
- Burges. A Tutorial on Support Vector Machines for Pattern Recognition
  http://research.microsoft.com/en-us/um/people/cburges/papers/svmtutorial.pdf
- Statistical Learning and Kernel Methods. – Bernhard Scholkopf
  MSR-TR-2000-23 - ftp://ftp.research.microsoft.com/pub/tr/tr-2000-23.pdf
- Muller, Mika, Ratsch, Tsuda, Scholkopf. An introduction to kernel-based learning algorithms. IEEE Trans. Neural Networks, 12(2):181-201, May 2001

Books [after the first they are quite deep]:

- Haykin. Neural Networks: A Comprehensive Foundation (2nd Edition)
  -chapter 6 « necessary and sufficient »
- Shawe-Taylor, Cristianini: Support Vector Machines ... Cambridge U.P., 2000
- Shawe-Taylor, Cristianini: Kernel Methods for Pattern Analysis. Cambridge U.P. 2004
- Scholkopf, Smola: Learning with kernels. MIT Press. Cambridge, MA, 2002.
- (The SLT bible) **Vapnik**. Statistical Learning Theory. Wiley, N.Y., 1998.
- (Compact form) **Vapnik**. The Nature of Statistical Learning Theory. Springer, N.Y., 1995 or very compact:
- V.N. Vapnik, An Overview of Statistical Learning Theory, IEEE Transactions on Neural Networks, 10 (5), 1999.

Summary of Main characteristics (pro and cons)

Dip. Informatica
University of Pisa

- Regularization embedded in the optimization problem (margin)
- Approximation of the theoretical structural risk minimization
  (bound to vc-dim of hyperplane minimized with the solution [hard-margin, no K.])
- Convex problem: **training always finds a global minimum**
- Implicit feature transformation using kernels

But

- Must choose kernel and kernel parameters
- Batch algorithm
- Very large problems are (were) computationally intractable
  - Problems with more than 20,000 examples *(are) were* very difficult to solve exactly with standard approaches.
  - However, many solution are proposed nowadays (including *gradient descent* approaches (!))

Advantages (repetita)



Dip. Informatica
University of Pisa

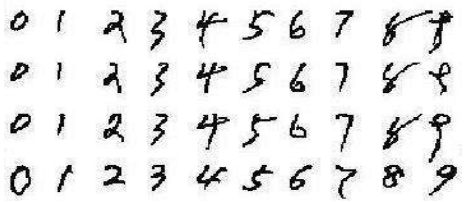
- Linear models (solver) with bound to the complexity in terms of margin (which is directly optimized)
  - Linear model (simplicity, compactness, …)
- Rich set of non-linear decision functions in the input (sample) space via Kernels
  - we are still doing a linear separation, but in a different space
  - Efficient (by kernels) treatment of the linear basis expansion
  - Rich set "not directly" implies high VC-dim (which for SVM depends on margin, K. parameters, C, etc.)

- Kernel trick → Kernel Methods (also without SVM): see later

SVM first (famous) application

Dip. Informatica
University of Pisa

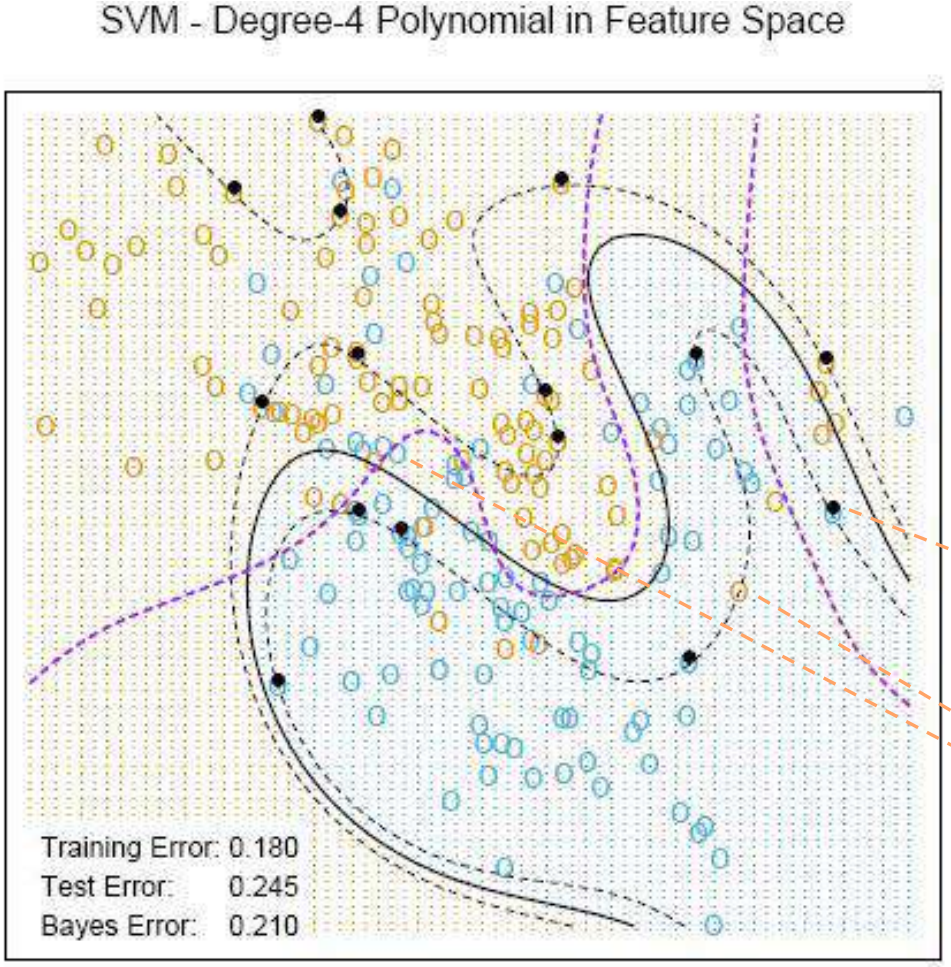
- The task we already know on handwritten digits (MNIST) represented an historical instance of successful applications for SVM



- E.g. 0.8% errors in ~1998 using a degree-9 polynomial SVM
- Which was noteworthy, comparable to LeNet, and obtained without years of developments on the architectures…
  - [the relevance of applications] Much more attention on SVM and SLT then followed…. (although related to 60'/70' studies)
  - [or] The relevance of good theory for successful applications

The mixture example: Polynomial Kernel results

Dip. Informatica
University of Pisa



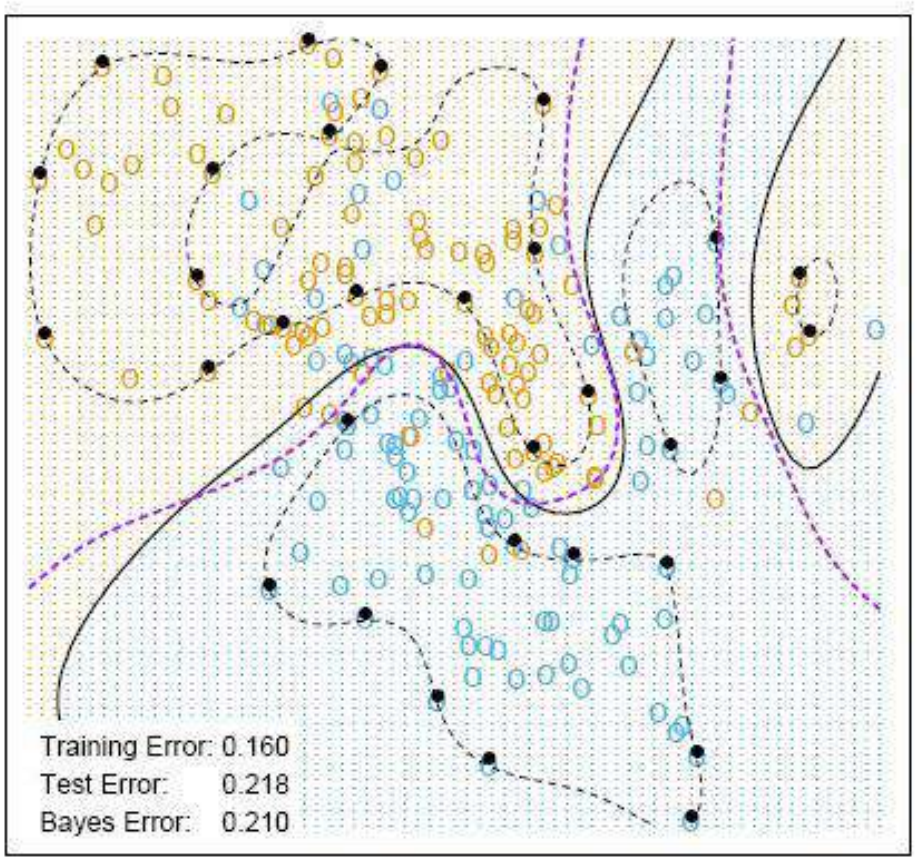
C tuned to the best performance
C=1(Hastie et al.)

42% of data results to be support vectors !

Support Vector: on the margin (slack=0)
or on the wrong side of the margin (slack >0)

The mixture example
RBF Kernel results

Dip. Informatica
University of Pisa



The radial basis kernel performs the best, as might be expected given the data arise from mixtures of Gaussians.

$\gamma = 1/(2\sigma^2) = 1$

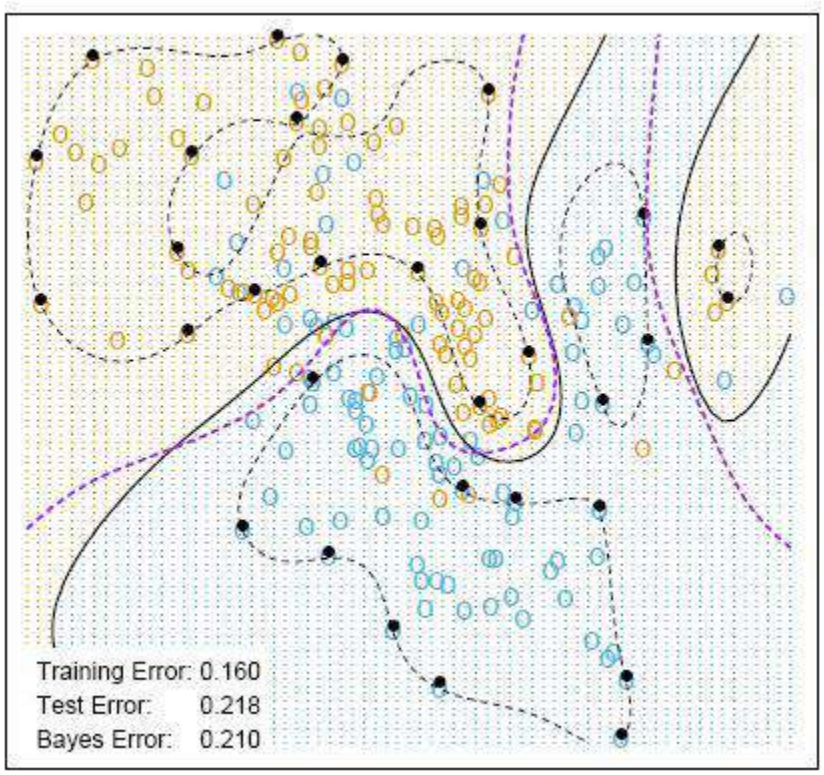
45% of data are support vectors !

$$k(\mathbf{x}, \mathbf{x}_i) = e^{-\frac{1}{2\sigma^2}\|\mathbf{x}-\mathbf{x}_i\|^2}$$

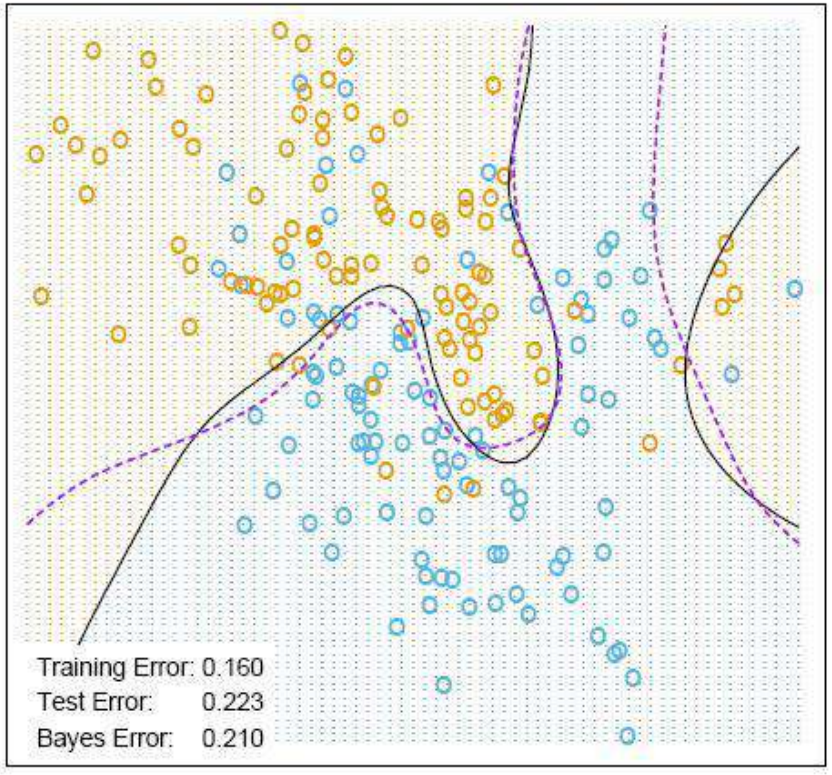
Do you remember NN results?

Dip. Informatica
University of Pisa

SVM - Radial Kernel in Feature Space



Neural Network - 10 Units, Weight Decay=0.02



SVM: practice of (I)

Dip. Informatica
University of Pisa

"Currently there exists no theory which *guarantees* that a given family of SVMs will have high accuracy on a given problem" [burges 98].

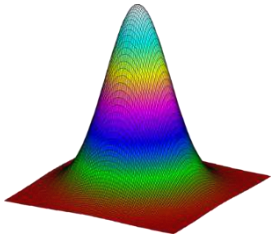
The nice property of hard-margin classifiers cannot be directly extended to soft-margin and kernels

(C parameter, Kernel can lead to infinite VC-dim of the SVM classifier)

E.g. :

$$h(\mathbf{x}) = sign(\sum_{i=1}^{N} \alpha_i d_i k(\mathbf{x}, \mathbf{x}_i))$$

- Gaussian RBF SVMs of sufficiently small width (σ) can classify an arbitrarily large number of training points correctly (like 1-NN only 1 SV point, the closet one to **x**, will contribute to the solution), and thus it can have infinite VC dimension (unless you regularize the model)
- On the opposite with large width of the Gaussian all SV points are considered and you get a sort of "global average", low VC-dim

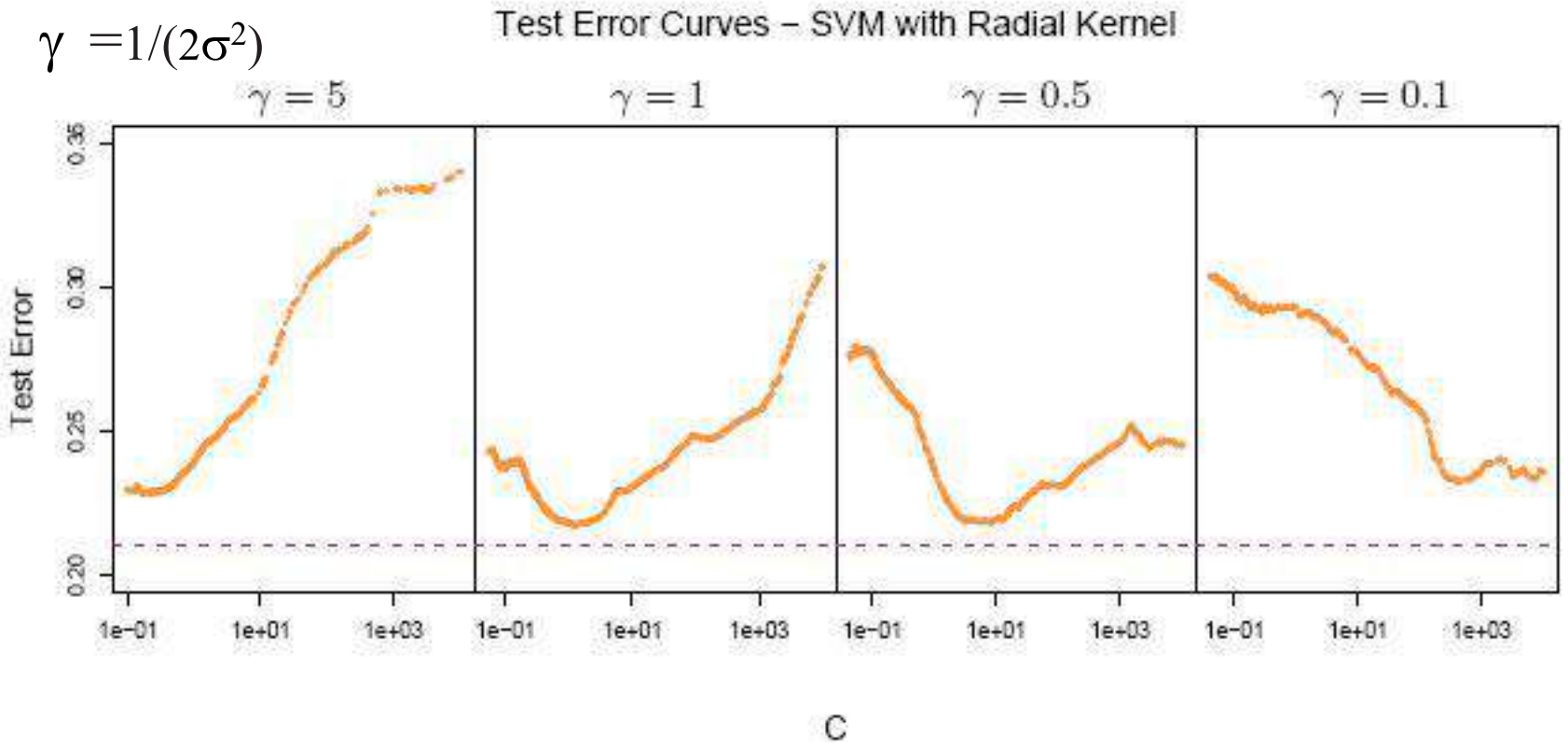


→ Controlling the width we can control the VC-dim

Critical role of Hyper-parameters on the performance

Dip. Informatica
University of Pisa

$\gamma = 1/(2\sigma^2)$



γ =5 (small σ, narrow peaked Kernels) require lower C value (max regularization). Hence, the best value of C depends also on the kernel hyper-parameters !

Often selected using cross-validation: take care !

SVM: practice of (II)

Dip. Informatica
University of Pisa

- To apply the SVM to practical problems, one needs to choose the type of kernel (and its parameters) and the C value (also e for regression).
- Rigorous selection requires an estimation of the complexity (VC-dim).
- Hyper-parameters affecting model complexity → hyper-parameters used for model selection
- In practice: again (carefully) empirical evaluation
  - Hyper-parameters fitting: tuning of hyper-parameters (C for soft-margin, gamma for RBF K., etc.) can introduce **overfitting** (use a validation set to set hyper parameters and then a test set, or double-nested CV, etc….)

SVM: practice of (III)

Dip. Informatica
University of Pisa

- Number of support vector: assumed low in the discussion: Not always true ! (see the examples at the beginning!)
- Performance: often good, *not always* the comparison is favorable w.r.t. other ML methods.
- E.g. Nowadays (after 2012/13) NN Deep model have shown to largely outperform previous records on image and speech recognition tasks!

SVM Folklore (and Exercise)

Dip. Informatica
University of Pisa

- Now you can answer by yourself to same typical erroneous issues on SVM (not *super* vector machine ;-)):

1. SVM can be used with default hyper-parameters values
2. I choose SVM because it does not fall into over-fitting
3. SVM has not problems using high-input dimension (does it solve the curse of dimensionality?*)
4. By SVM is like to find automatically the "number of units"
5. …

* Solution to 3: SVM does **not** solve curse of dimensionality of the input space !!
(as it proper deals with high dimensional in the feature space, not in the input space!)

Kernel Methods (I)

Dip. Informatica
University of Pisa

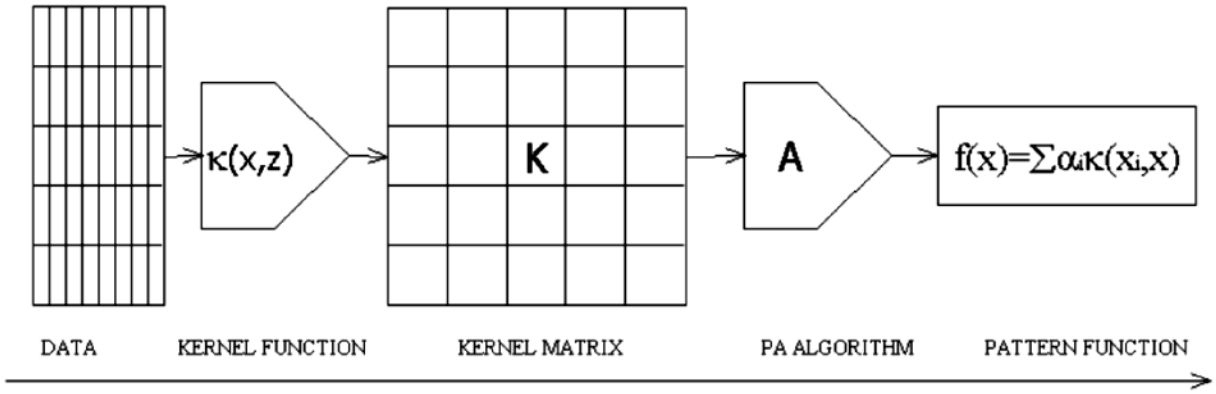
- Kernel trick → Kernel Methods (also without SVM)

- "Kernelization" of previous (traditional) approaches whenever you had a dot product inside your model or a similarity measure enabling them to operate in a new implicit high-dimensional space just specifying a *kernel* (and changing it)

Kernel Methods (II)

Dip. Informatica
University of Pisa

- Kernel trick → Kernel Methods (also without SVM)
  - Very efficient *fixed high-dim* **Linear basis expansion**
    - **Modular choice of the basis functions by the kernel**
    - **Unified theoretical framework for general dot product functions**



K can change without changing the learning machine!!!

Kernel Methods (III)

Dip. Informatica
University of Pisa

Moreover

- $K(s, t)$ can be related to a similarity measure comparing s and t

$$d_k(s,t) = \sqrt{k(s,s) - 2k(s,t) + k(t,t)}$$

- Similar objects (small distance) ←→ high value of the kernel
- The idea underpinning the use of a kernel function is to focus on the representation and processing of **pairwise comparisons** rather than on the representation and processing of a single object.
  - For some domain it may be easier to compare two objects instead of defining some abstract space of features where to represent a single object.

Kernel Methods (IV)

Dip. Informatica
University of Pisa

- An SVM is largely characterized by the choice of its kernel
  - Please, compare with the use of "adaptive" basis functions in NN :

  recall that the basis functions themselves were in NN adapted to data by fitting of w in phi (w of hidden units)

- Instead, the best choice of kernel for a given problem is still a research issue

Kernel: Advanced topics (I)

Dip. Informatica
University of Pisa

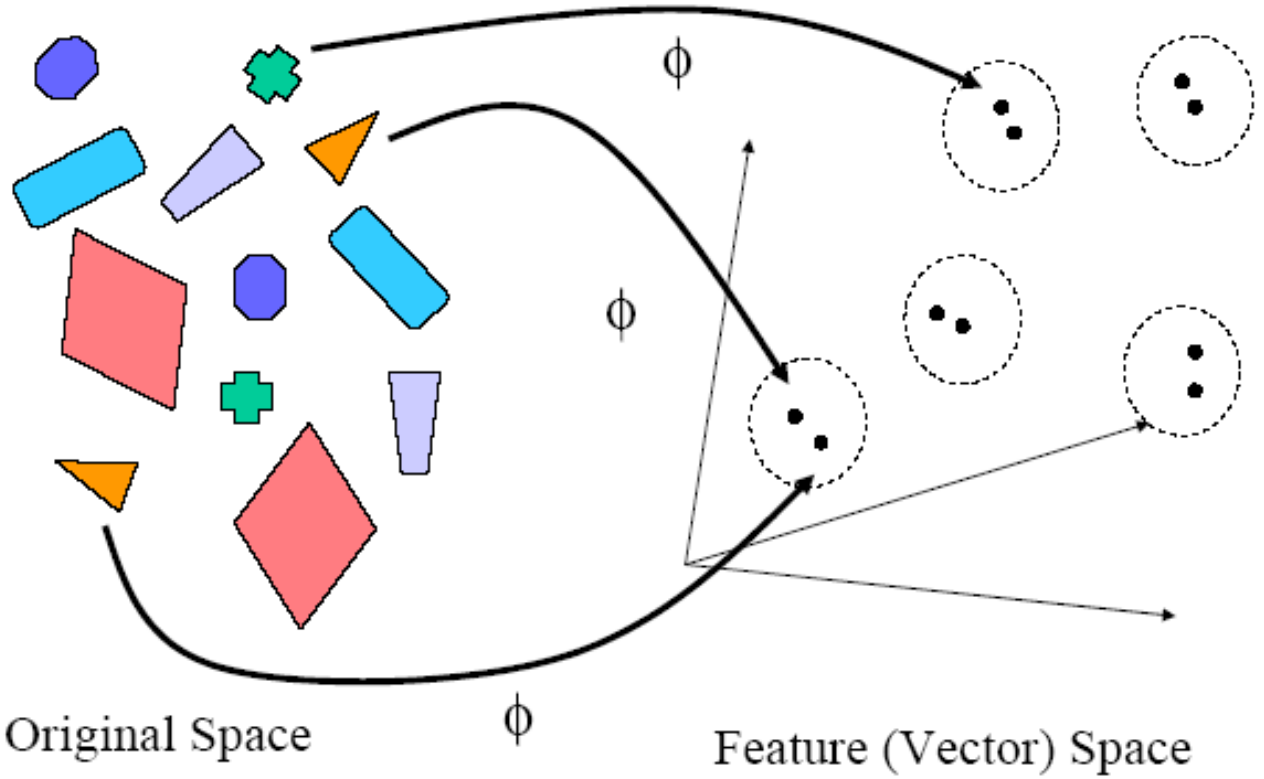
- $K(s, t)$ can be related to a similarity measure comparing s and t

$$d_k(s,t) = \sqrt{k(s,s) - 2k(s,t) + k(t,t)}$$

- *Exercise*: Re-think it as for the distance-based methods (see k-nn):
  - when is it a good kernel? *
  - when is it a bad kernel? **

A good kernel for “objects”

Dip. Informatica
University of Pisa



Kernel: Advanced topics (II)

Dip. Informatica
University of Pisa

- Design of new kernels: Need to choose a "good" kernel function for the specific domain and it can be extended to generic object in inputs (not only vectors!) :
  - e.g. for strings/trees/graphs
  - ad hoc for the data domain and task (e.g. to encode similarity measure useful in NLP, bioinformatics, …)*
  - efficiency (algorithm issues**), ….
  - adaptive kernels: a research issue

* e.g. edit-distance kernel for bioinformatics or language suffix-tree for strings rep. → the comparison can be made in various efficient ways

Note that the background knowledge is sperated form the solver

Notation summary

Progressive draft

Generally, we use a consistent notation among the most important course parts. Only on the need and in order to simplify, slight *local* variations are made and specified according to the context (as indicated in the specific lectures).

The following guide shows the general case and can help to summarize the main variations according to the different sub-cases.

DATA:

Pattern	x_1	x_2	x_i	x_n
Pat *1*	$x_{1,1}$	$x_{1,2}$	…	$x_{1,n}$
…	…	…	…	…
Pat *p*	$x_{p,1}$	$x_{p,2}$	$x_{p,i}$	$x_{p,n}$
…				

$\boldsymbol{X}$ is a matrix $l \times n$

l rows (patterns),

n columns (features/variables/attributes/components)

$p=1..l, \quad i=1..n$

We often need to omit some indices when the context is clear, e.g.:

- Each row, generic $\boldsymbol{x}$ (vector-bold): a raw in the table: (input) example, pattern, instance, sample, input vector, ….
- x_i or x_j (scalar): component i or j (given a pattern $\boldsymbol{x}$, i.e. omitting p)
- $\boldsymbol{x}_p$ (or rarely other indices e.g. $\boldsymbol{x}_i$) (vector-bold): p-*th* (i-*th*) raw in the table = pattern p (or i)
- $x_{p,i}$ (scalar) also as $(\boldsymbol{x}_p)_i$: component i of the pattern p
  or we can often use also $\boldsymbol{x}_{p,j}$ for the component j, *etc.*
- For the target we will typically use just y_p with $p=1..l$ (the same for d_p or t_p)

General:

- $\boldsymbol{x}$ is the input pattern
- o (or sometimes *out)* is the model output component (e.g. an the output of a unit in Neural Networks)
- $h(\boldsymbol{x})$ is the hypothesis/model output: the output of the model $h(\boldsymbol{x})$ can be of different nature according to the task (e.g. discrete for classification, a real number for regression).

- h is the hypothesis emitted by the learning algorithm to approximate the unknow target function f.
- $h_{\boldsymbol{w}}$ can be used to highlight the dependencies on the parameters $\boldsymbol{w}$.

Error and Risk: expected value of the "inner" loss/measure $L(h(\boldsymbol{x}),d)$ that changes according to the tasks and models (see first- intro lectures and validation lectures).

- Typically $E(\boldsymbol{w})=\frac{1}{r}\sum_{p=1}^{r} L(h(\boldsymbol{x}_p), d_p)$ over a generic set of r data (it can be measured for training, validation or test according to the context)
- A typical example is the MSE used for the Linear models and Neural Networks, for l training data:

$$E(\boldsymbol{w}) = \frac{1}{l}\sum_{p=1}^{l}(d_p - h_{\boldsymbol{w}}(\boldsymbol{x}_p))^2$$

- Risk function over all the data: R, see SLT and Validation lectures
- Empirical risk: R_{emp} = $E(\boldsymbol{w})$ error, surrogate of R on a finite set of data, typically used for the training data (see SLT, whereas $R_{emp} = R_{emp}(h, TR)$) or also for a validation/test part (see Validation lectures, specifying the h and the data split D in the form $R_{emp}(h, D)$, within the purpose of training, e.g. on D_{TR} in the simple hold-out, model selection, e.g. on D_{VL} in the simple hold-out, or model assessment, e.g. on D_{TS} in the simple hold-out, as specified in the model selection/assessment algorithms).

Objective function for model training: it can be of different nature according to the task and model (also changing the "inner" loss L) and it can (or not) include regularization aspects, e.g. for Linear models and Neural Networks with Tikhonov regularization the objective function is the following *Loss* function (see the lectures for details):

$$Loss(\boldsymbol{w}) (\text{or } Loss(h_{\boldsymbol{w}})) = \text{ error data term (MSE) + penalty term}$$

Specific cases

In Linear models/LTU (the basic):

As in the table above:

- Inputs: $\boldsymbol{x}$
- Outputs: $h(\boldsymbol{x})$
- Targets: typically y (regression); y or d (classification)

In Neural Networks:

- Inputs: $\boldsymbol{x}$ (loaded in the input layer). Also, $\boldsymbol{x}$ can represent a generic input to each unit, either from extern source (input pattern) or another (also hidden) unit according to the position of the unit in the network.
  As explained in the slide "The NN and their Units" with a picture: If we load the pattern $\boldsymbol{x}$ in the input layer, we can use the notation with $\boldsymbol{o}$ for both the inputs and the hidden units outputs. Hence, *inside the network*, the input to each unit t (either hidden and output

units) from *any source u* (through the connection w_{tu}) is typically denoted as o_u (this is the style used in the Backpropagation derivation).

- Outputs: $h(\boldsymbol{x}) = \boldsymbol{o}(\boldsymbol{x})$ at last layer (with more than one output units) and $h(\boldsymbol{x}_p) = \boldsymbol{o}(\boldsymbol{x}_p)$.
  In Backprop derivation we simplified using o_k (or o_u etc.) as output of the unit k (or u) for a pattern p, i.e. omitting p (and the same for other vectors used therein for a single pattern p).
  I.e. the subscript is used therein for the indices of the units (see also such usage explained above after the main data table for components of a vector omitting p) .
- Targets: d scalar (or vector $\boldsymbol{d}$ for multioutput networks)

In SVM:

- Inputs: **x**
- Output: $h(\mathbf{x})$
- Targets: d (as for classification above)

In SVM the bias w_0 is typically denoted as b

Slight variations to be consistent with Haykin chapter 6:
$l \rightarrow$ N (number of data), $n \rightarrow$ m (input dimension)
We use i and j for the patterns indices.

Special cases (different domains)

In the Introduction for Concept Learning (Inductive Bias) part

This is a particular case due to the discrete nature of the hypothesis space in propositional logic (and we partially use the nice framework from the Mitchel presentation, with very slight changes w.r.t. to the previous style)

Target function: d: X→{0,1} (c: X→{0,1} in the Mitchel book)
TR set couples <$\boldsymbol{x}$, $d(\boldsymbol{x})$> in TR (or D in the Mitchel book)
h: X→{0,1} (made with literals l_i of propositional logics)

- Inputs: $\boldsymbol{x}$
- Outputs: $h(\mathbf{x})$
- Targets: $d(\boldsymbol{x})$

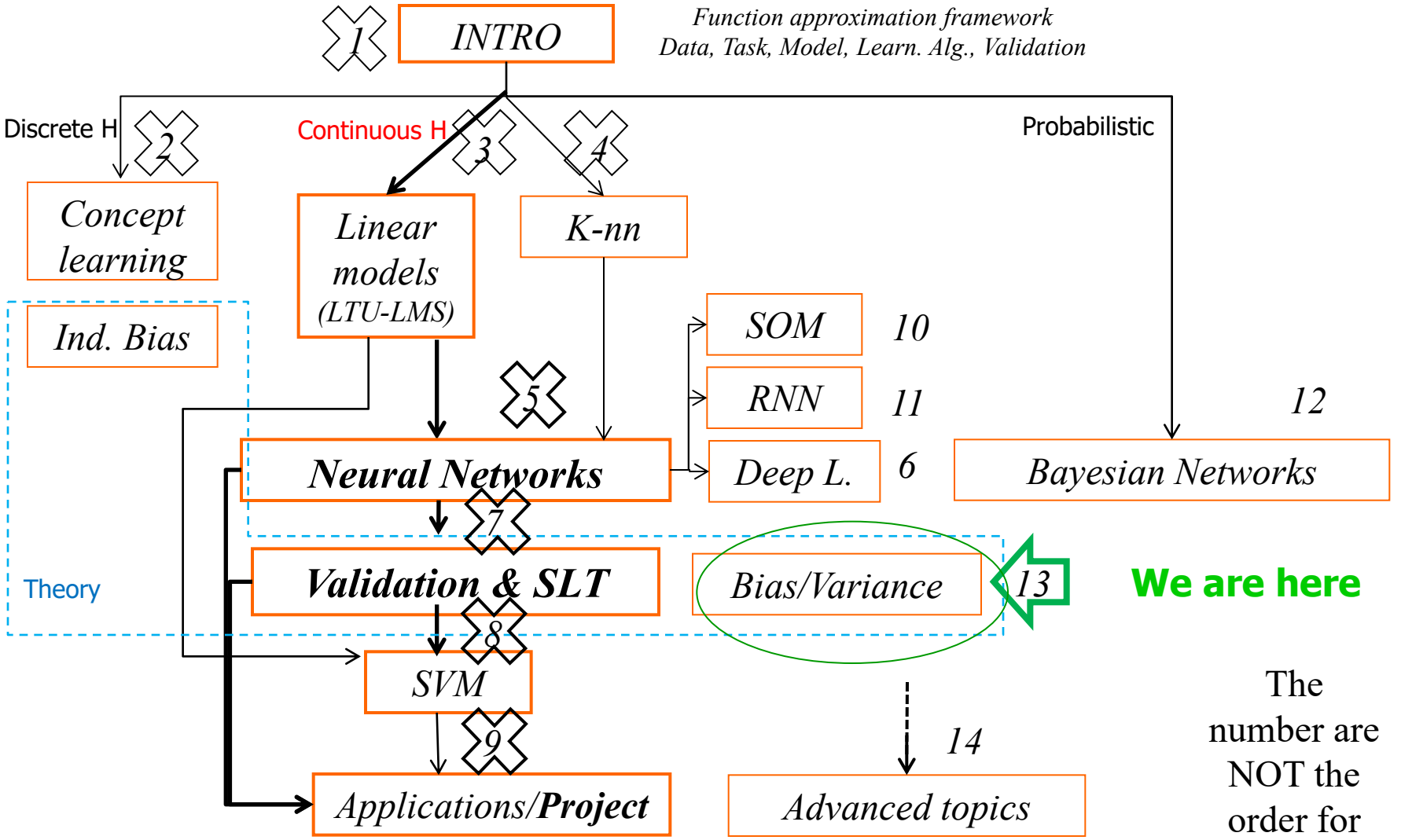
In RNN:

The domain changes from vectors to sequences/time-series:

- Inputs: $\boldsymbol{l}(t)$, components of a sequence
- Outputs: $o(t)$ or $y(t)$
- Targets: $d(t)$
- $\boldsymbol{x}(t)$ in RNN denotes the states values

ML Course structure
Where we are

Dip. Informatica
University of Pisa



The number are NOT the order for this year

Bias-Variance

Dip. Informatica
University of Pisa

- A training set is only one possible realizations from the universe of data: different training set can provide different estimate
- Decomposition of expected (on various TR) error at a point x:
  - **Bias**: quantifies the discrepancy between true function and $h(x)$ (averaged on data): if H is too small it is high
  - **Variance**: quantifies the variability of the response of model *h* for different realizations of the training data: due to high flexibility
  - **Noise**: because the label include random error
    - E.g. if for a given x there are more than one possible *d*

- Assume the **regression** scenario with target y *and* L_2 *(squared error loss)*

Credits: Pictures from Macskcassy, Bishop, Bengio

Bias-Var. Analysis in Regression

Dip. Informatica
University of Pisa

- Suppose we have examples $\langle \mathbf{x}, y \rangle$ where the true function is $y = f(\mathbf{x}) + \varepsilon$ and where ε is Gaussian noise with zero mean and standard deviation σ.
- In linear regression, given a set of examples $\langle \mathbf{x}_i, y_i \rangle \quad i=1..l$ we fit a linear hypothesis $h(\mathbf{x})=\mathbf{w}\mathbf{x}+w_0$, such as to minimize sum-squared error over the training data:

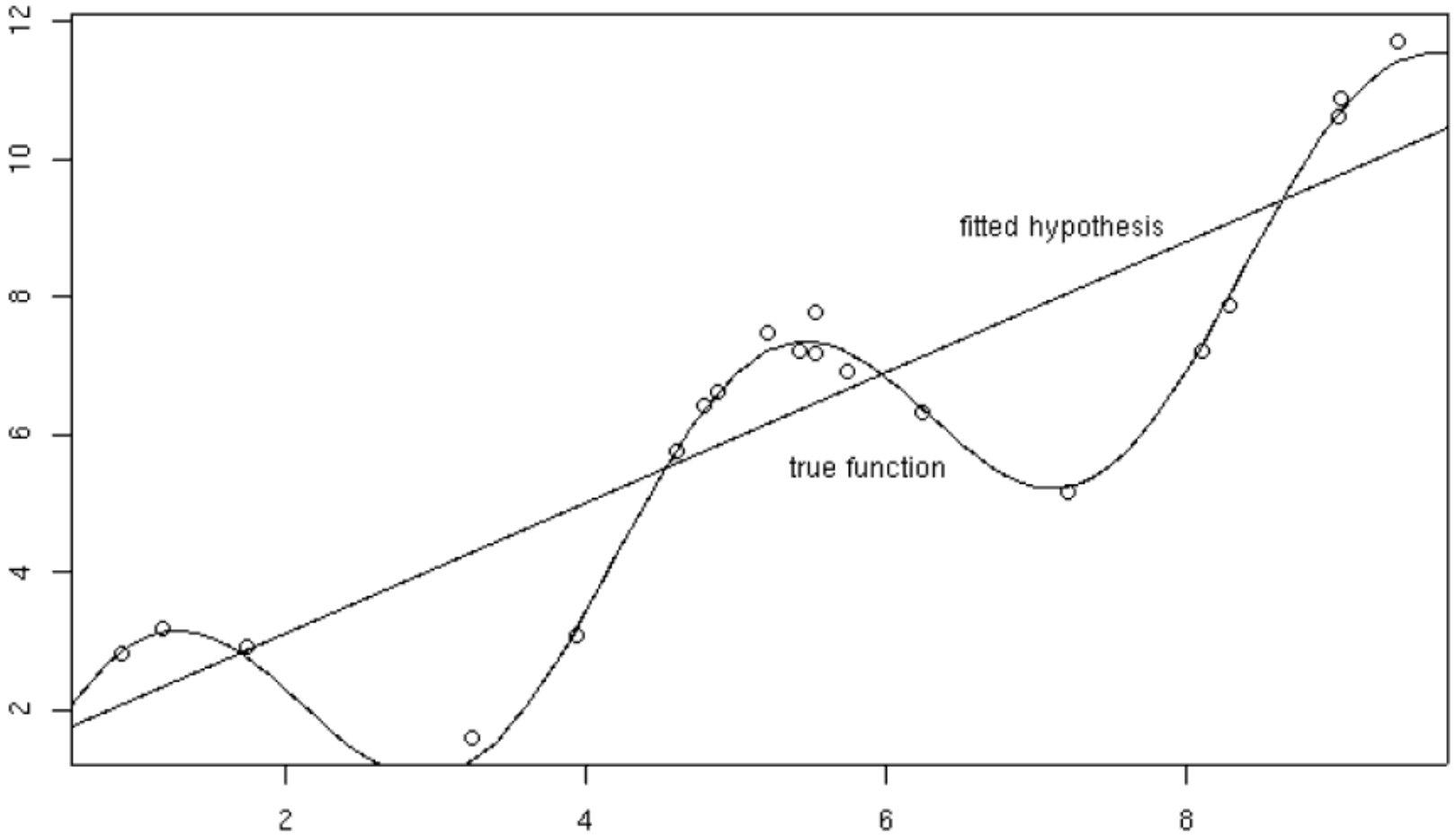
$$\sum_{i=1}^{l} (y_i - h(\mathbf{x}_i))^2$$

1) Because of the hypothesis class that we chose (linear hypotheses) for some function f, we will have a systematic prediction error
2) Depending on the data set we have, the parameters $\mathbf{w}$ that we find will be different.

Example: 20 points

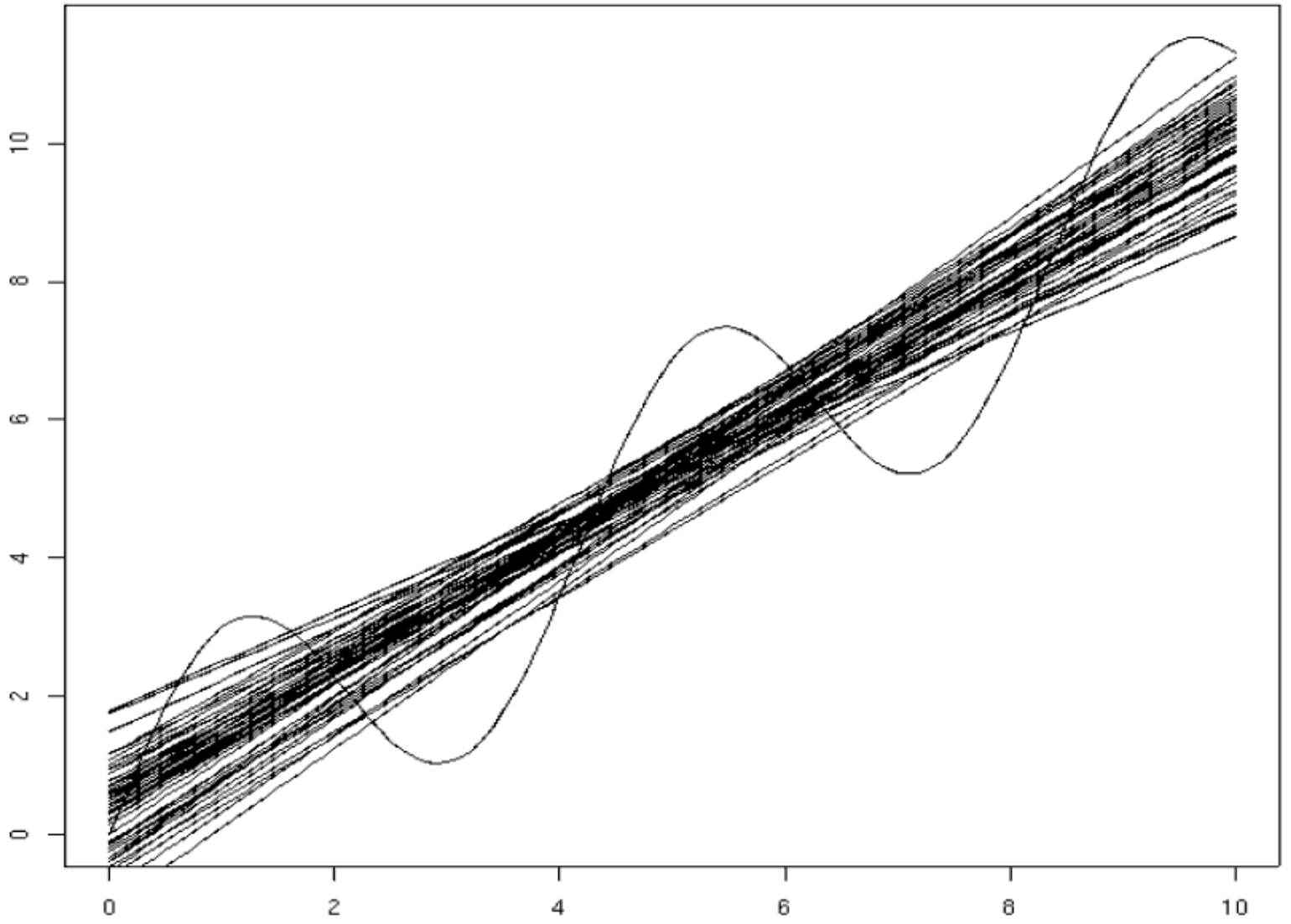
Dip. Informatica
University of Pisa

$$y = x + 2\sin(1.5x) + N(0,0.2)$$



Example: 50 fits (each on 20 points)

Dip. Informatica
University of Pisa



Bias-Var. Analysis

Dip. Informatica
University of Pisa

- Given a new data point **x**, what is the **expected prediction error?**
- Assume that the data points are drawn i.i.d.* from *a unique underlying probability distribution P*
- The goal of the analysis is to compute, for an arbitrary new point **x**,

$$E_P\left[(y - h(\mathbf{x}))^2\right]$$

Note: there is a different h (and y) for each different "extracted" training set :

where y is the value for **x** that could be present in a data set, and the expectation is over *all training sets* drawn according to P.

- We will decompose this expectation into three components:

  bias, variance and noise

* i.i.d: independent and identical distributed

Recall of basic statistics

Dip. Informatica
University of Pisa

- Let Z be a random variable with possible values z_i, $i=1..l$ and with probability distribution $P(Z)$
- The **expected value** or **mean** of Z is:

$$\bar{Z} = E_P[Z] = \sum_{i=1}^{l} z_i P(z_i)$$

- If Z is continuous the sum is replaced by an integral, and the distribution by a density function.
- The **variance** of Z is:

$$Var[Z] = E[(Z - \bar{Z})^2]$$
$$= E[Z^2] - \bar{Z}^2$$

(this last line is proven on the next slide)

Variance Lemma

Dip. Informatica
University of Pisa

- $Var[Z] = E[(Z - \bar{Z})^2)] = \sum_{i=1}^{l} (z_i - \bar{Z})^2 \, P(z_i) =$

  $\sum_{i=1}^{l} ({z_i}^2 - 2 z_i \bar{Z} + \bar{Z}^2) P(z_i) =$

  $\sum_{i=1}^{l} {z_i}^2 \, P(z_i) - 2\bar{Z} \sum_{i=1}^{l} z_i \, P(z_i) + \bar{Z}^2 \sum_{i=1}^{l} P(z_i) =$

  $= E[Z^2] - 2 \, \bar{Z} \, \bar{Z} + \bar{Z}^2 \cdot 1 =$

  $= E[Z^2] - \bar{Z}^2 = Var[Z]$

- We will use the form

$$E[Z^2] = \bar{Z}^2 + Var[Z]$$

Bias-Var. Decomposition (1)

Dip. Informatica
University of Pisa

$$E_P\left[(y - h(\mathbf{x}))^2\right] = E_P\left[h(\mathbf{x})^2 - 2yh(\mathbf{x}) + y^2\right]$$
$$= E_P\left[h(\mathbf{x})^2\right] + E_P\left[y^2\right] - 2E_P[y]E_P[h(\mathbf{x})] \quad *$$

- Let $\bar{h}(\mathbf{x}) = E_P[h(\mathbf{x})]$ denote the *mean prediction* of the hypothesis at **x**, when h is trained with data drawn from P
- For the first term, using the variance lemma*, we have:

$$E_P\left[h(\mathbf{x})^2\right] = E_P\left[(h(\mathbf{x}) - \bar{h}(\mathbf{x}))^2\right] + \bar{h}(\mathbf{x})^2$$

- Note that $E_P[y] = E_P[f(\mathbf{x}) + \varepsilon] = f(\mathbf{x})$
- For the second term, using the variance lemma*, we have:

$$E_P\left[y^2\right] = E_P\left[(y - f(\mathbf{x}))^2\right] + f(\mathbf{x})^2$$

* Lemma: $E[Z^2] = Var[Z] + \bar{Z}^2$

Bias-Var. Decomposition (2)

Dip. Informatica
University of Pisa

$$E_P\left[h(\mathbf{x})^2\right] + E_P\left[y^2\right] - 2E_P[y]E_P[h(\mathbf{x})]$$

- Putting everything together, we have:

$$\begin{aligned}
E_P\left[(y - h(\mathbf{x}))^2\right] &= E_P\left[(h(\mathbf{x}) - \bar{h}(\mathbf{x}))^2\right] + \\
&\quad \bar{h}(\mathbf{x})^2 - 2f(\mathbf{x})\bar{h}(\mathbf{x}) + f(\mathbf{x})^2 + \\
&\quad E_P\left[(y - f(\mathbf{x}))^2\right] \\
&= E_P\left[(h(\mathbf{x}) - \bar{h}(\mathbf{x}))^2\right] + \qquad \text{(variance)} \\
&\quad (\bar{h}(\mathbf{x}) - f(\mathbf{x}))^2 + \qquad (\text{bias})^2 \\
&\quad E_P\left[(y - f(\mathbf{x}))^2\right] \qquad (\text{noise})^2 \\
&= \text{Var}[h(\mathbf{x})] + \text{Bias}[h(\mathbf{x})]^2 + E_P[\varepsilon^2] \\
&= \text{Var}[h(\mathbf{x})] + \text{Bias}[h(\mathbf{x})]^2 + \sigma^2
\end{aligned}$$

- Expected prediction error = Variance + Bias^2 + Noise^2

Bias/Variance results

Dip. Informatica
University of Pisa

- Decomposition of expected (on various TR) error at a point $\mathbf{x}$:
  - **Bias**: quantifies the discrepancy between true function and *h(x)* (with *h(x)* averaged over different TR data) [*systematic error*]
    - E.g. Due to too small H, too rigid model

    $$[\bar{h}(\boldsymbol{x}) - f(\boldsymbol{x})]^2$$

  - **Variance**: quantifies the variability of the response of model *h* for different realizations of the training data
    - Due to too high flexibility

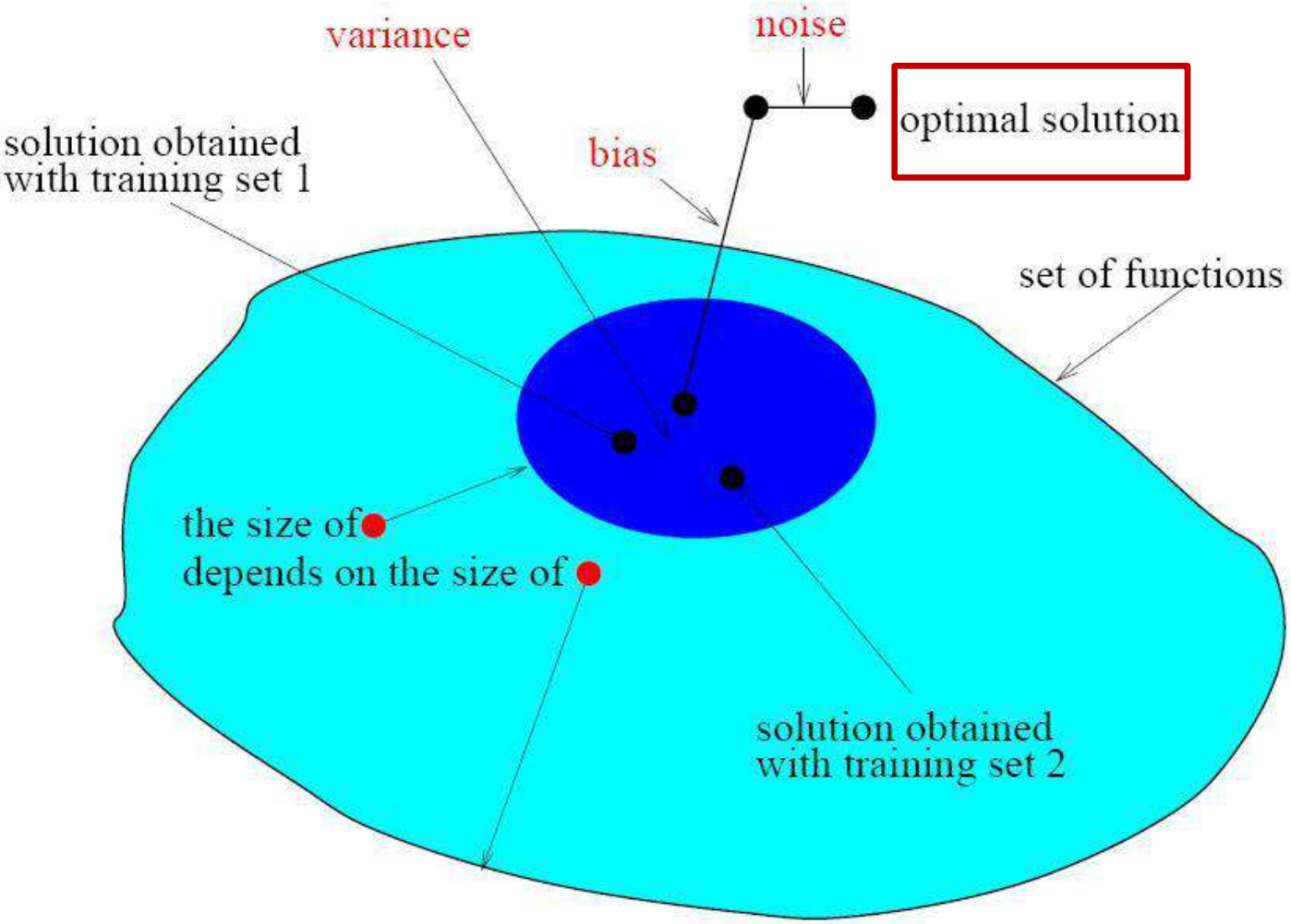
    $$E_P\left[(h(\mathbf{x}) - \bar{h}(\mathbf{x}))^2\right]$$

  - **Noise**: even the optimal solution could be wrong!
    - E.g. if for a given $\mathbf{x}$ there are more than one possible *d*

    [*irreducible*: does not depends on the model, we have a tolerance on the response (σ, sigma)]

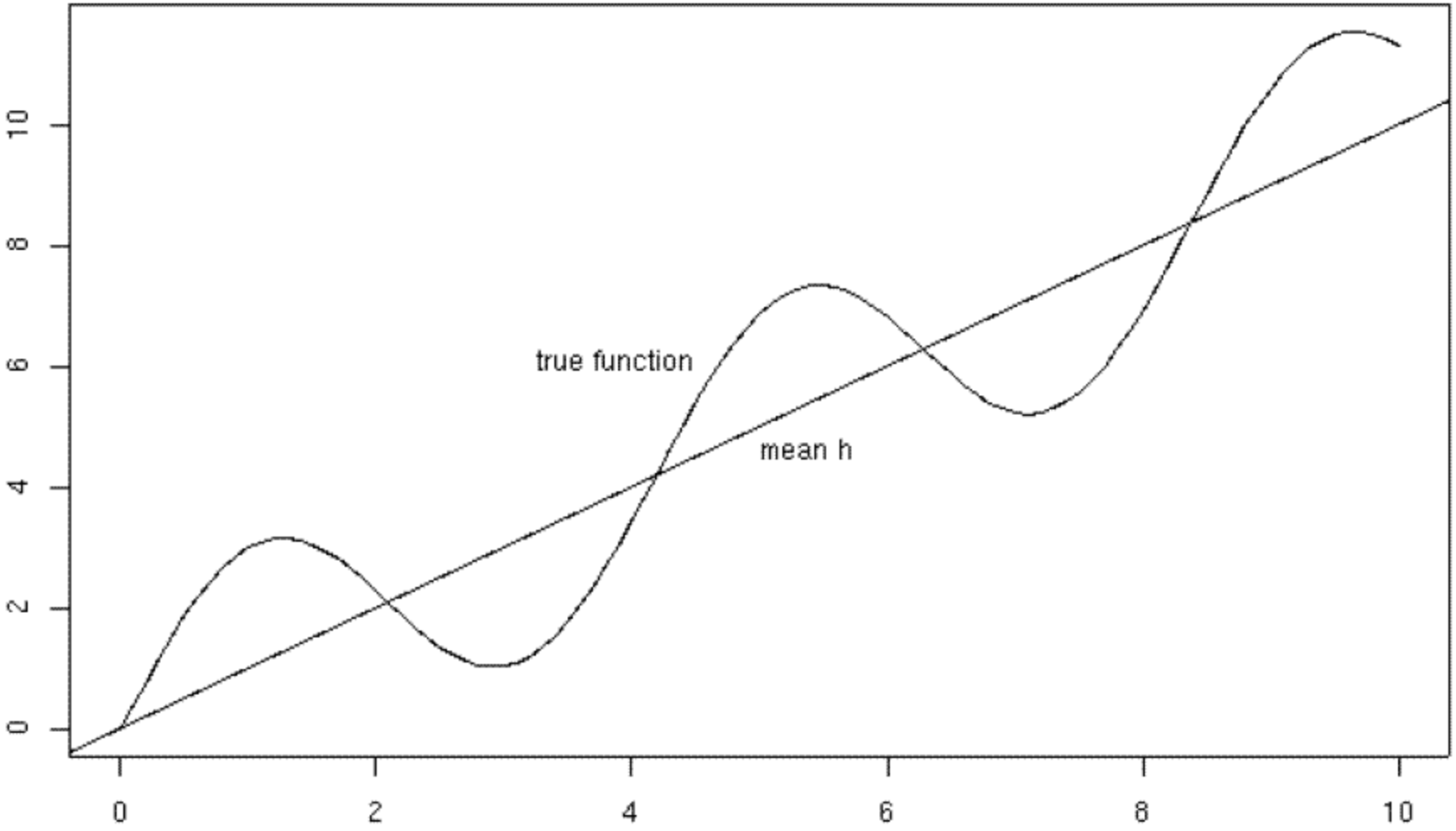
Bias-Variance (graphical view)

Dip. Informatica
University of Pisa



Bias

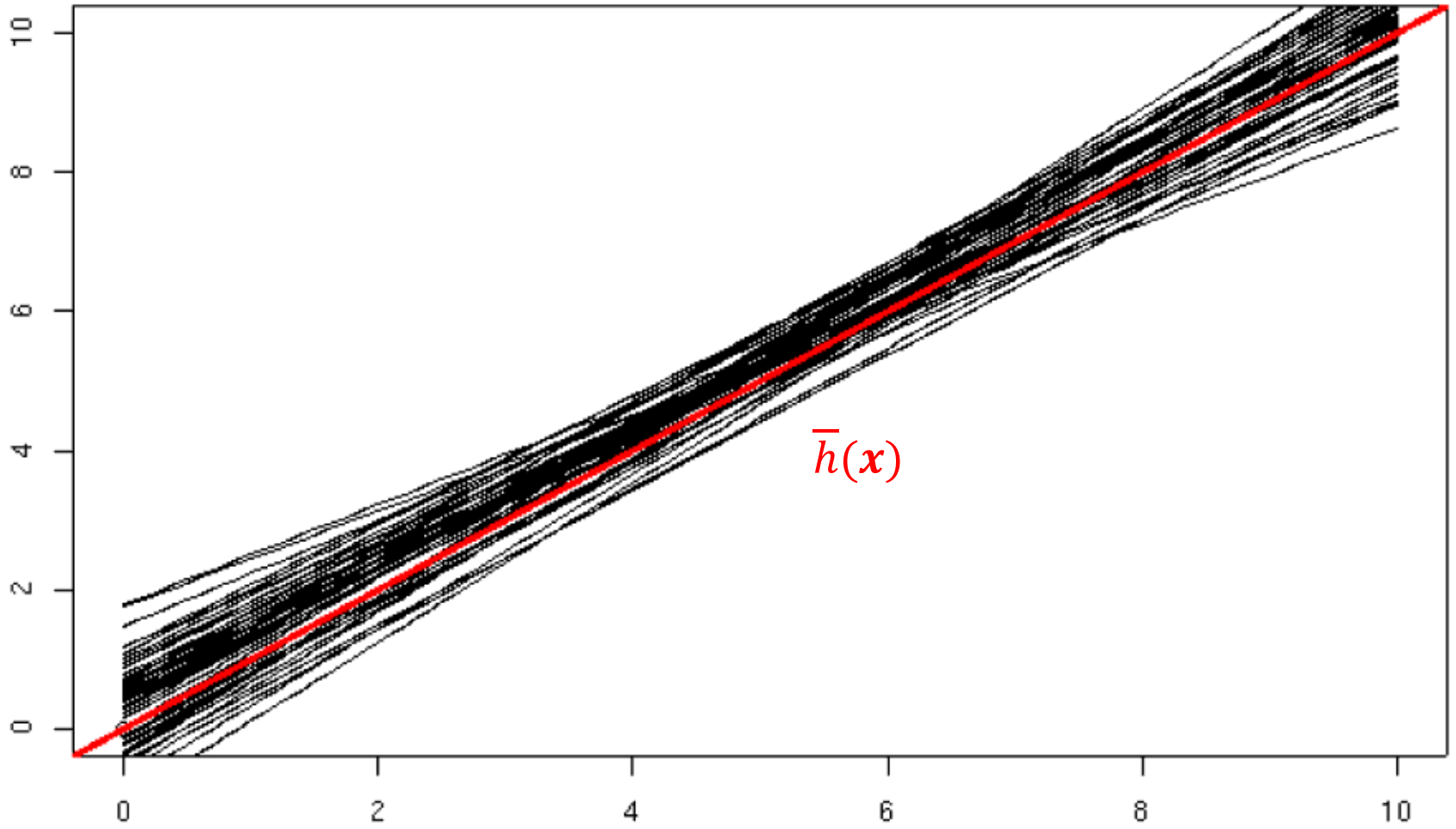
Dip. Informatica
University of Pisa



Note the high bias ! (the linear model is too rigid)

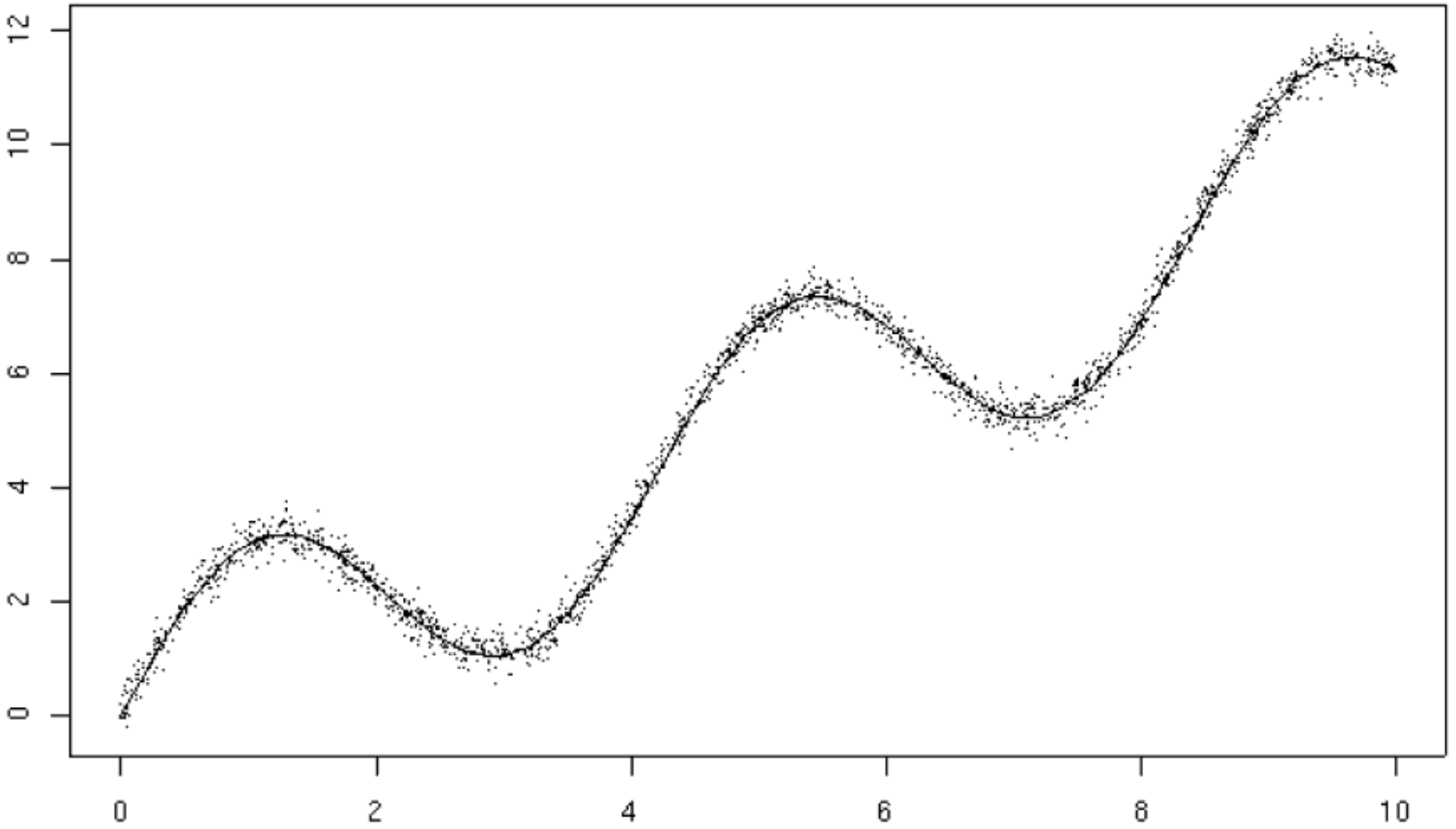
Variance

Dip. Informatica
University of Pisa



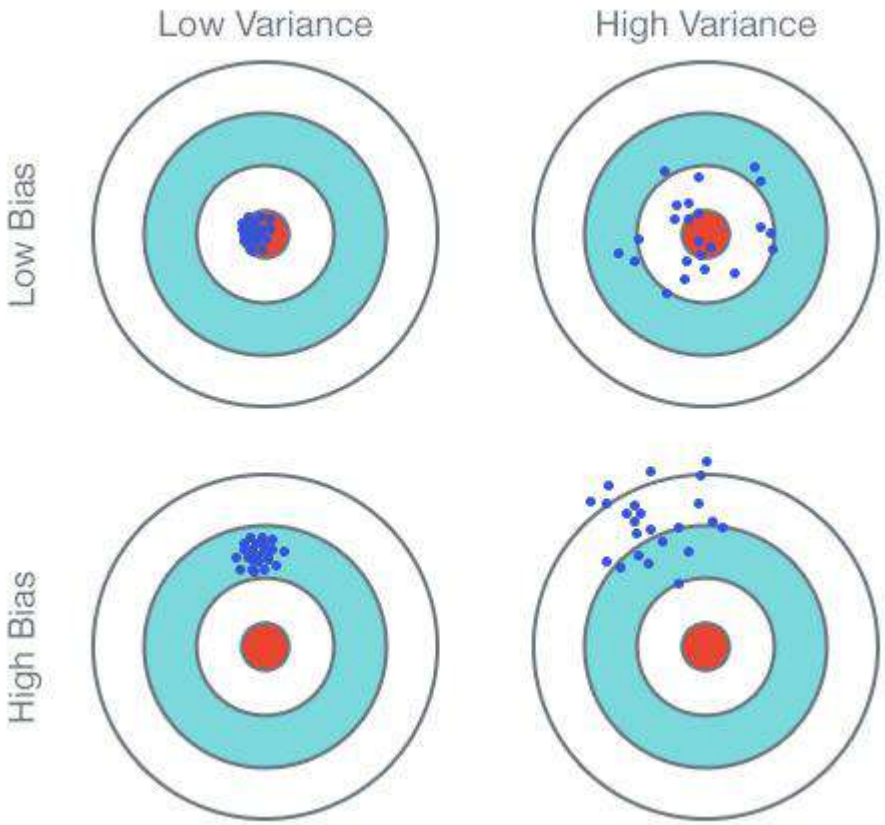
Noise

Dip. Informatica
University of Pisa



A nice graphical analogy for bias and variance

Dip. Informatica
University of Pisa





Darts game

Exercise: identify under and overfitting cases…
But the match of the analogy is not perfect for ML…

Bias/Var and Regularization

Dip. Informatica
University of Pisa

Recall Regularization:

$$\text{Loss}(\mathbf{w}) = \sum_p (d_p - o(\mathbf{x}_p))^2 + \lambda \|\mathbf{w}\|^2$$

Left (data) term (TR data) Penalty term

Varying λ we can obtain
- less regularized (complex) solutions (low λ)
- more regularized (less complex) solutions (high λ)

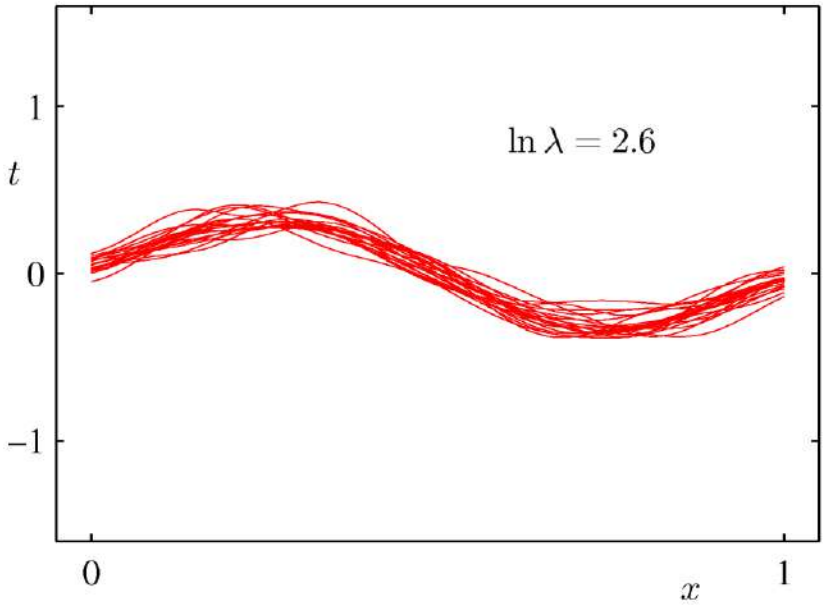
- The following example (from Bishop) (should be) using RBF network with LMS

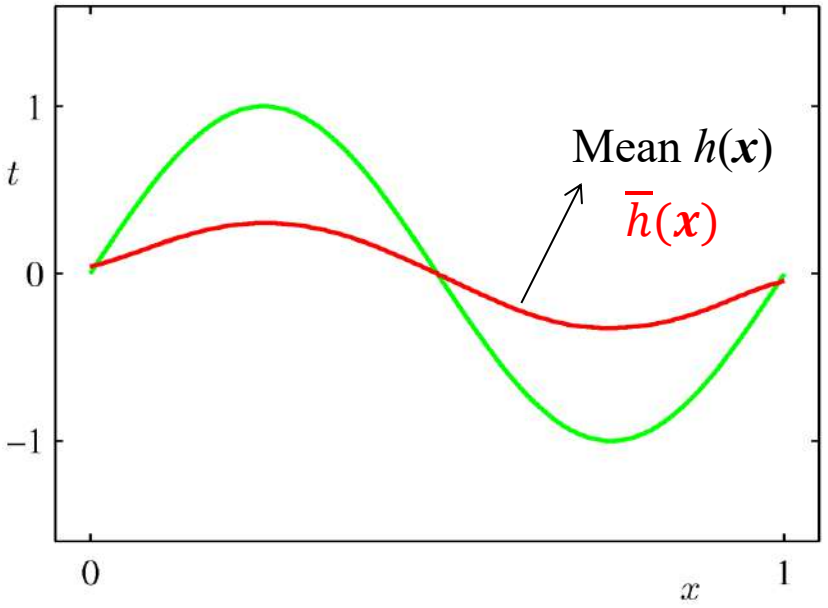
(By the way, note: that the intercept is typically left out of the penalty term)

The Bias-Variance Decomposition: high lambda

Dip. Informatica
University of Pisa

- Example: 25 data sets from the sinusoidal, varying the degree of regularization, lambda (λ)





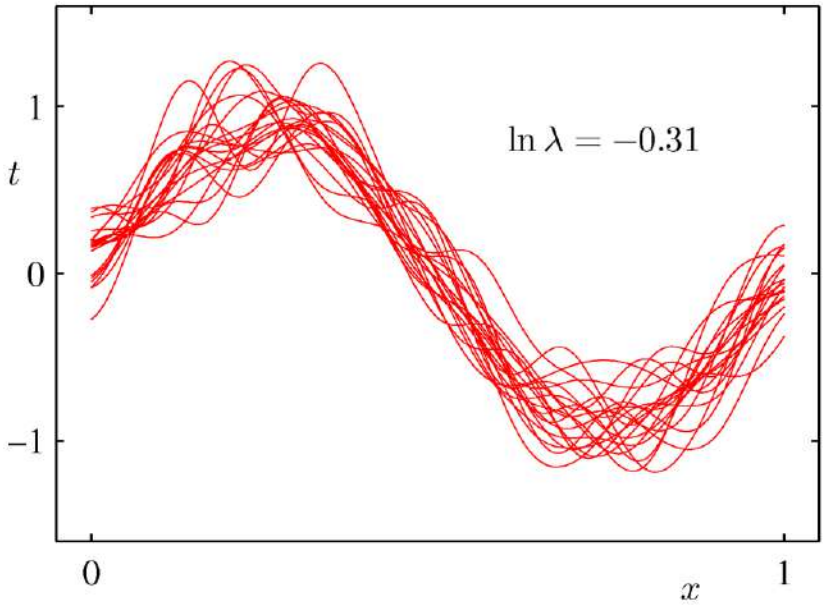
High bias, low variance

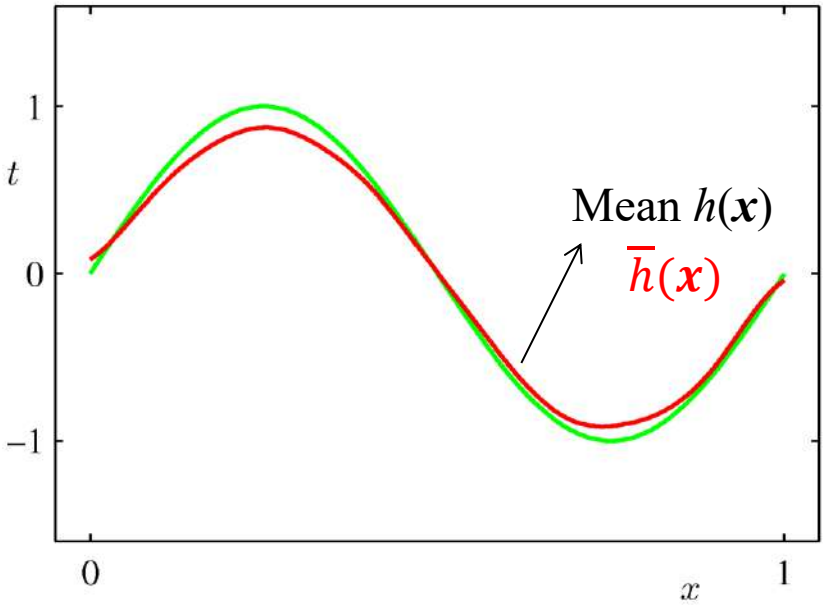
High λ increase the effect of "penalty term",
less dependent on left (data) term ("too rigid")

The Bias-Variance Decomposition: medium lambda

Dip. Informatica
University of Pisa

- Example: 25 data sets from the sinusoidal, varying the degree of regularization, lambda (λ).



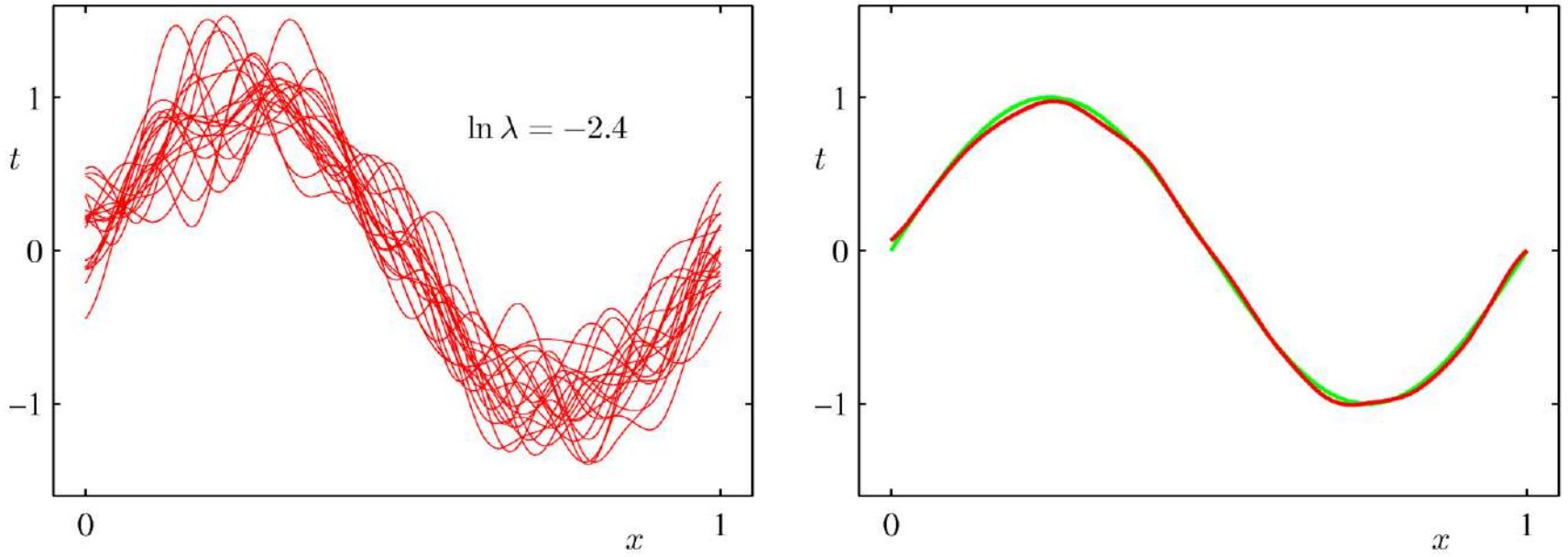


Lower bias, higher variance
Decreasing λ → we are more dependent
on the specific training data (left –data- term)

The Bias-Variance Decomposition: low lambda

Dip. Informatica
University of Pisa

- Example: 25 data sets from the sinusoidal, varying the degree of regularization, lambda (λ).



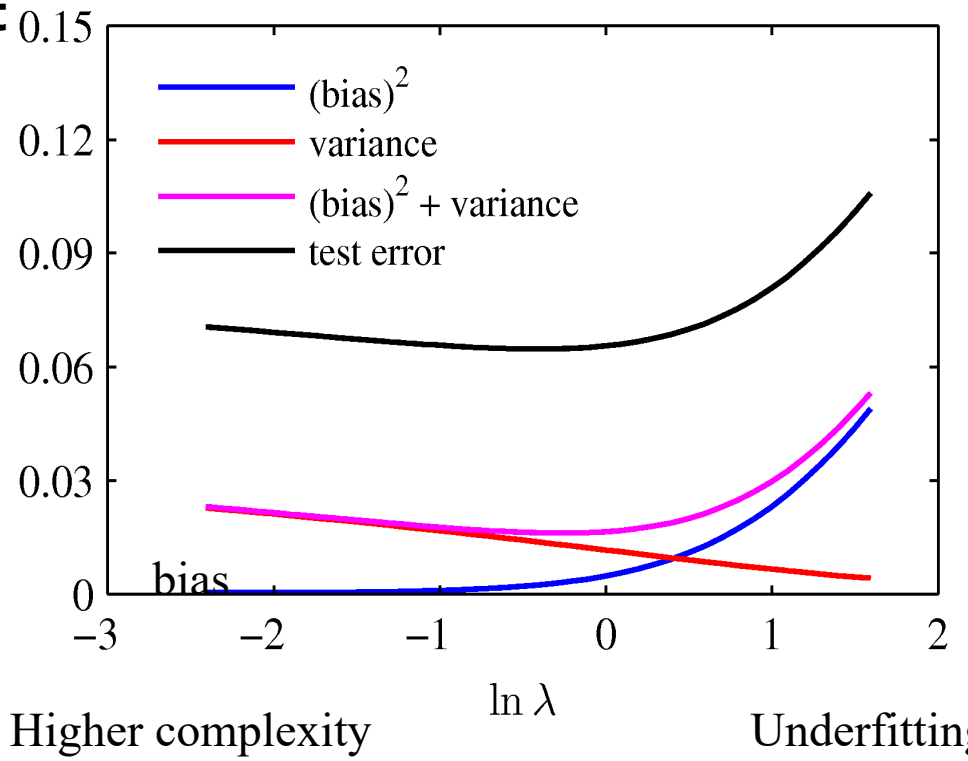
Very low λ: Low bias, high variance:
Test error (for a given TR) can be high (the variance component) but …
even if *h(x)* are quite variegated, they are correct "on average" (right plot)

The Bias-Variance Trade-off

Dip. Informatica
University of Pisa

- **From these plots, we note that an over-regularized model (large λ) will have a high bias, while an under-regularized model (small λ) will have a high variance.**

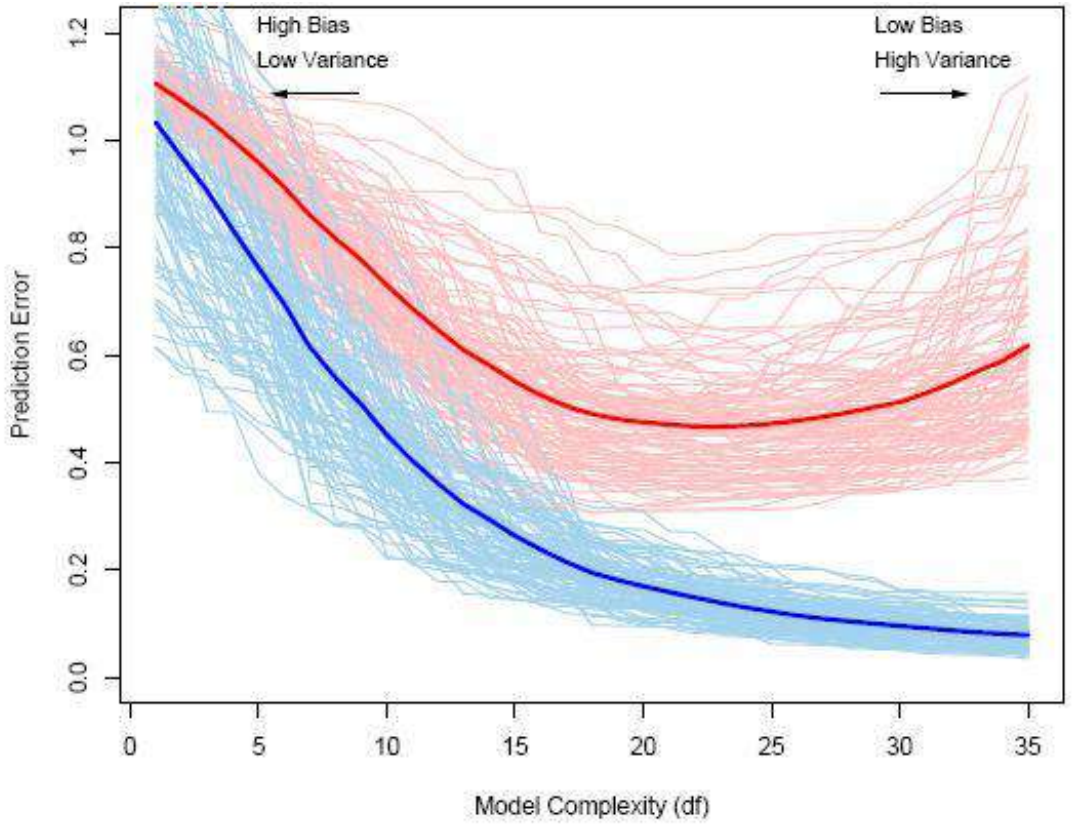
- **Again the trade-off!**



Note: complexity is higher on the left
(reverse of our standard plots)

Bias-Variance plot*
(with TR and TS)

Dip. Informatica
University of Pisa



unlucky

lucky

Complex model : lower bias, higher variance → possible overfitting

FIGURE 7.1. *Behavior of test sample and training sample error as the model complexity is varied. The light blue curves show the training error* $\overline{\text{err}}$*, while the light red curves show the conditional test error* $\text{Err}_{\mathcal{T}}$ *for 100 training sets of size 50 each, as the model complexity is increased. The solid curves show the expected test error* Err *and the expected training error* $\text{E}[\overline{\text{err}}]$.

Other ML topics

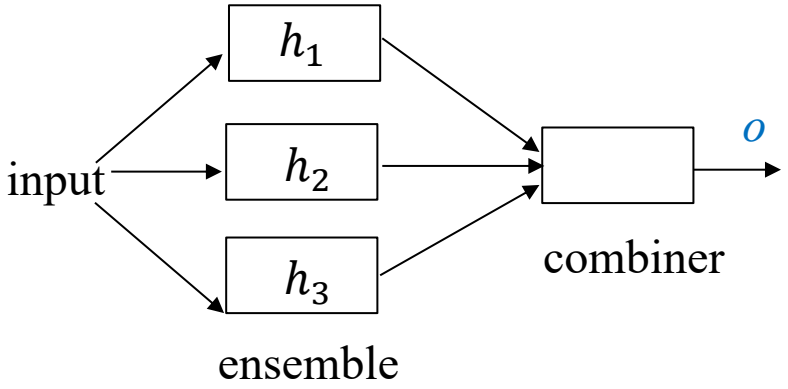
Dip. Informatica
University of Pisa

- Ensemble learning: bagging, boosting
- Feature selection
- PAC learning
  - E.g. Mitchell, Haykin
- Comparative analysis of inductive principia (SRM, MDL, Bayesian inference, Regularization) with respect to complexity control
  - Cherkassky, Mulier (chapter 2)

Ensemble Learning

Dip. Informatica
University of Pisa

- A first look to **ensemble learning**
- Take advantage of multiple models



Simplest case ("voting schema"):

- Regression
  - simple average committee $o(\boldsymbol{x}) = {}^{1}/_{K} \sum_{i=1}^{K} h_i(\boldsymbol{x})$ ⇨ $\overline{h}(\boldsymbol{x})$
  - $loss(E_i(h_i)) \leq E_i(loss(h_i))$ I.e.
  - ($mean_D$) square error of a committee ≤ mean of the ($mean_D$) square error
    e.g. $(t-o)^2$, t=5, 2 models → $((5-2)^2+(5-4)^2)/2 = 5 > (5-3)^2 = 4$
    or even: $((5-6)^2+(5-4)^2)/2 = 1$ while $(5-5)^2 = 0$
- Classification
  - Take a vote over many classifier (given they provide "diversified" responses): why ? (or classify after the mean of the continuous outputs)

Alternative ("stacking schema"): the combiner is again a ML model

Bagging (*bootstrap aggregating*)

Dip. Informatica
University of Pisa

- Combine many classifiers
- Bagging
  - Train *K* classifiers on different subsets of training set
  - Differentiate each training using bootstrap (resampling with replacement)
- Think to bias-variance (*see "low lambda" case slide: right figure*): $\overline{h}(\boldsymbol{x})$
- High variance models (with law bias) can perform well *on average*
- Or in other terms: the average of *h* reduces the variance
  - Regression: use the mean
  - Classification: voting of *K* classifiers
    - Likely to increase **margin** of a classifier
    - Indeed, average of *K* separation curves likely to be "in the middle"
      "In medio stat virtus"

Boosting (e.g. AdaBoost)

Dip. Informatica
University of Pisa

- If models have the same errors we have not advantage on ensembling

1. Differentiate each training concentrating on errors (more weight to difficult instances, e.g. more likely to be included in the TR of the next classifier) E.g.:
   - Train 1st classifier
   - Train 2nd classifier, weight more instances misclassified before
   - Etc.
2. Combine results by output weights (weighted vote for classifiers, with more weight to low error classifier)

- If not stopped, boosting will learn to classify correctly all instances from training set (using *weak learners* that can merely do better than random guessing, i.e. error less than ½ for binary classification) providing an incremental approach to construct complex models
- Can greatly improve performance of *weak learners*
- Even if bias is reduced the variance does not increase (resist to overfitting)
- As bagging, also likely to produce separation curves *in the middle* (i.e. max margin)
- But can suffer for noisy data (which are weighted more)! (it does not solve the underfitting/overfitting issue in itself)

Feature Selection (few words)

Dip. Informatica
University of Pisa

Find a *selection of the features* that are more informative for the problem at hand….

Great benefits for

- Dimensionality reduction (e.g. VC-dim effects for NN, curse of dim, …), filtering of irrelevant information and noise

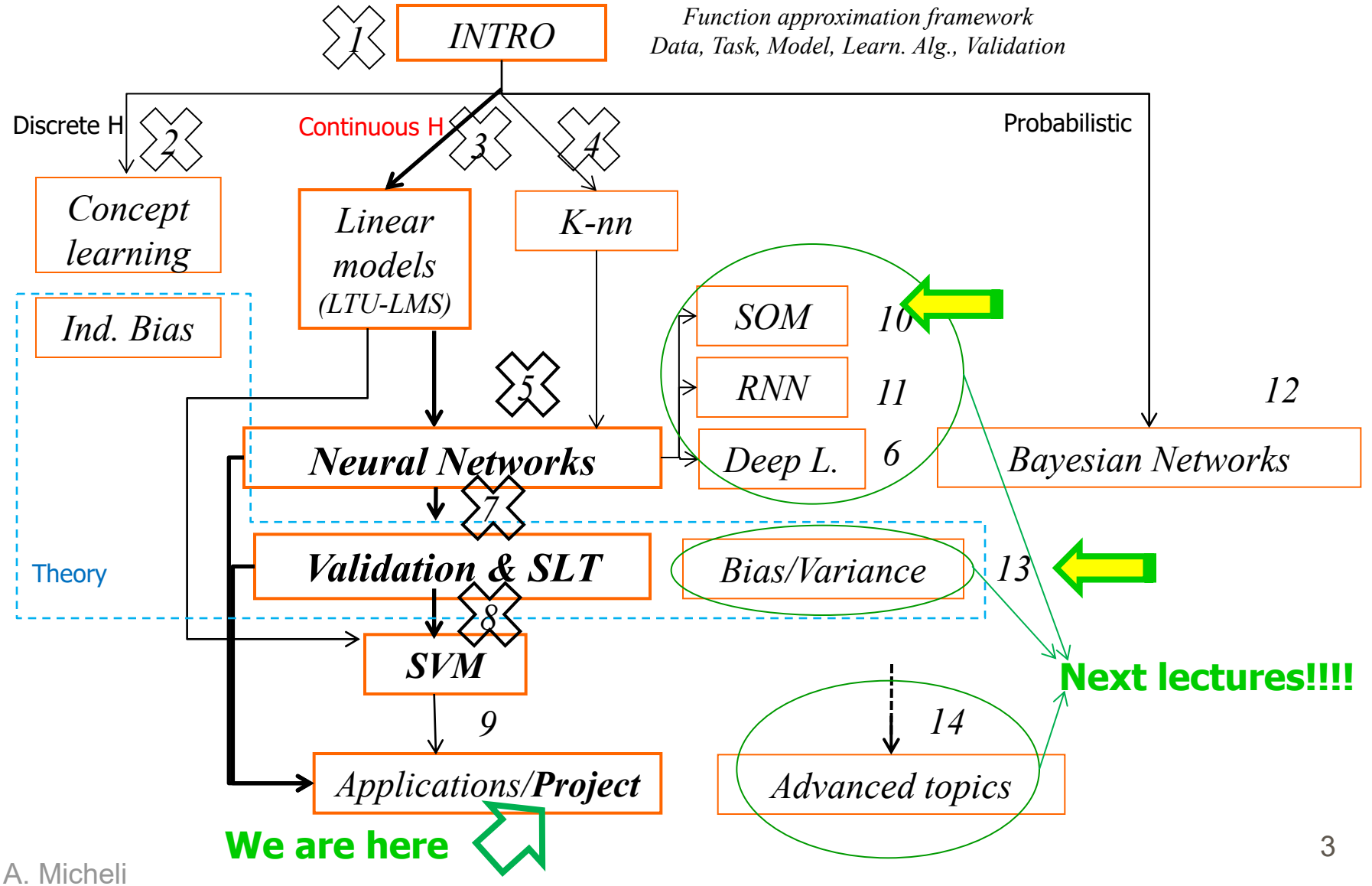
  (that can lead to great performance improvement)
- Interpretability (e.g. find a set of useful descriptors)

- It is computationally hard (think how many possible subsets of features can be composed for the learner, and consider the retraining of models)
- Typically, a heuristic search of the best subset (including or not the learner) by greedy or GA or other optimization techniques.
- Results are particularly sensible to the validation method: "*feature subset selection bias*", wrong use if selection is made on the entire data set (see slides on CV!)

A survey: I. Guyon, A.Elisseeff. "An Introduction to Variable and Feature Selection". JMLR vol.3:1157-1182, 2003

ML Course structure
Where we go

Dip. Informatica
University of Pisa



This Lecture

Dip. Informatica
University of Pisa

Sec. 1. The Exam (resume and details)

Sec. 2. The Project

Sec. 3. FAQ & other info (read by yourself)

Sec. 1. The Exam

Exam: schematic synthesis (REPETITA from introduction)

Dip. Informatica
University of Pisa

Summarizing, the exam will consist in:

- The project delivery by each group
- A general (pre)-Test for all the candidates at the date of the official exam sessions (as in esami.unipi.it, when the sessions will be published)
  - *ONLY FOR IN PRESENCE STUDENTS, not remote connection is allowed*
- Oral (group by group)

Exam: details
(REPETITA from introduction)

Dip. Informatica
University of Pisa

In the following order:

1. **Course lectures** (stay on-line with them!)
   - And perform the **Intermediate tests** during the course → I can assess your active participation to the course
2. **Project work** (you can start around in the middle of the semester)
3. **Project package delivery:** results, slides report etc. at the date specified in the Moodle for each session
4. **Oral:** start at the date of the official exam session, with questions in written form [in class **general (pre)-Test** at the date in esami.unipi.it] (it is in the same session of the prj delivery).
5. **Project presentation and discussion, and oral finalization,** according to a specific calendar for each group, i.e. as soon as possible in one of the following days/weeks (*ALL the students of the group must be present for the exam, i.e. making the exam in the same session*).

See more in the first Lecture and I also updated the FAQ section in the Moodle with the "pipeline for the project/exam"

Assessment methods
(REPETITA from introduction)





Dip. Informatica
University of Pisa

Exam:

- **Project** (Written exam)
  - Students have the opportunity to develop a project realizing/applying a learning system simulator (typically a simple neural network) and to validate it through benchmarks. A written report (slides) will show the results.
    - Great *opportunity* to apply the concepts by yourself
    - Great *opportunity* to show your concrete understanding and effort for the exam
  - Deadline: (around/or **more than**) 10/14 working days before the oral exam session (see the Moodle folder of your session for the exact deadline)
  - **See details in the lecture for project presentation → TODAY**
  - We include a *competition* with *blind-test*
    - which is part of the benchmark results in the prj
    - also some joint proposals with CM course
- **Oral exam** (dates according to the exams sessions)



Exam (II): Some details
(REPETITA from introduction)



Dip. Informatica
University of Pisa

- **Project**: we will discuss in a *specific lecture* all the details but from now it useful to know that:
  - It is made by a team of (2 or) 3 students (exceptions are very rare, subject to restrictions, MUST be justified by serious impairments and authorized in advance: see later)
    - A shared document *is open* to find partners (see Moodle)
  - It is made just one time, i.e. with 1 delivery
  - It is corrected/discussed jointly with the oral exam (at the exam session)
    - So all the team members need to do the exam in the session
  - It is an implementation of a NN/ML model or an application of ML SW (type A or B)

… (see Intro lectures and **later in this lecture**)

The main hint
(REPETITA from introduction)





Dip. Informatica
University of Pisa

A major hint *from past students*:

1. **FIRST study the course content**
2. **THEN apply for the project**

Good news

Dip. Informatica
University of Pisa



- This year we have **two** ML course assistants:
  - Domenico Tortorella <domenico.tortorella@phd.unipi.it>
  - Alessandro Dipalma <alessandro.dipalma@phd.unipi.it>
  - to help for the project development (but **not** for questions on the course!)
- Take the opportunity to interact with them during the course



- Available via email or de-visu or video-call each Wednesday 14.30-15.30, better by fixing an appointment by email in advance **with your group**

Delivery and deadlines for Exam and the Project

Deadlines and important notes: <formal exam> subscription and prj

Dip. Informatica
University of Pisa

1. Date for the **exams**: esami.unipi.it portal

- Therein you will find the "session date" = date for the oral (period beginning) (this holds for all the students) → General (pre) **Test slot**!
- Take care to register your name in the official UNIPI (esami.unipi.it) portal for exams (check the **deadline** in large advance): *It is a must*, only registered student can be admitted to the exam (all students rule)

2. The prj delivery has a different **deadline** (automatic in the Moodle): the deadline is *typically* =session date **–** (minus) ~10-14 working days (but see the exact deadline date and hour in the Moodle folder for your session, check it in large advance!):
   - especially at the first sessions a *greater time* is need to correct all the projects (or sometimes due to other, often formal, commitments)
- Of course, you CAN always deliver before the deadline!

- The **3 dates/deadlines** are different: two are for you (to register and to do the test at session date), one is for the prj delivery

Notes on timing (prj)

Dip. Informatica
University of Pisa

- You will have all the sessions of the academic year to deliver your project and make the exam

- For the first session (on January), I will take in account the shorter time you have to complete the project,
  - with a larger tolerance on the "quantity" (extension of your investigation)

Project General Rules: Material

Dip. Informatica
University of Pisa

- Deliver the report and material according to the **deadline** in the Moodle folder, with a **zip package** including (see also FAQ II):
  - **Code** (the code developed by you, usually well commented)
  - **The slides** (report) describing the implementation and the experimental aspects:
    - Details later for presentation time, slides, font, demo etc.
      (a file describing in detail the report, and a demo, is in the folder for the project)
    - Name of slides and zip files: See FAQ II (use team's surnames)
  - **Files** for the ML-CUP: "blind test set" result (1 file *team-name*_ML-CUP25-TS.csv) + a short abstract (see the next slides with details)

- Bring with yourself the code to the oral exam (printed or electronic version)

Project General Rules: How to deliver it

Dip. Informatica
University of Pisa

1. Use the Moodle (elearning platform) to deliver the Project (PRJ) a zip package: code, report, cup-files, short-abstract
   - See in Moodle the Section **Prj Student Material**
2. After the last upload (until the deadline):
   Send email at both micheli@di.unipi.it, domenico.tortorella@unipi.it, alessandro.dipalma@phd.unipi.it
   Subject: **[ML-2025]** Report by <your names>
   Include in the main text:
   - Your name(s) & email contact information in the main text
   - Your degree course (master programme)
   - The name of file used in Moodle (to find it)

It is a must **(!)** to use this **tag**

- Don't forget a CC to your colleague (*all the group members* must be included in *all* the communications).
- Use the Moodle to deliver all the other files, do ***not*** attach them by email!!!
- Projects no announced by an e-mail will not be considered !!!
- (be sure I received it: resend the email if I do not reply with an ack. after 5/6 days)

Notes on Oral: Modality

Dip. Informatica
University of Pisa

- The **orals** will be made group by group ***in presence*** (the room will be specified before each oral slot, typically in **my office** at the Dept. of Computer Science).
  - For special cases, due to *serious* and *motivated* logistic reasons, please advice me in the project delivery e-mail.

- Style: I'll ask answer first by **math language** (e.g. in the form of *equations* due to clarity, synthesis, …), then we can discuss on them

Notes on Oral: Timing

Dip. Informatica
University of Pisa

- Oral: as said before, I'll communicate the **date and time slot of the oral** interview and project discussion (for each group) as soon as possible after the project delivery

- First sessions: due to the high number of candidates the slots of the final orals of (the first sessions) especially the second session will be distributed over an extended **period of time** (from mid-February to over March, even if *most* of them will be earlier).
- If you have special cases for the dates, please **anticipate your problems** with the project delivery e-mail: i.e.
  - If necessary, you can express (motivated) need for an early oral priority or a preference for a late oral session (or no preference, i.e. also no reply to my email for prj ack., if your group has no special needs nor specific preferences).
  - *I'll try to help you* if the overall scheduling of requests allow me to do it.

Notes on Oral: FAQ

Dip. Informatica
University of Pisa

- **Do I need to have a prj approval (passed/not passed) before to come to the general rest?**
  - No, after project delivery, you can do the general Test.
    The prj and general test results will be discussed at the oral.

- **What happens if a student of the team at the general test gets a low score?**
  - I assume that all students are well-prepared after completing the project (see the slide "*The main hint*"). So, very low scores should be rare.
  - In any case, ALL team members will come to the oral and discuss the prj
    - Then *we will discuss case by case* (each student will be evaluated on his or her own, including the possibility of redoing an oral, which, however, fortunately is necessary only in very rare cases).

Sec. 2. The Project

The project at a glance

Dip. Informatica
University of Pisa

- In the following: A top-down presentation (first general info/rules than details)

In short (the **phases**):

I. Form a **group**

II. Prepare a **simulator**, made by you (prj type A) or from software libraries (prj type B)

III. Apply (**experimental part**) to all the benchmarks (including the CUP)

IV. Write your **slides** report (with all the info indicated in the demo & info files) and the **results for the cup** (following the provided file format)

V. Deliver the **package** till the deadline (see slides in Sec. 1)

VI. You will **present or discuss the slides** at the beginning of your oral

- *It is easy*, all the rest are details (although important)

The project

- **Premise** ("experimental", main aim, etc.)
- **Form a group (phase I)**
- **Type A, B, etc. (phase II)**
  - The simulators
- **The application (phase III)**
- **The slides** (format, style, etc.) **(phase IV)**
- **And slides presentation (phase VI)**

First premise
On the empirical approach (I)

Dip. Informatica
University of Pisa

- Experimental method (using ***empirical*** evidence, from experiments) is the basis of the "*scientific method*" per-se
- In our case, the *experiments* are the numerical simulations to test your hypothesis on the models and its quantitative parameters
  - You don't know the result in advance, you have to formulate your hypothesis and then to measure the empirical results/observations, to compare and to reason on the results etc.
  - Don't worry if this is a new experience for you (for many Computer scientists)
  - In any case, this experience will be useful to you as a ground for *your thesis*



First premise
On the empirical approach (II)

Dip. Informatica
University of Pisa

- You don't know the result in advance, …
- So, *"I feel lost", what have I to expect?*
  If you don't have any experience with "*experiments*", this is new for you, but it is *not* negative but REALLY a nice and useful experience you take the opportunity to gain !!!

- Exciting or lost? Break the ice !!!
- Nothing explodes: explore different cases, it is not dangerous! It is a numerical experiments 😉





Students: "*The challenging of implement different solvers and hyperparameters selection figured out to be interesting and one thing that makes us learn a lot.*
Initially the experience was a little disorienting for the huge amount of point to test, but going ahead the funny part was the same that at the start scare us."

Second premise: The real aim of the project

Dip. Informatica
University of Pisa

The real aim of the project is not to achieve a state –of–the art result (least of all frustrating you with your first ML experience)

- The NN or ML models implementation or usage is a mean, your understanding is my/ the objective
- Learning with NN or ML models is a mean, your learning on how such models work is my/the aim



YOU are the objective!

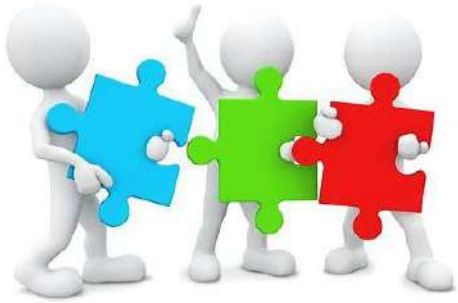
The project

- **Premise** ("experimental", main aim, etc.)
- **Form a group (phase I)**
- **Type A, B, etc. (phase II)**
  - The simulators
- **The application (phase III)**
- **The slides** (format, style, etc.) **(phase IV)**
- **And slides presentation (phase VI)**

I. Form a group (I)

Dip. Informatica
University of Pisa

- **Groups**: *it is assumed groups of (2 or) 3 students*
  - For the prj with CM see later

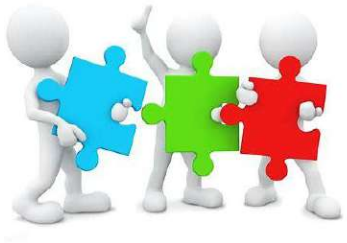


- Advantages of a team work:
  - The result is greater than the sum
  - Increase autonomy and never discourage
  - Mutual enrichment of experience
  - Best results (in the general sense) in the past editions
  - Of course, the evaluation (final mark) can be different for the students (the orals are different for each student)

I. Form a group (II)

Dip. Informatica
University of Pisa

- **Groups**: *it is assumed groups of (2 or) 3 students*



- [Repetita] Please, remember that the project is corrected/discussed *jointly* with all the students within the final oral exam and hence the oral is given at the same session (in the same day) for all the group candidates.

- Special (and are) case:
  To participate to the *extraordinary sessions (spring & autumn)* ALL team members must be eligible
  - Note: The extraordinary sessions are ONLY for the protected categories of students, see **https://didattica.di.unipi.it/en/extraordinary-exam-sessions/**

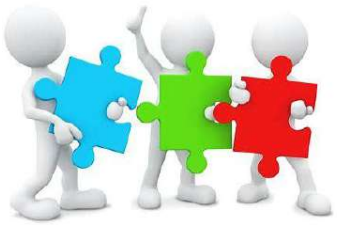
I. Form a group (III)

Dip. Informatica
University of Pisa

FAQ: **Can you help me to find partners to form a team?**

- To find partners (a team or the third partner) see the shared document with a link in the Moodle [**Prj partners searching page**].

- You can fill it with your preferences and contacts so that other students can contact you (and you the others) with a good matching.
- Connect with the **UNIPI account** to edit it.
- You can use it <u>also after</u> the first sessions!



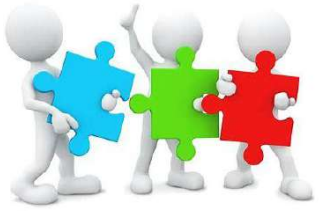
- 3 students are preferred: I invite also students in a team of two members to see the **<u>Prj partners searching page</u>** to form a team with 3 students

I. Form a group (IV)

Dip. Informatica
University of Pisa

Can we do a group of **1 student (by myself)**? **NO**

- **No prj can be done alone**, the project is a team work
- You lose the *opportunity* to do a prj, in case you will have instead an *additional exam part, we will decide on singular case basis* *
- Moreover, the qualification in this "*individual student*" category needs to satisfy 2 conditions:
  1. a preventive trial to search partners through the "Partners searching page"
  2. a preventive (I mean largely before the prj delivery deadline) agreement with the teacher (by email), motivated by a very special (serious and rare) impairments



The project

- **Premise** ("experimental", main aim, etc.)
- **Form a group (phase I)**
- **Type A, B, etc. (phase II)**
  - The simulators
- **The application (phase III)**
- **The slides** (format, style, etc.) **(phase IV)**
- **And slides presentation (phase VI)**

II. Possible Aims: Type of projects

Dip. Informatica
University of Pisa

A) Realize a ML/NN model simulator and apply it (implementation)

- Especially useful for students that will go to develop new models *
- Programming language is a free choice (C++, Python are popular for ML, or even environments as Matlab or R can be considered)
- A) with CM : a coordinate prj with CM

B) Extensive experimental applications of existing ML/NN simulators (comparison)

- Especially useful for students that will go to apply ML in other fields
- Simulator is a free choice (a list will be discussed in the next slides)

C) Contact me. Only in case there are serious reasons/impediment not to apply to A and B. Or special cases, e.g. for PhD students

Both **A)** or **B)**: participate to the "**ML-2025 cup**" competition.

There is **no** difference in the grade and in the global effort (≠aim)

Let us see details for A) and B) and then the cup details.

Type A) Model implementation

Dip. Informatica
University of Pisa

Realize 1 ML model simulator : typically a **MLP Neural Networks (with regularization techniques)**

*This is the **typical project case**, allowing you to realize a simple models by your original code, and to experiments all the variants that you like,* and have fun;-)

Examples:

- [**typical case**] **Implement a** (multilayer) **MLP with backpropagation, momentum, and L2 regularization:**
  try the regularization and other hyper-parameters effects
- Backprop with variants for the weights upgrade : Quick-prop, Rprop, … or other gradient based techniques. Or Cas-Cor, etc.

Other models:

- MLP for classification: LMS versus Cross-entropy*
- Bayesian models: contact me.
- **SVM**: see A) with CM

General note: you can exploit numerical libraries, e.g. NumPy, Armadillo (C++), ..

A) Model implementation (II)

Dip. Informatica
University of Pisa

- With a group of 3 students take the opportunity to try something *more than* the basic setting (I expected to see these), see the NN-part2 lectures with many variants *beyond* the topic with the **(!) symbol**, e.g. see cases with the  symbol), examples:
  - Comparison among different strategies for learning rate
  - Consider also the Nesterov momentum and compare
  - Comparison among different regularization approaches
  - Different architectures (the minimum is 1 and 2 hidden vs multilayers)
  - Different activation functions
  - Any variants to the training algorithm (beyond SGD, e.g. R-prop, Quick-prop, Cascade Correlation but see also CM projects)
  - Type A for NN + 1 other model from libraries (as for type B) to compare
  - Type <A with CM> (also implementing 2 models, e.g. NN and SVM)
  - …
- Not necessarly all them, e.g. just two, and then … *it is up to you*!
- Of course, a group of 2 can anyway consider some of them as well!

Special case <A) with CM>

Dip. Informatica
University of Pisa

A) with CM: A coordinate prj between ML and CM

- See a detailed list of proposals within the CM course
- You will provide **1 report for ML** with the basic A) results + the new results with the CM technique (comparison)
- AND **1 report for CM** (according to CM teachers' rules, and a different aim)

Categories (examples) [through CM approaches] :

- (improved) NN training by other gradient descent methods algorithms
  - see many examples in the lectures NN2 Heuristics (Conjugate gradients, …)

  [comparing w.r.t. the basic gradient descent with momentum and L2 regularization]
- Non-differentiable optimization for Piece-Wise linear functions
  - PWL or ReLU activation function or *L1 regularization*

  [comparing w.r.t. the basic gradient descent with momentum and L2 regularization]
- SVM/SVR implementation (through different approaches), applied with kernels...
  - [for this case see the FAQ for the NN comparison with NNs]

- **If** it is not in the CM list, or it includes a ML part not considered as basic prj A (see the FAQ), the proposal must be agreed by ML **AND** CM teachers!!!
- You can still compete for the ML-CUP with your results.

Notes on <A) with CM>

Dip. Informatica
University of Pisa

Further details on<A) with CM>:

- **More effort?** It is because *you like to try and show something more* (a volunteer challenging choice) and not something less! This fully exercises your full understanding of the singular parts (by adapting them to the combined construction)
- For may cases it is useful to refine the **literature** basis (provide by us) to see previous studies of the impact of the used methods for the ML area.
- So, we consider this **a plus** (if it is well done)

Notes:

- L2 regularization is the Tikhonov with norm-2 penalty
- L1 uses norm-1 penalty (we will discuss it later) (see lecture on linear models and NN-part2)
- Also ask for PWL if you are interested → ReLU will be (re)discussed also later

- Note that the aim is **not** to improve the performance/results obtained with the basic approach, but to critically exercise the use of CM approaches for ML (the aim is a critical analysis the results)*
- Unforeseen issues? You can go back: discussed by a case-based approach*

Notes on <A) with CM>: groups

Dip. Informatica
University of Pisa

You can do <A) with CM> either:

- In group of **3** (you have to agree this with the CM teachers with the modality specified by them)
- In a group of **3**, but only 2 are making the prj with CM, the third one is making only the ML prj (for my side)
- A group of 2

- Note: these are **two different** projects →
  - you are NOT forced to sustain the exam of ML and CM at the same session
  - provided that the results will be already present for ML in the report sent to me, you can do CM in any other time.

Type B) Compare models

Dip. Informatica
University of Pisa

- **Extensive experimental applications of existing ML/NN simulators** or *code made by LLM* (

):
- Compare different models/ existing tools, e.g.:
  - SVM and/or NN by a standard library versus basic models (linear, k-nn, naïve bayes, …) implemented by yourself or by standard software tools
  - Compare NN vs SVM vs other models (even not included in the ML program) within the same software tool
  - Compare 3 or more models even from different sw tools
  - Compare different software tools for the same model (e.g. SVM or NN)
  - Compare different software tools.
- (also for fairness wrt A) the B case implies a **larger effort on the comparison** among models (including accurate validation) and to the experimental part
- The report can include also evaluation on the sensitivity to the hyperparameters values (for different models), efficacy, efficiency, predictive performance (of course!) but also issues of tool usage, usability, richness of the set of hyperparameters etc. (for different tools)
- Repetita: you can still apply to the *CUP competition* just selecting the best model to apply.
- The code written by yourself (to use the libraries) must be included in the package

Further notes

Dip. Informatica
University of Pisa

- **LLM:** We well know that is possible to wrote code by LLM. In this case it is not a problem for us, it is PRJ B
  Of course, if you declare that is type A with your code and type A, but it is generated by LLM (or copied), etc → The prj fail, not passed → next session of exam

See the FAQ section (Sec. 3, end of these slides) for further aspects:

- On the boundary between A and B according to the usage of different libraries
- On the applications of different models for PRJ B to the provided benchmarks

FAQ: When is a project A or B?

Dip. Informatica
University of Pisa

- If you program the NN model and back-propagation (or any training algorithm), and the CV approach, from scratch and by your-self (independently from the software environment) is *type A*. E.g.:
  - A NN made by you within the standard Matlab is *type A* (but see in the FAQ on Matlab)
  - Instead, a NN using Matlab NN/DL *toolbox* is *type B*
  - The same for TensorFlow, PyTorch, Keras and other frameworks (see later): it is *type B*.

- Using high level libraries (SciKit learn, Keras etc.) is *type B*
  - However, if for example you just exploit part of the PyTorch features (e.g. the "automatic differentiation") but you implement the main training loop, the CV etc. you can indicate it in the report.
- Using LLM is type B

- Using numerical or graphical libraries in your code maintains it of *type A* prj.

Important: For both A) and B) cases

Dip. Informatica
University of Pisa

- For **NN heuristics** see the lectures on "Neural Networks: part 2"
  Try the effects of different configurations/ hyperparameters values according to your experimental schema

  (and explain the schema in the report)
  - In any case, for NN, include the momentum and a regularization approach (weight decay, early stopping, ….). See all the **(!)** indications
  - For type A: consider examples in the previous slide "A) Model implementation (II)"
  - For type B (*ça va sans dire*): if you use a library, please do not limit yourself to default values!

- You are free to choose the **model selection/assessment-evaluation** strategy:

  whose fitness for the problem at hand *is evaluated*:
  - Directly from the description in the report
  - Through the results on the blind test set

The project

- **Premise**
- **Type A, B, etc. (phase II)**
  - **The simulators (ML tools)**
- **The application (phase III)**
- **The slides (phase IV)**
- **And slides presentation (phase VI)**

Simulators/Software Tools

Dip. Informatica
University of Pisa

- Software / ML Tools to be used for the **B case**
- Or you can use a tool as an "***oracle***" to compare with your simulator for the **A case** (helping in assessing its correctness)

- If you use a library you must specify in the report the complete link to the source !!! (for both A and B cases, use a READ-ME file in the code package)

- In the following some examples: an exhaustive list is out of our scope and it is even impossible to keep it updated!!!
  - Developers make changes everyday!!! (big companies have a major role)

- The best one is the one that is more useful **FOR YOU**.
  - Check also the documentation and if it still have maintenance/developments (new releases, support, …)
  - Some of them have a reference in [JMLR]= http://jmlr.csail.mit.edu/mloss/

Simulators/Software Main (ML & NN & Deep NN)

Dip. Informatica
University of Pisa

- **Keras was** a high-level Neural Networks/Deep Learning library (Python) capable of running on top of either TensorFlow , Microsoft Cognitive Toolokit (formerly CNTK), Theano and others, (+Wrappers versus Scikit-Learn API).
  Nowadays it acts as an interface for TensorFlow library (see below) !
- **TensorFlow** *(Google)*, open source, C++/Python like (API, interface). Include deep neural networks. Released as open since November 2015. Now: *TensorFlow 2.**.
  - Used by Google for all their products, Photos, Gmail, speech recognition, …
- **PyTorch** is an open-source machine learning library for Python, based on Torch (was in C++). Neural Networks, Deep Learning. It exploits "automatic differentiation"*. Developed by *Facebook AI* research group (and Uber)!
  - **Torch** (NN, SVM, AdaBoost, K-nn, Bayesian, …renewed for deep learning/GPU, coll. with Facebook, Google, Twitter …development stopped in 2018)
- **Theano** (and Pylearn2): also for deep learning, GPU etc. wrappers with scikit-learn, … (stop dev. on 9/17)
- **Caffe** is a deep learning framework, originally developed at UC Berkeley. It is open source, C++, with a Python and MATLAB interfaces
- Others: https://en.wikipedia.org/wiki/Comparison_of_deep_learning_software

Simulators/Software Main (ML & NN & Deep)

Dip. Informatica
University of Pisa

- **Scikit-learn** [JMLR 2011]: Python open source, many tools for preprocessing, model selection, linear models, regularization, SVM, DT, … … NN was very limited, but from version 1.5 has been improved, still improving (but it is not comparable with the flexibility and completeness of Pytorch and Keras)
  - very *good documentation* with graphical examples: HAVE A LOOK!

Environments:

- **R** (Statistical Computing language): programming language and free software environment for statistical computing
- **Matlab**: multi-paradigm numerical computing environment and proprietary programming language (available at Unipi),
  - or as a simulator for NN ("Deep Learning Toolbox")
- and **GNU Octave** (free, Matlab compatible)
- **Wolfram Mathematica** (mathematical computation integrated tolls)

Just to witness the interest of the big companies



Dip. Informatica
University of Pisa

Just to witness what happens (not for your prj *):

- Google: google prediction (service) / **TensorFlow** *
- IBM: Watson
- Microsoft : Azure (platform and services)/ The Microsoft Cognitive Toolkit (formerly CNTK), is a deep learning framework developed by Microsoft Research.
- Amazon Machine Learning (service) / (old) DSSTNE: Deep Scalable Sparse Tensor Network Engine (open, Deep ML)
- Facebook (AI research lab): ~~FBLearner Flow,~~ → **PyTorch**…
  - (used by FB for computer vision and NLP, and also by Tesla, Uber, …)
- ……

Simulators for SVM

Dip. Informatica
University of Pisa

- **LIBSVM** (C++)
- SVM light (C)
- Torch (C++)
- JKernelMachines (Java library for learning with kernels [JMLR 2013])
- **Scikit-learn**

- Weka (Java)
- mySVM
- SVM in R
- ….
- http://www.support-vector-machines.org/SVM_soft.html

Popular Supporting Libraries

Dip. Informatica
University of Pisa

- **Pandas**: software library written for the Python programming language for data manipulation and analysis



- **NumPy** (*popular for NN!*) (**SciPy**), **Armadillo** (C++): numerical/linear algebra libraries: suggested also for prj of type A!!!



- https://en.wikipedia.org/wiki/List_of_numerical_libraries
- https://en.wikipedia.org/wiki/Comparison_of_linear_algebra_libraries

General note: you can exploit such numerical libraries for your code!

The project

- **Premise** ("experimental", main aim, etc.)
- **Form a group (phase I)**
- **Type A, B, etc. (phase II)**
  - The simulators
- **The applications (phase III)**
  1. MONK
  2. CUP
- **The slides** (format, style, etc.) **(phase IV)**
- **And slides presentation (Phase VI)**

III. Applications for the PRJ

Dip. Informatica
University of Pisa

Your results will include

1) MONK benchmarks results

2) The CUP data set results (competition)

In the report: results for **1)** and **2)**
for **both** prj A and B type

III. 1) MONK benchmark [repetita]

Dip. Informatica
University of Pisa

- Difficulty of assessing implementation correctness:
- A first test *("collaudo")* **Monk's data set**
  - **The results *must* be reported in the prj report: performance and learning curve plots for the 3 monk tasks**

    https://archive.ics.uci.edu/dataset/70/monk+s+problems

    NEW
  - 3 tasks of binary classification, small artificial data set, "not difficult" (a small NN with few units achieve a very high accuracy, up to 100%, with small time of convergence)
  - There is a report with previous results using MLP (and others ML models)
    - Chapter 9, "The MONK's Problems», S.B Thrun et al., CMU-CS-91-197, 123 pages

Solve it!

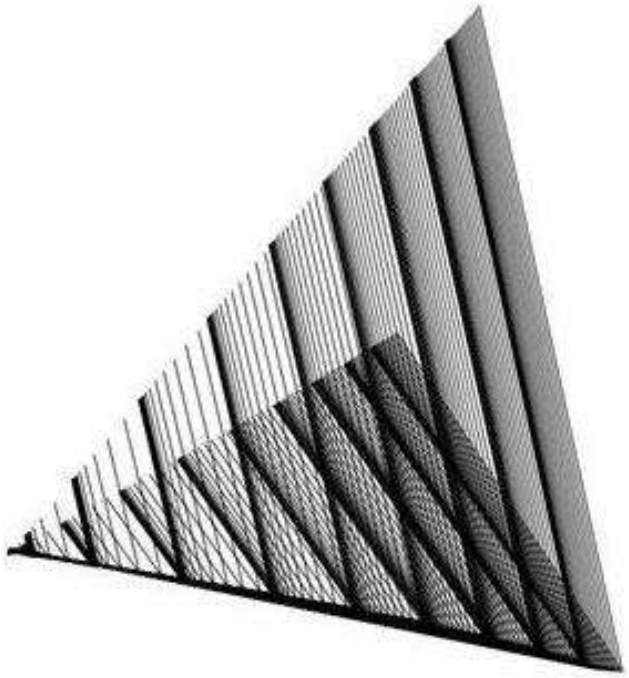
  - Input encoding: 6 variables with 2, 3 or 4 symbolic values each: 1-of-k → correspond to introduce 17 input units (see also sec 1.1 and check it!)
  - TS includes TR, which is a bad practice, but does not change here (since 100% accuracy is 100% accuracy also on the test set !!)

- Other sources of data sets for software tests: http://archive.ics.uci.edu/ml/ UCI Machine Learning Repository (hundres of data sets !)

III. 1) MONK benchmark
The Tech. Report

Dip. Informatica
University of Pisa

- S.B Thrun et al., «The MONK"s Problems-A Performance Comparison of Different Learning Algorithms», CMU-CS-91-197, 123 pages



The MONK's Problems

A Performance Comparison of Different Learning Algorithms

S.B. Thrun, J. Bala, E. Bloedorn, I. Bratko, B. Cestnik, J. Cheng, K. De Jong, S. Džeroski, S.E. Fahlman, D. Fisher, R. Hamann, K. Kaufman, S. Keller, I. Kononenko, J. Kreuziger, R.S. Michalski, T. Mitchell, P. Pachowicz, Y. Reich H. Vafaie, W. Van de Welde, W. Wenzel, J. Wnek, and J. Zhang

Carnegie Mellon University CMU-CS-91-197, December 1991

Monk data set results

Dip. Informatica
University of Pisa

- Please see examples of plots (learning curves for MSE and Accuracy) in previous lecture on NN:
  the "Neural Networks: part 2" lecture (**file**: ML-**-NN-part2-…pdf)
  - See "Other suggestions for your first trials" in the same lecture
  - and examples of hyperparameters were specified in the slides

  - Good results on the MONK benchmark does not guarantee the simulator correctness
  - Bad results on the MONK benchmark for sure require your revision of the code/setting

- Also: the results on Monk should be achieved using a **small network** (2-4 units): The test works if you achieve the state of the art results with few units (you have to not to improve the accuracy).
- For the CUP task, on the other hand, there are no constraints, because instead you have to do the best performance ...

III. 2) ML CUP !

Dip. Informatica
University of Pisa



ML CUP 2025



Dip. Informatica
University of Pisa

- I provide to you a **data set for the CUP**, with a training set and a blind test set (examples without target values)
- Apply the models that you are investigating selecting the final one that you think it is more accurate through the training set (used for training and validation, and your *internal test **) (*see the FAQ).
- Report in the "report (slides)" the TR/VALIDATION and your internal TEST errors, in the original scale i.e. MEE for the 2025 cup (see next slides).
- Apply your final model to the blind test producing an output for each example of the blind test set and record them into an output file
- Provide with the report and the output file containing the blind test results
- Test result: the accuracy on the blind test set will be automatically computed
- The final results will be summarized on the web site using your nicknames
- Glory to the winners! ;-)



Tasks and Data



Dip. Informatica
University of Pisa

- **Regression** on 4 target variables, i.e. t_1, t_2, t_3, t_4, in a 4D space
  - **4** target variables: 1 NN with 4 *linear* output units, or 3 NNs, 3 SVR, ….
- **500 training examples**
  - Column 1: pattern name (id)
  - 12 central columns: **12** variables with continuous values (from a noise source)
  - Last 4 columns: target of **4** continuous value variables : t_1, t_2, t_3, t_4
- **Blind test set**: 1000 patterns, with the same input format (of course without the 4 target columns)

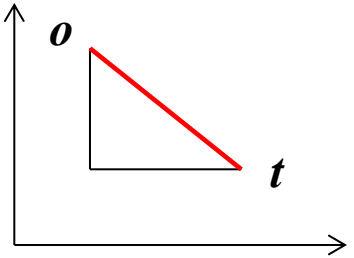
Task & Errors 2025

Dip. Informatica
University of Pisa

Report in the **original scale** the following error measure (**Euclidian distance**):

- **Mean Euclidian Error** (it is the error used for the competition performance evaluation), *where l=number of data, p=pattern, o=output, t=target*

$$E_{MEE} = \frac{1}{l}\sum_{p=1}^{l}\left\|\boldsymbol{o}_p - \boldsymbol{t}_p\right\|_2 = \frac{1}{l}\sum_{p=1}^{l}\sqrt{\sum_{m=1}^{4}\left(o_{p,m} - t_{p,m}\right)^2}$$



E.g. Distance between
2 points in 2 dim space

- Typically you will also observe the (root) mean squared error (that is the typical loss used for the LMS training approach)

$$E_{MSE} = \frac{1}{l}\sum_{p=1}^{l}\left\|\boldsymbol{o}_p - \boldsymbol{t}_p\right\|_2^2 = \frac{1}{l}\sum_{p=1}^{l}\sum_{m=1}^{4}\left(o_{p,m} - t_{p,m}\right)^2$$

Note that MEE≠RootMSE since sum(root(a), root(b)) ≠ root (sum(a,b))

What is provided 2025

Dip. Informatica
University of Pisa

- Training set
  - ML-CUP25-TR.csv
- Blind test set (without target)
  - ML-CUP25-TS.csv



!!!!!

WHERE:

- **ML section on Moodle: https://elearning.di.unipi.it/**
- **Folder:** ML-25-PRJ lecture & package *
- With files for cup data, info (slides and txt), slides-demo & report-info, cup results template

- Note: past editions **are NOT valid for this edition/ year**

What to produce 2025 (I)



Dip. Informatica
University of Pisa

- **Output file** (blind test results) in a simple text/*txt* format (comma-separated values, see FAQ III). The name must be: ***team-name*_ML-CUP25-TS.csv**
- Using the following format
  - First 4 rows are for comments:
    - # your names
    - # team (nickname max 8-10 char) for the web results
    - # data set name (ML-CUP25 v1)
    - # date (e.g. 2 Jan 2025)
  - Table with 1000 rows and 5 columns (comma separated values):
    - id, o_1, o_2, o_3, o_4 (i.e. 4 outputs of your models)
    - id ordered from 1 to 1000 (exactly as for the file ML-CUP25-TS.csv)

    See the demo file: ***template-example-with-random-outputs_ML-CUP25-TS.csv*** (which is filled with random-output values for demo)

PLEASE, double check the output file format!!!!



- If it is not OK we cannot evaluate it automatically → jump to the bottom of the rank !
- Repetita: you can send **only 1** *team-name*_ML-CUP-25-TS.csv file (assuming it is your best result)

What to produce (II)



Dip. Informatica
University of Pisa

- File: team-name_abstract.txt, in a simple text/*txt* format
- With a very short (5 rows) description of the **used final model** (e.g. NN/SVR/…) and validation technique.

- HENCE, you have to send (for the cup):
  - ***team-name*_ML-CUP25-TS.csv**
  - ***team-name*_abstract.txt**

- Along with (see the "Project general rules" slide at the beginning of this lecture)
  - Your code
  - Your written slides report (with results on 3 MONKs and the CUP)

Results



Dip. Informatica
University of Pisa

- Initially I will personally communicate the result to the single participant during each oral
  (for fairness with participants following in time)
- At the end of next year it is possible to award *the winners!*



- Criteria for the winner on the task: accuracy (MEE) and possibly the quality of final plot of the results (at the discretion of the jury ;-))

The project

- **Premise** ("experimental", main aim, etc.)
- **Form a group (phase I)**
- **Type A, B, etc. (phase II)**
  - The simulators
- **The application (phase III)**
- **The slides** (format, style, etc.) **(phase IV)**
  - And template demo & info files
- **And slides presentation (phase VI)**

The report: slides



Dip. Informatica
University of Pisa

- To help you: see the *demo*/template of the slides file: **ML-2025-PRJ-Slides-Demo** **https://docs.google.com/presentation/d/11dcEY9uq8fOqWfRqLosXQlEI9cHM54nOQPYyOet6Dag**
  - Check the last version (Vx.y)
  - Don't' change the demo **(make your copy!)**
  - Use your UniPi account to connect
  - A copy will be part of the package
- The demo file includes descriptions of the information to be provided:
  - these are mandatory to accept the report as valid for the exam
  - **double check in the demo files if you reported all the needed information** (see also the next slide)
- Please follow such basic organization for your slides
- English language

An other auxiliary file



Dip. Informatica
University of Pisa

- Use also this document with useful info for your project and slides content, file:
  ML-25-Report-info-v*.pdf

- It helps you, it is in the form of a report since in the past we used a report style (instead of slides), …
- But, this year don't write a report in that form, ***you have to prepare only slides and present/discuss them at your oral session***

Slide presentation/discussion(*): mandatory rules

Dip. Informatica
University of Pisa

- Time: you will have sharp time to present (or discuss *)
  - **Max. 16 minutes** for a group of 2 students
  - **Max. 18 minutes** for a group of 3 students
- I'll interrupt the presentation after this time, due to conciseness and fairness reasons
- I expect that all the members will present for the same time (so ~8 min. each for a group of two, or 6 min. each for a group of three)
- (*) In some cases, the presentation will be changed into a discussion. I'll advise you directly at the oral, so be prepared for a presentation

- This typically correspond to around
  - **Around 16-18** slides for a group of 2 students (no more, see appendix in the demo)
  - **Around 20** slides for a group of 3 students (no more, see appendix in the demo)
- Font: **font** $\geq$ 14 for the text (exception for some rare cases to 12 for secondary things), typically a **PDF** file (include a pdf copy in any case)
  - Small font slides will be rejected (we will control also with a tool)
  - Too full slides are not useful for our aims (see later)

Slide presentation: strong suggestions!

Dip. Informatica
University of Pisa

- Synthesis issue:
  - **Focus on what is relevant to show!**
  - Given that you provide all the necessary information in the slides (see the demo & info files), the choices regarding the synthesis of the all the other results depends on *your decisions/preferences*.
  - The *time and space limit* must not be forced, also because it is the constraint that imposes the synthesis in order to *exercise/test your choices.*
- Don't forget to ***empathize the novelties*** that you introduced in your model and/or any significant/critical analyses and ***any interesting finding/insight***
  1. [model/approach/techniques/implementation **novelties**] It is important to stress what you have been investigated beyond ordinary techniques (if any)
  2. [**Discussion**] Evaluate the effects of the novelties on the results.
     Also, don't forget a "Discussion/Analysis" answering "*what did you learn?*" on the Models/Hyperparameters/Results/Efficacy/Efficiency, selecting/highlighting what is *more significant in your opinion* (again, time and space constrains helps!)

Look at …

Dip. Informatica
University of Pisa

I'll in particular evaluate (not necessarily in this order):

- Completeness: adherence to requests (in particular see the [*] sign in the ML-25-Report-info file for the mandatory info)
- Correctness with respect to the ML concepts
  - In particular, clarity and correctness of the *validation/assessment procedure* and choice of the final model
- Depth: *introduced novelties supported by evidences*, and quality of the *analysis and discussion*
- Synthesis: focus on interesting aspects!
- Autonomy of the group
- Code quality & MONK results, Efficiency of your implementation
- Effort to achieve the best CUP result (not necessarily the result in itself)
- CUP results: The *blind test* result is just one of the possible parameters (not the most important): however, *very often* it strongly depends on the quality of the model and of the validation approach.
- Quality/clarity of the oral presentation and/or of the discussion
- (further details in Sec. 3)

*The ultimate aim is to evaluate the **maturity** of your approach*

Finally

Other hints/request for the prj

Screening phase

Dip. Informatica
University of Pisa

- Initially you will try different values of hyper-parameters
- (if you think interesting) significant cases of learning curves can be reported in the report
- **but for yourself** plot them for various combination*
  - →the screening phase is essential for yourself to learn from this experience [see next slide demo]

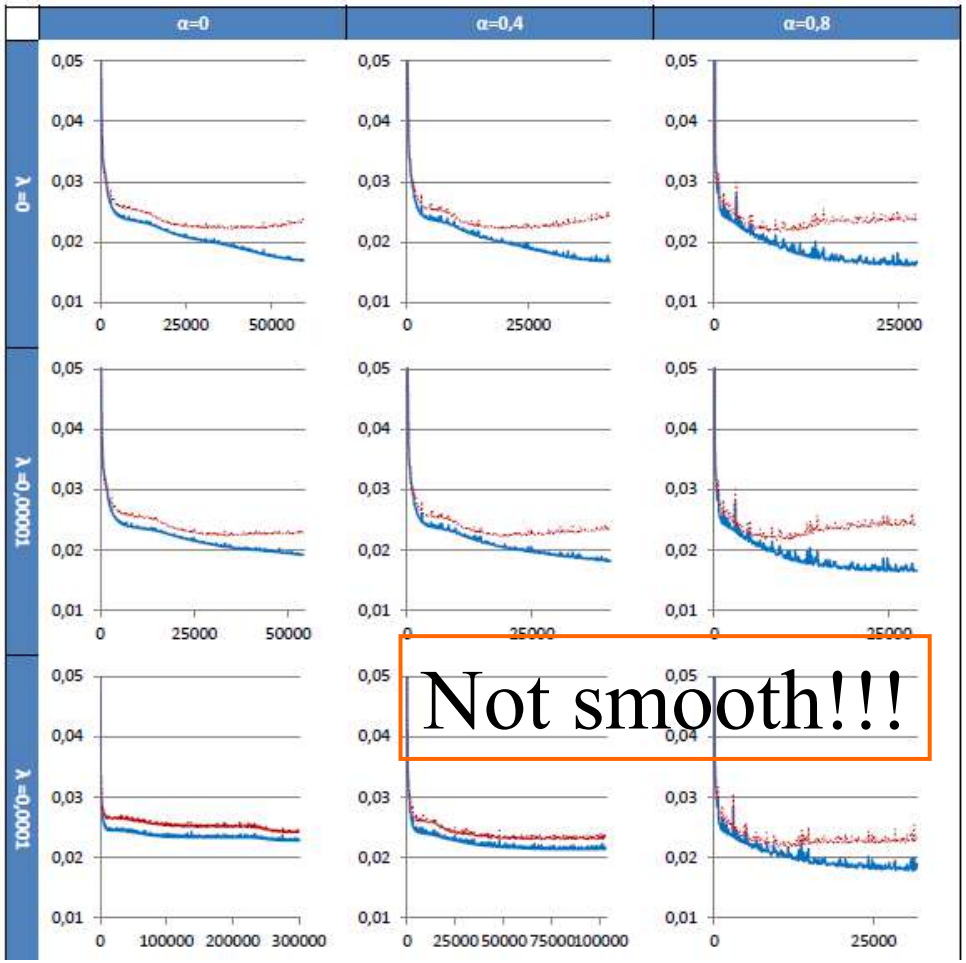


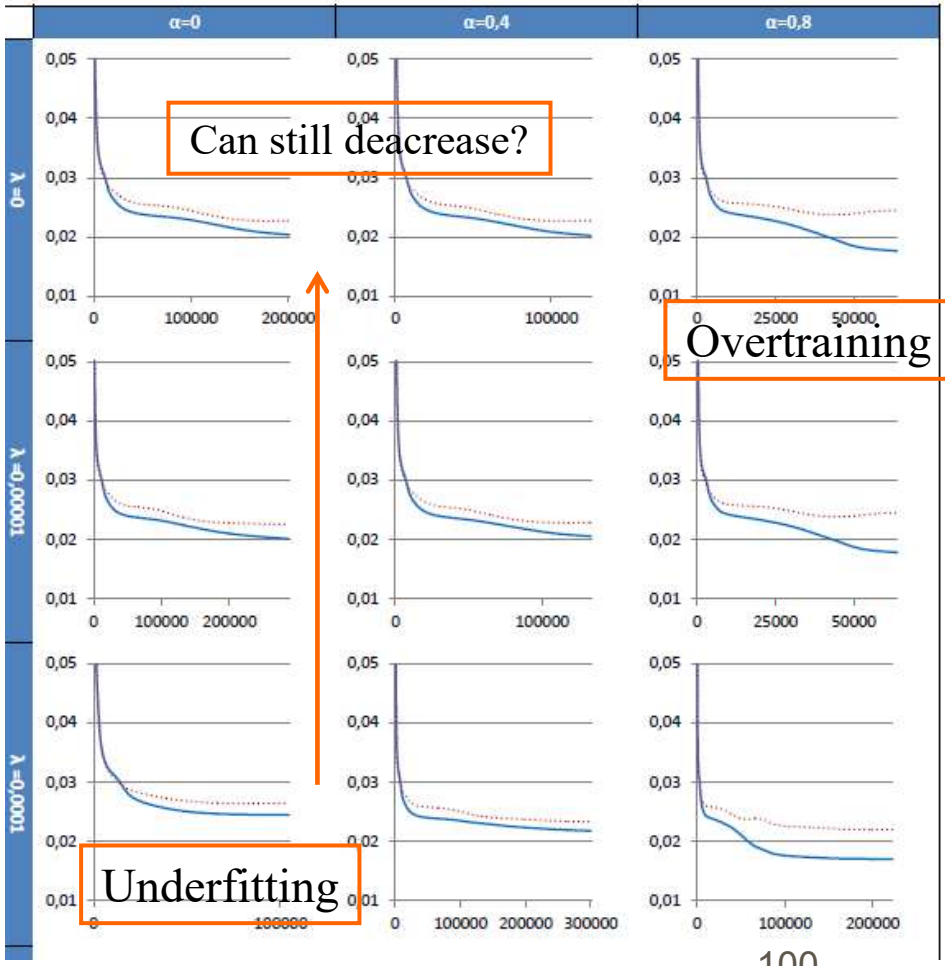
- Results of this kind (also from the grid search) could be part of the appendix

Screening demo

Dip. Informatica
University of Pisa

- Some instances of screening phase (don't mind of the specific demo hyper. values, look to behavior changes…and make the same with your values)





How many rules?

Dip. Informatica
University of Pisa

Again, everything is **easy**

1. You have a **task and dataset**
2. You have **to apply using your models** and to **report the results**
3. You have to respect the **deadlines**

But… many questions have been raised, so to help you I report the answer to many FAQ

- Please, consider them; the first rule of the FAQ: **read the FAQ**!
  - Please, *don't ask for questions already answered in the FAQ* in these slides or in the Moodle FAQ section (most of the questions that we typically receive can be answered with "Please, read the FAQ").
  - Also avoid to miss the described needs for the project
- Please, to avoid further rules "cluttering" *do not ask other rules* ;-)

Sec 3. FAQ & Other Info

How to evaluate your work: further info (Details I)

Dip. Informatica
University of Pisa

- **Autonomy** (is part of the evaluation)
- Soundness and quality of the (code) simulator (including any characteristics of modularity, efficiency, ...)
  - Note that more efficiency allows you to have more time for experimental part that is typically time demanding
- Proper operation and behavior:
  - e.g. see the regularity of learning behavior on the plots (see the FAQ), …
- **Pertinence of the choice** for the model selection and evaluation
- Extension and/or depth of your investigation
  - Model and/or algorithms variants
  - Hyperparameters range
  - Validation and assessment …(**accuracy/rigorousness of the validation**)

How to evaluate your work: further info (Details II)

Dip. Informatica
University of Pisa

- **Quality of the report**:

1. completeness (adherence to requests, see the demo & info files), 2. correctness, 3. synthesis, 4. depth.

And in general: organization, rigor, soundness and synthesis (no repetitions); motivations suitability; choices made; *breadth and depth* of the experimental investigation (although expressed in a concise way);
clarity of the validation schema
clarity on the choice of the final model used for the blind test
quality and depth of the analysis and discussion of your evidences & findings

- In case: difficulty of the challenge (if it is well done) can be a plus (e.g. <A) with CM>)

*The ultimate aim is to evaluate the **maturity** of your approach*.

FAQ 0 [repetita]



Dip. Informatica
University of Pisa

Typical serious matters to make possible the correction:

- Respect the presentation *time limits*
  - hence, take care of the **number of slides**
- Respect the *minimum font*
  - Too small font report will be rejected
- Respect the *format of the CUP results* (and range values):
  see the file "template-example-with-random-outputs_ML-CUP2*-TS.csv"

FAQ I: Deadline [Repetita]



Dip. Informatica
University of Pisa

- **Deadline**: see the beginning slides
  - Check the **exact deadline** for the project material delivery **in large advance** in the Moodle folder for each session, i.e. in Prj Student Material → Session xx (your session of interest)
  - Can be extended**?** No, **later is indeed in advance for the next session** (using the proper folder).
  - It is a strict automatic deadline: please, check it in large advance!
  - I'm sorry, I *cannot* (*) reply to emails asking to move the date (or to extend it for personal issues) of the exam/session or the deadline of the prj delivery.
  - (*) Please, image the case: If I do it for you, I have to do it for all the candidates (it is also a fairness matter) & session organization is lost.
    If you (the group together) cannot come, you simply can come in the next session ("appello"). Please, in case advise me in advance.
  - Thank you in advance for your understanding.



FAQ II (Practical)





Dip. Informatica
University of Pisa

- Is there any **CHECK LIST** for material delivery**?**
  - Previous slides on <Delivery and deadlines for Exam and the Project> and < What to produce> (2 slides)
  - For the **report**: see the demo & info files with [*] for mandatory info on the results
    - ML-2025-PRJ-Slides-Demo* **&** ML-25-Report-info*
  - Check that cup **result file** has the proper name and **format**
  - Provide the following files:
    - Your code (if original, else use a link to libraries in the report, add it to READ-ME file in the code package)
    - Your written slides report (with results on 3 MONKs and the CUP)
    - *team-name*_ML-CUP25-TS.csv
    - *team-name*_abstract.txt
- Deliver the package only in the Moodle and only as **zip file**!!!
- Only **1 delivery** for each group, by 1 member of the group
  - Use the **surnames** of the group components for the package file name
  - And for the pdf slide report file
- Send confirmation email (slide: "Project General Rules: How to deliver it")



FAQ III (Practical)



Dip. Informatica
University of Pisa

- **How the group communicate with teacher and assistants and for the final project delivery?**
  All the group member **must** be included in cc of the email, no answer will be sent to a singular partner of a group (also to avoid partial info among the group components)

- **Do we need a supercomputer/server/GPU to run the prj?**
  Not at all, this is a small task: you can use any machine or also external services (e.g. Colab) , **but typically your PC/laptops are sufficient** to conclude the prj in a reasonable time (also the constraints on the resources force you to focalize your effort, see the lect.s on validation)





FAQ III (Practical)



Dip. Informatica
University of Pisa

- **CSV** format for the CUP results: see the files, it is a comma-separated values without spaces after the comma. Each pattern is a row. The header of input file has some rows of description beginning with #.
  - ***Example:*** output_template_example-with-random-output-ML-CUP25-TS.csv (which is filled with random-output values for demo)

- Executable programs, libraries, … (**large files**): until the package is small size try to include everything in the package.
  If you have problems include (in the READ-ME file of the code package) a link to download the *** accessories files *** of great dimension
  - Check the zip archive size before submitting it. Please try to stay **within 5 MB**. If it is above 5MB, make sure you've cleaned up notebook outputs or that you haven't included hidden files by mistake. **In any case, the limit for the package is 20 MB**

- Plots , graphics etc.: it is mandatory to distinguish lines in the plots also using different lines symbols/style (to see them also in Black&White)



FAQ IV



Dip. Informatica
University of Pisa

- How many hidden layers do you suggest for the CUP?
  I strongly suggest you to try also two (or more) and compare (even if a single hidden level can result sufficient)
- How many trials (grid search dimension for the hyperparameters values)?
  How many you feel are useful in order to do a good model selection. Such choices are part of the quality of your work.
- Which trials should I report? All the significant cases with details (experimental evidences), the other can be mentioned/synthetized in the text. Such choices are part of the quality of your work.
- **MSE** or **MEE**? You can use MSE (LMS) for training and MEE to evaluate
- Is "**internal test" result** needed? **YES**, it is, so we can compare your results with the blind test results (for our didactics aims)
  See also the ML-25-Report-info file.
- Does the use of the **(internal) test data for model selection** aims influence our prj evaluation? **YES**!, using "test" data in any form for the model selection is considered **a serious issue**!



FAQ V



Dip. Informatica
University of Pisa

- What if my model **does not work well**?
  - Check the learning curve to find clues
  - HINT: Try to compare with respect to a known tool in the same condition
  - Take advantage of the Q&A class (with also the assistant)
  - Help of the assistant
  - But…: Autonomy is part of the evaluation

- **PRJ A**: For **efficiency** reasons, it is suggested an implementation that uses a matrix approach (in terms of batch size and layers of units), exploiting NumPy, versus the use of "for" cycles on vectors e.g. considering individual units



FAQ VI



Dip. Informatica
University of Pisa

Even more QUESTIONS?

- Please, read the following slides (other FAQ) before the next Q&A lecture!!!!
- They are related to frequent past errors!
- Please, read other FAQ in the Moodle page (I can update them)



FAQ (NOT read in class I)

Dip. Informatica
University of Pisa

Please, read them by yourself :

- [Report] It is important to report the final Training, Validation and the internal Test error by the MEE measure?
  Yes, it is a must. I cannot accept a report without these values.
- [Report] It is important to report how we obtained the final model (how selected)?
  Yes, it is a must. I cannot accept a report without this information.
- Can you provide to me an idea of the basic result for the CUP? Should I have near 0 errors like Monk?
  No, zero errors is ONLY for *some* of the Monks tasks, an exception. For reasons of equity, no indications can be given on the reference performance for the CUP**.**
- The error to be reported in the tables is without the penalty term? Yes, without it
- General note on plots in the report: to be comparable, use a point for each epoch (defined as the total number of training data): if you are using on-line or mini-batch provide the mean over an epoch for each point in the plot.
- What if my learning curve is unstable? Try to solve the issue! A smooth learning curve can be obtained for these tasks by a correct implementation and a suitable setting ! Else, I have to consider it as a *not good* result.

FAQ (NOT read in class II)

Dip. Informatica
University of Pisa

Please, read them by yourself :

- Can we use other methods beyond grid search?
  Yes, but first grid search approach is mandatory. It is needed to teach to you the meaning of the hyperparameters! Others? Yes, e.g. Random search : see Validation 1 (Valid1) lecture. But if you like, and only after the result with grid search
- Is it needed an advanced validation approach for Monks?
  No, Monk dataset is a benchmark to assess your code and system. You can also assess the validation process with Monk, but it is *not* needed to address this task by a complex CV approach.
- Plots with MEE or MSE?
  MEE for the tables (and this is mandatory!), for plots up to you (typically the quality of the learning behavior does not change)
- Usage of Early Stopping:
  See Validation 3 (Valid3) lecture

FAQ (NOT read in class III)

Dip. Informatica
University of Pisa

Please, read them by yourself:

- Does a NN for **Monk** converge always with few hundred of epochs"?
  The model converge easily in "most of the case" with the suggested setting (and also with other settings). But not "always" due to the sensitivity to the random initialization using a large range of the initial weights. Use a very small range for weights init.! Please see in the lecture NN2.
- So what accuracy have to be reported for the Monk, just the best over different trials/initializations?
  No, the mean accuracy over that trials. See the discussion above.
- How many units for Monk? Very few. Follow the manual as baseline: see a slide in this lecture!
- How many units for the CUP? You have to search by yourself, but it is completed unrelated with respect to the Monk task!!!
- What if a use a scaling/normalization of target values for the CUP ? It is not need to do it; if you do it, remember to report the MEE in the **original scale** (for the test result file and the tables on the report). It is a must in order to make possible the evaluation, i.e. the comparison! Take care: it is "popular" way to fail on the results

FAQ (NOT read in class IV)

Dip. Informatica
University of Pisa

Please, read them by yourself:

- Can we collaborate with other groups for specific issues? We encourage to exchange ideas and to solve some common issues with other students (e.g. is the ***w*** update rule correct in my code)? However, of course, the project will be completely independent among different groups, and you must show your autonomy in developing it. We suggest to follow your original path and exploit in autonomy this experience!

- What is the time to be reported when you say «Provide an estimation of the (training) computing time (and of your HW resources)" in the info doc for the report?
  A measure of the time like the mean time (e.g. in second/minutes/hours) for an epoch of training, or better for a full training of the final selected model, and/or a cycle of validation (and the description of the computer that you are using to make this training).

FAQ (NOT read in class V)

Dip. Informatica
University of Pisa

Please, read them by yourself:

- Making a prj with SVM, which plot can I do?
  See E.g. Slide "Critical role of Hyper-parameters on the performance" of <ML-**-SVM-other info> lecture (e.g. results w.r.t. hyper-parameters values, including epsilon-tube parameter in case of regression)
- Project B (or <A with CM>): If the winner is not a NN (or a methods with iterative training) which plot can I do?
  See the answer on SVM above (you can show effects of the hyper-parameters). Moreover, if you are considering a NN in the set of models, even if it is not the winner, you can report the learning curve for the selected NN. Explain the case in the report.
- Prj <A with CM>: comparison also on Monks?
  Using a NN the results on MONK are still needed (at least summarized if the not interesting for the basic or the advanced algorithms), and of course you have to apply to the CUP dataset. For the SVR (if only for regression) PRJs (with CM) applying to MONK is *not* required. But see the Q&A on do we need to implement also the NN? (next page) or you can also always use your SVM also in the form of a classifier.

FAQ (NOT read in class VI)

Dip. Informatica
University of Pisa

Please, read them by yourself:

- Making <A with CM> implementing a NN, do we implement the basic and advanced NN by ourselves? Yes. It is a type A prj.
  - In the CM report you will discuss the properties of the algorithms from the CM view point.
  - In the ML report you will discuss the "basic" NN and the *comparison* with the "advanced" algorithm developed with CM.

- Making <A with CM> implementing a SVM, do we need to implement also the NN? Up to you, in this case
  - you can develop the NN by yourself (type A prj) or
  - you can take the NN from a library (like for a type B prj, in this case it is a mix),

  but, in any case, you have to *compare* the results (for the ML report)
  General note: the slide for <A with CM> said: "More effort? It is because you like to try and show something more and not something less!", hence I strongly suggest to compare also with a NN

FAQ (NOT read in class VII)

Dip. Informatica
University of Pisa

Please, read them by yourself:

Type B Prj:

- Project B: Do we have to compare the models on both MONK and the CUP data sets? **Yes**, you have to compare ALL the models on both the Monk datasets & on the CUP dataset. You can synthetize using only the most interesting results and you can use the appendix for details

- Project B: Do we have to include the code?
  Yes, the code written by yourself, in case, e.g. to use the libraries/notebooks etc., must be included in the package

- Package size:
  "Check the zip archive size before submitting it. Please try to stay **within 5 MB**. If it is above 5MB, make sure you've cleaned up notebook outputs or that you haven't included hidden files by mistake. In any case, the **limit for the package is 20 MB**"

FAQ (NOT read in class VIII)

Dip. Informatica
University of Pisa

- What changes using **MATLAB/OCTAVE/R?** **[repetita]**
  - If you code by yourself → prj type A,
    but exploit your advantage (less time for the coding phase) to use more time in exploration/usage of advanced Matlab numerical computing functions/ possibly a comparison with other available models in the environment/extensive cross-validation/ impressive graphical results/…
  - If you use the NN/Deep Learning Toolbox → prj type B

- Can we apply our models also to a "personal" or other data sets?
  - It is not necessary. If someone wants to do it, it is always *appreciated* when students do more; however, such results should be included as appendix material in the appendix and only after the rest has been completed:
    *It does **not** replace the main requirements*, i.e. the results the provided benchmarks for MONK and CUP.

FAQ (NOT read in class IX)

Dip. Informatica
University of Pisa

- Project delivery: For the type B project, because its analysis is more focused on the comparison of different models, for the software part, can we deliver a Jupyter Notebook (at least for the models used with Python library) or do we have to send "pure" Python script?
  - Since this is a project that lends itself to using different libraries and frameworks, I would very well expect a Jupyter or Colab Notebook (if you are using them) rather than pure python code with various requirements to install in order to test it.
  - It's different for project A, where you will use only numerical libraries.
  - But it is good to remember to remove the output of the cells before delivery, to avoid that you also save the images or plots of each run. Just run "Cell -> Output cells -> Clear" before saving the notebook for delivery (to avoid cluttering and huge delivery in terms of space).
- Package size (any kind of project):
  Check the zip archive size before submitting it. **Please try to stay within 5 MB**. If it is above 5MB, make sure you've cleaned up notebook outputs or that you haven't included hidden files by mistake. In any case, the **limit for the package is 20 MB**

Motivational hints by past students (I)

Dip. Informatica
University of Pisa

Selection of general nice hints/motivations by past students reports conclusions (when it was a report):

- "The powerful libraries available nowadays provide a high level of abstraction and while this is useful for quickly getting things done, a naive use of a user with no solid knowledge of **the theoretical ground** may lead to a disastrous outcome. […] In conclusion, by implementing our own library, we feel that we are now much more aware of whats happening behind the scenes."
- "The limited time and computing resources **has pushed us to think** more about reasonable choices or shortcuts to speed up the searching for a fine model selection for the Cup dataset, showing us how necessary is the integration between artificial and human intelligence."
- "Last consideration is on how important is to **visualize results in the best way**. Not only it is fundamental for presentation itself, often it is the fastest way to understand how the model behaves in the hyperparameter space."
- "… the **«DIY» approach** enabled us to really grasp concepts from the ground up, to establish deep knowledge on the mechanisms seen during the course."

  DIY "Do it yourself" aka "self-made-culture"

Motivational hints by past students (I)

Dip. Informatica
University of Pisa

Selection of general nice hints/motivations by past students reports conclusions (when it was a report):

- "Lo sviluppo di questo progetto è stato sicuramente un'ottima motivazione per mettere in pratica quanto affrontato nel corso, sia in termini di implementazione e comprensione degli algoritmi alla base delle tecniche più moderne di Machine Learning, sia in termini di applicazione di una rigorosa metodologia per la valutazione e soprattutto la validazione di un modello, indispensabile per raggiungere risultati plausibili e confrontabili con quanto studiato in teoria. **La coordinazione che un lavoro condiviso come questo richiede è inoltre un ottimo 'training test' per lo sviluppo in gruppo**, ed il confronto continuo durante le diverse fasi dell'implementazione e validazione ha benificiato entrambi nella comprensione del lavoro svolto."

- "I **concetti teorici trattati a lezione** di cui fanno parte ad esempio le tecniche di regularization […] e le nozioni di underfitting/overfitting di un modello sui dati sono stati visti in un ambito più pratico e è stato molto utile ai fini didattici. Le richieste ci hanno permesso di scoprire più a fondo il mondo del Machine Learning e in particolare quello delle Neural Network, infatti abbiamo avuto l'occasione di mettere le mani su dinamiche e funzionamenti che dall'esterno risultano trasparenti."

- «Sono abbastanza soddisfatto del risultato ottenuto, ma più di tutto, sono contento di aver fatto il progetto A perché mi ha permesso di capire nei minimi dettagli ogni aspetto del funzionamento di una rete neurale e, collateralmente, ha anche migliorato le mie capacità di scrivere codice.»

Enjoy !

Dip. Informatica
University of Pisa

- … Now you can **enjoy** with ML !
- To stimulate your initiative, listen "rock music":
  - "La Follia" (Corelli)
  - Il "cimento dell'armonia e dell'invenzione", "L'estro armonico", "La stravaganza" (Vivaldi)



Future – just ahead

Dip. Informatica
University of Pisa

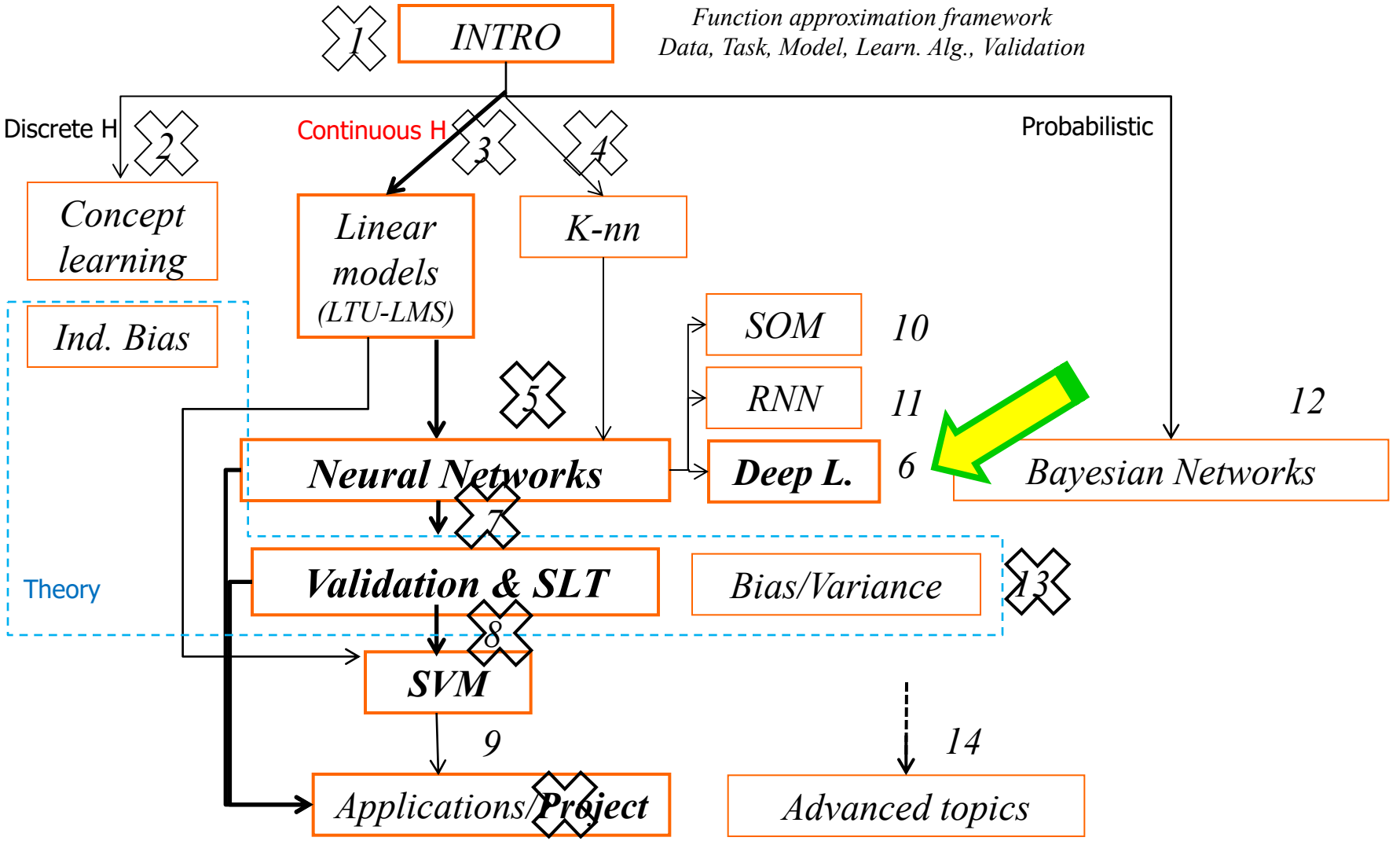


- CIML@UniPi
- Didactics: other courses (ISPR, HLT, CNS,…)
- General topics for ML applications
- CIML@Pisa research

2025: THIS PART WILL BE PRESENTED LATER, Just wait

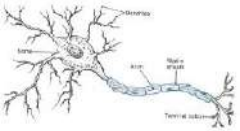
ML Course structure
Where we go

Dip. Informatica
University of Pisa



Course structure: zoom in the NNs box

Dip. Informatica
University of Pisa



1 unit NN

Perceptron

Adaline

LMS

As a linear model (previous lects)

Neural Networks

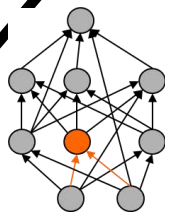
MLP: Mack. &Pits

LEARNING: Perceptron learning alg + Convergence theorem

Non-linear out (f)

LEARNING: Backprop

MLP: feed-forward NN

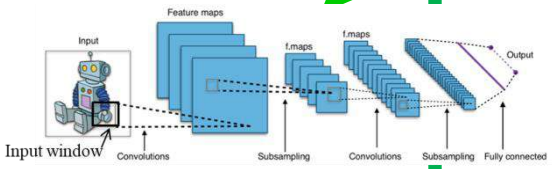


Heuristics for BP

CasCor

Regularization

We are here



Applications: **CNN** example

Introduction to recent NN paradigms (**Deep**, Random.)

Made later, after the PRJ (to favorite validation, SVM, …)

Applications

Dip. Informatica
University of Pisa

A huge number of successful applications in all the fields of applications seen for ML (see intro of the course) since his origins

- Performance
- Easy to be configured as a first approach for learning arbitrary functions whose general form is unknown in advance

(without explicitly assuming a prior statistical models)

Sub-symbolic tasks (especially sensorial data, motor control, visual processing) have been a workhorse of NN.

Exploited in robotics, pattern recognition, control, …

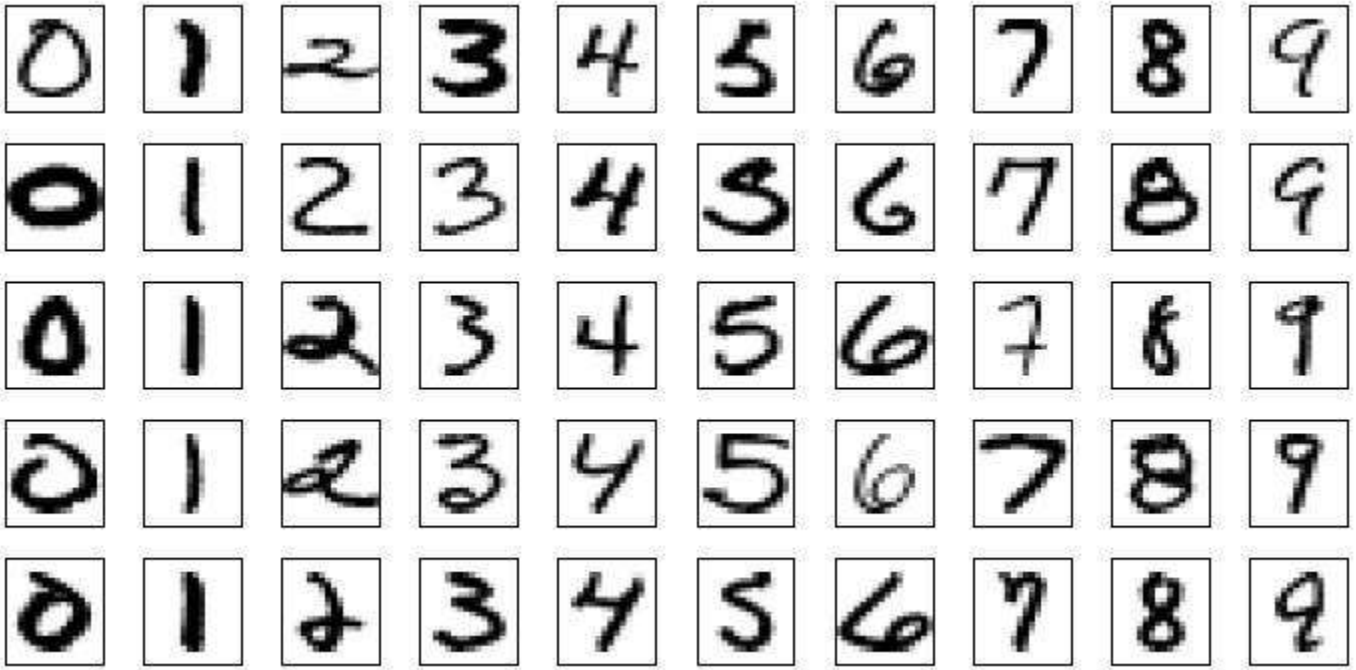
An example:

- Showing the flexibility of NN design
- and historical example on how to **change the design** dealing with specific applications
- A way to implement a not-fully connected FF-NN for a specific problem

see other more standard examples on Mitchell books for face recognition tasks (section 4.7) … *and nowadays even by newspapers!!!!*

Character recognition: classification

Dip. Informatica
University of Pisa



Examples of training cases from ZIP code data.

Each image is a (normalized) 16×16 8*-bit grayscale representation of a handwritten digit (Le Cun et al. 1990).*

© Hastie, Tibshirani & Friedman 2009

MNIST benchmark ("the *drosophila** of machine learning,")

First approaches

Dip. Informatica
University of Pisa

- Standard NN with 256 inputs (why?):
  - Misclassification rate of 5%/20% (many errors due to lack of invariance as rotations etc.)

- Basic idea: exploit architecture design to include prior information into NN
  - restricting the network architecture: local connections of receptive fields: extraction of local features
  - constraining weights by *weight-sharing* among different units → reduce the number of free-parameters while allowing more complex connectivity
    - The same operation on different part of the input (image)
    - Because the features of the characters should appear in different parts of the image (regardless the position, small rotations, etc)
  - "LeNet"/CNN/ConvNet: *Convolutional (neural) networks*

(**Refs.: see Haykin or DL book chap. 9**)

The name Convolutional

Dip. Informatica
University of Pisa

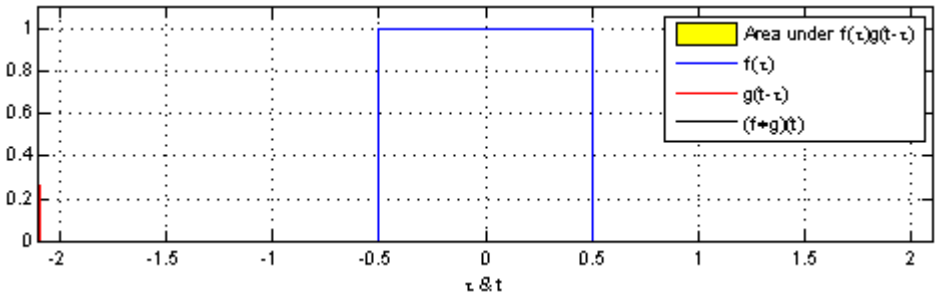
- The name Convolutional NN takes inspiration from the *Convolutional operator*

$$(f * g)(t) \stackrel{\text{def}}{=} \int_{-\infty}^{\infty} f(\tau) g(t - \tau)\, d\tau = \int_{-\infty}^{\infty} f(t - \tau) g(\tau)\, d\tau.$$

CONCEPT:
Weighted average of a function *f*,
weighted by another function *g*
moved over time (**sliding**)

We are skipping mathematical def. of "well defined" convolution etc.

Image *g* limited, defined (not zero) on an interval



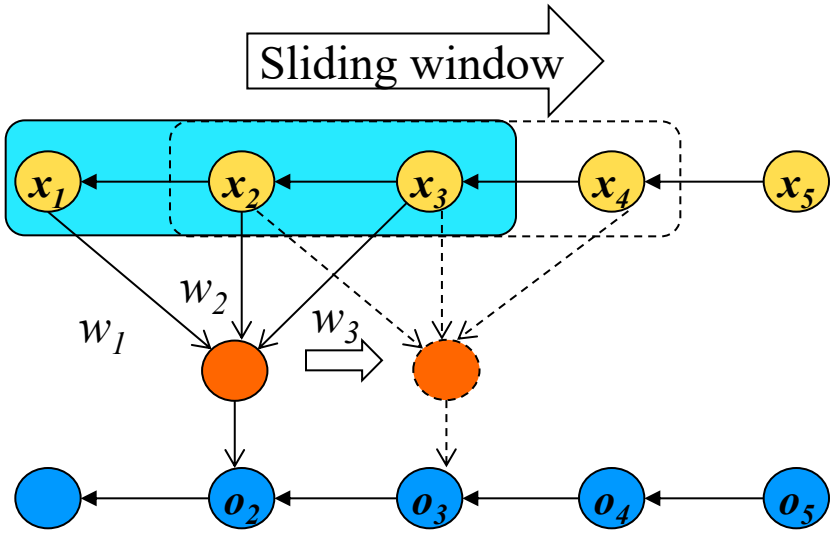
Animation at
https://en.wikipedia.org/wiki/Convolution#/media/File:Convolution_of_box_signal_with_itself2.gif

CNN details: Convolution

Dip. Informatica
University of Pisa

- A simple neural unit example over a stream

$$out_t = \sum_{i=1}^{3} w_i x_{t+i-2}$$



Exercise: compute o_2 and o_3 according to the equation above (start with t=2)

Weights are tuned as usual **by learning:**
An adaptive filter

We will call this **Time-Delay NN**

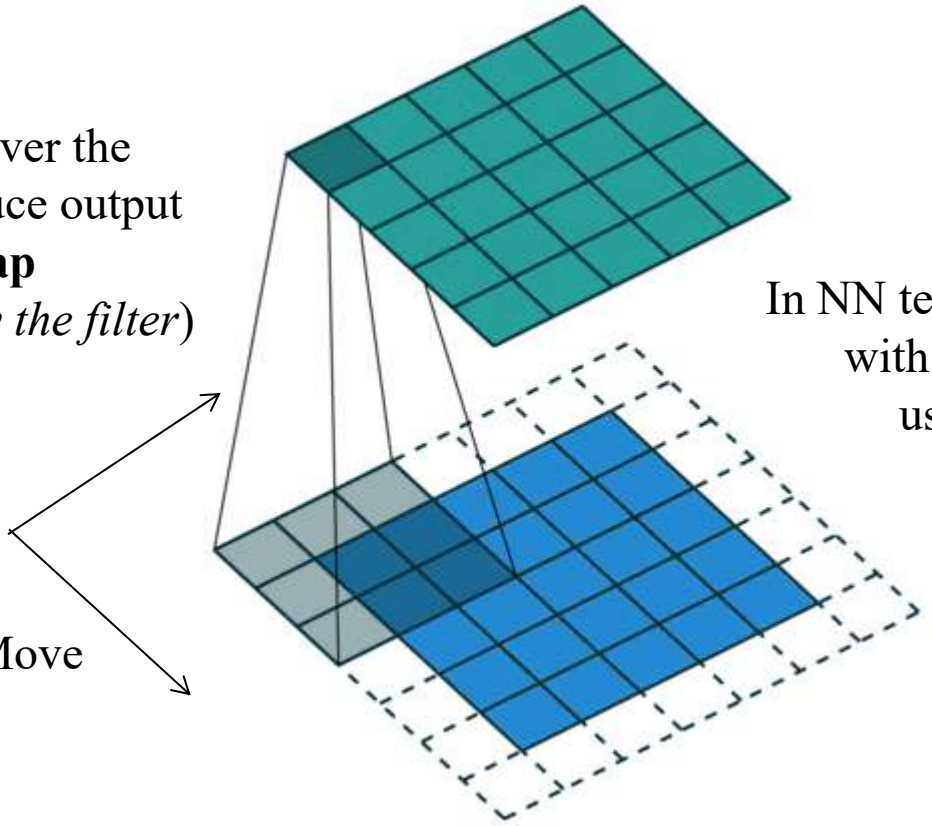
Weight sharing
for the two instances of units
More units operate as a "filter bank"

CNN details: 2D Convolution

Dip. Informatica
University of Pisa

- 3x3 kernel (unit *local receptive field*), 1 pixel shift (*stride* 1 for traversing)
- Padding is used to treat the border

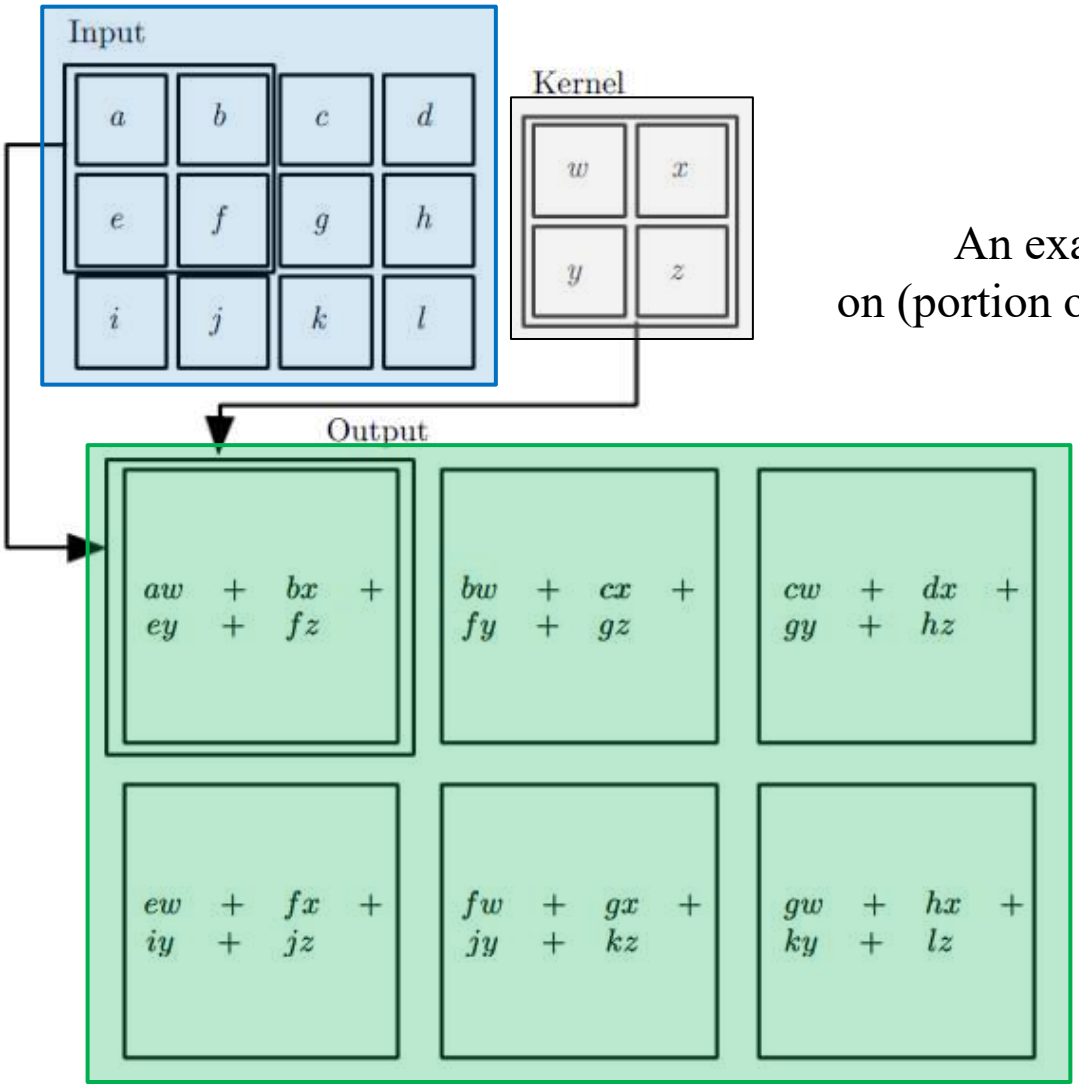
Move the kernel over the image (blu) to produce output (**feature**) **map** (*features extracted by the filter*)



In NN terms: we traverse the image with (*the same*) neuron that uses local connections (*receptive field*)

CNN details: 2D Convolution

Dip. Informatica
University of Pisa



An example of 2-D convolution
on (portion of) a 2-D (2 dimensional) image

Convolution in 2D



Dip. Informatica
University of Pisa

To have a look to an example of implementation over a 2D image ***I***, with a 2D kernel ***K***

$$S(i,j) = (I * K)(i,j) = \sum_m \sum_n I(m,n)K(i-m, j-n).$$

This form is used in many NN library for CNN:

$$S(i,j) = (I * K)(i,j) = \sum_m \sum_n I(i+m, j+n)K(m,n).$$

Written as *cross-correlation* form

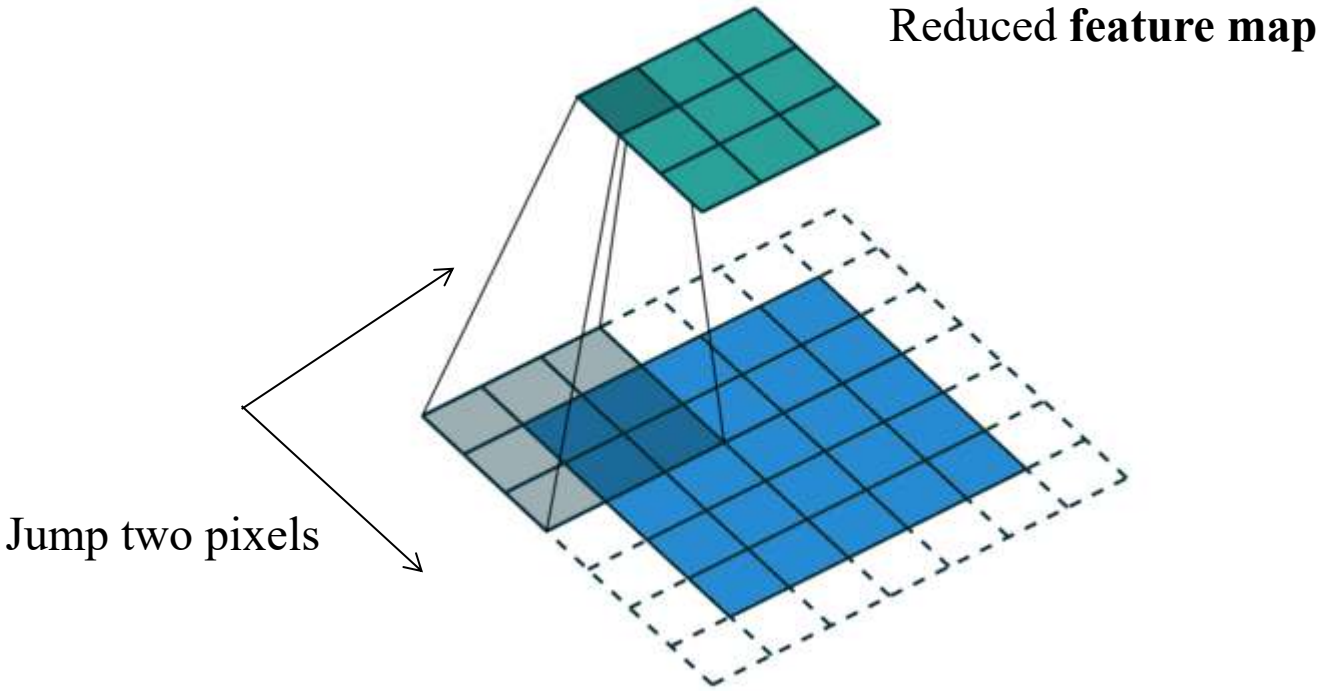
DL book : 9.1 for details

Not for the oral!!!

CNN details: 2D Convolution Stride > 1

Dip. Informatica
University of Pisa

- Down/Sub-sampling: moving jumping two pixels
- Aka: "stride"



A network of “filters”

Dip. Informatica
University of Pisa

The units' weights are a ***filter*** trained to detected some features/pattern in the image

Architecture design

- Local and small receptive field (kernel): by this architecture constraints, learnt "filters" produce the strongest response to a spatially local input pattern (similarly to the animal visual cortex).
- The same unit/filter is applied across the entire image: allow for features to be detected regardless of their position in the visual field, thus constituting the property of *translation invariance*.

- Learnable filters: we can learn the filters that in traditional algorithms for image processing were hand-engineered. This independence from prior knowledge and human effort in feature design is a major advantage.
- Stacking many layers: feature maps unit represent larger and larger areas of the original image (assembling areas of the previous feature maps) . See later.

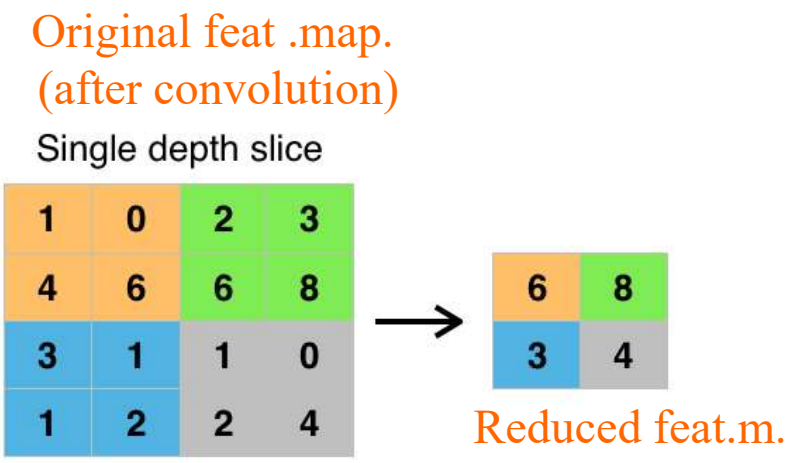
CNN details - Pooling

Dip. Informatica
University of Pisa

- Reduce the feature map by
  - subsampling (see before, stride > 1) or
  - operating a mean (average or weighted average) or
  - **max pool** operation (most common)

  over the neighbors of each output

In practice instead of producing a value for each input pixel, we produce a value for a set (rectangular) of pixels taking the *mean* or the *max* of the kernel/neuron outputs (feature map)



Max pooling with a 2x2 filter and stride = 2

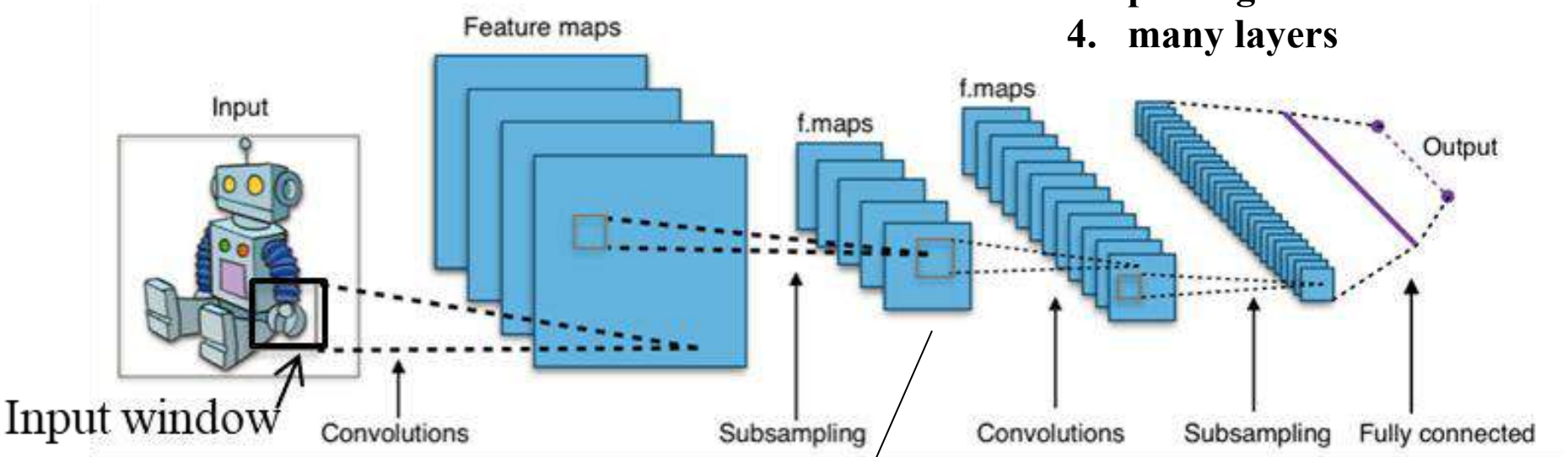
- Pooling also further helps to make the representation become approximately *invariant* to small translations of the input.
  - Because the max/mean has small changes translating few inputs → e.g. a high output is still high for a slight translate version

CNN as a whole

Dip. Informatica
University of Pisa

- CNN exploits weight sharing to set-up a shifting window/*local field* of the units over a segment of the input signal, also extended to 2D images and re-apply for many layers (feature maps) (activating pooling subsampling/average operators)

1. **local connections**
2. **shared weights**
3. **pooling**
4. **many layers**



Feature maps
from different filter banks

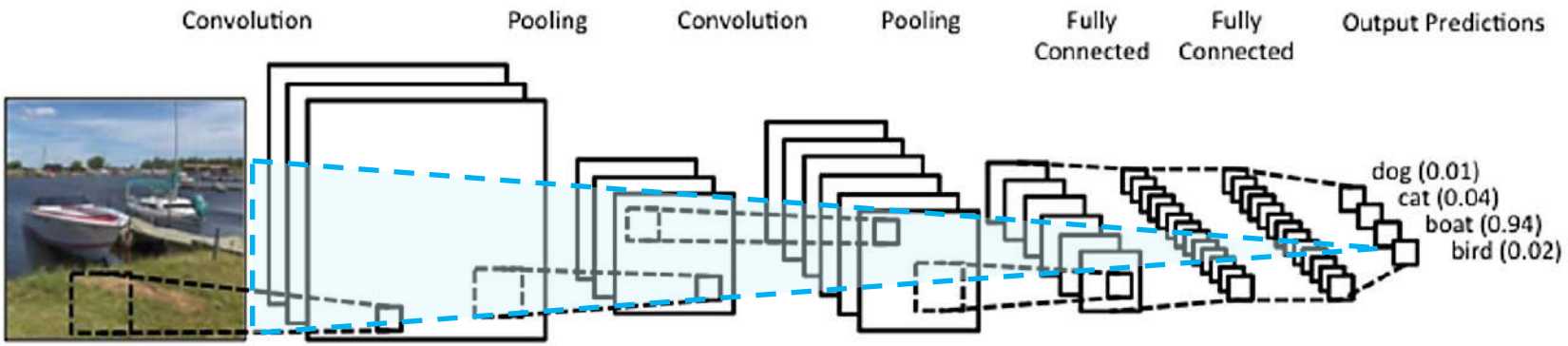
Note: the number of filters or units (i.e. the features maps) can be increased in higher layers

Second example

Dip. Informatica
University of Pisa

- Same number of feature maps after pooling

1. **local connections**
2. **shared weights**
3. **pooling**
4. **many layers**



Cone from the image to final maps

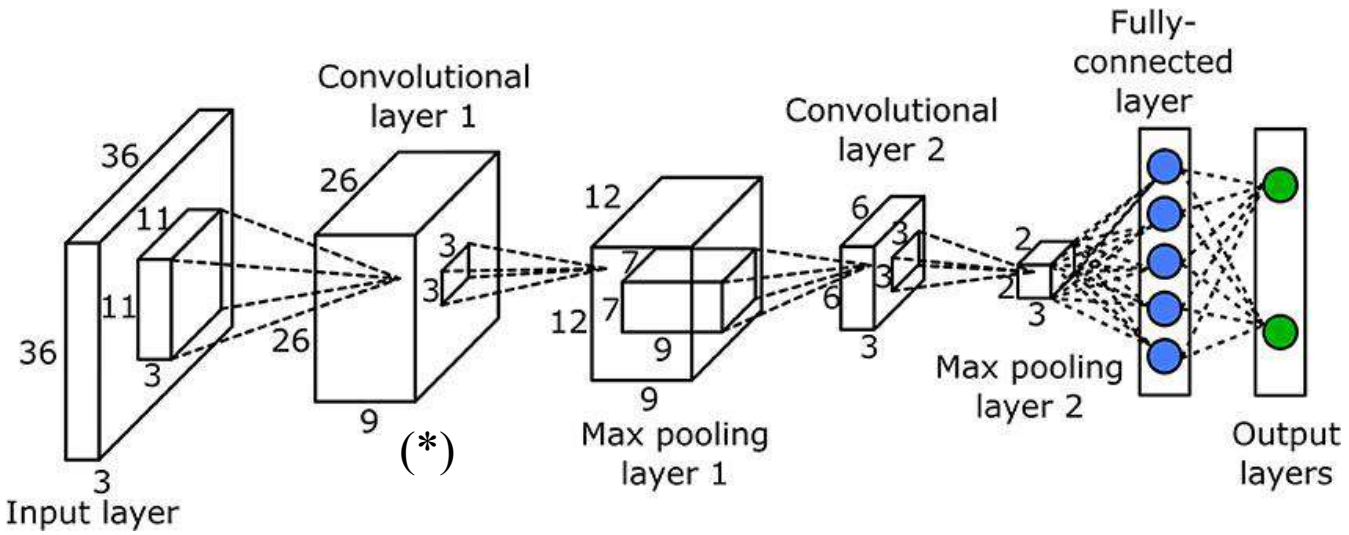
(the receptive field of final units is larger than the receptive field of the units in the first layers (we reduced the feature maps many times); they are indirectly connected to large part of the image)

Note: the number of filters or units (i.e. the feature maps) can be increased in higher layers

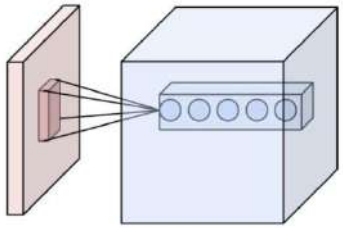
Third example

Dip. Informatica
University of Pisa

- To be precise (example with numbers and a different view)



Color image
RGB

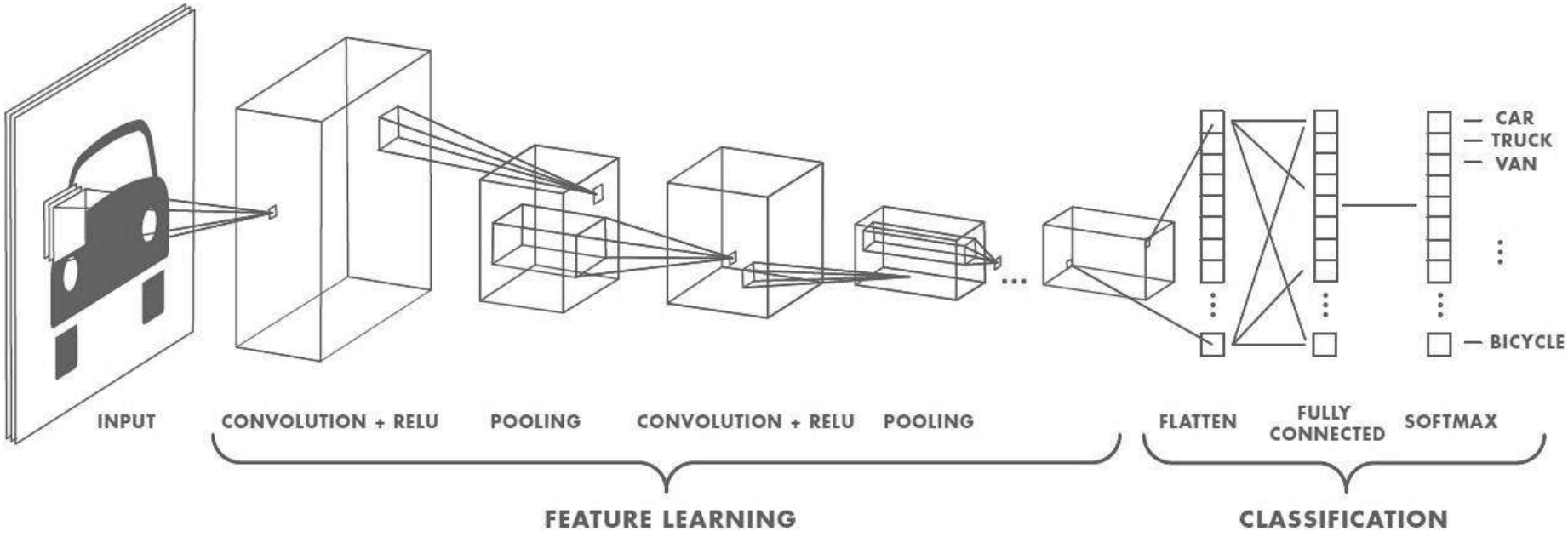


Depth of this volume = number of neurons (features maps)

4-th example

Dip. Informatica
University of Pisa

- ~ AlexNet



Summary of advantages

Dip. Informatica
University of Pisa

1. **local connections**
2. **shared weights**
3. **pooling**
4. **many layers**

- Local connections and weight-sharing:
  1. Detect local motifs/patterns in images
  2. and invariant to translation (invariant to location of the pattern)
  - Reduce the number of free-parameters (keeping the connections): regularization!
- Pooling (merge role)
  3. Reduce the dimension of the representation (in each layer → pyramid of layers)
  - Help the invariance to small shift and distortions (that does not have effect of the pool result, e.g. max progress just the hits of a pattern)
- A historical instance of **deep** neural network
  4. 1 & 2 applied to all the features maps*: progressive abstracting of features from images
  - Making possible the composition of primitives (low-level features, learned one time) for all the possible combinations in the successive layers (see the next DL lectures)
- Visual system (neuroscience) analogies have been studied

How to use?

Dip. Informatica
University of Pisa

- Training is typically made by *back-propagation*, with the many heuristics that you know (and some specialization of course, e.g. for weight-sharing, the pooling etc.)
- Give the typical usage of **huge networks** and large amount of data, many hyper-parameters are fixed by experience (which unfortunately is not your case yet) or by experts suggestions (!)
  - As it would be very expensive to run cross-validation on large hyper-par. search spaces

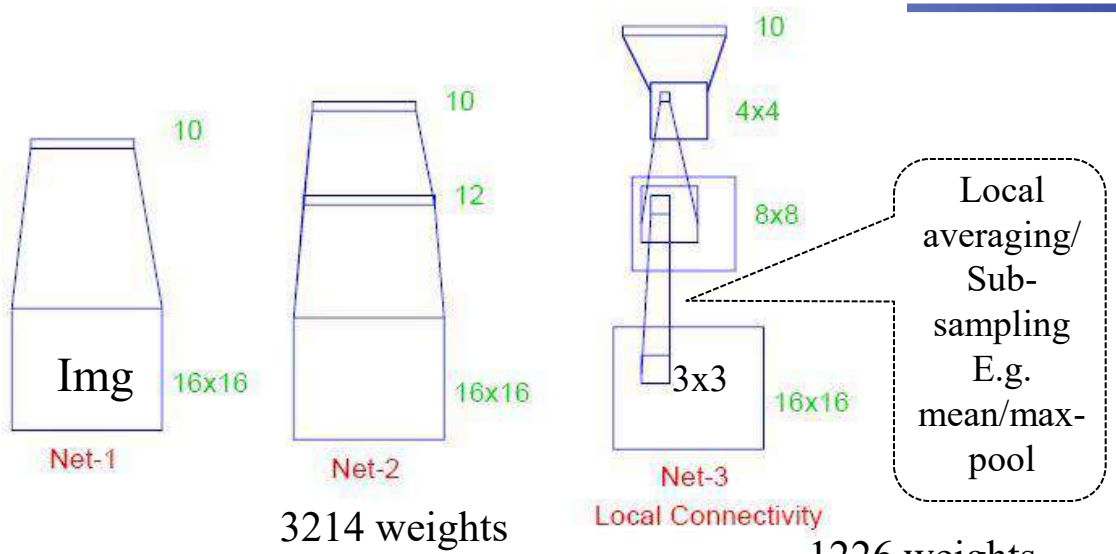
- Discussion on
  - the issues of training and
  - on the regularization effects and
  - good behavior of such huge networks, trained with large amount of data,

  will be discussed later (within deep network framework)

LeCun architectures
also MNIST benchmark

Dip. Informatica
University of Pisa

No hidden layer

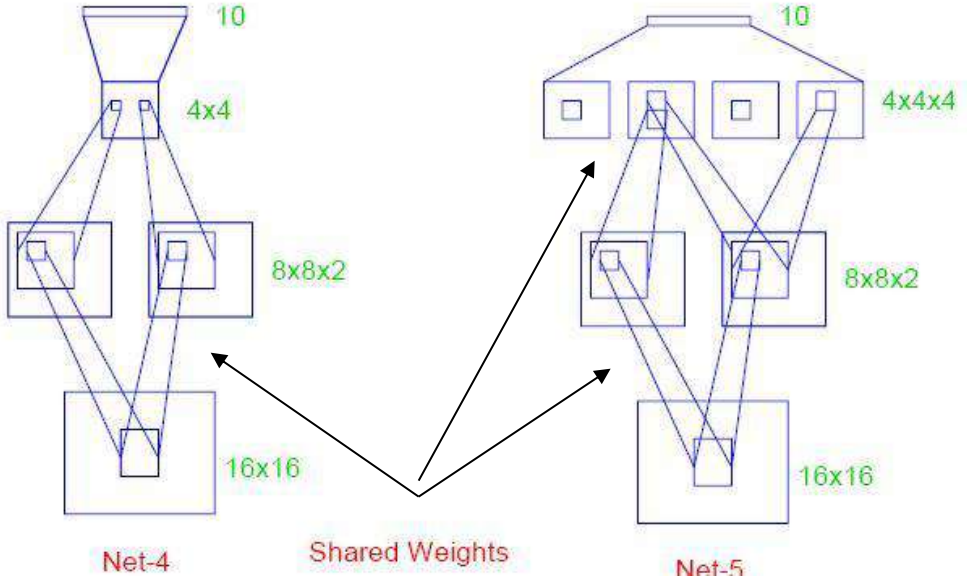


3214 weights

1226 weights

Architecture of the five networks used in the ZIP code example.

© Hastie, Tibshirani & Friedman 2009



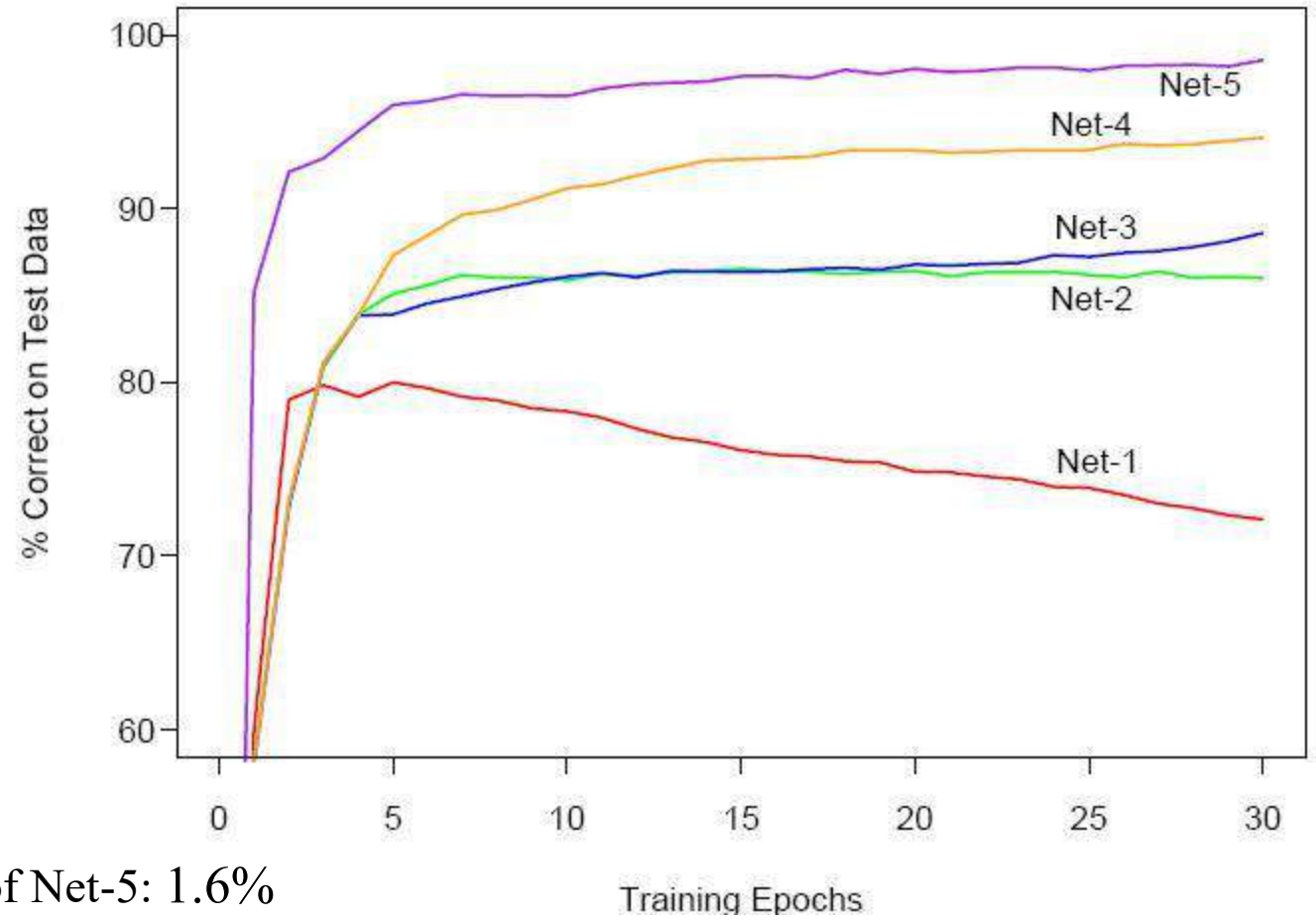
Net	Conn.	Weights
1	2570	2570
2	3214	3214
3	1226	1226
4	2266	1132
5	5194	1060

Test Performances curves

Dip. Informatica
University of Pisa

Test performance curves, as a function of the number of training epochs, for the five networks of Table 11.1 applied to the ZIP code data (Le Cun, 1989)

Note: TR error zero for all of them over a reduced set of images (~300)



Misclassification rate of Net-5: 1.6%
With 5194 connections
1060 weights

Nowadays: up to 0.8% (NN and SVM) and even less…

MNIST Handwritten Digit Recognition Benchmark

Dip. Informatica
University of Pisa

- Full data set: 60000 images for training 10000 for test.
- Progress (1998-2012) of the results at http://yann.lecun.com/exdb/mnist/
- OR https://en.wikipedia.org/wiki/MNIST_database

- Progress by Deep Learning:
  - E.g. MLP or CNN with many layers
  - Recent methods pre-train each hidden CNN layer one by

  one in an unsupervised fashion then use supervised learning to achieve **0.39%** (2006)/**0.35%** (2011)

- Best: (2012) **0.23%** error (committee of conv. networks) (2016): **0.21%**, ….Nowadays?
- But (warning!) too many uses of the test data to be still significant (especially for improvements of 0.1% or similar)

Deep MLP with Backprop on MNIST via GPU

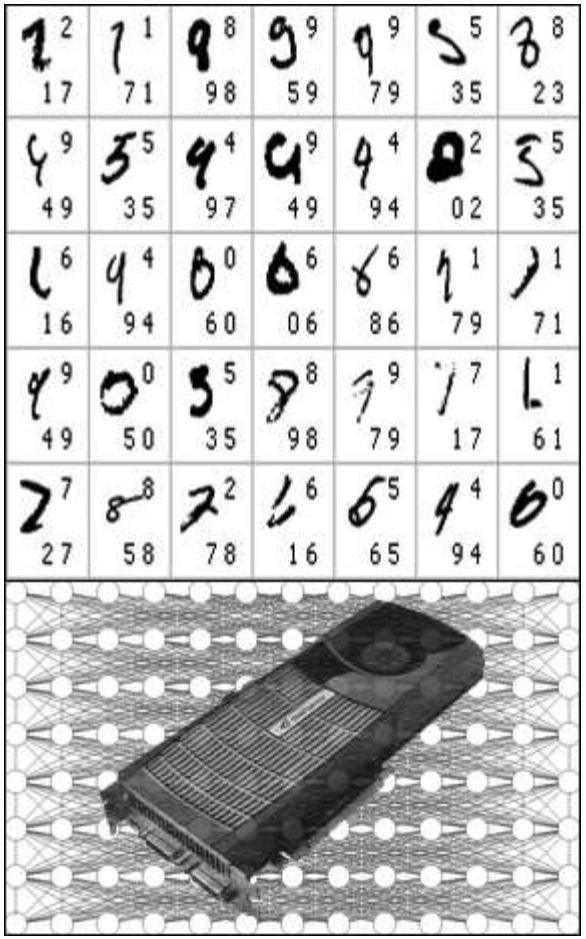
Dip. Informatica
University of Pisa

But is it really true that deep BP-MLPs do not work at all (gradient vanish, training time etc.), or do they just need more training time?

Paper in Neural Computation, 2010
http://arxiv.org/abs/1003.0358:

*"Good old on-line back-propagation for plain MLP yields a very low **0.35%** error rate on the famous MNIST .... All we need to achieve this best result (as of 2010) are many hidden layers, many neurons per layer, numerous deformed training images, and graphics cards to greatly speed up learning".*

- 6-layer NN 784-2500-2000-1500-1000-500-10 (on GPU): 12 million weights !!! With 2 hours of training.
- Complexity control issue: deforming images to get much more training instances (*)

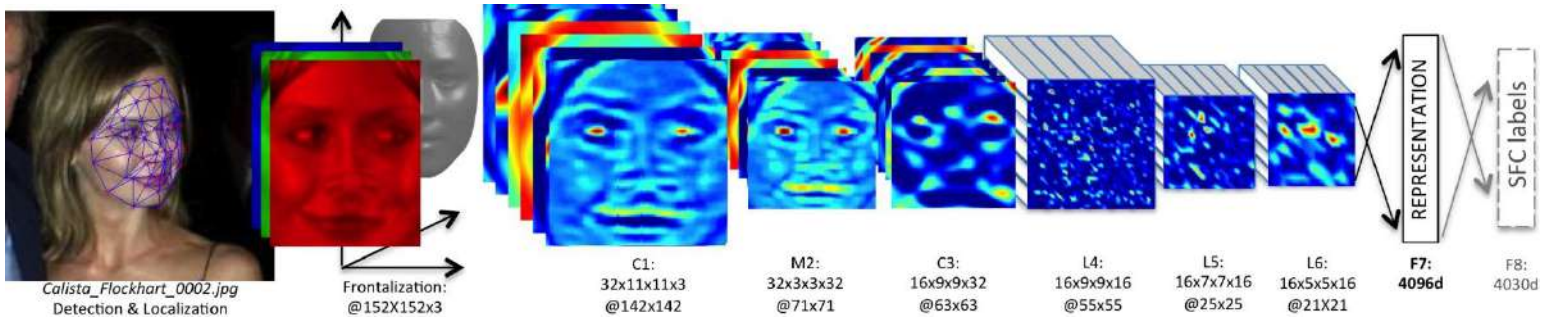


Large MLP and GPU

An instance on a "no-more recent" result (CVPR, 2014)

Dip. Informatica
University of Pisa

- **Face recognition** combining (**deep**) **Neural Networks** and other ML approaches. Note: *Convolutional networks* !!! 9 layers → deep.
- Starting from four million facial images belonging to more than 4,000 identities



- Asked whether two photos show the same person, DeepFace answers correctly 97.25% of the time … just a shade behind humans (97.53%).
- From a news on newspaper LaRepubblica April14 ;-)

Modern CNN

Dip. Informatica
University of Pisa

- Many variants and specialized architecture/models for processing of images → "Intelligent Systems for Pattern Recognition" course
  - High impact in Pattern Recognition and Computer Vision
- Neuroscientific basis of CNN → "CNS" course
- Efficient implementations through tensor representation of matrixes, exploiting GPU (many ML library for DL)

- Even using the features by a models already trained with large benchmarks e.g. from ***Alexnet*** (winner of the ImageNet object recognition challenge) [Krizhevsky *et al.* NIPS 2012]
- E.g. in Matlab. Also by matconvnet toolbox: googLeNet , vgg, e ResNet *.
- Many pre-trained CNNs for image classification, segmentation, face recognition, and text detection are available.

- In the following: ImageNet examples + tensors

ImageNet Example

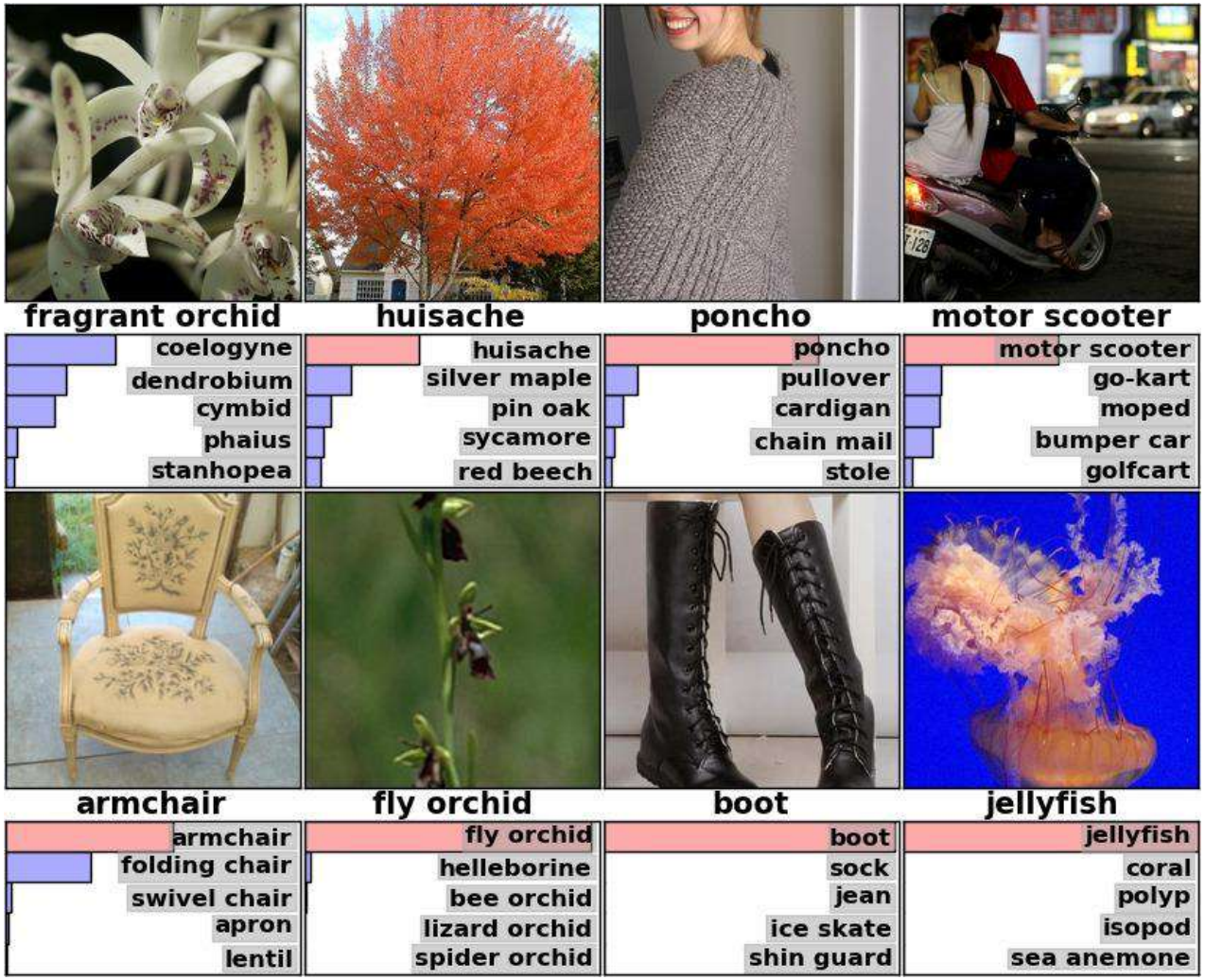
Dip. Informatica
University of Pisa

- Krizhevsky, Sutskever*, Hinton**, *ImageNet Classification with Deep Convolutional Neural Networks*, NIPS 2012
- We trained a large, deep convolutional neural network to classify the **1.3 million** high-resolution images in the ILSVRC-2010 ImageNet training set into the 1000 different classes.
- On the test data, we achieved top-1 and top-5 error rates of 39.7% and (from ~26% to) 18.9% which is considerably *better than the previous state-of-the-art results*.
- The neural network, which has **60 million parameters and 500,000 neurons**, consists of **five convolutional layers**, some of which are followed by max-pooling layers, and two globally connected layers with a final 1000-way softmax.
- To make training faster, we used *non-saturating* neurons and a very efficient **GPU implementation** of convolutional nets.
- To reduce overfitting in the globally connected layers we employed a new **regularization** method that proved to be very effective (*dropout*).

* Sutskever: Chief Scientist of OpenAI
** Hinton: Turing award 2019

Results demo

Dip. Informatica
University of Pisa



Top-5 predictions for random images from the test set

How to parallelize linear operations on GPU





Dip. Informatica
University of Pisa

Matrix multiplication (i.e., dot product) is a typical linear operation performed efficiently on GPU

$$(\mathbf{AB})_{ij} = \sum_{k=1}^{m} A_{ik} B_{kj}$$

Parallelizing the sum
or a set of independent products (for each *i,j*)

$$A_{ik} B_{kj}$$

$\mathbf{A}$ Input matrix

Many kernel operations

$$\mathbf{W_1}, \mathbf{W_2}, \ldots, \mathbf{W_5}$$

We can build **by hand** a block matrix in order to perform many kernel operations as a matrix multiplication on GPU

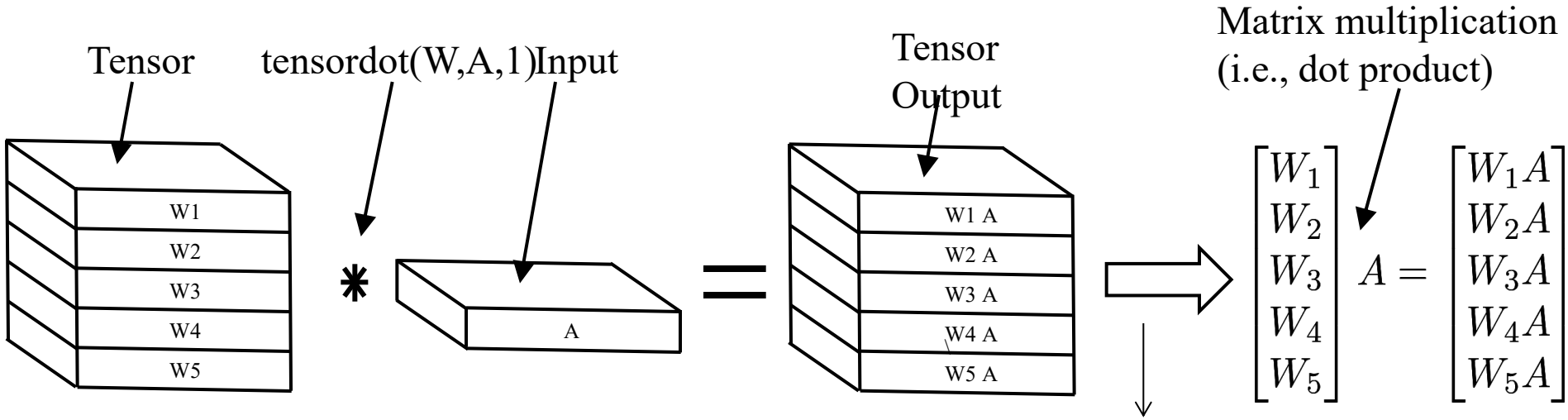
$$\begin{bmatrix} W_1 \\ W_2 \\ W_3 \\ W_4 \\ W_5 \end{bmatrix} A = \begin{bmatrix} W_1 A \\ W_2 A \\ W_3 A \\ W_4 A \\ W_5 A \end{bmatrix}$$

Tensor Notation for GPU

Dip. Informatica
University of Pisa

In computer programming a **Tensor** is a multi-dimensional matrix with a linear operation called **tensordot**

Solution!: In **Tensorflow,** or other libraries (e.g. NumPy), we can perform many linear operations (i.e., dot products) as a **tensordot** between **Tensors**



Why use Tensor notation in Machine Learning?

The use of **Tensor Notation** allows to parallelize many linear operations (i.,e., matrix multiplication) between **tensors** on GPUs without construct complex block matrices by hand

"Correspond to"
but more useful
if written as *

Beyond CNN: an example



Dip. Informatica
University of Pisa

Capsule (Neural) Networks, [CapsNet] November 2017

- S. Sabour, N. Frosst, G. E Hinton, Dynamic Routing Between Capsules https://arxiv.org/abs/1710.09829

- Using specialize subnetworks for image processing
  - A "capsule" is a subset of neurons within a layer that outputs both an "instantiation parameter" indicating whether an entity is present within a limited domain and a vector of "pose parameters" specifying the pose of the entity relative to a canonical version. [position and orientation]
  - *State-of-the-art** performance on MNIST and is considerably better than a convolutional net at recognizing highly overlapping digits.
  - Most interestingly, without transforming image preprocessing
- Routing mechanism (instead of Max-pooling)
  - The brain must have a mechanism for "routing" low level visual information to what it believes is the best capsule for handling it.

Other architectures (guide slide)

Dip. Informatica
University of Pisa

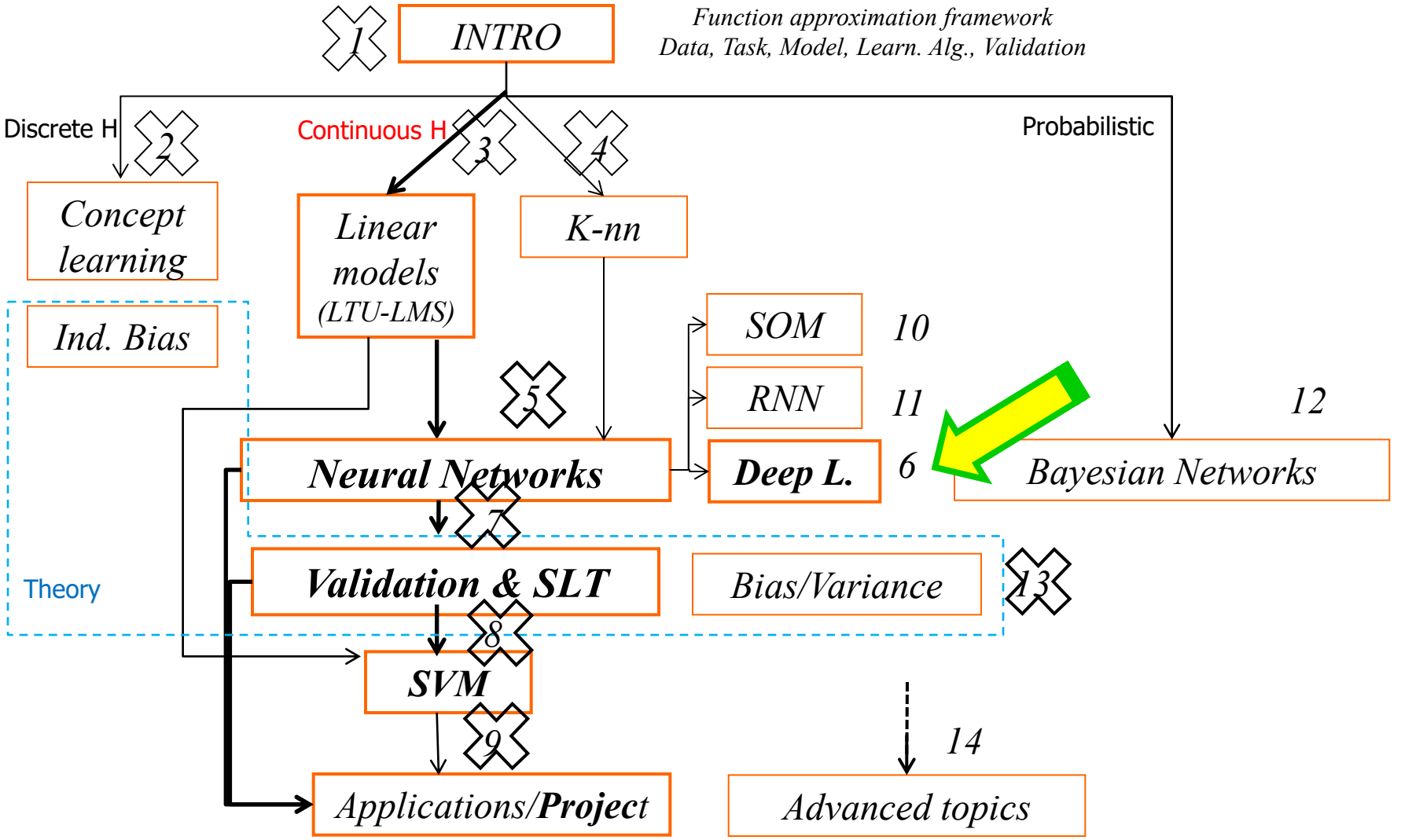
Introduction to recent NN paradigms

- Next lectures: Other architectures…..
  - Deep learning
  - Random weights NN

AKA: *Deep* learning and *extreme* learning ;-)

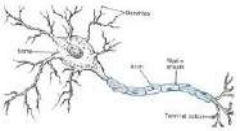
ML Course structure
Where we go

Dip. Informatica
University of Pisa



Course structure: zoom in the NNs box

Dip. Informatica
University of Pisa



1 unit NN

Perceptron

Adaline

LMS

As a linear model (previous lects)

Neural Networks

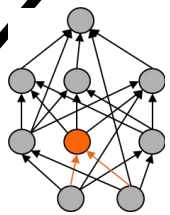
MLP: Mack. &Pits

LEARNING: Perceptron learning alg + Convergence theorem

Non-linear out (f)

LEARNING: Backprop

MLP: feed-forward NN

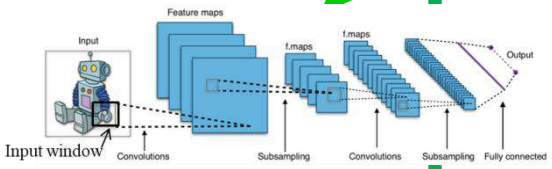


Heuristics for BP

CasCor

Regularization

We are here



Applications: **CNN** example

Introduction to recent NN paradigms (**Deep**, Random.)

Made later, after the PRJ (to favorite validation, SVM, …)

Initial messages: Aims in the ML course

Dip. Informatica
University of Pisa

1. Aims for DL in this course and … overlapping with other courses?
   - (also valid for CNN, RNN, etc)
- In this course the perspective is to see different NN **architectures** (which is a topic part of the ML course)
  - AI curriculum: In the following courses you will study details, variants aimed at applications to pattern recognition, NLP, etc.
- Moreover, we need this introduction to recent paradigms for the NN models (we will see that many aspects and techniques are useful for the basic setting)
- And, also, not all the ML attendants can follow the other courses.

2. Send to me questions for the next Q&A! (tag: [ML-25-Q])

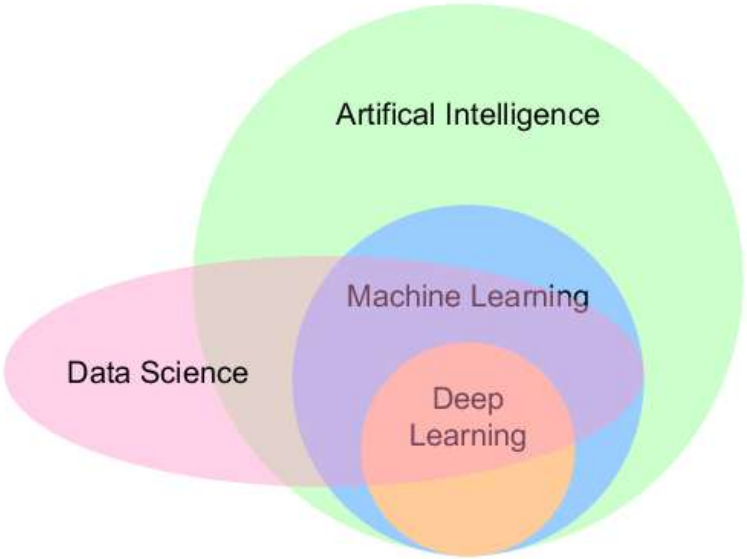
Other architectures (guide slide)

Dip. Informatica
University of Pisa

Introduction to recent NN paradigms

- Other architectures.....
  - Deep learning
  - Random weights NN

AKA: *Deep* learning
and *extreme* learning ;-)



Not all the story but nice view at a glance

Deep Learning- guideline

Dip. Informatica
University of Pisa

- **Basic (or rapid introduction)**
- Insights (do we need many layers? + Representation learning and distributed representation)
- Techniques (some)
- Applications → further courses in AI (PR, NLP, …)

Deep Learning quotation

Dip. Informatica
University of Pisa

REF for overiview: DL book Chapter 1

- "*Deep learning allows computational models that are composed of multiple processing layers to learn representations of data with multiple levels of abstraction.*
These methods have dramatically improved the state-of-the-art in speech recognition, visual object recognition, object detection and many other domains"

LeCun, Y., Bengio, Y., & Hinton, G. (May 2015). *Deep learning*. Nature, *521*(7553), 436-444.

Deep Learning: *short history*

Dip. Informatica
University of Pisa

- Starting from 2010 there was an impact of **deep learning** in industry: The use of deep architectures provided new performance results and new insights for new NN approaches
  - Large data sets are nowadays available (e.g. millions of images)
  - Powerful computing system (HPC) are nowadays available (enabling factor, e.g. GPU)
  - New ideas to easy the training (see later)

➔ Learning in deep architectures it is no longer "unfeasible"

- New successful state of the art results for complex task and competitions in *speech recognition* , *machine translation** and *image recognition* tasks attracted many practitioners and research to deep learning approaches.
- (In the past it was expected an improvement changing the scale of models and data, but not necessarily such *jump* of performance)
- All the big CS companies are basing their RD strategies on such approaches

* e.g. *google (neural) translator since summer 2016, DeepL, etc.*

Deep Learning framework

Dip. Informatica
University of Pisa

- It is a general framework including different models, such are:
  - deep Neural Networks (aka a MLP with many layers)
  - *convolutional* NNs→ successful for computer vison
  - deep belief networks and generative approaches
  - recurrent and recursive NNs → *active research in Unipi-CIML*
  - …
- By contrast: "Shallow models" (e.g. a NN with few layers, say less than 3)

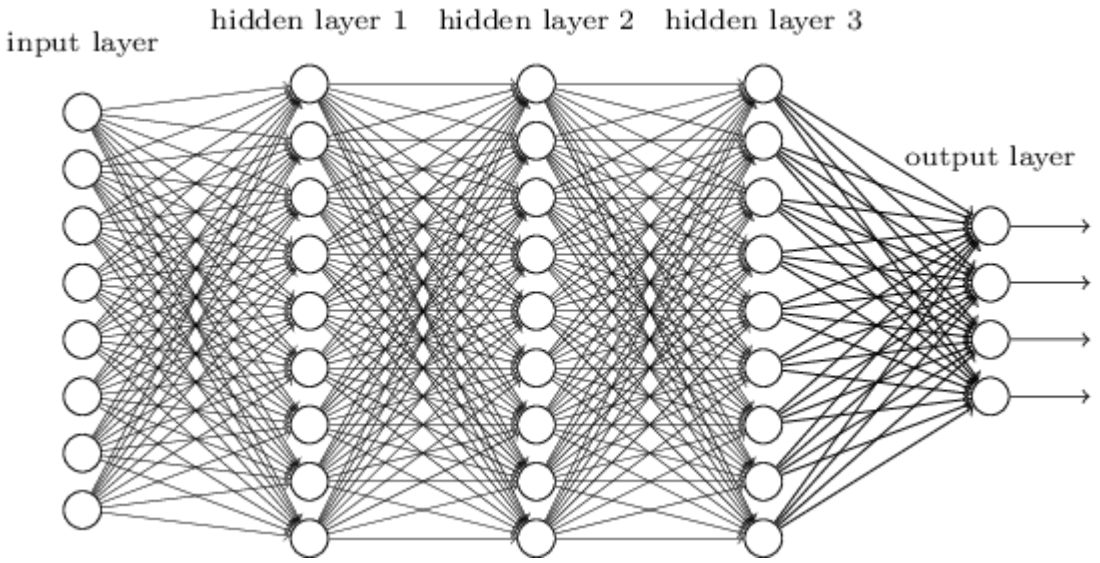
- And give to us the opportunity to focus on some relevant concepts, not discussed enough up to now e.g.
  - Representation learning
  - Distributed representation
  - Other techniques

How to build it?

Dip. Informatica
University of Pisa

- Many approaches to implement deep learning

- We will introduce approaches well rooted in your background through this course
- Basically by building a MLP with many layers and e.g. pre-training techniques



A more general view

Dip. Informatica
University of Pisa

Common aspects among different approaches:

- Multiple layers of nonlinear processing units
- Supervised or unsupervised learning of feature representations in each layer, with the layers forming a hierarchy from low-level to high-level features (different levels of abstraction).
- Hence, hierarchical sparse/distributed representation of the input data

- (differences according to the specific models, e.g. max pool operators for CNN, energy based training for RBM - Restricted Boltzmann Machines , etc.)

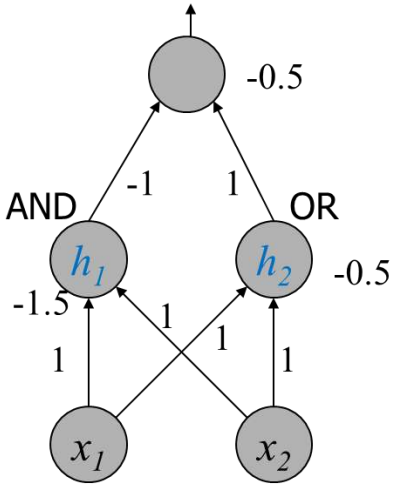
REPETITA:
XOR by a Two Layers Network

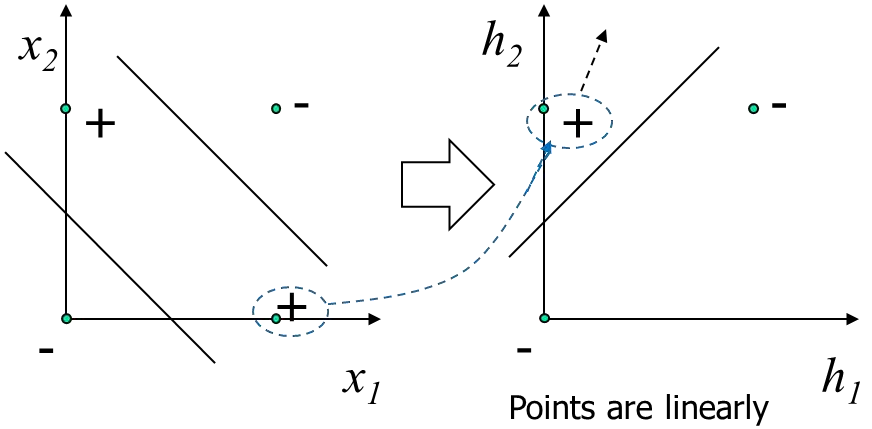
Dip. Informatica
University of Pisa

$x_1 \oplus x_2 = x_1 \cdot \overline{x_2} + \overline{x_1} \cdot x_2$ let : $h_1 = x_1 \cdot x_2$ (dot = **and**) (+ = **or**)

$h_2 = x_1 + x_2$

then we have : $x_1 \oplus x_2 = \overline{h_1} \cdot h_2$





E.g. (1,0) becames $h_1=0$ $h_2=1$

Points are linearly separable in the *new space!*

- Concept of internal **(re)representation** of input variables via internal (hidden) units,
- developing **high level (hidden) features,** is a key factor in NN
- The representation in the hidden layer makes easier the task to the output layer (last unit)
- Such **composition** of sub-/intermediate operations to perform a complex task can be extended through many layers of **abstraction**
- Learning internal distributed representation of the data

Conceptual ingredients: hierarchical abstraction

Dip. Informatica
University of Pisa

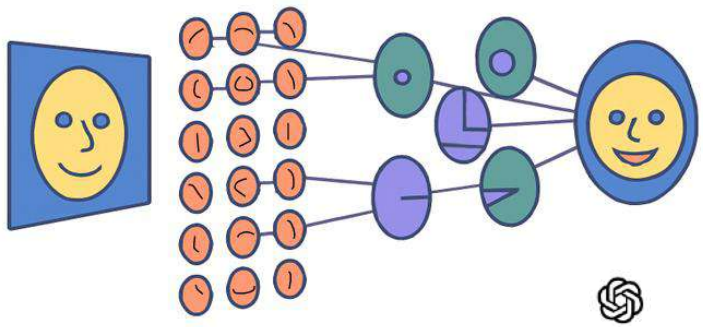
- Unsupervised or semi-supervised feature learning and *hierarchical feature* extraction instead of features engineering (extracted by experience, e.g. the texture from images).

Move to the concept of "**learning representations of data**"

- Increase level of **abstraction through different layers**

Example: An observation (e.g., an image) can be represented in many ways such as a vector of intensity values per pixel, or in a more abstract way as a set of edges, regions of particular shape, etc..

  - Edges form motifs, motifs assemble into parts, parts form objects…. *Many levels*.



- Performs best when input information has some form of structure, e.g. spatial, temporal, ...

An example (with CNN) I

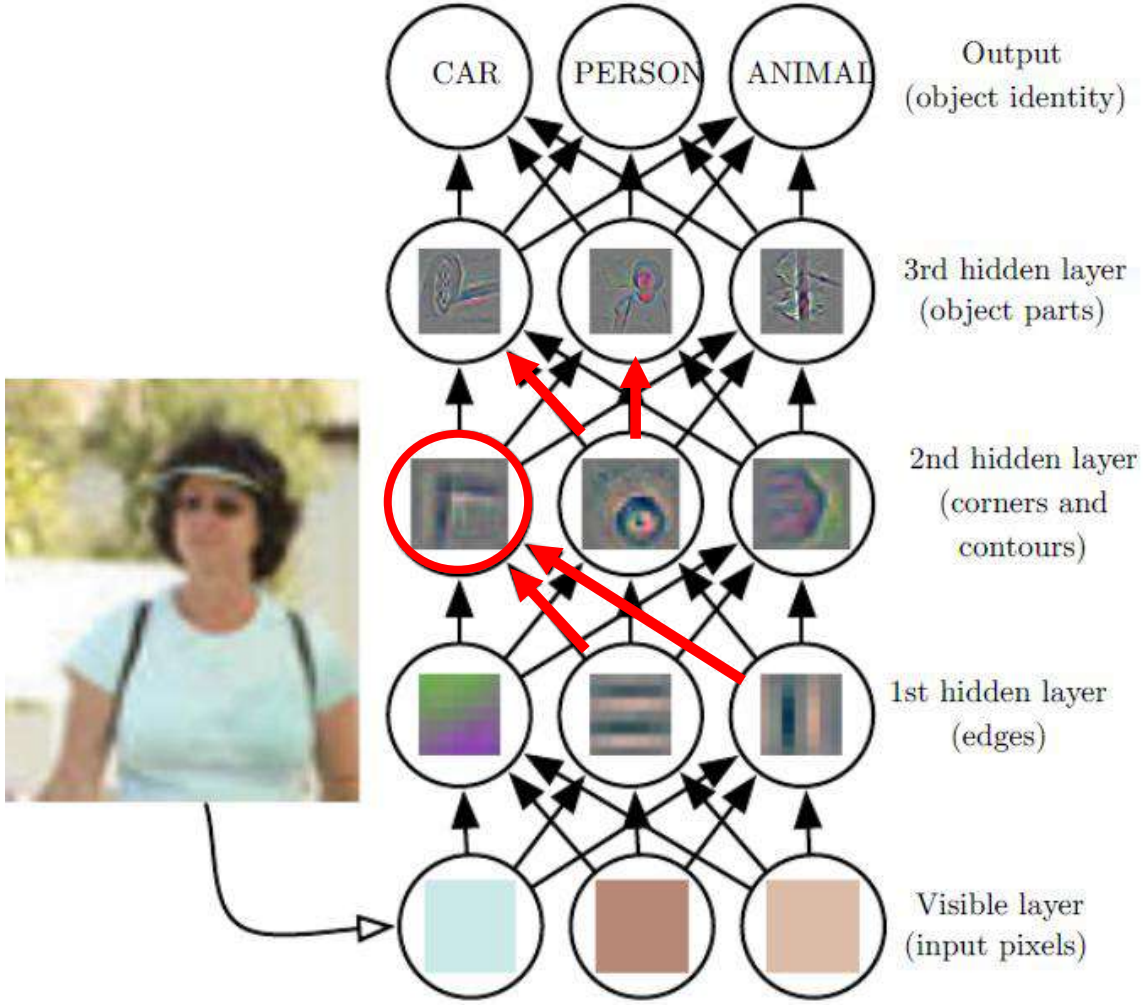
Dip. Informatica
University of Pisa

From pixels to object identity:
Deep learning resolves this difficulty by breaking the desired complicated mapping into a series of **nested simple** mappings, each described by a different layer of the model.
A series of ***Hidden layers*** **extracts increasingly abstract features from the image.**

Note: The new representation simplify the classification task at the last layer.

Biological visual system analogies

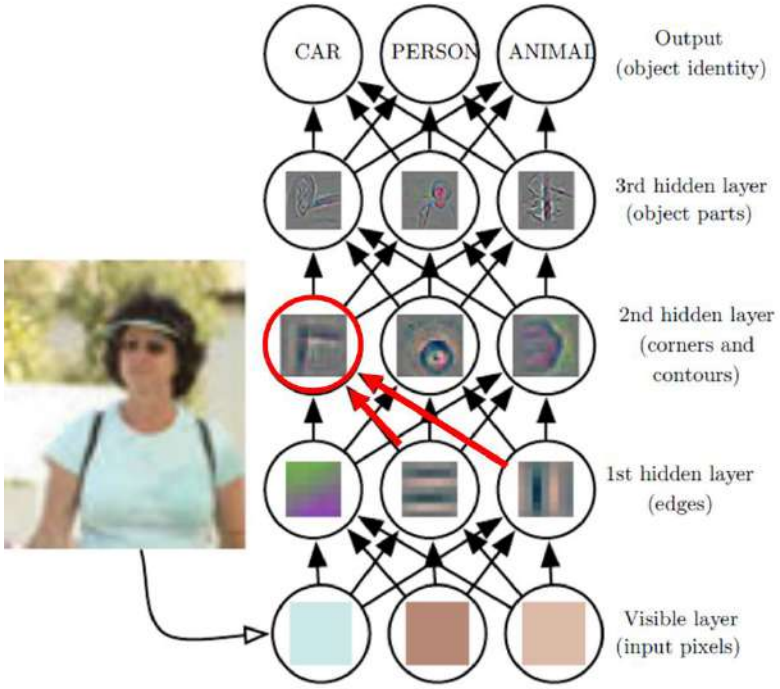
The images here are visualizations of the kind of feature (pattern) represented by each hidden unit



An example (with CNN) II

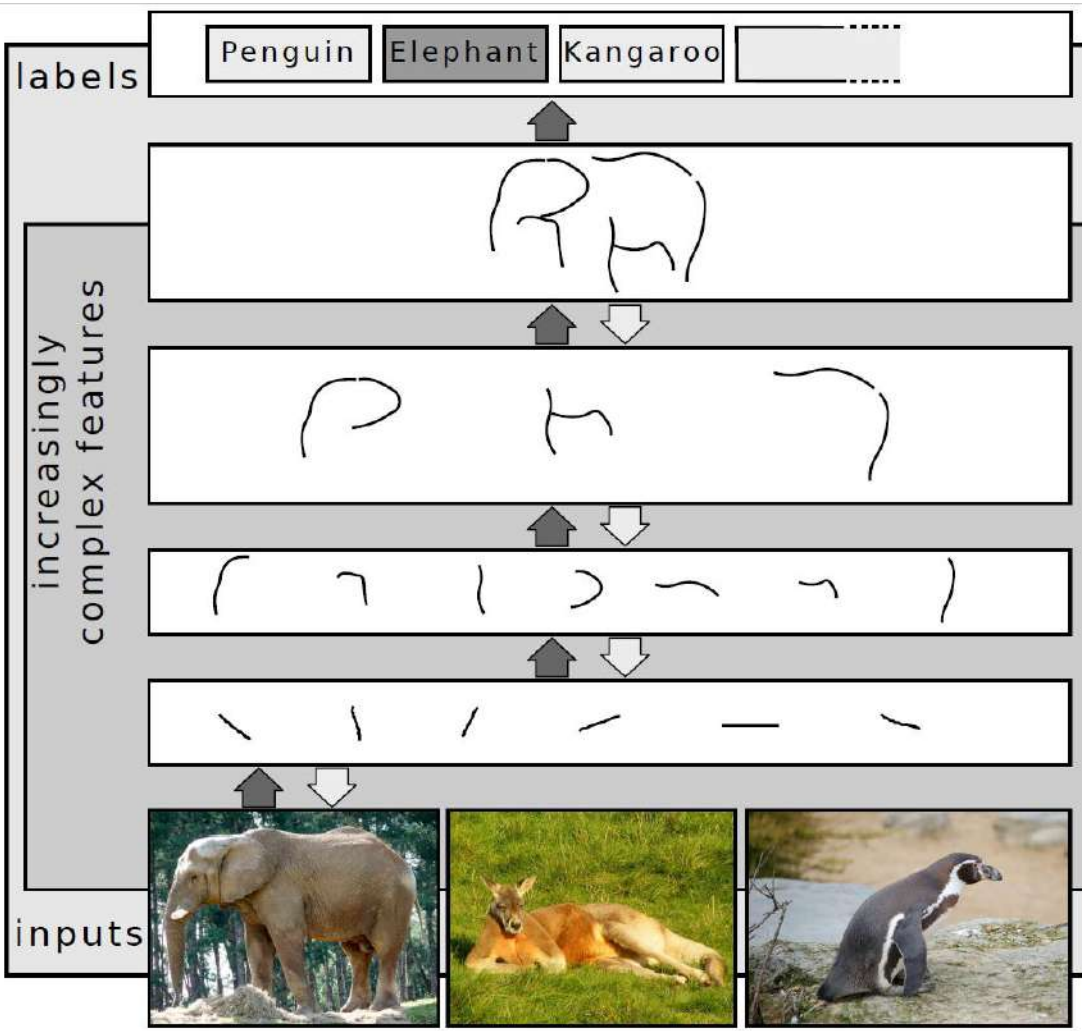
Dip. Informatica
University of Pisa

- In NN these *abstract features* are **learned** from examples!
  - ***Back propagation*** → the model learns how to represent what is need to solve/help at the last layer developing a *suitable internal representation*:
    Error at the output layer to better discriminate cars and persons → *delta* through the layers, the ***units will specialize*** accordingly to provide high level features for this, e.g. the wheel for the car, a hand for the persons, and so on back …
  - The relationship between the automatically developed features and the reality (the inductive bias w.r.t the nature of the language or images etc) is a ***fascinating aspect*** (we will discuss it later)
  - However, the **first filters** extract linear or horizontal lines as a simple *convolutive linear filter* can do. Usurpingly we can do these as a first step.



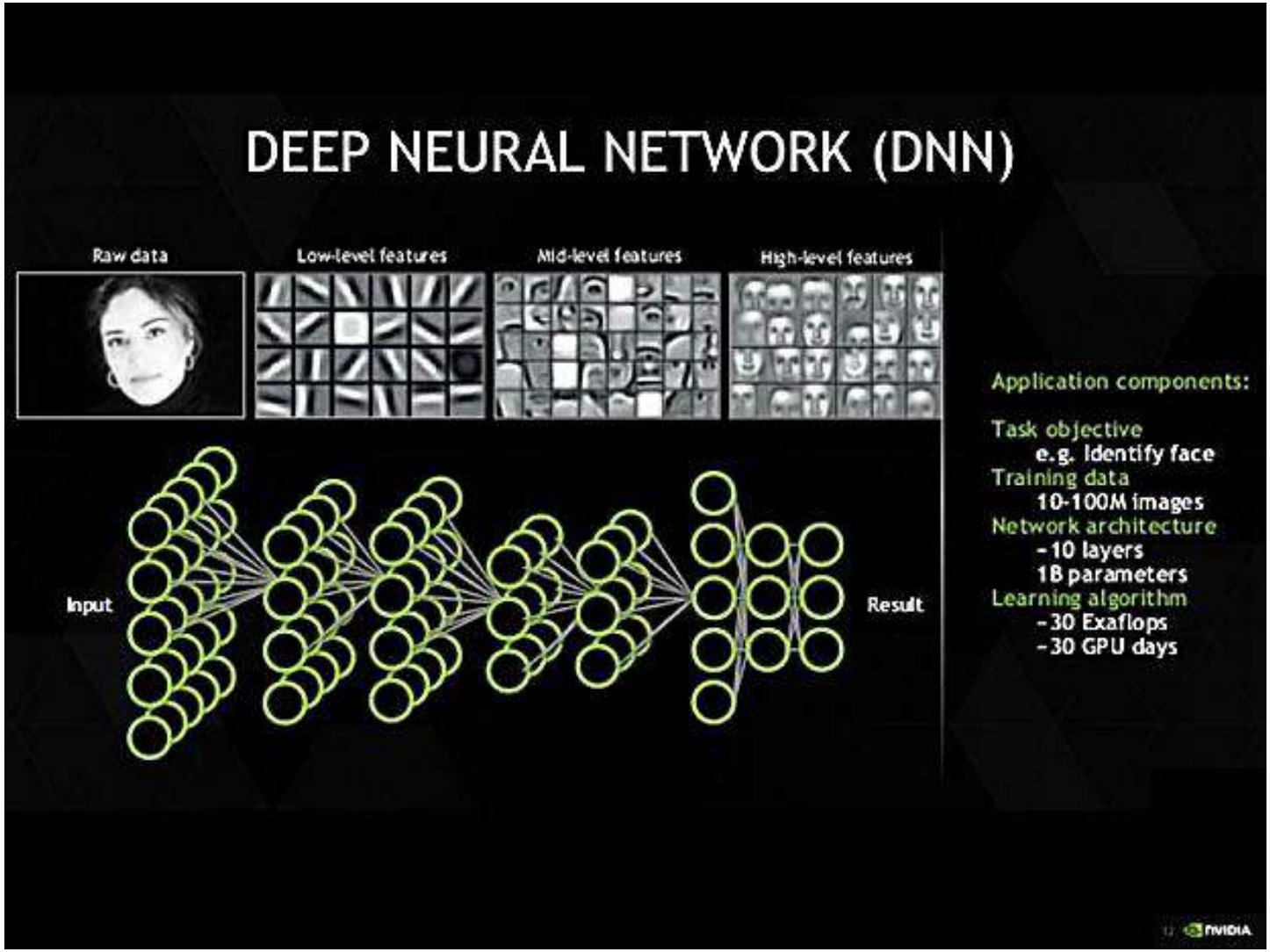
Another example

Dip. Informatica
University of Pisa



Another example

Dip. Informatica
University of Pisa



Combine to generalize (Img)

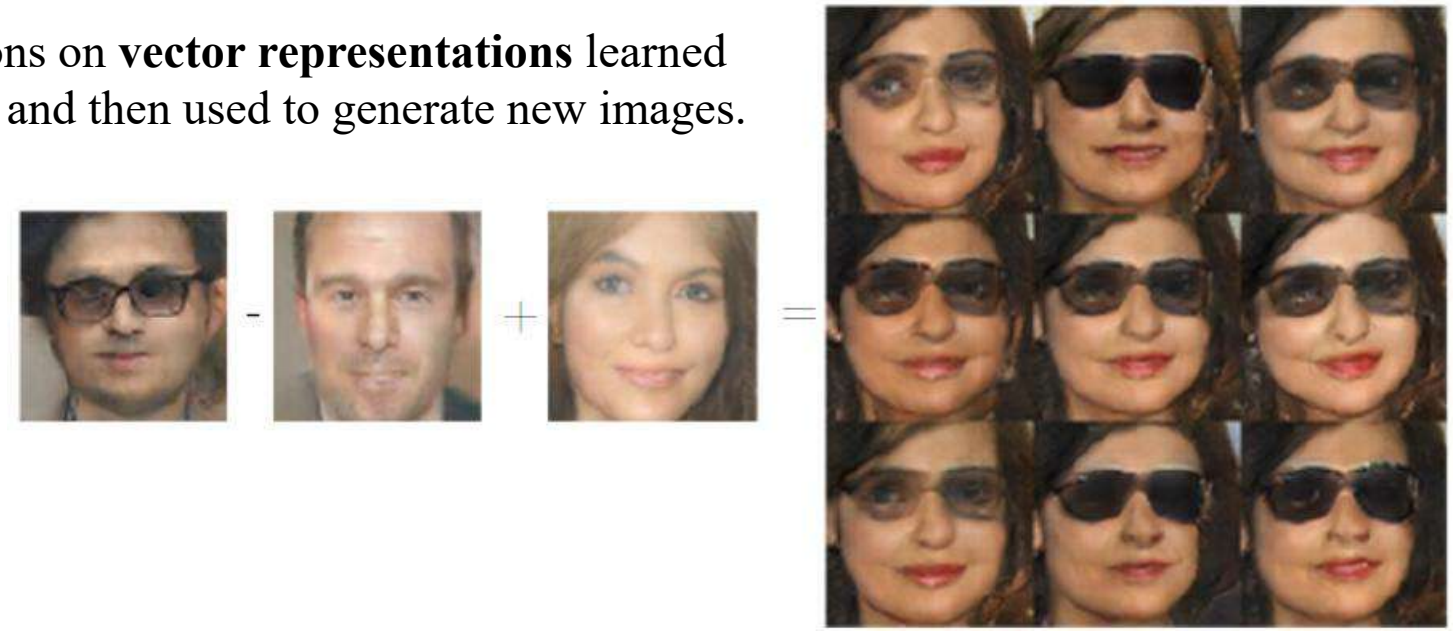
Dip. Informatica
University of Pisa

- And the abstract features can be "reused" at the higher levels
  - E.g. gender/ glasses/ ….
- And combine them to generalize to unseen (during training) cases:
  - Woman without glasses
  - Man with glass

→ Woman with glasses can be easily represented as well.

- See the "distributed representation" section in the following

Operations on **vector representations** learned by model and then used to generate new images.

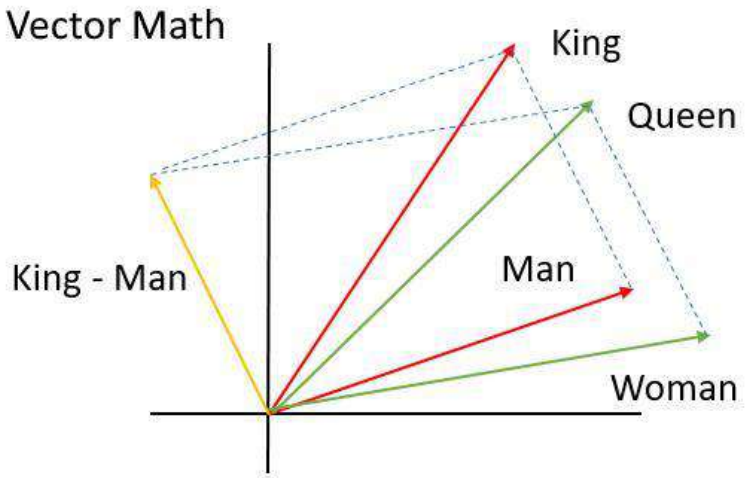


Combine to generalize (NLP)

Dip. Informatica
University of Pisa

Representation (Rep) in the vector space developed by the Deep model (inner layers)

- Rep(king) - Rep(male) + Rep (female) = Rep(queen) [gender relationship]
  - In this case, the vector difference between *king* and *male* captures the concept of monarchy
- Rep(Paris) – Rep(France) + Rep(Poland) = Rep(Warsaw) [concept of capitol]
  - In this case, the vector difference between *Paris* and *France* captures the concept of capital city



Techniques: why? [preview]

Dip. Informatica
University of Pisa

In general, for deep learning:

- Deeper networks often are able to use less units in each layer → less parameters → Less training data to achieve a good generalization
- *But many layers may be harder to be optimized* (trained), hence:

- Emphasis on methods to improve
  - *Gradient descent* (also gradient vanish trough layers, …)
  - *Regularization* (due to large networks) (e.g. CNN strong architectural by w. sharing)
  - *Better exploitations of data* (e.g. semi-supervised learning, reinforcement learning cycles, multi-task learning, generative adversarial learning etc.)

…. These deserve to be presented (some of them) in the following

Also, but already discussed before:

- Larger datasets from industry and real applications (e.g. millions of images)
- Software infrastructures (also public available)
- HPC improvements (GPU and over)



Deep Learning- guideline

Dip. Informatica
University of Pisa

- Basic (or rapid introduction)
- **Insights**
  - Do we need many layers?
  - Representation learning
  - Distributed representation
- Techniques (some)
- Applications → further courses in AI (PR, NLP, …)

Deep Learning- guideline

Dip. Informatica
University of Pisa

- Basic (or rapid introduction)
- **Insights**
  - **Do we need many layers?**
  - Representation learning
  - Distributed representation
- Techniques (some)
- Applications → further courses in AI (PR, NLP, …)

Example from logic networks (I)

Dip. Informatica
University of Pisa

General idea (**no-flattening**):

- *when a function can be compactly represented by a deep architecture,*
  it might need a very large architecture to be represented by an insufficiently deep one.

- We will start with an example from logic circuits
- And we generalize the concept using different points of view

Example from logic circuits (II)

Dip. Informatica
University of Pisa

- A two-layer circuit of logic gates can represent any Boolean function
- Any Boolean function can be written as a sum of products, i.e. a disjunctive normal form:
  - AND gates on the first layer with optional negation of inputs, and
  - OR gate on the second layer

- What is interest for us? Our point becomes:
  - With depth-two logical circuits, most Boolean functions require an *exponential* (with respect to input size) number of logic gates to be represented [*].
  - Let us see an example

[*] I. Wegener, *The Complexity of Boolean Functions*, 1987.

The problem example: the parity function

Dip. Informatica
University of Pisa

- An extension of the XOR problem to N input variables
- Aka **odd parity: return 1 iff** there are an odd number of 1 over N binary (0/1) inputs (bits b_i or variables x_i)
- Implement odd parity Boolean function by logic gates
- Remember the XOR

$$x_1 \oplus x_2 = x_1 \cdot \overline{x_2} + \overline{x_1} \cdot x_2$$

NOT

AND OR

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

- We assume to have also the NOT(x_i)
- With 2 input variables, 3 gates are sufficient

Solution with 2 layers

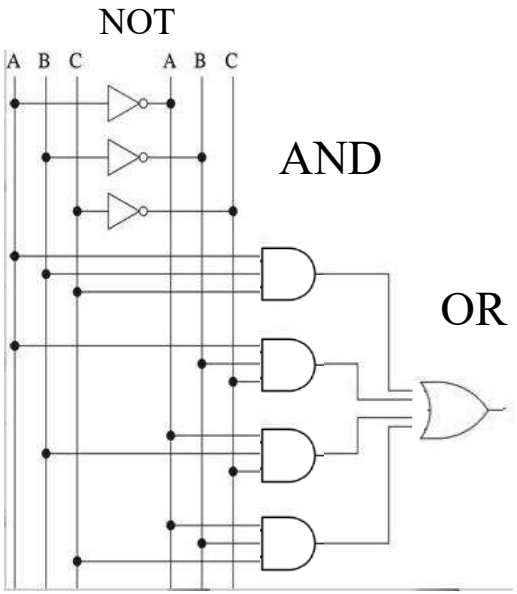
Dip. Informatica
University of Pisa

- For 3 inputs: 4 AND + 1 OR: **5 gates**

$$x_1x_2x_3 + x_1\overline{x_2x_3} + \overline{x_1}x_2\overline{x_3} + \overline{x_1x_2}x_3$$

1 iff inputs are 111 or 100 or 010 or 001

⇨ *All the combinations of the positive patterns*



- For 4 inputs: 8 AND + 1 OR: **9 gates**
- For 8 inputs: 128 AND + 1 OR: **129 gates**

- For N inputs: $2^N/2+1 = \mathbf{2^{N-1} + 1}$ **gates**
  An exponential number of gates !!!

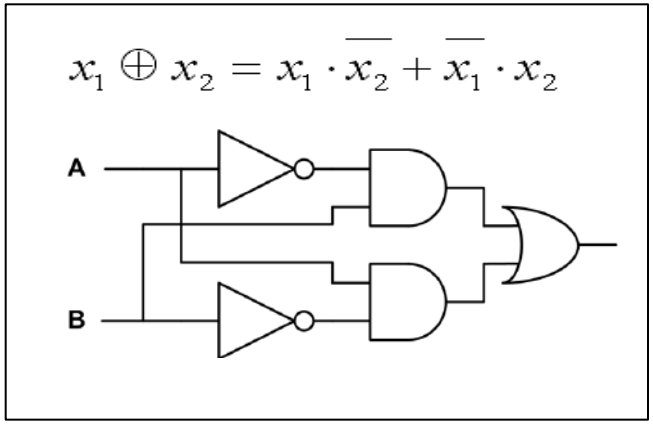
Solution with logN layers

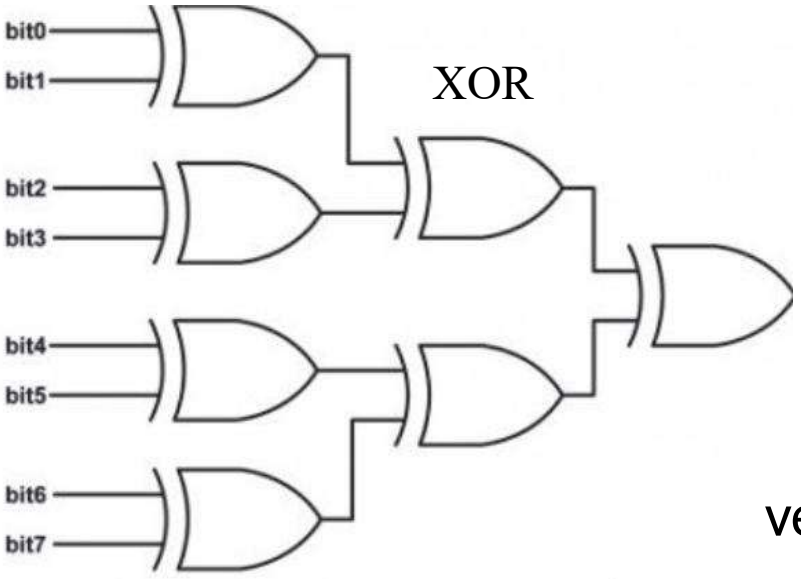
Dip. Informatica
University of Pisa

- Polynomial number of gates:
- complete tree* with N-1 (XOR) internal nodes
- 3 gates for each node: total 3(N-1) gates

* A complete binary tree with N leaves has N-1 internal nodes

XOR (with 3 AND/OR gates) ⟶





With more layers,
for 8 inputs only
3x7= **21 gates**
versus **129 gates** with 2 layers.

Example from logic circuits: Discussion

Dip. Informatica
University of Pisa

- However, this does not hold for all the classes of functions
- To define the class of the functions that benefits (with an exponential gain) from a multilayer structure is still an open research issue
- No-flattening theory (see later)

NN case [Repetita]: How many layers? Overview

Dip. Informatica
University of Pisa

- The *univ. approx. theorem* is a fundamental contribution
  - It shows that *1 hidden layer is sufficient* in general, but it does not assure that a "small number" of units could be sufficient (it does not provide a limit on such number)
  - Even if it is possible to find boundaries on such number (for many f. families)
- But is also possible to find "*no flattening*" results (**on efficiency, not expressivity**):
  - 1. cases for which the implementation by a single hidden layer would require an exponential number of units (w.r.t *n input dim.*), or exp. non-zero weights,
  - 2. while more layers can help (it can help for the number of units/weights and/or for learning (*) such approximation)
  - (But is it easy to optimize (training) a MLP with many layers?)

(*) Learnability: How rapidly can the neural network learn good parameters for approximating a function?

1. The case of shallow networks

Dip. Informatica
University of Pisa

REF: DL book Sec 6.4

- Cases with exp. number of units for 1-layer networks
- This is easiest to see in the binary case:
  the number of possible binary functions on vectors **v** in $\{0,1\}^n$ is 2^{2^n}
- This *may* require an exponential number of units
  - E.g. in the worst case, an exponential number of hidden units (possibly with one hidden unit corresponding to each input configuration that needs to be distinguished) may be required.
  - See the "parity" example

- Practical issue for the dimension of the network
- But also, it implies to become difficult **to learn** with few examples for complex tasks (this is more relevant for ML)

2. No flattening of deep networks

Dip. Informatica
University of Pisa

- There exist families of functions which can be approximated **efficiently** by an architecture with depth greater than some value *d*, but which require a much larger model if depth is restricted to be less than or equal to *d* (with up to an exponential gain)
  - Examples for logic gates (1986) → see also again the example at the beginning
  - Examples for NN with standard activation functions (1990-1994)
  - Examples with ReLU (2014)
  - …
  - …

Theoretical analysis

Dip. Informatica
University of Pisa

Can it be proved? (open theoretical research)

- No-flattening **theorems**:

  compositional functions than can be well implemented by a *deep NN* and that **cannot** be implemented retaining the same efficiency while flattening the NN (e.g. from linear to exponential number of units).

- But there is no guarantee that in general your task share such property

- A complete list of *no-flattening theorems* would show exactly when deep networks are more efficient than shallow networks

Why?

Dip. Informatica
University of Pisa

Why/when does it happen? Open line of research, e.g. essentially…..

- Deep models can exploit ***compositionality*** of internal representations:

  An exponential gain in representational power, due to the fact that simpler concepts represented in a layer of the network can be exploited **as primitives** by the next layer to represent more complex concepts,

  avoiding explicit *combinatorial* representation (and learning) of features

Compositionality, examples …

Dip. Informatica
University of Pisa

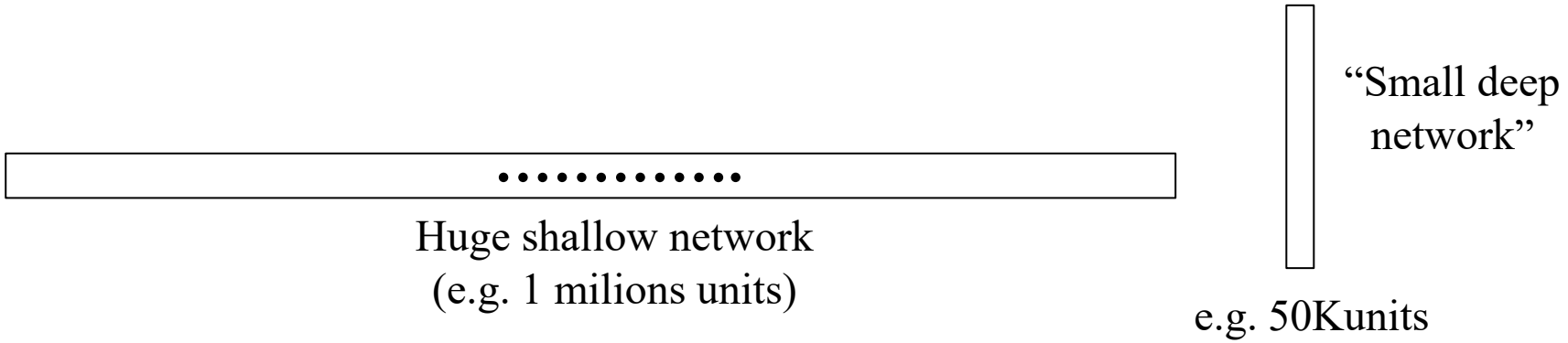
Open issue: Studies on the *compositionality of the target function* (may explain the success?)

- E.g. you have to learn each word/sub-pattern but **not directly all the possible combinations** (i.e. you can learn without having to see all the configurations), also considering them in different contests (we will see specific examples)

Less is better!

Dip. Informatica
University of Pisa

- So, do **not** look at Deep Networks as huge model w.r.t. to small shallow models (just because you see huge instances of models for applications, *deep* is not necessarily equivalent to *big*)
- You can see them as compact models w.r.t. to potentially larger shallow models for the same task (even exponentially more compact)
- Hence, they are something less → more efficient →
- But it is **not** just for computational advantage:
- **Learning** view again: Less units and less weights help learning on complex task to achieve good generalization with/requiring less examples (i.e. making it feasible).



Inductive bias arguments (I)

Dip. Informatica
University of Pisa

REF: DL book Sec 5.11 and 6.4

Common aspects/inductive bias:

- Choosing a deep model encodes a very general belief that the function we want to learn should involve composition of several simpler functions.
  Or nested factor of variation.

- *If our task matches this **bias** of course the deep shape of the learner is suitable*.
  - And it happens that generalization is better due to the use of many layers (see next slides examples)

Typical examples of these kinds of tasks (**hierarchical**) are for:

- The structure of images (composition of sub-graphical parts)
- The structure of language (both text and speech), also music , …?
- …. New fields are under discovering

Inductive bias arguments (II)

Dip. Informatica
University of Pisa

REF: DL book Sec 5.11 and 6.4

Common aspects/inductive bias:

- *If our task matches this **bias*** (composition of several simpler functions) *of course the deep shape of the learner is suitable*.

  - And even: May be that ***nature*** (not only images or languages) have a *hierarchical* structure itself ?
    - Multi-scale modeling, i.e. phenomena seen at different scale levels in space and time, symmetrical for rotations and translation, etc.
    - E.g. "a dog looks the same translating it" → this simplifies the task of pattern recognition and the *set of functions to be learned*.

    - "For reasons that are still not fully understood, our universe can be accurately described by polynomial Hamiltonians of low order"

      Lin, Tegmark, Rolnick, D. *Why does deep and cheap learning work so well*? J. of Statistical Physics, see the Bib.

Inductive bias arguments

Dip. Informatica
University of Pisa

REF: DL book Sec 5.11 and 6.4

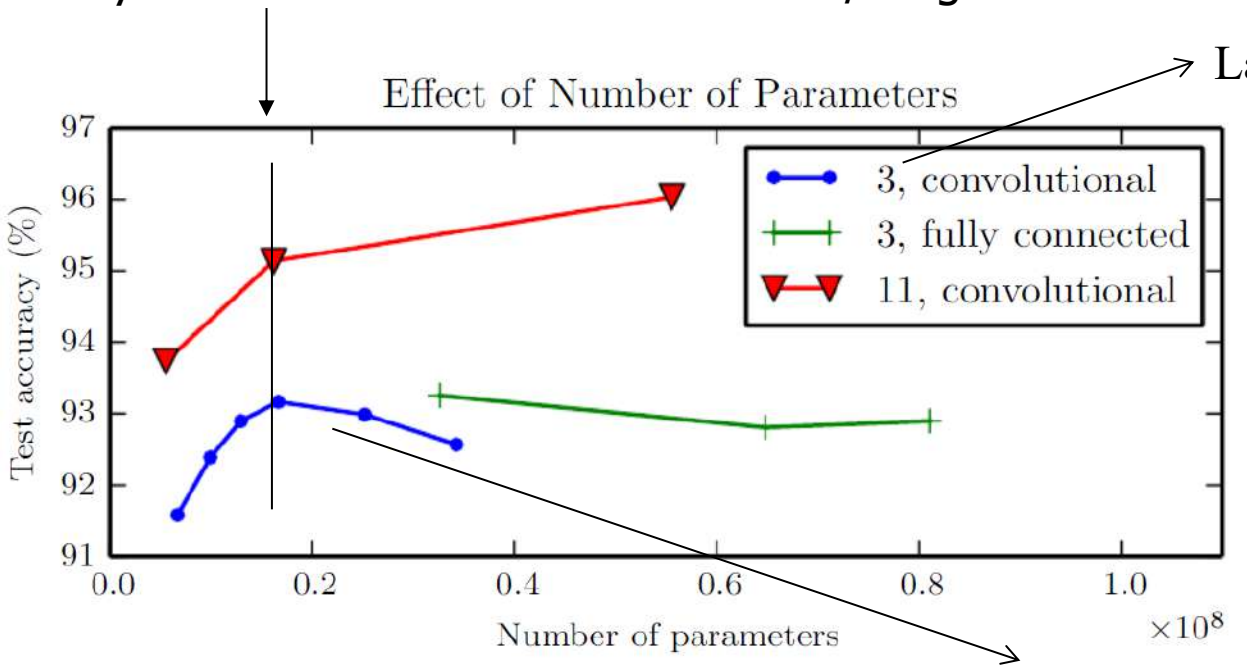
Common aspects/inductive bias:

- *If our task matches this **bias*** (composition of several simpler functions) *of course the deep shape of the learner is suitable*.
  - But, of course, **not** necessarily this holds for *all* the data and tasks
  - However, note that it is still a bias quite general,
  - much more general than other ad hoc constraints for specific tasks (e.g. specific phi in LBE or engineered features)
  - But, again, note that nothing is *magic* without assumptions !!!

Example: Empirical results

Dip. Informatica
University of Pisa

- *If our task matches this bias of course the deep shape of the learner is suitable.*
- In such cases deeper models tend to perform better
- And it happens, for instance, that generalization is better than a model with less layers but the same number of units/weights





Street View House Numbers dataset, ICLR 2014

And more arguments

Dip. Informatica
University of Pisa

There are other arguments investigating the properties of the Deep models, e.g.:

- Curse of dimensionality (number of examples needed in high dimensional space, next slide)
- Manifold learning (probability distribution for *real* images and texts seems concentrated in a manifold using only a fraction of the volume all the possible *random* images and texts, and we should be able to exploit it)

Let us see the first (even if it will clearer later discussing distributed representation advantages)

Curse of Dimensionality and Bias (I)

Dip. Informatica
University of Pisa

- Many learners rely on **local approximation** (local smoothness prior) e.g. the case of K-nn, local kernels, but also Decision Trees etc.
- But often the **smoothness assumption** (or local constancy prior) is not enough
- They need training examples to generalize surrounding and they need *many examples* to cover the entire space (that is huge in high dim., due to the *curse of dim.*), i.e. all the possible regions (when we have different target function behavior)
  - Specifically, to distinguish *O(k)* regions in input space, all these methods require *O(k)* examples.
- Or on the other way: Additional assumptions on the underlying data generating distribution (aka inductive bias) are needed to work with *O(k)* examples on exponential number of regions (in *k*) (to generalize non-locally)
- Note that we will see again this argument in the "distributed representation" discussion

Curse of Dimensionality and Bias (II)

Dip. Informatica
University of Pisa

- One can approach the issue making strong, *task-specific assumptions* (tailored to each task, *loosing the generality*)
- **Instead**, as we discussed before, the deep learning framework chooses a *quite general* inductive bias related to composition of functions, so again:
  - *The core idea in **deep learning** is that we assume that the data was generated by the **composition of factors** or **features** (different sub-regions representations), potentially at multiple levels in a hierarchy*
- This allow:
  - to achieve a **potential exponential gain** between number of examples and number of regions that can be distinguished, as we explained before (and more later on representation learning and distributed rep.), whenever the assumption holds
  - to actually **generalize non-locally**
  - (And we find again that the **learning** benefits with less examples)

General Practical issues

Dip. Informatica
University of Pisa

- How many layers?
- How many units?

No general answer yet

[→ Model selection…*]

In general for deep learning:

- Deeper networks often are able to use less units in each layer →
  → less parameters →
  → Less training data to achieve a good generalization
- But many layers may be harder to be optimized (trained)

* A note on model selection: hyper-par of this class of models (very large NN) may be excessively high, experience makes a difference in fixing many architectural aspects.

Deep Learning- guideline



Dip. Informatica
University of Pisa

- Basic (or rapid introduction)
- **Insights**
  - Do we need many layers?
  - **Representation learning**
  - Distributed representation
- Techniques (some)
- Applications → further courses in AI (PR, NLP, …)

Abstracting: Representation Learning

Dip. Informatica
University of Pisa

REF: DL book Chapt 15

With DL other more general concepts are emerging

- "*Representation learning is a set of methods that allows a machine to be fed with raw data and to automatically discover the representations needed for detection or classification.*"
  LeCun, Y., Bengio, Y., & Hinton, G. (May 2015). *Deep learning*. Nature, *521*(7553), 436-444.

- Deep-learning methods are representation-learning methods with multiple levels of representation (i.e. introduce a specific kind of reps. expressed in terms of other simpler reps).
- *But the concept is more abstract and it can be applied for many models (e.g. NN in general*)

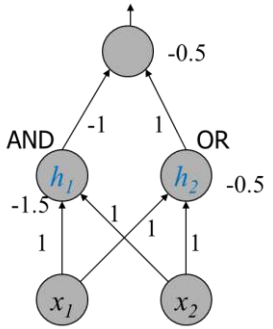
- Note that learning the representation, or *features learning* , is in most of the cases refereed to raw data as images or text strings (not if you provide already a small set of features…)

Basic ideas (I)

Dip. Informatica
University of Pisa

- Many information processing tasks can be very easy or very difficult depending on how the information is represented (everywhere in CS)

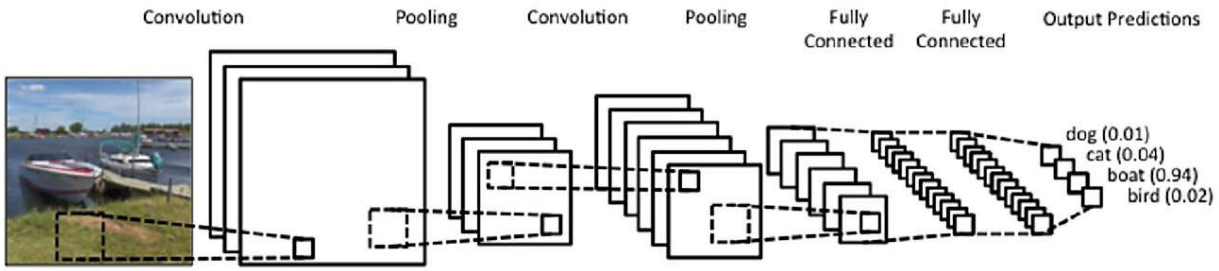
- In ML: a **good representation** is one that makes a subsequent learning task easier.
- Manual design of features is difficult, even decades of years for specific communities (language, images, ….)
  - A.k.a. to manually engineer the [phi] ***ϕ(x)*** **of LBE**
- Supervised learning in NN MLP is an example that leads to an automatic representation at every hidden layer taking on properties that make the output layer task easier.
  - See the XOR example in our ML course (NN1 slides)
  - A.k.a. ***ϕ(x,w)*** **is learned**, finding the best *w* (see NN1 lectures)



Basic ideas (II)

Dip. Informatica
University of Pisa

- Another example:
  - In the CNN we used the *raw image* as input letting the model automatically (learn to) extract the salient features of the image for the task at hand
  - No specific pattern or engineered features have been specified in advance



- **DL** search for a representation on *multiple hierarchical levels*: How to obtain or exploit it (beside the known full training on MLP and CNN)?

How to obtain or exploit hidden representation: Specific cases

Dip. Informatica
University of Pisa

How to obtain hidden representation (historical case):

- Semi-supervised learning: we can learn a representation for the unlabeled data and then use it to solve supervised tasks → ***pretraining*** approaches

How to exploit the hidden representation:

- The learned representation can be used for other tasks → ***transfer learning*** approaches

Let me start with the first aspect: pretraining

Pretraining (preview)

Dip. Informatica
University of Pisa

"Greedy Layer-Wise Unsupervised Pretraining"

- It was the first approach to make possible to train a deep supervised networks ~2006 (other than CNN or RNN)
- Unsupervised learning as pretraining to capture the shape of the input distribution (layer by layer)
- It makes easier the training of the whole network (initialized in such way)
  - Instead of an *end-to-end* training from the output layer to the lower layers with a gradient descent (that was difficult for deep models)
- E.g. by **autoencoders** (see next slides)

Why does it help?

Dip. Informatica
University of Pisa

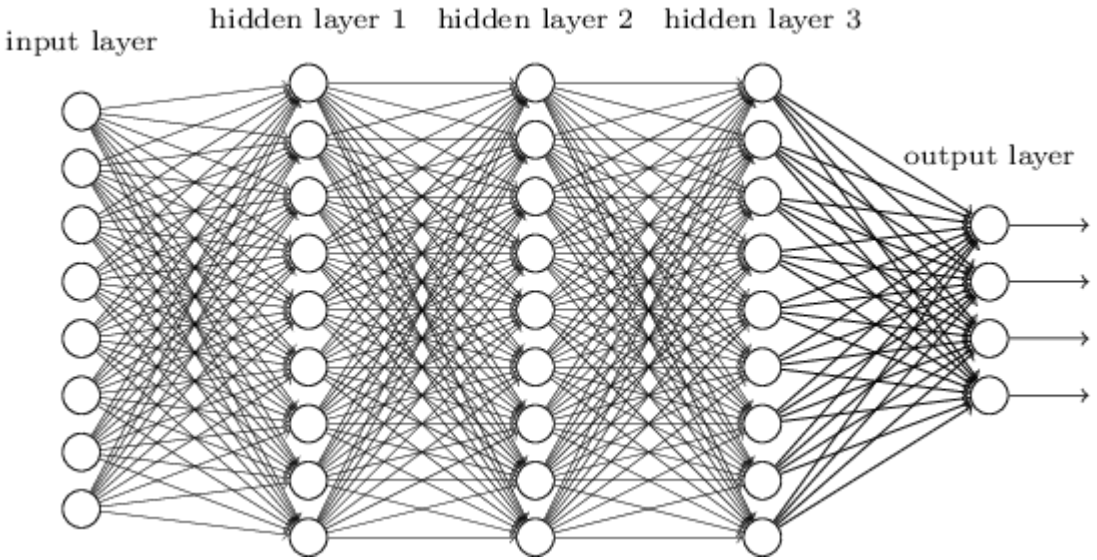
- ***Greedy layer-wise***: each layer is optimized independently in an ***unsupervised*** way, constituting a ***pre-training*** for the final fine-tuning of the network
- It works both in terms of:
  - Good initialization strategy
  - Regularization: herein not in terms of simpler model but in terms of discovering features that simplify the unsupervised process (regularities extracted in an unsupervised way)
    - If the true underlying functions are "complicated" and shaped by regularities of the input distribution, unsupervised learning can be a more appropriate regularizer.
  - Reducing the variance of the estimation (training)process (as pre-trained models tend to arrive in a smaller region after training)

How to build it?

Dip. Informatica
University of Pisa

- Many approaches to implement deep learning

- Let me introduce an approach well rooted in your background through this course
- Basically by building a MLP with many layers and pre-training techniques

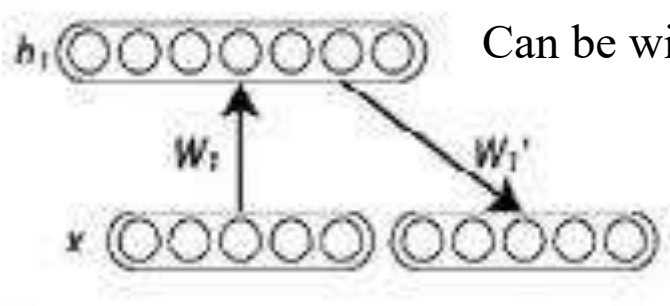


A premise: autoencoders

Dip. Informatica
University of Pisa

REF: DL book Sec 14.1/14.2

- An *autoencoder* is a neural network that is trained to attempt to copy its input to its output.
- Internally, it has a hidden layer *h* that describes a *code* used to represent the input.
- The network may be viewed as consisting of two parts:
  - an encoder function $h = f(x)$ and
  - a decoder that produces a reconstruction $r = g(h)$.



Can be with less units, see next slide

Autoencoders type

Dip. Informatica
University of Pisa

Note: there are many form of auto encoders

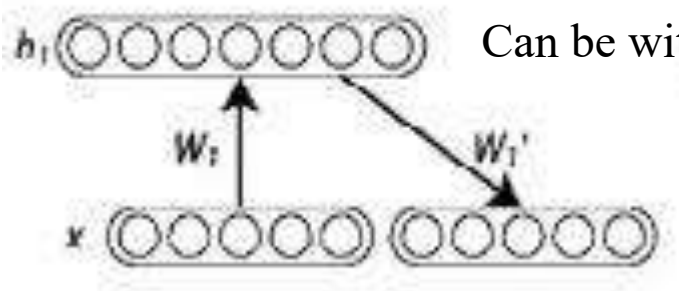
Also they can be

- **Undercomplete**: hidden layer is smaller than the input: force to capture the most *salient features* of the training data by architectural constraints
  - E.g. you obtain a PCA by linear decoder and a LMS approaches
- **Overcomplete**: hidden layer is grater than the input, but we use regularization to make the constraints for *sparsity* of the representation, *robusteness* to noise and/or other properties of interest beside the trivial (for the over complete case) input-output "copy" capability

- Unsupervised learning!

A premise: autoencoders Example of use

Dip. Informatica
University of Pisa



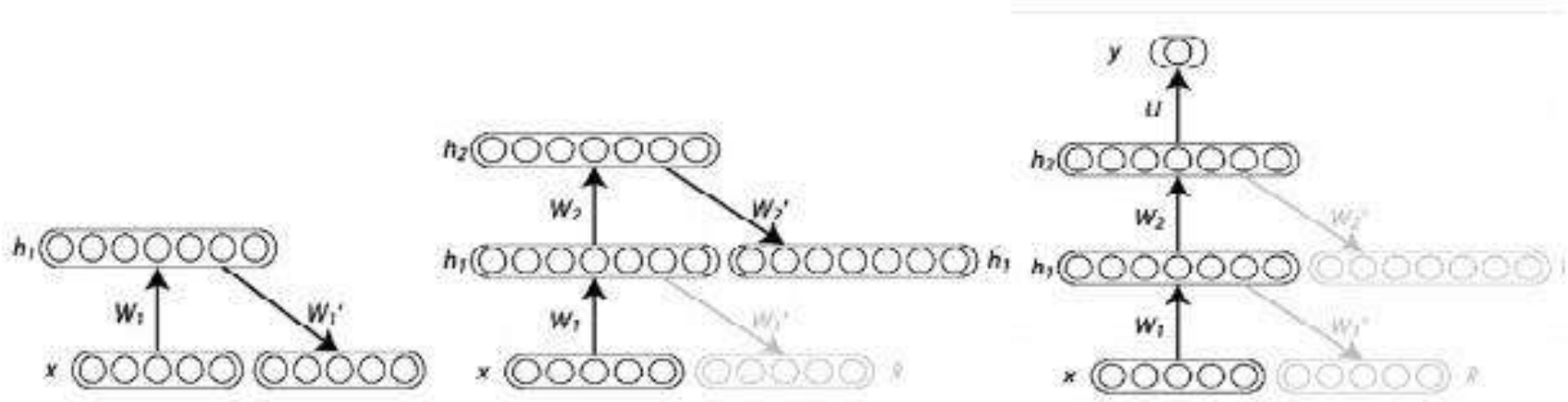
Can be with less units

- Usage: e.g. denoising autoencoders and auto-associative memories:
  - Training on a set of images, finding a good representation of them*
  - * "a *good representation is one that can be obtained robustly from a corrupted input and that will be useful for recovering the corresponding clean input*"
  - Retrieval (reconstruction) of the original image after presenting a noised version and repetitions of encoding/decoding phases

Layer-wise (pre)training

Dip. Informatica
University of Pisa

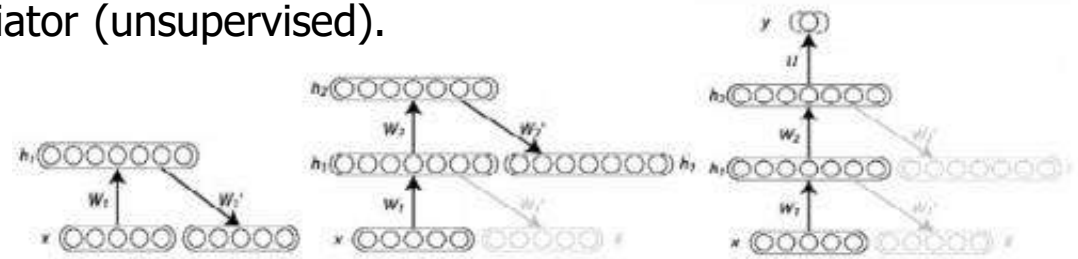
- Unsupervised (wrt to the global task): it can exploit also unlabeled data for the *layer by layer* training
- Auto-encoders used in different DL models in the form of:
  - RBM Restricted Boltzmann Machines (RBMs), which are particular energy-based models → Deep belief networks, e.g. Hinton et al 2006
  - Stacked (NN) (denoising) autoencoders, e.g Bengio et al 2007 ←



The algorithm for DL (by Y.Bengio)

Dip. Informatica
University of Pisa

1. Train the first layer as an autoassociator to minimize the reconstruction error of the raw input (unsupervised wrt the global task: we only need unlabeled examples).
2. The hidden units' outputs in the autoassociator are now used as input for another layer, also trained to be an autoassociator (unsupervised).
3. Iterate as in (2) to add the desired number of layers.



4. Take the last hidden layer output as input to a supervised layer and initialize its parameters (either randomly or by supervised training, keep the rest of the network fixed).
5. Fine-tune all the parameters of this deep architecture with respect to the supervised criterion.

The hope is that the unsupervised initialization in a greedy layer-wise fashion has put the parameters of all the layers in a region of parameter space from which a good local optimum can be reached by local descent

Note: Despite many initial successful cases, the general role of pre-training is nowadays under critical revision by researchers.

Needed?

Dip. Informatica
University of Pisa

- Pre-training can yield improvements for some tasks (especially in NLP) but not in other
- In NLP it allows to exploit large text corpora to learn (distributed) representations of words

- It allowed to start DL!
- But is difficult to be managed (e.g. effects of hyperparemeters divided in two phases, pre- and full- retraining)
- "*Today unsupervised pretraining is largely abandoned*" (DL book Chap 15): the techniques for DL (see in the following) are often sufficient to make a good training from scratch
  - i.e. *end-to-end* backprop. over the deep network
- But still an open field….

Some (modern) short notes on (auto)encoders

Dip. Informatica
University of Pisa

- *Variational autoencoders* use learning algorithms in the Bayesian framework;
  - the latent space is in this case composed by a mixture of distributions instead of a fixed vector
- *Neural Machine Translation* (NMT) can be implemented by autoencoders whereas the output does not match the input - it is in another language:
  - texts are treated as sequences to be encoded into the learning procedure, while on the decoder side sequences in the target language(s) are generated.

Nowadays, we have other techniques:

- The nowdays well-know *transformers* (see GPT, *Generative Pre-trained Transformer*) are encoders/decoders neural networks, plus an attentive mechanism (to learn over sequences, see a next lecture)
  - They can exploit only encoder part (e.g. BERT Bidirectional Encoder Representations from Transformers, used for various NLP tasks)
  - Or the decoder part, for generative purposes (e.g. OpenAI's GPT series for autoregressive text generation)

How to obtain or exploit hidden representation: Specific cases

Dip. Informatica
University of Pisa

How to obtain hidden representation (historical case):

- Semi-supervised learning: we can learn a representation for the unlabeled data and then use it to solve supervised tasks
  → ***pretraining*** approaches

How to exploit the hidden representation:

- The learned representation can be used for other tasks
  → ***transfer learning*** approaches

And now let see the second aspect: transfer learning

Transfer Learning

Dip. Informatica
University of Pisa

- When we use the representation discover in a model to improve another model
- We assume there exist features that are useful for the different settings or tasks, corresponding to underlying factors that appear in more than one setting.
  - Before: from the autoencoder to the classifier (a form of TL)
- But also we can use a trained model for another task (same inputs, changing the target) (***multi-task learning***) → implicit regularization
- Or changing the input domain (but sharing features), e.g. sentiment analysis for customer for videos or for customer for tv and smartphones (***domain adaptation***)
- Or take advantage of pre-trained models from a *larger datasets*…:
  e.g. using AlexNet etc.

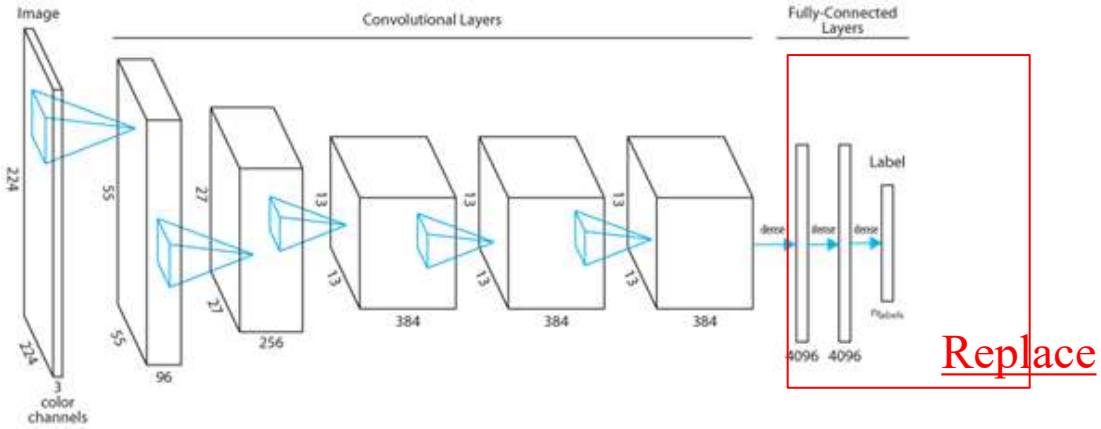


Note: these in bold are sub-fields of ML, not just terminologies!

AlexNet

Dip. Informatica
University of Pisa

- AlexNet is a Convolutional Neural Network (CNN) that has been trained on ~1.2 million images from the ImageNet Dataset
- With 1000 object categories, such as keyboard, mouse, pencil, and many animals. As a result, the model has learned rich feature representations for a wide range of images.
  - the primitives or basic features (low layers) are shared among different images
- The network has 5 convolutional layers and 3 fully connected layers.
- You can *quickly transfer learned features to a new task* using a smaller number of new training images for your new task .
- E.g. Transfer the layers to the new classification task by replacing the last 3 layers for your task and use training for these last 3 layers.
- You can fine-tune a pretrained AlexNet convolutional neural network to perform classification on a new collection of images.



Other examples of pre-trained networks

Dip. Informatica
University of Pisa

- Many pre-trained CNNs for image classification, segmentation, face recognition, and text detection are available.
- E.g. AlexNet in Matlab (Neural Network/DL toolbox)
- Also MatConvNet toolbox, VGG (Oxford) , ResNet (Microsoft), GoogLeNet, ...
- Similarly in other DL libraries (Caffe, Keras, etc.)

Deep Learning- guideline



Dip. Informatica
University of Pisa

- Basic (or rapid introduction)
- **Insights**
  - Do we need many layers?
  - Representation learning
  - **Distributed representation**
- Techniques (some)
- Applications → further courses in AI (ISPR, NLP, …)

Abstracting: Distributed representation

Dip. Informatica
University of Pisa

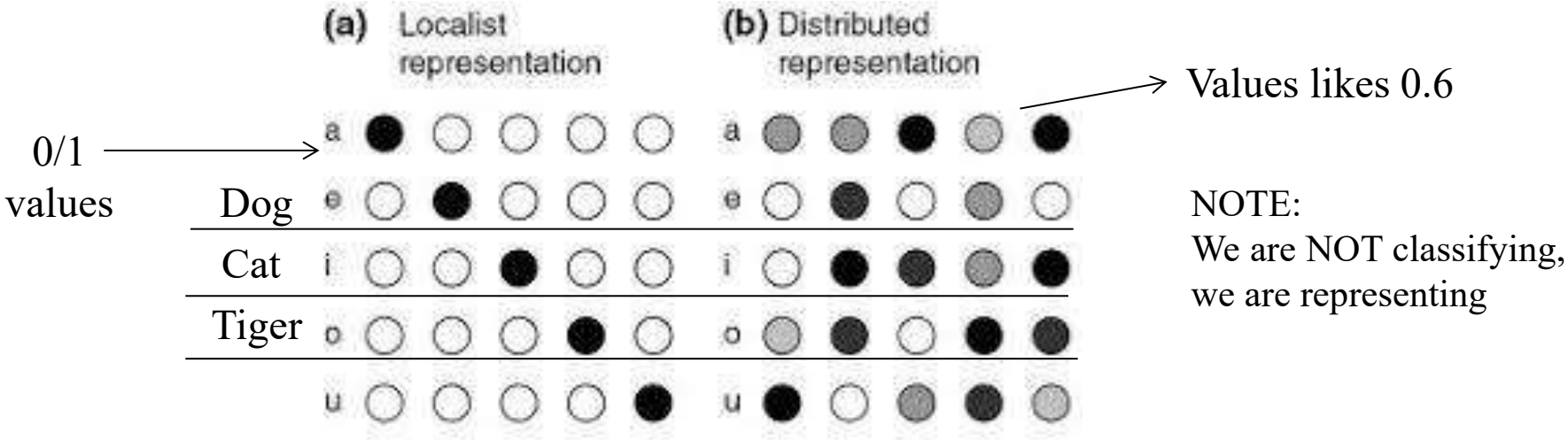
REF: DL book Sec 15.4

- "*In a distributed rep. their elements (the features) are not mutually exclusive and their many configurations correspond to the variations seen in the observed data.*"
  LeCun, Y., Bengio, Y., & Hinton, G. (May 2015). *Deep learning*. Nature, *521*(7553), 436-444.

- Deep-learning methods exploit distributed representation with multiple levels of representation.
- *But the concept is more abstract and it can be applied for many models*, e.g. NNs in general (over hidden units) or probabilistic graphical models (multiple latent variables)*

Symbolic vs distributed

Dip. Informatica
University of Pisa



NOTE:
We are NOT classifying, we are representing

- One-hot representation
- Only one unit is 1, the other 0
- *Distance* always *root(2)* among them

- Vectors of real numbers (dense)
- Each symbol/concept represented as the set of unit activations
- Enable to share the similarity between more than two concepts
- Can share some (*learned) features*
- The *distance* reflects the meaning (our knowledge or such learning)

- ❖ Richer and smooth representation
- ❖ Shared attributes allows generalization among different concepts(see later)

Input or internal rep.?

Dip. Informatica
University of Pisa

Question: Symbolic vs distributed refers to the input or to the hidden layers rep.?

- Here is general, but learning acts on the internal representation (representation learning aspect)
- In general, distributed rep. is used:
  - **In input** if you have background knowledge to fix it (difficult and rare)
    - Or if you fix it arbitrarily it can hamper the task solving
  - **Automatically** (*by learning*) if you can use distribute rep. internally to the model (e.g. hidden NN layers)
    - Typically, one-hot (not informed) rep. is used anyway for the inputs, and the model is let to develop internally the distribution rep. need for the task at hand
- Distinguish **models** able to exploit distributed rep. internally or not (see later for examples)

More in detail

Dip. Informatica
University of Pisa

We will discuss in the next few slides:

- **Count the difference** between symbolic and distributed representation
- **Shared attributes** advantages for learning and generalization, with word-embedding example

Count the difference in the discrete case (I)

Dip. Informatica
University of Pisa

- Blu car, blu bike, red car, red bike: 4 concepts

- But you do not need 4 neurons
- 2 neurons are sufficient (to represent the 2^2 concepts):
  - 1 for Blu/Red
  - 1 for Car/Bike

- and the neuron describing redness is able to learn about redness from images of cars and bikes, not only from images of one specific category of objects.

Count the difference (II)

Dip. Informatica
University of Pisa

- **Discrete case**: Distributed rep. can use *n* features/variables with *k* values to describe k^n different concepts
  - E.g. in binary code: 2^n (K=2) configurations versus *n* possible symbolic rep.s (1-hot)
- Also, e.g., if you encode by *binary* features a data of dim. *N,* with Distr. Rep. we can assign a unique code to exp. number of regions $O(2^n)$
- Hence ML models based on **distributed rep.** (e.g. **NN** with hidden units) is able to distinguish exponentially many more regions than the non-distributed one.
- Examples of **ML models** that rely on **non**-distributed rep.:
  - **k-nn**: 1 or few prototypes for each input: cannot share info with all the training data, and the input component cannot be controlled to be similar each other,
  - **RBF Kernels**: reduce to k-nn as seen in SVM lecture,
  - **Mixture of Gaussian**: similar,
  - **Decision Trees**: only one leaf is activated (and the path from the root)

Shared attributes: *disentangling concepts*

Dip. Informatica
University of Pisa

The same example again (*but for the meaning*)

Distribute representation

	Car (Bike)	Red (Blue)
Red Car	1	1
Red Bike	0	1
Blu Car	1	0
Blu Bike	0	0

Localist representation

Red Car	1	0	0	0
Red Bike	0	1	0	0
Blu Car	0	0	1	0
Blu Bike	0	0	0	1

Sharing a column (feature) we *can* disentangle the concept of "redness" (shared among different objects)

- *Disentangling* the 2 red-blu/car-bike concepts by distributed rep. we can learn about the distinction between Car/Bike, or about the color, without having to characterize all of the configurations of the other features by examples covering all of these combinations of values.
- *Statistical separability* → make easer to *generalize* to configuration unseen during training* (see also the example on "woman with glasses" at the beginning)
  - We can easily classify blu bike having seen the other 3 cases (e.g. "I don't like it" because not red, and the model learnt that I like red objects, with localist rep. Blu bike is completely new)

Shared attributes

Dip. Informatica
University of Pisa

We will further discuss the advantage of sharing attributes in 2 steps (see below in orange)

- Using examples for **word representation:**
- **Symbolic**, one-hot for each word (raw symbols have dimension equal to the vocabulary size), VS **distributed** rep. (vector space)

- Sharing some features between *dog* and *cat* words allows the model:

1. to treat them similarly (when needed by the context)

2. to automatically learn such similarity

- The same can be said for patterns shared in different objects, etc.

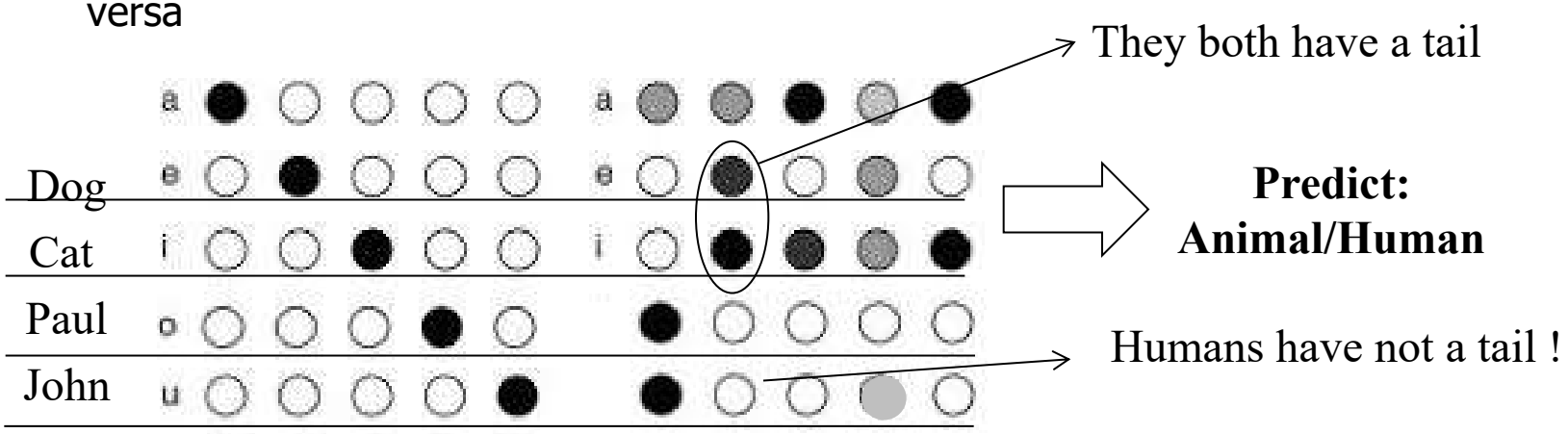
1. Treat dog and cat similarly

Dip. Informatica
University of Pisa

- Instead of using one-hot for each word (vocabulary size)
- Sharing some features between *dog* and *cat* allows the model

1. to treats both similarly, transferring information (by learning) from each training sentence to an exponentially large number of *semantically related* sentences (sharing the same features)

  - For example, if *dog* and *cat* map to representations that share many attributes,
  - then sentences that contain the word *cat* can inform the predictions that will be made by the model for sentences that contain the word *dog*, and vice-versa



2. Learn to be similar Word Embedding

Dip. Informatica
University of Pisa

- Sharing some features between dog and cat allows the model

  2. to automatically make close their rep. due to similar contexts (discovering by learning *common semantic features*) :

- **Learned word-embedding** :
  - one-hot rep. of words: each word is at the same distance from each other, *root(2)*,

  VS

  - vector space representation allowing *different similarity* between words (see for instance "word2vec" techniques)

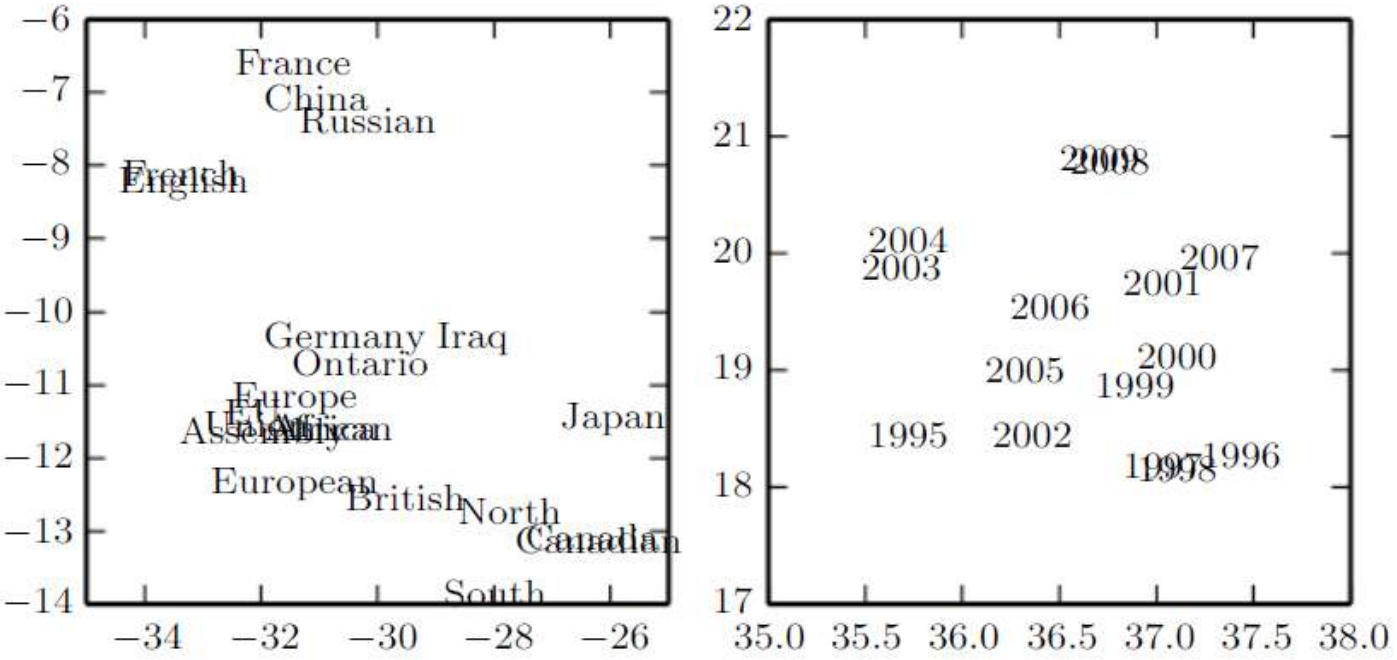
- This often results in words with similar meanings being neighbors, preserving the *semantic similarity* between the words (Semantic distribution of words in the text) e.g. see next slide
- And the models can generalize much better!

Word embedding results

Dip. Informatica
University of Pisa

Zooms in on specific areas of a ***learned word embedding space*** to show how *semantically similar* words map to representations that are close to each other,

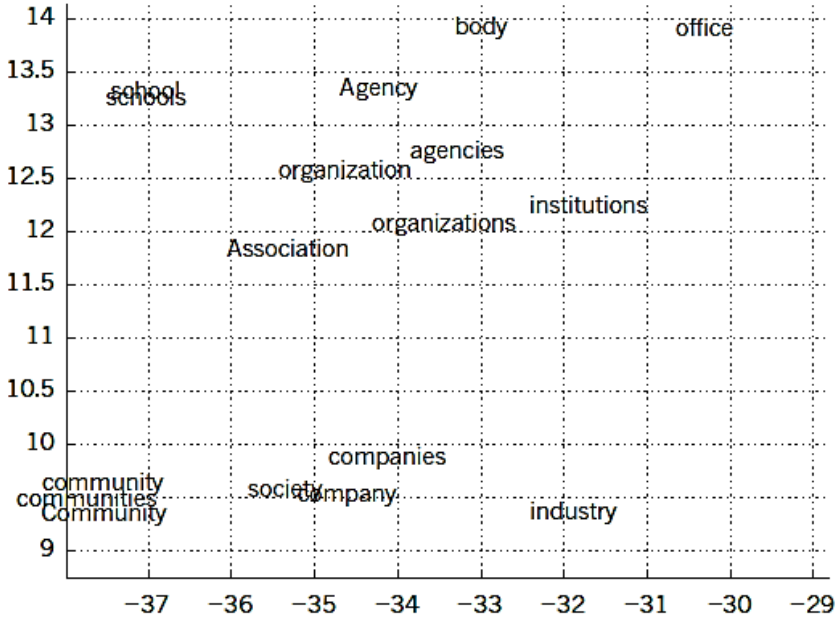
i.e. hidden layers activations projected in 2D from a *neural translation machine*: countries are in the same zone, *numbers* (*years*) as well



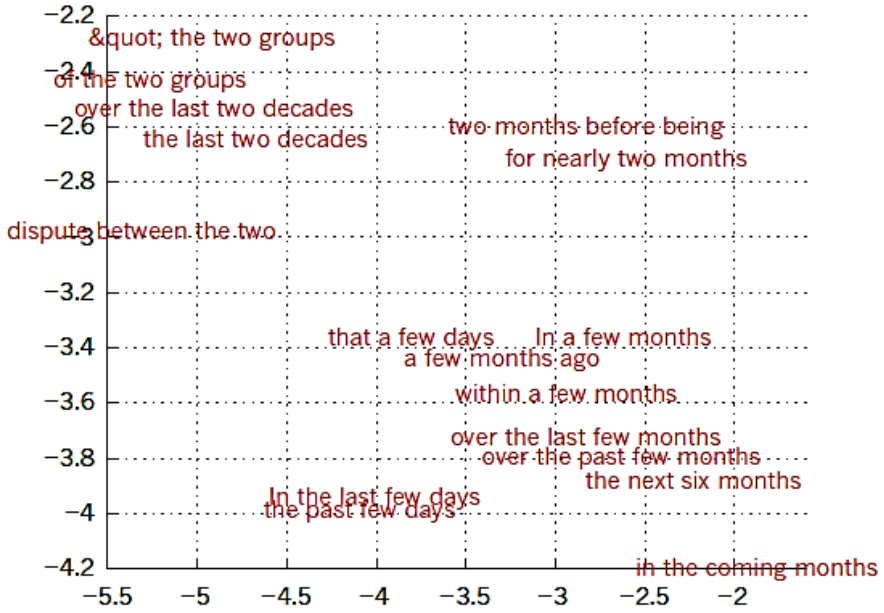
It **automatically** "discovers" *countries* or *years* from learning

Word embedding results (2)

Dip. Informatica
University of Pisa



Word embedding space (projected in 2D)



Phrases learned by an English-to-French encoder–decoder recurrent neural network

One can observe that semantically similar words or sequences of words are mapped to nearby representations.

The distributed representations of words are obtained by using *backpropagation* to jointly learn a representation for each word and a function that predicts a target quantity such as the next word in a sequence (for language modelling) or a whole sequence of translated words (for machine translation)

Word embedding results (3)
A short discussion on DL results

Dip. Informatica
University of Pisa

- The model learns not only the grammar (this is not a surprise indeed): the learned representations are *semantically* significant
- And the model **learn them automatically**, just feeding with text used as training, **not other knowledge** is provided!

- **Issue: why our semantic corresponds to what is learnt by the (Deep) NN?**
- Probably the language (and images) intrinsically shows a (hierarchical) structure in itself and the Deep NN has the proper *inductive bias* to learn that structure (in a distributed fashion)
- E.g. real images are just a small fraction of white-noise images

- But this is still an open field of research!

Beyond NN

Dip. Informatica
University of Pisa

- The debate on distributed representation extend to the debate between
- **Logic-inspired** paradigms for cognition
- **Neural-network**-inspired paradigms for cognition

- For instance, recent successes in language translation raise doubts on the need of symbolic approaches to understand sentences
- …

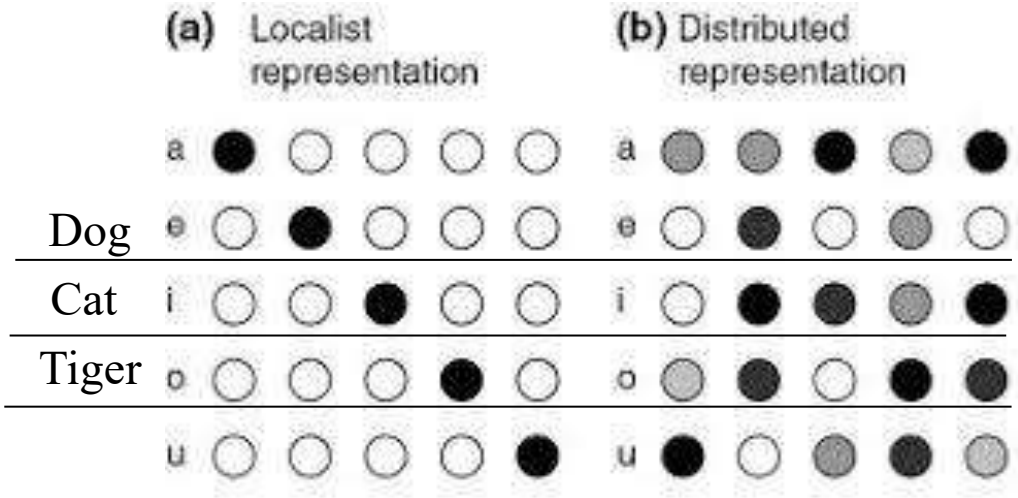
Note: of course, distributed rep. affects also internally the models , not just for the initial input description!

Interpretability

Dip. Informatica
University of Pisa

A drawback of the many advantages of distributed representation:

Interpretability of a *distributed representation* is less easy



And what for deep distributed rep.?

Dip. Informatica
University of Pisa

- DL Exploit distributed representation through many layers,
  - obtained them composing different level of abstraction
  - or by a hierarchy of reused features

- And as we already discussed such compositionality can lead to a further exponential boost to statistical efficiency (on the top to the exponential boost by distributed rep.)
  - two different potential exponential advantages (over not-distributed and shallow models).

- *Globally, **DL** models learn a distributed rep. of data finding (or better disentangling) shared (by distributed rep.) causal factors that generates the data through different level of abstraction*

- And **representation learning** is the more general concepts emerged from these studies

Combined application: Image + Text

Dip. Informatica
University of Pisa

- Combine CNN and RNN for the generation of image captions [Image source + language generation]: Impressive results, look where the attention is focused by the lighter patches generating each word (bold)



A woman is throwing a **frisbee** in a park.



A **stop** sign is on a road with a mountain in the background



A little **girl** sitting on a bed with a teddy bear.



A giraffe standing in a forest with **trees** in the background.

Vinyals, O., Toshev, A., Bengio, S. & Erhan, D. Show and tell: a neural image caption generator. *ICML* http:// arxiv.org/abs/1502.03044 (2014).

Medical example: skin cancer

Dip. Informatica
University of Pisa





- A deep neural network trained on 130.000 cases, much more cases than a clinician could see in (many) lifetimes.
- Capable of classifying *skin cancer* with a level of competence comparable to certified dermatologists (Nature 2017)
- It can work with an **app**

Deep Learning- guideline

Dip. Informatica
University of Pisa

- Basic (or rapid introduction)
- **Insights**
  - Do we need many layers?
  - Representation learning
  - Distributed representation
  - **Further topics**



- Techniques (some)
- Applications → further courses in AI (ISPR, NLP, …)

Other general insights

Dip. Informatica
University of Pisa

- **Smoothness through network structure:**
  For a given number of parameters deeper networks impose more smoothness than shallow ones
  - Each layer works on the already smooth surface output by the previous layer

- Deeper networks seem to learn better, for the same number of total neurons
  - Implicit smoothness constraints
  - As opposed to explicit constraints from more conventional classification models

Recent advancements



Dip. Informatica
University of Pisa

- In the following: Recent insight topics that can deserve your attention for further (personal) studies

- Also as possible SEMINARS at the end.
- We accept volunteers … (write to me)

Non-overfitting puzzle

Dip. Informatica
University of Pisa

- Huge NN with apparent lack of overfitting, why?
- « *A main puzzle of deep networks revolves around the absence of overfitting despite large overparametrization and despite the large capacity demonstrated by zero training error on randomly labeled data.* »

- Gradient descent enforces a form of implicit regularization
- Convergence to the maximum margin solution
- Robustness w.r.t. curse of dimensionality implications (see before)

- Poggio, T., Kawaguchi, K., Liao, Q., Miranda, B., Rosasco, L., Boix, X., Hidary, J. and Mhaskar, H., 2017. Theory of Deep Learning III: explaining **the non-overfitting puzzle**, arXivpreprintarXiv:1801.00173

Two (surprising?) theorems

Dip. Informatica
University of Pisa

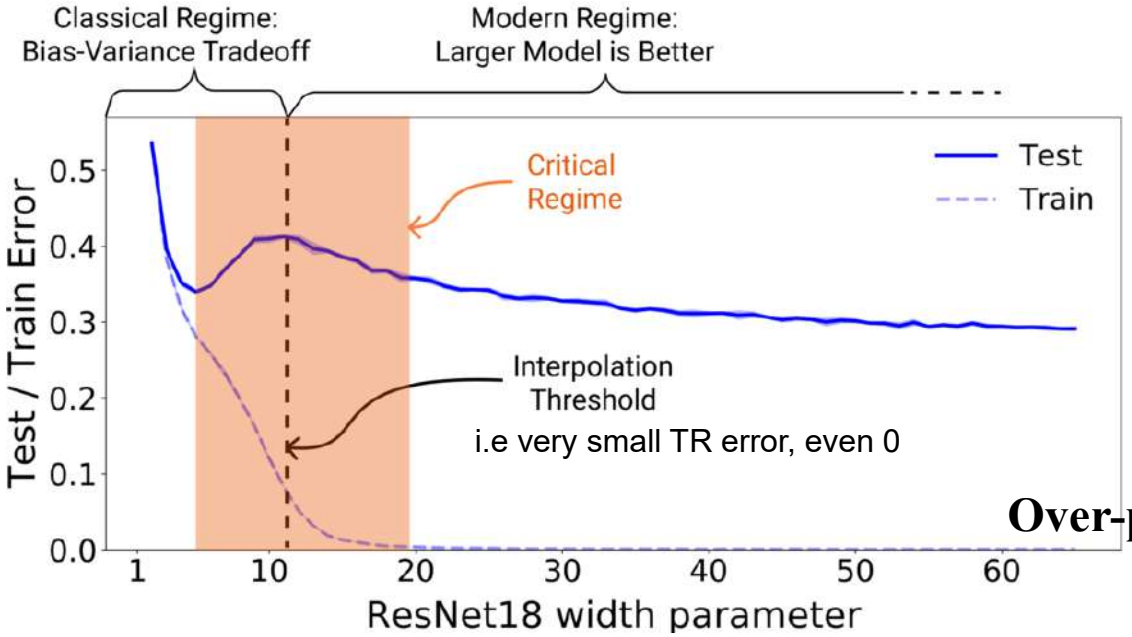
For DeepNN training:

- Variants of SGD (batch normalization and weight normalization, see later) perform minimization *equivalent to maximize (normalized) margin*
- Even standard gradient descent *maximize* L_2 *margin* (without explicit normalization or regularization)
  (or implicitly it *controls the complexity* through an "implicit" unit L_2 norm constraint)

- Poggio et al. Theoretical Issues in Deep Networks: Approximation, Optimization and Generalization, PNAS 2019

New Insights on training large deep models

Dip. Informatica
University of Pisa



Training and test error as a function of model size, for ResNet18s of varying width on CIFAR-10 with 15% label noise.

Over-parametrization

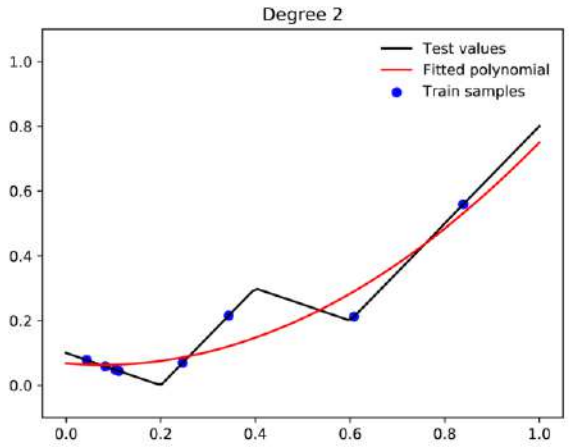
- Deep Double descent phenomenon occurs in CNNs, ResNets, and transformers *(millions of parameters)*: performance first improves, then gets worse (classical U-shape), and then **improves again** *with increasing model size*, or training time. This effect is often avoided through careful regularization.
- While this behavior appears to be fairly universal, we don't yet fully understand why it happens. *Further study of this phenomenon are an important research directions: "Effective Model Complexity" studies w.r.t number of data etc. or re-interpreting classical SLT etc.*

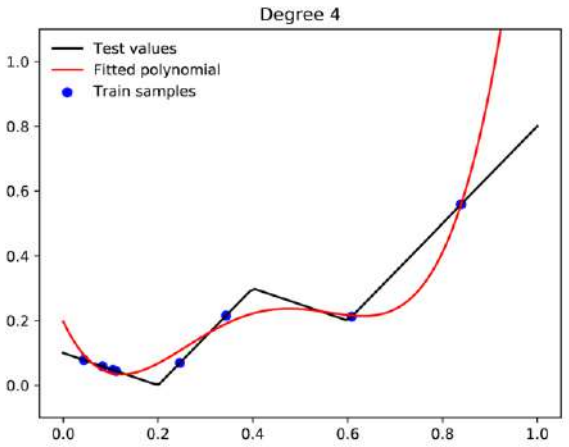
P.S. NOT necessarily in your case (small models for the prj)

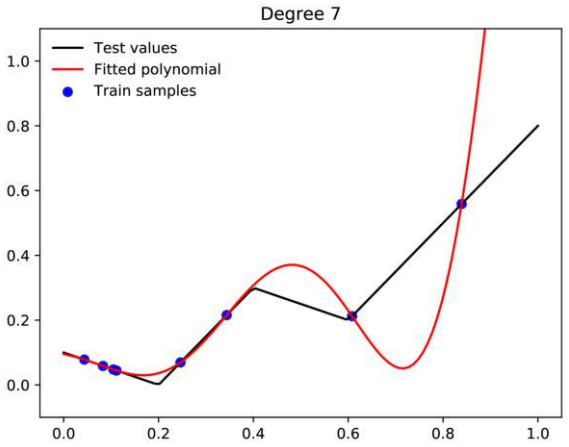
A toyish example on polynomial regression with regularization (I)

Dip. Informatica
University of Pisa

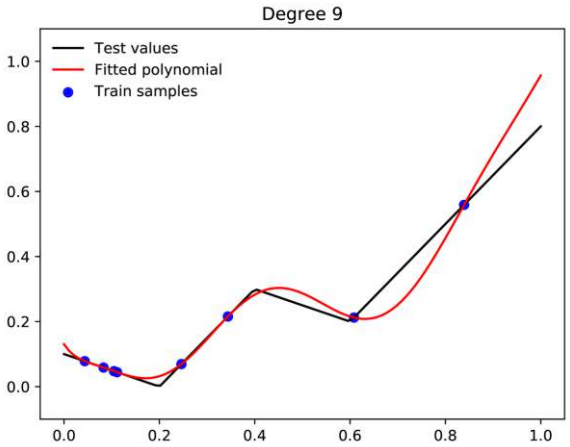
Anyway, let's us as see a simple case
and that the *regularization* is still need (and in part explain the phenomena)

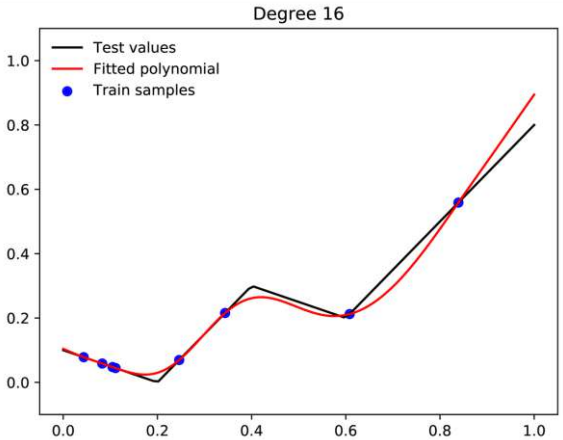






After zero TR error, when the degree increases, the fit is only driven by the penalty term, and the polynomial gets more and more "piece-wise linear"

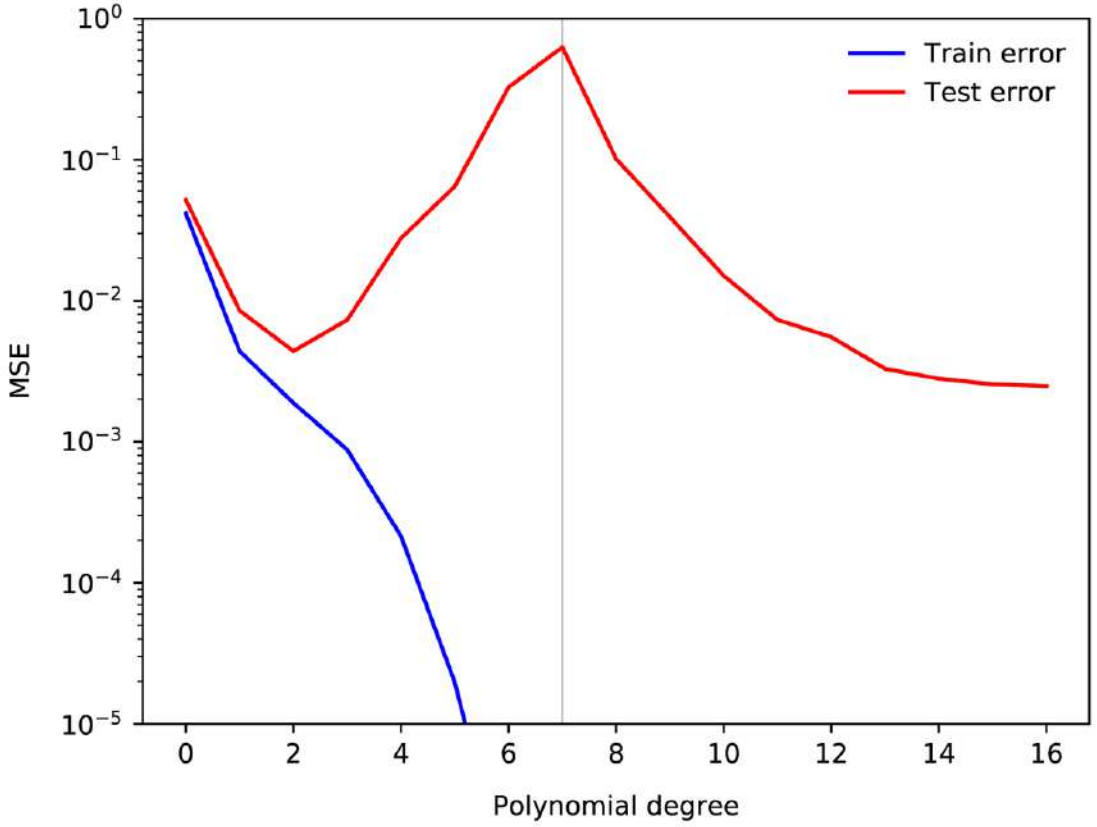




A toyish example on polynomial regression with regularization (II)

Dip. Informatica
University of Pisa

If we look at the [median over 250 runs] of the MSEs,
we get the pretty "double descent"!



Credits: François Fleuret

Much more on Double descent

Dip. Informatica
University of Pisa

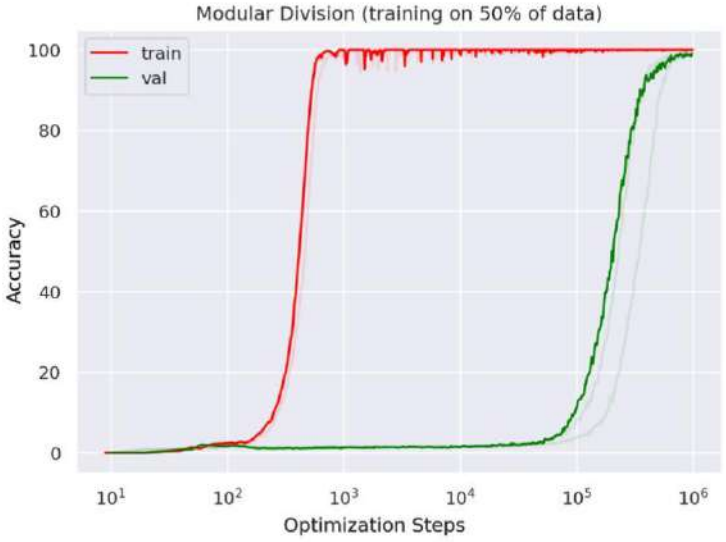
- E. H. Lee and V. Cherkassky, "*Understanding Double Descent Using VC-Theoretical Framework*," in IEEE Transactions on Neural Networks and Learning Systems, vol. 35, no. 12, pp. 18838-18847, Dec. 2024
  - or Eng Hock Lee, Vladimir Cherkassky. VC Theoretical Explanation of Double Descent,. https://arxiv.org/pdf/2205.15549
- L. Oneto, S. Ridella, and D. Anguita, "*Do we really need a new theory to understand over-parameterization?*" Neurocomputing, vol. 543, p. 126227, Jul. 2023, doi: 10.1016/j.neucom.2023.126227

Grokking

Dip. Informatica
University of Pisa

- Also related concepts: **Grokking** is a recently discovered phenomenon where generalization is achieved long after a model overfits to the training data (focus on the number of training steps/epochs)

For a fixed architecture, the neural network starts to generalize many epochs after reaching the interpolation threshold (i.e. when in theory it would have overfitted)



- See e.g.:
  - Light intro: Power, Burda, Edwards, Babushkin, Misra. *Grokking: Generalization beyond overfitting on small algorithmic datasets,* https://arxiv.org/pdf/2201.02177
  - Miller, O'Neill, Bui: *Grokking Beyond Neural Networks: An Empirical Exploration with Model Complexity*. Transactions on Machine Learning Research, 2024, https://openreview.net/pdf?id=ux9BrxPCl8
  - Z. Liu, E.J. Michaud, M. Tegmark. *Omnigrok: Grokking beyond algorithmic dat,* https://arxiv.org/pdf/2210.01117

Redundancy

Dip. Informatica
University of Pisa

- «*We demonstrate that there is significant redundancy in the parameterization of several deep learning models.*
- *Given only a few weight values for each feature it is possible to accurately predict the remaining values. [...]*
- *In the best case we are able to predict more than 95% of the weights of a network without any drop in accuracy.*»

- Denil, M., Shakibi, B., Dinh, L. and de Freitas, N., 2013. *Predicting parameters in deep learning.* In Advances in Neural Information Processing Systems (pp. 2148-2156).

Compressing the parameters

Dip. Informatica
University of Pisa

- Or compressing the parameters without accuracy loss

- Han, S., Mao, H. and Dally, W.J., 2015. *Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding*. arXiv preprint, arXiv:1510.00149.

- So, small network seems enough!

Lottery ticket hypothesis

Dip. Informatica
University of Pisa

Why, then, do we train large networks?

Frankle and Carbin answered: with the lottery ticket hypothesis:

- Good performance depends on the *lucky* initialization of one/more subnetworks.
- Large networks have exponentially more of these subnetworks.
- Testable hypothesis: Retraining pruned network on *same initialization* should provide similar performance.

- Frankle, J. and Carbin, M., 2018. The lottery ticket hypothesis: Training pruned neural networks. *arXiv preprint arXiv:1803.03635*.

Randomization

Dip. Informatica
University of Pisa

- The role of randomization (for randomly-initialized network and for random weights neural networks) is attracting more attention.

- Ulyanov, D., Vedaldi, A. and Lempitsky, V., 2017. *Deep image prior.* 2018 IEEE CVPR.
- Giryes, R., Sapiro, G. and Bronstein, A.M., 2016. *Deep neural networks with random gaussian weights: A universal classification strategy?.* IEEE Transactions on Signal Processing, 64(13), pp.3444-3457

- See also next lecture

Deep Learning- guideline

Dip. Informatica
University of Pisa

- Basic (or rapid introduction)
- Insights (do we need many layers?)
- **Techniques (some)**
- Applications → further courses in AI (PR, NLP, …)

Techniques: why? [repetita]

Dip. Informatica
University of Pisa

In general for deep learning:

- We have seen many advantages of deep NN
- But many layers may be harder to be optimized (trained), hence:

- Emphasis on methods to improve
  - gradient descent (also gradient vanish trough layers, grad. clipping,…)
  - regularization (due to large networks)
  - better exploitations of data (e.g. semi-supervised learning, reinforcement earning cycles, multi-task learning, generative adversarial learning etc.)

…. These deserve to be presented (some of them) in the following

Also:

- Larger datasets from industry and real applications (e.g. millions of images)
- Software infrastructure (also public available)
- HPC improvements (GPU and over)

Our approach to the following overview

Dip. Informatica
University of Pisa

Methodological note:

- At this time in the course, it is *intentional* to do *not* add to many technical details and many other arguments to the course.
- You can deep the topic of *your* interest
- And you should have sufficient background to do it by yourself

- So, I'll just provide an overview of some interesting aspects
- (Essential) Bibliography references are at the end of each sub-part in the following

- See it in this way: I'm exploiting the idea of *distributed representation* avoiding to report all the possible combination ;-)

Key techniques for DL

Dip. Informatica
University of Pisa

For training of layered NN:

- Originally: pretraining approaches
  - But still in domains as NLP (word-embedding)
- Nowadays (in the popular libraries): You already know them
  - **SGD** with momentum (with decay on η, mini-batch) or Adam etc.
  - **ReLU** activation functions (hidden units)
  - Use of max log-likelihood (**cross-entropy**) loss with soft-max for output layers [also to avoid saturation/small gradients effects]
  - **Regularization**:
    - Early-stopping/Weight -decay/ You already know them
    - Drop-out or batch-normalization (for CNN/ sigmoid f.)

Gradient issues

Dip. Informatica
University of Pisa

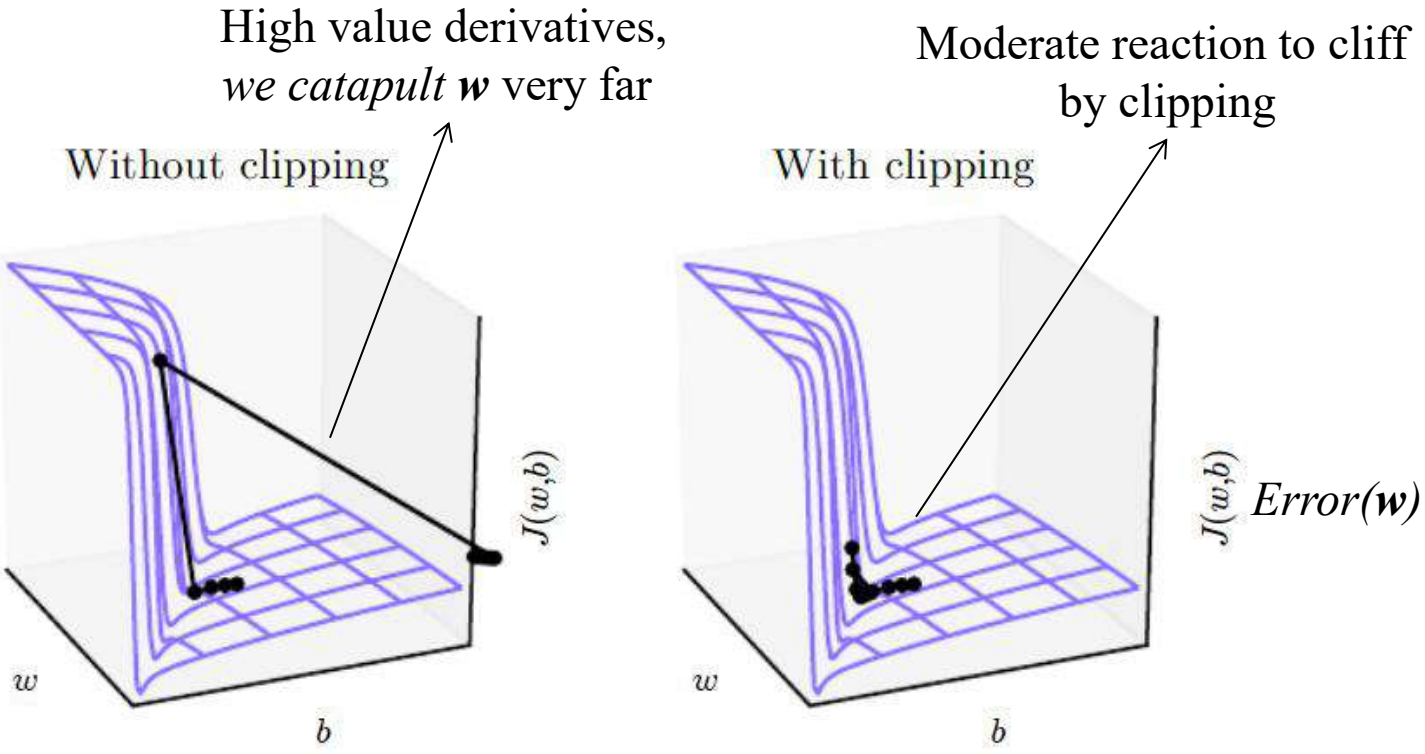
- What happens to the magnitude of the gradients as we backpropagate through many layers?
  - If the weights are small, the gradients shrink exponentially *(gradient vanish issue)*.
    - Especially down-sized by Jacobian of sigmoidal function
    - See also later
  - If the weights are big the gradients grow exponentially*.

- We are going to have a look to some easy approaches or discuss the issues

Clipping gradient

Dip. Informatica
University of Pisa

- Repetition of multiplication through many layers can introduce cliffs in the cost function
- Clipping: If $||\boldsymbol{g}|| > v$ then $\boldsymbol{g} = v\,\boldsymbol{g} / ||\boldsymbol{g}||$, $\boldsymbol{g}$: gradient and v: norm threshold
  So to move in the gradient direction, but bounding the weight update



Deep Learning book: Sections 8.2.4, 10.11.1

Gradient Vanish: a note

Dip. Informatica
University of Pisa

Most of the heuristics try to deal with the problem of **gradient vanish** that hampers the training in early deep NN in the low layers:

- Traditional activation functions such as the *hyperbolic tangent* function have gradients in the range (−1, 1), and backpropagation computes gradients by the *chain rule* (repeated through the layers).
- In *d*-layer network, this has the effect of multiplying *d* of these small numbers to compute gradients of the "front" layers (close to inputs),
- meaning that the gradient (error signal delta) decreases exponentially with *d* while the front (close to the input) layers training becames very slowly.
  - Of course if weights grows the backpropagated signals can increase (too much? Exploding case, but much more rare for tanh/sigmoidal)

There are many **approaches** to deal with it:

- From Rprop, to short-cut connections, up to randomized NN (see next slides) (see also further course for the RNN case)
- Let see the new ones (ReLU, …)

Skhetch of the idea [tech#]

Dip. Informatica
University of Pisa

- Let M last layer, l current layer
- $\boldsymbol{h}_l = F_l(\boldsymbol{h}_{l-1})$
- $F = F_M \circ \cdots \circ F_1$ the layers composition

→ $\partial F \,/\, \partial w_l = \partial\, F_M \,/ \partial \boldsymbol{h}_{M-1} \cdot \ldots \cdot \partial F_l \,/ \partial w_l$

→ $|\partial F \,/\, \partial w_l\,| \approx \rho^{M-l+1}$ assuming $\|\, \partial Fi \,/\, \partial \mathbf{h}i \,\| \approx \rho$

- $\rho \lessgtr 1 \Rightarrow$ vanishing/exploding of the gradient

- We can discusse it at a Q&A

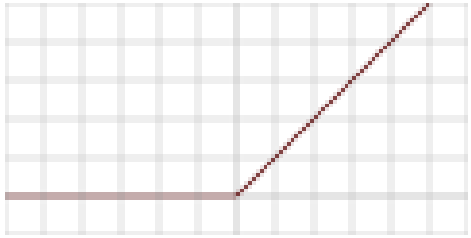
Important topic

ReLU activation function

Dip. Informatica
University of Pisa

REF: DL book Sec 6.3

- ReLU (Rectified Linear Unit):



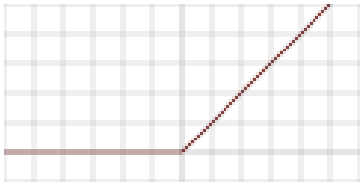
$$f(x) = \max(0, x)$$

$$f(x) = \begin{cases} 0 & for \quad x < 0 \\ x & for \quad x \geq 0 \end{cases}$$

- *Y. LeCun, L. Bottou, G.B. Orr and K. Müller (1998). "Efficient BackProp"* http://yann.lecun.com/exdb/publis/pdf/lecun-98b.pdf
- *X. Glorot, A. Bordes, Y. Bengio (2011). Deep sparse rectifier neural networks*
- *In DL BOOK: Section 6.3.*

ReLU: Discussion (I): *f'*

Dip. Informatica
University of Pisa

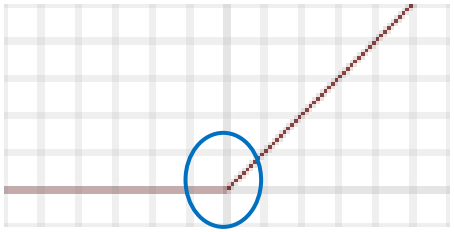


- Why use it for DL (since 2009)?
  - Simplified/Better gradient propagation through many layers (see previous slide for gradient vanish)
  - Avoid **saturation** effect of **sigmoidal functions**, where **gradient** shrink to small (see NN-part1: always $f'(x) < 1$), **while ReLU always has $f'(x) = 1$ for positive net**
  - Hence the gradient can be better propagated back through many layers, memento:

$$\delta_j = \left(\sum_{k=1}^{K} \delta_k w_{kj}\right) \cdot f'_j(net_j) \quad \text{if } t = j \text{ (hidden unit)}$$

ReLU: Discussion (II): In 0?

Dip. Informatica
University of Pisa



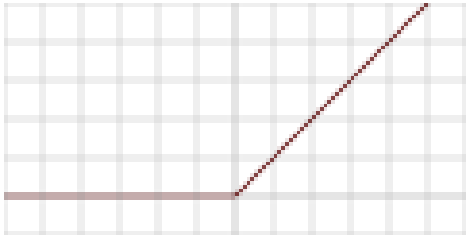
- Gradient in 0? Warning: It is Not differentiable in 0
- But often (in ML libraries) is assumed simply to be 0 (left derivative) or 1(right deriv.): approximation is "acceptable and safe" because
  - already input/numerical approximation make evaluation on $f(0)$ unlikely to be really for a true vale $x=0$.
  - So $f'(x) = 0$ if $x<0$, $=1$ if $x>0$, and arbitrarily 0 or 1 in zero.

- To be more principled: CM approaches for PieceWise Linear functions (via Non-differentiable optimization, etc.)

ReLU: Adavantges

Dip. Informatica
University of Pisa

- Efficient gradient propagation: w.r.t. vanish (previous slide)
- Sparse activation: E.g., in a randomly initialized network, only about 50% of hidden units are activated (having a non-zero output).
- Efficient computation: only comparison, addition and multiplication.
- Still neuroscience inspiration (*Glorot et al, 2011)*
- Faster and effective training of **deep** neural architectures



Also for ReLU

Dip. Informatica
University of Pisa

- Tricks:
  - Start with positive nets , e.g. bias to a small, positive value, such as 0.1.
  - This makes it very likely that the rectified linear units will be initially active for most inputs in the training set and allow the derivatives to pass through.

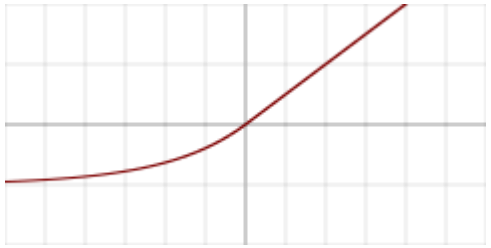
- Many generalization (see also the NN –part1 lecture and/or sec 6.3.1 DL book)
- An updated list (Note: many perform comparably!): https://en.wikipedia.org/wiki/Activation_function

- In general they avoid some drawbacks but still retaining the linear part, which in general made the model easer to be optimized…

Examples beyond ReLU

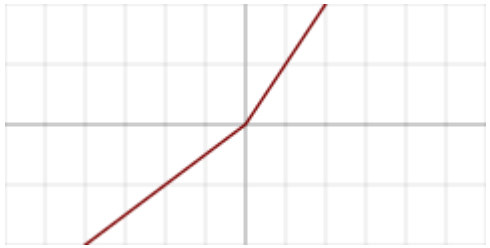
Dip. Informatica
University of Pisa

- ELU (Exponential linear unit)



$$f(\alpha, x) = \begin{cases} \alpha\left(e^x - 1\right) & \text{for } x \leq 0 \\ x & \text{for } x > 0 \end{cases}$$

- Leaky ReLU



$$f(x) = \begin{cases} 0.01x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$$

- Learning also on the left part (neurons are not turned off), mean activation closer to zero, …

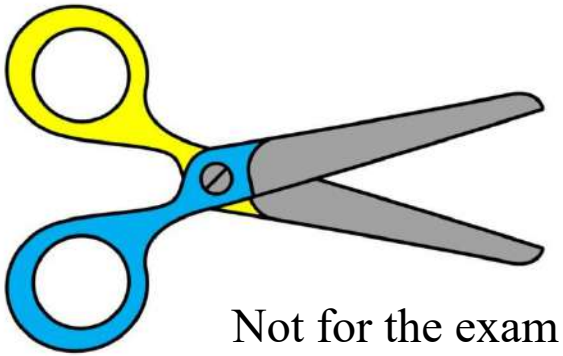
- An updated list https://en.wikipedia.org/wiki/Activation_function

Other

Dip. Informatica
University of Pisa

- Hessian Free Optimization (already discussed)
- Smart Initialization: pre-training techniques
  - Pre-training, aka "A good start is half the work"
  - Drive weights in a region where it is easy to reach better (local) minima
  - e.g. auto-encoders seen before

If we will have enough time we will discuss details later



Not for the exam
(unless you use it)

Batch normalization - Overview

Dip. Informatica
University of Pisa

- **Batch Normalization** is a method that *normalizes* each batch by calculating each individual batch statistic such as mean and variance for each layer (reparametrization).
  - Normalize each matrix [databatch x activation of units] with mean and variance (shifting values to zero-mean and unit variance)
  - Include it in backpropagation
- As normalizing the input is a standard approach, BN helps by making the data flowing between intermediate layers of the network to stay normalized.
  - Regularization effect (due to the noise added at training time by the mini-batch size)
  - Achieving faster learning and higher overall accuracy for DL (it deserve to be mentioned due to its relevance to allows CNN/sigmoidal deep models to be efficiently trained).

- To deepen: DL book Sec. 8.7.1. or ICML 2015 (Ioffe, Szegedy, Batch Normalization: *Accelerating Deep Network Training by Reducing Internal Covariate Shift*).

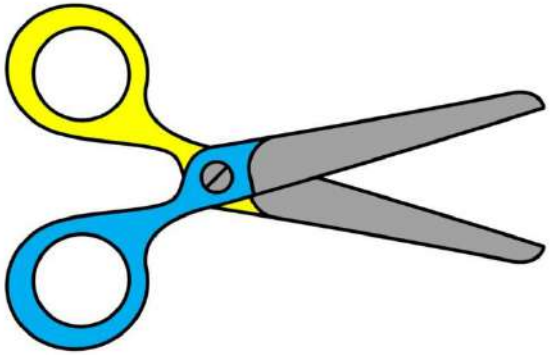


Batch normalization - Details

Dip. Informatica
University of Pisa

Include it in backpropagation: http://kratzert.github.io/2016/02/12/understanding-the-gradient-flow-through-the-batch-normalization-layer.html

If we will have enough time we will discuss details later



Not for the exam
(unless you use it)

Important topic

Dip. Informatica
University of Pisa

Dropout - Overview

REF: DL book Sec 7.12

- DropOut is a method that randomly selects a subset of the network *during its training*

- It combines (can be explained in terms of)
  - Ensembling (implicit bagging, different sub-networks)
    - But computationally inexpensive
  - Regularization

Concepts that you already know!

- "*Dropout: a simple way to prevent neural networks from overfitting*" JMLR 2014
- Section 7.12 DL book

From Bagging to Dropout

Dip. Informatica
University of Pisa

BAGGING (REPETITA)

- Combine many classifiers/models
- Bagging
  - Train n classifiers/models on different subsets of training set
  - Differentiate each training using bootstrap (resampling with replacement)
  - Output the average outcome
- High variance models (with law bias) can perform well on average

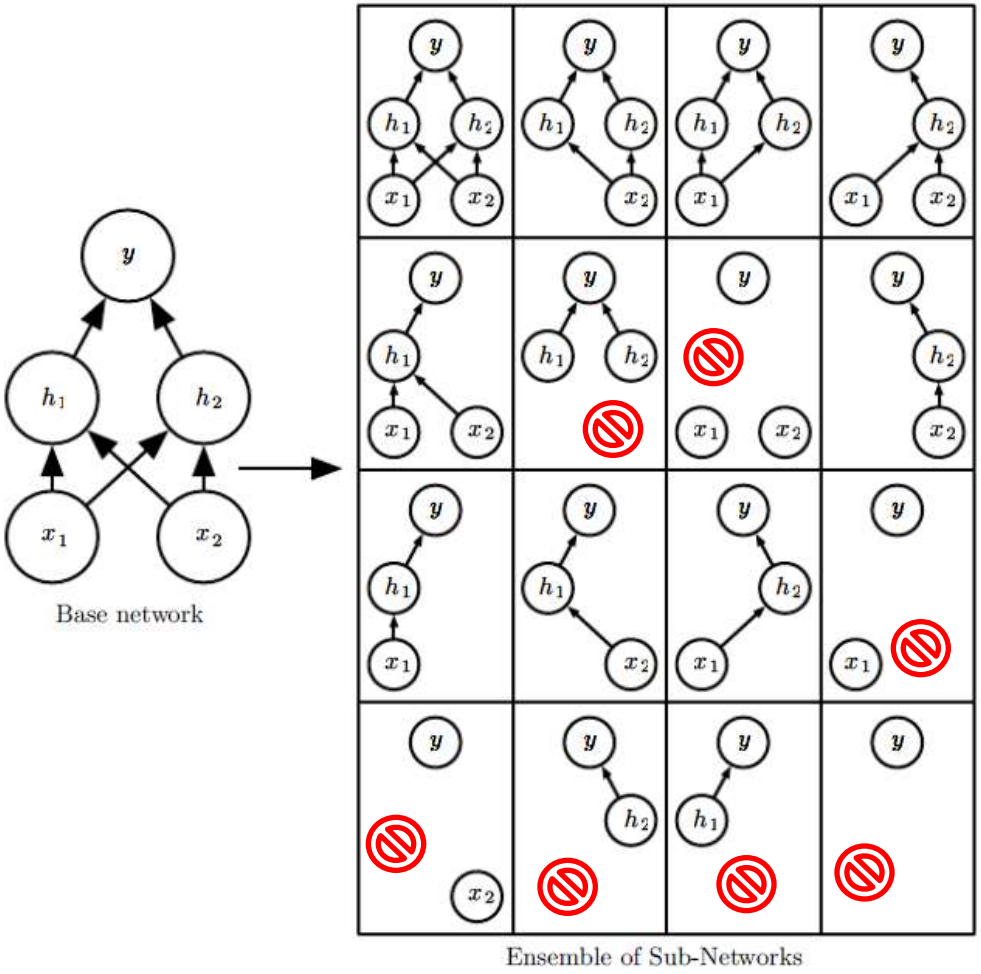
DROPOUT aims to approximate (and exploit) this process,

- but with an exponentially large number of (sub) neural networks
  - while a test time we will have still only a single network!
- and also aims at maximize the diversity of the ensemble preventing complex co-adaptations on training data

Dropout: picture

Dip. Informatica
University of Pisa

- Dropout trains the ensemble consisting of all sub-networks that can be formed by removing non-output units from an underlying base network
- In the example some networks cannot work
- With larger network the probability of no path from input to output is low.



Dropout Basic Algorithm

Dip. Informatica
University of Pisa

- Each time we load an example into a minibatch, we randomly sample a different binary mask to apply to all of the input and hidden units in the network.
  - The mask for each unit is sampled independently from all of the others.
  - The probability of sampling a mask value of one (causing a unit to be included) is a hyperparameter fixed before training begins. Typically, e.g. **0.8** for input units, **0.5** for hidden units
  - Each unit in the network is multiplied by the corresponding mask, (zero is like removing, "**drop out**", the unit → build sub-networks)
- We then run forward propagation, back-propagation, and the learning update as usual (training a subset of units at time).
- The removed units are then reinserted into the network with their original weights.

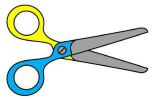
Dropout insights

Dip. Informatica
University of Pisa

- **Parameter sharing** among sub-networks: only a small fraction of the possible sub-networks are each trained for a single step (many are not explicitly trained per-se), but the *parameter sharing* causes the remaining sub-networks to arrive at good settings of the parameters.
  - Aka Many paths sharing the same links….
  - This allows us to treat an exponential number of models with a tractable amount of memory
  - 10-20 masks are typically enough!

- **Approximation to the predictions of the entire ensemble:** To do it better, *weight scaling inference rule*: multiple the weights with probability of inclusion (typically 0.5) to estimate the expected unit output during forward phase
  - Simulate what happens during training for each unit input (expected value of the output of any unit is the same as in the training stages)
  - No other details….





Dropout and Regularization

Dip. Informatica
University of Pisa

- Dropout regularization effect:
  - avoiding training all units on all training data, reduce units interactions (and diversify as useful for committees)
  - variance reduction as for bagging
  - insert structured noise (recognize a face with the hidden unit for the nose, compare with standard regularization by adding deformed inputs with random noise)

  Moreover, interestingly it *regularizes* each hidden unit to be not merely a good feature but a feature that is good in many contexts (different sub-networks).

- It can be show that for linear regression is equivalent to L2 weight decay, using different lambda for each input [Wager et al. 2013]

- Can be used for any model that uses *distributed representation* and SGD training
- Can be extended to any kind of random modification (not only to exclude units).

L1 regularization

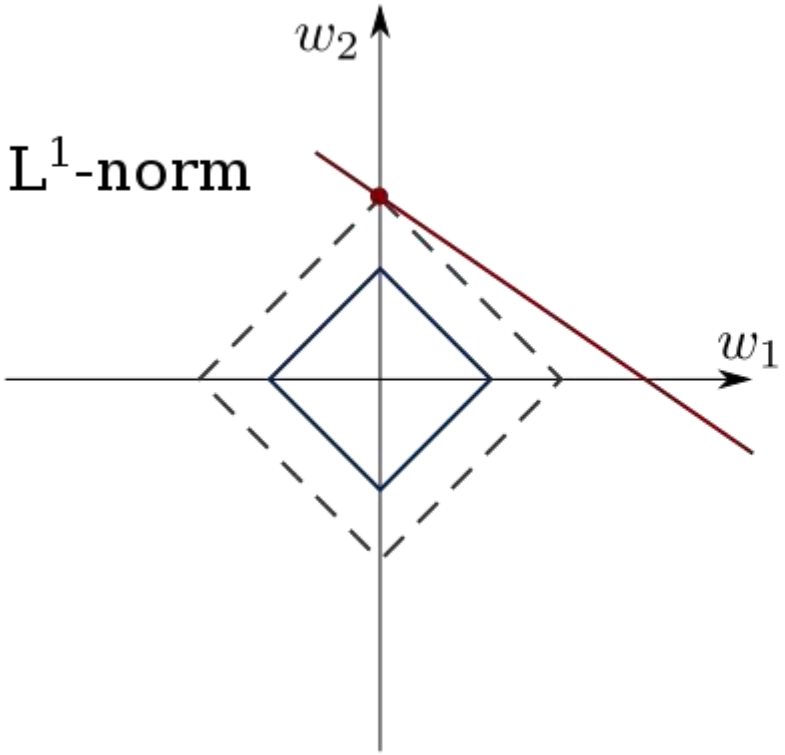
Dip. Informatica
University of Pisa

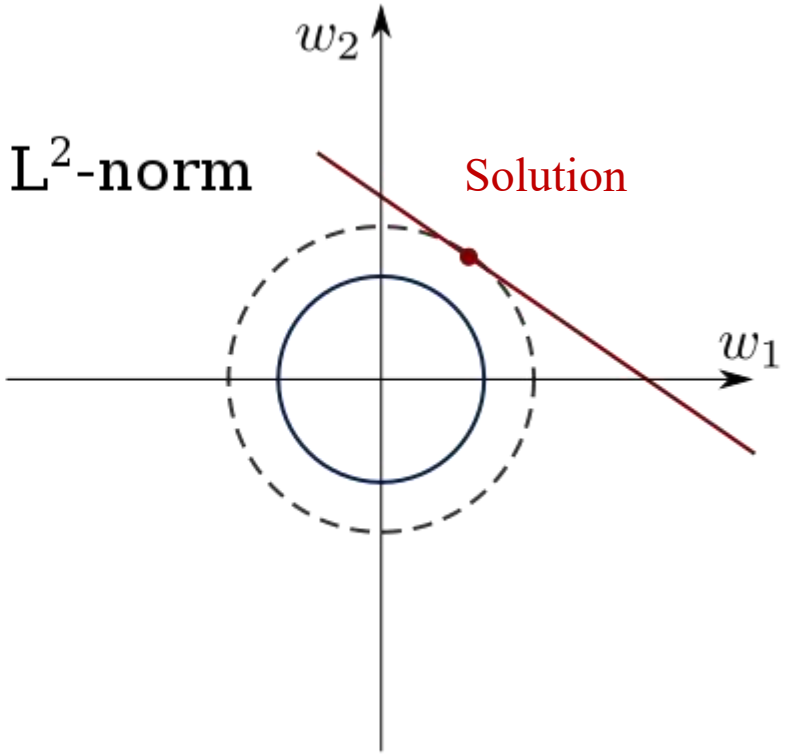
- Use L1-norm (sum of abs. values) instead of L2-norm in the penalty term
- LASSO alg. for linear models: it favorites *features elimination* (weight zero) wrt to L2 (which often only shrinks the size of the weights without setting any of them to zero) producing a simpler final models
- Solution can be approximated but in general requires non-differentiable optimization techniques.
  - the extension to non linear-modes, like NN, needs special approaches, e.g. smooth approximation of the L1 norm, etc
  - nondifferentiable optimization, sub-gradient, smoothed gradient approaches ... can be seen in the CM course
- This is a general topic NOT related to DL (we left it many lectures ago).
- For this year we refer to the CM course



L1 vs L2 regularization

Dip. Informatica
University of Pisa





Adversarial Learning

Dip. Informatica
University of Pisa

- Adversarial training
- **Generative adversarial network**
  - two neural networks contesting with each other
  - one network generates candidates and the other evaluates them.
- E.g. 2017: astronomical images and many other applications
- An example of possible SEMINAR at the end.



GAN producing photorealistic images of imaginary celebrities

Dip. Informatica
University of Pisa

These people do not exist!



- https://research.nvidia.com/publication/2017-10_Progressive-Growing-of

Deep Learning- guideline

Dip. Informatica
University of Pisa

- Basic (or rapid introduction)
- Insights (do we need many layers?)
- Techniques (some)
- **Applications** → further courses in AI (ISPR, NLP, …)
- And **Discussion**

Successful applications in …

Dip. Informatica
University of Pisa

Aka consider DL for

- NLP (including speech recognition, machine transition, sentiment analyses, question answering…)
- Computer vision (processing images and video, with many tasks including object recognition, and many many applications areas from medical applications to image synthesis, …) with performances never achieved with previous methods.
- GO game solved with DL

- More at the **NLP** and **ISPR** courses (e.g. in the AI curricula)

Challenges

Dip. Informatica
University of Pisa

- Theoretical analysis and framework
- Optimal architecture design, e.g.
  - How many layers? Which type of units is the best? … (as empirical CV approach could be excessively expensive in this context)
- Efficiency for big data
- Processing streams of data (on-line)
- Structured data (sequences, trees, graphs)
- …

DL conclusions

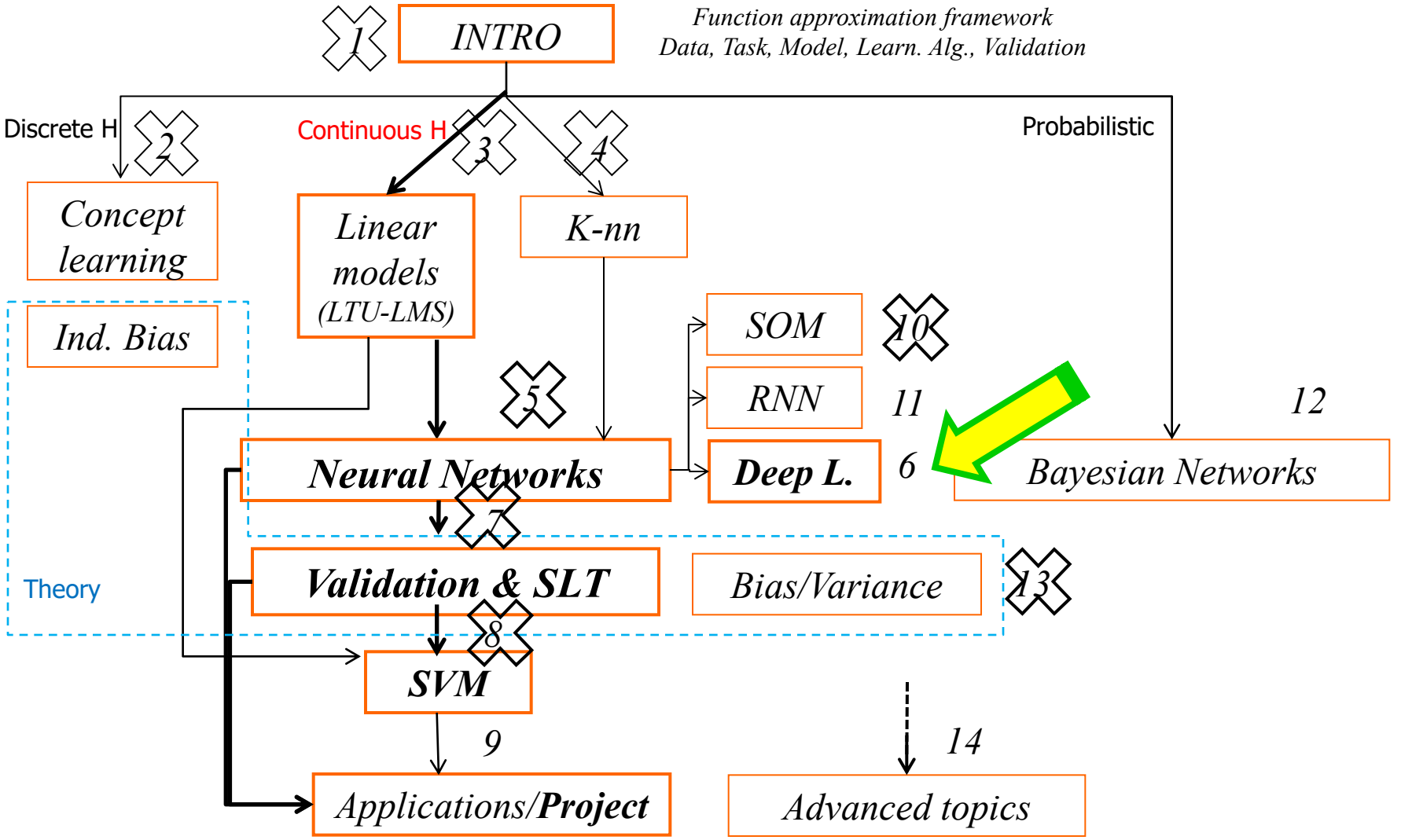
Dip. Informatica
University of Pisa

- The recent **technical/data** advancements made possible to learn with deep and very large NN/models
- Many **successful applications** have been developed, whereas DL provides the state-of-the-art solution (especially where compositionality for features is relevant: visual, language, …)
- The insights suggest (two different) **potential advantages** (even exponential) in the representation and learning capability of such construction
- The **theoretical studies** and the extension of the **conceptual frameworks** (e.g. representational learning) and of the **models** and **applicative domains** are growing at an unprecedented rate

- Globally, it is becoming more and more evident how Machine learning makes it relatively easier to develop sophisticated AI (but more in general software) systems working for **real-world solutions**

ML Course structure
Where we go

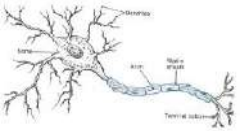
Dip. Informatica
University of Pisa



Course structure: zoom in the NNs box

NEW

Dip. Informatica
University of Pisa



1 unit NN

Perceptron

Adaline

LMS

As a linear model (previous lects)

Neural Networks

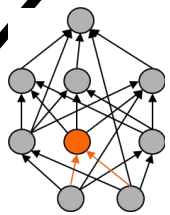
MLP: Mack. &Pits

LEARNING: Perceptron learning alg + Convergence theorem

Non-linear out (f)

LEARNING: Backprop

MLP: feed-forward NN

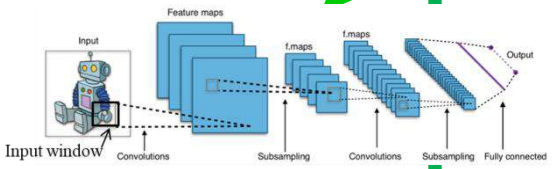


Heuristics for BP

CasCor

Regularization

We are here



Applications: **CNN** example

Introduction to recent NN paradigms (Deep, **Random.**)

Made later, after the PRJ (to favorite validation, SVM, …)

Other architectures (guide slide)

Dip. Informatica
University of Pisa

Introduction to recent NN paradigms

- Other architectures…..
  - Deep learning
  - **Random weights NN**

AKA: *Deep* learning and *extreme* learning ;-)



Randomized Machine Learning & Random weights NNs



Introduction

Dip. Informatica
University of Pisa

A randomized approach employs a degree of randomness as part of its constructive strategy



- **Randomness** can be viewed as an **effective part** of **ML approaches**
  - Randomization can **enhance** several aspects of ML **methodologies**
    - data processing, learning, hyper-parameter selection
  - Randomization as a key factor for **building** efficient **ML models**

- **Focus**: Randomized Neural Network Approaches

Randomness in Machine Learning

Dip. Informatica
University of Pisa

Randomness can be present at different levels:

- To **enhance** predictive **performance**
- To **alleviate difficulties** of classical ML methodologies

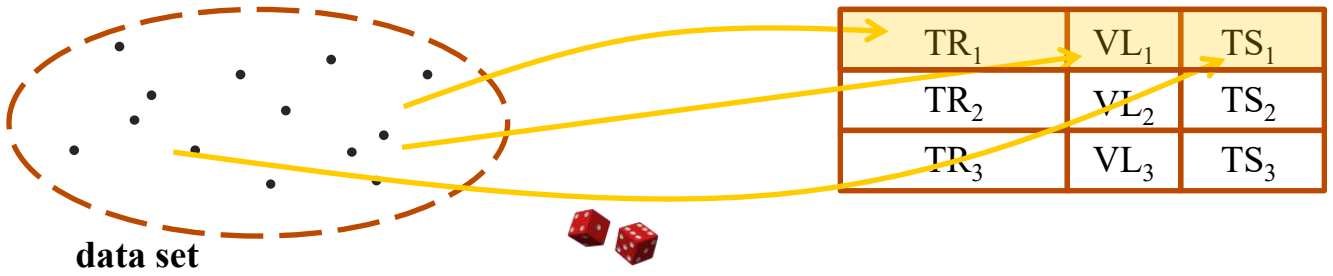
Randomness in Specific Aspects of ML:

- Data Processing
- Learning and hyper-parametrization selection
- In the core design of the model

Randomness in Machine Learning

Dip. Informatica
University of Pisa

- **Data splitting**
  - Hold-out, k-fold cross validation, leave-one-out

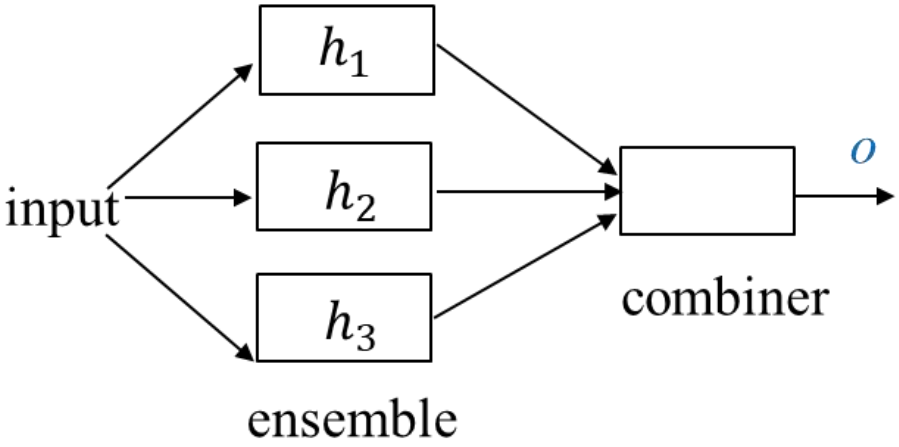


- **Data Generation**
  - Bootstrap, Boosting, Balancing techniques
- **Learning**:
  - e.g. observation order (shuffling for mini-batch)
  - Random initialization of weights in NN
  - …
- **Hyper-parameters** selection (including random search)
- …

Specifically: In the enesmbling

Dip. Informatica
University of Pisa

- Random differentation among models, e.g.:
  - different weights init. in NN,
  - (partial) randomness for sampling different TR sets in bagging and boosting,
  - …

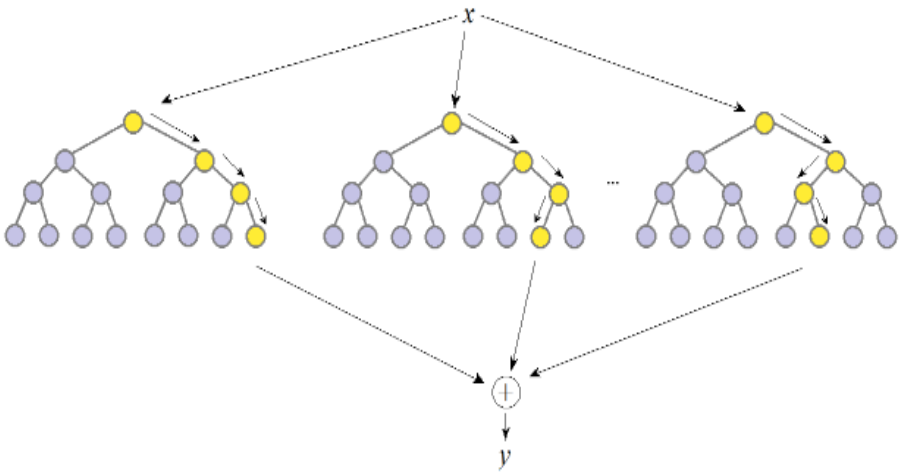


Intrinsically Random Models

Dip. Informatica
University of Pisa

Randomness as a key element **of the ML model**

- Random neurons of Restricted Boltzmann Machines
- Extremely Randomized Trees, **Random Forests**





- Pool/ensemble of randomized decision trees
- Each Decision Tree is built from samples randomly drawn from the training set



- Randomized selection of the input variables to choose for splitting the nodes
- **Ensembling + Randomness advantages**

- Regularization: **Drop-out** for NN training (even if in this case randomization is not necessary per-se)

Random weights NNs: overview

Dip. Informatica
University of Pisa

Is possible to exploit the **MLP/RNN architectures** even **without** long training cycles in the hidden layers?

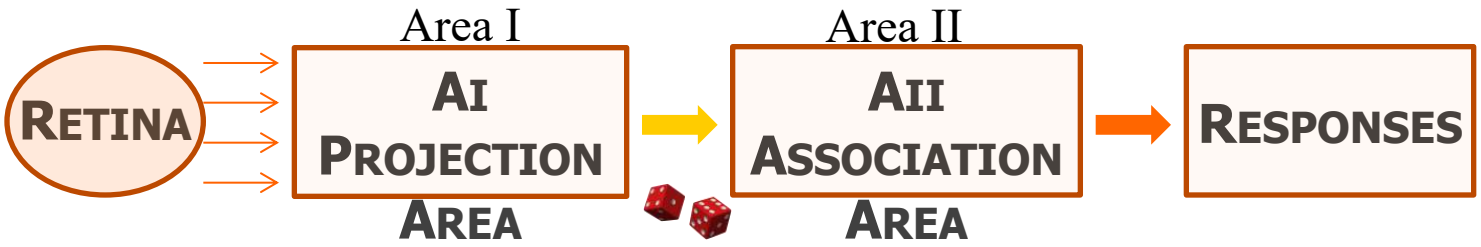
- Long tradition for *randomized NN*, *random weights NN*, *Random projection architectures*, e.g. Random Vector Functional Link Neural Network (RVFL) … since its origin
- Recently, new popularization with various names, including *ELM (Extreme Learning Machine)*
- Basic idea:
  - A network containing a **randomly** connected hidden layer(s), i.e. weights are **fixed** after random initialization
  - train **only** the output weights (linear model: by pseudoinverse/ridge regression)

  overcoming the problems associated with the complex and computationally demanding training algorithms traditionally used to train neural networks.

Randomized Neural Network Approaches

Dip. Informatica
University of Pisa

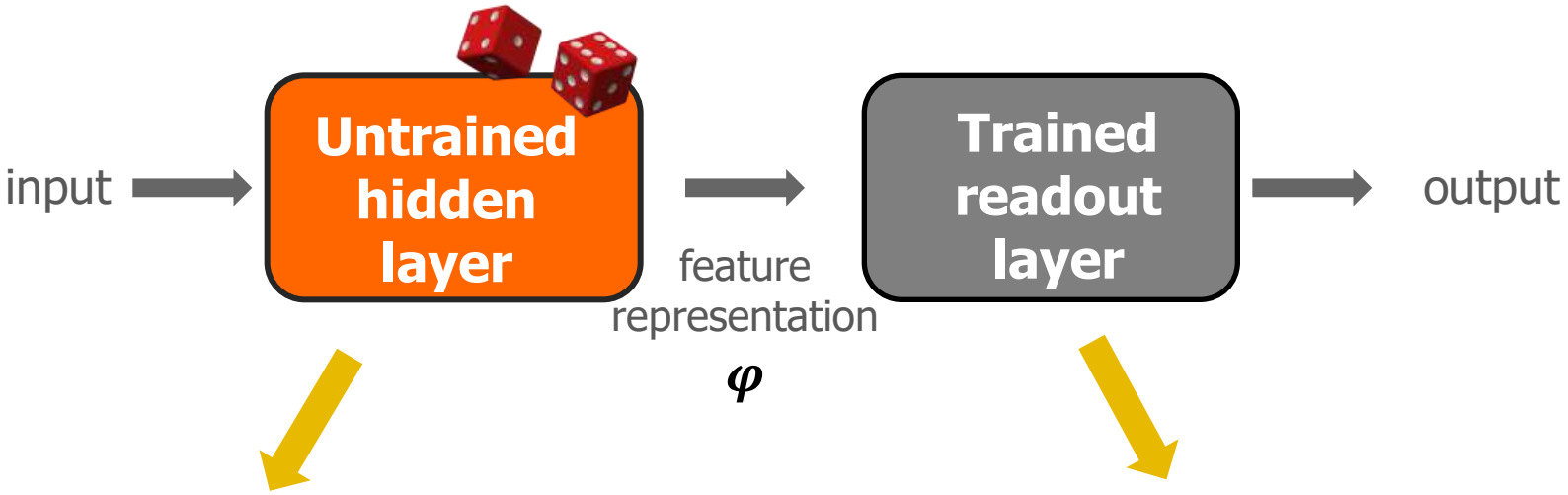
- In the Neural Network (NN) area randomness gave rise to a rich set of models
- **Roots are in the pioneering work on the Perceptron**



- Other pioneering works pointed out the **relevance of learning in the output layer** w.r.t. the hidden layer
- **Recently** randomized NNs delineate a **research line** of **growing interest**
- Particularly suitable when **efficiency** is of primary relevance (since training is restricted to the last layer!)

General Architectural Structure

Dip. Informatica
University of Pisa



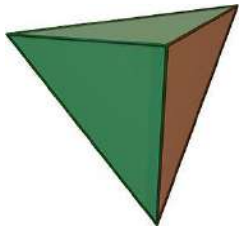
- **Non-linearly** embed the input into a **high-dimensional feature space** where the problem is more likely to be solved linearly (**LBE**)
- **Theoretically grounded** basis (Cover's Th. → next slide)

- Combine the features in the hidden space for **output computation**
- Typically implemented by linear models

Cover Theorem

Dip. Informatica
University of Pisa

- *"A complex pattern-classification problem, cast in a high-dimensional space nonlinearly, is more likely to be linearly separable than in a low-dimensional space, provided that the space is not densely populated".*
  - In particular, using a deterministic mapping: Assuming l examples, lift them onto the vertices of the l-1 dim triangle (simplex). Now, every binary partition of the samples is linearly separable



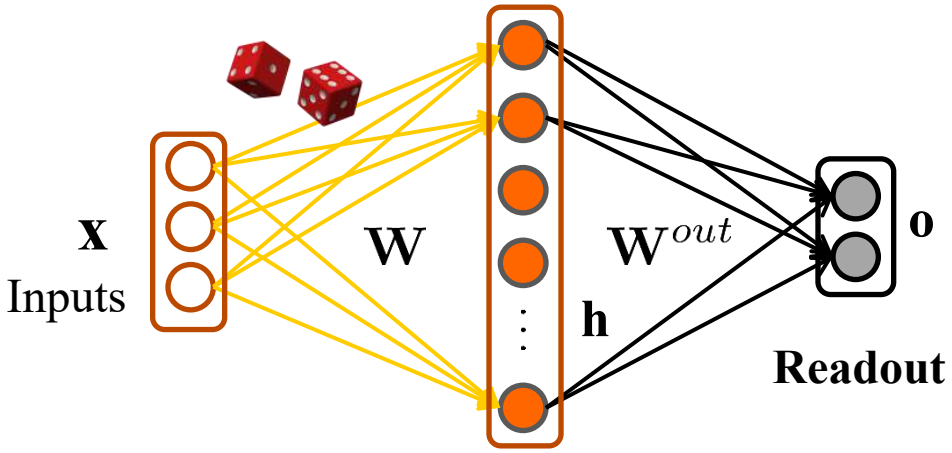
Cover, T.M., *Geometrical and Statistical properties of systems of linear inequalities with applications in pattern recognition*, IEEE Transactions on Electronic Computers, **1965**.

- Note: The same property exploited for kernels !!!

Feed-forward Randomized NNs

Dip. Informatica
University of Pisa

- Input-output relation is implemented through an **untrained** hidden layer
- The hidden layer implements **randomized** **large** **basis expansion** !



$\mathbf{W}$ hidden layer weight matrix

$\mathbf{W}^{out}$ readout weight matrix

f hidden layer non-linearity

TO BUILD:

- Fill $\mathbf{W}$ with random values (e.g., Gaussian random noise) (using many units)
- Training for the Readout (see the next slide)

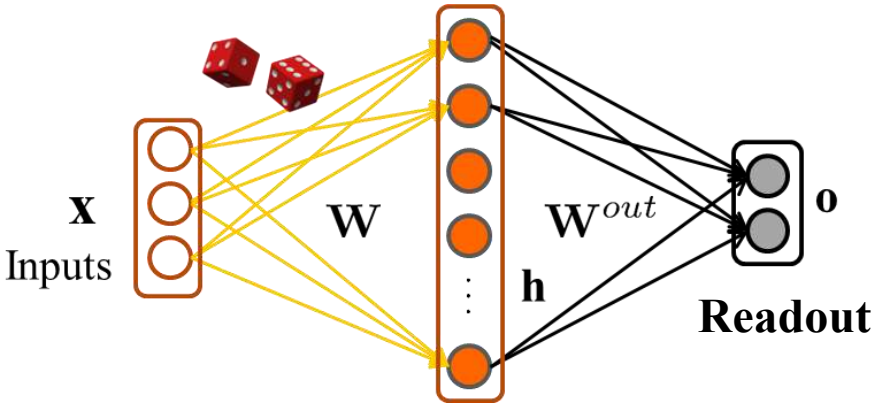
TO USE:

- Compute the hidden layer activations $\mathbf{h}$ as usual for neural units
- Compute the outputs $\mathbf{o}$ resulting in: $\mathbf{o}(\mathbf{x}) = \mathbf{W}^{out} f(\mathbf{W}\mathbf{x})$
- Apply a threshold (or not linear functions if needed for classification)

Feed-forward Randomized NNs

Dip. Informatica
University of Pisa

- Input-output relation is implemented through an **untrained** hidden layer
- The hidden layer implements **randomized** **basis expansion** !



$\mathbf{W}$ hidden layer weight matrix

$\mathbf{W}^{out}$ readout weight matrix

f hidden layer non-linearity

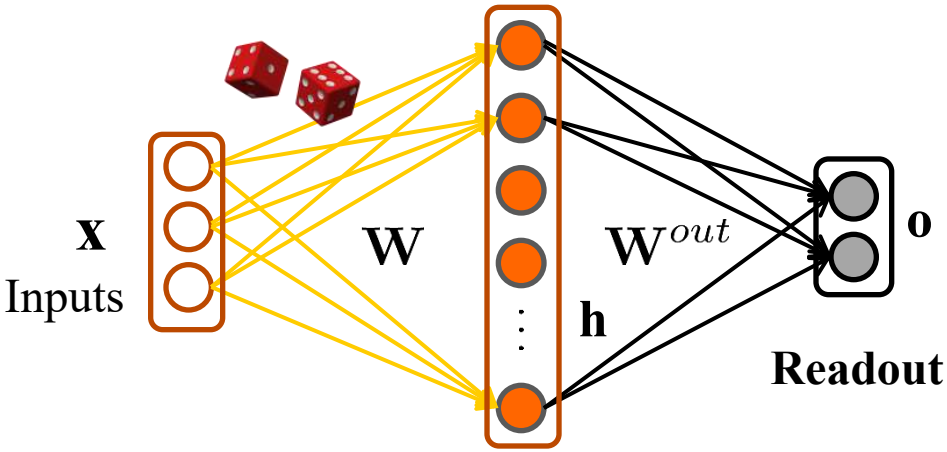
TO LEARN: given a set of l training example $(\mathbf{x}_p, \mathbf{d}_p)$, as we did for linear models: $argmin_{\mathbf{w}}\ Error(\mathbf{W}^{out})$ *in* L_2 by the direct method:

- Estimate $\mathbf{W}^{out}$ by least-squares fit (LMS) computed using the pseudoinverse: $\mathbf{W}^{out} = \mathbf{H}^{+}\mathbf{d}$, where $\mathbf{H} = f(\mathbf{WX})$
- OFTEN implemented with the L_2 *regularization* (useful due to the high number of units): $\mathbf{W}^{out} = (\mathbf{H}^T\mathbf{H} + \lambda\mathbf{I})^{-1}\mathbf{H}^T\mathbf{d}$
- or also by L1

Feed-forward Randomized NNs

Dip. Informatica
University of Pisa

- Input-output relation is implemented through an **untrained** hidden layer
- The hidden layer implements **randomized** **basis expansion (φ)**



$\mathbf{W}$ hidden layer weight matrix

$\mathbf{W}^{out}$ readout weight matrix

f non-linearity of the expansion

Major reference literature **models**:

- Random Vector Functional-Link (**RVFL**)
- Extreme Learning Machine (**ELM**)
- Radial Basis Function (**RBF**) networks **with random centers**

Pro/Cons

Dip. Informatica
University of Pisa

- Large set of hidden unit plays an interesting role: projection non-linearly expands the dimension of an input vector so as to make the data more separable
- A large number of units can provide a sufficient "basis expansion" to solve the task at hand (with a theoretical ground, including universal approx. theorems)
- *Extremely* ***efficient***
- Many variants and improvement exists, e.g.:
  - incremental learning adding hidden neurons (see CC)
  - pruning, etc.

  They are often linked to past researches with a different name.

- Of course focused models, with smaller number of *adaptive* units (*basis functions*) trained on the specific task at hand appear still "more elegant" (there is a debate, but at least it is worth for you knowing also about this "**random features**" versus "learned features" approach)

- Extended to RNN → ESN (next lectures and CNS course)

Useful for

Dip. Informatica
University of Pisa

- **Big Data** (due to extreme efficiency)
- **Tiny implementations** suitable for distributed embedding e.g. in resource-limited devices, e.g. WSNs, IoT, edge computing…
- **Deep Learning,** where training algorithms are particularly demanding
  - Randomized NNs can **alleviate the computational burden** of training
  - **Deep Feed-forward Randomized NN**: Networks consisting in stacks of RVFL/ELM modules (e.g. TNNLS 2016)
  - Also randomized **CNN** approaches
- **Recurrent Randomized NN**
  - where efficiency of training is more demanding (wrt feedforward MLP)
  - with a solid ground on the introduced bias by *system dynamic* analysis (e.g. related to stability, learnability, etc.)
  - (we will introduce RNN in a next lecture)

Conclusions

Dip. Informatica
University of Pisa

- Randomness is <u>not</u> an enemy,
  it is a friend if you control it
- Also in other fields of CS (randomized algorithms is a research field), in Math (probability, also probalistic ML approaches)

- Note that we are starting to see many times the same concepts (e.g. Cover theorem, LBE, regularization, feature representation, …):
- The course is now «mature»

FAQ

Dip. Informatica
University of Pisa

- **Can ELM be used for the project?**
  Yes, but as an additional part to the implementation of the standard MLP with SGD (fully trained with backprop). Easy to be implemented as a subpart of the fully trained FFNN/MLP.

More information

Dip. Informatica
University of Pisa

Examples (**ELM**):

- G.B. Huang, Q.Y. Zhu, C.K. Siew, "Extreme learning machine: theory and applications", Neurocomputing, vol. 70 (1), pp.. 489-501, 2006.

  (just to have a reference, even if criticism on the mathematical part exists)
- G.B. Huang, Q.Y. Zhu, X. Ding, and R.Zhang. "Extreme Learning Machine for Regression and Multiclass Classification". *Trans. Sys. Man Cyber. Part B* 42, 2), pp. 513-529, 2012. (ELM + regularization (also with SVM))

Short Surveys:

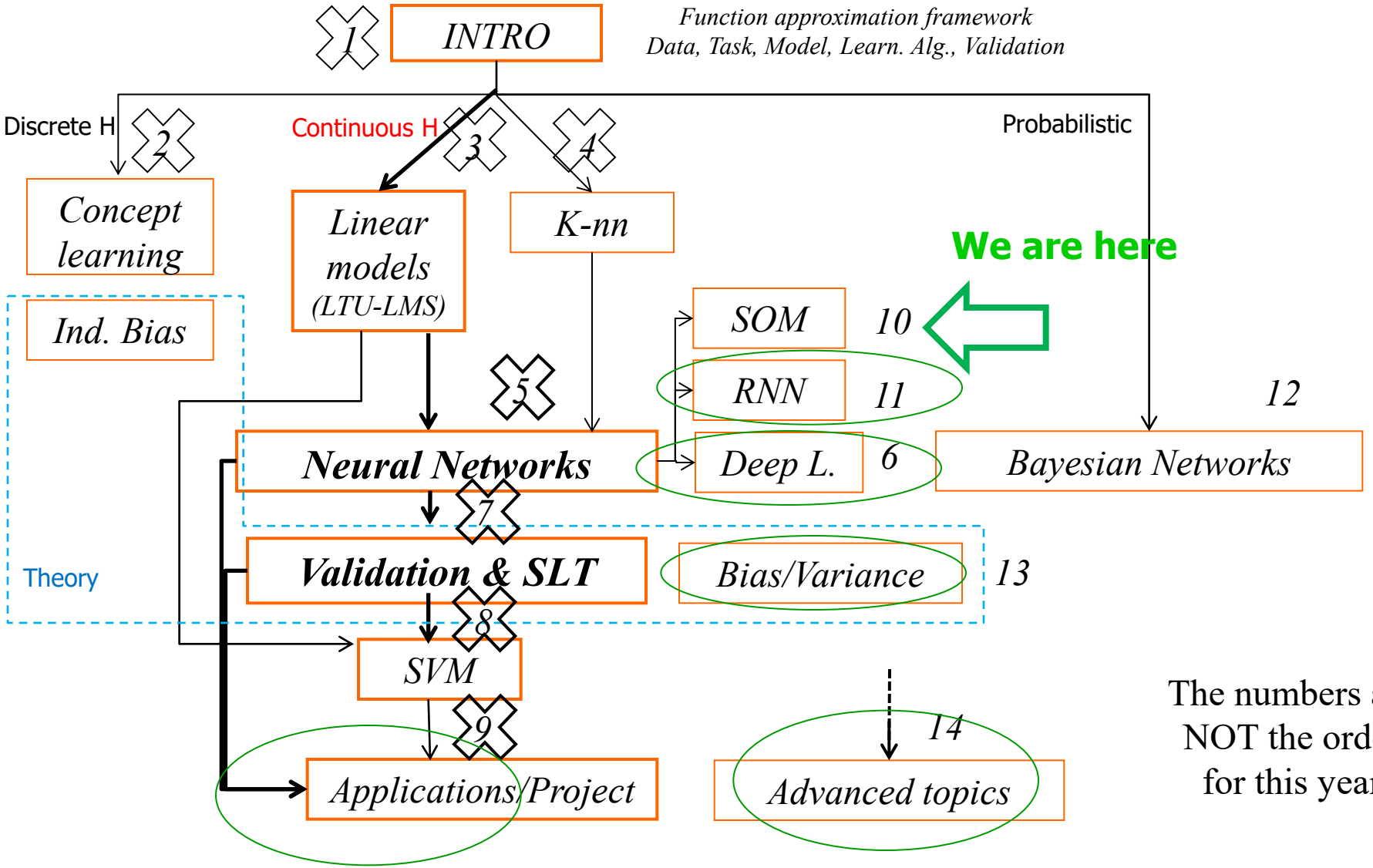
- Y. Miche, B. Schrauwen, A. Lendasse, "Machine Learning Techniques based on Random Projections", ESANN 2010 proceedings, 295-302, 2010.
- C. Gallicchio, J.D. Martín-Guerrero, A. Micheli, E. Soria-Olivas, "Randomized Machine Learning Approaches: Recent Developments and Challenges" ESANN 2017, pp. 77-86

Recent Surveys:

- S. Scardapane, W. Dianhui. "Randomness in neural networks: an overview." Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery 7.2 (2017): e1200.
- Le Zhang, P.N. Suganthan. "A survey of randomized algorithms for training neural networks." Information Sciences 364 (2016): 146-155.
- C. Gallicchio, S. Scardapane. "Deep randomized neural networks." Recent Trends in Learning From Data (2020): 43-68. Springer.

ML Course structure
Where we are

Dip. Informatica
University of Pisa



The numbers are NOT the order for this year

Tasks: Unsupervised Learning

Dip. Informatica
University of Pisa

Unsupervised Learning: No teacher!

- TR = set of unlabeled data <***x***>
  - Clustering: find *natural groupings* in a set of data
  - Dimensionality reduction/ Visualization/Preprocessing: multidimensional data are projected in a lower dimensional space
    - Principal Component Analyses, Multi-Dimensional Scaling, Independent Component Analysis, …
  - Modeling the data density (not in this course)

Utility

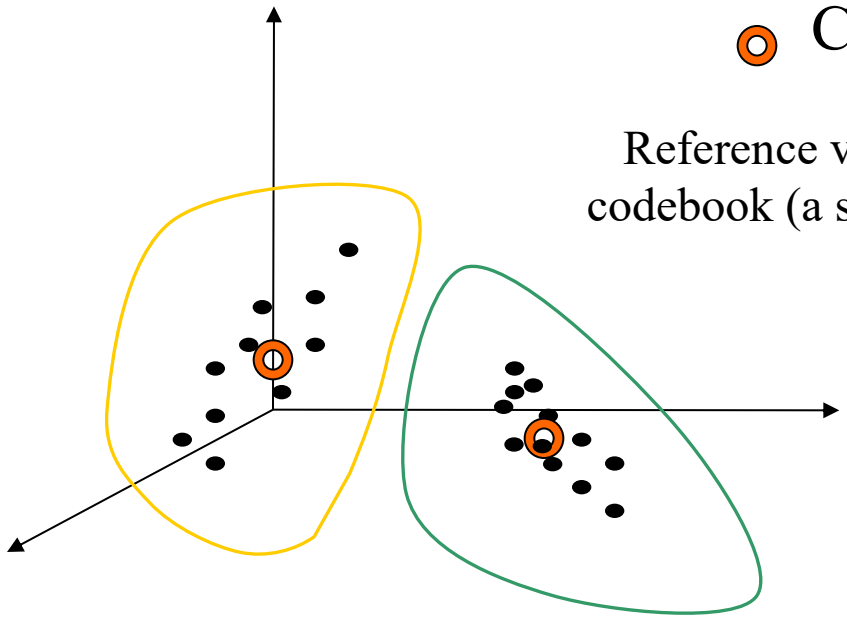
- Example (Taxonomy): organize life forms: what are mammals ?
- Example (human learning): babies learn to recognize ("cluster") familiar faces before they can understand human speech
- Unlabeled data is much cheaper to obtain
  - Examples: e.g. 10 million images (sampled frames from YouTube) with state-of-the-art results on object recognition (22,000 categories)

Clustering

Dip. Informatica
University of Pisa

Partition of data into clusters (subsets of "similar" data):

patterns within a valid cluster are more similar to each other than they are to a pattern belonging to a different cluster



Centroids (or prototypes)

Reference vector, centroid, cluster center, codebook (a set of codevectors) , or prototype vector, …

A guideline

Dip. Informatica
University of Pisa

- We mainly focus on Clustering/Vector Quantization
- The clustering problem has been addressed in many contexts and by researchers in many disciplines;
- *Thousand of different algorithms* !
  - this reflects its broad appeal and usefulness as one of the steps in exploratory data analysis.
  - **QUESTION**: how many of you already know the "K-means" **?**
- We don't try to list them, but we go directly to a simple approach and we have the opportunity to look at a historical NN models:
  - the **SOM/Kohonen maps** !
- We use a Vector Quantization perspective
- for Partitioning clustering

A premise: Similarity Measures

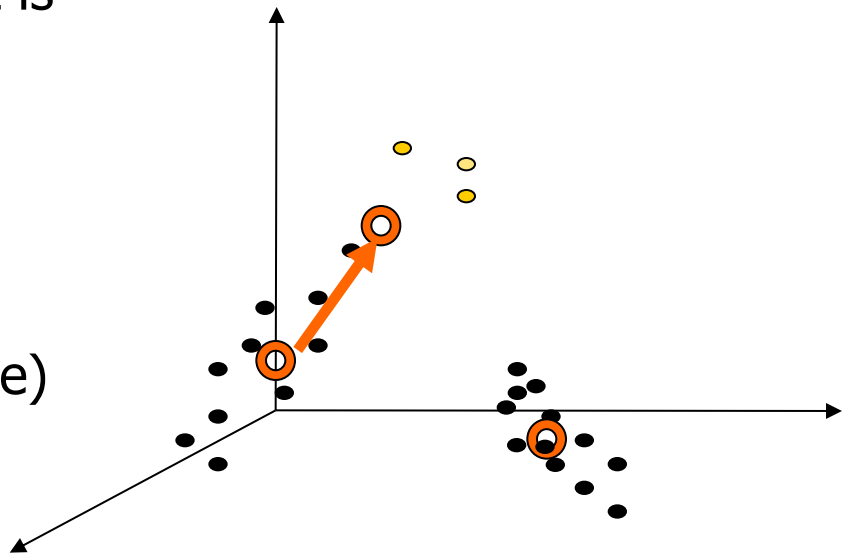
Dip. Informatica
University of Pisa

- Metric dominance ! The assumed distance tell us which are the similar examples (Pattern proximity)
- The clustering algorithm decides the grouping basing on that metric

- There are many possible metrics
- Although Euclidian metric is popular it is just a choice !
- *See Nearest Neighbored lecture !*
- May have more deep implications:

  Soft outlier weight by using

  (1-Gaussian centered in the prototype)



Vector Quantization

Dip. Informatica
University of Pisa

- Vector quantization techniques encode a data manifold, e.g.,

a submanifold $V \subseteq R^n$, utilizing only a finite set $\boldsymbol{w} = (\boldsymbol{w}_1, \ldots, \boldsymbol{w}_K)$ of reference or "codebook" vectors $\boldsymbol{w}_i \in R^n$, $i = 1, \ldots, K$.

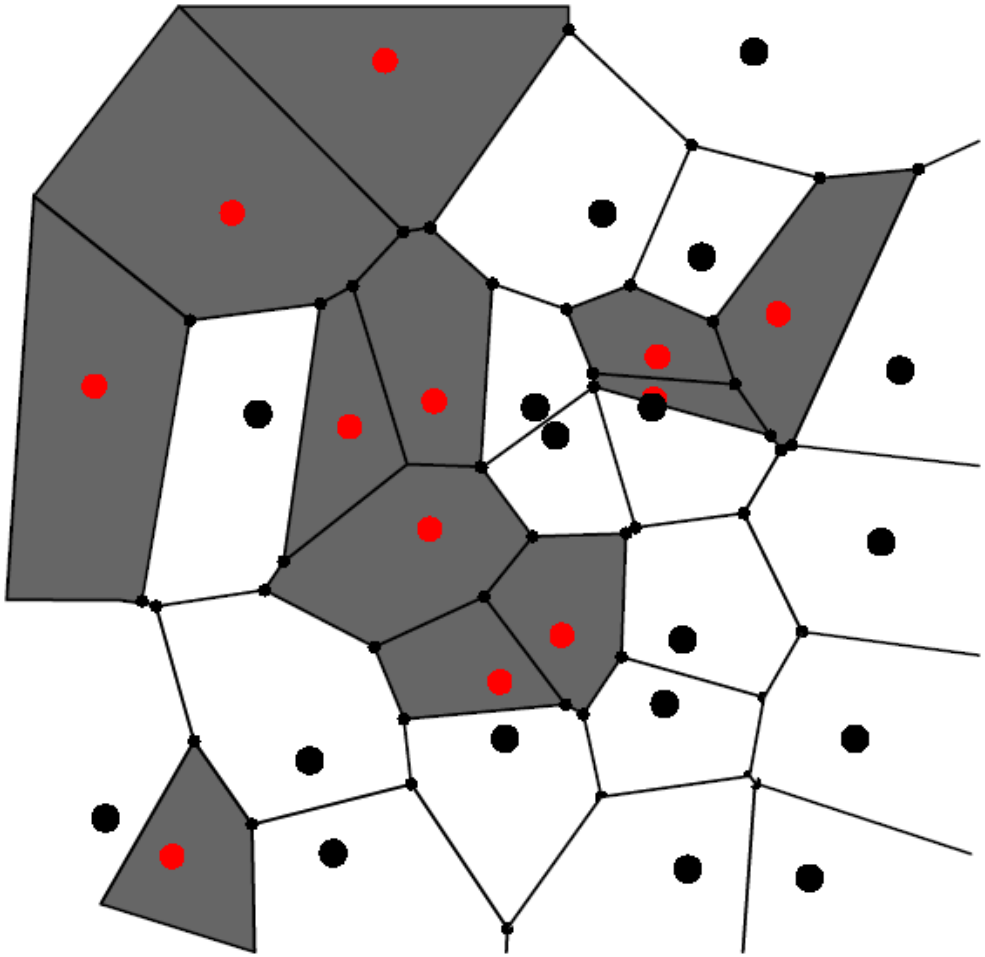
- A data vector $\boldsymbol{x} \in V$ is described by the best-matching or "winning" reference vector $\boldsymbol{w}_{i^*(\boldsymbol{x})}$ of $\boldsymbol{w}$ for which the *distortion error* $d(\boldsymbol{x}, \boldsymbol{w}_{i^*(\boldsymbol{x})})$ is minimal
- This procedure divides the manifold V into a number of subregions

$$V_i = \left\{ \boldsymbol{x} \in V \; s.t. \left\| \boldsymbol{x} - \boldsymbol{w}_i \right\| \leq \left\| \boldsymbol{x} - \boldsymbol{w}_j \right\| \forall j \right\}$$

- called Voronoi polyhedra, out of which each data vector $\boldsymbol{x}$ is described by the corresponding reference vector $\boldsymbol{w}_{i^*(\boldsymbol{x})}$

Voronoi diagram

Dip. Informatica
University of Pisa



Each cell consisting of all points closer to *the bold* ***black/red points*** than to any other patterns.

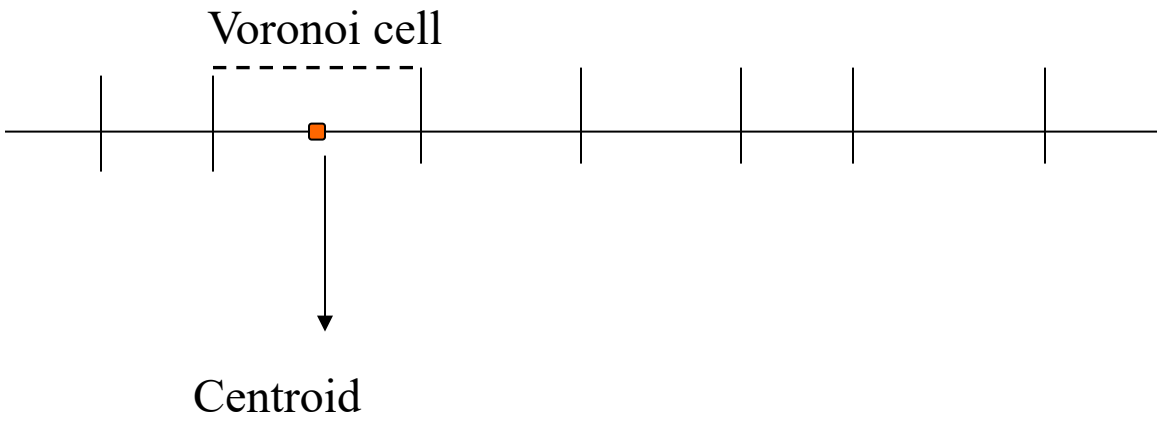
The segments of the Voronoi diagram are all the points in the plane that are equidistant to two patterns.

NOTE: for us the problem is: **given the dataset, find centers such that MIN Loss E**

In general Hard to be computed:
Optimal vector quantization codebook is an NP-complete problem

Example: Quantization in 1D

Dip. Informatica
University of Pisa



- E.g. In digital signal processing, **quantization** is the process of approximating ("mapping") a continuous range of values (or a very large set of possible discrete values) by a relatively small ("finite") set of ("values which can still take on continuous range") discrete symbols or integer values
  - so we have a quantization/distortion error
- *Music in your digital device* !!!!

VQ and Clustering

Dip. Informatica
University of Pisa

- Note that clustering has a more general objective of finding *interesting/useful* groupings of data
- *Interesting* is often implicitly defined via the computational procedure by itself and not necessarily measured by the minimum Quantization error
- However, VQ provides an useful framework, and VQ algorithms can be (and are often) used for clustering and we use this nice perspective

Clustering and Vector Quantization

Dip. Informatica
University of Pisa

- Goal: optimal partitioning of unknown distribution in *x*-space into regions (clusters) approximated by a cluster center or *prototype*.
- I.e a set of vector quantizers $\boldsymbol{x} \rightarrow c(\boldsymbol{x})$ *or* $\boldsymbol{w}_{i^*(\boldsymbol{x})}$
  continuos space → discrete space (the number of centers)
- considering the squared error distortion

- We evaluate $d(\boldsymbol{x}_i, c(\boldsymbol{x}_i)) = \left\| \boldsymbol{x}_i - c(\boldsymbol{x}_i) \right\|^2$
- The average value over the distribution of inputs is the average (expected) distortion or reconstruction or quantization error. This is our *Loss function*
- We use $\boldsymbol{w}$ as symbol for the prototype vectors

Quantization error function

Dip. Informatica
University of Pisa

$$E = \int f(\mathrm{d}(\boldsymbol{x}, \boldsymbol{w}_{i^*(\boldsymbol{x})}))\, p(\boldsymbol{x}) d\boldsymbol{x} = \int \left\| \boldsymbol{x} - \boldsymbol{w}_{i^*(\boldsymbol{x})} \right\|^2 p(\boldsymbol{x}) d\boldsymbol{x}$$

(squared error criterion)

$$E = \sum_{i}^{l} \sum_{j}^{K} \left\| \boldsymbol{x}_i - \boldsymbol{w}_j \right\|^2 \delta_{winner}(i, j)$$

Discrete version

$\delta_{winner}(i, j)$ characteristic function of the receptive field of $\boldsymbol{w}_j$
1 if j is the winner for $\boldsymbol{x}_i$, 0 otherwise

$p(\boldsymbol{x})$ is the probability distribution of $\boldsymbol{x}$

Minimize E

Dip. Informatica
University of Pisa

$$E = \int f(\mathrm{d}(\boldsymbol{x}, \boldsymbol{w}_{i^*(\boldsymbol{x})}))\, p(\boldsymbol{x}) d\boldsymbol{x} = \int \left\| \boldsymbol{x} - \boldsymbol{w}_{i^*(\boldsymbol{x})} \right\|^2 p(\boldsymbol{x}) d\boldsymbol{x}$$

Minimizing E (in $\boldsymbol{w}$)→ vector quantization

- Minimize E through an optimal choice of $\boldsymbol{w}$
- The set of reference vectors $\boldsymbol{w}$ that minimizes E is the solution of the VQ problem

- *Note:* the integrand of E is not continuously differentiable:
  - Indeed $\boldsymbol{w}_{i^*(\boldsymbol{x})}$ have ***discrete*** changes (w.r.t. $\boldsymbol{x}$).
  - However, we compute the derivatives locally (fixing $\boldsymbol{x}$ on a Voronoi cell) …

On-line K-means

Dip. Informatica
University of Pisa

$$E = \sum_i \sum_j \left\| \boldsymbol{x}_i - \boldsymbol{w}_j \right\|^2 \delta_{winner}(i, j)$$

- Taking the derivatives of E with respect to the $\boldsymbol{w}_j$ yields the learning rule of VQ
- Lloyd and MacQueen's well-known K-means clustering algorithm (on-line version) for any $\boldsymbol{x}_i$

$$\Delta \boldsymbol{w}_{i^*} = \eta \delta_{winner}(i, i^*)(\boldsymbol{x}_i - \boldsymbol{w}_{i^*})$$

Learning rate

- Note that we change only the winner prototypes, adapting it to $\boldsymbol{x}$
  - A stochastic gradient descent → local minima
  - Method dependent on initialization
  - Moreover, the number of clusters to find must be provided

Basic K-means Batch Alg.

LBG(Linde, Buzo, Gray) or generalized Lloyd algorithm

Dip. Informatica
University of Pisa

(1) Choose *k* cluster centers to coincide with *k* randomly-chosen patterns or *k* randomly defined points inside the hypervolume containing the pattern set.

(2) Assign each pattern to the closest cluster center (the winner)

(3) Recompute the cluster centers (geometric centroid, i.e. **mean**) using the current cluster memberships

(4) If a convergence criterion is not met, go to step 2. E.g.

- no (or minimal) reassignment of patterns to new cluster centers,
- minimal decrease in squared error.

Exercise: from on-line to batch alg: sum (mean) over patterns the on-line delta (without eta, i.e. eta=1) and apply to obtain $\boldsymbol{w}_{\text{new}} (= \boldsymbol{w}_{\text{old}} + \ldots)$

Details

Dip. Informatica
University of Pisa

- Given the cluster centers $\boldsymbol{w}_1 \ldots \boldsymbol{w}_K$ (vectors of dim n, as $\mathbf{x}$)
- (2) For each $\mathbf{x}$ the winner is (the nearest prototype):

$$i^*(\mathbf{x}) = \arg\min_i \left\|\mathbf{x} - \boldsymbol{w}_i\right\|^2$$

Norm 2

find i s.t dist. is min

Euclidian distance !

$$\| \mathbf{x} - \boldsymbol{w}_i \|^2 = \sum_{j \to n} (x_j - w_{ij})^2$$

- Now $\mathbf{x}$ belong to cluster i^*
- (3) For each cluster i the new mean (centroid) is:

$$\boldsymbol{w}_i = \frac{1}{|\,cluster_i\,|} \sum_{j: x_j \in cluster_i} \mathbf{x}_j$$

Vector

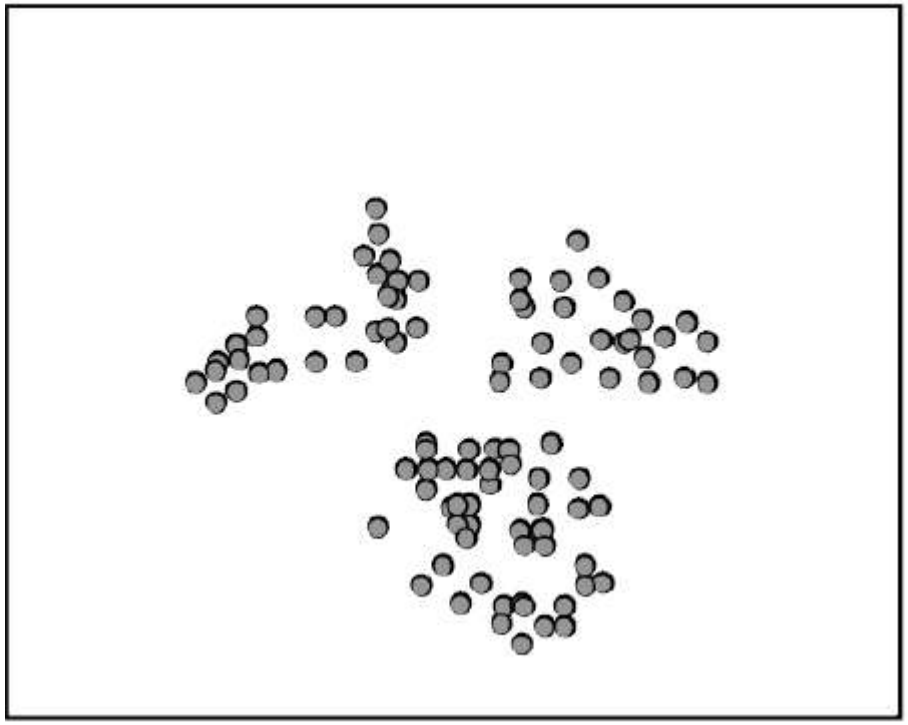
Vectors

Number of cluster members

Note: || ||² is the square of norm 2, hence the sum of squares (no root)

K-means example (1)

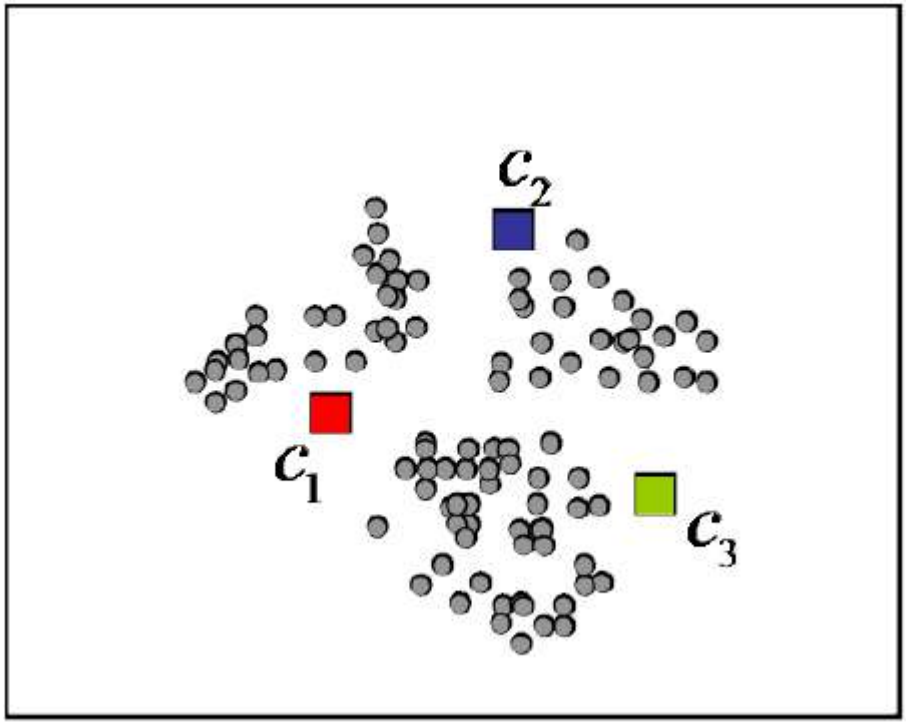
Dip. Informatica
University of Pisa



Input data

K-means example (2)

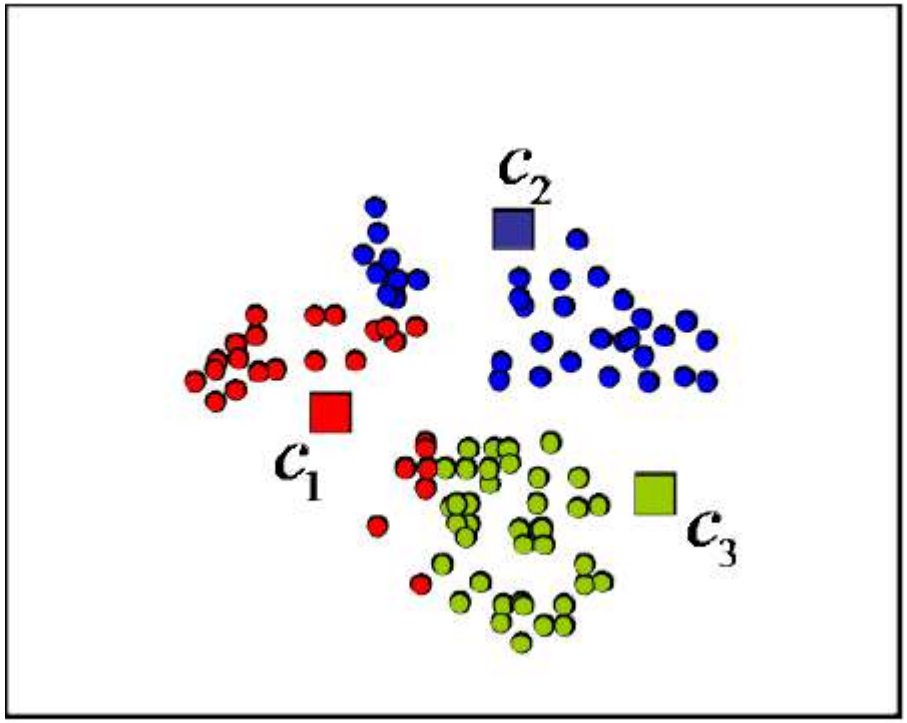
Dip. Informatica
University of Pisa



Initialize K = 3 prototypes

K-means example (3)

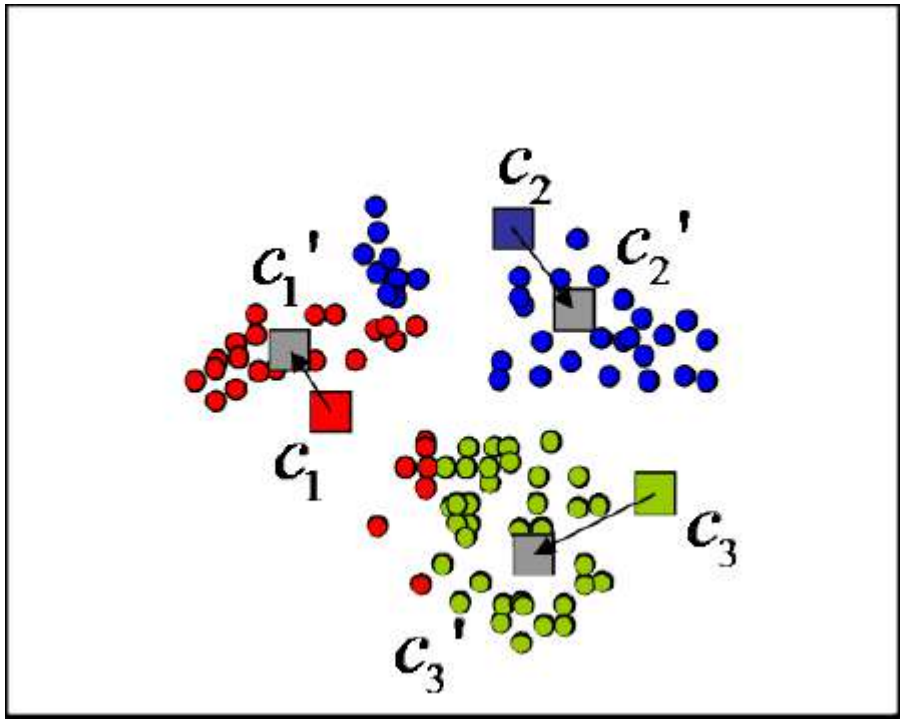
Dip. Informatica
University of Pisa



Assign samples to the nearest prototype/cluster

K-means example (4)

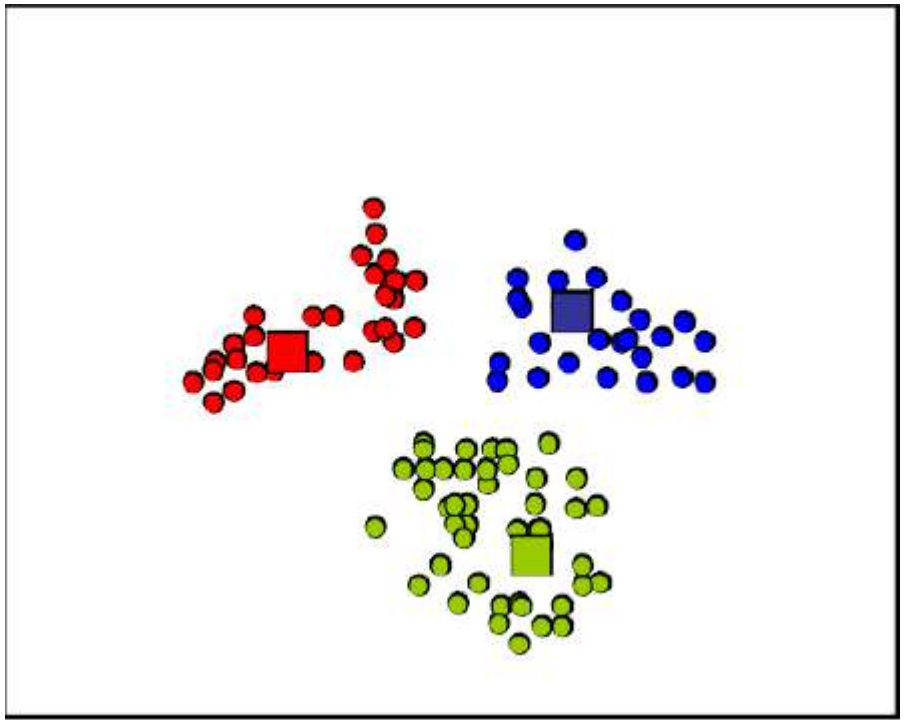
Dip. Informatica
University of Pisa



Compute the updated prototypes C'

K-means example (5)

Dip. Informatica
University of Pisa



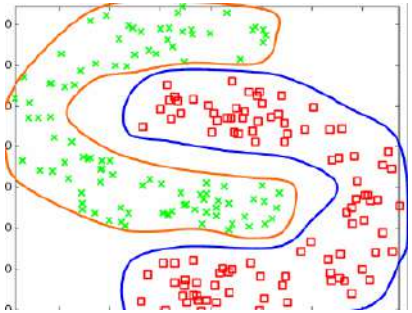
Update cluster assignment

K-means (+/-)

Dip. Informatica
University of Pisa

- The *k*-means is the simplest and most commonly used algorithm employing a squared error criterion
- The *k*-means algorithm is popular because it is easy to implement, and its in general efficient

- The number *K* of clusters to find must be provided
- Local minima of E make the method dependent on initialization and only sub optimal choice of the refence vectors !
- It can work very well for compact and hyperspherical clusters.
- No visualization properties !
  - K-means does not allow us to project data in a lower dim space
  - Note that the indexing of these $\boldsymbol{w}$ can be made in an arbitrary way, whereby this mapping is still *unordered*.



NOT hyperspherical clusters: what happens?

Soft-max

Dip. Informatica
University of Pisa

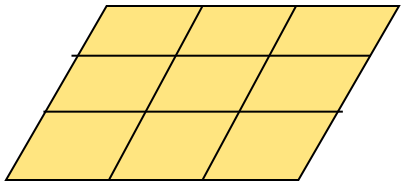
- To avoid confinement to local minima, a common approach is to introduce "soft-max" adaptation rule I.e.
- Not only the winning reference is modified
- The adaptation to x affects all cluster centers depending on their proximity to x, typically with a step size that decreases with the distance $d(x,w_i)$
- Maximum-entropy clustering (using Gaussians distance)
- An instance of this strategy in NN: the **Kohonen self-organizing maps**, where the proximity among reference vectors is defined on a ordered map, I.e. a neural grid where references vectors are arranged (and associated to units)

Unsupervised learning in NN: e.g. SOM

Dip. Informatica
University of Pisa

- **Self-organizing map** (**SOM**) neural networks, also called **Kohonen maps**, from the author's name (1981)

- Consist of N neurons located on a regular low-dimensional, usually 2-D, grid (lattice).
  - Each unit can be identified by the coordinate on the map
  - Each unit receives the same input x
  - Each unit has a weight w

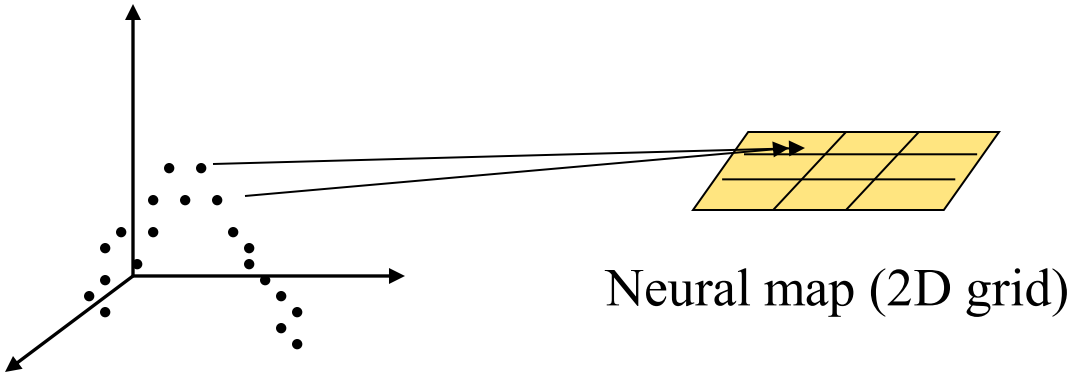


Neural map (2D grid): 3 X 3 neurons

Unsupervised learning in NN: SOM overview (AIM)

Dip. Informatica
University of Pisa

- SOM: learn a map from input space to a lattice of neural-units
  - topology-preserving map:
    - neighboring units respond to similar input pattern
    - data points close in the input space are mapped onto the same or **nearby map units***
  - visualization properties of the SOM output feature map (2D)



The unit whose weight vector is closest to the input is the **winner.**

SOM overview: Use of SOM

Dip. Informatica
University of Pisa

- **Clustering**: clusters can be identified on the map
  (rapidly find structures underlying the data)

- **Pattern recognition and vector quantization**: the input is transformed into the closest neuron weights, i.e. the reference or "codebook" vector;

- **Data compression**: the input is transformed into the indexes, or digital codes, of the winner unit;

- **Projection and exploration**: the distribution of multidimensional data is visualized in a lower dimensional space (the 2D map).

SOM overview: Competitive Learning

Dip. Informatica
University of Pisa

- **Competitive Learning** is an adaptive process in which neurons gradually become sensitive/specialized to different input categories/ set of samples

- Competitions among neurons for an input data→ the winner is allowed to learn more.

SOM overview: Homunculus in our brain

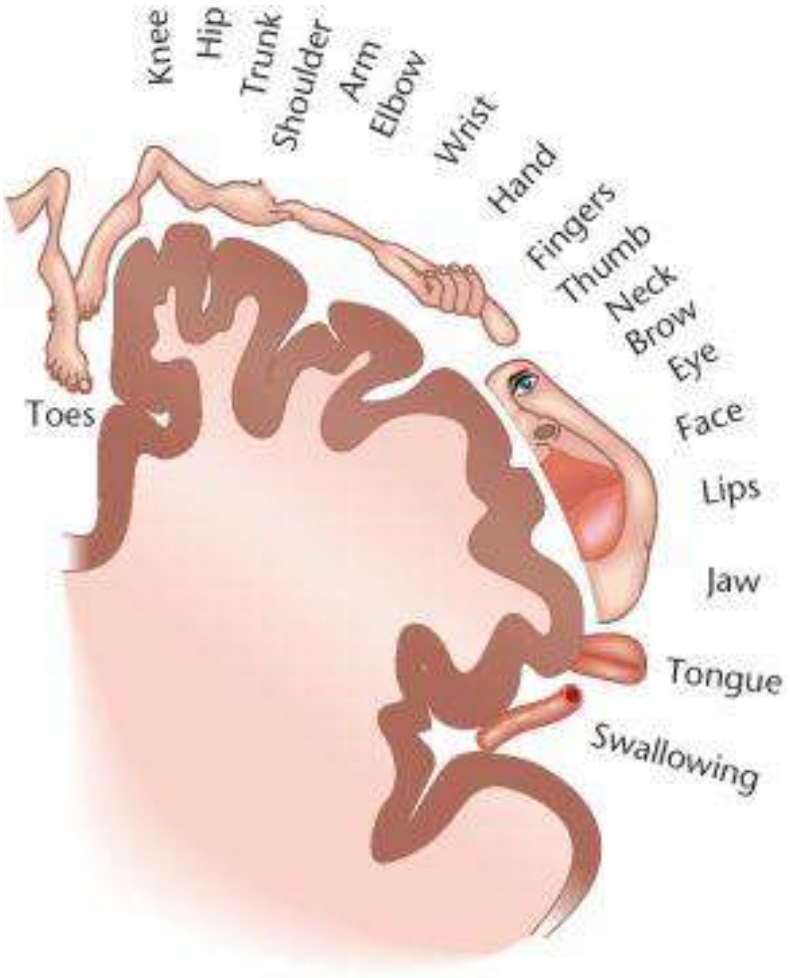
Dip. Informatica
University of Pisa

Somatosensory cortex (Motor cortex is similar)

Topologically ordered** (feature) **map

- **Adjacent neurons** come to represent coincident source of activations (in the body)
- Larger area of cortex for more sensitive body parts

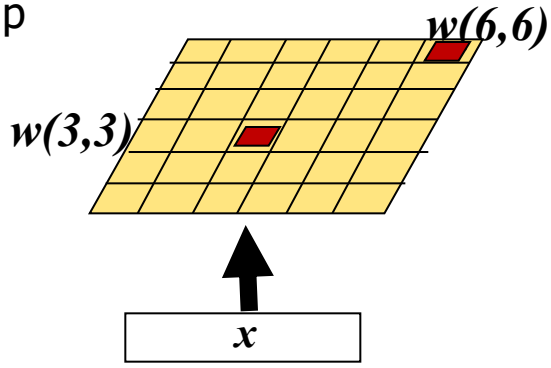
- In a **topology-preserving map**, units located physically next to each other will respond to classes of input vectors that are likewise next to each other.
- The nature (biology) exploited this mechanism!



SOM model and algorithm

Dip. Informatica
University of Pisa

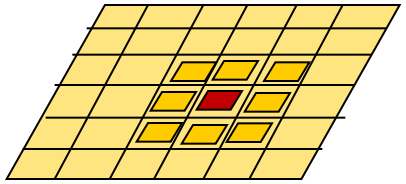
- The aim of SOM algorithm is to learn a map from vectors to a discrete output space (the coordinates of the unit on the map)
  - Each unit can be identified by the coordinate on the map
  - Each unit receives the same input x
  - Each unit has a weight w
  - Dim(w)=dim(x) = n



Learning Alg.

- The map is randomly initialized (weight values)
- Drawn a sample input x
- **Competitive stage**: the winner is the unit (with w) most similar to x
- **Cooperative stage**: upgrade the weights of the units that have topological relationships (*on the map*) with the winner unit (soft-max).
- Continues until convergence (no changes)

KEY FACTOR: neighbors on the map!!!!
(instead, standard soft-max used proximity in the data space)



SOM Learning Alg. (1)

Dip. Informatica
University of Pisa

Competitive stage

The current input vector $\boldsymbol{x}$ is compared with all the unit weights using an *Euclidean* distance criteria.

Standard SOM adopts a "winner-take-all" strategy (we already seen this in the K-means):

The winner is

$$i^*(\boldsymbol{x}) = \arg\min_i \left\| \boldsymbol{x} - \boldsymbol{w}_i \right\|$$

where i is an index on the map (e.g. coordinate in 2D)

SOM Learning Alg. (2)

Dip. Informatica
University of Pisa

Cooperative stage

- The weight of the winning unit and the weight of the units in its neighborhood are moved closer to the input vector (*Hebbian learning*)
- At iteration *t*

$$\boldsymbol{w}_i(t+1) = \boldsymbol{w}_i(t) + \eta(t) h_{i,i^*}(t)[\boldsymbol{x} - \boldsymbol{w}_i(t)]$$

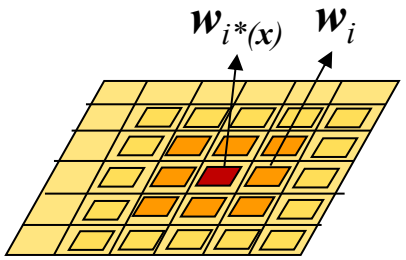
$i^*(\boldsymbol{x})$

- where *h(t)* is the neighborhood function (or neighborhood kernel), i.e. $h_{i,j}(t)$ is a function that decreases monotonically for increasing ||*i*-*j*||.

$$h_{i,i^*}(t) = exp\left(-\frac{\|\boldsymbol{r}_i - \boldsymbol{r}_{i^*}\|^2}{2\sigma_{nh}^2(t)}\right)$$

Coord. of unit *i*

width



- Learning rate and neighbourhood radius values (σ) are decreased as function of iteration *t*
  - neighbourhood wide at the beginning (global ordering) and then width and height slowly decrease during learning (convergence)
  - In practice, the shape of the neighborhood is chosen from standard geometrical form (rectangular, hexagonal etc. on the SOM grid) to include a finite set of the map units.

The Emerging Topological Order

Dip. Informatica
University of Pisa

- It is important to stress that the update rule of the cooperative stage is fundamental to the formation of topographically ordered maps.
- In fact, the weights are not modified independently of each other but as topologically related subsets.
  - For each subset similar kinds of weight updates are performed.
  - For each step a subset is selected on the basis of the neighborhood of the current winner unit (spatially, in the lattice).
  - Hence, topological information is supplied to the map because both the winning unit and its lattice neighbors receive similar weight updates that allow them, after learning, to respond to similar inputs.
- Although this seems quite natural, a formal proofs of ordering is difficult (see Kohonen book in the bibl.)

- Exercise: focalize the differences between K-means and SOM approaches.

SOM training on a 2D uniform data distribution

Dip. Informatica
University of Pisa

2D map, similar reference vectors (weights) are connected by lines

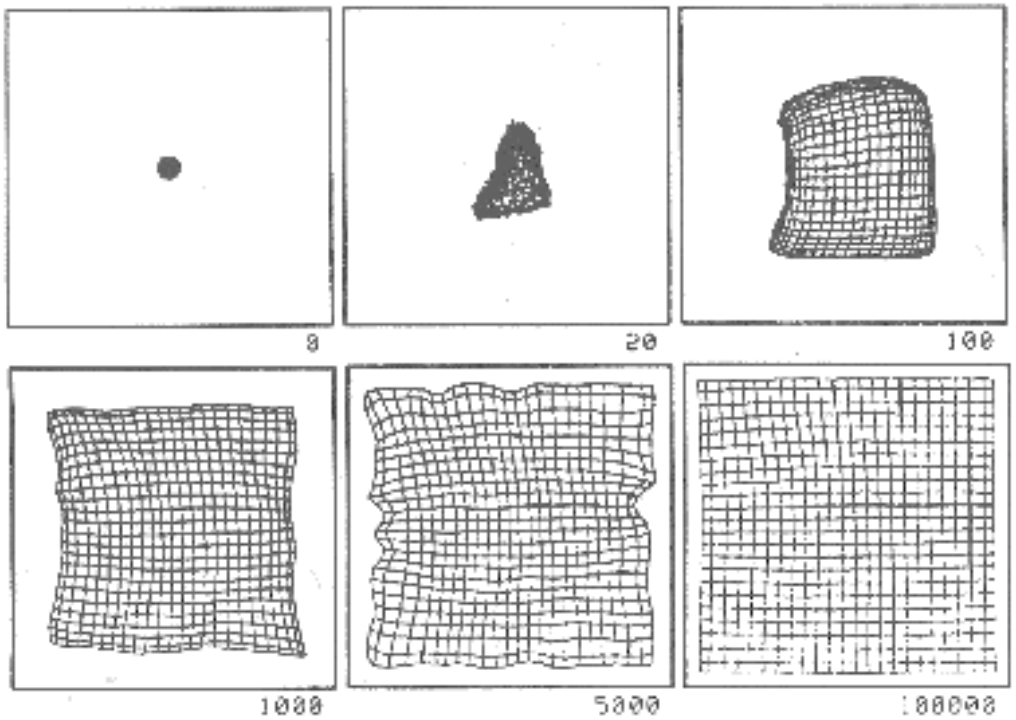
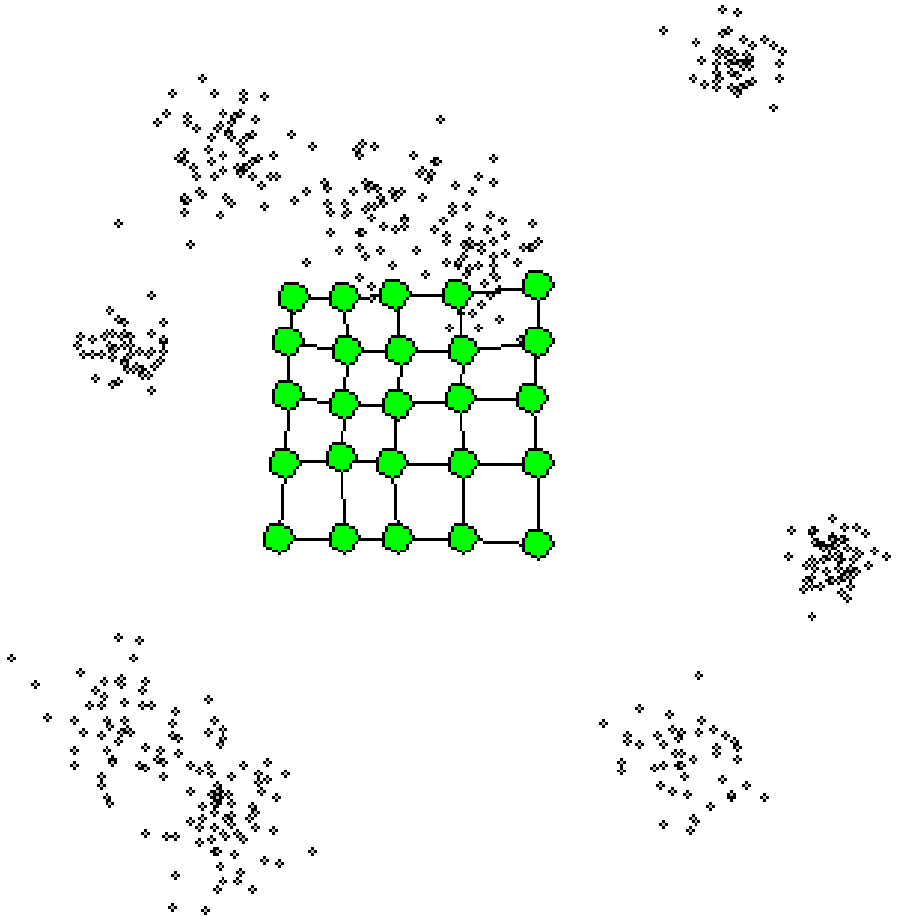


Fig. 3.4. Reference vectors during the ordering process, square array. The numbers at lower right-hand corner indicate learning cycles

Training the SOM (animated gif)

Dip. Informatica
University of Pisa

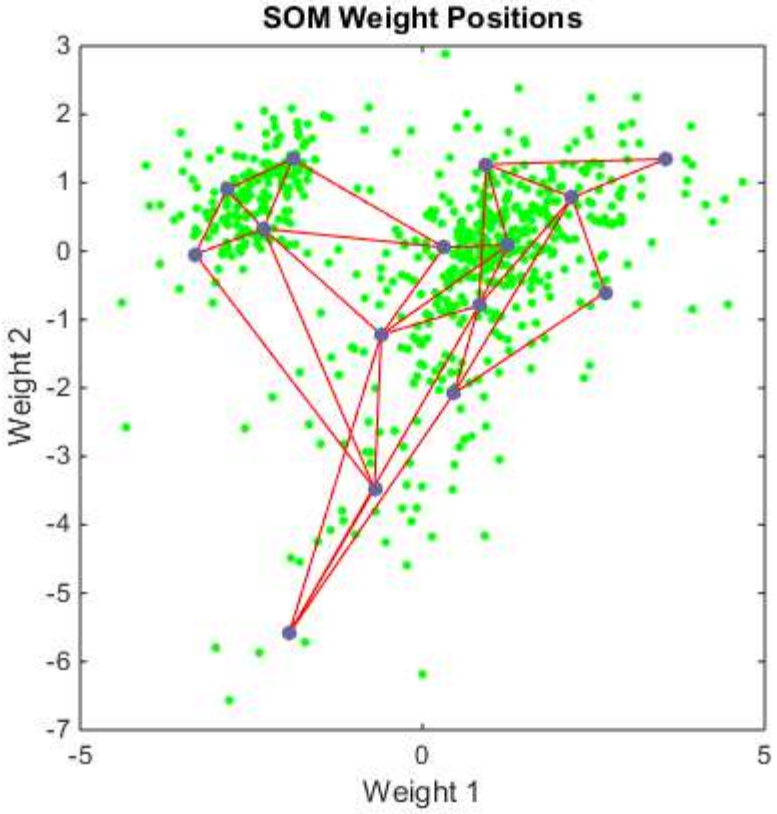
Units "go to" represent the data clusters



Animated GIF: https://media.giphy.com/media/11qH9uGmve6yFW/giphy.gif

Static image (after training on a different data set)

Dip. Informatica
University of Pisa



But for another movie, e.g.:
https://www.youtube.com/watch?v=k7DK5fnJH94

Visualization of SOM

Dip. Informatica
University of Pisa

- The map can be used to visualize different features of the SOM and of the data represented on the map. E.g.
  - **density** of the reference vectors (color density proportional to hits)
  - **distance** between prototype vectors of neighboring units (U-matrix):

    where dark color are used to indicate large distance between adjacent map units (prototypes) and light color are used to indicate small distances between adjacent map units. Next example.
  - Sammon maps (using lines)…
- Providing the SOM with easy interpretability!

An Applicative Example

Dip. Informatica
University of Pisa

- Data set of welfare indicators of different countries used to investigate the economic relationships between their economic systems.

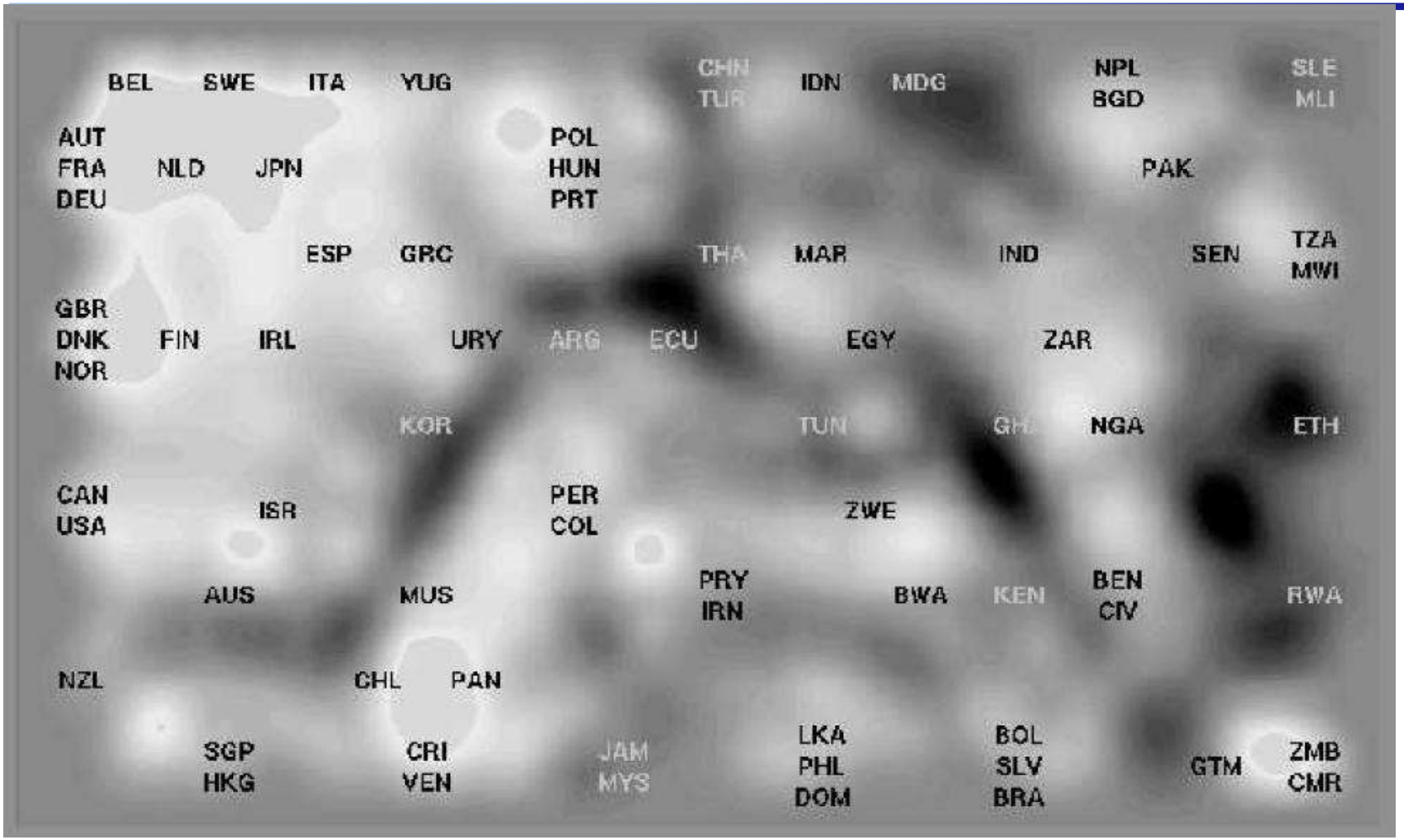
- 39-Dim data (Welafare descriptors), 77 countries, according to World Development Report 1992

- On the map display, the grey level of the point that is in the middle of two map units is determined by the scaled distance value between the pair of corresponding reference vectors (w) and then spatially smoothed. (light color are used to indicate small distances between adjacent map units)

- S. Kaski, T. Kohonen. Exploratory data analysis by the self-organizing map: Structures of welfare and poverty in the world. 1995

An Applicative Example

Dip. Informatica
University of Pisa

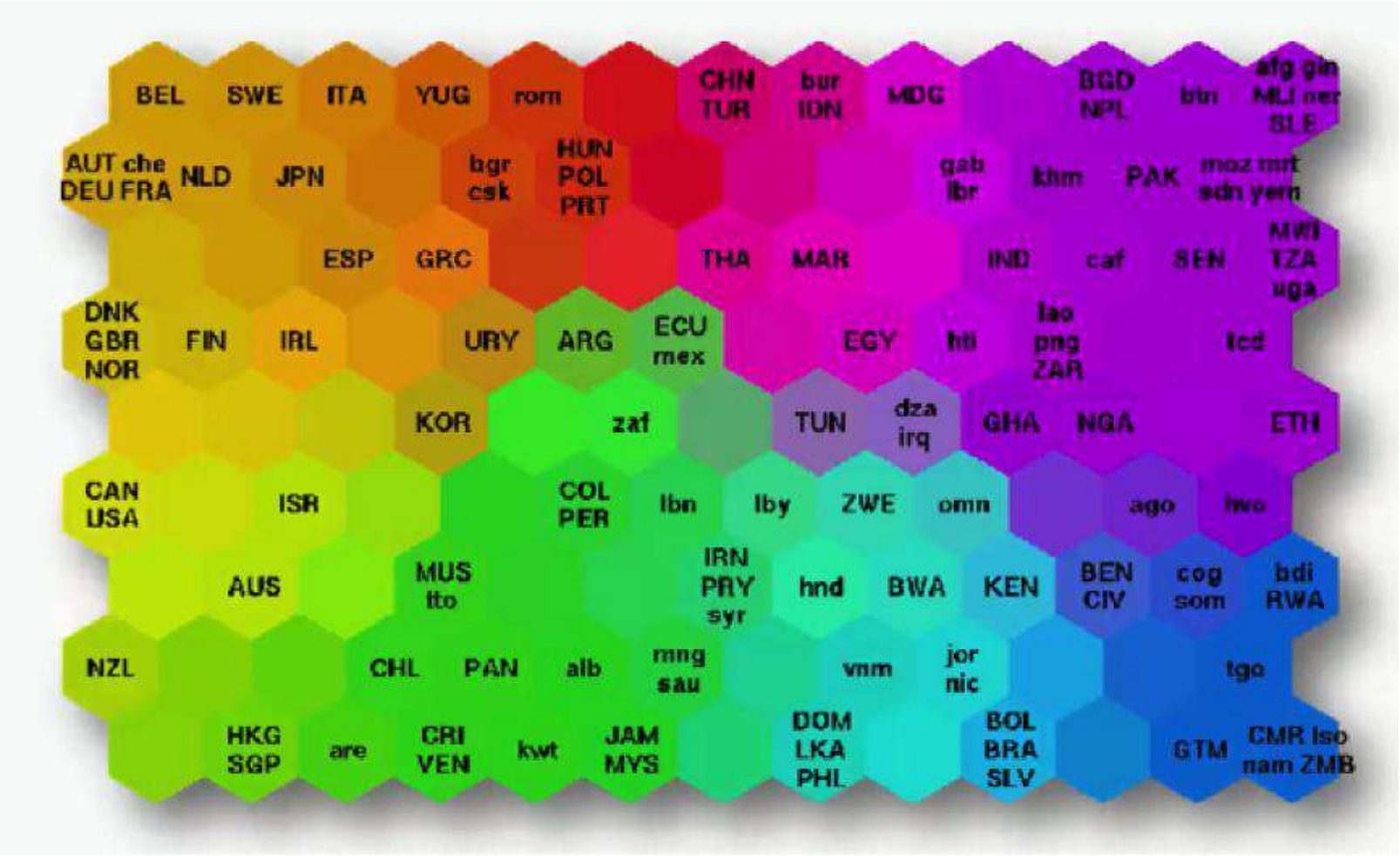


A smoothed distance matrix of 13 X 9 SOM lattice (illustrating the clustering structure)

Note that there is no need for a priori assumption on the shapes of the clusters

Colored image with cells

Dip. Informatica
University of Pisa



Evaluation (just few words)

Dip. Informatica
University of Pisa

- How is the output of a clustering algorithm evaluated?
- What characterizes a 'good' clustering result and a 'poor' one?

- Subjective: little in the way of 'gold standards' exist in clustering except in well-prescribed subdomains.

- Objective measures (no treated here): e.g. quantization error (but note that depends on the number of centroids)!
- But not "trivially" the label of a underlining classification task, called ***purity*** (entropy, mutual inf., …) (else you can use a classifier)!
- Quality of model/clustering versus quality of results (domain dependent)

Other topics in Unsupervised

Dip. Informatica
University of Pisa

- Hierarchical Clustering → *dendrograms*
- Principal Components, Curves and Surfaces Analysys
- Association Rules [DM]
- Independent Component Analysis and Exploratory Projection Pursuit
- Multidimensional Scaling
- t-distributed stochastic neighbor embedding (t-SNE)
- Generative Topographic Mapping (**GTM**)
- **ART** (adaptive resonance theory) NN
- …

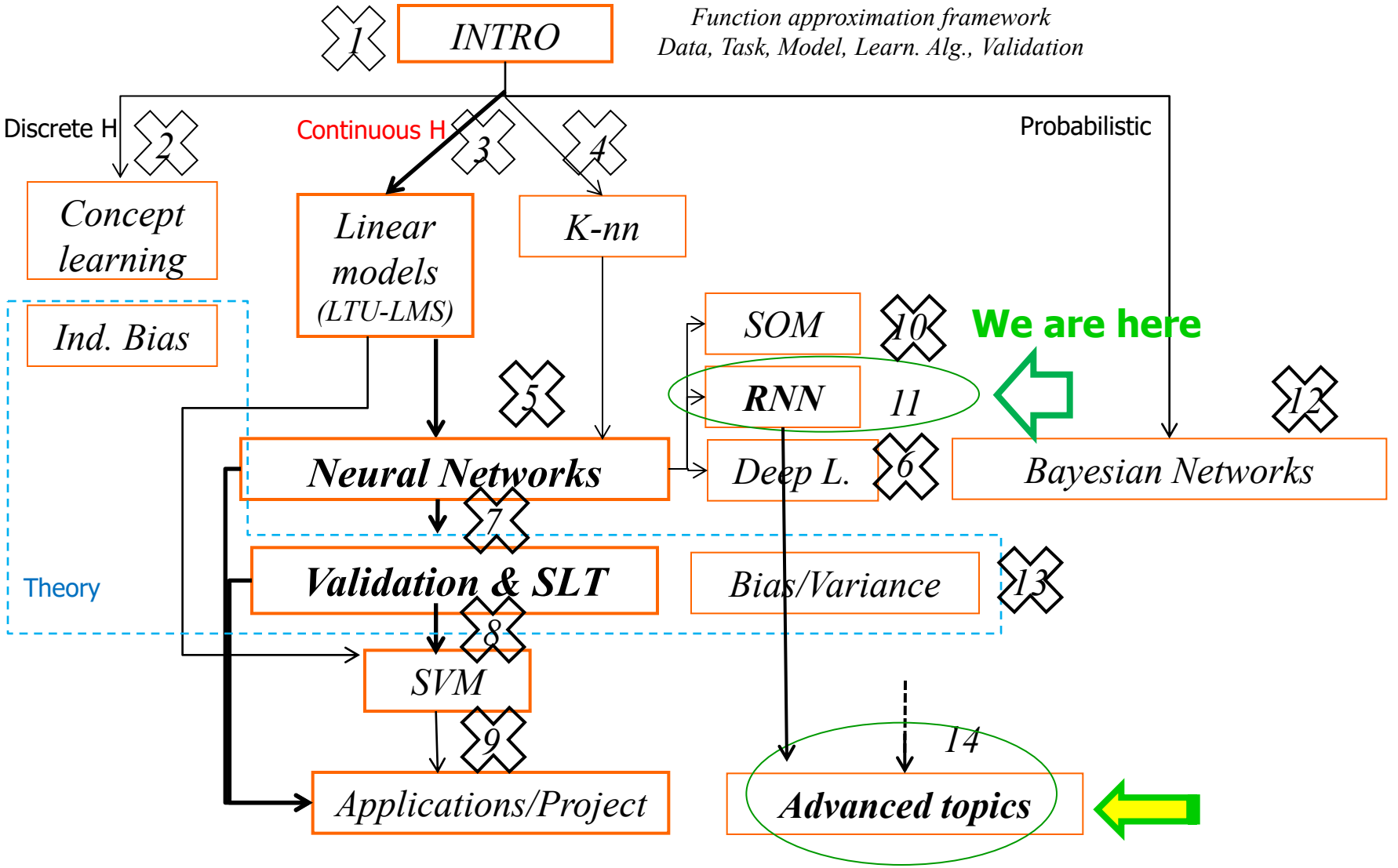
Exercises

Dip. Informatica
University of Pisa

- K-means: See before from on-line to batch alg.
- Resume the difference between SOM and K-means

ML Course structure
Where we are and go

Dip. Informatica
University of Pisa



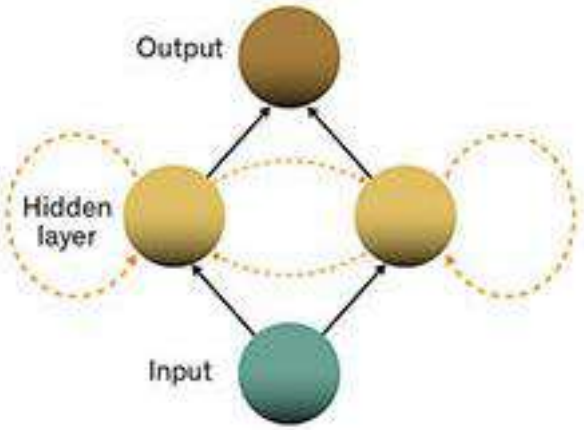
Intro to RNN

Dip. Informatica
University of Pisa

IEEE SPECTRUM

- IEEE Spectrum (magazine) 26 Jan 2016
- **"The Neural Network That Remembers"**
  - **With short-term memory, recurrent neural networks gain some amazing abilities**

A recurrent neural network includes connections between neurons in the hidden layer [yellow arrows], some of which **feed back** on themselves.



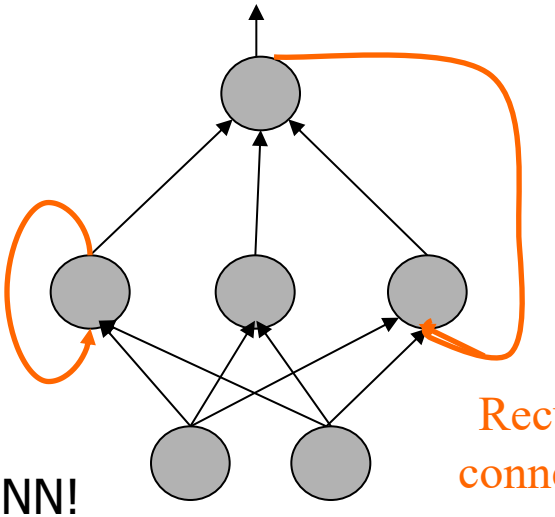
Feedforward versus Recurrent

Dip. Informatica
University of Pisa

- **Feedforward**: direction: input→ output
- **Recurrent** neural networks: A different category of architecture, based on the addition of *feedback loops* connections in the network topology,
  - The presence of **self-loop** connections provides the network with dynamical properties, letting a memory (**states**) of the past computations in the model.
  - This allows us to extend the representation capability of the model to the processing of sequences (and structured data).

Recurrent neural networks:

- See (these) separate slides in this course
- They will be the subject of the (further developed) ISPR/**CNS** courses
- **Neurobiological** plausibility
  - nervous system/biological NN are recurrent NN!



Recurrent connections

Why sequential data?

Dip. Informatica
University of Pisa

- Whenever the output of the model depends on the *history* of the inputs – e.g. *time*: dynamical models
- Or the domain is composed by varying-size sequences
  - Dynamical processes. Signal processing (Filters, Control). Robotics*
  - Language* (Speech recognition, NLP, Formal languages, IR*)
  - Vision, Reasoning (temporal events in AI):
  - Temporal series: financial forecasting, Signal processing *
  - Genomics/Proteomics (Bioinformatics*)
- 2 examples:
  - (human) Activity recognition (from sensor data)
  - Structure protein prediction from the sequence of aminoacids

Applicative Examples

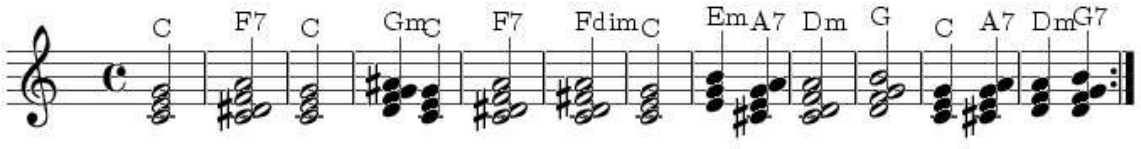
Dip. Informatica
University of Pisa

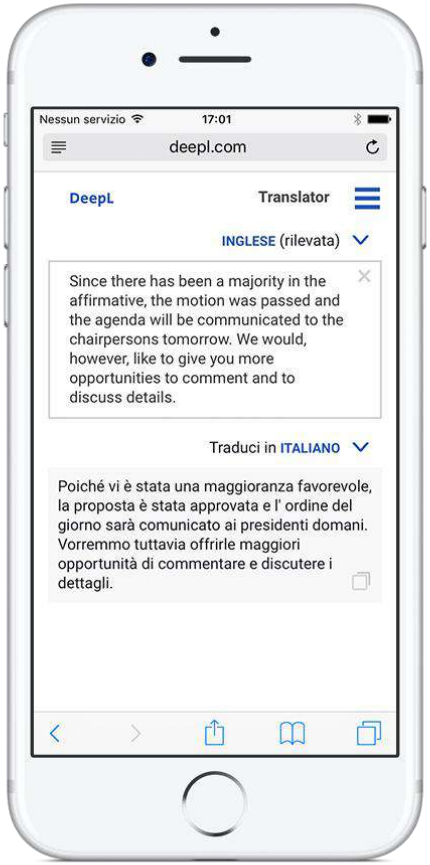
RNN (and related approaches) have been the reference approach for sequence processing,

especially for (state-of-the-art results):

- Speech recognition/processing
- Text processing
  - Machine translation etc.…

- Or even more (fun):
  - music composition
  - text/speech generation
  - …





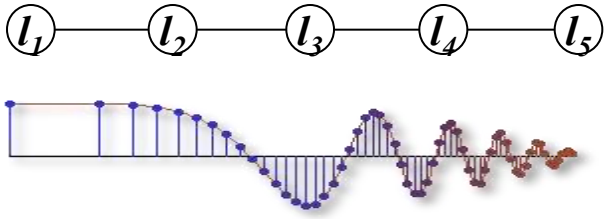
From flat to structured data

Dip. Informatica
University of Pisa

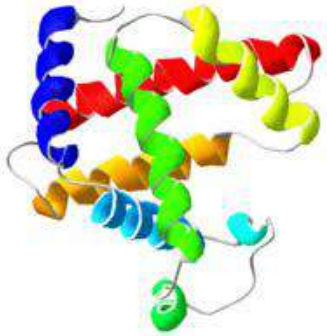
- **Flat: vectors** (as in the rest of this ML course)
- **Structured:** Sequences, trees, graphs, multi-relational data

stringa_in_italiano

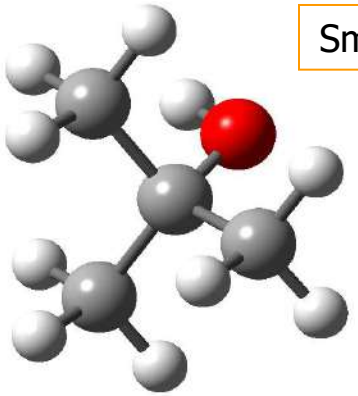
Strings



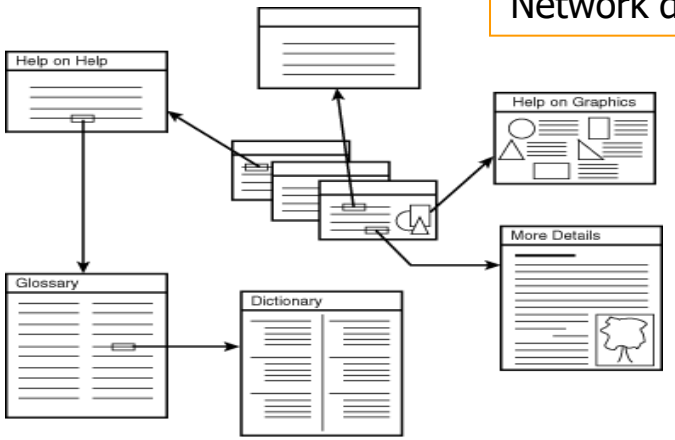
Series/
temporal stream



Proteins



Small molecules

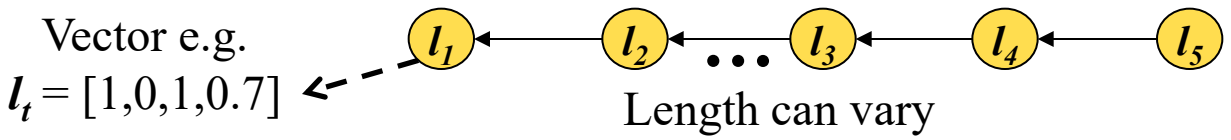


Network data

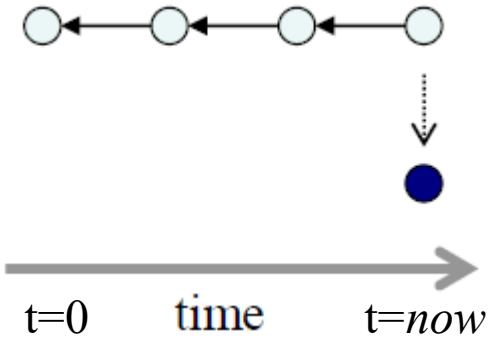
A new input Domain & Trasductions

Dip. Informatica
University of Pisa

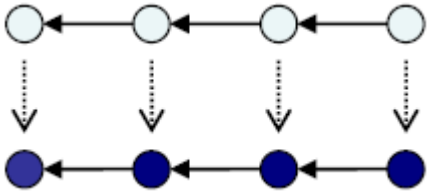
- Data: discrete sequence of vectors (serial order, e.g. time)
- Tasks:



Sequences recognition/classification: output at the end



Sequence Transduction, Next Step Prediction: output for each input step

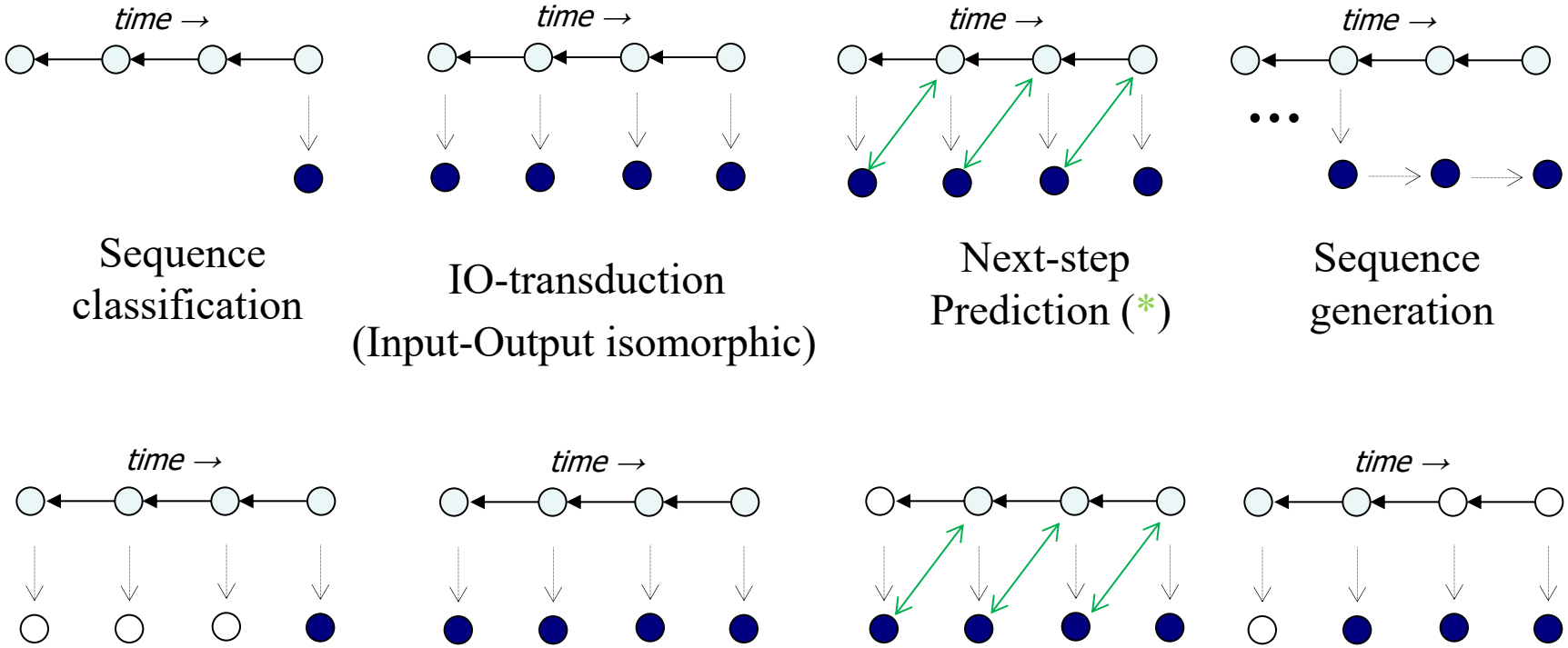


Each output is a vector (discrete or continuous values)

Transduction in the sequential domain: a mapping
Input sequence → Output value or sequence

Sequence Transduction: details

Dip. Informatica
University of Pisa



A **general transduction** where empty/full node can represent the different cases above

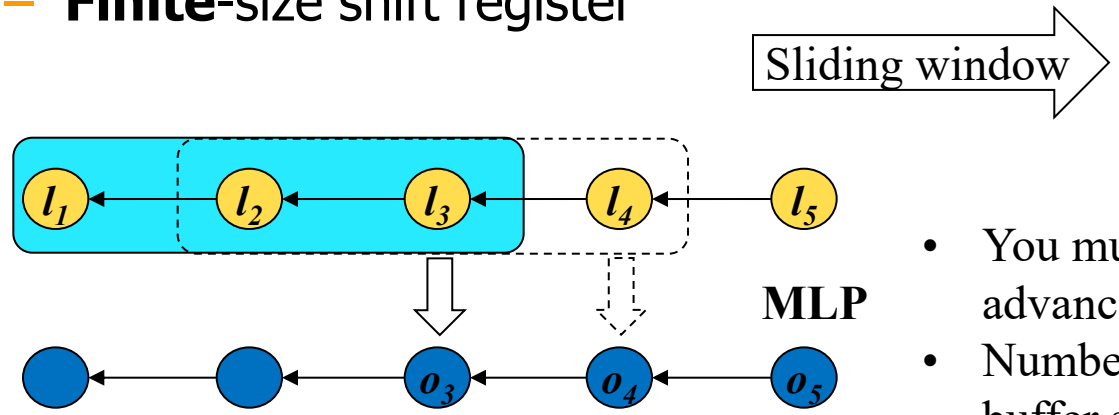
**Autoregressive*: next-step prediction with just 1 sequence → nowadays popular task with generative models!

Memory in NN: 1. IDNN

Dip. Informatica
University of Pisa

- Aim: Output depends on the previous inputs
- Temporal processing in NN
- 1. Finite memory/spatial approach:
  - Input delay neural networks (in Time-delay NN = TDNN class)
    - Since delays are in the NN connections to act as input memory
  - **Finite**-size shift register

See beginning of CNN lectures



CNN extended this idea to 2D images!

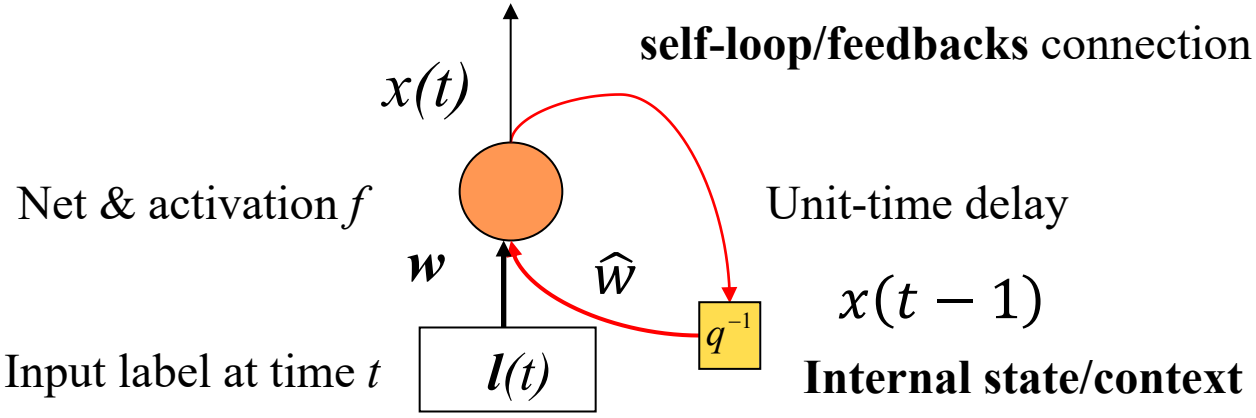
- You must know the window dim in advance
- Number of weights depends on the buffer size!!!

- If the buffer includes sufficient information we can solve with a MLP training algorithm! (e.g. NETalk, 1988)

Memory in NN: 2. RNN - 1 recurrent unit

Dip. Informatica
University of Pisa

- A Recurrent Neural **unit** with feedbacks in the architecture
- It uses current input + states information



$x(0)=0$

$$x(t) = \tau(\boldsymbol{l}(t), x(t-1)) = f(\boldsymbol{w}^T\boldsymbol{l}(t) + \hat{w}x(t-1) + \theta)$$

New part !!

f = (sigmoidal) activation function
$\hat{w}$ = recurrent weight
θ = bias

- The **state** summarizes the past information (context): it changes its value according to the stream of inputs
- The *encoding* of the past is **adaptive** (due to the free weights of the model)

An example

Dip. Informatica
University of Pisa

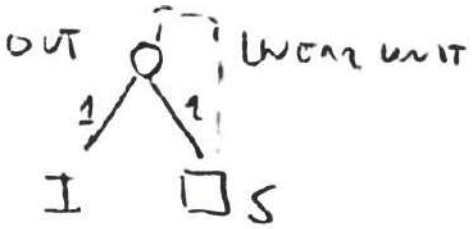
Exercise: realize a 1-unit RNN (find the values for the 2 weights w e w-hat) which ouputs the sum of "1" received in input so far.

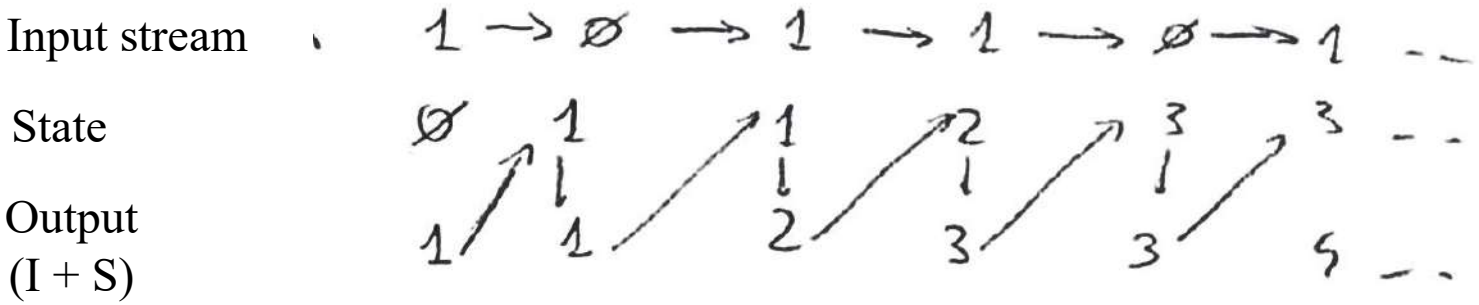
Input stream: 1 and 0

Drawn the flow of input/state/ouput values time-by-time for an example (101101)

Solution ;-)

Dip. Informatica
University of Pisa





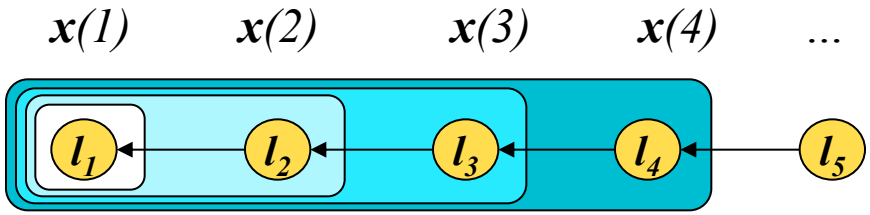
- Note that the state accumulates the sum of "1" received in input so far:
- The RNN summarizes the past subsequence information
- IDNN cannot do it for varying size sequences!!!

State transition system - recurrent model

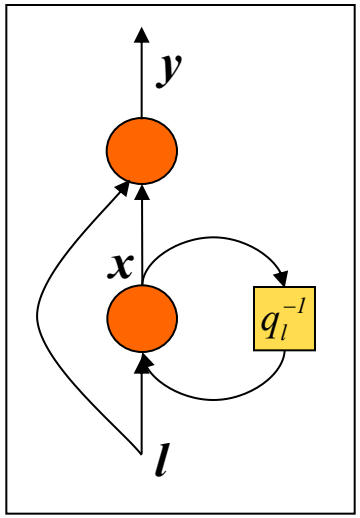
Dip. Informatica
University of Pisa

Given $x(0) = 0$

$$\begin{cases} \mathbf{x}(t) = \tau(\mathbf{x}(t-1), \mathbf{l}(t)) \\ \mathbf{y}(t) = g(\mathbf{x}(t), \mathbf{l}(t)) \end{cases}$$



State formation over time
Sub-sequences encoding/ embedding



Graphical model

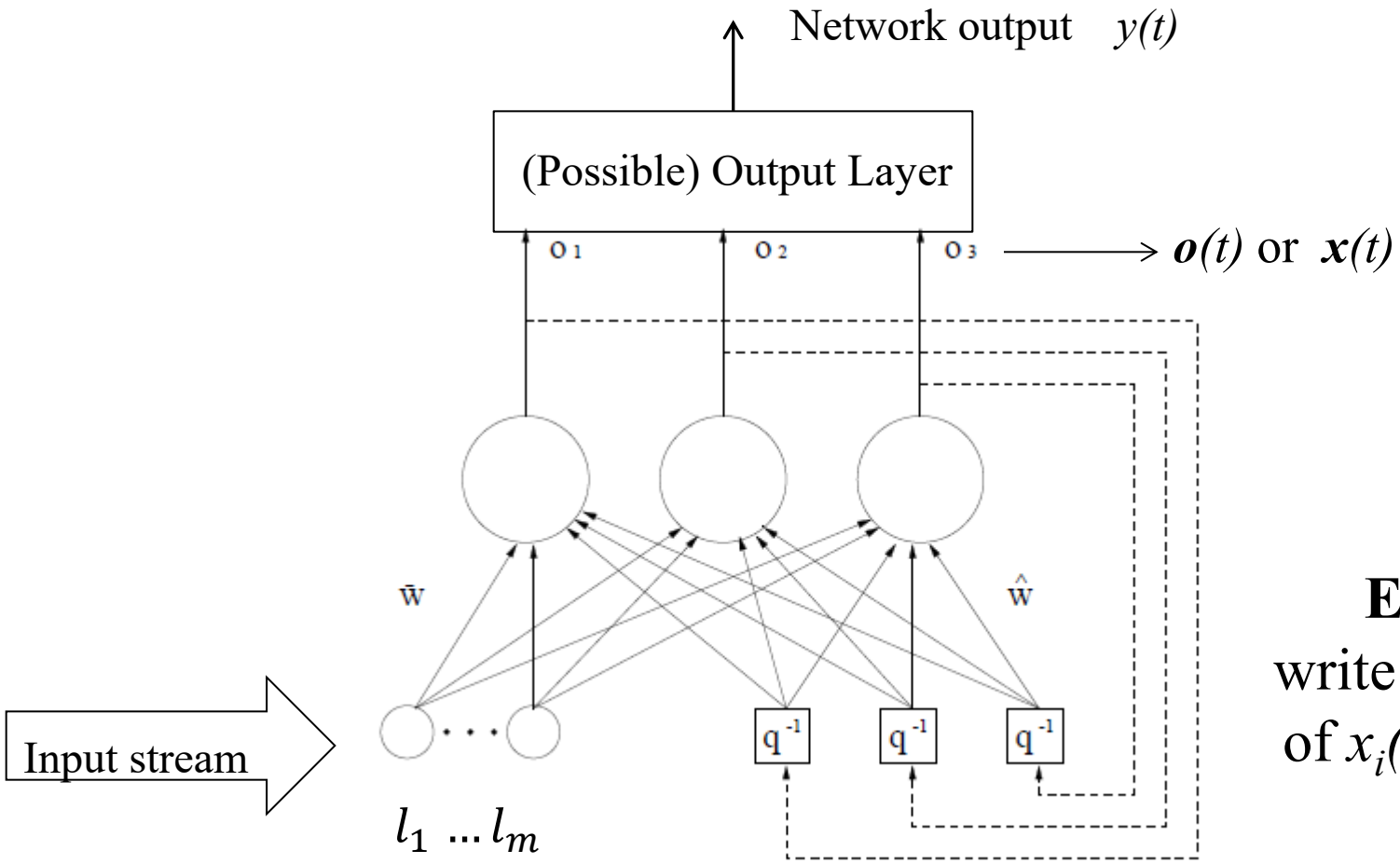
E.g. also **HMM**

- Tau (τ) is the *state transition* (next-state) function realized by a NN
- The state summarizes the past inputs
- Note that the state have continuous values (real numbers)
- $x(t)$ can be a set of states (to compose a network of recurrent units)

Recurrent Neural Network

Dip. Informatica
University of Pisa

- Many hidden recurrent neurons (“**Simple RNN” – Elman**)



Exercise:
write the equation of $x_i(t)$ (and then of $\boldsymbol{x}(t)$)

Properties

Dip. Informatica
University of Pisa

- Of course many architectures are possible, however the "Simple RNN" seen before is already powerful:
- Universal approximators of *non-linear* dynamical systems
- Turing equivalence (e.g. any automata can be simulated)
- Further: RNN as a (non-autonomous) *dynamical non-linear system* → dynamics studies → chaos theory (e.g. fractals theory …)

Properties of RNN

Dip. Informatica
University of Pisa

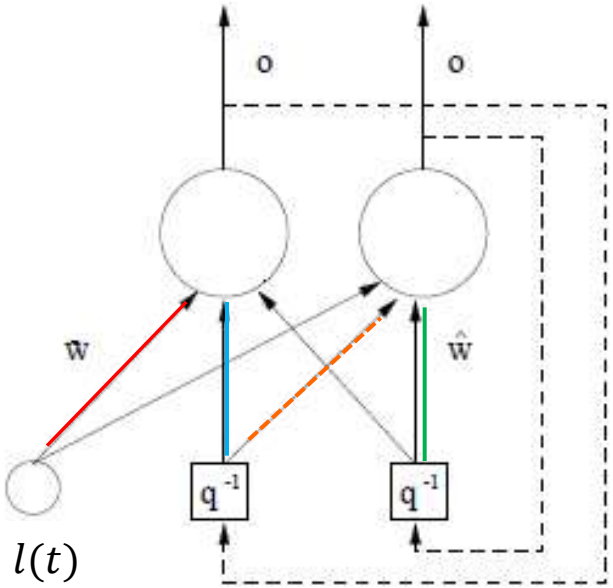
Recurrent NN models are based on the following **assumptions:**

- Causality: A system is causal if the output at time t_0 (or node v) only depends on inputs at time $t<t_0$ (depends only on v and its descendants)
  - necessary and sufficient for *internal state*
- Stationarity: time invariance (after model training), i.e. the state transition function τ is independent on node v of the sequence
  - i.e. τ is the same in any time t, regardless of the dimension of sequences (useful to process data of variable length with a fixed size of the model)
- Adaptivity: transition functions are realized by NN (with free weight parameters), hence they are **learnt** from data
  -

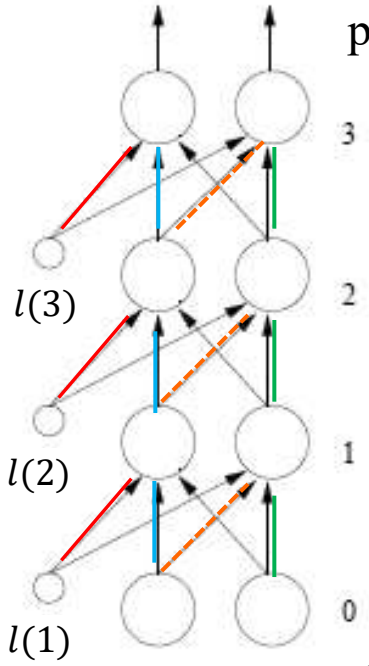
Unfolding

Dip. Informatica
University of Pisa

- Unrolling through time (i.e. input sequence) of the same model
- Build a feedforward MLP (called *encoding network*) equivalent (on the given k steps) to the RNN (given a specific input sequence)
- 1 model-replica for each input step
- Stationarity → ***weight sharing*** (across unfolded layers) !!
- Nice! : On this enc.net. we can apply back-prop learning algorithm !!!



Input label $l(t)$



Unfolding a 2-units RNN over 3 input steps

We will obtain a different enc.net. for each input sequence

MANY LAYERS:
Deep?

Learning algorithms

Dip. Informatica
University of Pisa

The learning algorithm must account the set of encode transitions developed by the model for each step of the inputs.

- **BPTT** (back-propagation through time) and
- **RTRL** (real time recurrent learning)

supervised learning algorithms are designed for recurrent neural networks.

They compute, in different style, (the same) gradient values of the output errors across an **unfolded** over time network that is equivalent to the recurrent one (encoding networks).

- The vanish of gradient over this deep network can make difficult to learn Long-term dependencies

Advanced models

Dip. Informatica
University of Pisa

Rapid growing research field, many variants:

- **LSTM** (long-short term memory): try to solve the gradient vanish by using "gate units" able to select the past/gradient flow.
  - Note: born to address the need to take long memory: not suitable for short series!
  - Hence, despite their popularity, do not use them acritically
  - See applications later (next slide)
- **GRU** (gated recurrent units): simplified LSTM
- **BRNN**: Bi-directional RNN: to consider both the left and right context
- … Often fusing DeepNN + RNN

- But also (to deal with gradient vanish issues):
  - Hessian-Free optimizer
  - Pre-training techniques …

- Many included in the popular DL ML libraries: Caffe(2), PyTorch, Keras, Matlab, CNTK, …

Popular applications

Dip. Informatica
University of Pisa

It has been for many years a reference method for sequence processing, especially for (with often state-of-the-art results)

- Speech recognition/processing
- Text processing (Machine translation etc.…)
- Some data (even if not updated):
  - In 2015 LSTM(RNN) used in 2 billion Android phones. 1 billion iPhones (siri).
  - In 2016 almost 30% of the awesome Google datacenters computational power for inference was used for LSTM(RNN).
  - 2017: LSTM(RNN) used in Alexa, Facebook (30 billion translations every week).
- And lots of fun…E.g.
  - Generating text/speech/music …
  - See also our previous examples on the caption generations, language learning (deepNN lectures), ….

Another nice example

Dip. Informatica
University of Pisa



Human user inputs a text and
the algorithm generates a handwritten version of it.

Generate pen trajectories by *predicting one* (x,y)
point at a time

Graves, *Generating Sequences With Recurrent Neural Networks*
https://arxiv.org/abs/1308.0850

Related approaches

Dip. Informatica
University of Pisa

- **SOM/unsupervised NN** for sequential domains
- Hidden Markov Models- **HMM** (model the probability distribution of state transitions)
- **Transformers**
- **Randomized (R)NN: Reservoir Computing**: ESN/LSM (see next slides)

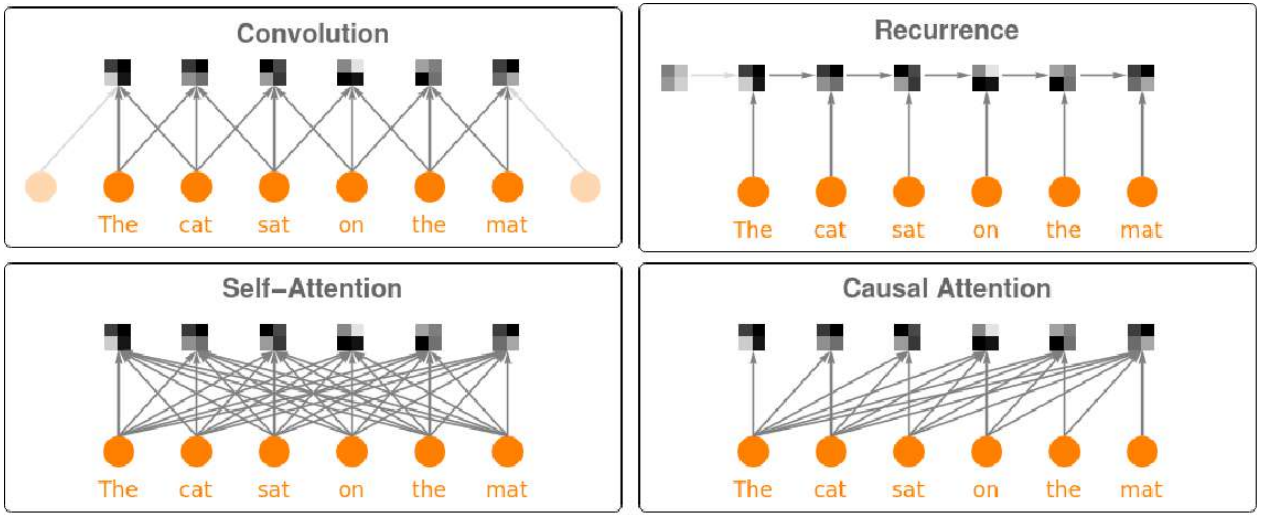
- Distance based models (string matching)
- **Kernel** for strings

- Grammatical inference
- **ILP**

Transformers

Dip. Informatica
University of Pisa

- A **transformer** is a deep learning architecture, proposed in 2017, that relies on the parallel multi-head attention mechanism
- Attention mechanism allows a model to access any preceding point along the sequence

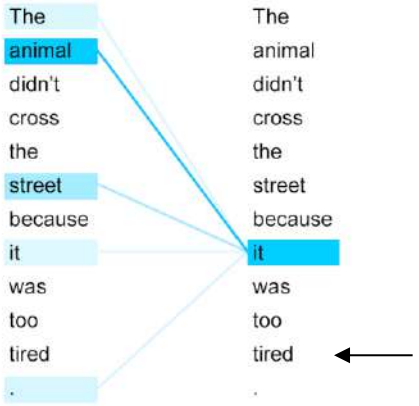


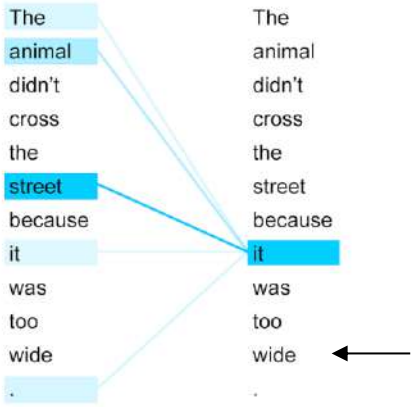
- *Encoder and/or decoder* + feedforward neural networks complete the architecture for different purposes

Transformers: Attention

Dip. Informatica
University of Pisa

- The **attention layer** weights all previous states according to a learned measure of relevance, providing, potentially, *information about far-away tokens*.
  This proved to be especially useful in language translation,
  where far-away context can be essential for the meaning of a word in a sentence
- First used to improve RNN models and then even alone (Paper: *Attention is all you need*)
- Processing all tokens simultaneously, calculating weights between them in successive layers, or better between their embedding
  - just scaled dot-product units using query and word vectors to search *the highest correlations amongst words* within a sentence. **E.g** "see that girl" whereas "that" is identified as the correlated to word "girl", or the examples on the right: *why the difference?*
- It can be applied also to patches (instead of words) of ***images*** (vision transformers), etc.





See further courses in these applicative fields e.g.
IS**PR/GDL** & **HLT** !

GPT

Dip. Informatica
University of Pisa

Generative

Pre-trained

Transformer

Used to generate text, speech, images, code …



A 2023 *Time* cover:
"The AI Arms Race Is Changing Everything"

Randomized RNN Toward ESN

Dip. Informatica
University of Pisa

- Can we exploit directly this *state machine* to encode sequences and then use the resulting embedding to learn the output mapping?
- Yes, even with *random connected recurrent networks* with fixed (*untrained*) **random weights**!!! (please refer to NN lectures for "*random weights NN*")

- Fractal machine + **Reservoir Computing**(RC) /ESN/liquid machine class of RNN
- Very **efficient** (no hidden recurrent unit learning at all), with the capability to solve well task *under specific conditions (bias)*.
  - Ability to intrinsically discriminate among different input sequences in a suffix-based fashion without adaptation of the recurrent parameters (see later)

Echo State Networks

Dip. Informatica
University of Pisa

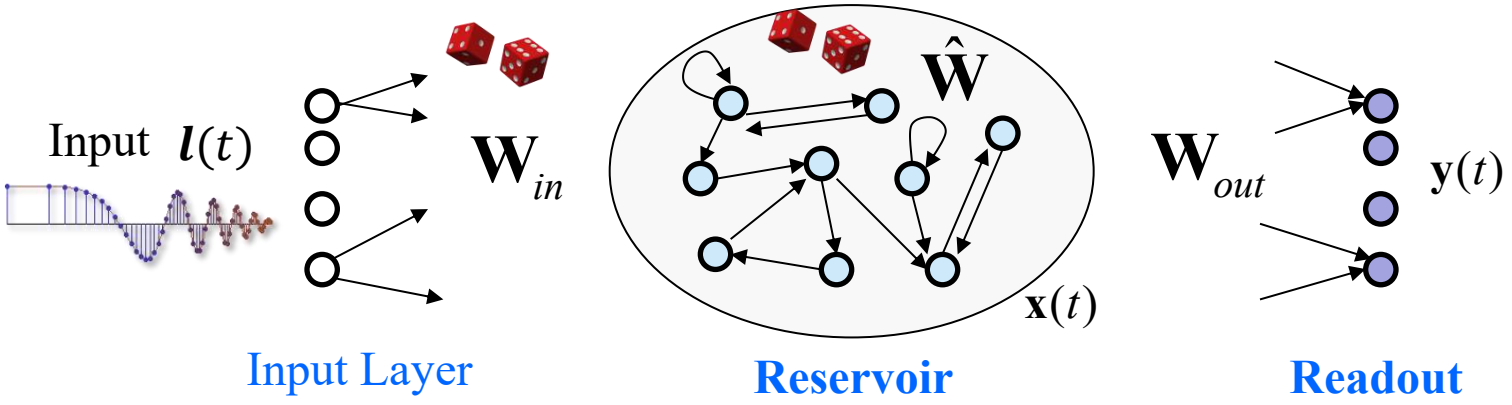
- Emerging paradigm for <u>efficiently</u> modeling Recurrent NNs
- An Echo State Network (**ESN**) consists in a

Large *reservoir* of sparsely connected recurrent units

Untrained after random* initialization

Simple feed-forward *readout* of linear units

Trained by efficient linear methods, e.g. *ridge regression* (*closed-form solution)*



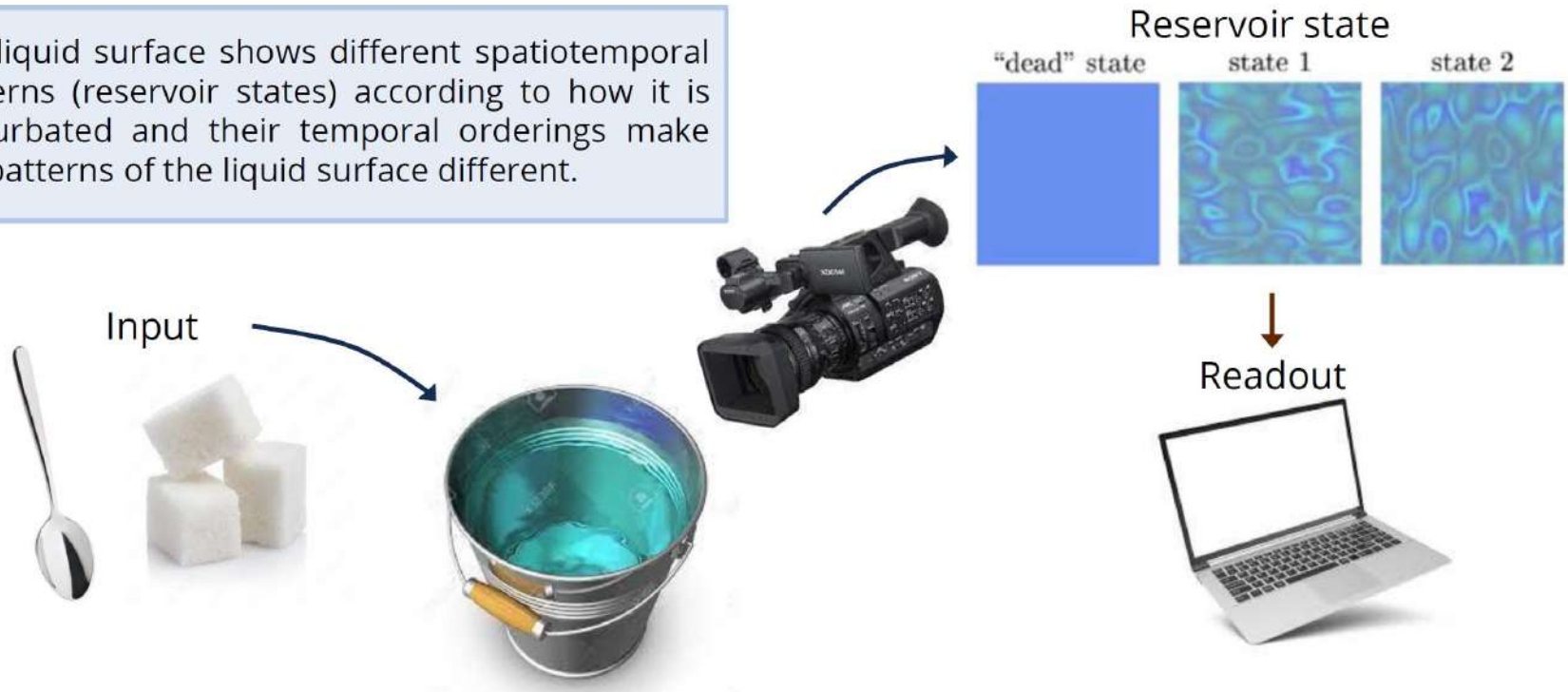
- *Echo State Property*: contractivity of the state transition functions (limiting *spectral radius ρ* of reservoir weights, i.e. stability of the dynamical system)
- Recurrent state asymptotically depends (*echo*) *only* on the *history of the inputs* (and not on the initial state conditions)

Reservoir Computing

Dip. Informatica
University of Pisa

Liquid-based reservoirs

The liquid surface shows different spatiotemporal patterns (reservoir states) according to how it is perturbated and their temporal orderings make the patterns of the liquid surface different.

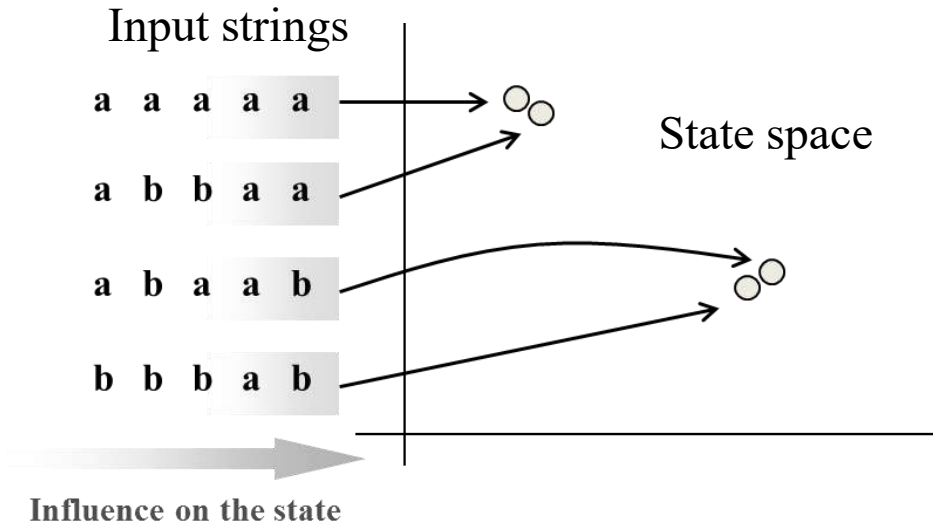


C. Fernando and S. Sojakka, Lecture Notes in Computer Science 2801 (Springer, Berlin, 2003), p. 588.

Credits. G. Milano

Insight: the state space Markovian organization

Dip. Informatica
University of Pisa

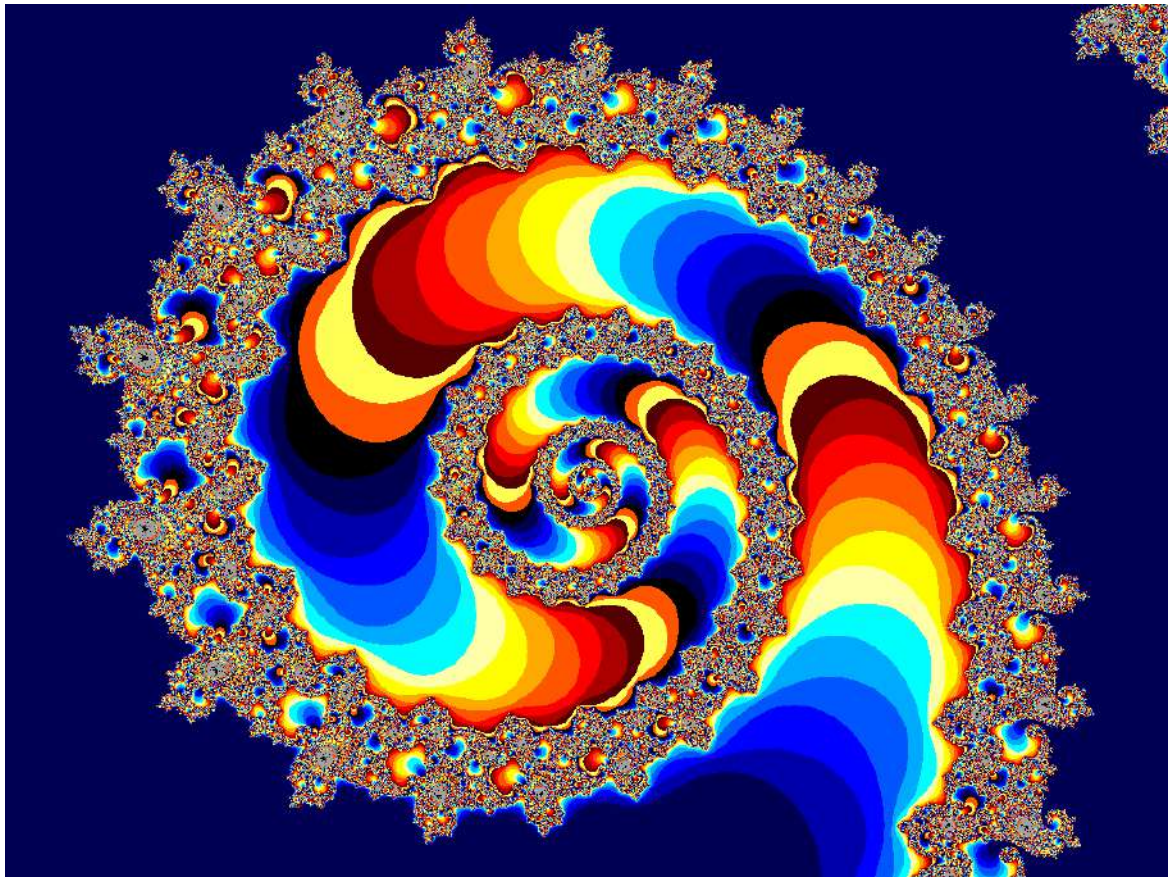


- Reservoir acts as a projection (embedding) of input sequences in the vector of states ***x(t)*** values
- Ability to intrinsically discriminate among different input sequences in a *suffix-based fashion* without training of the recurrent weights
  - The states are close to each other proportionally to the length of the common suffix
  - Easy to learn Markovian task (also for RNN but ESN are of course much more efficient!)
  - Architectural bias for recurrent models with learning !

C. Gallicchio, A. Micheli, "*Architectural and Markovian factors of echo state networks*", Neural Networks, 2011

Fractals

Dip. Informatica
University of Pisa

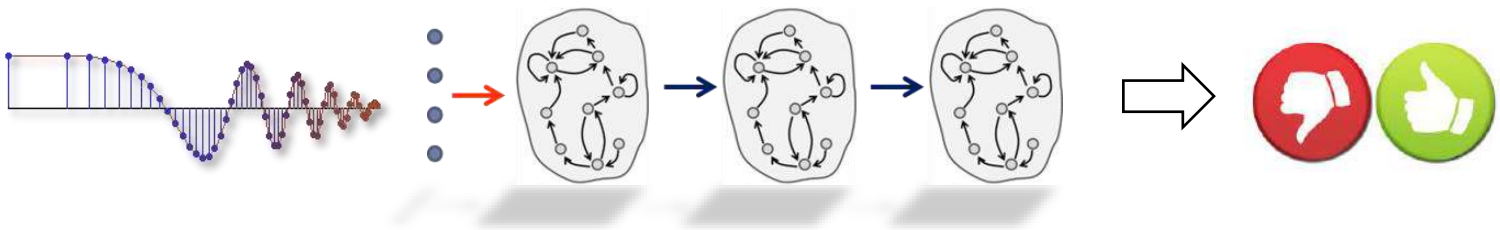


ESN/RNN are related to dynamical systems studies, chaos theory, fractals …

RC + Deep?

Dip. Informatica
University of Pisa

- RC+ Deep: Yes, we can.
  - And we have deepRNN power+ efficiency of ESN.
- Analysis by key questions, e.g.:
  - Why stacking recurrent layers?
  - What is the inherent (independent from learning) architectural effect of layering?
- Deep Echo state Networks have been studied (and still continue), with many results introduced by the CIML group in Pisa:
- Deep layered organization of RC models show advantages in terms of
  - occurrence of multiple time-scale
  - increasing of richness of the dynamics (hierarchical)

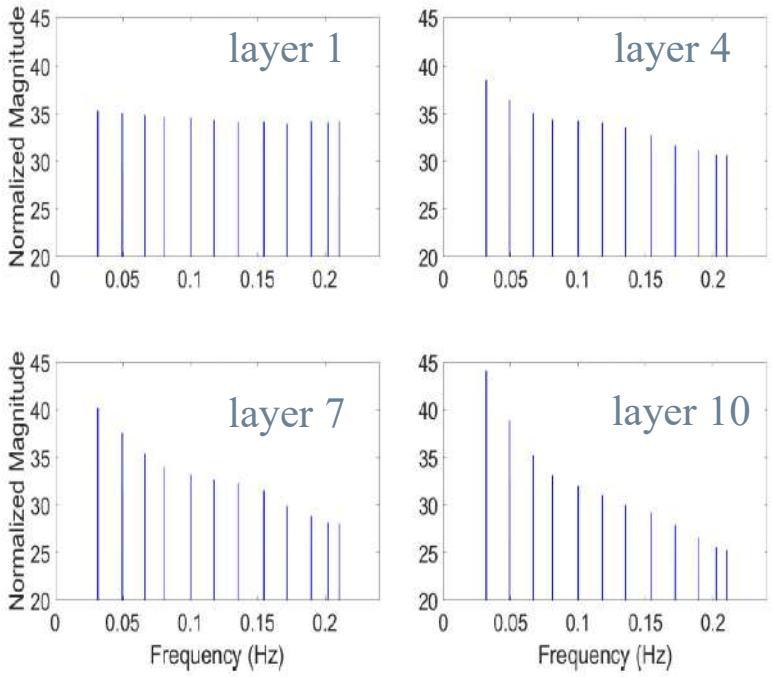


Hierarchical Temporal Features in DeepESN

Dip. Informatica
University of Pisa

Structured representation of temporal data through the deep architecture (multi time-scale)

Frequency Analysis (here in the linear DeepESN [*])



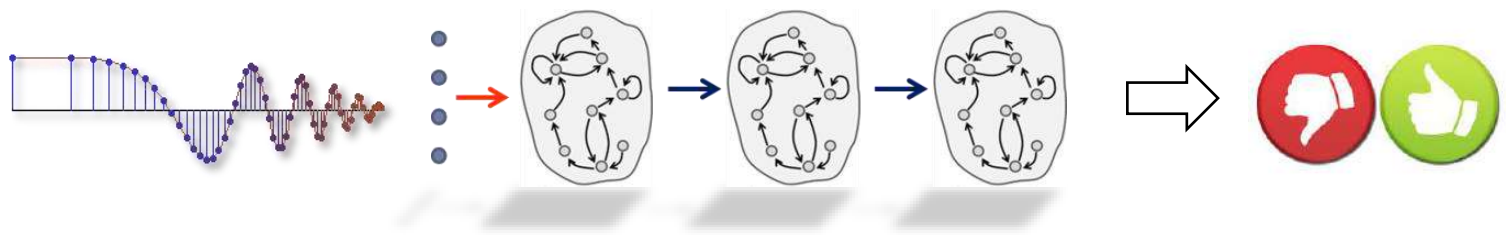
- Diversified magnitudes of FFT components in states (from inputs)
- **Multiple frequency representation**
- Ordered along the network's hierarchy
- **Higher layers focus on lower frequencies (larger time scale)**

[*] Gallicchio, Micheli, Pedrelli. **Hierarchical Temporal Representation in Linear Reservoir Computing** WIRN (2017) https://arxiv.org/abs/1705.05782

Deep RC references

Dip. Informatica
University of Pisa

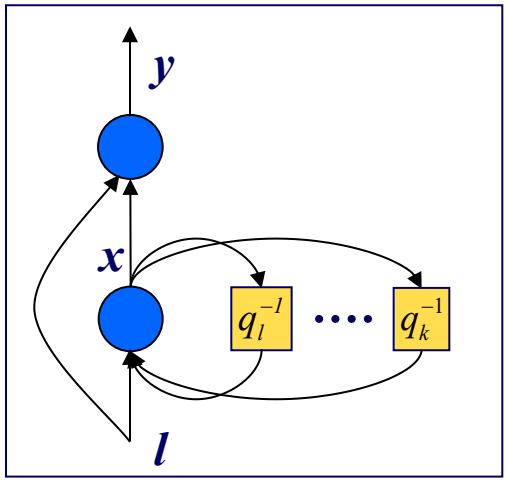
- Many results:
  - C. Gallicchio, A. Micheli, L. Pedrelli, "*Deep Reservoir Computing: A Critical Experimental Analysis*", Neurocomputing, 2017
  - C. Gallicchio, A. Micheli, "*Echo State Property of Deep Reservoir Computing Networks*", Cognitive Computation, 2017
  - C Gallicchio, A Micheli, L Silvestri, "*Local Lyapunov Exponents of Deep RNN*", 2018.
  - C. Gallicchio, A. Micheli, L. Pedrelli. *Design of deep echo state networks* . Neural Networks (2018), Vol. 108, December 2018
  - … many others
- **A short guide/overview**:
  - C. Gallicchio, A. Micheli, "*Deep Echo State Network (DeepESN): A Brief Survey*", arXiv:1712.04323



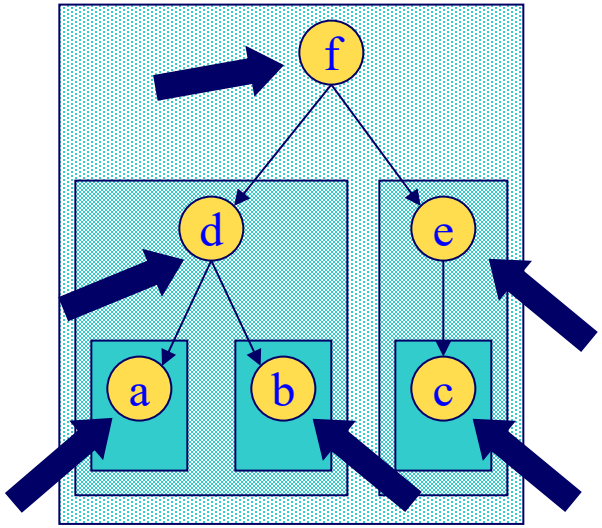
Toward Structured Domains (SD)

Dip. Informatica
University of Pisa

- Can we extend this recurrent approach to rooted trees?
- Yes, *Recursive Neural Networks*
  - Boot-up encoding of input trees
  - Extending states for children of vertices of the tree



Graphical model for trees

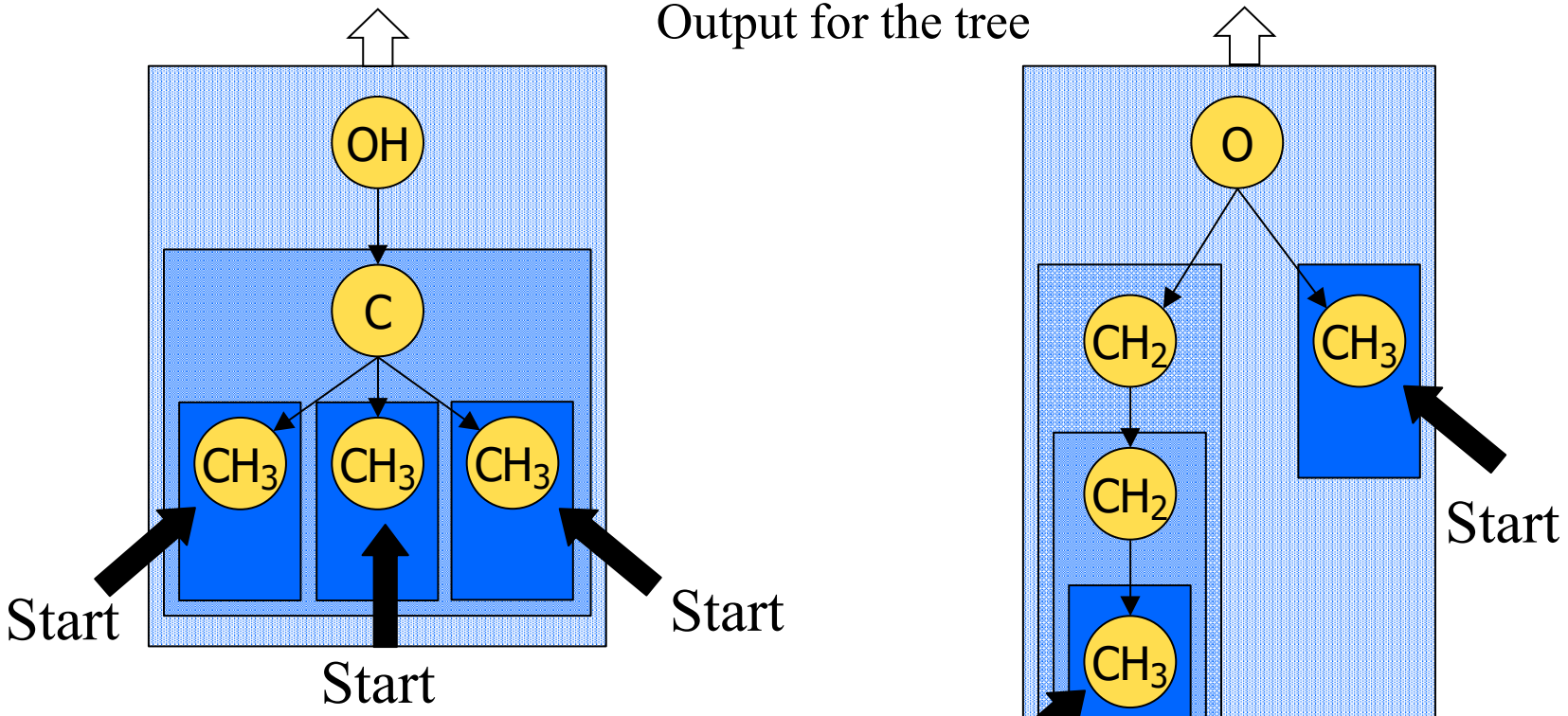


Unfolding the encoding process through structure

If we change the tree structure, the encoding change accordingly ! See next slide

RecNN over different structures

Dip. Informatica
University of Pisa



Examples on different trees (for chemical fragments):
Unfolding (encoding) through different structures.
Weight sharing: units are the same for the all the vertices of a tree and for all the trees in the data set!!!

ToC

- **Introduction**
  - Motivations
  - Historical notes



- **Main Course**
  - Introduction to Deep Graph Networks
  - Open Issues in Deep Graph Learning



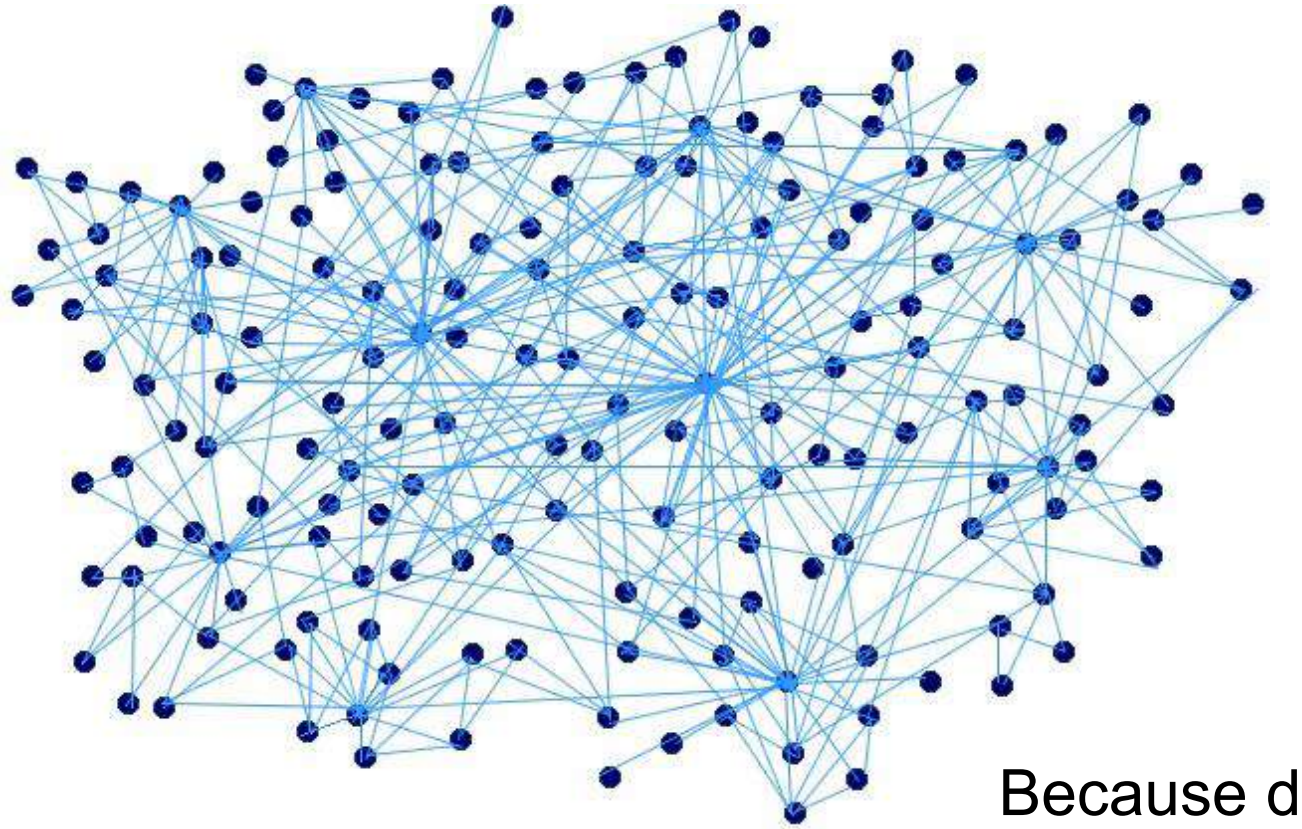
- **Our Recent Proposals**
  - Overview of some advancement in my lab
    (to open new collaborations among us)
  - Efficiency, Expressiveness, Graph XAI, Extensions



- **Other Approaches** (Kernels)
- **Perspectives & Conclusions**

Overview & Motivations

Why Structured/Graph Data?

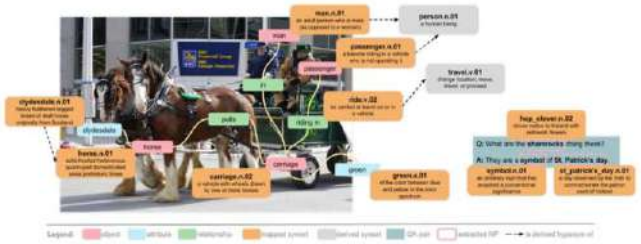


Because data have relationships

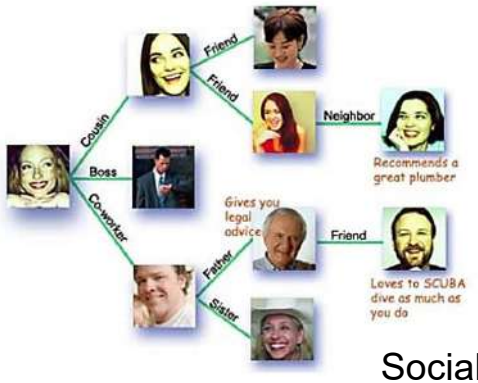
Examples: Graphs & Networks



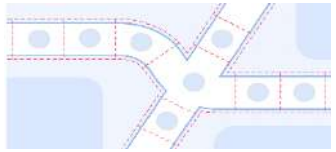
Abstraction of images



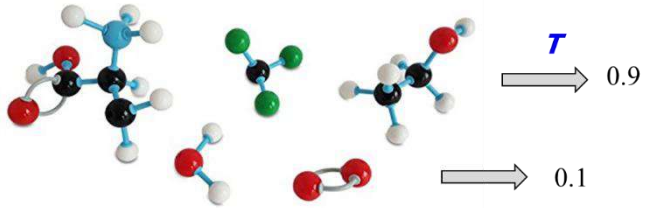
Scene understanding networks



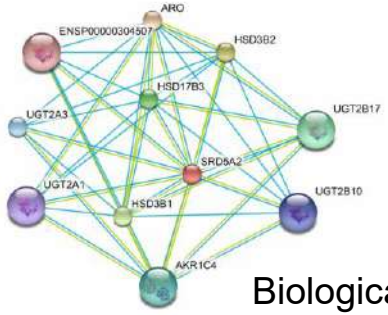
Social networks



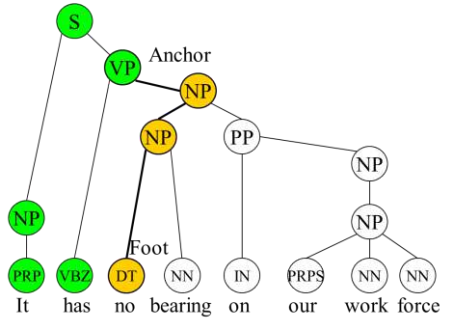
Transportation networks
(traffic predictions on G.maps@DeepMind)



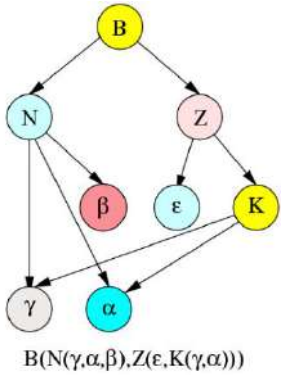
Small molecules (drug design)



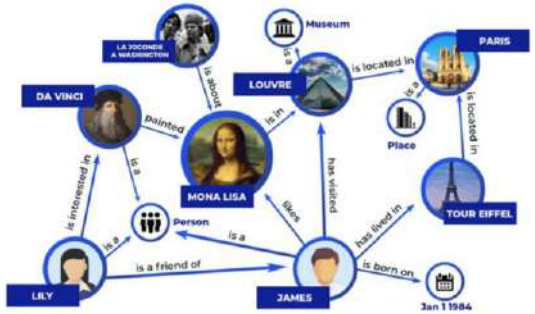
Biological networks
(proteins)



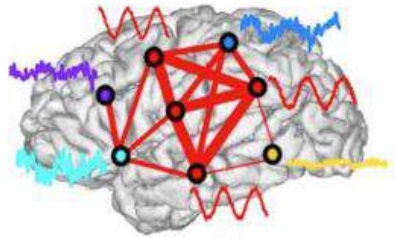
Language parsing



Logic terms



Knowledge graphs



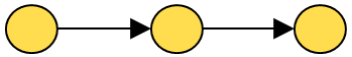
Brain (networks) modeling

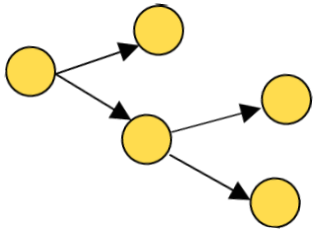
The scenario, terms (and trends)

Input Domain

Flat (vectors) (e.g. F.F.NN) | **Sequences** (RNN) | **Structures** (trees/graphs)

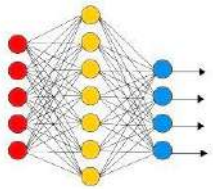

vector label [….]

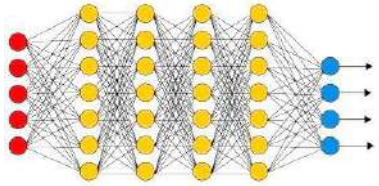




Layering

Shallow | **Deep** (for both F.F.NN and RNN)





Input rep. at multi-level of abstraction

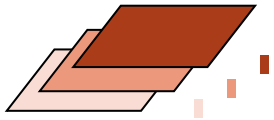
"Deep and wide" approaches: **high computational cost**!

Efficiency issue

Fully E2E trained NN | **Randomized/Incremental NN**

$\Delta \mathbf{w}$





Focus

Move toward "deep and *efficient*" approaches for SD

The Question:





Why learning on graphs is attracting more and more research and applications?



- Many **SOLUTIONS** to previously difficult applicative problems, not respecting the *real nature* of the data (the relationships)
  - Instead of moving *data to models* we move *models to data*



- Many **NEW RESEARCH** problems arise when the domain is changed from flat to structured



Historical Notes

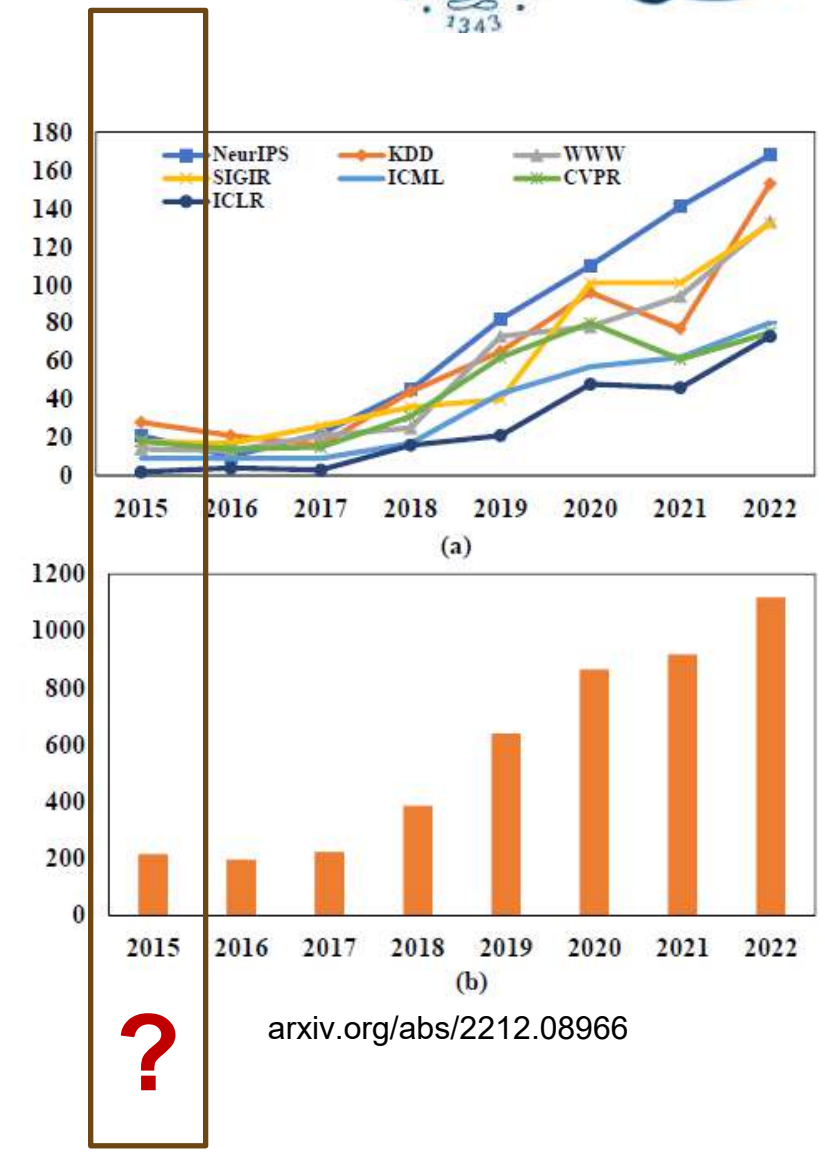
DGN: Evolutions

- Not surprisingly, nowadays, a world-wide “exploding” area:
  - Research literature is rapidly growing
  - And concrete real-world applications

Number of GL publications in recent years **on top venues**, including NeurIPS, KDD, WWW, SIGIR, ICML, CVPR and ICLR. Figures (a) and (b) depict the number of GL publications at top conferences and the **total number**, respectively

- Including different terminologies and perspectives, e.g.:
  - *Graph Representation Learning,*
  - *Learning on/for/with graphs,*
  - *Deep learning for graphs*
  - ***Deep graph networks (DGN)** framework* [Neural Netws. 2020]
  - *…*

We will see neural models: Neural networks for graphs / Graph Neural Networks (GNN) / Graph Convolutional networks (GCN)/ …

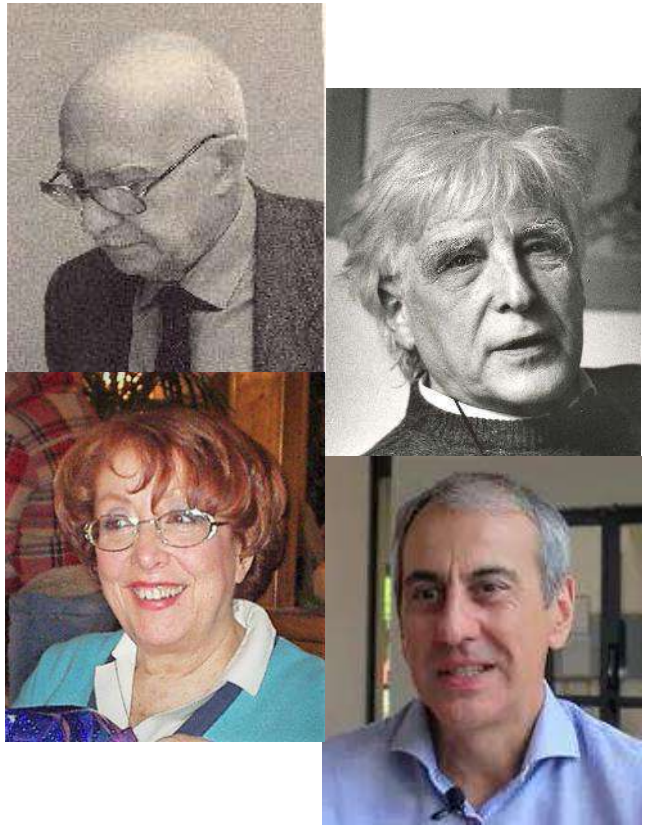


arxiv.org/abs/2212.08966

DGN: Origins, a Premise

The deep root and the path (timeline)

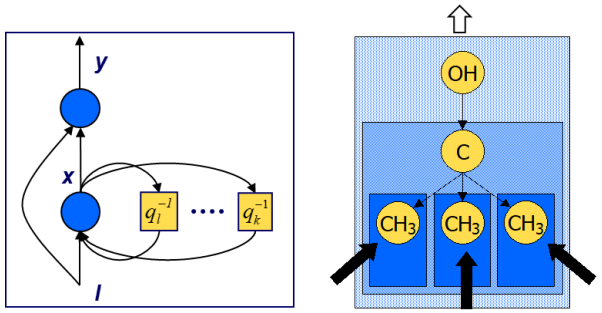
- **Edoardo Renato Caianello** (Napoli, Italy), 60's: Neural networks, Cybernetics
- **Valentino von Braitenberg**: Biocybernetics
- **Antonina Starita**: Pisa "Neurolab", 80's: Computational Intelligence and Bioengineering
- **Alessando Sperduti** (Pisa & Padova), 90's
- **Me ;-)**



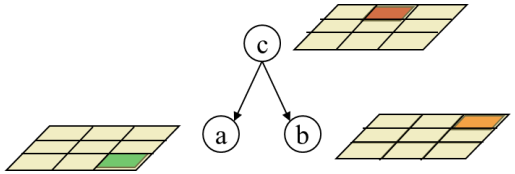
DGN: Origins (the roots, … and the trees)

Origins of NNs for **Structured Data**, mainly an Italian history:

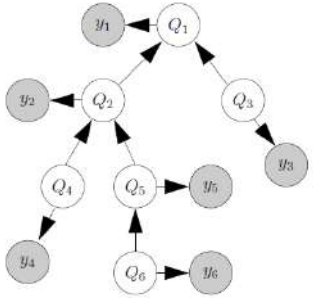
- Started with works on Trees, DPAGs etc. by *Recursive Neural Networks*
  (A. Sperduti, A. Starita,, P. Frasconi, M. Gori, B. Hammer, A. Micheli, M. Bianchini, F. Scarselli, M. Hagenbuchner, …)
  since **end of 90's**, continuing for more than 25 years developing a large family of models, **e.g.**



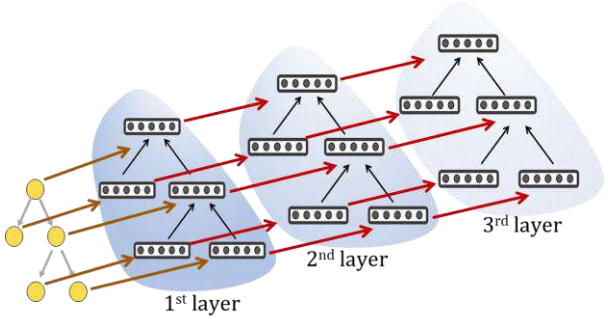
Recursive Neural Networks (1997-2000)



Self-Organizing Map for Structures (2003-2005)



Bottom-up Hidden Tree Markov Models (2012)



Tree ESN and Deep RC for Trees (2010-2018)

Origins of NNs for Structured Data (Trees/DPAGs/…): some references





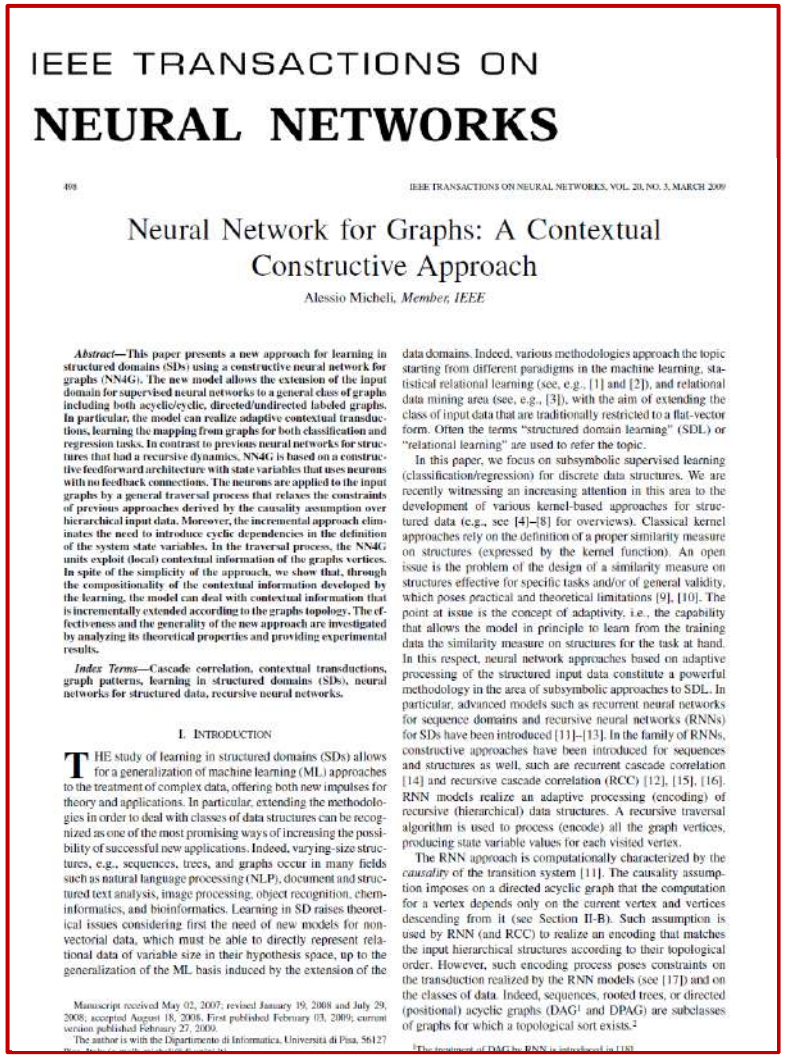
- A. Sperduti, A. Starita. Supervised Neural Networks for the Classification of Structures,IEEE Transactions on Neural Networks. Vol. 8, n. 3, pp. 714-735, 1997.
- P. Frasconi, M. Gori, and A. Sperduti. **A General Framework for Adaptive Processing of Data Structures**, IEEE Transactions on Neural Networks. Vol. 9, No. 5, pp. 768-786, 1998.
- A.M. Bianucci, A. Micheli, A. Sperduti, A. Starita. **Application of Cascade Correlation Networks for Structures to Chemistry**, Applied Intelligence Journal (Kluwer Academic Publishers), Special Issue on "Neural Networks and Structured Knowledge" Vol. 12 (1/2): 117-146, 2000.
- A. Micheli, D. Sona, A. Sperduti. Contextual Processing of Structured Data by Recursive Cascade Correlation. IEEE Transactions on Neural Networks. Vol. 15, n. 6, Pages 1396- 1410, 2004.
- B. Hammer, A. Micheli, and A. Sperduti.
  Universal Approximation Capability of Cascade Correlation for Structures.
  Neural Computation. Vol. 17, No. 5, Pages 1109-1159, (C) 2005 MIT press
- B. Hammer, A. Micheli, A. Sperduti, M. Strickert.
  Recursive Self-organizing Network Models. Neural Networks, Elsevier Science. Volume 17, Issues 8-9, Pages 1061-1085, 2004.

- C. Gallicchio, A. Micheli. **Tree Echo State Networks**, Neurocomputing, volume 101, pag. 319-337, 2013.
- C. Gallicchio, A. Micheli, **Deep** Reservoir Neural Networks for Trees. *Information Sciences 480, 174-193, 2019*

DGN: Origins

20th anniversary!

- Moved to **general graphs** (pioneered in Pisa and Siena, Italy, ***2005-2009***):
  1. Recursive NN: **GNN** [1], GraphESN [3] (see later)
  2. Layering/Convolutional: **NN4Gs** [2] in constructive version, and then continued as **Graph Convolutional NN** (**GCN**)
     - A line of research attracting more and more world-wide research since 2015…
- Deep Graph Networks (DGN) framework [4]



IEEE TRANSACTIONS ON

NEURAL NETWORKS

498 IEEE TRANSACTIONS ON NEURAL NETWORKS, VOL. 20, NO. 3, MARCH 2009

Neural Network for Graphs: A Contextual Constructive Approach

Alessio Micheli, *Member, IEEE*

Abstract—This paper presents a new approach for learning in structured domains (SDs) using a constructive neural network for graphs (NN4G). The new model allows the extension of the input domain for supervised neural networks to a general class of graphs including both acyclic/cyclic, directed/undirected labeled graphs. In particular, the model can realize adaptive contextual transductions, learning the mapping from graphs for both classification and regression tasks. In contrast to previous neural networks for structures that had a recursive dynamics, NN4G is based on a constructive feedforward architecture with state variables that uses neurons with no feedback connections. The neurons are applied to the input graphs by a general traversal process that relaxes the constraints of previous approaches derived by the causality assumption over hierarchical input data. Moreover, the incremental approach eliminates the need to introduce cyclic dependencies in the definition of the system state variables. In the traversal process, the NN4G units exploit (local) contextual information of the graphs vertices. In spite of the simplicity of the approach, we show that, through the compositionality of the contextual information developed by the learning, the model can deal with contextual information that is incrementally extended according to the graphs topology. The effectiveness and the generality of the new approach are investigated by analyzing its theoretical properties and providing experimental results.

Index Terms—Cascade correlation, contextual transductions, graph patterns, learning in structured domains (SDs), neural networks for structured data, recursive neural networks.

I. Introduction

THE study of learning in structured domains (SDs) allows for a generalization of machine learning (ML) approaches to the treatment of complex data, offering both new impulses for theory and applications. In particular, extending the methodologies in order to deal with classes of data structures can be recognized as one of the most promising ways of increasing the possibility of successful new applications. Indeed, varying-size structures, e.g., sequences, trees, and graphs occur in many fields such as natural language processing (NLP), document and structured text analysis, image processing, object recognition, chem-informatics, and bioinformatics. Learning in SD raises theoretical issues considering first the need of new models for non-vectorial data, which must be able to directly represent relational data of variable size in their hypothesis space, up to the generalization of the ML basis induced by the extension of the data domains. Indeed, various methodologies approach the topic starting from different paradigms in the machine learning, statistical relational learning (see, e.g., [1] and [2]), and relational data mining area (see, e.g., [3]), with the aim of extending the class of input data that are traditionally restricted to a flat-vector form. Often the terms "structured domain learning" (SDL) or "relational learning" are used to refer the topic.

In this paper, we focus on subsymbolic supervised learning (classification/regression) for discrete data structures. We are recently witnessing an increasing attention in this area to the development of various kernel-based approaches for structured data (e.g., see [4]–[8] for overviews). Classical kernel approaches rely on the definition of a proper similarity measure on structures (expressed by the kernel function). An open issue is the problem of the design of a similarity measure on structures effective for specific tasks and/or of general validity, which poses practical and theoretical limitations [9], [10]. The point at issue is the concept of adaptivity, i.e., the capability that allows the model in principle to learn from the training data the similarity measure on structures for the task at hand. In this respect, neural network approaches based on adaptive processing of the structured input data constitute a powerful methodology in the area of subsymbolic approaches to SDL. In particular, advanced models such as recurrent neural networks for sequence domains and recursive neural networks (RNNs) for SDs have been introduced [11]–[13]. In the family of RNNs, constructive approaches have been introduced for sequences and structures as well, such are recurrent cascade correlation [14] and recursive cascade correlation (RCC) [12], [15], [16]. RNN models realize an adaptive processing (encoding) of recursive (hierarchical) data structures. A recursive traversal algorithm is used to process (encode) all the graph vertices, producing state variable values for each visited vertex.

The RNN approach is computationally characterized by the *causality* of the transition system [11]. The causality assumption imposes on a directed acyclic graph that the computation for a vertex depends only on the current vertex and vertices descending from it (see Section II-B). Such assumption is used by RNN (and RCC) to realize an encoding that matches the input hierarchical structures according to their topological order. However, such encoding process poses constraints on the transduction realized by the RNN models (see [17]) and on the classes of data. Indeed, sequences, rooted trees, or directed (positional) acyclic graphs (DAG[1] and DPAG) are subclasses of graphs for which a topological sort exists.[2]

Manuscript received May 02, 2007; revised January 19, 2008 and July 29, 2008; accepted August 18, 2008. First published February 03, 2009; current version published February 27, 2009.

The author is with the Dipartimento di Informatica, Università di Pisa, 56127

[1]The treatment of DAG by RNN is introduced in [18].

[1] F. Scarselli, M. Gori, A.C.Tsoi, M. Hagenbuchner, G. Monfardini. **The graph neural network** model, *IEEE Transactions on Neural Networks*, 20(1): 61–80, **2009**.

[2] A. Micheli**. Neural network for graphs**: a contextual constructive approach, *IEEE Transactions on Neural Networks*, 20 (3): 498-511, **2009**.

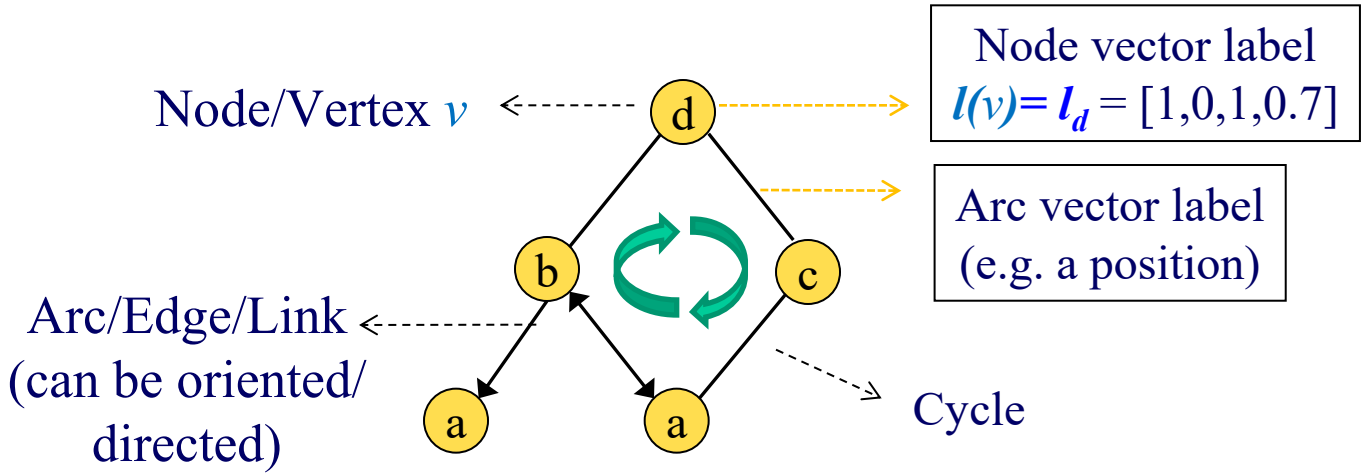
[3] C. Gallicchio, A. Micheli. **Graph Echo State Networks**, *IJCNN, 2010.*

[4] D.Bacciu, F. Errica, A. Micheli, M. Podda. A gentle introduction to deep learning for graphs, *Neural Netws. 2020*

Introduction to Deep Graph Networks

Our graphs (in the following)

Labeled graphs g



Call **A** the adjacency matrix of the graph

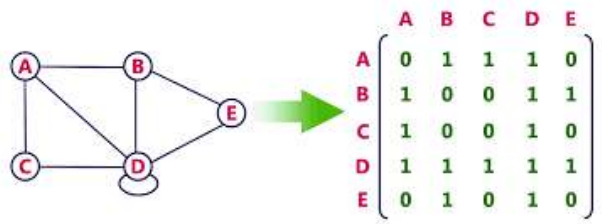
Graph Representations

The problem: there has been no systematic way (of general validity for any task) to extract features or metrics relations between examples for SD

- A representation learning instance (extended to SD)!

  - **Features based** representations are incomplete (or strongly task-dependent, e.g. topological indexes)

  - **Adjacent/incident matrix** representations (or other fixed-sizes representations). Issues:
    - Over-dim./incomplete (wasteful by padding/lose inf.)
    - Alignment among different graphs (!)
    - Topological order (it makes difficult the generalization)



  - "*The ability to treat the proper* ***inherent nature of the input data*** *is the key feature for a successful application of the machine learning methodologies*"

Learning Models for SD

SD + ML = Learning in Structured Domains

Instead of moving *data to models*
(e.g. Graphs into vectors or trees into sequences,
with alignment problems, loose of information, etc.)
we move *models to data*

Example (graphs): Molecules and Transductions

- The problem: there has been no systematic way to extract features or metrics relations between examples for SD
- **Goal**: to learn a mapping between a structured information domain (SD) and a discrete or continuous space (*transduction*)



*Property/Activity = **T**(Structure)*

T



Property Value (regression)
Toxic (yes/no) (classification)

QSPR: Correlate chemical structure of molecules with their properties

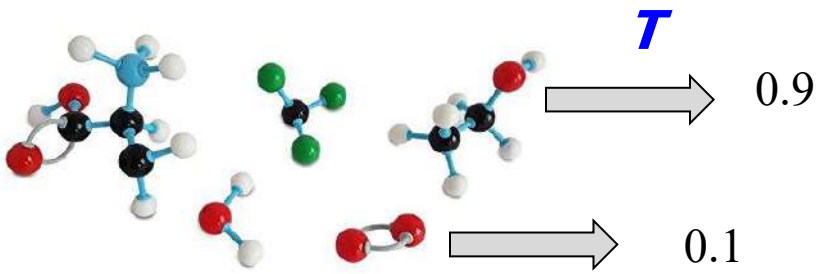
Can we predict directly from structures ?

Molecules are not vectors !
Molecules can be more naturally represented by varying size ***structures***

Learning a Transduction

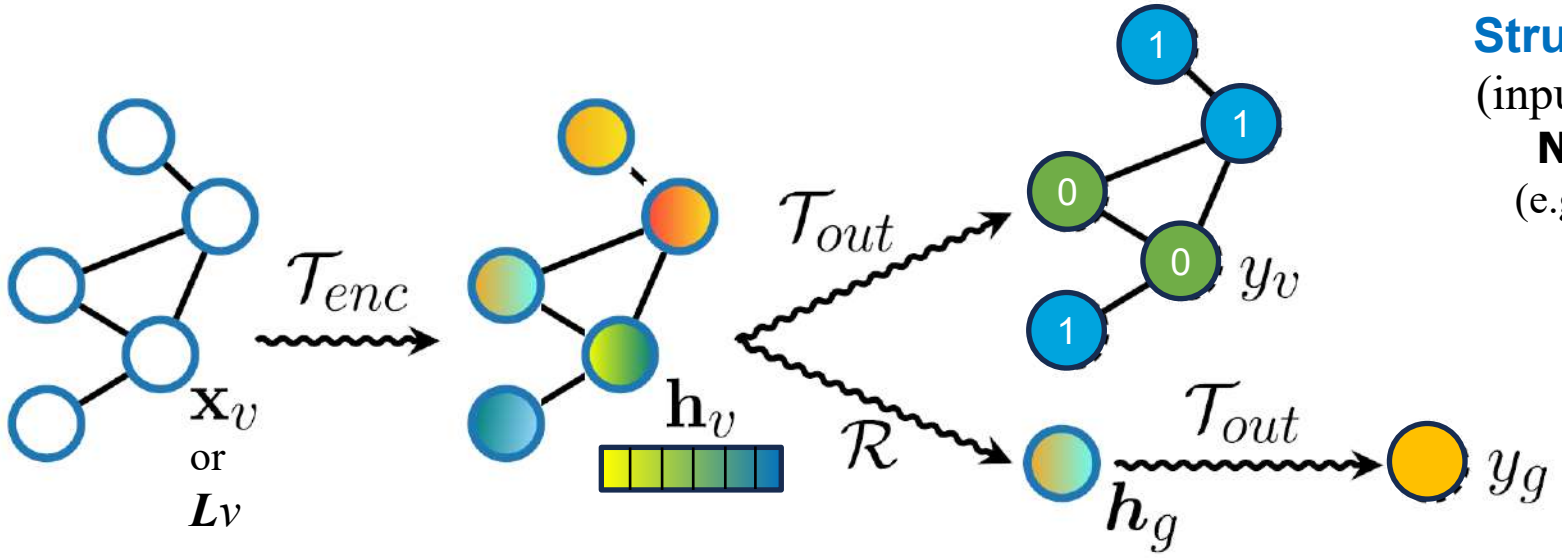
- **Goal**: to learn a mapping between a structured information domain (SD) and a discrete or continuous space (*transduction* ***T***).
- Start with this problem: classify variable size graphs
  - For instance, classify different graphs (or regression task) starting from a training set of know couples as in the molecules example

- Given a set of examples ($graph_i$,$target_i$)
- Learn a hypothesis, i.e a mapping ***T****(graph)*



Tasks and Transductions on Graphs

A transduction on graphs can be either



Structure-to-Structure
(input-output isomorphic)
Node based tasks
(e.g. node classification)

Structure-to-Scalar/Element
Graph based tasks
(regression/classification on graphs)

Labeled
Input graph

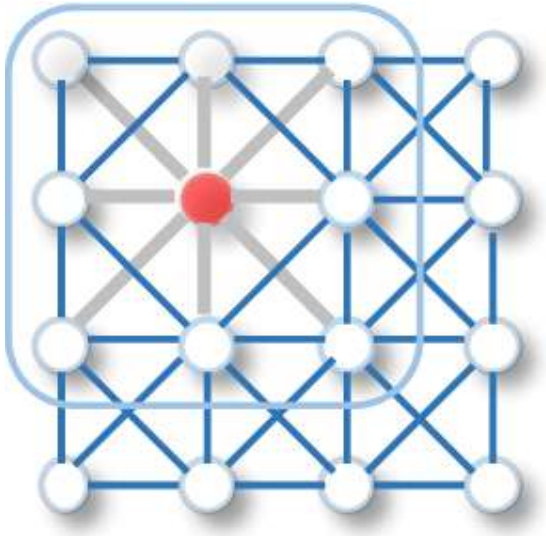
Node/Graph embedding
(latent space)
[hidden units values]

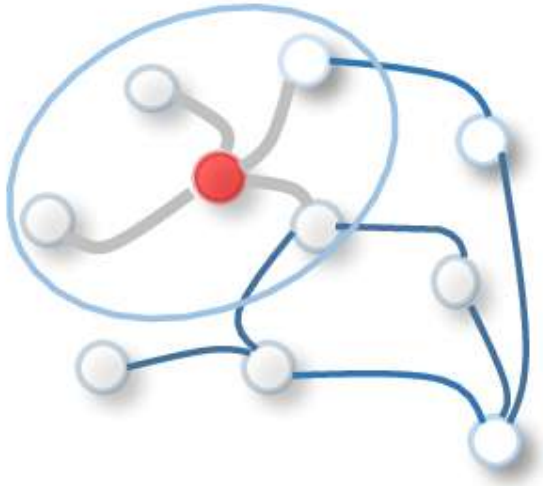
Or (in general) it can also be
non-isomorphic

And, … **Edge based tasks**
(e.g. link prediction)

How DGNs work? Layering/ Convolutional approches:

CNN and CGN both visit each node of the input data, but …





CNN: Convolutional kernel can be applied on a 2D regular grid (image matrix)

- The CNN takes a weighted average of the pixel values in the neighbor window
- The **neighbor** of a vertex are **ordered** and have a fixed size

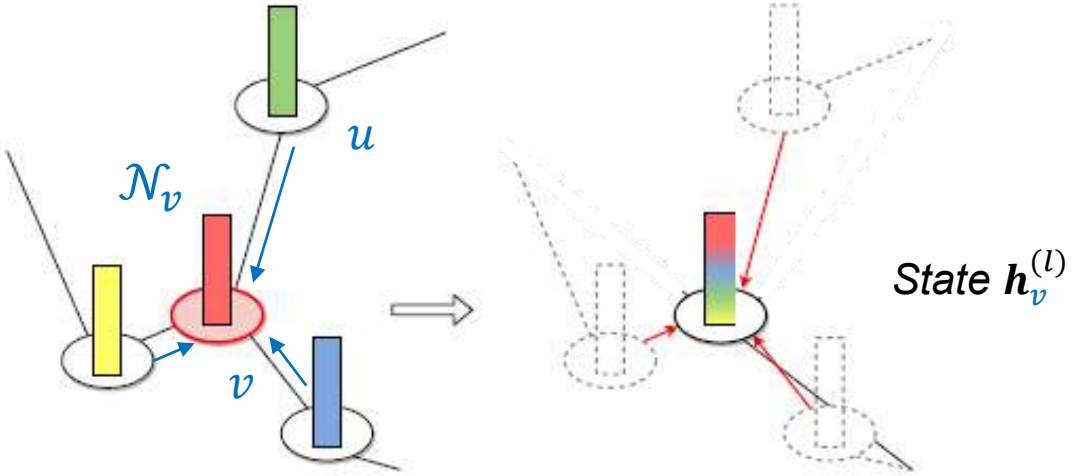
CNN cannot be immediately applied:
For **graphs** the **neighbors** of a vertex are **unordered and variable in size**

- We constrain the receptive field to **neighbors**, and we can exploit *weight sharing* to be permutation invariant so to *abstract from any ordering of the nodes*
- And **we extend from local neighbor through layering**: see next slides

How DGNs work? «Message Passing» Concept

$$\boldsymbol{h}_v^{(l)} = AGG_{\boldsymbol{W}^{(l)}}(\boldsymbol{L}_v, \boldsymbol{h}_v^{(l-1)}, \{\boldsymbol{h}_u^{(l-1)} : u \in \mathcal{N}(v)\}) \quad l = 1, \ldots, L$$

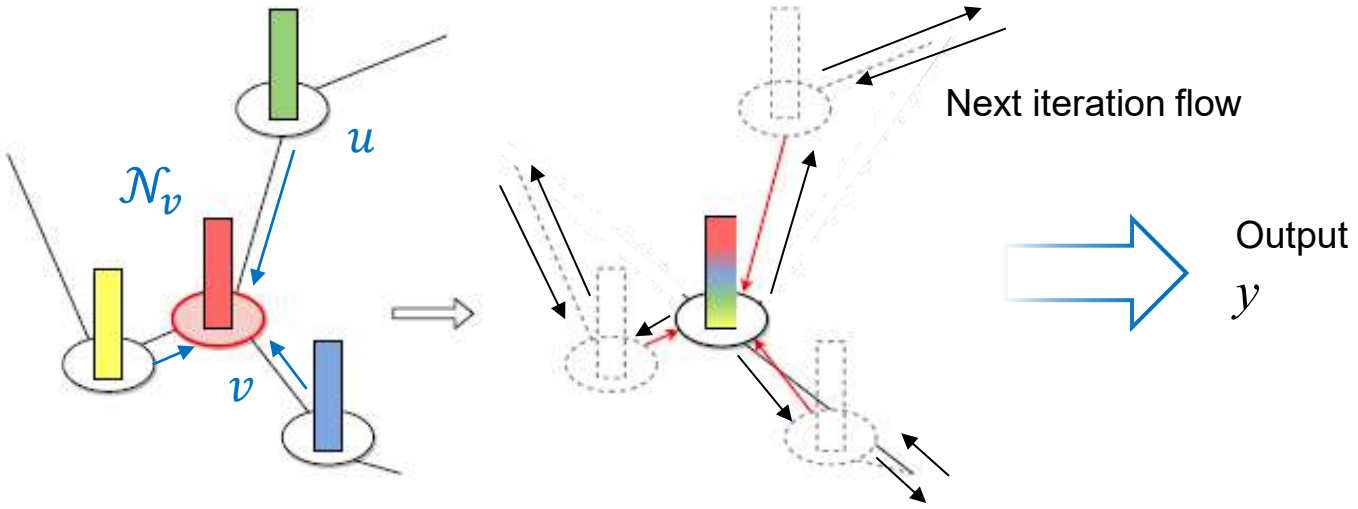
Message



- Nodes *"talk" each other* to inform about themselves and to be aware of the surrounding structure, by *messages*
- We compute for each node a *State* $\boldsymbol{h}_v^{(l)}$ combining the *messages* from $\mathcal{N}_v$ by *neural networks* with free parameters $\boldsymbol{W}$, allowing to to learn the node embedding

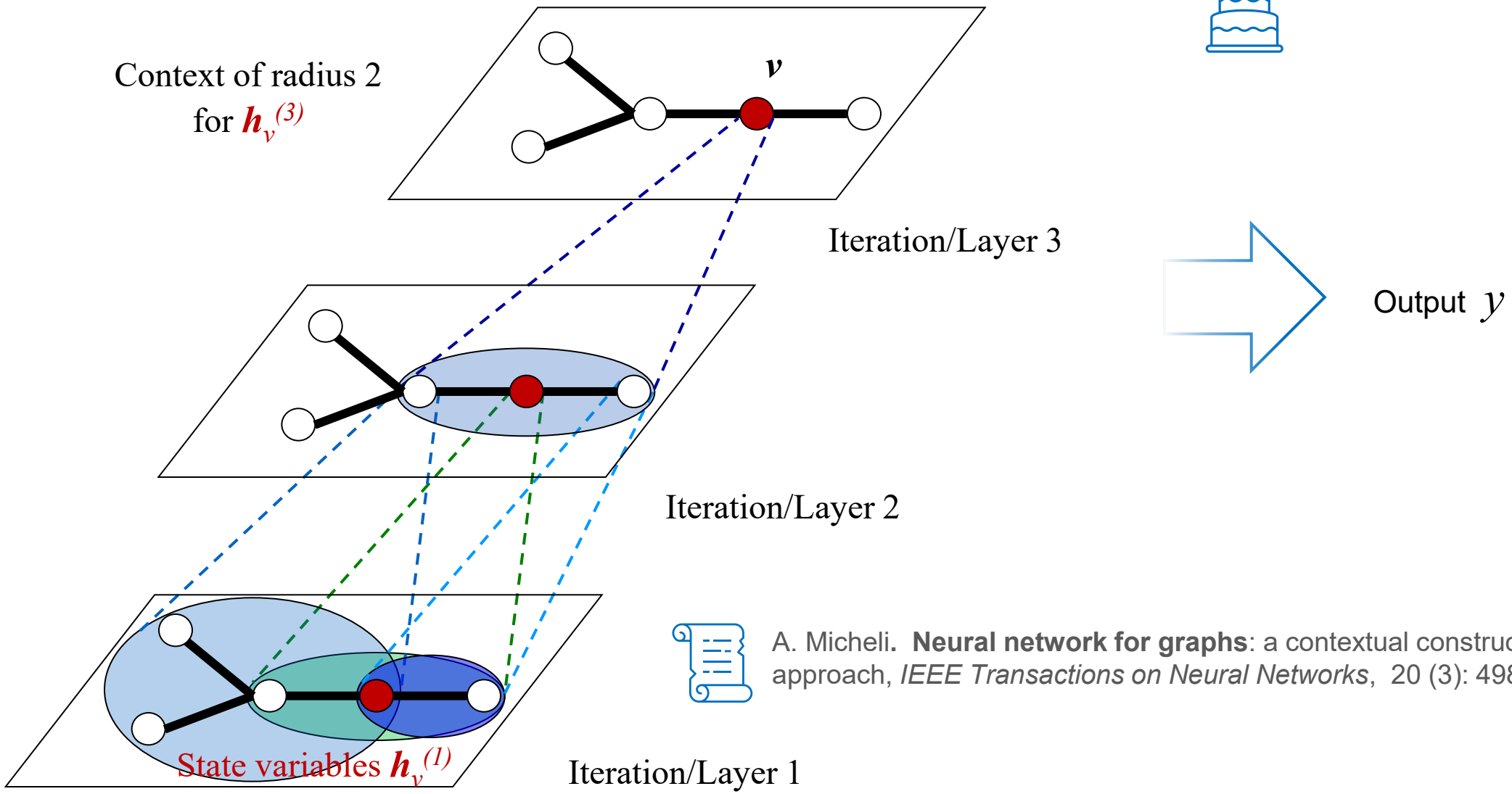
How DGNs work? «Message passing» Concept (II)

$$\boldsymbol{h}_v^{(l)} = AGG_{\boldsymbol{W}^{(l)}}(\boldsymbol{L}_v, \boldsymbol{h}_v^{(l-1)}, \{\boldsymbol{h}_u^{(l-1)} : u \in \mathcal{N}(v)\}) \quad l = 1, \ldots, L$$



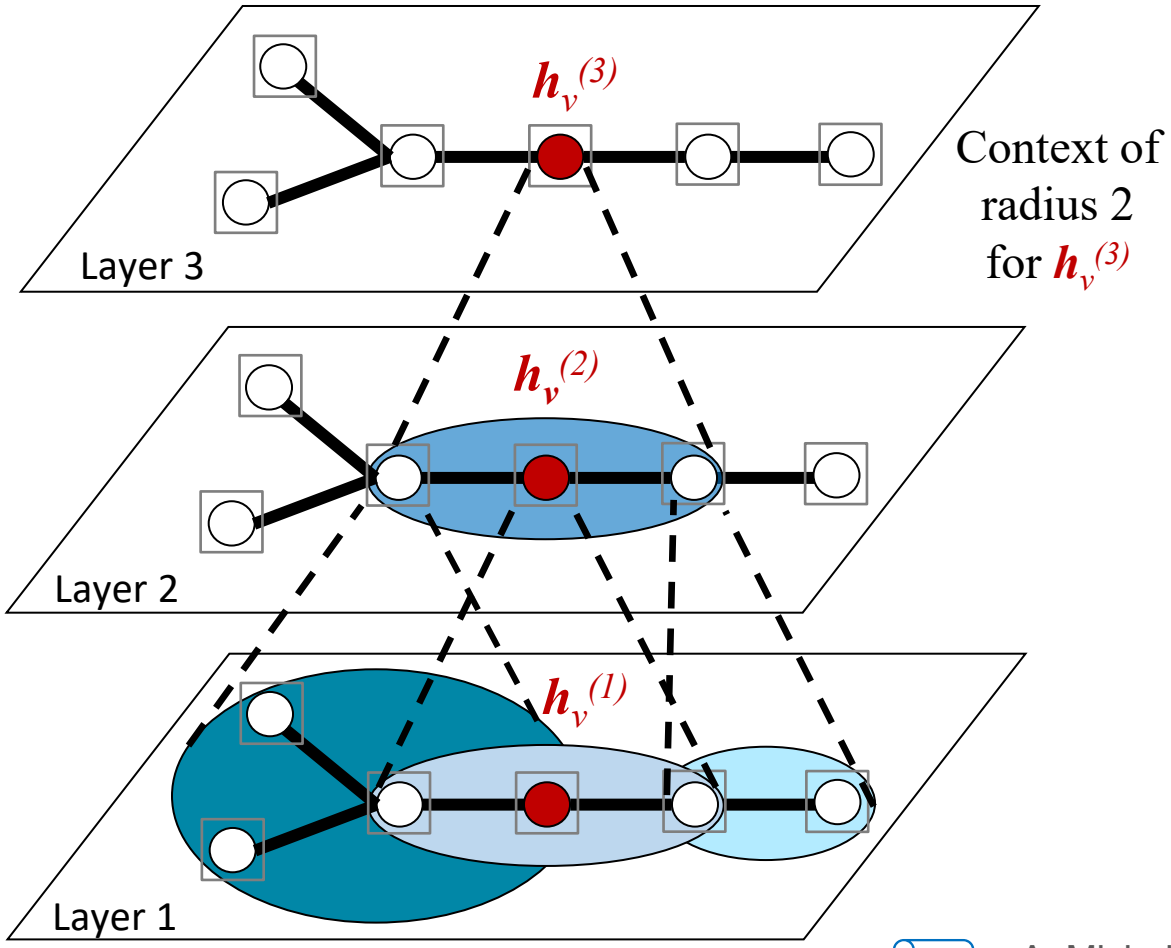
- The state does *not* depend only on local *neighborhood* $\mathcal{N}_v$: **Iterations** of this MP mechanism allows the **spread/diffusion of information** for all the nodes to all the others
- Finally, an **output** can be emittited **for each node** or **for the whole graph** through a permutation-invariant aggregation of node representations, also called *global pooling* (e.g. a sum or a mean operator)

«Message Passing» Concept: Iterations in GCN (the movie)



A. Micheli. **Neural network for graphs**: a contextual constructive approach, *IEEE Transactions on Neural Networks*, 20 (3): 498-511, **2009**

GCN & NN4G (2005-2009) Summary



Iterations l of MP are new layers

- For each node we compose the context/ receptive fields through layers (compositionality)
- Depth (DL) is functional to the context growth

→ DGNs are *inherently deep*

- Training the model we learn how to represent the graph:
  - E2E training for general **GCN**s
  - Layer-wise (incremental) training for **NN4G**
    - (automatic # of layers, no gradient vanish issues, etc)

A. Micheli. **Neural network for graphs**: a contextual constructive approach, *IEEE Transactions on Neural Networks*, 20 (3): 498-511, **2009**

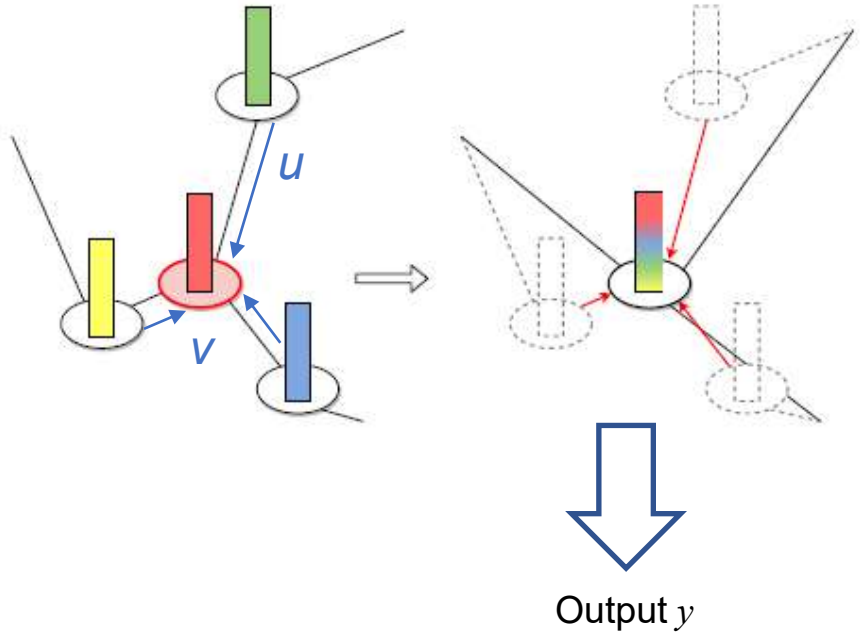
Message Passing: Details

- Iteration (over l) of

$$\boldsymbol{h}_v^{(l)} = AGG_{\boldsymbol{W}^{(l)}}\left(\boldsymbol{L}_v, \boldsymbol{h}_v^{(l-1)}, \{\boldsymbol{h}_u^{(l-1)} : u \in \mathcal{N}(\mathrm{v})\}\right) \quad l = 1, \ldots, L$$

Message

- $AGG_{\boldsymbol{W}^{(l)}}$ aggregates (propagates) messages from the neighbors nodes,
  - it includes a permutation invariant operator over the set of neighbors $\mathcal{N}$ (e.g. a sum),
  - and combinations of the NN units' activations in $\boldsymbol{h}$, where $\boldsymbol{W}$ are the free model parameters. $\boldsymbol{L}_v$ (or $\boldsymbol{x}_v$) are the node's labels.
  - $\mathcal{N}$ is according to the adjacency matrix **A** of the graph.
- Applying this computation to each node of the graph corresponds to a parallel, unordered visit of the graph at each iteration l



Message Passing: Implementations

$$\boldsymbol{h}_v^{(l)} = AGG_{\boldsymbol{W}^{(l)}}(\boldsymbol{L}_v, \boldsymbol{h}_v^{(l-1)}, \{\boldsymbol{h}_u^{(l-1)}: u \in \mathcal{N}(\mathrm{v})\}) \quad l = 1, \ldots, L$$

- We have a large flexibility for implementing these operators, with many model variations (**A** is the adjacency matrix of the graph):
  - **GCN** [Kipf, Welling, ICLR 2017] $\boldsymbol{h}^{(l)} = \mathrm{relu}\,(\widehat{\mathbf{A}}\,\boldsymbol{h}^{(l-1)}\,\boldsymbol{W}^{(l)})$, $\widehat{\mathbf{A}}$ degree normalized
  - **GraphConv** [Morris et al., AAAI 2019] $\boldsymbol{h}^{(l)} = f\,(\boldsymbol{W}_1^{(l)}\boldsymbol{h}^{(l-1)} + \boldsymbol{W}_2^{(l)}\mathbf{A}\boldsymbol{h}^{(l-1)})$
  - **GAT** [Veličković et al., ICLR 2018]: Attention-based aggregation (by $\alpha_{v,u}$)



pytorch-geometric

We named as **DGN** framework (including NN, Bayesian, etc) [Neural Netws. 2020]

NN4G (GCN): The Equations

Message

$$\boldsymbol{h}_v^{(l)} = AGG_{\boldsymbol{W}^{(l)}}(\boldsymbol{L}_v, \boldsymbol{h}_v^{(l-1)}, \{\boldsymbol{h}_u^{(l-1)} : u \in \mathcal{N}(v)\}) \quad l = 1, \ldots, L$$

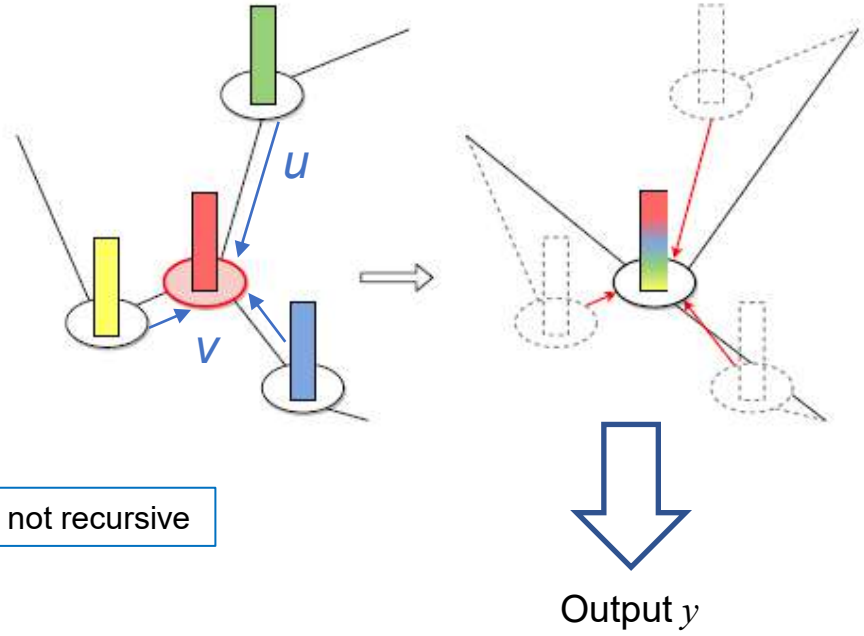
- NN4G computes a (one) state $\boldsymbol{h}_v^{(l)}$ for each layer and for each vertex v in this way:

$$\boldsymbol{h}_v^{(l)} = \begin{cases} h_v{}^{(1)} = f\left(\sum_{j=0}^{L^v} \bar{w}_{1j} L_j(v)\right) & \text{First layer } (l=1) \\ h_v{}^{(l)} = f\left(\sum_{j=0}^{L^v} \bar{w}_{lj} L_j(v) + \sum_{j=1}^{l-1} \hat{w}_{lj} \sum_{u \in N(v)} h_u{}^{(j)}\right) & l = 2, \ldots, N \end{cases}$$

Previous frozen layers: It is not recursive

Current Label of v

Context from the Neigh.s from (all) previous layers



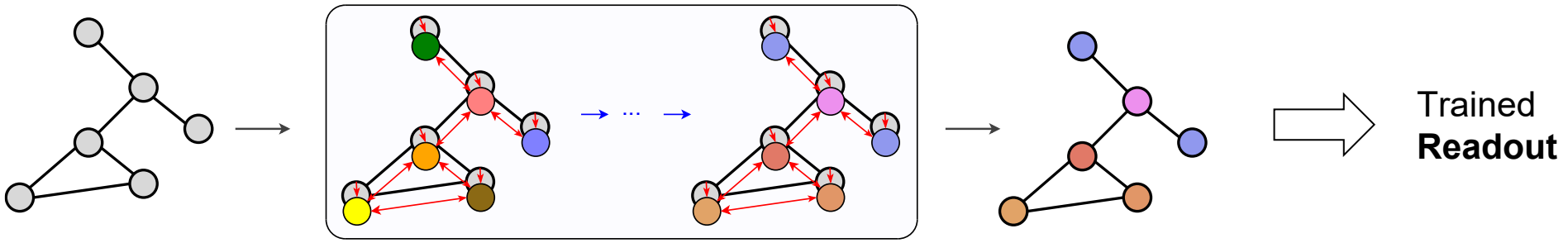
A. Micheli. **Neural network for graphs**: a contextual constructive approach, *IEEE Transactions on Neural Networks*, 20 (3): 498-511, **2009**

GNN-GESN (2005-2010)

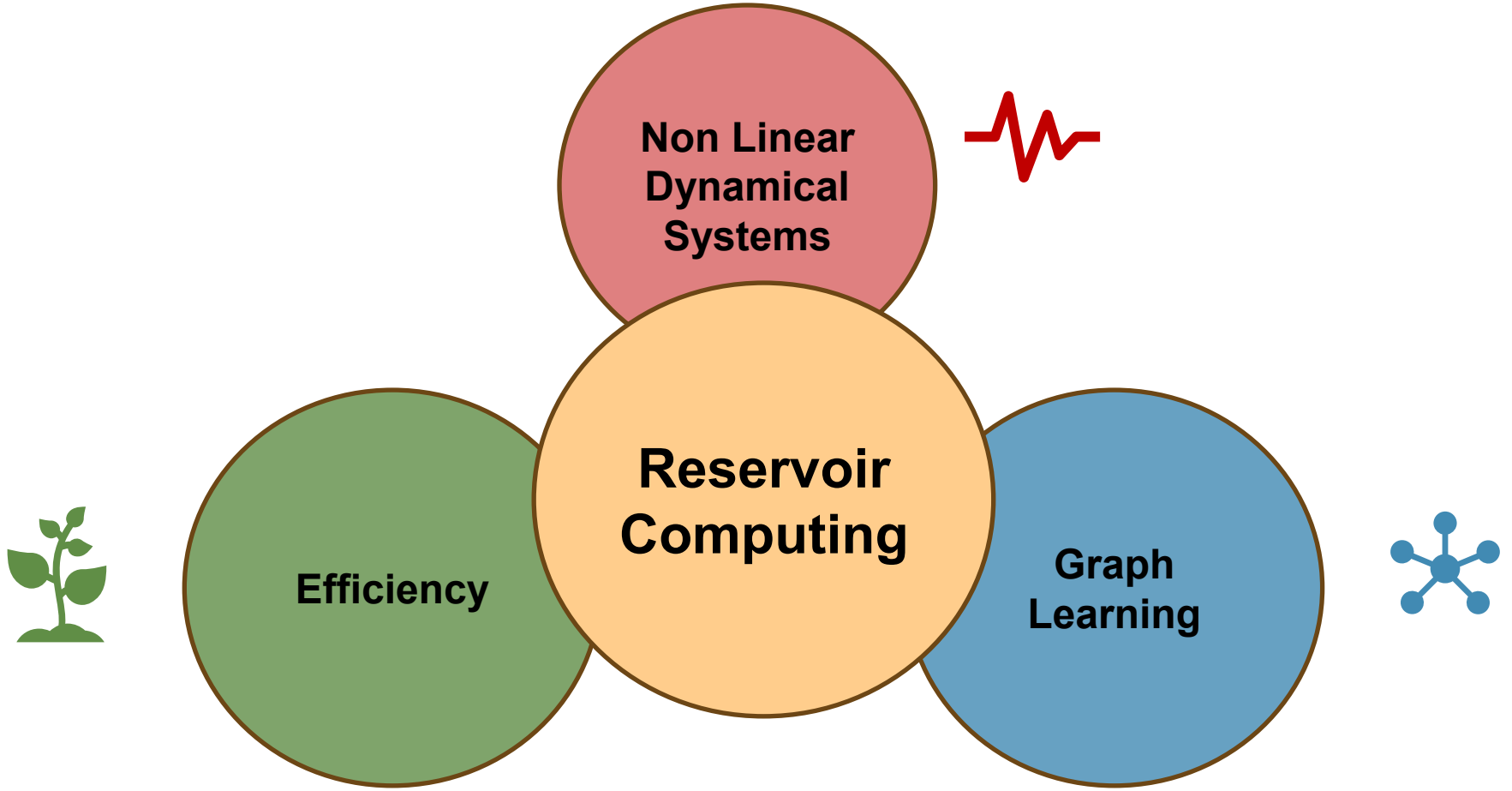
- For the *recursive approaches* (GNN, GESN) the diffusion process is similar, but l is the number of MP iterations within the same layer (or different layers with shared weights among layers)

$$\boldsymbol{h}_v^{(l)} = AGG_{\boldsymbol{W}^{(l)}}(\boldsymbol{L}_v, \boldsymbol{h}_v^{(l-1)}, \{\boldsymbol{h}_u^{(l-1)} : u \in \mathcal{N}(\mathrm{v})\})$$

- Dynamical system: typically requires convergence to a fixed point as graph embedding, constraining (MP) state dynamics to be contractive
  - **GNN** with training for the embedding [Scarselli, et al., TNNLS 2009]
  - **GESN** (extending **randomized** Echo State Networks for sequences): **untrained** message-passing function (but direct control of the dynamical system parameters) [Gallicchio, Micheli, IJCNN, 2010; AAAI 2020]



GraphESN: Reservoir Computing for Graphs



GraphESN: Reservoir Computing for Graphs

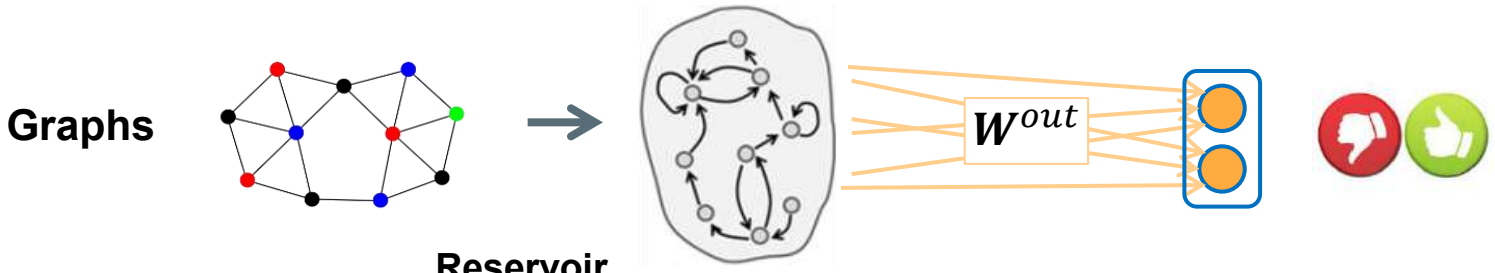


We can extend RC to efficient graph learning: GraphESN (GESN)

- Origins: Gallicchio, Micheli. *Graph echo state networks*, IJCNN 2010
- Deep: Gallicchio, Micheli. *Fast and Deep Graph Neural Networks*. **AAAI 2020**

Reservoir: Abstract randomized **encoder machine for graphs**

- **No training** (No backprop)



Untrained iterative state computation & **node embedding**

closed-form solution

GESN: The Equations

Message

$$\boldsymbol{h}_v^{(l)} = AGG_{\boldsymbol{W}^{(l)}}(\boldsymbol{L}_v, \boldsymbol{h}_v^{(l-1)}, \{\boldsymbol{h}_u^{(l-1)} : u \in \mathcal{N}(v)\}) \quad l = 1, \ldots, L$$

GraphESN compute state variables (here a vector $\boldsymbol{h}$ of reservoir units) for each vertex v in this way:

GraphESN state transitions (recursive) for $\boldsymbol{l}$ **iterations** until convergence

$$\boldsymbol{h}_v{}^{(l)} = tanh\left(\sum_{j=0}^{L^v} \bar{w}_{lj} L_j(v) + \sum_{u \in N(v)} \widehat{\boldsymbol{W}} \boldsymbol{h}_u{}^{(l-1)}\right)$$

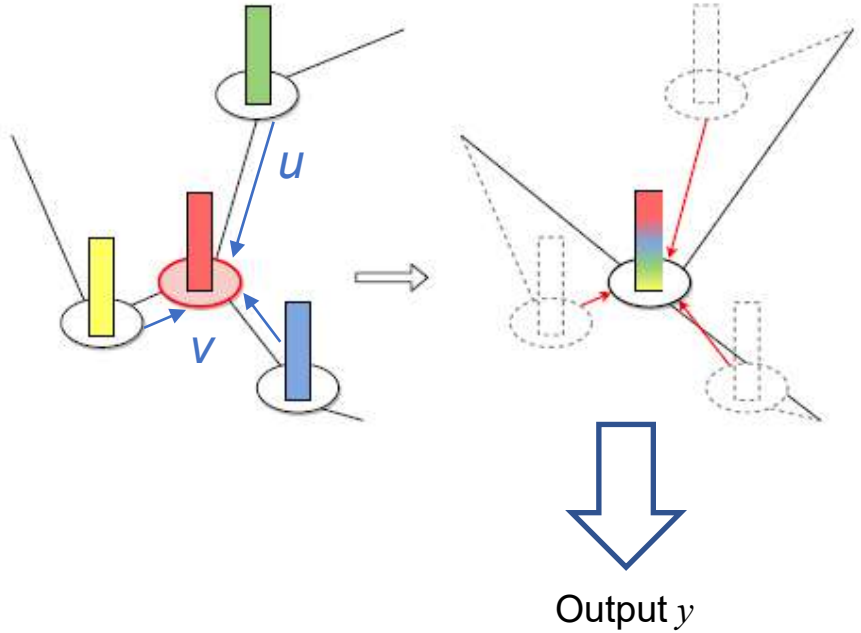
Connectivity of the Reservoir

Context from the Neigh.s recursively from previous iteration

Think to a RNN, with initial null states for the Neighbors

$$y(\boldsymbol{g}) = \mathbf{W}_{out} \mathrm{X}(\boldsymbol{h}(\boldsymbol{g}))$$

Output for a graph $\boldsymbol{g}$ by *global pooling* X and then by a linear **Readout** trained by pseudo-inversion/Ridge Regression

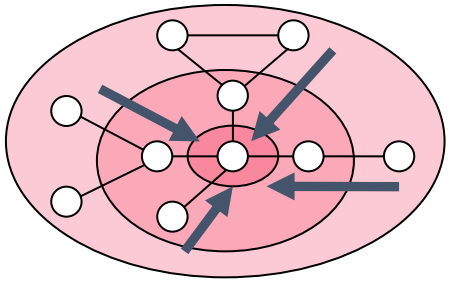


[Gallicchio, Micheli, IJCNN, 2010; AAAI 2020]

GESN: Iterations

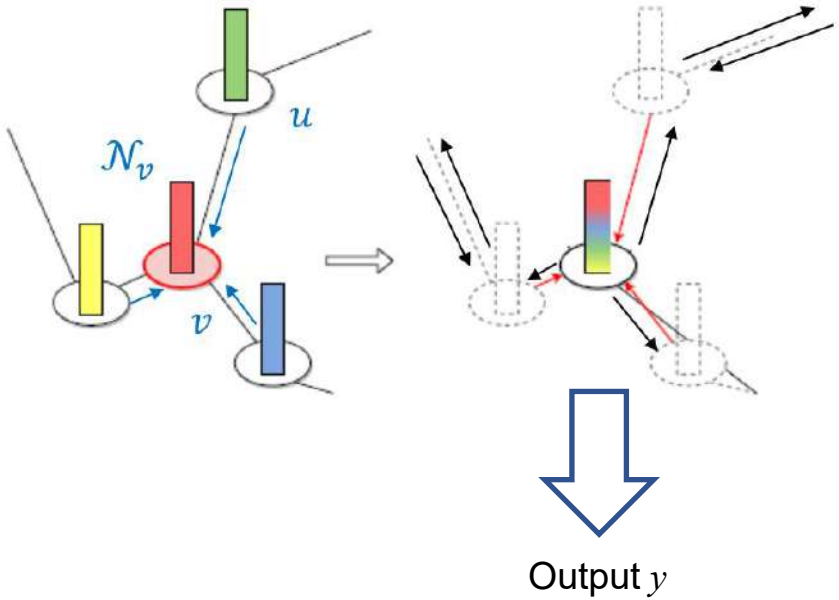
Message

$$\boldsymbol{h}_v^{(l)} = AGG_{\boldsymbol{W}^{(l)}}(\boldsymbol{L}_v, \boldsymbol{h}_v^{(l-1)}, \{\boldsymbol{h}_u^{(l-1)} : u \in \mathcal{N}(\mathrm{v})\}) \quad l = 1, \ldots, L$$



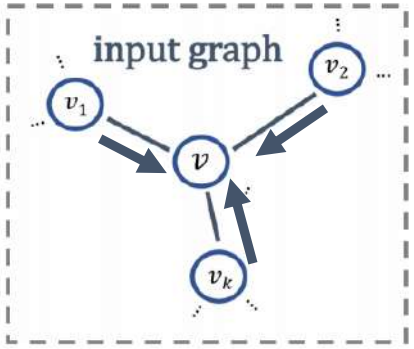
Context evolution, with the recursive iterations, of the state, shown for the vertex in the center

Not just local processing, by *diffusion on graph*

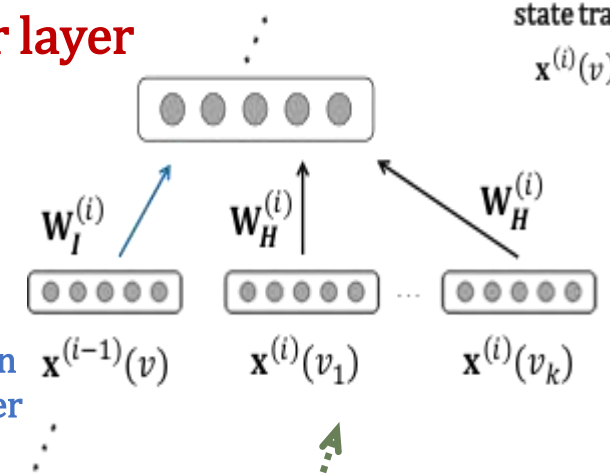


1. FDGNN: DeepGESN

Gallicchio, Claudio, and Alessio Micheli. **"Fast and Deep Graph Neural Networks."** AAAI 2020



i-th reservoir layer



embedding in previous layer

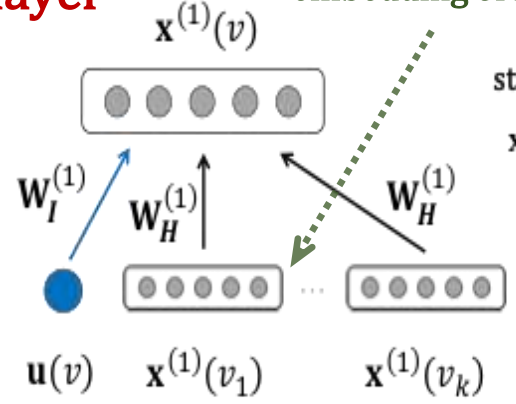
state transition function

$$\mathbf{x}^{(i)}(v) = \tau^{(i)}\left(\mathbf{x}^{(i-1)}(v), \mathbf{x}^{(i)}(v_1), \ldots, \mathbf{x}^{(i)}(v_k)\right)$$
$$= \tanh\left(\mathbf{W}_I^{(i)}\mathbf{x}^{(i-1)}(v) + \sum_{v_j \in N(v)} \mathbf{W}_H^{(i)}\mathbf{x}^{(i)}(v_j)\right)$$

Exploit embedding from previous layer to develop a higher order embedding

1-st reservoir layer

embedding of neighbors



vertex label

GraphESN

state transition function

$$\mathbf{x}^{(1)}(v) = \tau^{(1)}\left(\mathbf{u}(v), \mathbf{x}^{(1)}(v_1), \ldots, \mathbf{x}^{(1)}(v_k)\right)$$
$$= \tanh\left(\mathbf{W}_I^{(1)}\mathbf{u}(v) + \sum_{v_j \in N(v)} \mathbf{W}_H^{(1)}\mathbf{x}^{(1)}(v_j)\right)$$

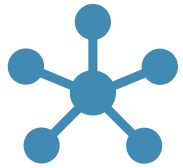
Fixed point eq.

Open Issues in Deep Graph Learning



Efficiency

Scaling training to very large graphs, e.g. >100K nodes



Expressiveness

Deep learning models may fail to learn meaningful node representations



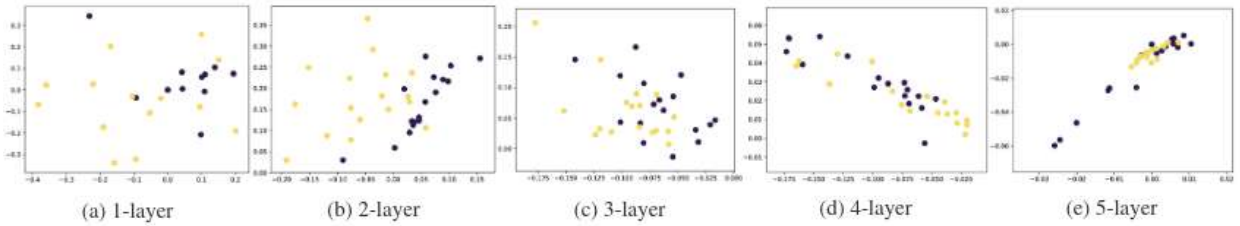
Theory

Characterization of DGN models

Expressiveness: the Over-* cases

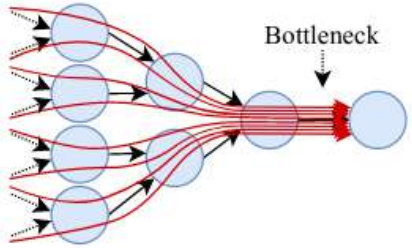
- **Over-smoothing**
  - Accuracy typically declines beyond 4-5 layers
  - Node representations collapse in deeper layers
  - Message-passing acts as diffusion
    - *high-level representations become similar*

[Li & al. , AAAI 2018]



- **Over-squashing**
  - Bottleneck: receptive fields increase exponentially with depth in a fixed-size node embedding dimension [Alon, Yahav, ICLR 21].
  - Exponential decrease in **sensitivity** of node repres. to the input features, as the number of layers $\boldsymbol{h}_v^{(L)}$ increases, computed as the Jacobian:

$$\left\| \frac{\partial \boldsymbol{h}_v^{(L)}}{\partial \boldsymbol{x}_u} \right\|$$

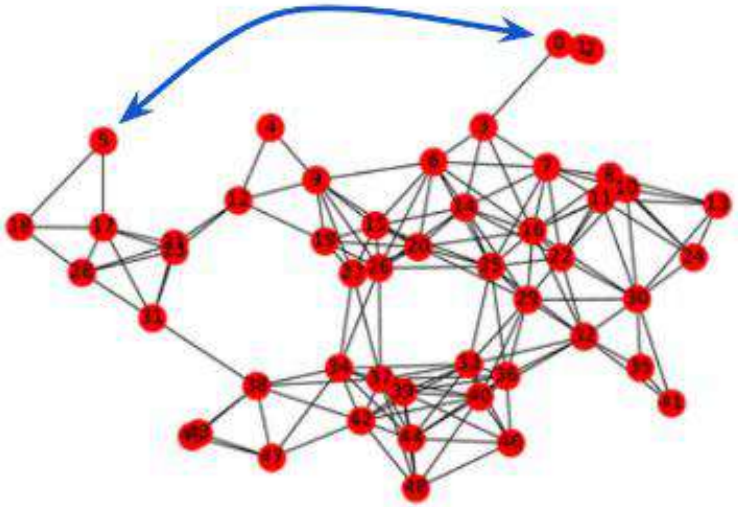


- **For E2E training**: difficulty of backpropagating the gradient through many (MP) layers

- The interplay among them is an open research issue

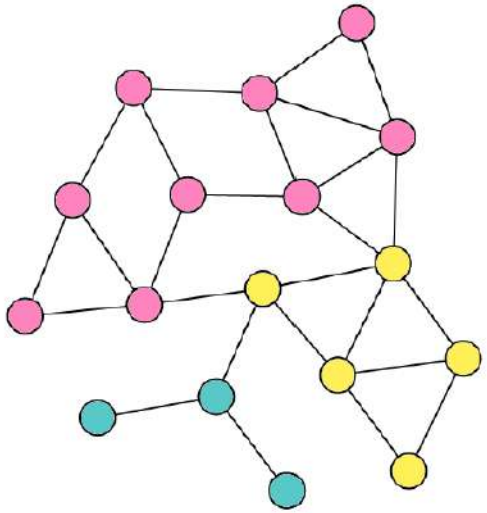
• Impact 1: Under-reaching

- **Impact 1: Under-reaching**
  - Over-* may prevent to learn interactions between distant nodes

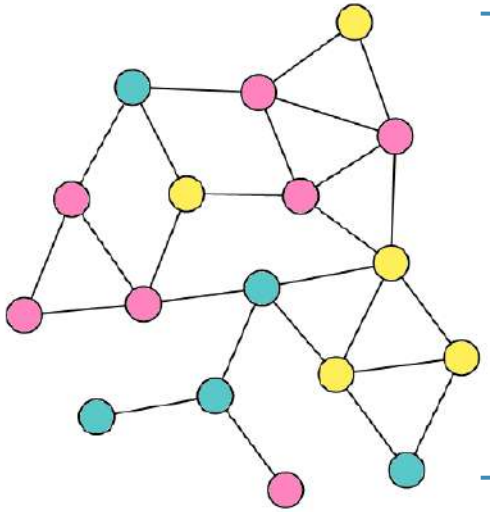


Impact 2: Heterophily Issue in Node Classification

- **Homophilic graphs**: nodes in the same neighborhood mostly share the same class
- **Heterophilic graphs**: nodes belonging to the same class are generally further apart (predictions that rely chiefly on a node's immediate neighbors can be misleading)



high homophily



low homophily

(!)

Potentially difficult tasks for standard MP

- impact of the over-*
- bias toward «homogeneous» graphs

A. Micheli, D. Tortorella. *Addressing heterophily in node classification with graph echo state networks* . Neurocomputing (2023).

Theory: Isomorphism and Weisfeiler-Leman (WL) Test

- **How many graphs can DGN learn to distinguish?**
  - *Vanilla DGN cannot be more discriminating than WL test*, example:



- 1-dim. Weisfeiler-Leman graph isomorphism heuristic
  - Check a relaxations of graph isomorphism
  - Polynomial time heuristic
- **DGNs expressiveness:**
  - Message-passing similar to WL coloring
  - MP upper bounded by WL *expressiveness*
  - ➢ There exist versions of DGNs that go beyond 1-WL
- **Characterization of models by bias analysis** → see later

- K. Xu, W. Hu, J. Leskovec, and S.Jegelka, *How Powerful are Graph Neural Networks?* ICLR 2019
- G. A. D'Inverno, M. Bianchini, M. L. Sampoli, and F. Scarselli, *On the approximation capability of GNNs in node classification/ regression tasks*, Soft Computing 2024

- C. Morris, M. Ritzert et al. *Weisfeiler and Leman Go Neural: Higher-order Graph Neural Network*, AAAI 2019

Our Proposals



EFFICIENCY ANALYSIS

Beyond E2E backpropation in DGN

Reservoir Computing for Graphs: Efficiency

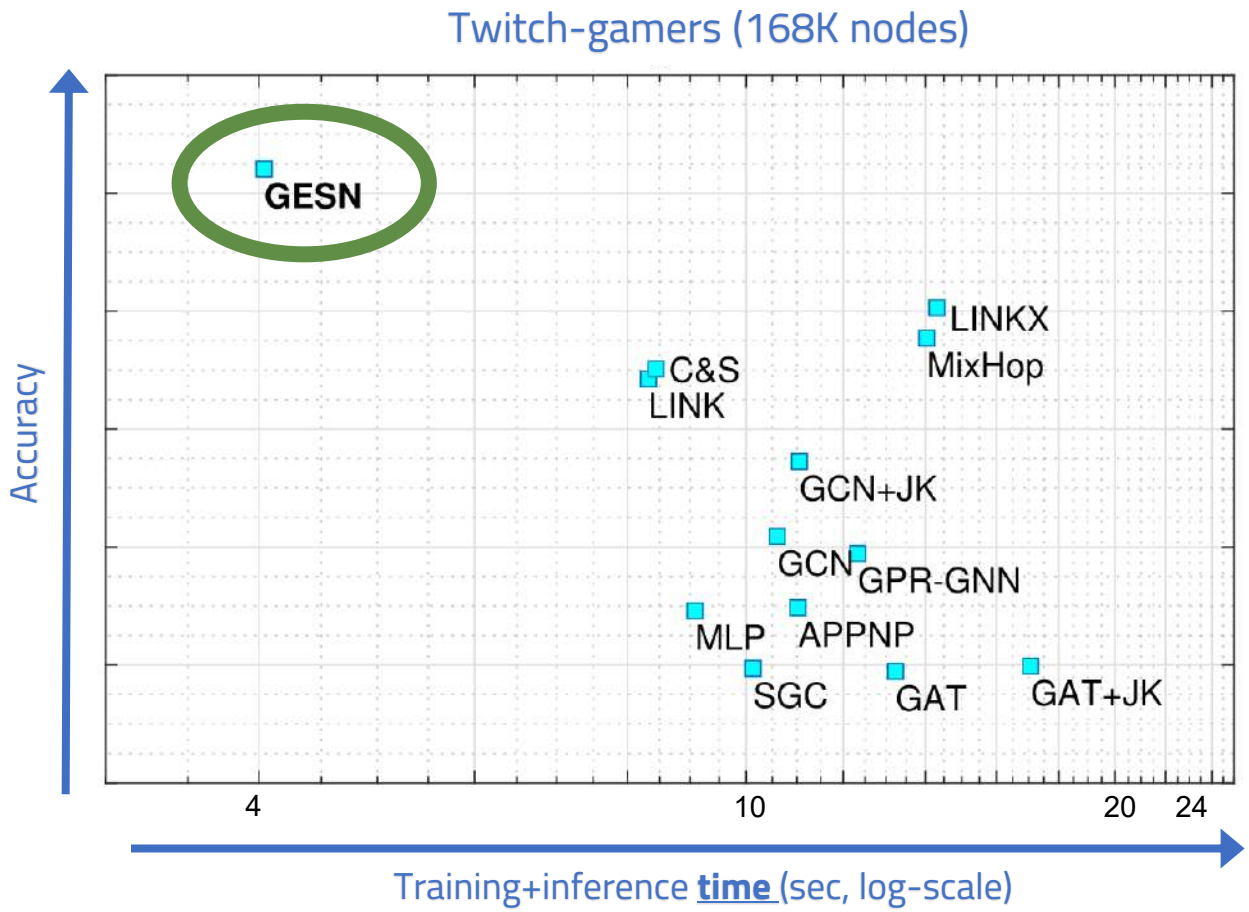
Accuracy equal or better than most end-to-end trained models

Graph classification:

- GESN is up to **900** times faster
- Time in seconds vs hours even in small data sets…

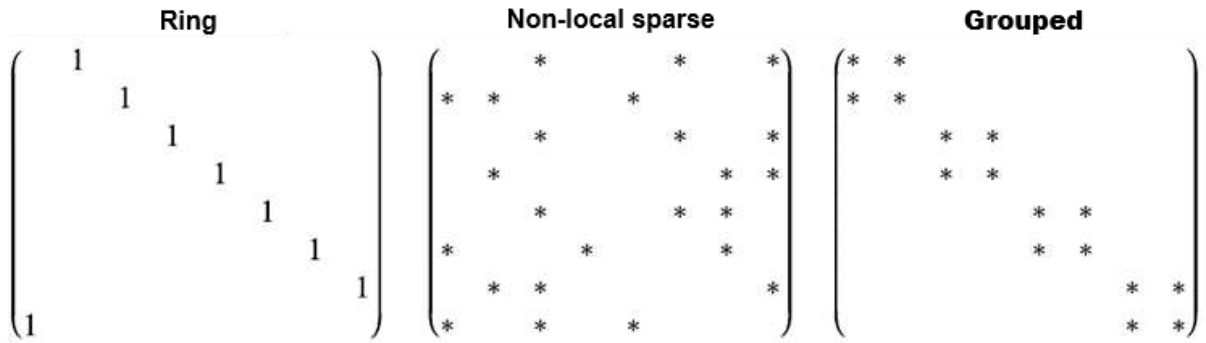
Node classification:

- A speed-up upto **10x** compared to other models
- Training scales easily to very large graphs, e.g. **>100K nodes**

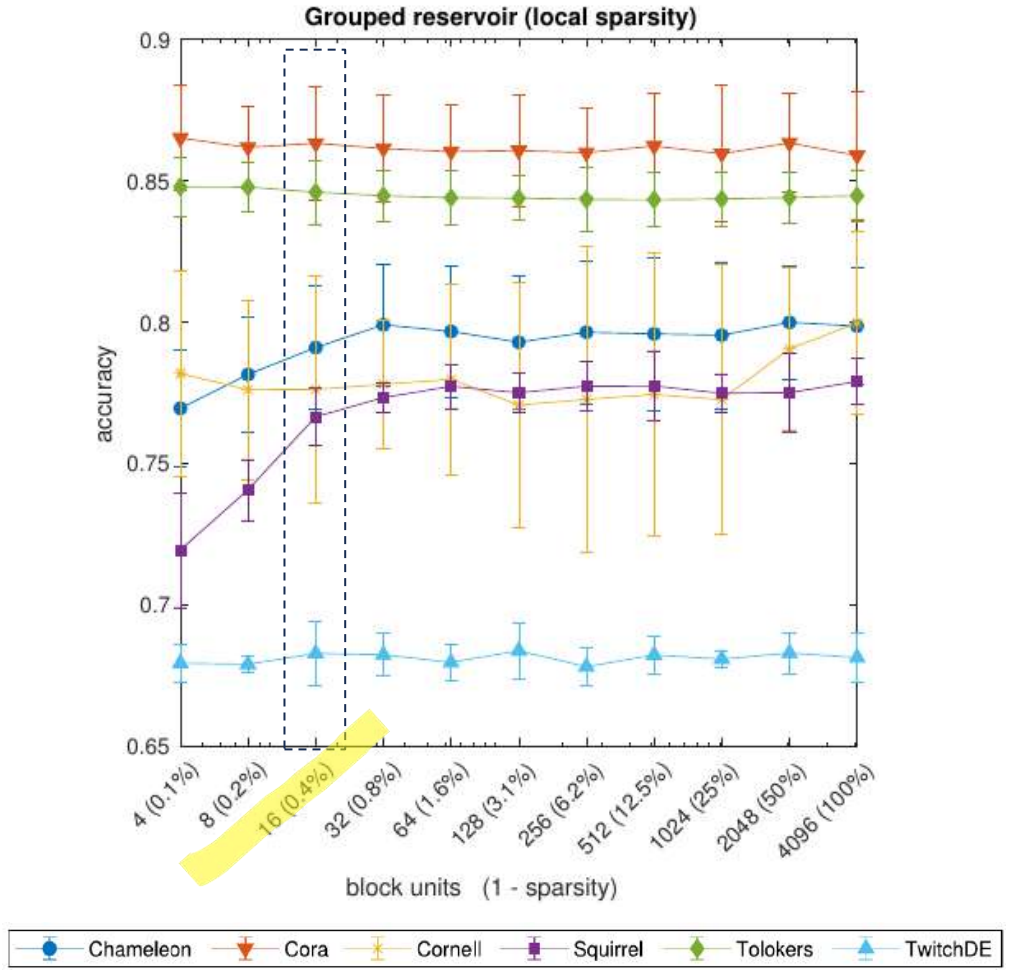


Micheli, Tortorella. *Addressing hererophily with graph echo state networks*. Neurocomputing, 2023

Sparse Design of GESN: Even more efficient



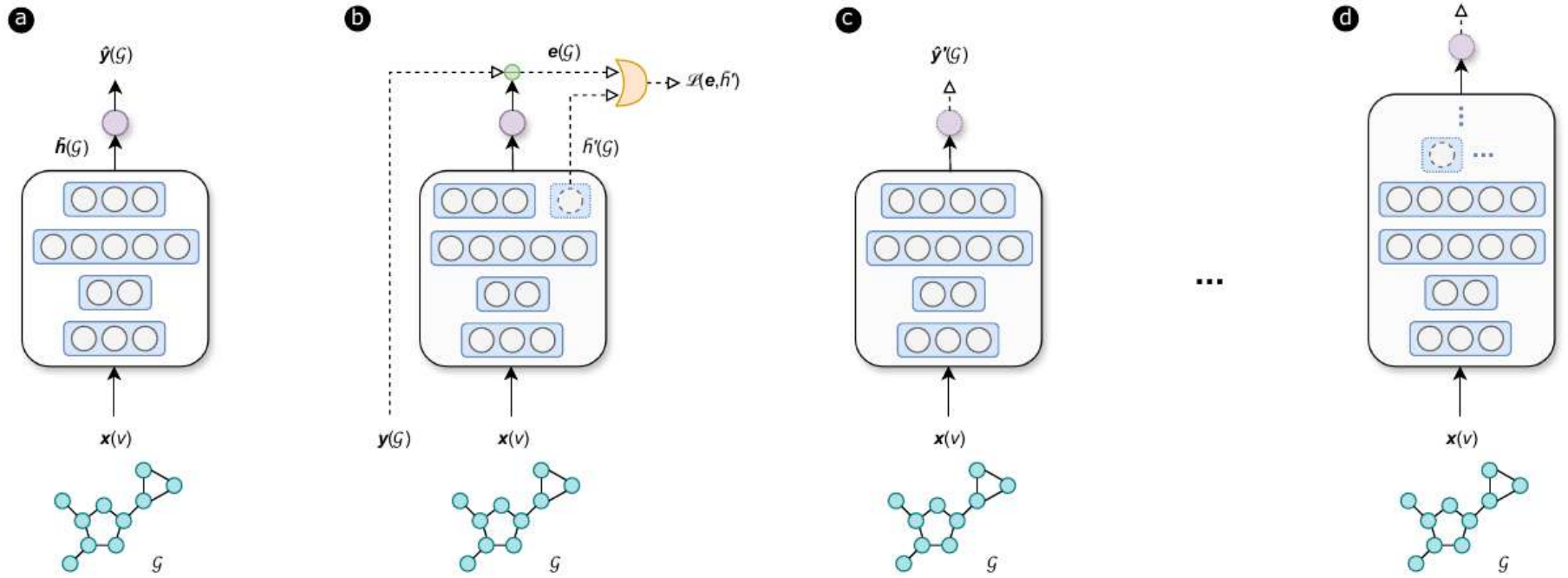
Block-diagonal structure



Micheli, Tortorella. *Designs of Graph Echo State Networks for node classification*. Neurocomputing, 2024

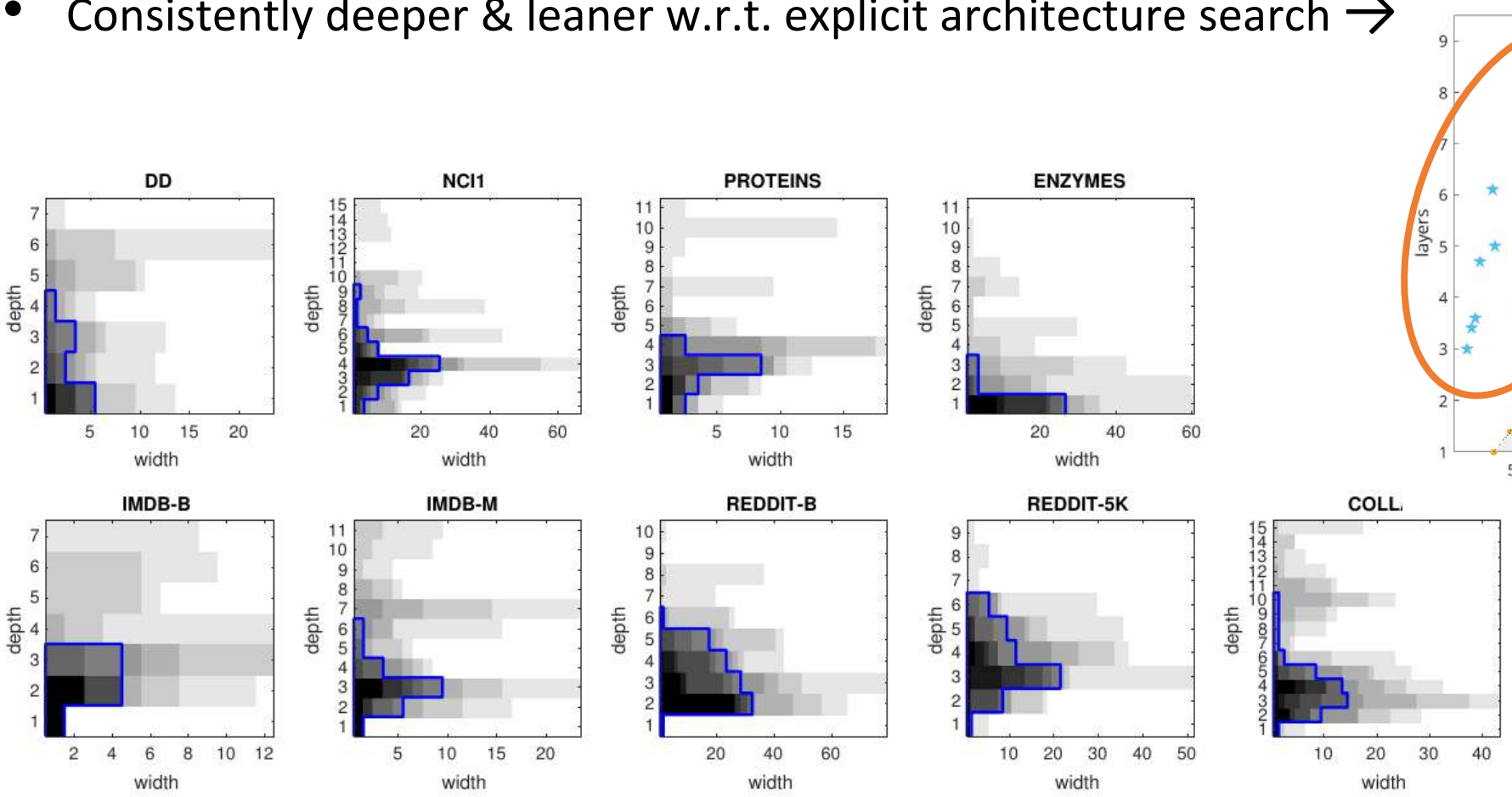
Automatic Architecture design by NN4G+ (New)

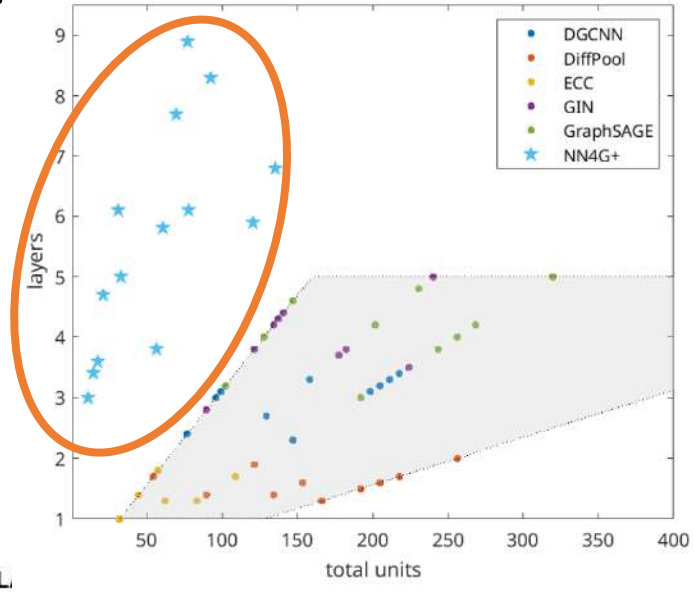
- NN4G+ is built incrementally, with *double synergic expan*sion (*New*!)
  - We *train 1 unit at a time*: **no E2E backprop** through layers
  - A layer is greedily expanded until validation accuracy no longer improves
  - New layers are added until validation accuracy no longer improves



NN4G+ Architectures Are Tuned to Tasks (New)

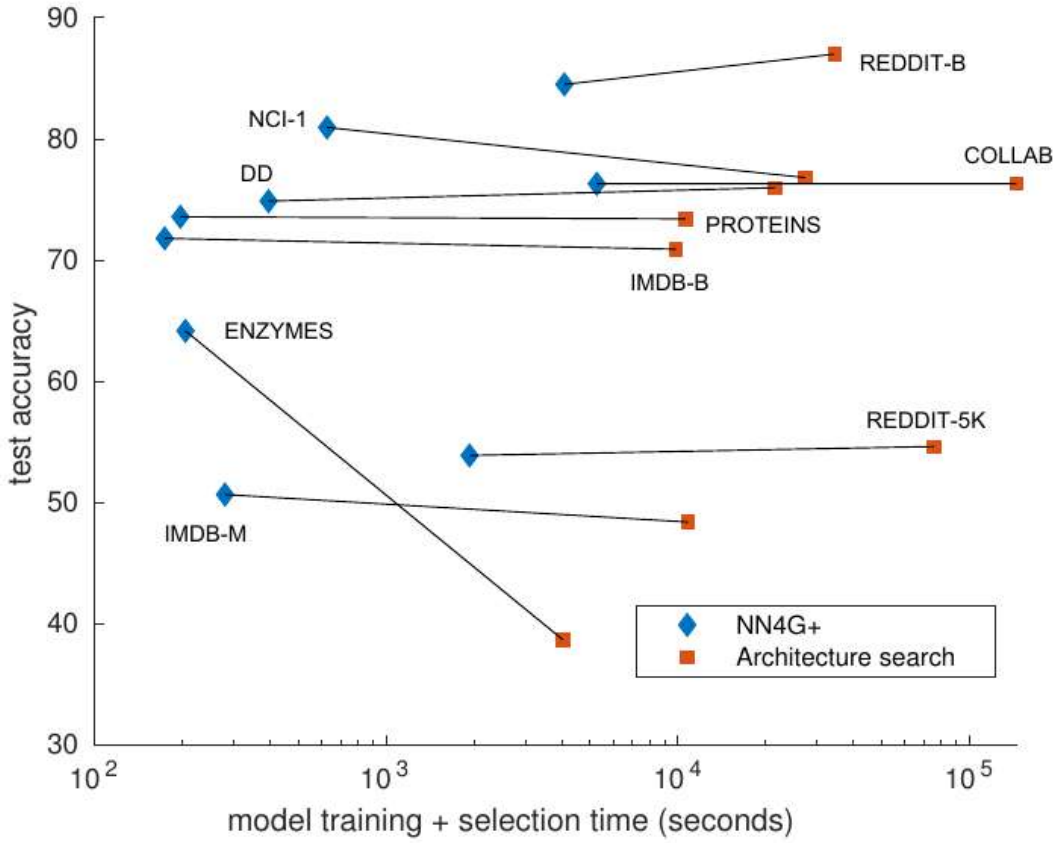
- Consistently deeper & leaner w.r.t. explicit architecture search →

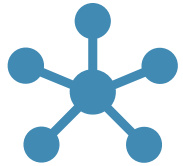




NN4G+ is *Efficient* and Effective (New)

- In general, NN4G+ constructs a neural architecture in *minutes* instead of *hours* w.r.t. E2E trained models
- In a fair comparison against *explicit architecture selection*, NN4G+ **speeds up >10×** without penalizing accuracy





EXPRESSIVENESS: OVER* ISSUES ANALYSIS

- The depth is necessary in DGN but …
- What are the impacts of the issues in Deep Graph Networks?

Expressiveness: Some Tools for the Analysis





- (1) Models that *decouple* the MP inductive bias of DGNs from E2E training issues (*going beyond backprop for DGN* 🌿 *)
  - **NN4G**: incremental, *avoiding gradient propagation through layers*
  - **GESN**: *no training* for the embedding, direct control of (fixed) spectral norm $\|\boldsymbol{W}\|$, imposing $\|\boldsymbol{W}\| > 1$ to prevent the *sensitivity* w.r.t. the input to be exponentially vanishing (*over-squashing* upper-bound)
- (2) Looking inside the models' layers
  - Spectral analysis (graph signals)
  - Accuracy (proxy) ←

GCN $$\left\| \frac{\partial \boldsymbol{h}_v^{(L)}}{\partial \boldsymbol{x}_u} \right\| \leq \left(\prod_{l=1}^{L} \|\boldsymbol{W}^{(l)}\| \right) \cdot \left(\mathbf{A}^{\mathrm{L}}\right)_{u,v}$$

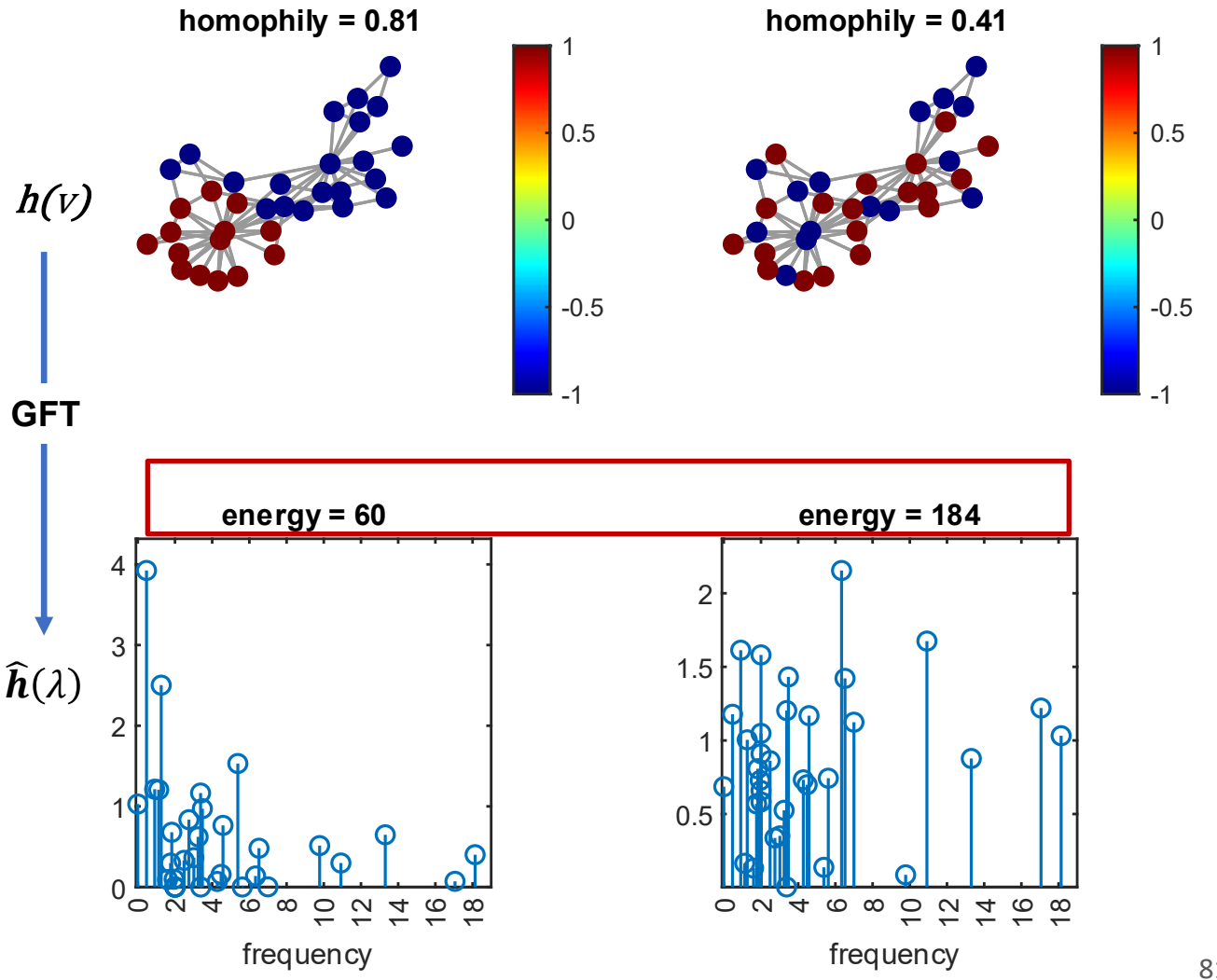
GESN $$\left\| \frac{\partial \boldsymbol{h}_v^{(L)}}{\partial \boldsymbol{x}_u} \right\| \leq \underbrace{\sum_{l=0}^{L} \|\boldsymbol{W}\|^l \, \|\boldsymbol{W}_{in}\|}_{Lipschitz} \; \underbrace{(\boldsymbol{A}^L)_{u,v}}_{topology}$$

Spectral Analysis & Etherophilic Graphs (Tech)

- (High) spatial variability of the graph signal, (high) frequency for graphs, are related to low homophily
  - i.e. larger spectra by *Graph Fourier Transform* (GFT) for etherophilic graphs (on the right)
- **Etherophilic tasks require to preserve high frequencies**
- The Dirichelet energy $E(\boldsymbol{h})$ of model layers $\boldsymbol{h}$ can **measure** this
- Spatial view (embedding variability):

$$E(\boldsymbol{h}) = \sum_{v \in \mathcal{V}} \sum_{u \in \mathcal{N}_v} ||\boldsymbol{h}_v - \boldsymbol{h}_u||^2$$

- **Spectral view**: *concise measure* of the *distribution* on the *graph frequency spectrum*
  E.g. high frequency signal → high energy

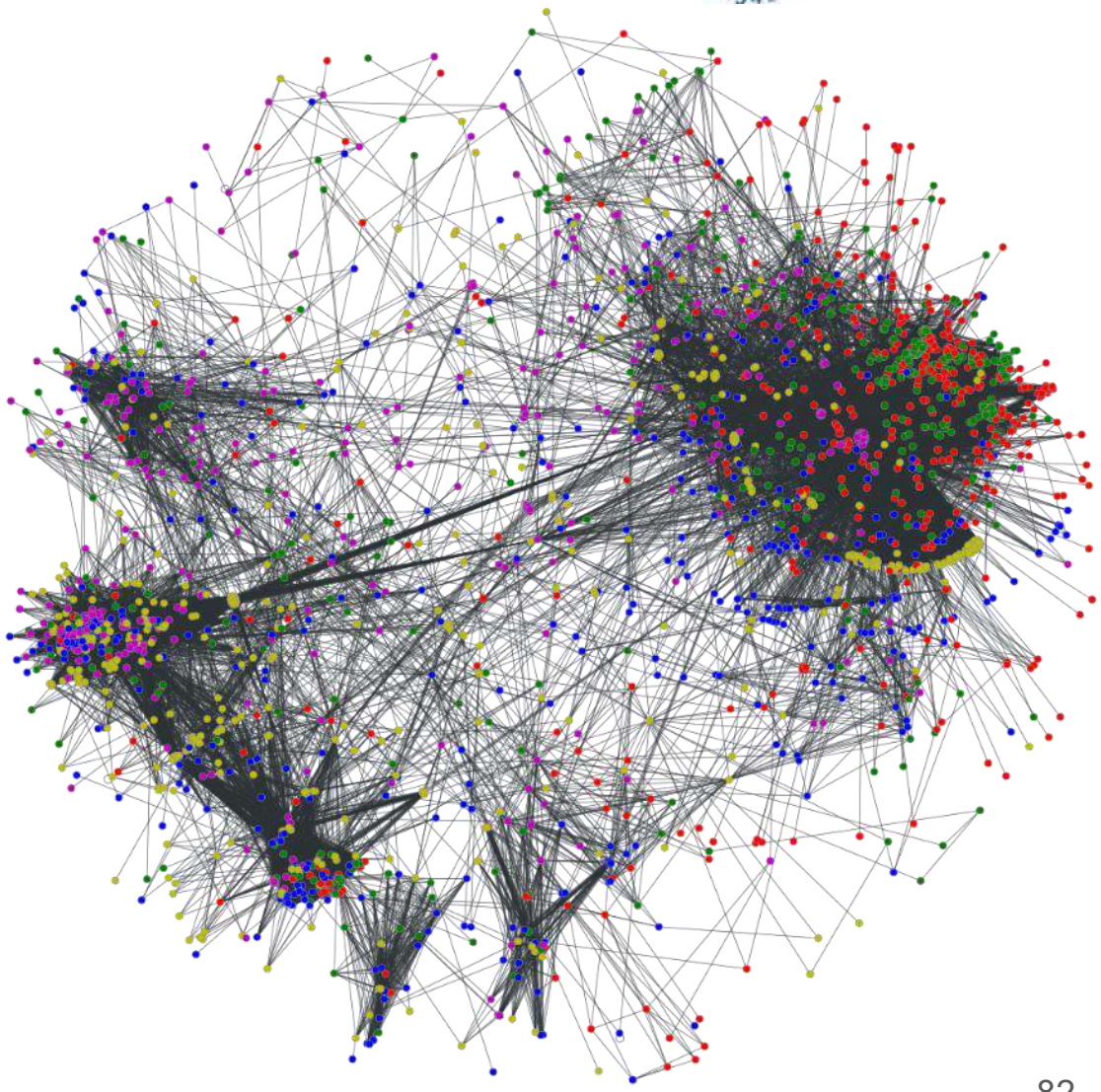


The Task: Chameleon (a «stress test»)

- Chameleon [Rozemberczki et al, 2019; Pei et al., 2020]
- Page-page network on a specific topics in Wikipedia
  - nodes represent web pages (bag of words)
  - edges are mutual links between pages
- **5** categories in term of the average monthly traffic of the web page (node classification)
- Nodes: 2277, Edges: 31400
- Degree min/median/mean/max = 1/12/27.6/732
- Diameter: **11** (length of the "longest shortest path“)
- Homophily, intra-class edges ratio [0-1]: **0.23**

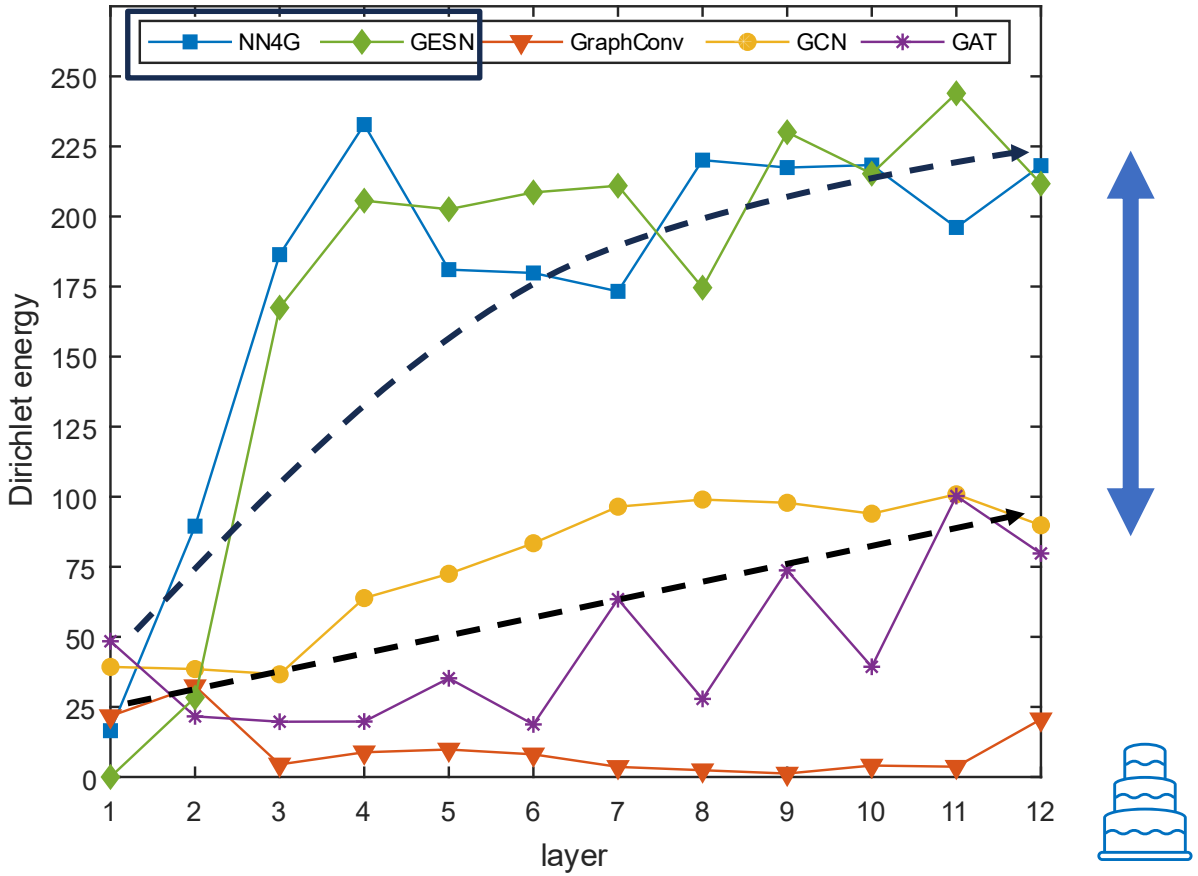
$$\frac{intra\text{-}class\ edges}{total\ edges}$$

Heterophilic



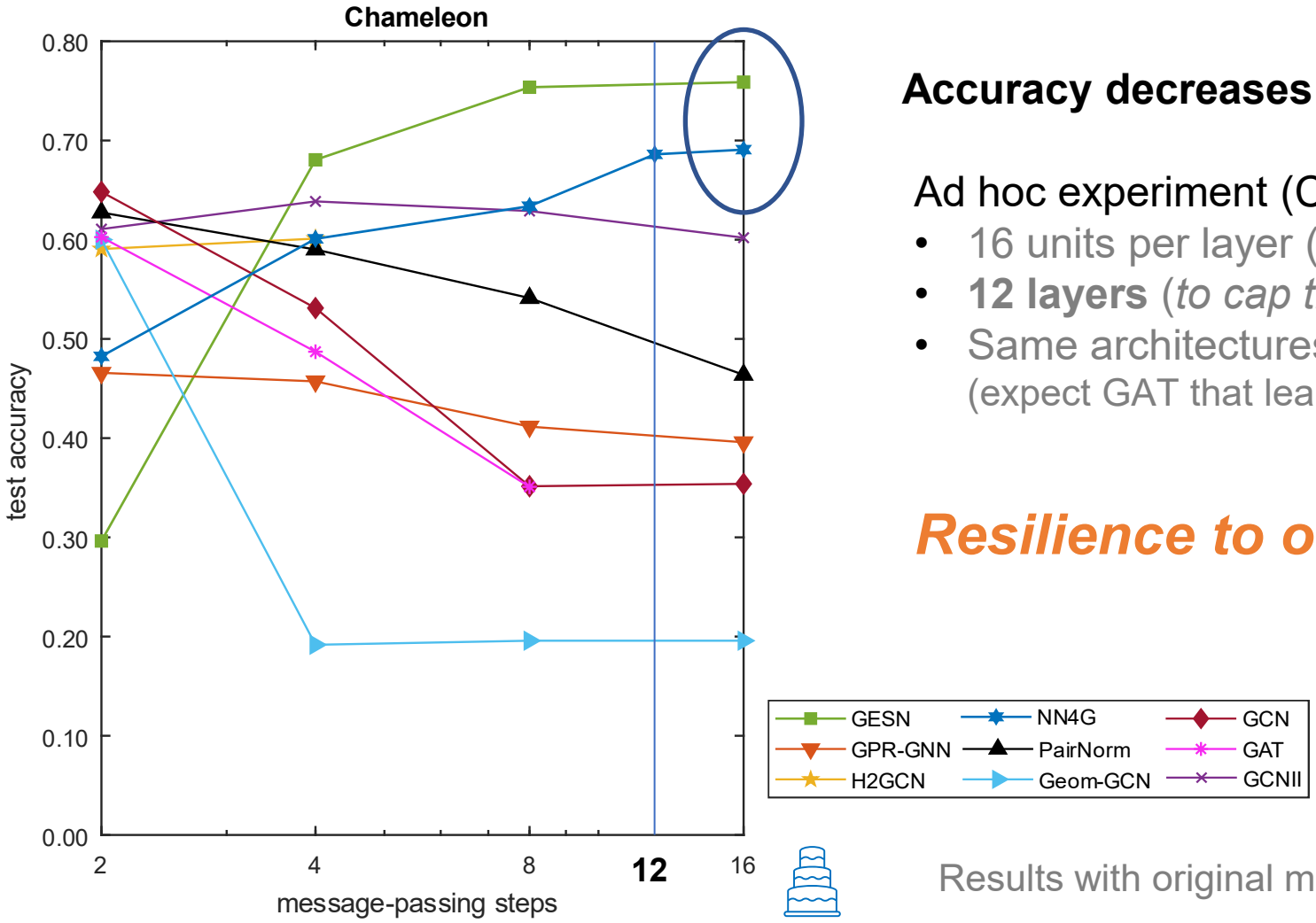
Spectral Analysis (Tech)





NN4G and GESN better preserve energy through layers (MP)

Impact of Depth on Accuracy



Accuracy decreases with MP, except for NN4G and GESN

Ad hoc experiment (Chameleon):

- 16 units per layer (expect GESN)
- **12 layers** (*to cap the graph diameter*)
- Same architectures and units configurations
  (expect GAT that learn edge weights via attention mech.)

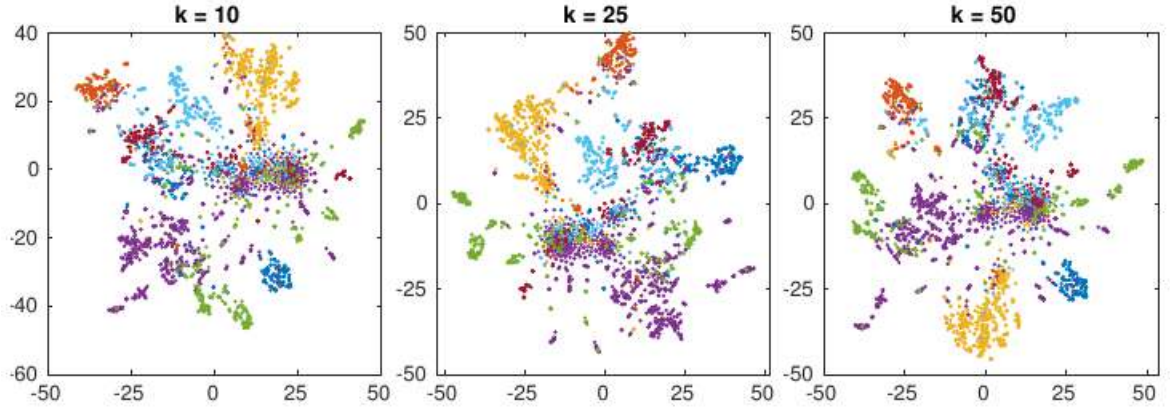
Resilience to over-*

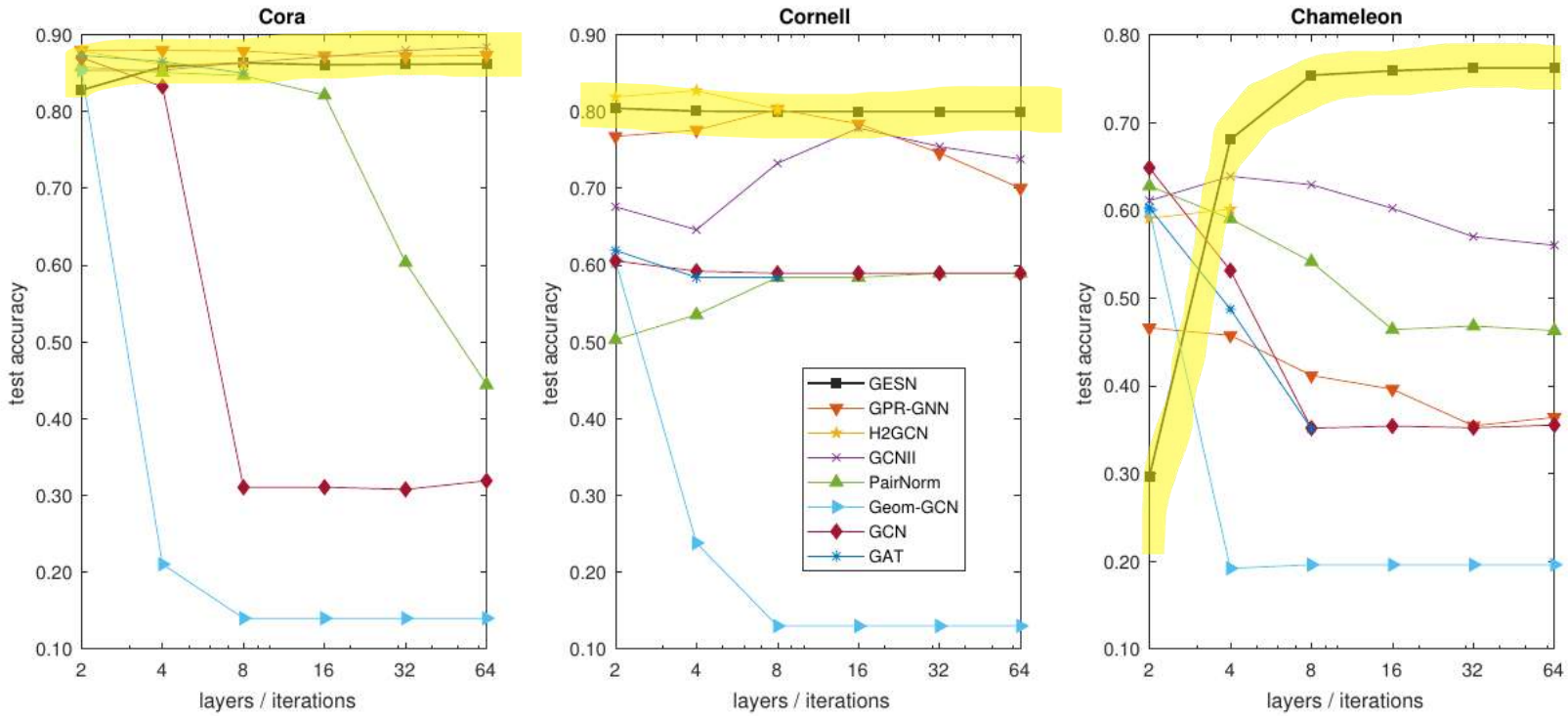
GESN	**73.7**
NN4G	68.3
GCN	57.8
GAT	55.0
GraphConv	33.8

Results with original models [Yan, Y., et al. ICDM 2022]

Resilience to over-smoothing

Node representations in **GESN** do not collapse as the number of MP steps increases





2D projection of node embeddings for the *Cora* graph at different MP steps.
Colors = classes, qualitatively showing how well separable are the node representations.

Micheli, Tortorella. *Addressing hererophily with graph echo state networks*. Neurocomputing (2023)

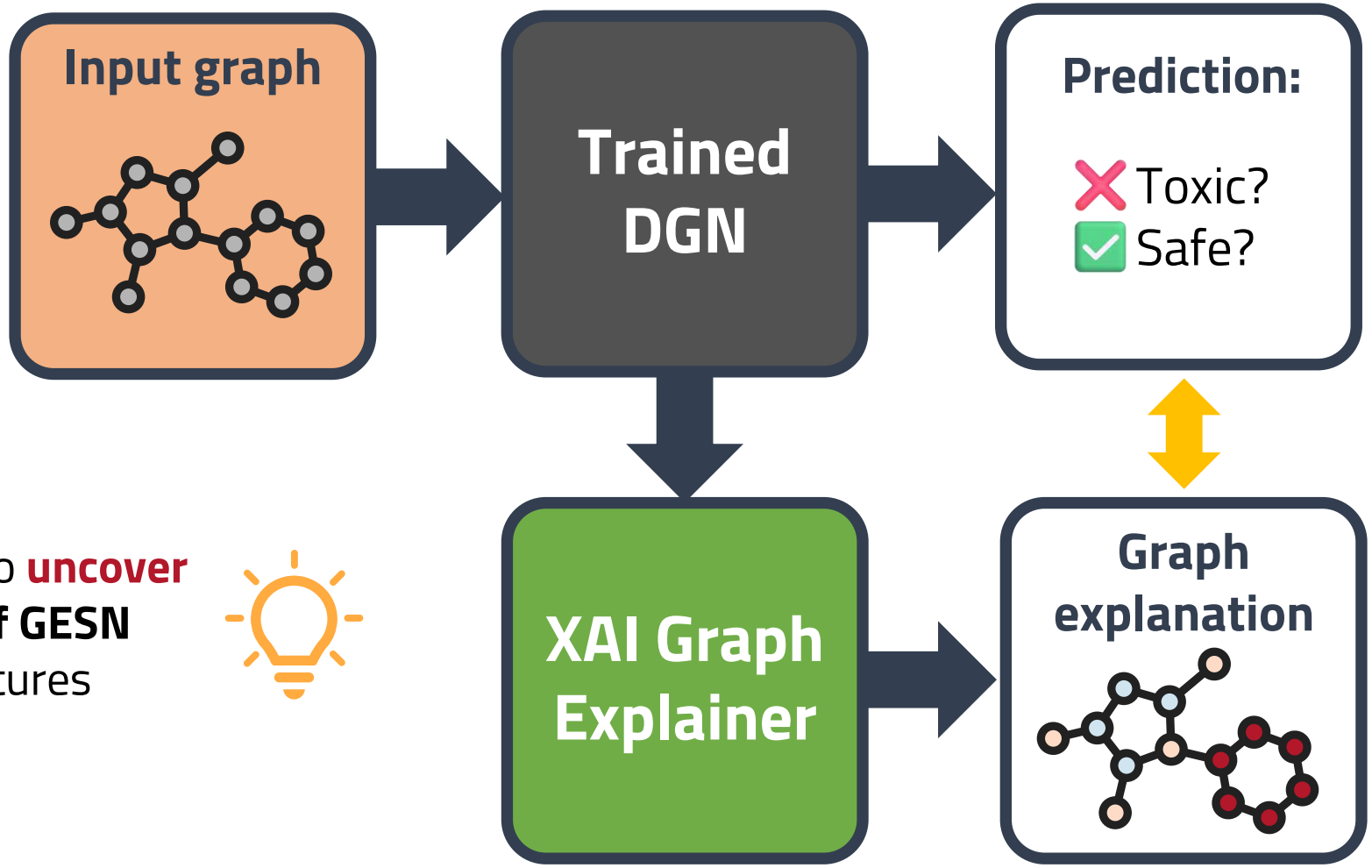


CHARACTERIZATION OF DGN MODELS

- Further characterization of the models inductive bias
- By the means of **Graph XAI**

Graph XAI as an Investigation Tool

A **graph explainer** derives an importance score for each node of the graph and hence **relevant subgraphs**



We exploited these approaches to **uncover the properties (inductive bias) of GESN** and other different DGN architectures

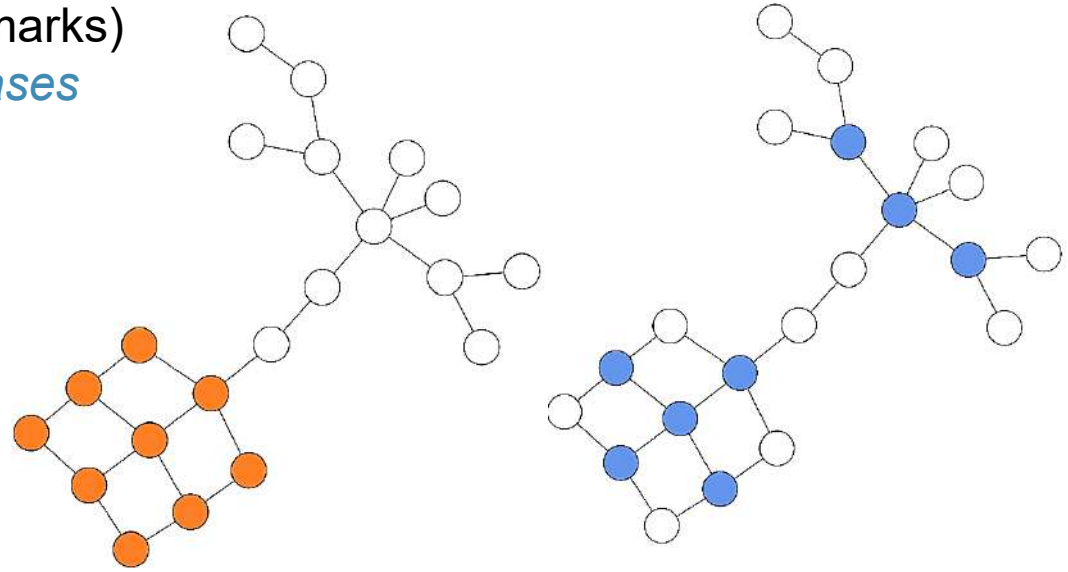
Explainability 4 Graphs: Our Investigations

With the same level of accuracy (100% for these benchmarks)
GCN and GESN (RC based) show different inductive biases

CAM Explainer

Dataset	Motif	
	GESN	GIN
BA2grid	0.806	**0.956**
BA2Motif	0.769	**0.911**
GridHouse	0.696	**0.903**

Dataset	Node degree	
	GESN	GIN
BA2grid	**1**	0.822
BA2Motif	**1**	0.812
GridHouse	**1**	0.847



Motif Ground Truth

Node Degree Ground Truth

Plausibility of different DGNs in detecting (different) Ground Truth (GT)

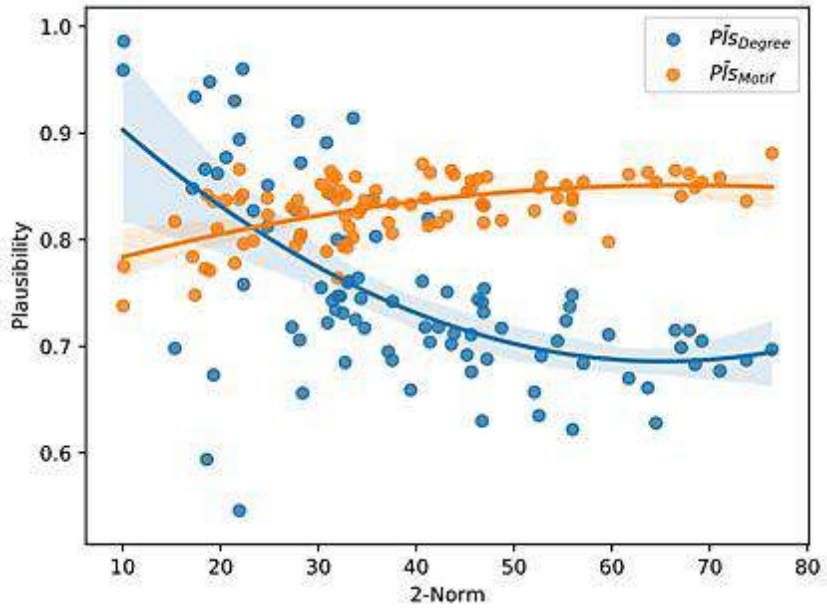
Plausibility measures the alignment (by the AUROC score) between explanation mask and GT

- Some architectures (GCN) prefer detecting **motifs** (left),
- some (GESN) prefer detecting **local degree patterns** (≥3) (right)

Explainability 4 Graphs: Our Investigations (II)

- We studied how to correlate to the inductive biases of the models, *also in terms number of layers (**depth**) and weights values*

  - As **more layers** are added to a DGN, the detection strategies transit from *degree-first* (low-order structures at the lowest layers) to *motif-first*
  - But also the **weights** (training) influences with a shift of explanatory pattern



Plausibility of 2-layered GIN varies as the **weights** 2-norm increases (BA2Motif dataset)

- Node degree pattern
- Motif pattern

→ Richness of capabilities/inductive biases in landscape of DGNs

Explainability 4 Graphs: Our Investigations (III)

The detection policies and *model inductive biases* can be measured and correlated to concepts of **graph theory** (Katz centrality, Fiedler eigenvectors)

Work in progress

- *Real world tasks (e.g. toxicology)*
- *More in-depth analysis of model details*
- *More complex patterns*
- *New benchmarks development ("OpenGraphXAI")*

Fontanesi, Micheli, Podda, Tortorella. *Bridging XAI and spectral analysis to investigate the inductive biases of deep graph networks.* Machine Learning, accepted.

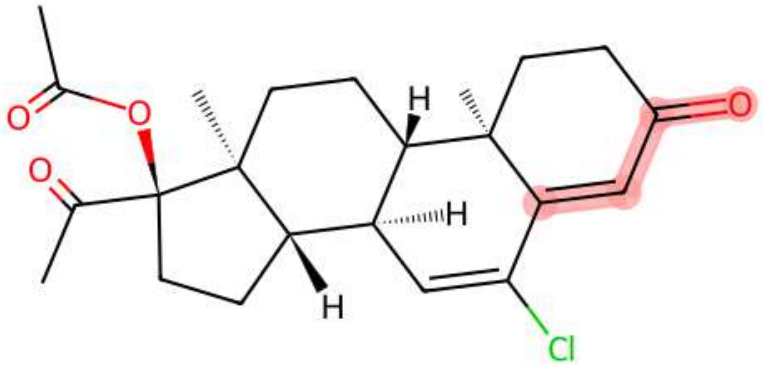
Appplications: example on Tox21 P53

Michael acceptor



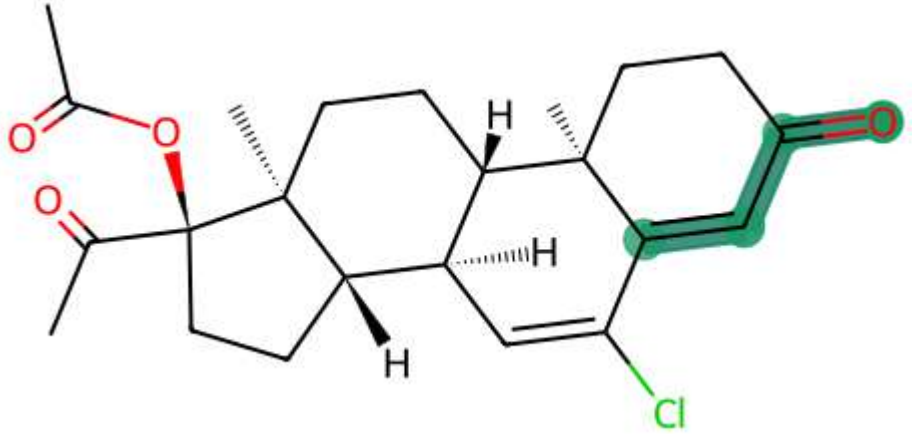
XAI: Identify the (known) fragment(s) (structural patterns) that influences the activation

Structural Pattern: **Michael acceptor**
Smart: "[C]=[C]-[C](=[O])"



Name: chlormadinone acetate
Smile:[H][C@@]12CC[C@](OC(C)=O)(C(C)=O)[C@@]1(C)CC[C@@]3([H])[C@@]2([H])C=C(Cl)C4=CC(=O)CC[C@]34C

XAI Result



Top4 Nodes

Graph XAI: Emerging Messages







- Relevance to **know your models** (which bias you want to exploit during learning)
- **The truth is not unique!** There exist **multiple valid** *solutions* (*same classification accuracy*) from different models but having *different explanatory patterns*
- This is particularly interesting **for graphs**, where there is **not** the same “natural” visual interpretation that we can have on images

- The **GT is rarely available** in real-world contexts **for graphs**: It is extremely necessary to introduce accurate annotated benchmarks for **graph XAI** from real data
- We proposed **OpenGraph XAI** with more than 15 novel datasets derived from molecular datasets and a software framework able to generate **2000+ XAI benchmarks** with controlled GT

Fontanesi, M., Micheli, A., Podda, M., Tortorella, D.: Addressing the scarcity of benchmarks for graph xai (2025), https://arxiv.org/abs/2505.

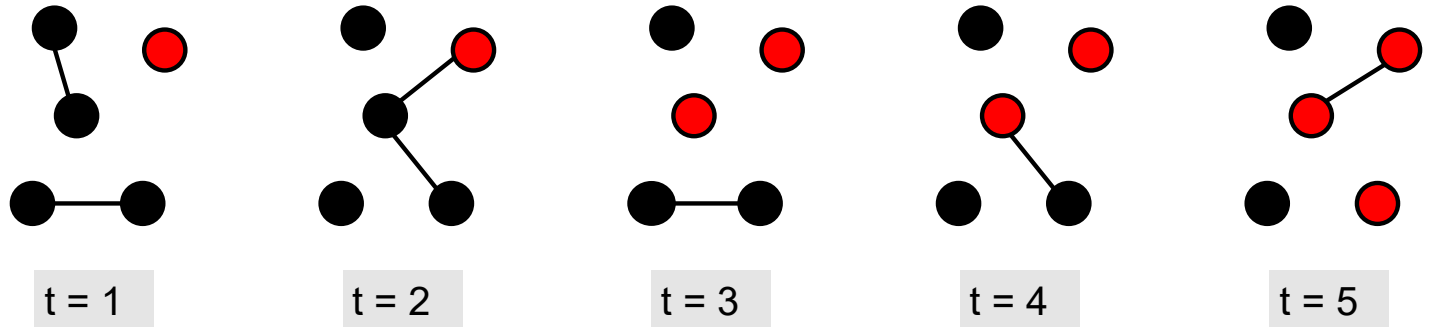




EXTENSIONS

Scope: Dynamical/temporal graphs

- Dynamic/Temporal Graphs represent relationships or node values that evolve throught time
  - E.g. Infection-spreading models, content dissemination on social networks, signals on nodes...



A. Micheli, D. Tortorella. *Discrete-time Dynamic Graph Echo State Networks*. Neurocomputing (2022)

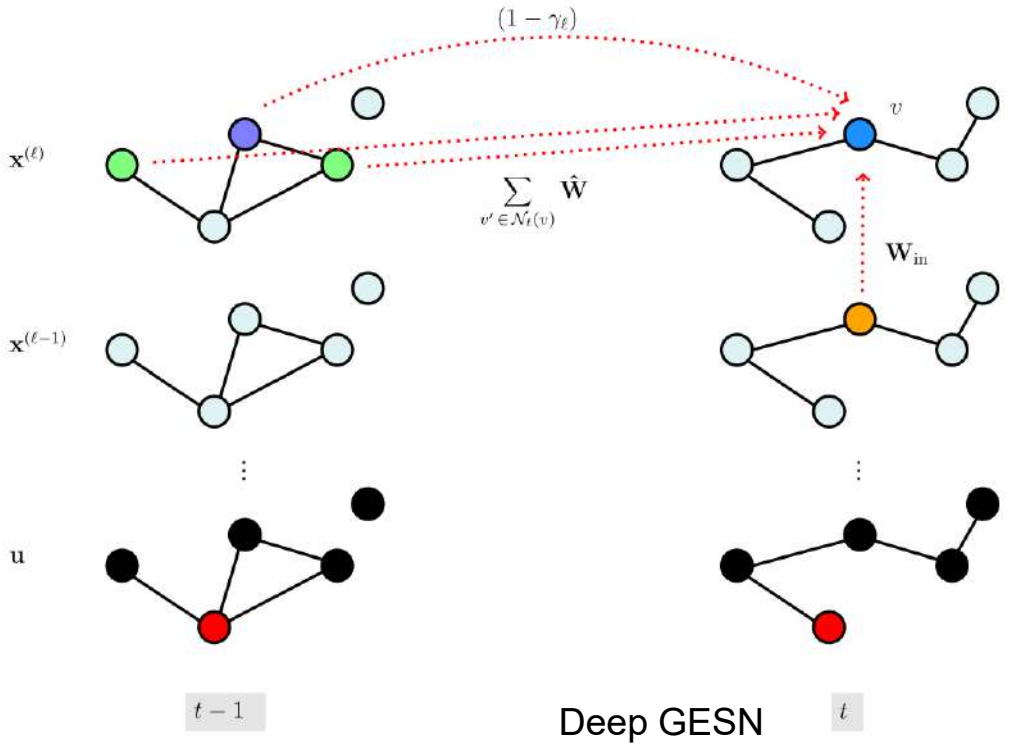
Dynamic Graph Echo State networks: Results

- Multi-layer reservoir defined by the node-wise state transition function, implementing a spatio-temporal graph convolution:

$$\mathbf{x}_t^{(\ell)}(v) = \gamma_\ell \tanh\left(\mathbf{W}_{\text{in}}^{(\ell)}\mathbf{x}_t^{(\ell-1)}(v) + \sum_{v' \in \mathcal{N}_t(v)} \widehat{\mathbf{W}}^{(\ell)}\mathbf{x}_{t-1}^{(\ell)}(v')\right) + (1-\gamma_\ell)\,\mathbf{x}_{t-1}^{(\ell)}(v)$$

- **Stability condition**: $\left\|\widehat{\boldsymbol{W}}^{(\ell)}\right\| < \frac{1}{\lceil \|\boldsymbol{A}_t\| \rfloor}$ for all layers (time-dependent, geometric mean of the graph series)

- More or as effective as (10) TGNs (fully-trained Temporal Graph neural Nets) but …
  - **100x** training speed-up w.r.t. TGNs
  - **10x** inference speed-up w.r.t. TGNs



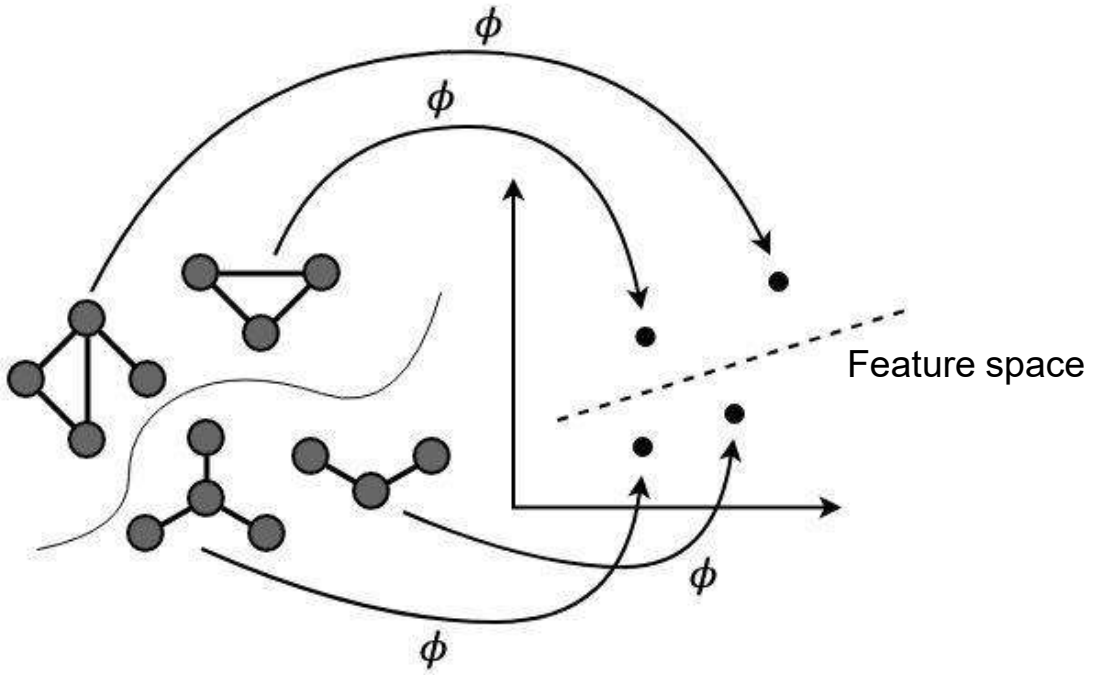
Deep GESN

Micheli, Tortorella. *Discrete-time Dynamic Graph Echo State Networks*. Neurocomputing (2022).

Other approaches to Learn in SD: Example

And Kernels?

- Large set of proposals (out of the scope)



Kernel methods for SD



Kernel modularity/ definition: easily can be extended to graphs of different types: sequences, trees, **graphs** (directed, undirected, cyclic/acyclic, …)

Implicit embedding of data into Euclidean space by specifying an inner product

$$k(x, x') = \langle \phi(x), \phi(x') \rangle$$

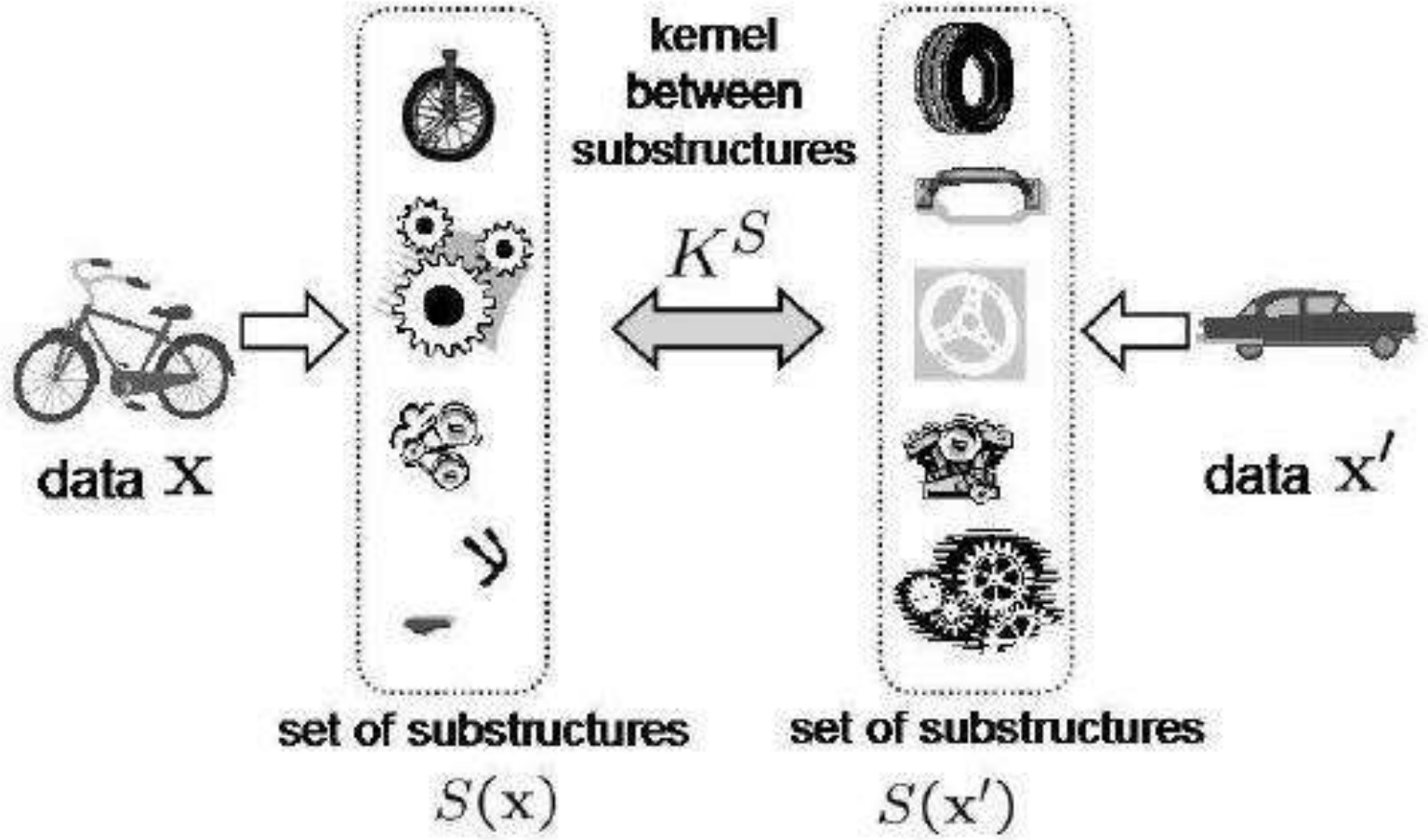
Data $\boldsymbol{x}$ can be of various types, including structured data

Define similarity between data

Example: Marginalized Kernel

- Similarity between 2 graphs based on the *common label paths* obtained by *random walks* in the graphs
- Kernel function defined as the inner product of the count vectors, averaged over all possible label paths
- Computational complexity of the kernel function (typically) *scales quadratically with the graph size*

Convolution Kernels



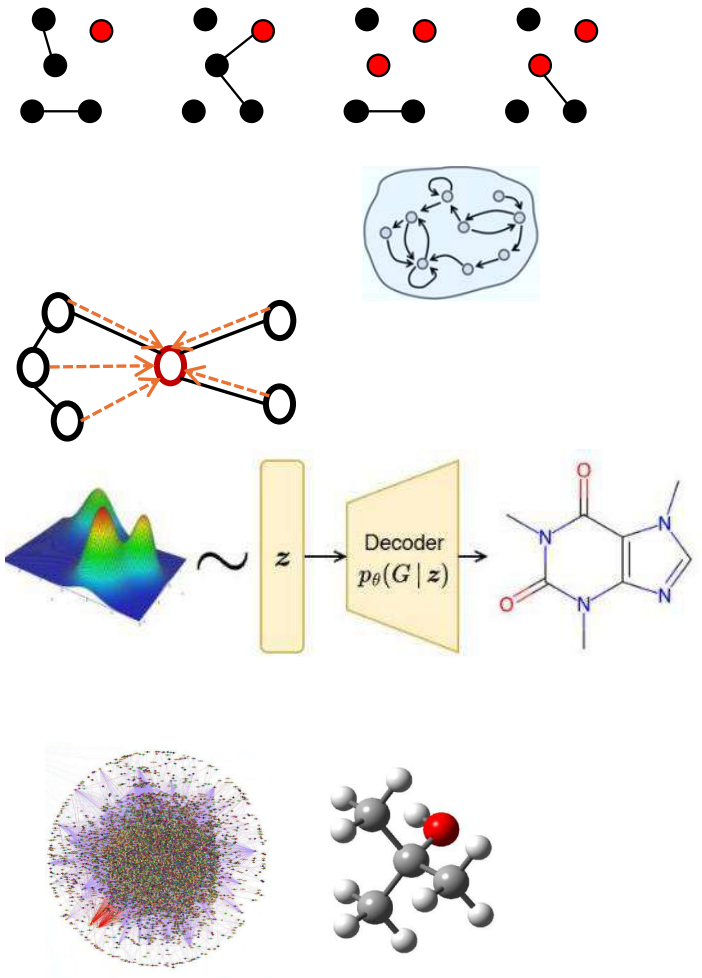
Kernels for SD: discussion

- Efficiency: the cost is critical (due to sub-structure decomposition)
- Adaptivity warning: Kernel for structure: *knowledge-driven methods*
- Similarity measure fixed prior to learning
  - i.e. no adaptivity/learning in the encoding into the feature space
    - Pro: we can exploit prior knowledge by the Kernel (to engineering a specific problem)
    - Cons: For SD ***no universality is known***:
      - The choice of the kernel function is critical
      - E.g. Computing any complete graph kernel *is at least as hard as* `**Graph Isomorphism**'
    - Adaptive kernels studies…

Perspectives & Conclusions

Perspectives (Examples)

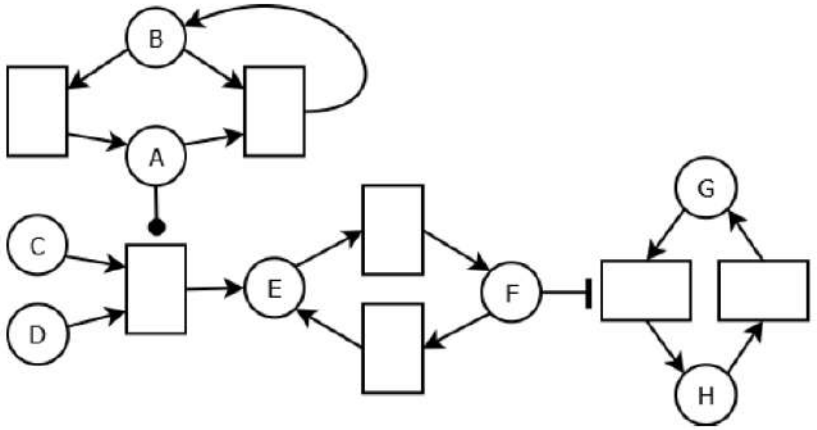
- **Scope and extensions**: Dynamics graphs, heterogenous graphs, hypergraphs, etc.
- Evolution **RC** based models
- **Graph Transformers** (attention mechanism considering all the nodes)
  - similarities with MP, many variants, not always better performances, efficiency issue ($O(n^2)$)
- **Graphs generation**
- **Foundation models**
- **XAI** can help to **uncover** properties of DGN and to properly use them, but we are just at the beginning
- **Trustworthy** concepts
- **Applications**, e.g. Biological networks, PPI; Cheminformatics, …



Applications (Example 1)

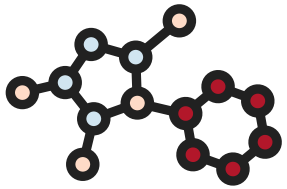
Biochemical Pathways (**BPs**) are sets of chemical reactions among biomolecules
They model **cell functionality**

Structure described by **graphs** (e.g. **Petri Nets)**
Goal: predict dynamical proprieties associated to the pathways (robustness, sensitivity, etc.)



Fontanesi, M., Micheli, A., Milazzo, P., & Podda, M. (2023). Exploiting the structure of biochemical pathways to investigate dynamical properties with neural networks for graphs. *Bioinformatics,* 39(11)

Applications (Example 2)

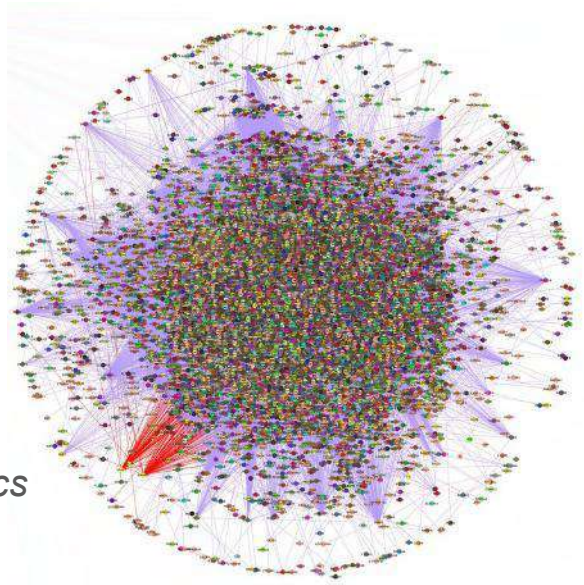



- Chemical, social networks benchmarks
- **Biology**:
  - Protein-Protein Interaction Network (**PPI**)
    - *Nodes* represent proteins
    - *Arcs* represent physical associations between two proteins
  - **Predicting sensitivity** (perturbation analysis) on PPI
  - Starting from biochemical pathways (Biomodels) and mapping to PPI subgraphs (BioGRID, **60K nodes**)



  - **Results**:
    - **Efficiency** vs numerical simulation on biochemical pathways (ODE)
    - **High performance** even without protein embeddings, meaning that DGN is relying on PPI topology
  - **Potential usage**:
    - Disease understanding
    - Drug design and repurposing
    - Precision/Personalized medicine

PAN-HUB prj on antibiotics resistance



Dipalma, et al. *BMC Bioinformatics* (***2025***)

Conclusions



- It is possible to do deep learning for graphs efficiently

  (without fully E2E training)

  - Models as GESN and NN4G allow to efficiently build DGN with high accuracy
  - At the same time enabling the investigation of the MP biases



- ML for graphs is a concrete reality

  - It can provide new solutions for complex domains where the **managing of relationships** is relevant to achieve suitable modeling of the solutions
    - If you have data with relationships (serial, hierarchical, networks), any domain where you find structured data, do *not* use a flat representation … we can work together on them!
  - While still **under development**







General conclusions

Many open issues & extensions



Many applicative fields

We are open to collaborations!

Your Thesis and more



"tie a knot in your handkerchief"

CIML group @ C.S. Dep. - University of Pisa





- **Faculty** (9): Prof. A. Micheli (coordinator), Prof. D. Bacciu, Prof. C. Gallicchio, Dr. A. Carta, Dr. A. Ceni, Dr. A. Cossu, Dr. L. Passaro, Dr. M. Podda, Dr. D. Tortorella
- 5+ post-doc researchers
- **12+ PhD students (the young energy!)**

- More than **25 years** of research in ML/AI

- Part of the cairne.eu Research Network
  (Confederation of Laboratories for Artificial Intelligence Research in Europe)

CAIRNE

Site: *ciml.di.unipi.it*



Computational Intelligence & Machine Learning Group

Special Thanks







- **Claudio Gallicchio**, Assoc. Prof.
- **Marco Podda**, Researcher (FAIR)
- **Luca Pedrelli**, Post-Doc (THE)
- **Domenico Tortorella**, Researcher (PAN-HUB)
- **Michele Fontanesi**, PhD student (FAIR)
- **Alessandro Dipalma**, PhD student













Projects

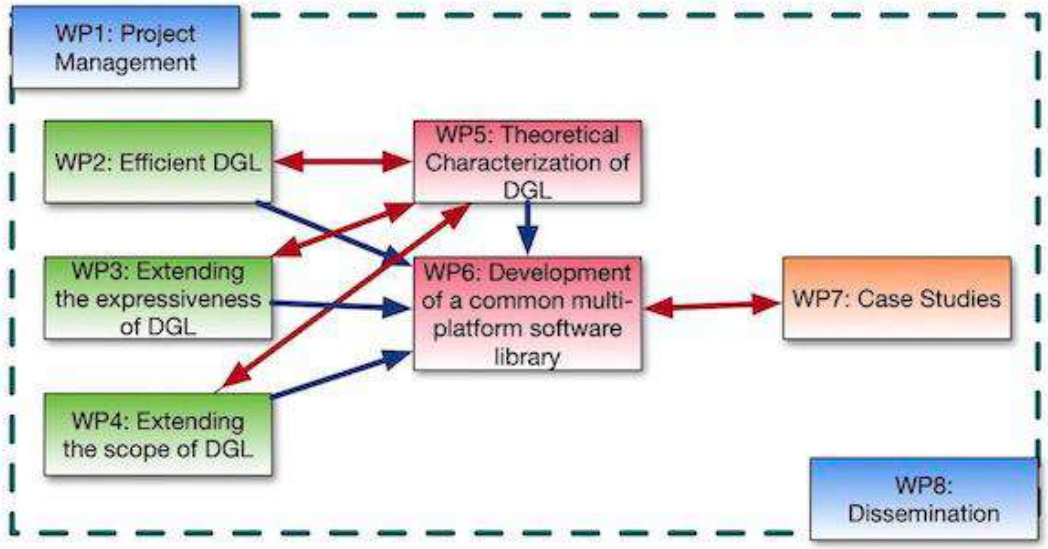
- Research for extension of **DGN** to **continual learning**, **automatic model design** within the **trustworthy** concepts (*explainability*, robusteness, fairness, privacy) is open….
  (graphs are discrete structures!)
- PNRR PE00000013 "FAIR - Future Artificial Intelligence Research" - Spoke 1 "**Human-centered AI**", funded by the European Commission under the *NextGeneration EU programme*. 2023, 3 years
- WP on *Human centered Lifelong Learning for Complex Data*





Projects

- PRIN (national project), 2Y, starting Feb. 2025
- **DEEP-GRAPH: Design and Theory of Deep Graph Learning**
  - University of Padova (Prof. A. Sperduti)
  - University of Pisa (Prof. A. Micheli)
  - University of Siena (Prof. F. Scarselli)



Ministero dell'Università e della Ricerca

PRIN
Progetti di Ricerca di Rilevante Interesse Nazionale

CIML group @ UNIPI

Contact:
Alessio Micheli
micheli@di.unipi.it

Thank You



Dipartimento di Informatica
Università di Pisa - Italy

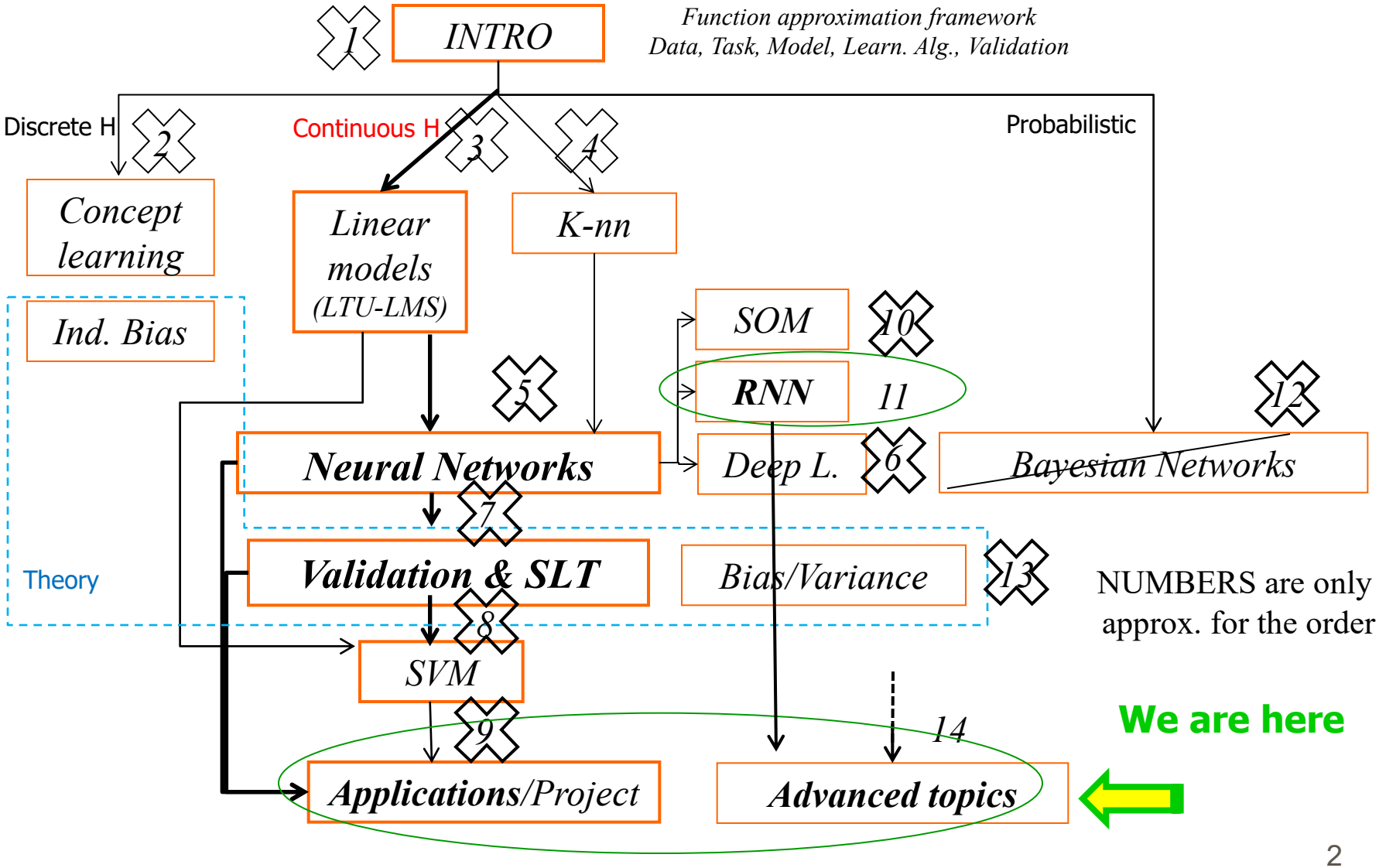


Computational Intelligence & Machine Learning Group

Site: ciml.di.unipi.it

ML Course structure
Where we are and go

Dip. Informatica
University of Pisa



Program (page 1)

Dip. Informatica
University of Pisa

- **Introduction:**
  - Critical contextualization in CS, learning as function approximation (example)
  - Computational learning tasks for predictions, design components, hypothesis space, model, learning algorithm, Loss and learning tasks.
- **Basic concepts and models**:
  - Concept Learning, dimension of discrete spaces, version space, Inductive Bias (language and search bias) and its role.
  - Complexity over an example (polynomials), Generalization and overfitting concepts, inductive principles, introduction to SLT and validation
  - Linear models: LTU, LMS algorithms (direct and iterative solutions): development and limitations. Linear basis expansion (LBE). Regularization
  - K-Nearest Neighbors: algorithm, characteristics, comparison, curse of dimensionality

Topics are NOT ordered according to the lecture number

Program (page 2)

Dip. Informatica
University of Pisa

- **Neural Network (NN) models:** architectures and learning algorithms
  - Biological and Artificial Neural Networks, Perceptron and LTU, Perceptron learning algorithms, Adaline, computational properties and comparison
  - Perceptron convergence theorem. Activation functions, MLP and NN architectures, NN properties: NN as LBE, universal approximation.
  - Derivation of the Back-propagation algorithm for learning in MLP
  - Hyper parameters and heuristics for the NN learning. Issues and techniques to improve the training (e.g.W init. and multiple minima, momentum, minibatch, learning rate setting and variants, …). Early stopping and Regularization.
    NN constructive approaches: Cascade Correlation.
  - Discussion on the characteristics of NNs.
  - Applicative examples (NN specific architecture design): Convolutional networks
  - Deep Learning: advantages and flattening analysis, autoencoders, representation learning and distributed learning concepts, other insights, issues and specific techniques including ReLu, Drop-out, pre-training, etc…
  - Randomized NN (with Cover Theorem)
  - NN architectures: Recurrent Neural Networks (intro)

Topics are NOT ordered according to the lecture number

Program (page 3)

Dip. Informatica
University of Pisa

- **Principles of learning processes and general practical aspects**:
  - Evaluation (Risk estimation) and Model Selection: difference and combined use, index (error measure) for classification and regression
  - Analytical or re-sampling approaches. Hold-out, K-fold Cross-validation schemas. Special cases, typical errors.
  - Statistical Learning Theory (SLT): complexity and shattering, VC-dimension (examples), basic SLT/VC-bound, Structural Risk Minimization
  - Bias-Variance decomposition. Outline of ensemble learning (bagging, boosting)
- **Support Vector Machines and Kernel-based models**:
  - Linear model and margin (hard margin), Vapnik theorem, soft-margin
  - Non-linear tasks: Kernels.
  - SVM for regression.
  - Critical analysis of characteristics and hyperparameters, discussion and comparisons.

Topics are NOT ordered according to the lecture number

Program (page 4)

Dip. Informatica
University of Pisa

- **Unsupervised learning:**
  - Usage, Vector Quantization, Self-Organizing Maps
- **Introduction to the Applications:** benchmark examples**,** applicative project
- **Introduction to advanced approaches:** Learning for Structured data (learning on graphs)
- **Introduction to the Applications:** examples (and research demos)

Topics are NOT ordered according to the lecture number

Program (page 5): Summary

Dip. Informatica
University of Pisa

Beyond the model details and techniques,
as transversal concepts should be emerge:

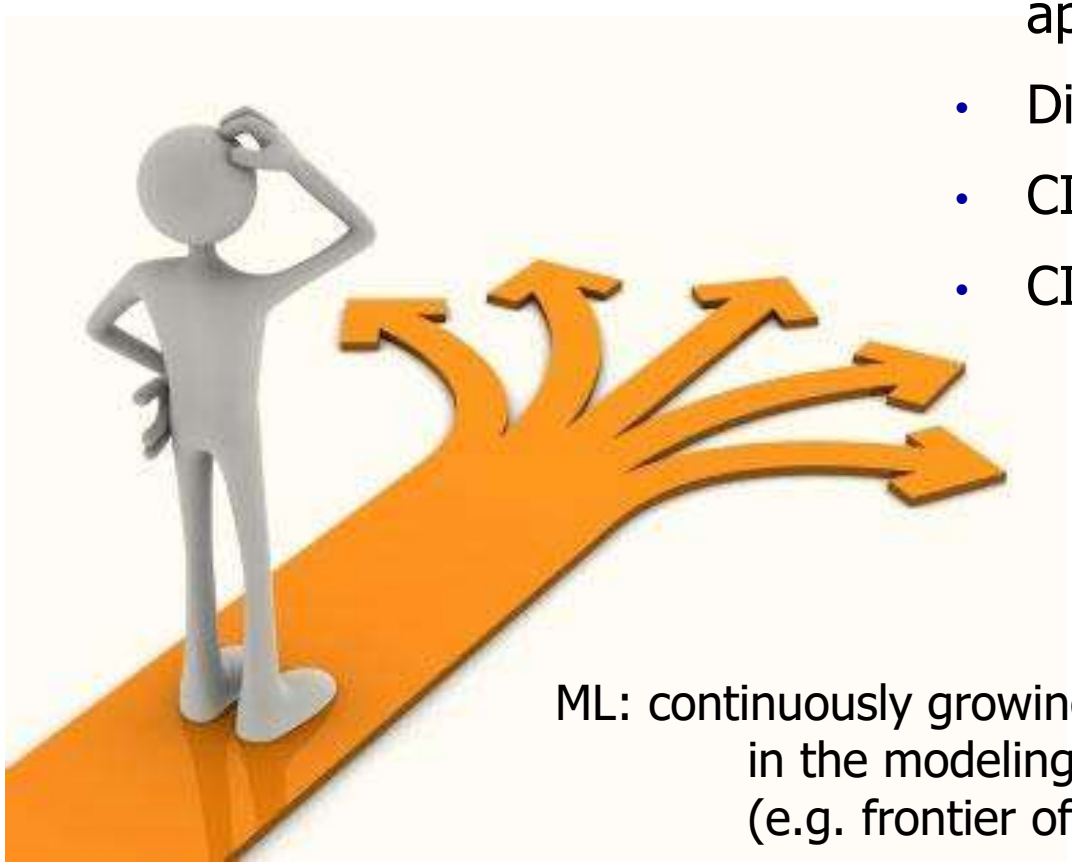
- The relevance and roles of *model complexity* and *regularization f*or *generalization* purposes
- The relevance and roles of a proper, extensive and rigorous *model selection (validation) and assessment*
- That **research in ML is open, and you are welcome to join !**



"tie a knot in your handkerchief"

Future – just ahead

Dip. Informatica
University of Pisa



- General topics for ML applications
- Didactics: CNS course
- CIML@Pisa
- CIML@Pisa research examples

ML: continuously growing area
in the modeling-theoretical and applied aspects
(e.g. frontier of scientific research)

A more general look to applications

General challenges

Dip. Informatica
University of Pisa

- Build autonomous Intelligent/learning machines:
  - Robotics, HRI, search engines, …
- Build powerful tools for emerging challenges in intelligent data analysis
  - Tools for the "data scientist"
- Open new areas of applications in CS: innovative interdisciplinary open problems
  - Fantasy is your limitation !

Challenges by Source of data

Dip. Informatica
University of Pisa

2000's

- Bio-Cheminformatics (*-omics, personalized medicine, toxicology)
- Web data

Recent "buzzwords"

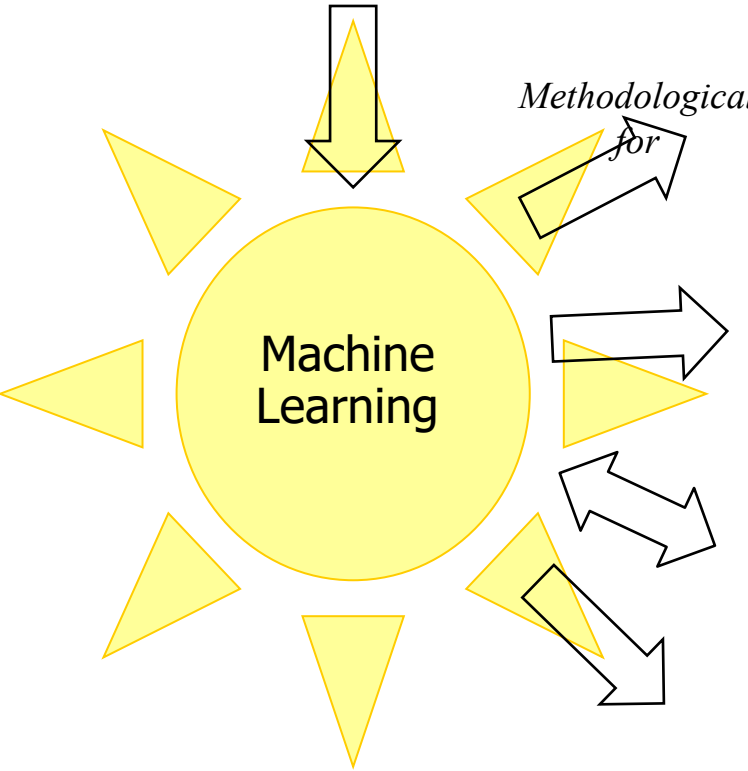
- Smart systems (design smart things and make human life better) : from *Smart Cities* to *Ambient Assisted Living* to the *Internet of Things* (One trillion =10^{12} devices with sensors of all kinds are expected to be connected to the Internet by 2022 (IEEE news March 2014))
- Social, Big data : e.g. social network data, or again IoT

Growing data sources opens up a huge application area for ML and related areas (also in terms of employment opportunities)

ML & other areas

Dip. Informatica
University of Pisa

Computational mathematics for learning and data analysis
Parallel and distributed systems: paradigms and models



Human language technologies (AI)
Generative and deep learning (AI)
Computer vision (AI)

Computational neuroscience (AI/ING)
Robotics (AI)
Semantic web (AI)

Algorithm engineering (KD)
Data mining (KD)
Information Retrieval (KD)
Mobile and cyber-physical systems (ICT)

See (link):
Artificial Intelligence

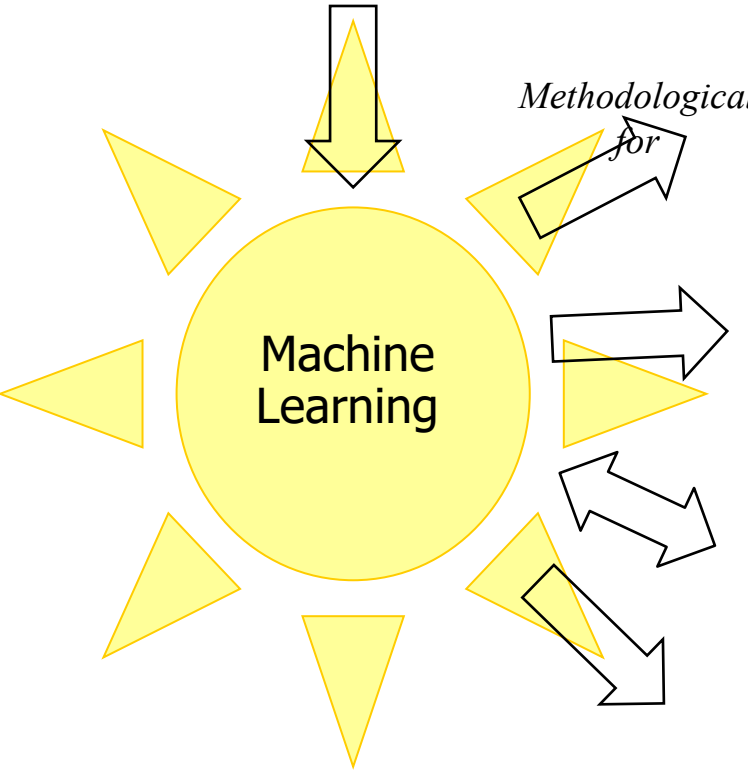
But also others:*

Bioinformatics (KD) Advanced databases (KD)
Big data analytics (WBI) …

ML & other areas

Dip. Informatica
University of Pisa

Computational mathematics for learning and data analysis
Parallel and distributed systems: paradigms and models



Human language technologies (AI)
Generative and deep learning (AI)
Computer vision (AI)

Computational neuroscience (AI/ING)
Robotics (AI)
Semantic web (AI)

Algorithm engineering (KD)
Data mining (KD)
Information Retrieval (KD)
Mobile and cyber-physical systems (ICT)

But also others:*

Bioinformatics (KD) Advanced databases (KD)
Big data analytics (WBI) …

CNS - General Info

Dip. Informatica
University of Pisa

- **CNS** (**Computational neuroscience**) 6 CFU SSD: INF/01 is part of *Applied Brain Science (12 CFU)* - Master programme in Bionics Engineering
- **CNS** (6 CFU) - Master programme in Computer Science

- **Instructors** (2024):
  - ~~Alessio Micheli~~
  - Claudio Gallicchio
  - Andrea Ceni
  - Seminars from experts in CNS

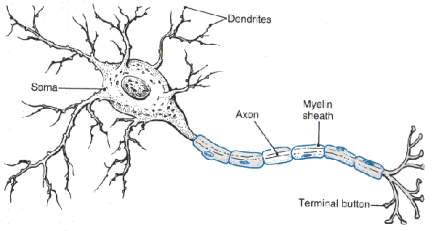


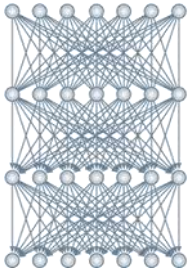
Objectives CNS

Dip. Informatica
University of Pisa

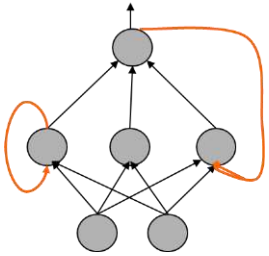
- Introduction to the basic knowledge of the CNS, considering both the **bio-inspired neural** modelling **and computational** point of view.
- Gain practical knowledge on simple CNS models by lab experience

- Including, as for Syllabus,
  - Bio-inspired neural modelling
    - Neuroscience modeling
    - E.g. spiking neuron models and synaptic plasticity





  - Advanced computational learning models:
    - Unsupervised learning: Hebbian and Hopfield Networks
  - Recurrent neural networks
    - Dynamical models for temporal data
    - Also Reservoir Computing



Our approach to CNS

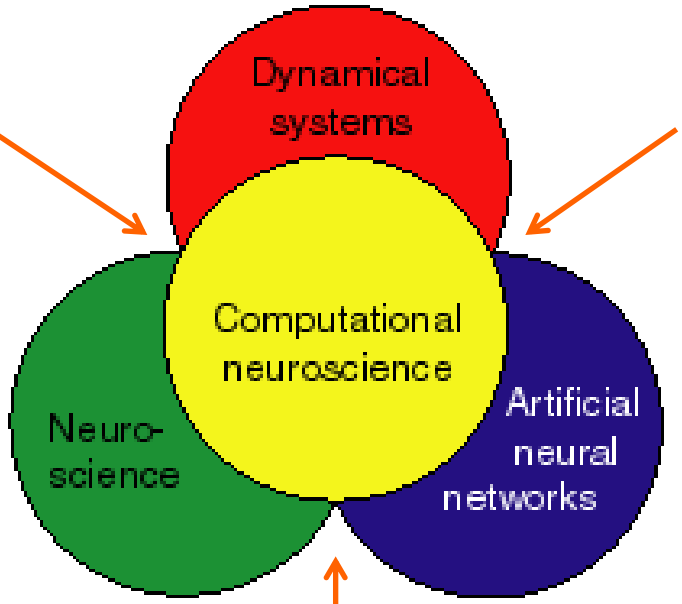
Dip. Informatica
University of Pisa

Study of the information processing properties of the structures involved in the nervous system dynamics

Neuroscience modeling

Biological realism is essential

how NN works?



Recurrent Neural Networks

Introduce effective ANN learning systems

Computational Models for learning

CNS Q&A





Dip. Informatica
University of Pisa

- **Attendance**: strongly advised. We develop the LAB in the class time, through Hands-on Labs in Python and MATLAB, *interleaving* theoretical and practical sessions.
- **Where?** "Polo Ingegneria" Via Giunta Pisano, in a lab class!
- **Are you ready?** Yes, for the ML and Math background, and then:
  - Python and MATAB are easy to be learnt
  - (simple) differential equations for the Neuronal Dynamics Modelling (first part) will be gently introduced for the practical usage (CS point of view)
- The **Exam** include a *seminar* or a project + Oral (details will be provided)

- Further info: "unipi esami" portal for the programme

Future – just ahead

Dip. Informatica
University of Pisa



- General topics for ML applications
- Didactics: CNS course
- **CIML@Pisa**
- **CIML@Pisa research examples**

CIML Group - People

Dip. Informatica
University of Pisa

- **Faculty** (9): Prof. A. Micheli (coordinator), Prof. D. Bacciu, Prof. C. Gallicchio, Dr. A. Carta, Dr. A. Ceni, Dr. A. Cossu, Dr. L. Passaro, Dr. M. Podda, Dr. D. Tortorella
- 4+ post-doc researchers
- **12+ Phd students (the young energy!)**

- More than 25 years of research in ML/AI

- Part of the CAIRNE.eu Research Network (Confederation of Laboratories for Artificial Intelligence Research in Europe)



Computational Intelligence & Machine Learning Group
https://ciml.di.unipi.it/

CIML Group - People

Dip. Informatica
University of Pisa

More than 10 PhD students …





New calls each year, for

- PhD positions in **Computer Science** or
- the **National** PhD in **Artificial Intelligence**



"tie a knot in your handkerchief"

Computational Intelligence & Machine Learning Group
https://ciml.di.unipi.it/

Motivations to a look at Ongoing CIML Projects

Dip. Informatica
University of Pisa

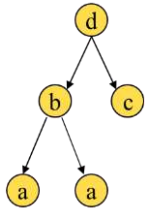
- To have a touch on *research on ML* @ di.unipi
- Some examples of projects @ CIML-Pisa
- On request + Motivational*: **University: didactics & research***



CIML Research Area (overview)

Dip. Informatica
University of Pisa

- Learning in Structured Domains (SD)
  - Pioneering since the 90's the development of *Recursive NN* for **trees** and **graphs**
  - New models by RC, Kernels and generative approaches
  - Focus on efficient deep learning for SD



- Intelligent sensor networks (ML+WSN/IoT)



  - FP7: Rubicon (self-learning robot ecologies), Doremi (personalized healthcare/monitoring of human daily activities)
  - H2020: TEACHING (adaptive CPSoS) [2020-2022], TAILOR (Foundations of Trustworthy AI - Integrating Reasoning, Learning and Optimization) [2020-2023], EMERGE [2023-2026] …
  - NextGeneration EU programme: FAIR [2023-2025], THE [2022-2024], …
  - BrAID (Brugada syndrome and Artificial Intelligence applications to Diagnosis)
- Interdisciplinary applications (med., bio., chem., eng., robotics, …)
- Industrial collaborations
- …

Computational Intelligence & Machine Learning Group
https://ciml.di.unipi.it/

Main core: new model developments

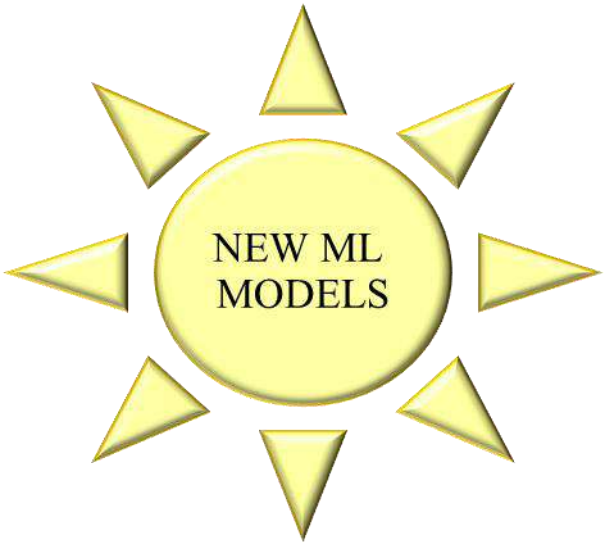
Vertical approach to ML:

Theoretical analysis

New models development

Applications

- **Neural Networks**
- Reservoir Computing
- Kernel-based
- Generative-probabilistic
- **Deep learning**



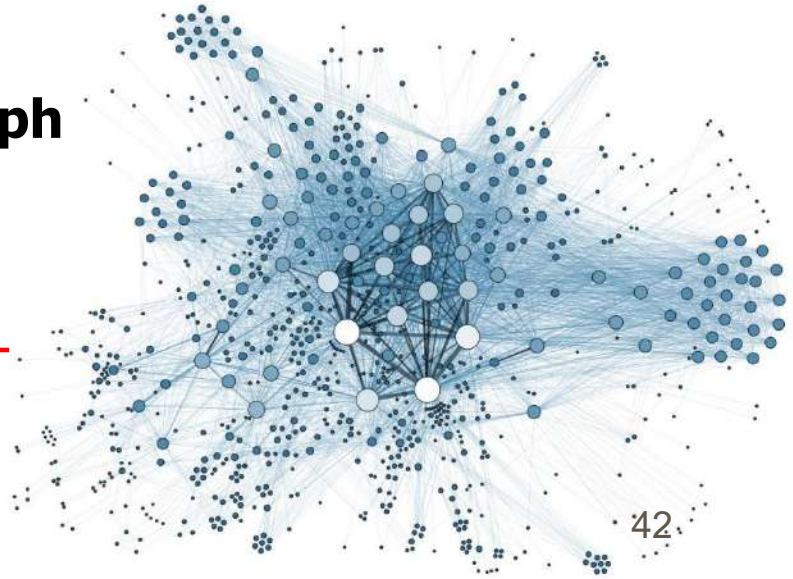
See the previous lecture slides on SDL

An example: an open area of research @CIML-Pisa

Dip. Informatica
University of Pisa

- Deep Learning for SD (the *research core*!)
  - **Analysis of properties** of *Recurrent Neural Networks* with *deep* networks
  - **New models** to conjugate efficient RNN modeling and **deep** learning aspects/advantages.
  - **New kernels** from generative or dynamical models
  - Extension of ML to **Structured/Graph Domains** and **applications**

See the previous lecture slides on SDL



Interdisciplinary applications

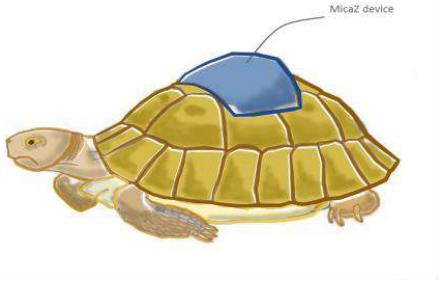
Vertical approach to ML:

- **Theoretical analysis**
- **New models development**
  - (see previous lecture learning for SD)
- **Applications**

Application Overview @CIML



Chem/Bio-informatics



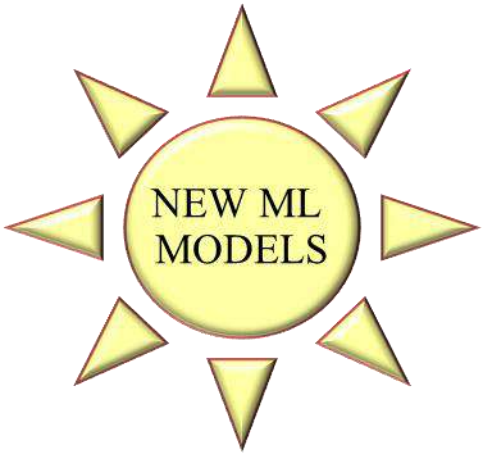
Pattern recognition on animal activity

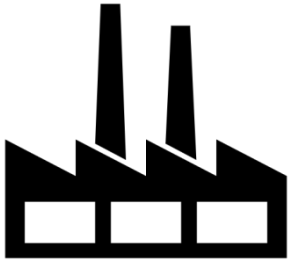


Sensor data
Smart *
IoT

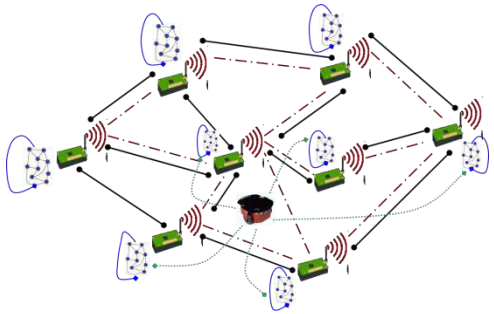


Health care (projects)





Data analysis for industrial leaders



Intelligent Sensor Networks (EU project)

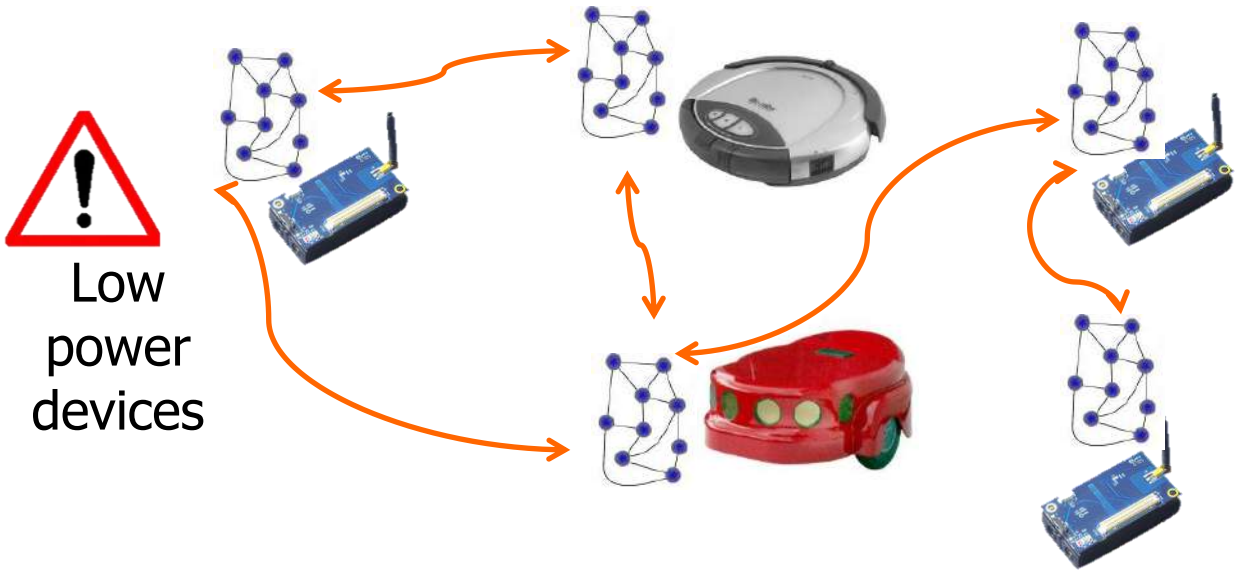


PAST PROJECTS
Examples

A Learning System for Robotic Ecologies

Dip. Informatica
University of Pisa

A distributed, adaptive, and self-organizing learning system comprising independent learning modules residing on ecology nodes



Low power devices

Why embedding?

Information fusion

Adaptive filtering

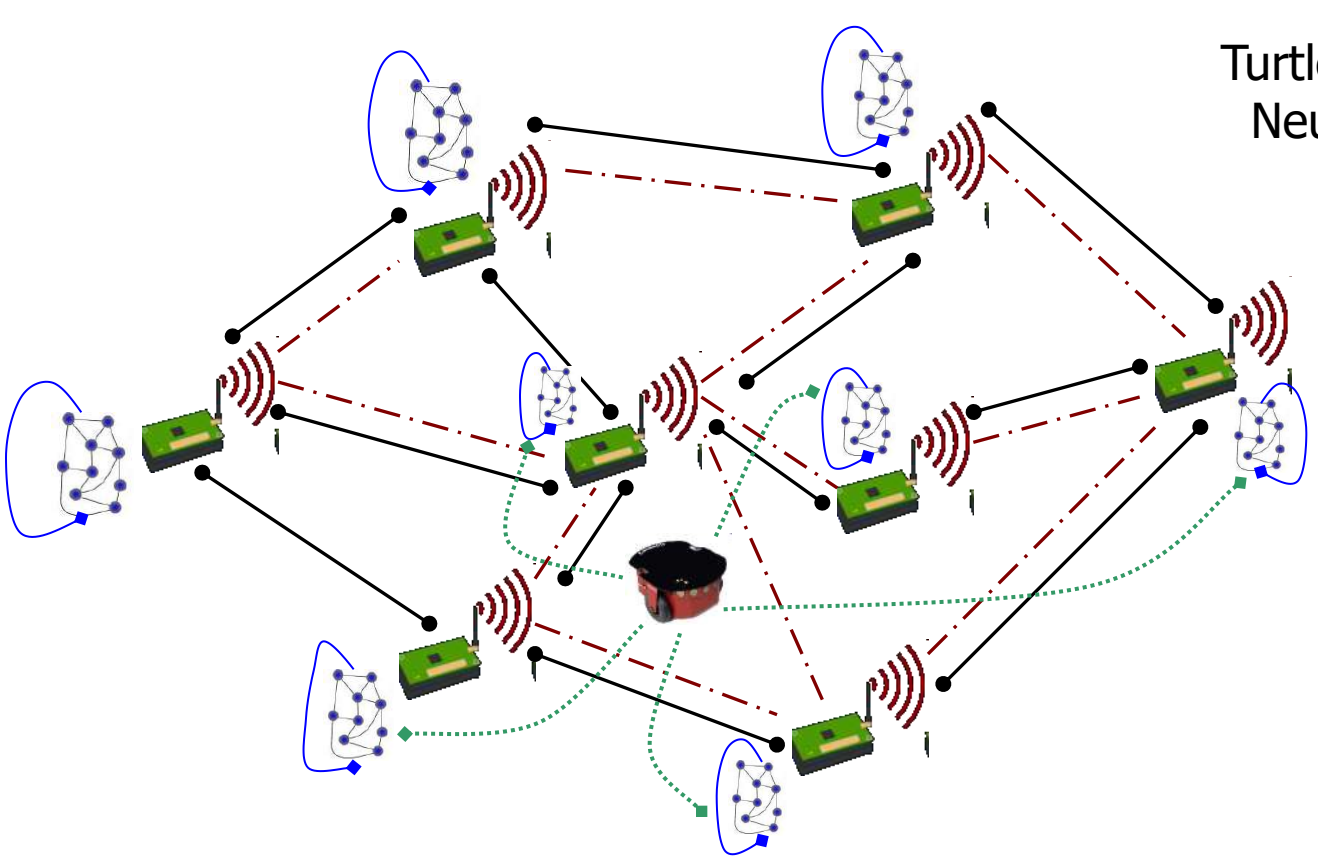
Energy saving

Learning modules interact to realize a distributed prediction

Robot Ecologies
RUBICON (EU FP7)

Dip. Informatica
University of Pisa

- Best with CNS/ISPR (and even better with Sensor Network course)
- Dynamical models for Wireless Sensor Networks data



TurtleBot @ Neurolab



WSN Node

WSN connection

Synaptic connection

Learning module

Distributed NNs and Cooperative Cognitive Process

Robot Ecologies (EU FP7)

Dip. Informatica
University of Pisa



www.unipi.it

Examples of applicative scenarios: Robot Navigation

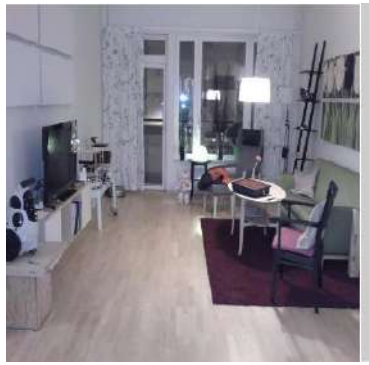
Dip. Informatica
University of Pisa

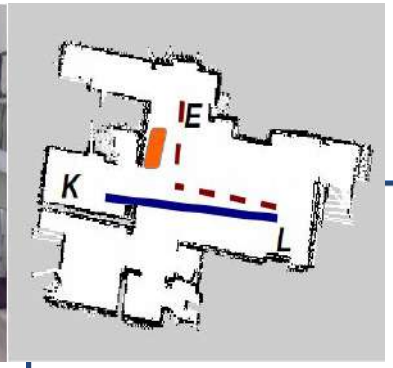


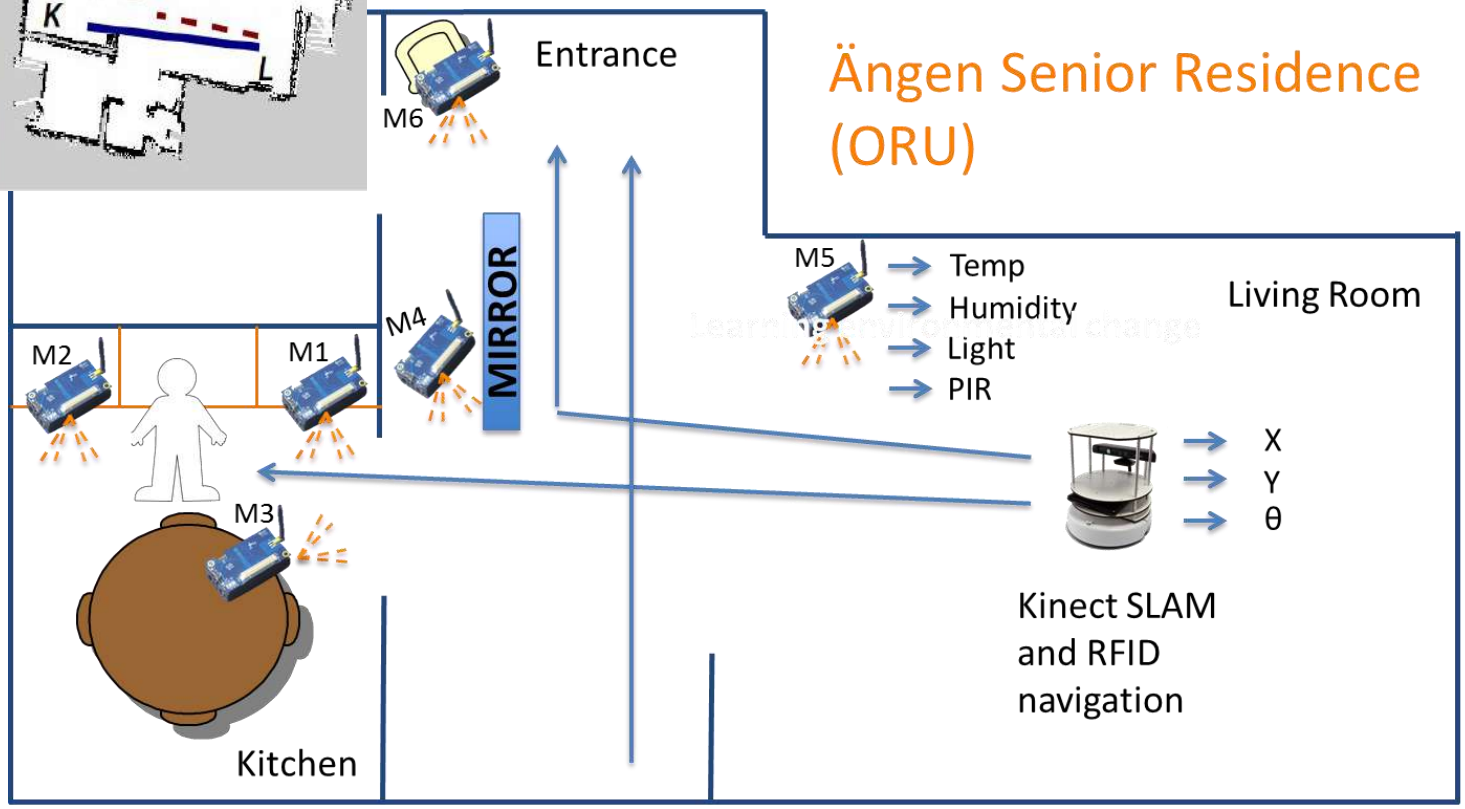
Robot navigation in Stella Maris Hospital
Using noisy signals from RSSI Anchors

Examples of applicative scenarios: Learning for Robot Ecology

Dip. Informatica
University of Pisa







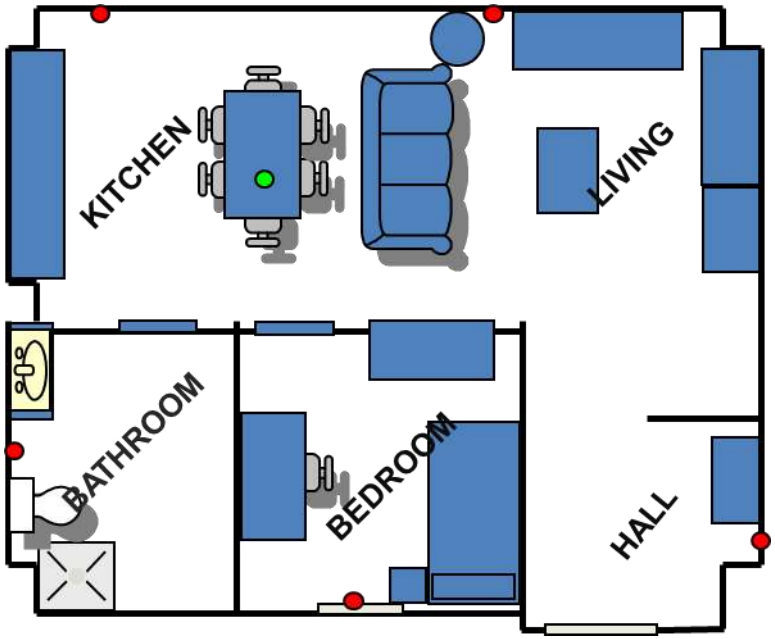
Learning/ adaptation to user's preferences and changes to the environment

Examples of applicative scenarios: Ambient Assisted Living

Dip. Informatica
University of Pisa

- Predicting event occurrence and confidence of Human activities (from cooking to sleeping) basing on local sensors

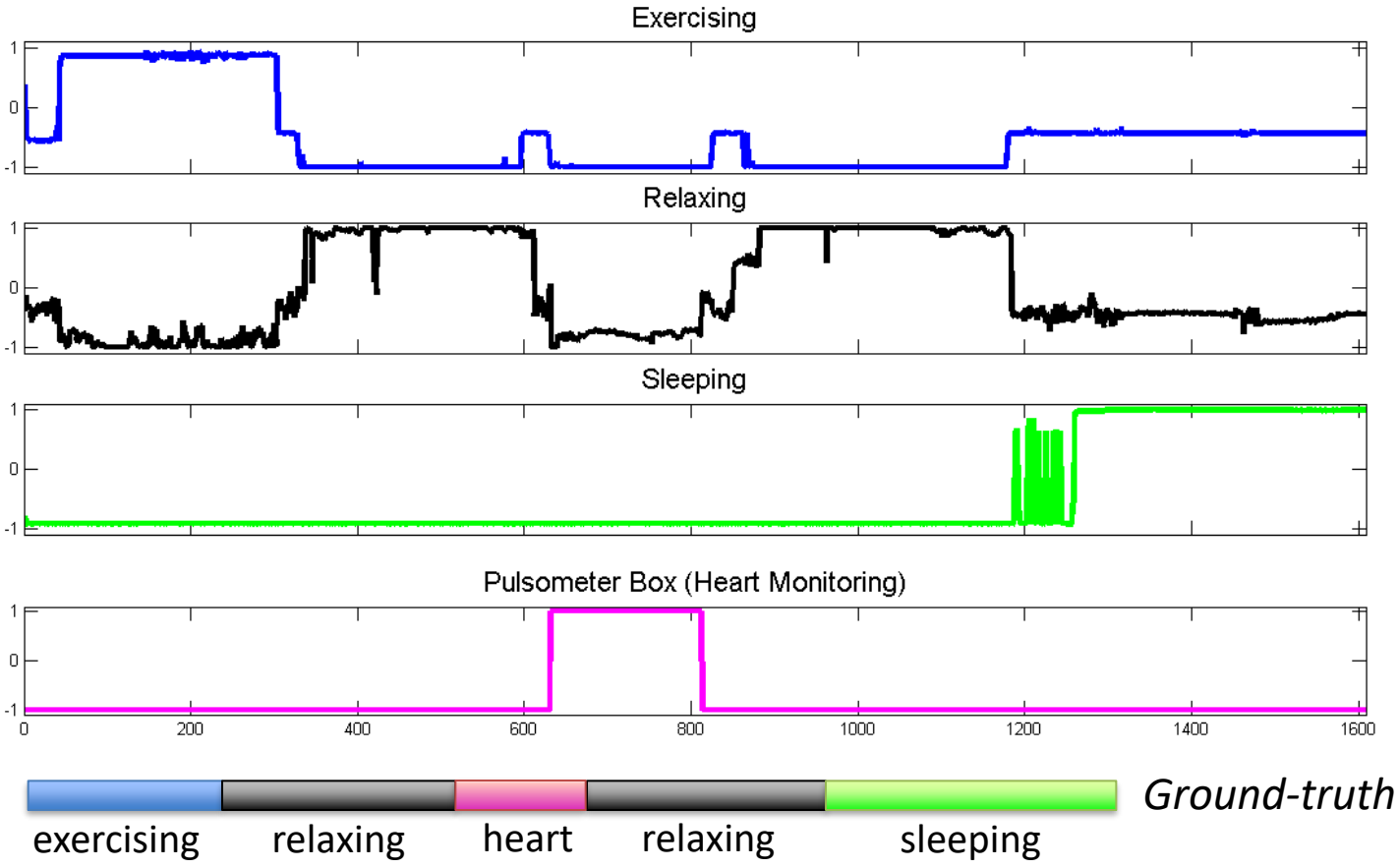




AAL scenario at TECNALIA HomeLab
(Bilbao, Spain - 2014)

Human Activity Recognition

Dip. Informatica
University of Pisa

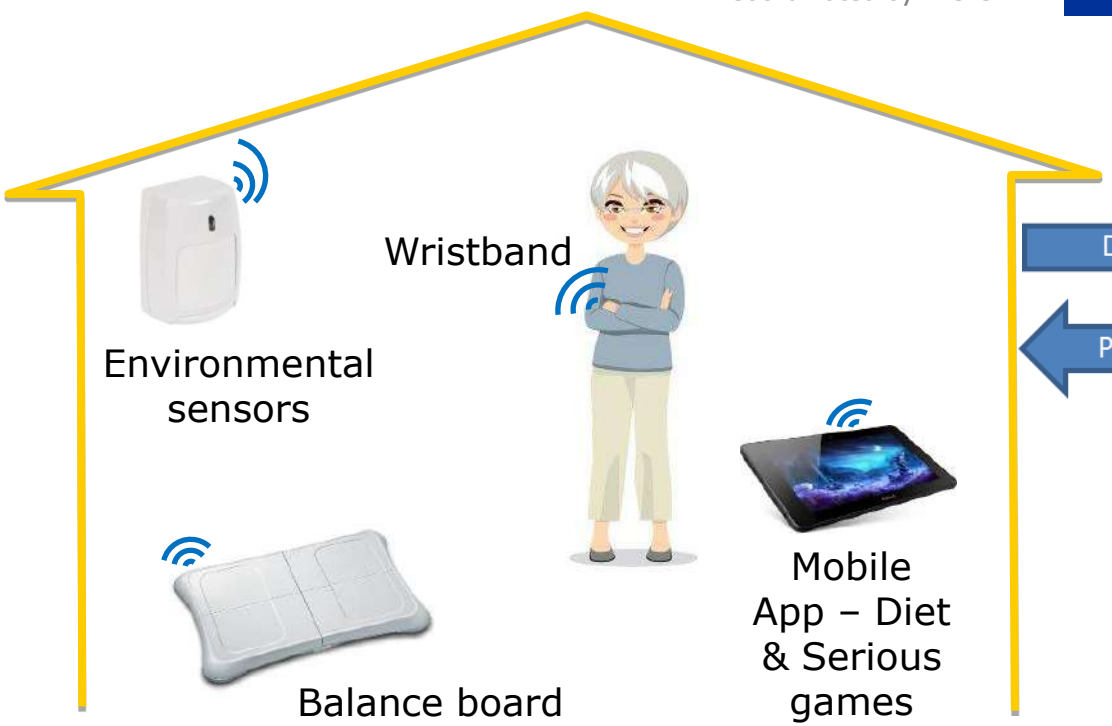


Outputs *of ESN Neural Networks (efficient models for temporal data)*
Inputs *are signals from environmental sensor.*

Decrease of cOgnitive decline, malnutRition and sedEntariness by elderly empowerment in lifestyle Management and social Inclusion

Dip. Informatica
University of Pisa

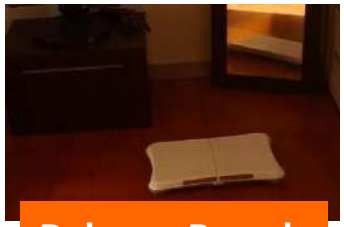
EU FP7
Coordinated by IFC-CNR



- Monitor user activities and assess compliance with lifestyle protocol
- Personalize the lifestyle protocol



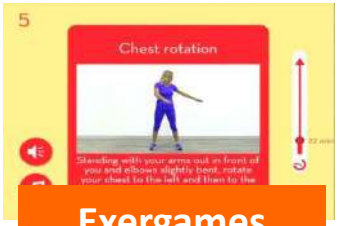
WSN



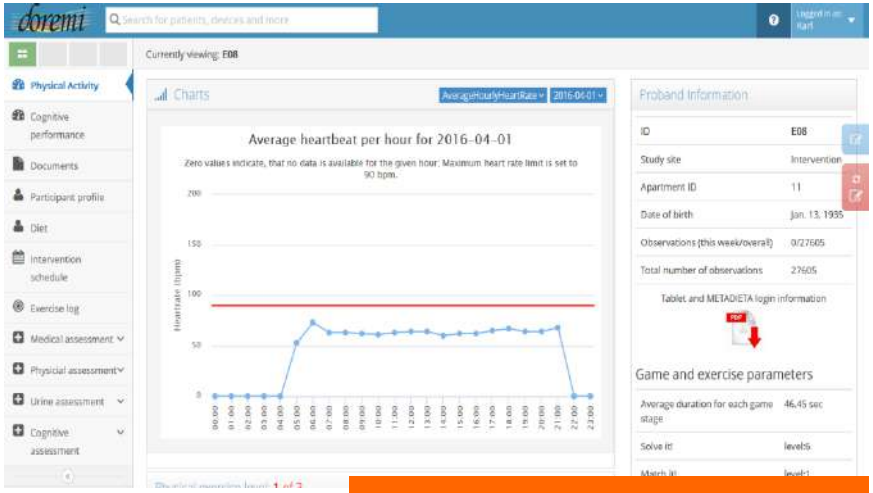
Balance Board



Bracelet



Exergames



Web-based Dashboard



Diet App.



Cognitive Games





feedback

DOREMI (EU FP7)

Dip. Informatica
University of Pisa

- Analysis of dynamical/temporal sensor data for predictive modeling of environmental context and user activities

- **Recognition of behavioral pattern for medical purposes, e.g.:**
  - Balance assessment
  - Level of physical activities/ Kcal energy consumption
  - Social interactions



from accelerometers and heart rate by a user bracelet and environmental sensors (real-world data!)

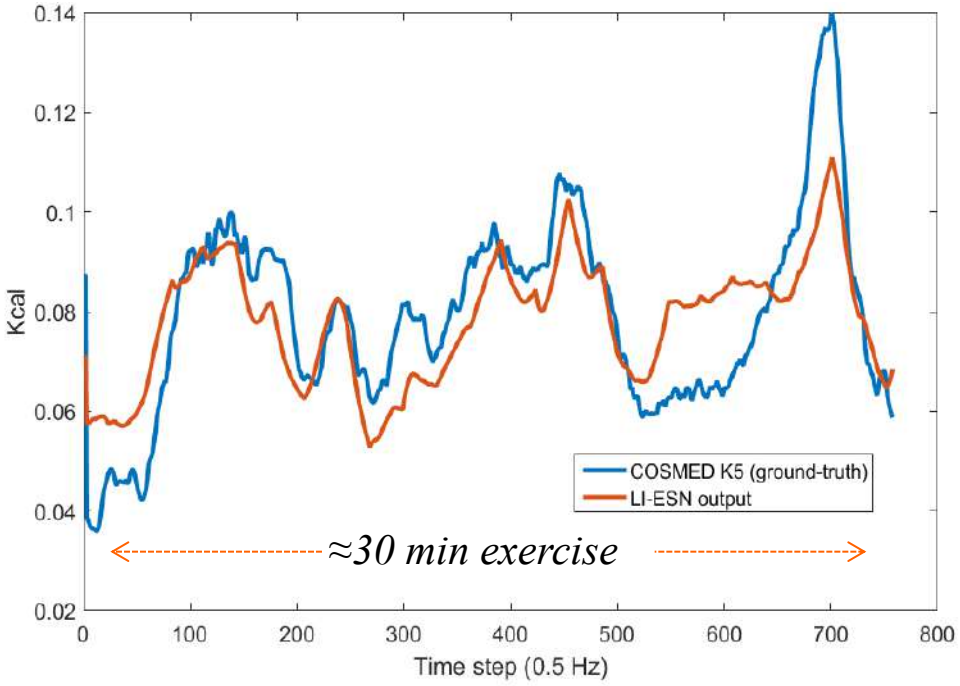
DOREMI (start nov. 13): Decrease of cOgnitive decline, malnutRition and sedEntariness by elderly empowerment in lifestyle Management and social Inclusion

Physical Activity Expenditure Estimation

Dip. Informatica
University of Pisa

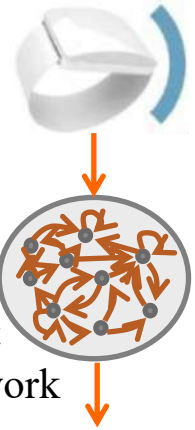
Automatic system for estimating calories consumption from heart rate (HR) measurements (by a bracelet)

- Ground-truth calories consumption (Kcal) from COSMED K5



Target vs Output on a test sequence

- Improves literature approaches
- Tuned for elderly (by learning on a group of volunteers)
- Dramatically improves the accuracy of commercial devices (≈20 times better)



Recurrent Neural Network

VS



Balance Skills Estimation

Dip. Informatica
University of Pisa

An unobtrusive automatic system for balance assessment in elderly

- Berg Balance Scale (BBS) test: 14 exercises/items (~30 min.)
- Autonomous, unobtrusive & simple: the patient can use it even without supervision by clinicians
- Save time for monitoring balance stability (1 vs 14 ex.): 10 sec.
- Recurrent Neural Networks for time-series allow to automatically exploit the richness of the signal dynamics



Protection of animal populations (species preservation)

Dip. Informatica
University of Pisa

- Pattern recognition on tortoise activity (stream from sensor data) in natural parks (Unipi)



PLOS ONE

Localizing tortoise nests by neural networks
PLoS ONE, 11(3), 2016.

Tortoise@ recognizes the digging phase using neural networks

Dip. Informatica
University of Pisa

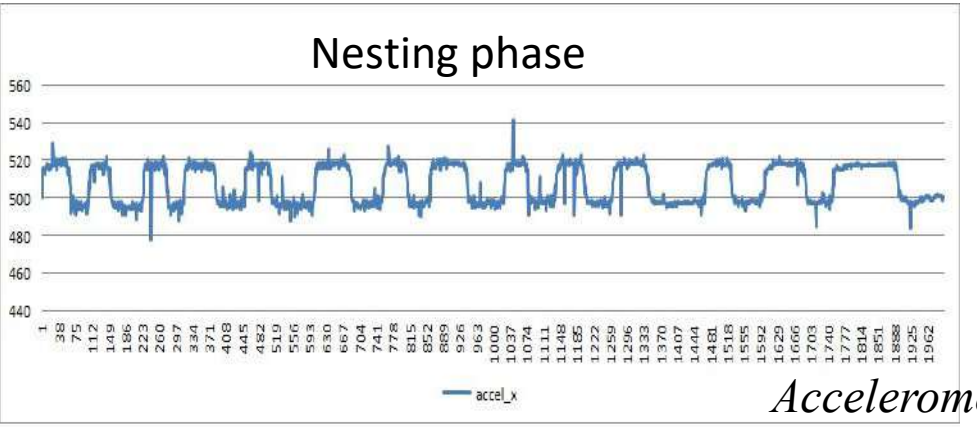


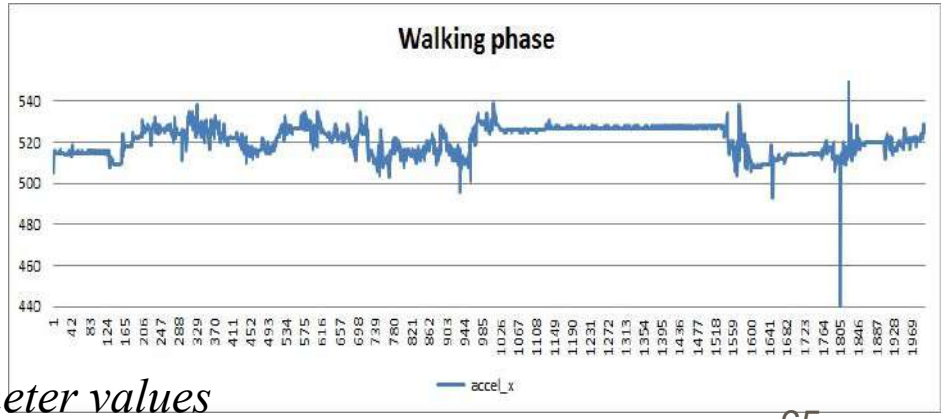


Eating phase



Walking phase





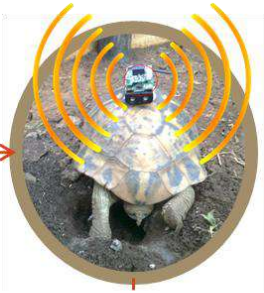
Accelerometer values

The final result ;-)

Dip. Informatica
University of Pisa

A device with a ML classifier identifies the excavation activity of the tortoises.





When the excavation phase is recognized, the device sends a message to the herpetologists.





The herpetologists take the eggs and protect them in a protection center.



But also....

Dip. Informatica
University of Pisa



Penguins

Seals

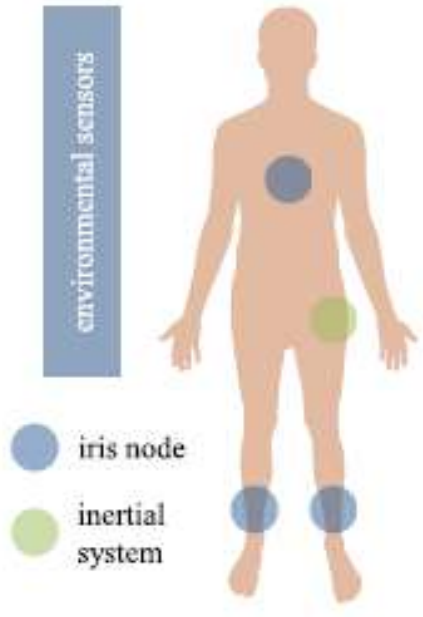


With Australian universities collaborations

And for Humans …

Dip. Informatica
University of Pisa

- Human Activity Recognition by ESN



- **Classification of human daily activities** from RSS data generated by sensors worn by the user
- Input: temporal sequences of RSS values (6 dimensional vector for each time step, noisy data)
- Target: classification of human activity (bending, cycling , lying, sitting, standing, walking)
- Extremely good accuracy (≈ 0,99) and F1 score (≈ 0,96)
- 2nd Prize at **2013** EvAAL International Competition

EvAAL

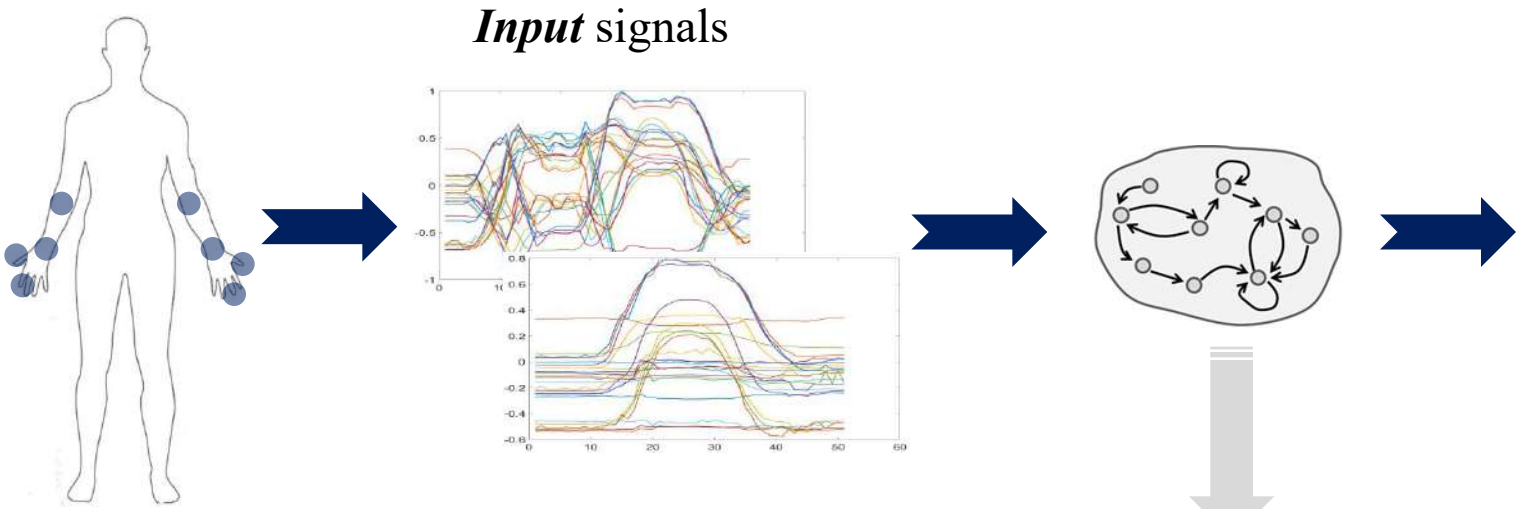
UCI Machine Learning Repository

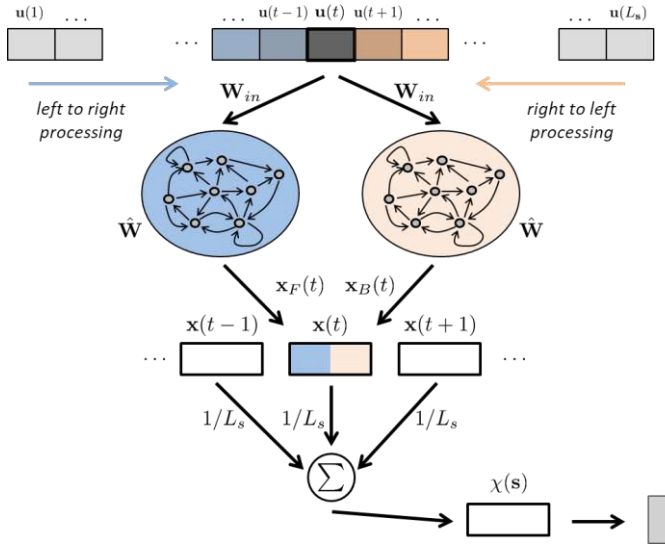
Dataset is available online on the UCI repository

http://archive.ics.uci.edu/ml/datasets/Activity+Recognition+system+based+on+Multisensor+data+fusion+%28AReM%29

Human Gesture Recognition by ESN

Dip. Informatica
University of Pisa



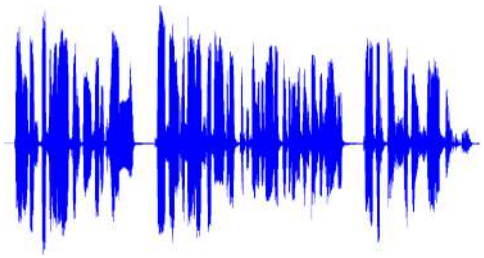


- Experimentally assessed in the AALTD2016 International challenge (ECML/PKDD**2016**)

Validation Set	Test Set
97.8%	94.4%

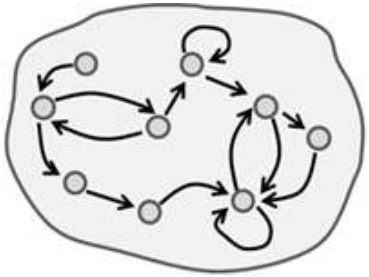
Emotion Recognition from Speech by ESN

Dip. Informatica
University of Pisa



Inputs

Waveform of speech signal



Average reservoir state



Emotion Class



- Speech Processing
- **EVALITA 2014 – Emotion recognition track (Sentiment Analysis)**
- INPUT: Time-series representing the waveform of speech directly (no explicit feature extraction required)
- OUTPUT: emotion (anger, disgust, fear, joy, sadness, surprise, and neutral)
- Efficiency and simplicity of the system
- Promising performances achieved

Neonatology

Dip. Informatica
University of Pisa

A machine learning approach to *estimate preterm infants survival*

- **Aim**: *Early* estimation *of* mortality *risk* of very preterm neonates
  - from features such are birth-weight, gestational age, sex and other physiological variables (12) measured in the first minutes
- Allow to help doctor decision for immediate therapy, to better advise parents, etc.
- Data set of more than 28000 infant records (from the Italian Neonatal Network)
- Relevance of an "**open data**" approach!
- **On-line** PISA (Preterm Infants Survival Assessment) predictor! [http://pisascore.itc.unipi.it/]

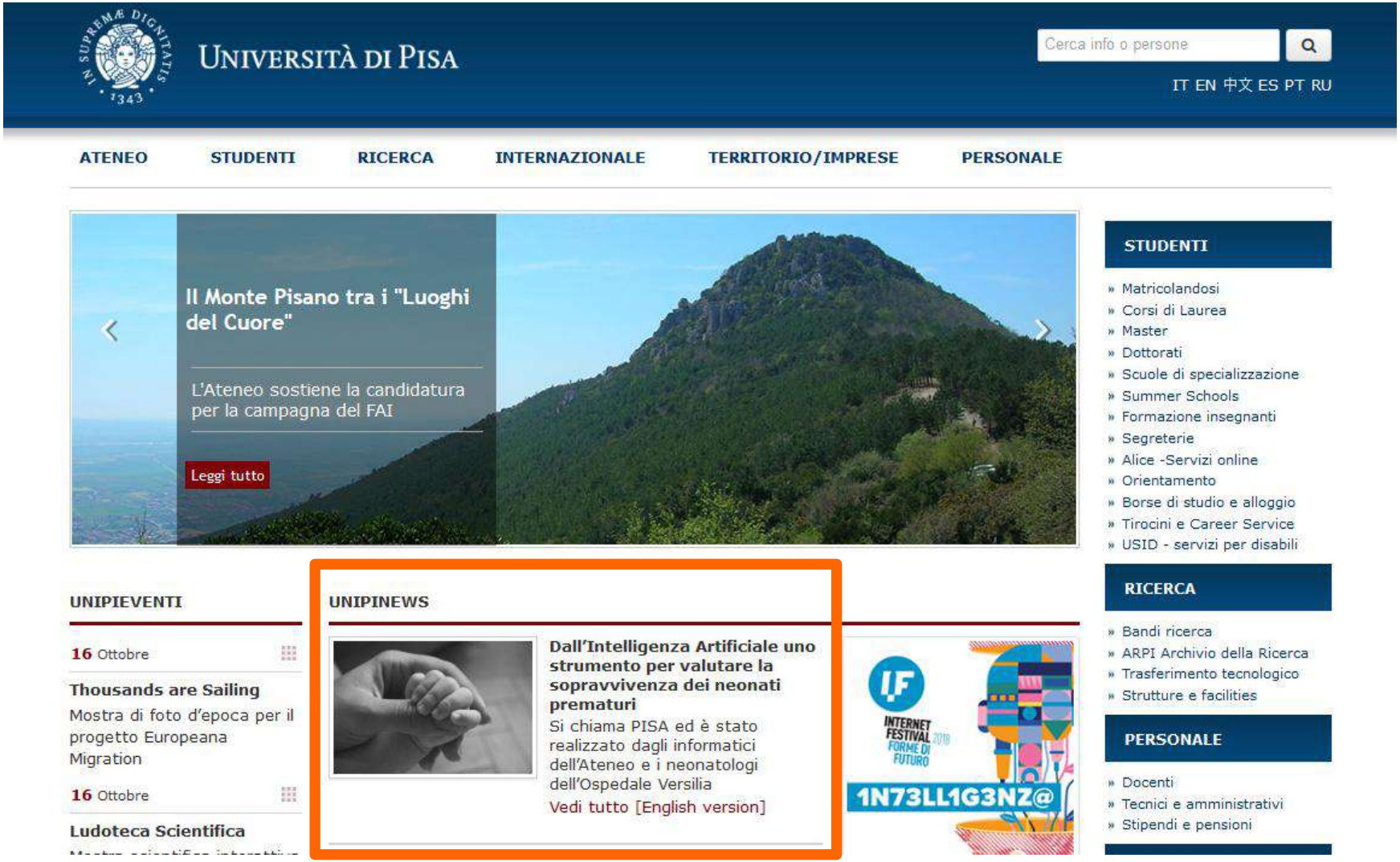




A machine learning approach to estimating preterm infants survival: development of the Preterm Infants Survival Assessment (PISA) predictor
Scientific Reports - Nature (2018) vol. 8:13743.

Unipi main page 16 Oct 2018

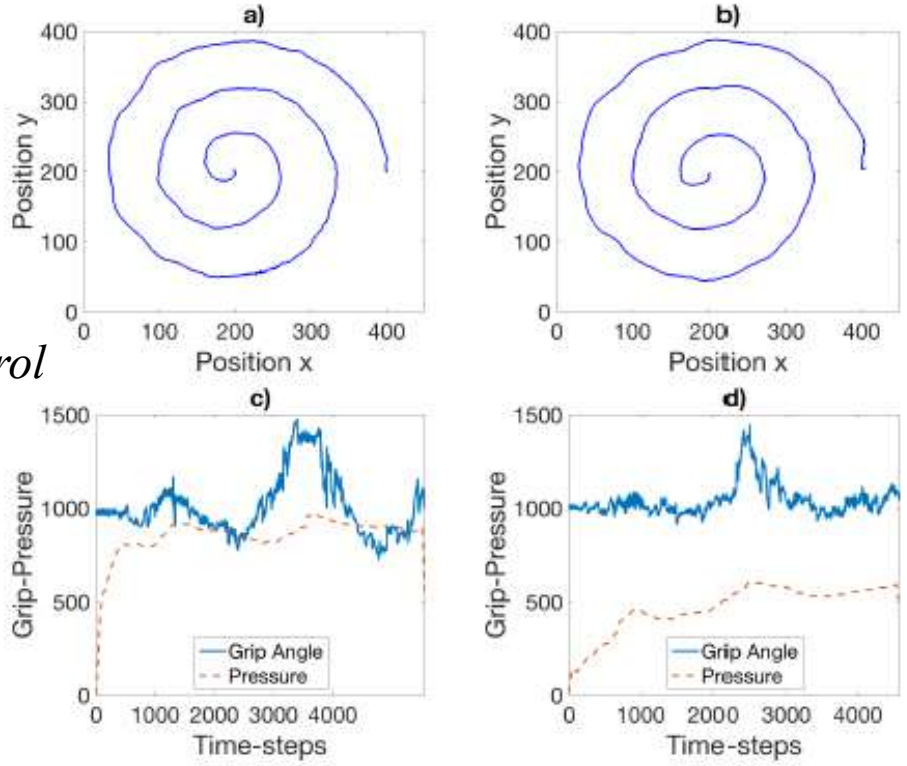
Dip. Informatica
University of Pisa



Parkinson

Dip. Informatica
University of Pisa

- The identification of Parkinson diseases is performed by analyzing the **time-series** collected from a tablet device during the sketching of spiral tests
- by a **deep recurrent neural networks** (Deep ESN)



Control

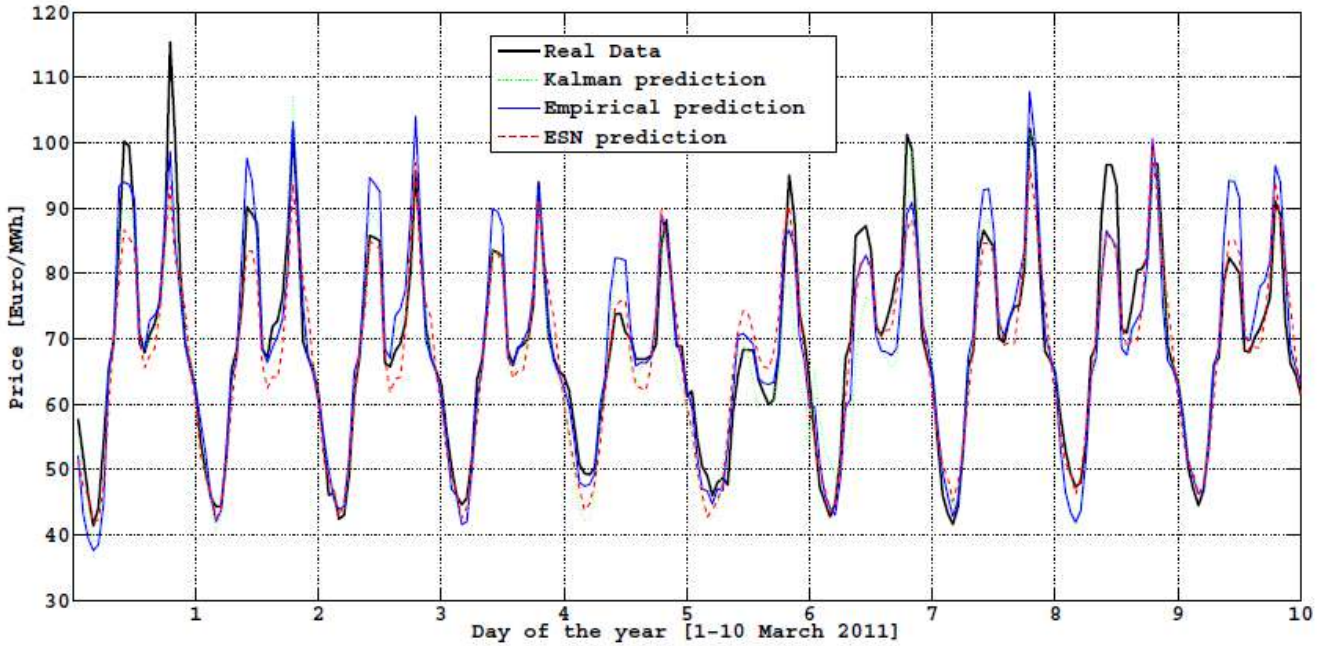
Patient

REF:
C. Gallicchio, A. Micheli, L. Pedrelli. Deep Echo State Networks for Diagnosis of Parkinson's Disease. ESANN April 2018

Applications of ESN for smart grid applications

Dip. Informatica
University of Pisa

- Prediction of the Electricity price on the Italian Market
  - one day-ahead hourly electricity price forecast: most accurate price predictions imply significant economic savings in many smart grid activities



Accurate prediction of hourly electricity price (less than 10% MAPE error)

Earthquake PRJ

Dip. Informatica
University of Pisa

- **ASPIS** PRJ with INGV: Istituto Nazionale di Geofisica e Vulcanologia also CS and Physic dep. + 3 companies
- *«Allerta sismica precoce per infrastrutture sensibili»*
  - EN: Early seismic alert for sensitive infrastructures
- VIRGO @ EGO (European Gravitational Observatory).

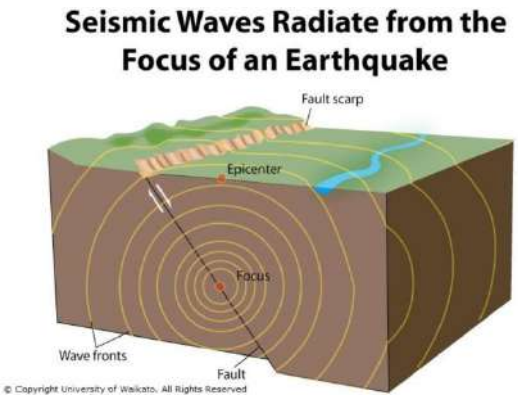


The **Virgo** (3-km long) interferometer antenna is designed to *detect gravitational waves* predicted by the general theory of relativity.
Cascina, Pisa.

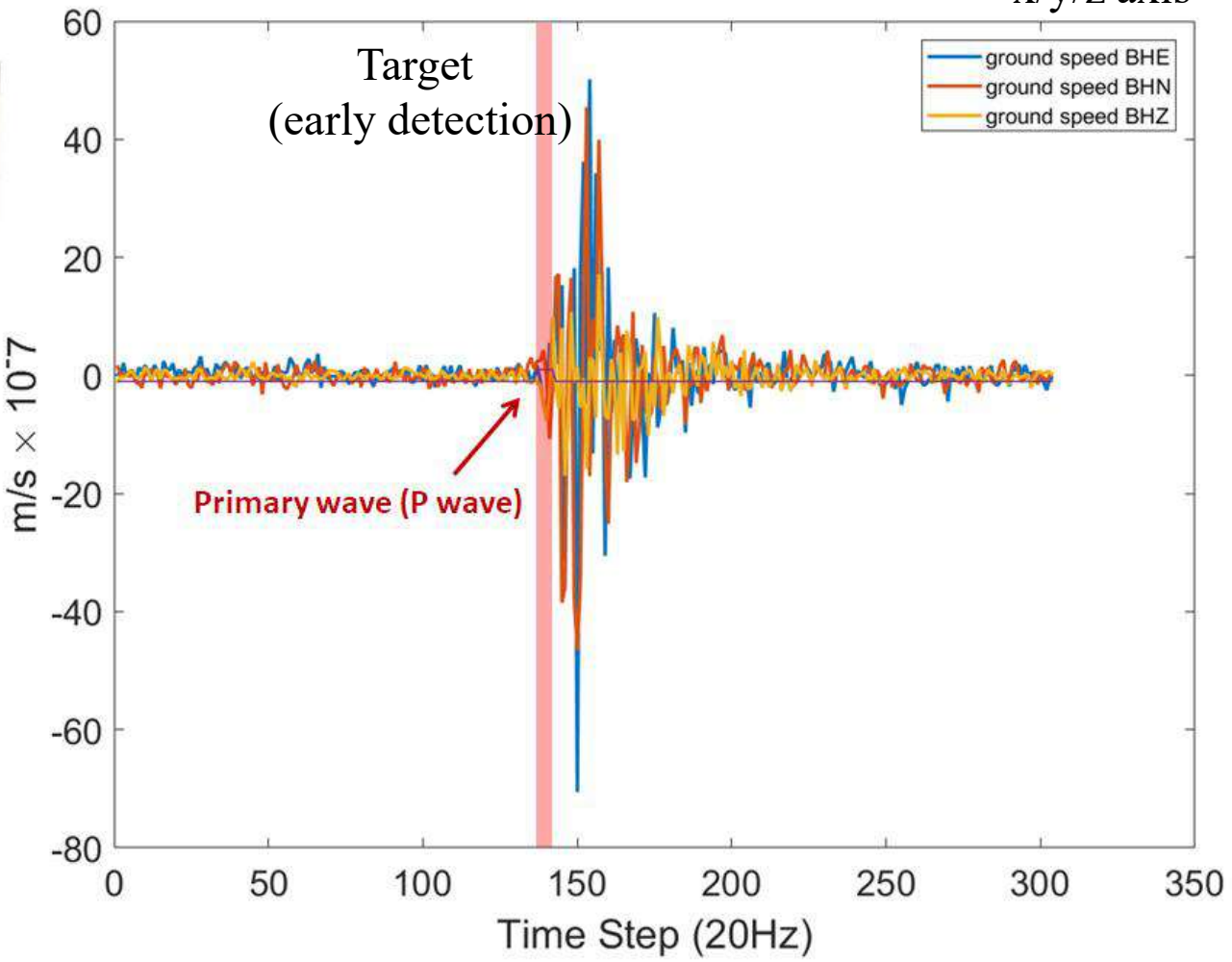
- But early detection can be useful in many other scenario (to save human life!)

Example: seismic waves

Dip. Informatica
University of Pisa



x/y/z axis

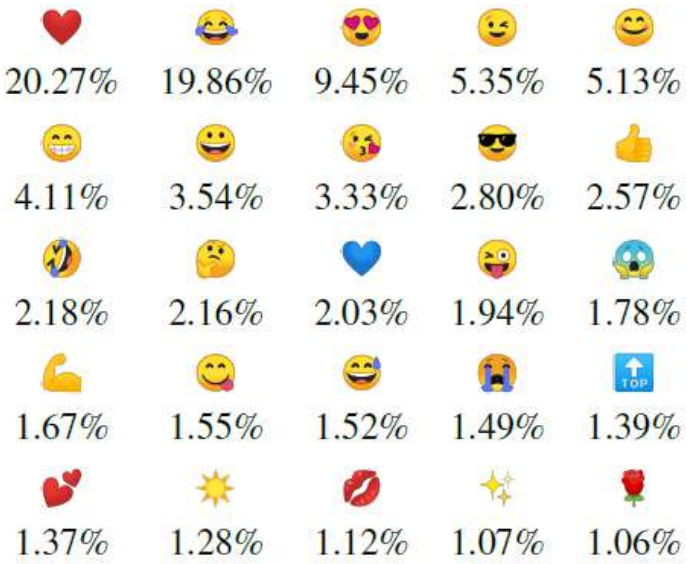


Thesis are still avaliable

Trees: ITA Emoji 2018

Dip. Informatica
University of Pisa

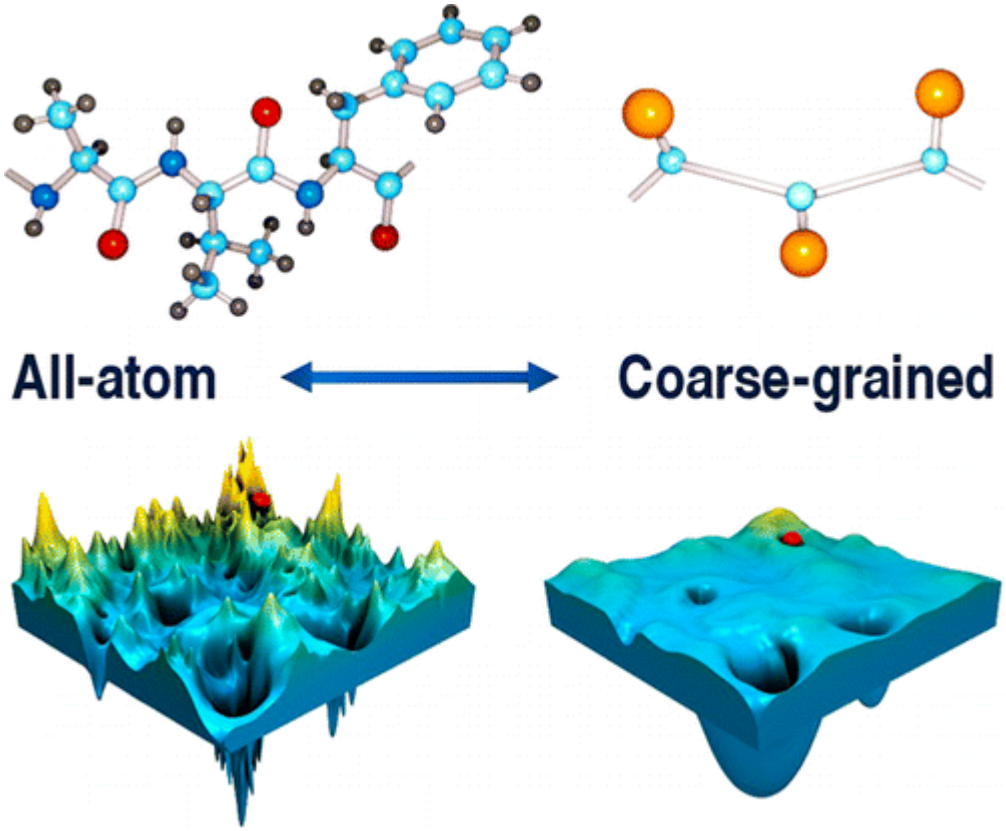
- Emoji Prediction via Tree Echo State Networks
- Italian tweets → Parser → Tree (grammar relations and word ambedding) → TreeESN → Emojis class



Emojis under consideration and their frequency within the dataset.

Graphs: Coarse grained models for Proteins (Biophysics with Univ. of Trento)

Dip. Informatica
University of Pisa





By NN for graphs!



NEW PROJECTS (on going): and Thesis proposals



Main on-going projects and developments

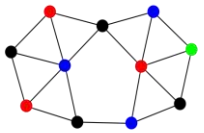


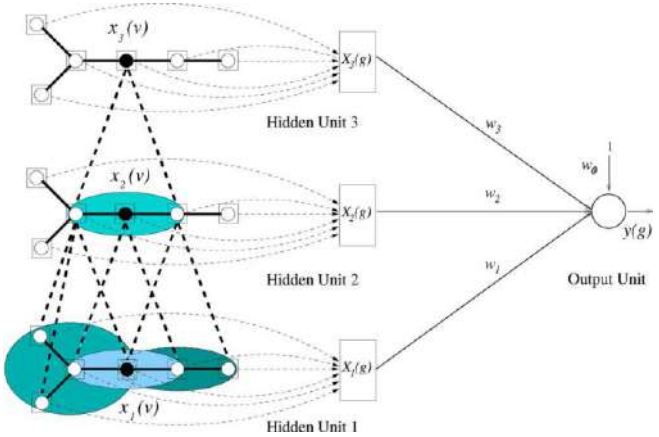
"tie a knot in your handkerchief"

Model developments (I)

Dip. Informatica
University of Pisa

- **ML/Deep NN for graphs** area: see the SDL lecture and its bibliography, e.g.:
  - Constructive approaches by *Deep NN*: **NN4G basis**
    - E.g. heterogenous pool in the cascade correlation construction





A. Micheli. **Neural network for graphs**: a contextual constructive approach, *IEEE Transactions on Neural Networks*, 20 (3): 498-511, **2009**

- **Thesis are ready to start**

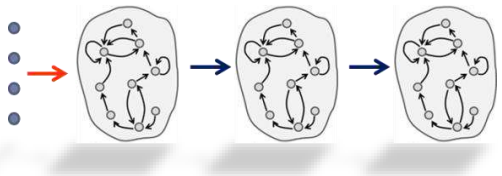


Many new variations are possible following the current developments and our proposals aiming at involving **you** in papers on top-tier venues for ML (ICML, AAAI, NeurIPS and Journals)

Model developments

Dip. Informatica
University of Pisa

- **RC/ESN** area
  - Analysis of basic properties and architectures (typically simplified architectures, like rings)
  - **Deep ESN**: further advancements (typically simplifying the architecture)



  - **Extensions to SD** (trees, **graphs**): further advancements
    - (e.g. properties analyses, comparison with trained GNN models, sw …)
  - *New:* **over-smoothing issues for graphs** → See lect. On SDL learning with references, e.g. Micheli, Tortorella. *Addressing hererophily with graph echo state networks*. Neurocomputing, 2023
  - *New:* **temporal Graphs** by Dynamic-GraphESN & **Continual Learning** → See lect. On SDL learning with references, e.g. Micheli, Tortorella. Discrete-time Dynamic Graph Echo State Networks. Neurocomputing (2022)

Thesis are ready to start

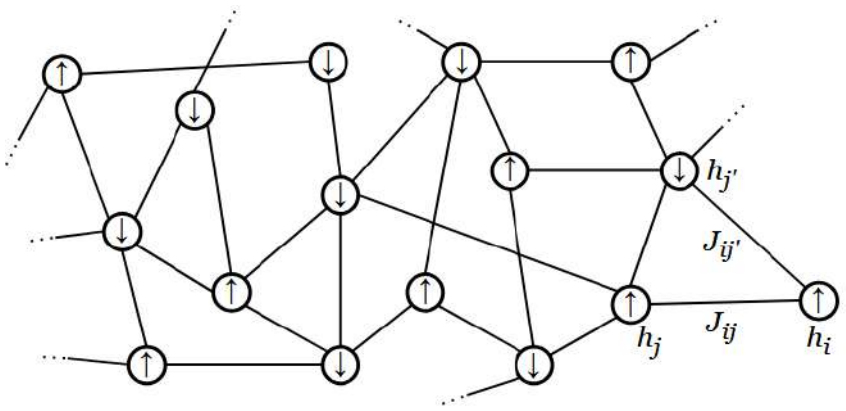


Many new variations are possible following the current developments and our proposals aiming at involving **you** in papers on top-tier venues for ML (ICML, AAAI, NeurIPS and Journals)

Another recent example: Ising Models for Graphs

Dip. Informatica
University of Pisa

- Objectives
  - Efficient HW implementation of Graph RC
- Means:
  - Node representations as Ising spin configurations
  - Sampled from Boltzmann distribution (energy minima)
- How can we implement it?
  - Non-conventional hardware
  - Spintronics, optoelectronics, …
- Who: Collaboration with… TDB



$$\mathcal{H}(\mathbf{h}) = -\mathbf{h}^\top \underbrace{(\mathbf{A} \otimes \widehat{\mathbf{W}})}_{\mathbf{J}} \mathbf{h} - \mu \underbrace{(\mathbf{W}_{\text{in}}\,\mathbf{x})^\top}_{\mathbf{b}^\top} \mathbf{h}$$

Spin coupling (according to the graph topology, see the fig.)

External magnetic field

Tortorella, Brau, Micheli, "Encoding Graph Topology with Randomized Ising Models" *ESANN 2025 Proceedings*, pp. 271–276.

➢ **Thesis are ready to start**

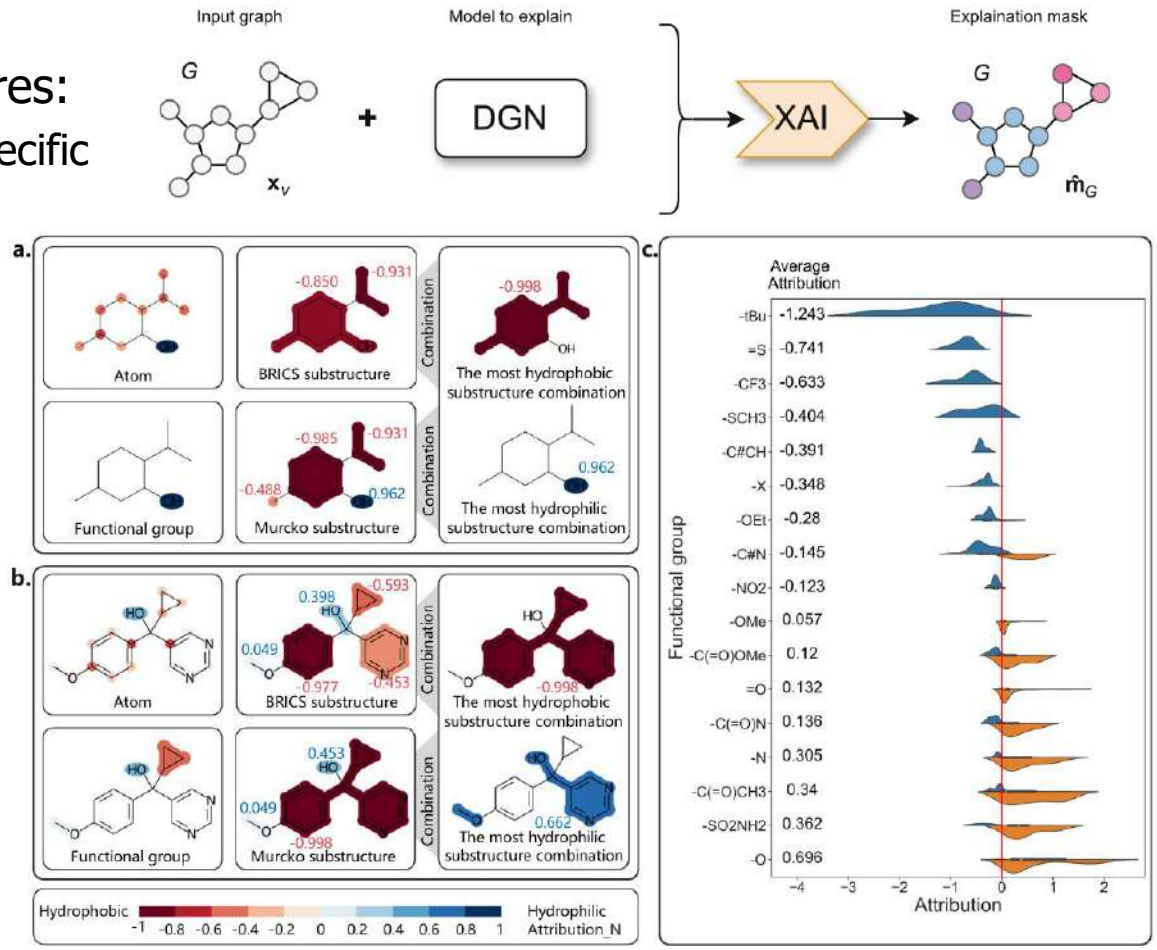
An applicative example: Drug Discovery via Graph Learning & Graph XAI

Dip. Informatica
University of Pisa

- Motivations:

  Discovering new drugs requires:
  - Finding new molecules with specific target activity
  - Lab trials are too costly
- Objectives:
  - Learning for graphs to support Drug Discovery process
  - Why should a molecule be a potential drug?
- Who
  - Collaborations with Helmholtz Munich (Institute of Research)

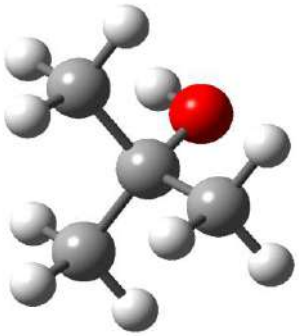
➢ **Thesis are ready to start**



Explaining learning model predictions

Cheminformatics & Toxicity

Dip. Informatica
University of Pisa



QSPR: Correlate chemical structure of molecules with their properties

Property Value e.g Toxicity:

$$\begin{cases} +1 \\ -1 \end{cases}$$

- Means: again,
  - Learning for graphs
  - Graph XAI
- Who
  - Collaborations with *Mario Negri Institute* of Pharmacological Research - Department of Environmental Health Science

➢ **Thesis are ready to start**

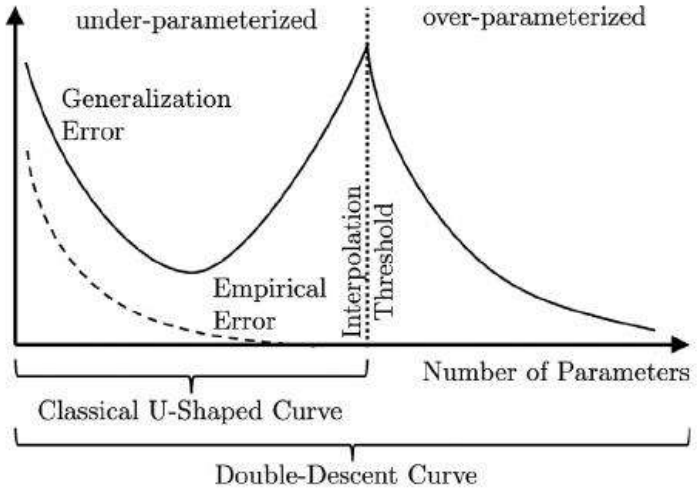
Thesis **related to NN for graphs and Graph XAI**

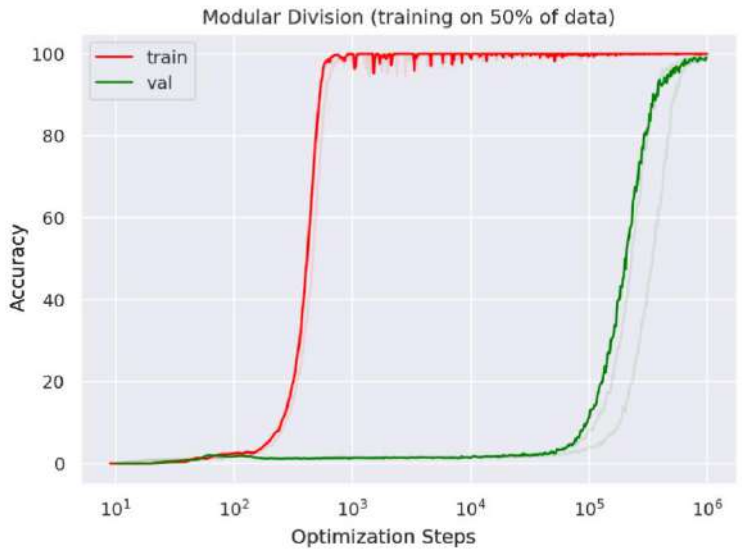
Theory: Investigating Grokking and Double descent

Dip. Informatica
University of Pisa

- Motivations
  - Some learning models do not follow the classic underfitting/overfitting generalization curve
- Objectives
  - How can we explain it?
- Means
  - Model capacity measures beyond VC-Dim
  - Advancing learning theory
- Who
  - Collaborations with UniGe

➢ **Thesis are ready to start**





PNRR Project NextGenerationEU

Dip. Informatica
University of Pisa

- **FAIR** – Future Artificial Intelligence Research is the Italian project approved in the call **"**Extended Partnerships: **Artificial Intelligence. Fundamental aspects"** of the NRRP.
- 3 years, started on January 2023
- The FAIR project brings together 350 researchers and is developed in 10 spokes



  - Spoke 1: *Human-centered AI* (Leader: **UNIPI**), e.g.:
  - Objecyive 1: "human-in-the-loop" machine learning and reasoning
  - WP1.3 - Human centered *Lifelong Learning for Complex Data*

Finanziato dall'Unione europea NextGenerationEU
Ministero dell'Università e della Ricerca
Italiadomani
PIANO NAZIONALE DI RIPRESA E RESILIENZA

WP1.3 FAIR Objectives

Dip. Informatica
University of Pisa

WP1.3 - Human centered Lifelong Learning for Complex Data
Lead: UNIPI, PIs: **Alessio Micheli**, **Fabrizio Sebastiani**.

- study how deep learning architectures (transformers, graph NN, GANs, autoencoders, …) can **continuously and incrementally learn new concepts**, considering also the "human in the loop" and small/scarce data.
- push forward the frontier of **learning in complex dynamic domains** (streams; time series; trajectories; graphs; knowledge graphs; multimedia, time-varying networks)
- **Cross-modal/**cross-language lifelong **learning**

- Other Key Personnel: Davide Bacciu (Unipi), Vincenzo Lomonaco (Unipi), Alessandro Lenci (Unipi), Fabrizio Falchi (CNR), Gualtiero Fantoni (Unipi)

FUTURE AI RESEARCH

Trustworthy for graphs

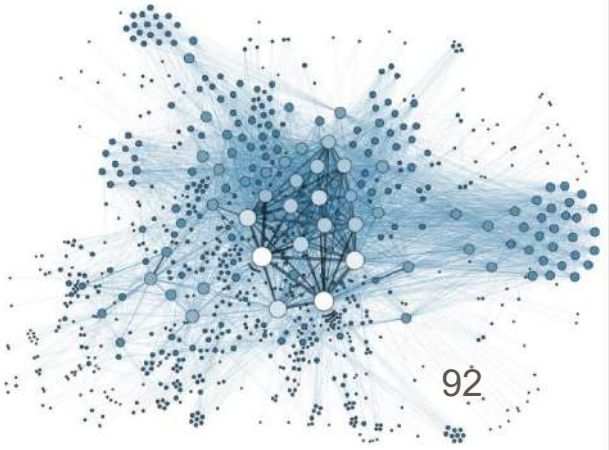


Dip. Informatica
University of Pisa

Trustworthy AI for complex data (structured domains):

New models development and analysis considering

- **Explainability**
  - Interpretate the outcome of a NN for graphs, e.g. in the molecular domain, why this drug his effective?
  - Or connecting to knowledge graphs, bridging symbolic and sub-symbolic reasoning, etc.
- Robustness
  - Perturbations and adversarial attacks to graph NN
  - Difficult: effect small perturbations on graph structure (e.g. to induce to misclassify a person in a social network). It can be useful also for medical data analysis, including COVID data.
- Fairness and privacy preserving
  - e.g. in social data analysis and again COVID data
- *NEW*: Integration with the **continual learning**



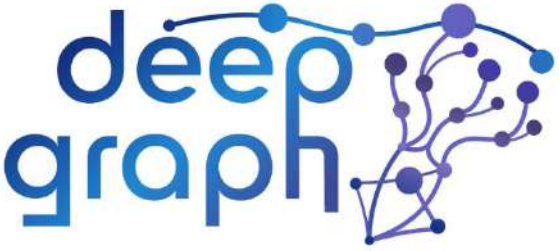
On going research: Deep Graph



Dip. Informatica
University of Pisa

- PRIN (national project), 2Y, **starting Feb. 2025**
- DEEP-GRAPH: Design and Theory of Deep Graph Learning
  - University of Padova (Prof. A. Sperduti)
  - University of Pisa (Prof. A. Micheli)
  - University of Siena (Prof. F. Scarselli)

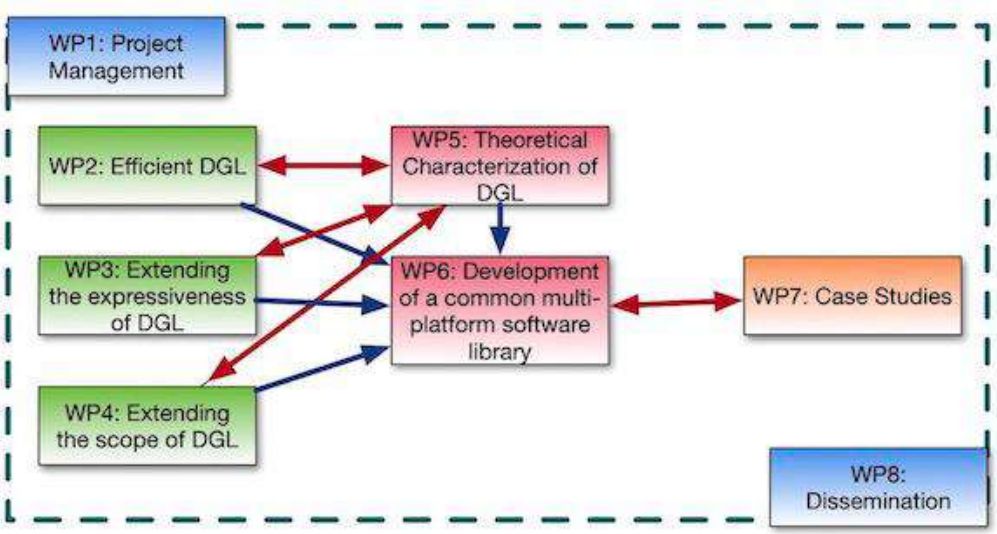






Ministero dell'Università e della Ricerca

PRIN
PROGETTI DI RICERCA DI RILEVANTE INTERESSE NAZIONALE



Thesis are ready to start

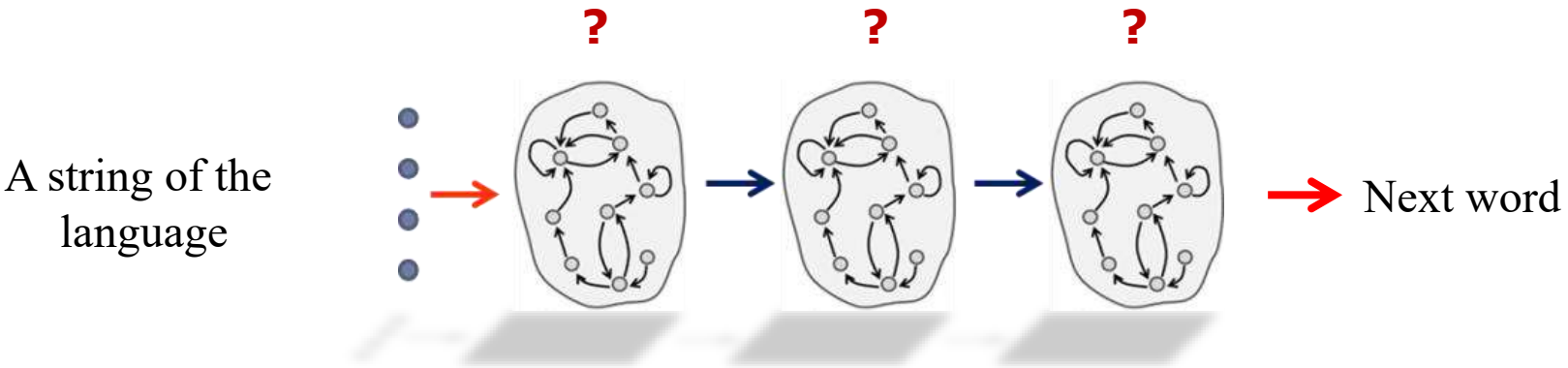


Further Developments and Applications

NLP related

Dip. Informatica
University of Pisa

- Trade-off accuracy/efficiency using RC approaches for NLP (vs LLM)
- Analysis of the representations developed in the internal layers of deep ESN (RNN)
  - I.e. analyses of different levels of abstraction for language encoding
- With Computational Linguistic researchers



Thesis are ready to start



A recent project in Cardiology: BrAID

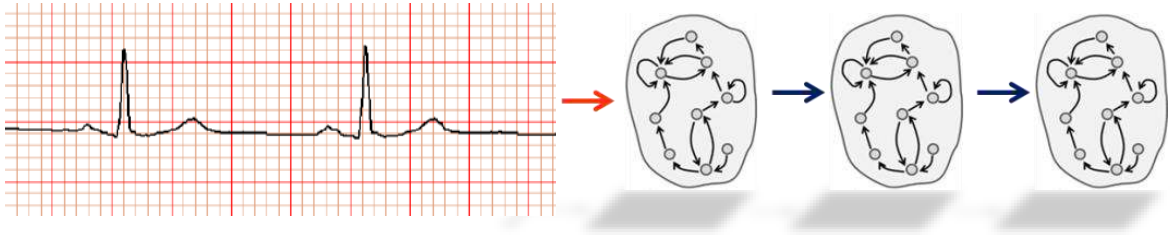
BrAID Project
Brugada syndrome and **A**rtificial **I**ntelligence applications to **D**iagnosis

Regione Toscana

braid

- Brugada syndrome (BrS): not common but dangerous disease (sudden cardiac death)
  - No symptoms
  - BrS is diagnosed by identifying characteristic patterns on the electrocardiogram (ECG)

ECG (electrocardiographic) signal



Brugada **diagnosis alert**: allowing advantages in term of *screening automatization*



Results: accuracy over 90%, w.r.t. **88.0%** by clinicians (blind test)

Vozzi, Pedrelli, Dimitri, Micheli, et al. Echo state networks for the recognition of type 1 Brugada syndrome from conventional 12-LEAD ECG, Helyon, 2024

BrAID now and next

Dip. Informatica
University of Pisa

State-of-the-art:

- We concluded the data collection, so we havenow a good dataset with positive (BrS) and negative cases
- We have very good results applying ESN (over 90% of accuracy, with cardilogit at 88)
- We can develop (future projects):
  - **extended data sets** by other collaborations
  - **new approaches** (including the usage of different sources of data, ECG and features, filtered versions, or ECG different leads, omics data, etc.)
  - **new models** (e.g. CNN, RNN, advanced and *hybrid approaches,* interpretable models, comparing with other approaches)
  - compare the results and analysis with **clinical centers** (clinical feedbacks).

Thesis are ready to start



braid

Bioinformatics

Dip. Informatica
University of Pisa

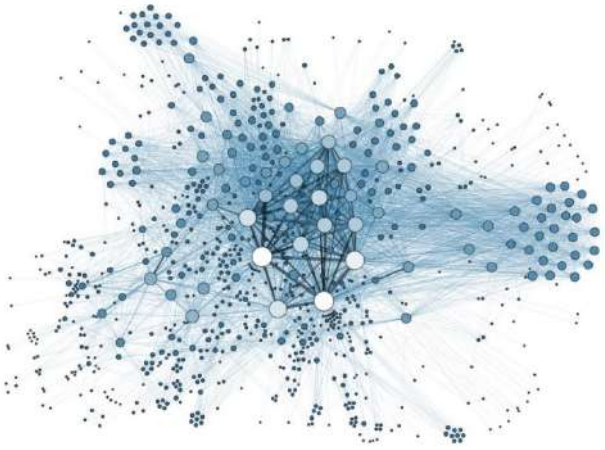
Under construction! Ready to apply for **tree/graph data** on:

- System biology (graphs/networks) (with P. Milazzo) *on-going (next slides)*

- Bioinformatics:
  - Bio-molecular Networks → next slides
  - Prediction of proteins function → next slides
  - Pan-genome analysis (by graph representation)



On going: Thesis available!



THE: PNRR Project NextGenerationEU

Dip. Informatica
University of Pisa

- **THE- TUSCANY HEALTH ECOSYSTEM**
- ***Ecosistema dell'innovazione sulle scienze e le tecnologie della vita in Toscana***



- *Spoke: Precision medicine & personalized healthcare*
- System biology, AI and BigData technologies for omics and computational biology
  - Investigation of **machine learning** for structured data in the omics and computational biology domain: *proteins, drugs and genomic structures*.

Finanziato dall'Unione europea NextGenerationEU
Ministero dell'Università e della Ricerca
Italiadomani PIANO NAZIONALE DI RIPRESA E RESILIENZA

PAN-HUB



Dip. Informatica
University of Pisa

Multidisciplinary and Interregional Research and Clinical Trial Hub for Contrasting Pandemics and Antimicrobial Resistance (PAN-HUB)

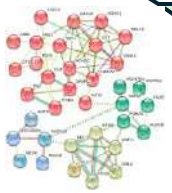
Finanziato dall'Unione europea NextGenerationEU

FSC Fondo per lo Sviluppo e la Coesione

Ministero della Salute

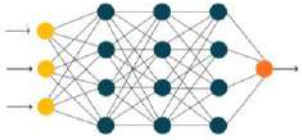


SYSTEMS BIOLOGY

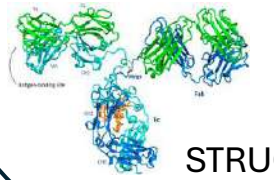




BIOINFORMATICS



MACHINE LEARNING



STRUCTURAL BIOLOGY

ANTIBIOTICS RESISTANCE

- **In-silico antibody design**: antigen-antibody interaction prediction
- Prediction of antibiotic resistance and clinical outcome of bacteria infections
- In-silico identification/generation of **therapeutic compounds** modulating response to infections

EPIDEMICS

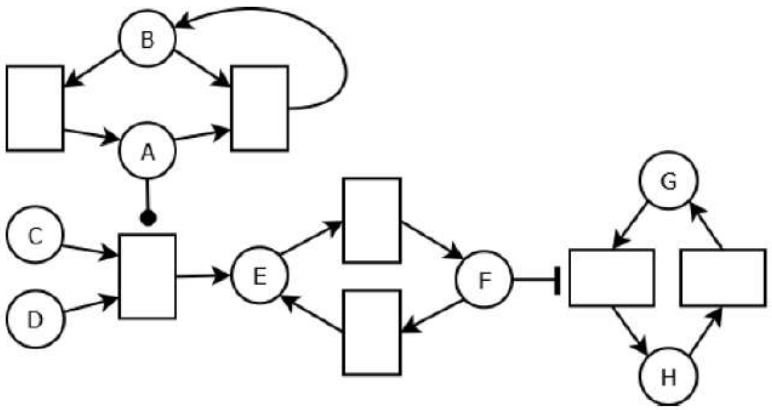
METHODS

APPLICATIONS

Classification of Biochemical Pathway Robustness with Neural Networks for Graphs

Dip. Informatica
University of Pisa

- **Biochemical Pathways** (**BPs**) are sets of chemical reactions among biomolecules
- They model **cell functionality**
- Structure described by **graphs** (e.g. **Petri Nets**), dynamical properties analyzed with Ordinary Differential Equations (ODEs)
- Entities (**nodes**): species (circle) or reactions (square)
  Relations (**arcs**): produce, enhance, block
  - there are also Input/Ouput nodes: induced sub-graphs of paths for different I/O nodes

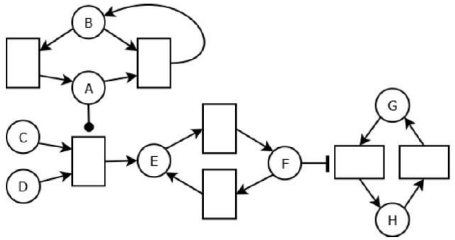


Classification of Biochemical Pathway Robustness with Neural Networks for Graphs

Dip. Informatica
University of Pisa

- **Input:** Pathway Petri nets (their correspondent **directed** graph), for each pair of species in a given net
- **Output:** an associated **robustness** value (0/1)

- Direct prediction for the Petri nets topology, avoiding costly ODE simulations
- Does the networks include enough information to predict robustness? Or other properties?
- Can we extend to Protein-Protein interaction (PPi) networks? (next slide)



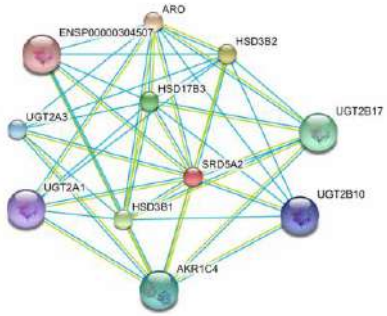
- Best paper award @ **BIOSTEC-BIOINFORMATICS 2020**
  Bove, P.; Micheli, A.; Milazzo, P. and Podda, M. (2020). Prediction of Dynamical Properties of Biochemical Pathways with Graph Neural Networks. In Proceedings of the 13th International Joint Conference on Biomedical Engineering Systems and Technologies - Volume 3: BIOINFORMATICS, pages 32-43
- Fontanesi, M., Micheli, A., Milazzo, P., & Podda, M. (**2023**). Exploiting the structure of biochemical pathways to investigate dynamical properties with neural networks for graphs. *Bioinformatics,* 39(11)

Bio-molecular Networks

Dip. Informatica
University of Pisa

A Bio-molecular network is a graph representation of

- **Relationships** (of which there are many types) amongst a group of biomolecules.
- **Vertices** or nodes represent biomolecules, including macromolecules such as genes, **proteins**, and RNAs, or small biomolecules like amino acids, sugars, and nucleic acids



*They can be **very large** (up to milions of nodes)*

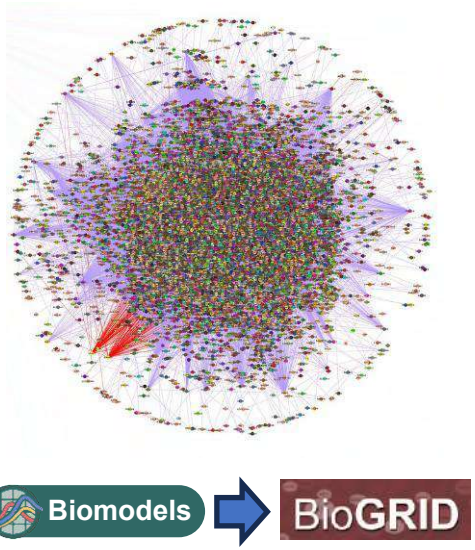
In particular, other networks data/problems to extend the application of ML/NN for graphs are:

- **PPI**: Protein Protein Interaction Networks: Proteins (interactions or similarities or ontologies) networks for drug or oncology research
  - These graphs contain a relevant information that can be exploited for conducting different kinds of analysis, such as automatic function prediction, disease gene prioritization, drug repositioning or discovery, etc.

On going: Thesis available!

PPI Networks: first results

- Protein-Protein Interaction Network (**PPI**)
  - Nodes represent proteins
  - Arcs represent physical associations between two proteins
- **Predicting sensitivity** (perturbation analysis) on PPI
- Starting from biochemical pathways (Biomodels) and mapping to PPI subgraphs (BioGRID, **60K nodes**)



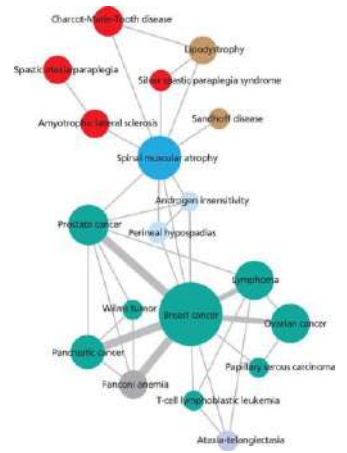
- **Results**:
  - **Efficiency** vs numerical simulation on biochemical pathways (ODE)
  - **High performance** even without protein embeddings, meaning that DGN is relying on PPI topology

PAN-HUB prj on antibiotics resistance

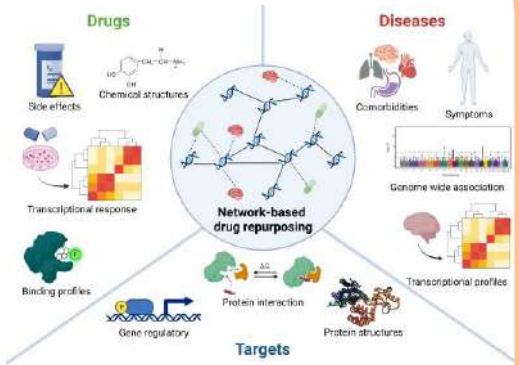
Dipalma, Fontanesi, Micheli, Milazzo, Podda. Sensitivity analysis on protein-protein interaction networks through deep graph networks *BMC Bioinformatics (**2025**)*

Impacts/Potential usage

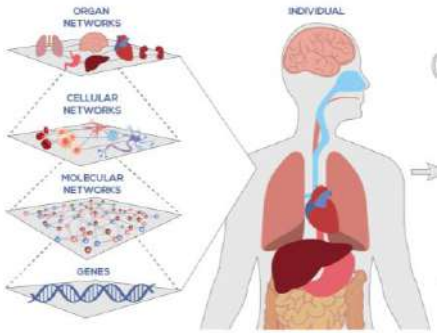
Dip. Informatica
University of Pisa



Disease understanding



Drug design and repurposing



Precision/Personalized medicine

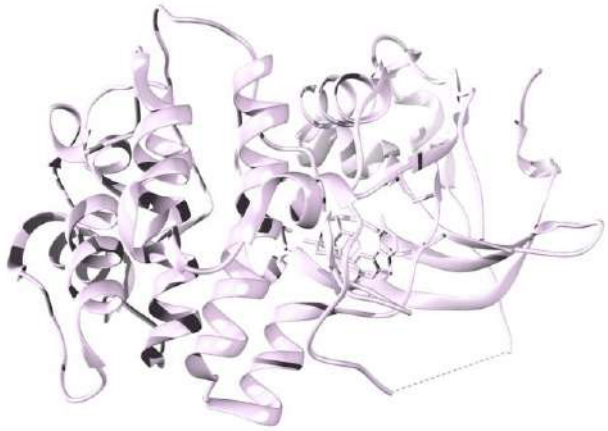
Prediction of proteins function

Dip. Informatica
University of Pisa

- Prediction of proteins function
- Using gene ontology graphs
- Possibility to partecipate to CAFA challenge
  - There are many proteins in the databases for which the sequence is known, but the function is not.
  - The Critical Assessment of Functional Annotation (CAFA) is an ongoing, global, community-driven effort to evaluate and improve the computational annotation of protein function.

Approaches can include:

- SVM/**Kernel** building by efficient computation of common sub-graphs between pairs of proteins (with R. Grossi)
- Use of **NN for graphs**

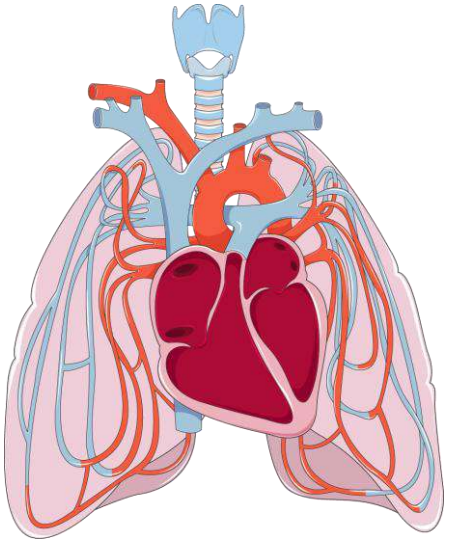


On going: Thesis available!

Pulmonary Arterial Hypertension

Dip. Informatica
University of Pisa

- Objectives
  - Predictive risk estimation in pulmonary arterial hypertension
- Means:
  - Classifiers + Survival analysis
  - Importance analysis (XAI) of clinical features (echocardiographic variables)
  - Data sets of clinical data are already available
- Who
  - Collaboration with Prof. Badagliacca, Dep. Clinical Intern, Anesth. and Cardiovascular Sciences, Sapienza University of Rome
  - USA research and clinical centers



On going: Thesis available!

Data Analysis for Industrial Leaders: Example

Dip. Informatica
University of Pisa

- Objectives
  - to develop new approaches for improving the efficiency of urban public transport
- Means:
  - to combine machine learning techniques with combinatorial optimisation methods
- Who
  - Collabaration with Prof. Antonio Frangioni, Andrea Cardia (PhD student), M.A.I.O.R



M.A.I.O.R.
A CLEVER DEVICES COMPANY

MAIOR designs and develops software solutions for service planning, resource scheduling and company performance analysis in the transportation industry

On going: Thesis available!